

Universidade de Vigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D^a. Tatiana María Quintas Rodríguez

para a obtención do Título de Graduado en Enxeñaría Informática

**Desarrollo y evaluación de modelos de aprendizaje automático
para la predicción de la biodiversidad del suelo**



Setembro, 2026

Traballo de Fin de Grao N^o: EI 25/26-120

Titor/a: Javier Rodeiro Iglesias

Cotitor/a: Antonio Adrián González Pedrouzo

Área de coñecemento: Linguaxes y Sistemas Informáticos

Departamento: Informática

Dedicatoria

Agradecimientos

Índice

1. Introducción	1
2. Objetivos	2
3. Antecedentes y contexto	3
4. Resumen de la solución propuesta	6
5. Marco teórico o práctico	8
5.1. Fundamentos del aprendizaje automático	8
5.1.1. Tipos de aprendizaje automático	9
5.1.1.1. Aprendizaje supervisado	9
5.1.1.1.1. Minimización del riesgo empírico y capacidad del modelo	9
5.1.1.1.2. Caso particular: regresión de salida múltiple	10
5.1.1.2. Aprendizaje no supervisado	11
5.1.1.2.1. Formalización mediante una función objetivo interna	11
5.1.1.3. Aprendizaje semi-supervisado	12
5.1.1.3.1. Formalización del objetivo combinado	12
5.1.1.4. Aprendizaje por refuerzo	13
5.1.1.4.1. Formalización mediante una función objetivo interna	14
5.2. Selección de variables	15
5.2.1. Métodos de filtrado	15
5.2.2. Métodos de envoltura	15
5.2.3. Métodos de embebido	16
5.2.4. Variables empleadas	16
5.3. Modelos de aprendizaje automático empleados	18
5.3.1. Regresión Ridge	18
5.3.2. Random Forest	19
5.3.2.1. Random Forest multisalida	20
5.3.3. XGBoost	21
5.3.3.1. XGBoost multisalida	22
5.3.4. RegressorChain	23
5.3.5. Redes neuronales	24
5.3.5.1. MLP multisalida	24
5.3.5.2. MLP con función de pérdida personalizada	25
5.3.6. Comparativa	25
5.4. Métricas de evaluación y validación	26
5.4.1. Métricas de regresión	26
5.4.1.1. Coeficiente de determinación	26
5.4.1.2. Raíz del error cuadrático medio	26
5.4.1.3. Error absoluto medio	27
5.4.2. Métricas de clasificación	27
5.4.2.1. Exactitud (<i>Accuracy</i>)	27
5.4.2.2. Precisión, recall y F1	27
5.4.2.3. Coeficiente de Kappa de Cohen	28
5.4.2.4. Matrices de confusión	28
5.5. Arquitectura general	29
5.5.1. Capa de ingesta de datos	30
5.5.2. Capa de procesamiento de datos	31
5.5.3. Capa de modelado predictivo	32
5.5.4. Capa de evaluación de modelos	35
5.6. Herramientas empleadas	37
5.6.1. Lenguaje de programación, control de versiones y librerías	37
5.6.2. Servicios de obtención de datos remotos	37
5.6.3. Programas de redacción y documentación	38
5.6.4. Programas de desarrollo de código	38
5.6.5. Programas de creación de diagramas e ilustraciones	39
6. Planificación y seguimiento	40
6.1. Planificación	40
6.2. Puntos críticos	41
6.2.1. Desarrollo y parametrización de los modelos predictivos	42
6.2.2. Evaluación de modelos y control de calidad (QA)	42
6.2.3. Estrategias de mitigación	42
6.3. Seguimiento	43

6.4. Justificación de desviaciones	43
7. Principales aportaciones	44
8. Conclusiones	45
9. Vías de trabajo futuro	46
10. Referencias	47
11. Anexos	50
11.1. Informe EDA	50
11.2. Modelos empleados	51
11.2.1. Modelo de Regresión Ridge	51
11.2.1.1. Fundamento y configuración	51
11.2.1.2. Entrenamiento	51
11.2.1.3. Explicabilidad y variables más relevantes	52
11.2.1.4. Resultados sobre eval.csv	52
11.2.2. Modelo Random Forest	63
11.2.2.1. Fundamento y configuración	63
11.2.2.2. Entrenamiento	63
11.2.2.3. Explicabilidad y variables más relevantes	63
11.2.2.4. Resultados sobre eval.csv	64
11.2.3. Modelo Random Forest Multisalida	75
11.2.3.1. Fundamento y configuración	75
11.2.3.2. Entrenamiento	75
11.2.3.3. Explicabilidad y variables más relevantes	76
11.2.3.4. Resultados sobre eval.csv	76
11.2.4. Modelo RegressorChain con Random Forest	86
11.2.4.1. Fundamento y configuración	86
11.2.4.2. Ensemble of Chains (ECC)	86
11.2.4.3. Entrenamiento	86
11.2.4.4. Explicabilidad y variables más relevantes	87
11.2.4.5. Resultados sobre eval.csv	87
11.2.5. Modelo XGBoost	98
11.2.5.1. Fundamento y configuración	98
11.2.5.2. Entrenamiento	98
11.2.5.3. Explicabilidad y variables más relevantes	98
11.2.5.4. Resultados sobre eval.csv	99
11.2.6. Modelos XGBoost multisalida	109
11.2.6.1. Fundamento y configuración	109
11.2.6.2. Entrenamiento	109
11.2.6.3. Explicabilidad y variables más relevantes	109
11.2.6.4. Resultados sobre eval.csv	110
11.2.7. Modelos MLP mutlisalida	122
11.2.7.1. Fundamento y configuración	122
11.2.7.2. Entrenamiento	122
11.2.7.3. Explicabilidad y variables más relevantes	122
11.2.7.4. Resultados sobre eval.csv	123
11.2.8. Modelos MLP con función de pérdida personalizada	132
11.2.8.1. Fundamento y configuración	132
11.2.8.2. Entrenamiento	132
11.2.8.3. Explicabilidad y variables más relevantes	133
11.2.8.4. Resultados sobre eval.csv	133
11.3. Scripts y notebooks auxiliares	144
11.3.1. Script de automatización de notebooks	144
11.3.2. Script de automatización general	147
11.3.3. Notebook de división de datos	148
11.3.4. Notebooks para la comparación de modelos	153
11.3.4.1. Comparador de regresión	153
11.3.4.2. Comparador de clasificación	170
11.3.4.3. Comparador entre ramas	184
11.3.5. Notebook para la comparación de variables (Prueba I)	200
11.3.6. Notebook para la comparación de barrido de rs (Prueba III)	207
11.3.7. Notebooks para la prueba de ensamblado (Prueba IV)	215
11.3.7.1. Notebook de ensamblado de predicciones sin meta-modelos (Prueba IV.I)	215
11.3.7.2. Notebook de ensamblado de predicciones con meta-modelos (Prueba IV.II)	231
11.4. Pruebas llevadas a cabo	248

11.4.1. Prueba de eliminación de variables	248
11.4.2. Prueba de sensibilidad al random state	248
11.4.3. Prueba de barrido de random_state	248
11.4.4. Prueba de ensamblado de predicciones	248
11.5. Distribución de ramas y notebooks	249
11.5.1. Notebooks empleados	252
11.5.1.1. Comparación de resultados	253
11.5.1.2. Pruebas de ensamblado	254

Índice de figuras

Figura 1	Diagrama con las diferentes fases de CRISP-ML(Q)	6
Figura 2	Arquitectura de software del proyecto	29
Figura 3	Diagrama de Gantt con la planificación del TFG.	41
Figura 4	Diagrama de distribución de ramas	250

Índice de tablas

Tabla 1	Glosario de abreviaturas.	7
Tabla 2	Glosario de términos.	8
Tabla 3	Variables predictoras empleadas	16
Tabla 4	Variables objetivo empleadas	17
Tabla 5	Comparativa de modelos base	25
Tabla 6	Tabla de los datasets resultantes de la fase 1	32
Tabla 7	Tabla de submódulos empleados de la librería scikit-learn	33
Tabla 8	Tabla de submódulos empleados de la librería PyTorch	34
Tabla 9	Tabla de submódulos empleados de la librería scikit-learn (para evaluación)	36
Tabla 10	Planificación del TFG.	40
Tabla 11	Distribución de horas y porcentajes de esfuerzo estimados.	41
Tabla 12	Resultados de Ridge sobre eval.csv, por target.	52
Tabla 13	Resultados de Random Forest (salida única) sobre eval.csv, por target.	64
Tabla 14	Consenso de hiperparámetros en Random Forest multisalida.	75
Tabla 15	R ² de Random Forest multisalida sobre eval.csv, por target.	76
Tabla 16	Consenso de hiperparámetros en RegressorChain.	86
Tabla 17	R ² de RegressorChain (ensamble de 10 cadenas) sobre eval.csv, por target.	87
Tabla 18	Resultados de XGBoost (salida única, ensamble de 5 modelos) sobre eval.csv, por target.	99
Tabla 19	Resultados de XGBoost multisalida (ensamble) sobre eval.csv, por target.	110
Tabla 20	Consenso de hiperparámetros en MLP Multisalida.	122
Tabla 21	R ² del MLP multisalida sobre eval.csv, por target.	123
Tabla 22	Combinaciones de arquitecturas para MLP Custom Loss	132
Tabla 23	R ² del MLP con pérdida personalizada sobre eval.csv, por target.	134
Tabla 24	Distribución de ramas por objetivo/prueba	249
Tabla 25	Resumen de las ramas creadas.	251
Tabla 26	Notebooks de entrenamiento de modelos.	252
Tabla 27	Notebooks de comparación de resultados.	253
Tabla 28	Notebooks de pruebas de ensamblado.	254

Glosario de Términos

En este glosario se incluyen definiciones que puedan ser de utilidad y aclaraciones sobre las diferentes abreviaturas empleadas a lo largo de esta memoria.

Sigla	Significado
API	Interfaz de Programación de Aplicaciones (<i>Application Programming Interface</i>).
ASV	Variante de Secuencia de Amplicón (<i>Amplicon Sequence Variant</i>). Unidad taxonómica empleada en los targets de biodiversidad microbiana/molecular (bacterias, hongos, eucariotas, oomicetos, cercozoos) obtenidos por metabarcoding.
CDS	Copernicus Climate Data Store.
CRISP-ML(Q)	<i>Cross-Industry Standard Process for Machine Learning with Quality Assurance</i> .
DEM	Modelo de Elevación Digital.
ECC	<i>Ensembles of Chains</i> (Ensamblados de Cadenas), variante de RegressorChain empleada en este TFG.
GEE	Google Earth Engine.
MAE	Error Absoluto Medio.
MDI	<i>Mean Decrease Impurity</i> (Reducción Media de Impureza), medida nativa de importancia de variables en modelos de árboles.
MDP	Proceso de Decisión de Markov (<i>Markov Decision Process</i>).
ML	<i>Machine Learning</i> (Aprendizaje Automático).
MSE	Error Cuadrático Medio (<i>Mean Squared Error</i>).
NetCDF	Formato empleado por los archivos con extensión .nc, generalmente asociado a los datos descargados desde el CDS.
QA	<i>Quality Assurance</i> (Aseguramiento de la Calidad).
R2	Coefficiente de determinación.
RFE	<i>Recursive Feature Elimination</i> (Eliminación Recursiva de Variables).
RMSE	Raíz del Error Cuadrático Medio.
SHAP	<i>SHapley Additive exPlanations</i> .
SOB4ES	<i>Integrating SOil Biodiversity to Ecosystem Services</i> , proyecto europeo en cuyo marco se desarrolla este TFG.
TSS	<i>True Skill Statistic</i> , métrica de rendimiento habitual en modelos de distribución de especies.
VC	Dimensión Vapnik-Chervonenkis, medida de la capacidad de un espacio de hipótesis.
VIF	Factor de Inflación de la Varianza (<i>Variance Inflation Factor</i>).

Tabla 1: Glosario de abreviaturas.

Término	Definición
Accuracy	Proporción de predicciones correctas sobre el total, calculada tras la discretización de la predicción numérica en niveles ordinales de biodiversidad.
Bagging	<i>Bootstrap aggregating</i> . Técnica de ensamblado que entrena cada modelo base sobre una muestra aleatoria con reemplazo del conjunto de entrenamiento original, reduciendo la varianza del conjunto sin apenas aumentar el sesgo.
Cross-Validation (Validación cruzada)	Técnica de evaluación que divide el conjunto de datos en varios pliegues, entrenando y evaluando el modelo de forma repetida sobre distintas particiones, para obtener una estimación más robusta del rendimiento real que con una única partición.
F1-score	Media armónica entre Precisión y Recall, que penaliza con más fuerza que una media aritmética cuando una de las dos es baja aunque la otra sea alta.
Gradient Boosting	Técnica de ensamblado que construye modelos de forma secuencial, donde cada nuevo modelo corrige los errores cometidos por los anteriores. XGBoost es una implementación de esta técnica.
Kappa de Cohen	Métrica de concordancia entre la predicción y el valor real que corrige el nivel de acierto esperable por azar. Más robusta y fiable que el Accuracy, especialmente cuando las clases están desbalanceadas.
Matriz de confusión	Tabla que contrasta las clases predichas frente a las clases reales, permitiendo identificar en qué niveles de biodiversidad concreta se equivoca más el modelo.
Minimización del riesgo empírico (ERM)	Principio que aproxima el riesgo teórico, desconocido, de un modelo mediante el error medio calculado sobre el conjunto de entrenamiento disponible.
Overfitting (Sobreajuste)	Fenómeno por el cual un modelo aprende patrones específicos del conjunto de entrenamiento, incluido su ruido, en lugar de la relación subyacente, perjudicando su capacidad de generalización a datos nuevos.
Precisión	Proporción de muestras etiquetadas por el modelo como pertenecientes a una clase que realmente lo eran.
Recall	Proporción de casos realmente pertenecientes a una clase que el modelo logra detectar correctamente. Especialmente informativo cuando las clases están desbalanceadas.
Regularización	Método estadístico empleado para reducir los errores causados por el sobreajuste de los datos de entrenamiento, penalizando la complejidad del modelo (p. ej. L2 en Ridge, L1 en Lasso).
Underfitting	Fenómeno opuesto al sobreajuste: el modelo carece de capacidad suficiente para capturar las relaciones reales presentes en los datos, presentando un rendimiento pobre incluso sobre el propio conjunto de entrenamiento.

Tabla 2: Glosario de términos.

1. Introducción

El suelo alberga aproximadamente el 59% de la biodiversidad de nuestro planeta. Este conjunto de organismos genera un gran impacto en el ciclo de nutrientes, el mantenimiento de las estructuras del suelo y la salud de los ecosistemas. En última instancia, la vida que encontramos en los suelos representa la base funcional de los ecosistemas terrestres^[1].

Por estos motivos, resulta fundamental poder controlar y monitorizar el estado de los suelos y su biodiversidad. Sin embargo, nos encontramos con numerosos desafíos a la hora de medir dicha biodiversidad, entre los cuales destacamos los siguientes:

1. El número elevado de especies.
2. Las dificultades metodológicas según el grupo de especies estudiado.
3. La falta de procesos estandarizados.
4. El elevado coste de los diversos análisis a realizar^[2].

Debido a dichos desafíos y limitaciones, surge la necesidad de desarrollar nuevas técnicas que permitan y faciliten conocer los niveles de biodiversidad en el suelo, asegurando también la fiabilidad y la calidad de los resultados.

En el marco del proyecto europeo SOB4ES («Integrating SOil Biodiversity to Ecosystem Services: testing cost-effectiveness of Soil Biodiversity indicators and the provision of soil biodiversity-based Ecosystem Services to build better land management solutions that effectively implement the EU Soil Strategy»^[3]), en el que se sitúa el presente TFG, se han obtenido mediciones de múltiples propiedades físicas, químicas y ambientales de más de 400 puntos geográficos diferentes a lo largo del continente europeo y sus proximidades. Adicionalmente, se ha evaluado la biodiversidad de dichos suelos a través de múltiples características.

2. Objetivos

El objetivo del presente trabajo es realizar un estudio de los datos del proyecto SOB4ES para desarrollar, entrenar y evaluar modelos de aprendizaje automático que sean capaces de aproximar los valores de biodiversidad para nuevos puntos geográficos, a partir de sus propiedades físicas, químicas y ambientales.

A partir de este objetivo central se proponen los siguientes objetivos específicos:

1. Analizar e integrar los datos provenientes del proyecto europeo SOB4ES con datos climáticos, satelitales y edafológicas externas procedentes de bases de datos geoespaciales y de diversos servicios API como Google Earth Engine (GEE)^[4] y Copernicus (CDS)^[5].
2. Preparar y armonizar los datos disponibles mediante un proceso reproducible de limpieza, normalización y homogeneización de las distintas fuentes heterogéneas, obteniendo un conjunto de datos consistente que sirva de base fiable para el análisis y modelado posterior.
3. Seleccionar y justificar, desde un punto de vista metodológico, las técnicas y herramientas de análisis de datos y aprendizaje automático más adecuadas a las características del problema (tamaño muestral reducido, múltiples indicadores correlacionados entre sí, datos heterogéneos procedentes de fuentes distintas).
4. Entrenar, comparar y evaluar métricamente diferentes algoritmos y modelos de aprendizaje automático supervisado para aislar y determinar la solución algorítmica más precisa, robusta y con mayor capacidad de generalización.
5. Garantizar el rigor metodológico y la reproducibilidad de los resultados obtenidos, mediante la aplicación sistemática de una metodología de gestión de proyectos de ciencia de datos (CRISP-ML(Q)) que permita mitigar riesgos y sesgos a lo largo del proceso de investigación.

3. Antecedentes y contexto

El proyecto europeo SOB4ES, en cuyo marco se desarrolla este TFG, surge como respuesta a la necesidad de la Unión Europea de contar con indicadores fiables y de bajo coste sobre la biodiversidad del suelo, en contexto de la Estrategia de Suelos de la UE (*EU Soil Strategy*) [3]. Tradicionalmente, la evaluación de la biodiversidad edáfica se ha basado en muestreos de campo y análisis de laboratorio, que si bien son precisos, presentan las limitaciones ya descritas en la Introducción: coste elevado, falta de estandarización metodológica entre grupos y especies, y dificultad para escalar todo a nivel continental o internacional.

Ante estas limitaciones, en los últimos años se ha extendido el uso de modelos de aprendizaje automático supervisado para estimar indicaciones ambientales y de biodiversidad a partir de variables predictoras de obtención más económica (datos climáticos, satelitales y edafológicos), reduciendo así la dependencia de mediciones directas exhaustivas sobre el terreno.

En un plano más general, la revisión de Wadoux^[6] sitúa este tipo de aplicaciones dentro de un marco disciplinar más amplio: propone una taxonomía de la inteligencia artificial en ciencia del suelo, construida a partir de una clasificación previa de la Comisión Europea y refinada mediante revisión de literatura y consulta a personas expertas, que distingue tres grandes dominios: percepción e interacción, razonamiento y toma de decisiones, y aprendizaje y predicción. Los modelos de aprendizaje supervisado empleados en este TFG se enmarcan dentro de este último dominio, señalado por la propia revisión como el más consolidado entre las aplicaciones actuales de IA a la ciencia del suelo, lo que sitúa este trabajo dentro de una tendencia disciplinar más amplia y no como un caso aislado.

Varios trabajos han abordado directamente el modelado de biodiversidad del suelo haciendo uso del aprendizaje automático, con resultados que nos pueden servir de referencia para contrastar con los resultados obtenidos en este TFG (véase Conclusiones):

1. Salako *et al.*^[7] emplearon **Random Forest** para modelar la distribución espacial de la riqueza y densidad (ind./m²) de tres grupos ecológicos de lombrices (epigeas, endogeas y anécicas) en Alemania, usando como predictores variables de suelo y clima sobre distintos usos del terreno.

Encontraron que cada grupo ecológico responde a predictores distintos: las especies epigeas están mayoritariamente condicionadas por el clima en zonas forestales, mientras que las endogeas, el grupo más diverso, dependen sobre todo de la textura del suelo (contenido de arcilla y limo).

2. Phillips *et al.* [8] compilaron el conjunto de datos de revisado de mayor escala sobre lombrices de tierra (6928 puntos de muestreo en 57 países) para predecir riqueza de especies, abundancia y biomasa mediante tres modelos lineales mixtos generalizados (GLMM), uno por cada métrica, a partir de 12 variables ambientales agrupadas en seis temas (suelo, precipitación, temperatura, retención de agua, cobertura del hábitat y otras).

- A diferencia del resto de estudios de esta lista, no emplean aprendizaje automático en sentido estricto, sino un **modelo estadístico clásico** validado mediante *10-fold cross-validation*.
- Se incluye en estos antecedentes por ser, con diferencia, el estudio de mayor escala sobre biodiversidad de lombrices de tierra revisado, y porque se compara explícitamente el error de un modelo que combina propiedades de suelo medidas en campo con variables de SoilGrids frente a un modelo que usa únicamente SoilGrids (una capa global derivada, no medida *in situ*).
- La comparación *in situ* + remoto vs. solo remoto es la más directamente equiparable, dentro de la literatura revisada, a la que plantea este TFG (SOB4ES + Google Earth Engine/Copernicus).
- Encuentran que las variables climáticas resultan más determinantes que las edáficas o la cobertura del hábitat, y que la riqueza y abundancia locales tienden a ser mayores en latitudes altas, lo cual es un patrón inverso al de la biodiversidad aérea.

3. Un estudio de modelado conjunto multiespecie^[9] entrenó una **red neuronal profunda multi-tarea** para predecir la distribución de 77 especies de lombrices en Francia a partir de variables climáticas, edáficas y de cobertura del terreno, empleando SHAP para identificar los principales factores ambientales. El modelo conjunto alcanzó un TSS ≥ 0.7 y mejoró notablemente las predicciones para especies raras frente a los modelos tradicionales de distribución de especies (basados en una sola especie).

4. Un estudio sobre biodiversidad multi-trófica del suelo mediante metabarcoding de ADN ambiental [10] comparó sistemáticamente **LightGBM, Random Forest y una red neuronal (ANN)** para predecir la abundancia relativa de 51 grupos tróficos (bacterias, hongos, protistas, oligoquetos, insectos, colémbolos y otros metazoos), usando cuatro configuraciones de variables predictoras: datos ambientales de baja resolución, datos *in situ* de alta calidad, *embeddings* de imágenes satelitales, y una combinación de ambos.

Random Forest obtuvo consistentemente el mejor rendimiento, ligeramente por encima de *LightGBM*, y los datos *in situ* de alta calidad superaron sistemáticamente tanto a los datos de baja resolución como a los derivados de imágenes satelitales. Un resultado directamente relevante para este TFG, que también combina variables *in situ* (SOB4ES) con variables remotas de menor resolución (Google Earth Engine, Copernicus).

5. Un estudio de predicción del microbioma del suelo [11] comparó seis modelos de aprendizaje automático clásico (incluyendo *Random Forest* y *Gradient Boosting*) frente a un modelo de aprendizaje profundo para predecir la frecuencia relativa de comunidades bacterianas y fúngicas a partir de factores ambientales.

Gradient Boosting obtuvo el mejor resultado a nivel de filo ($R^2 = 0.57$), y *Random Forest* y *Gradient Boosting* empataron como mejores modelos a nivel de grupo funcional ($R^2 = 0.45$); las arquitecturas más sofisticadas de aprendizaje profundo (CNN, *transformers*) obtuvieron resultados peores, atribuido explícitamente al tamaño reducido de los datos de entrenamiento disponibles. Esto es una conclusión especialmente relevante para este TFG, cuyo conjunto de datos (400 muestras) es de un orden de magnitud similar.

En conjunto, estos antecedentes apuntan de forma consistente a que los modelos basados en árboles, como *Random Forest*, *Gradient Boosting* y *XGBoost*, pueden igualar o hasta superar a los modelos basados en redes neuronales cuando el conjunto de datos es reducido (como suele ocurrir en los datos relacionados con la biodiversidad). Asimismo, apuntan a que los datos *in situ* de alta calidad siguen siendo, por ahora, más informativos que los derivados exclusivamente de fuentes remotas con resoluciones bajas, conclusión a la que también apunta, desde un modelo estadístico distinto y a una escala mucho mayor, Phillips et al. [8]. Estas dos hipótesis pueden ser contrastadas empíricamente con los resultados de este TFG.

Este enfoque indirecto no es exclusivo de la biodiversidad del suelo. El uso de variables climáticas, satelitales y edáficas como predictoras de propiedades del ecosistema está ya consolidado en ámbitos afines, de los cuales se resaltan los siguientes:

- **Estimación del carbono orgánico en el suelo:** Tian et al. [12] emplean *Random Forest* y *quantile regression forests* sobre 45616 observaciones de suelo y variables de teledetección para mapear la densidad de carbono orgánico en toda la Unión Europea a 30 metros de resolución y cuatro intervalos de profundidad, obteniendo un R^2 de 0.63 en validación independiente.
- **Clasificación de usos del terreno:** Rodríguez-Galiano et al. [13] muestran que *Random Forest*, aplicado sobre imágenes Landsat y un modelo digital del terreno en el sur de España, clasifica 14 categorías de cobertura del suelo con una precisión del 92% y supera de forma significativa a un único árbol de decisión. Esto da un resultado que anticipa, en otro dominio, la ventaja del *bagging* frente a un modelo individual, que también es un elemento que se discute dentro de este TFG.
- **Predicción del rendimiento agrícola a partir de series temporales de NDVI:** La revisión de Shawon et al. [14] sobre 97 estudios publicados entre 2017 y 2024, encuentra que *Random Forest* y *Gradient Boosting Trees* están entre los algoritmos más empleados para predecir el rendimiento de los cultivos combinando temperatura, tipo de suelo e índices de vegetación como NDVI. Esta es la misma combinación de datos y la misma familia de algoritmos empleados en este TFG, pero usados para resolver un problema de predicción distinto.

La disponibilidad reciente de catálogos satelitales abiertos de alta resolución y de reanálisis climáticos globales ha reducido significativamente la barrera de entrada para poder aplicar estas técnicas a nuevos problemas ecológicos, entre los cuales se encuentra el que aborda este TFG.

Frente a estos antecedentes, la aportación diferencial que puede proporcionar este TFG es doble:

1. Por un lado, la integración de múltiples fuentes de orígenes heterogéneos en un mismo *pipeline* reproducible y versionado.
2. Por otro lado, la comparación de múltiples algoritmos de aprendizaje supervisado bajo un mismo marco de validación, en lugar de limitarse a un único modelo, lo que permite discutir con evidencia empírica cuál generaliza mejor ante un conjunto de datos comparativamente reducido.

A esto se suma el hecho de que ni trabajos como el de Phillips et al. [8] ni el estudio multitrófico [10] emplean fuentes de acceso de uso extendido como Google Earth Engine o Copernicus, ni combinan múltiples grupos biológicos del suelo de forma simultánea, como se hace en este TFG, al utilizar los datos del **proyecto SOB4ES** [3].

Este TFG se sitúa bajo el contexto siguiente: comienza a partir de los datos previamente recolectados por el proyecto SOB4ES en más de 400 puntos geográficos europeos, y los complementa con fuentes de datos remotas de acceso abierto, con el objetivo de evaluar hasta qué punto los modelos de aprendizaje automático supervisado pueden aproximar la biodiversidad del suelo a partir de variables indirectas, relativamente más baratas y escalables que las obtenidas mediante el muestreo directo.

4. Resumen de la solución propuesta

La solución propuesta se basa en un enfoque integral de ciencia de datos, diseñado para abarcar desde la recopilación de múltiples fuentes de información hasta la evaluación de modelos predictivos.

Para estructurar este proceso, se ha adoptado la metodología de desarrollo CRISP-ML(Q) [15], que permitirá definir y realizar el trabajo de manera estructurada. Además de la metodología CRISP-ML(Q), a lo largo del desarrollo de este trabajo se aplicará en conjunto una metodología de desarrollo iterativa incremental [16].

CRISP-ML(Q) (*Cross-Industry Standard Process of Machine Learning with Quality Assurance*) es el estándar de facto para la gestión del ciclo de vida en proyectos de aprendizaje automático. A diferencia de la ingeniería de software tradicional o la minería de datos clásica (CRISP-DM [17]), el desarrollo de modelos predictivos exige una aproximación principalmente experimental, no lineal e iterativa, además de ser un proyecto en el cual el rendimiento de los modelos puede verse degradado con el tiempo.

Para dotar a este proceso de un marco de ingeniería robusto y estricto, CRISP-ML(Q) organiza el flujo de trabajo en seis fases cíclicas interconectadas entre sí, integrando así, de forma transversal, los diversos controles de QA (*Quality Assurance*) en cada una de las transiciones.

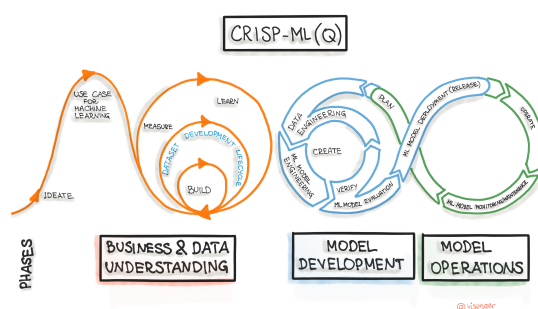


Figura 1: Diagrama con las diferentes fases de CRISP-ML(Q)

En el diagrama anterior, podemos distinguir las siguientes fases:

1. **Comprensión del negocio y de los datos:** Definición del problema predictivo y auditoría inicial de la viabilidad de las fuentes proporcionadas.
2. **Preparación de los datos:** Fase de ingeniería de características y de datos, limpieza, unificación y estandarización de los datos de las fuentes base. Junto también con la obtención de datos externos.
3. **Ingeniería del modelo:** Selección de algoritmos/tipos de modelo, entrenamiento formal, ajuste de hiperparámetros y su consecuente desarrollo de los modelos necesarios.
4. **Evaluación del modelo:** Validación con los correspondientes parámetros de evaluación del rendimiento estadístico frente a datos de control ciegos (nunca empleados en entrenamiento ni en otras pruebas anteriores a la evaluación).
5. **Despliegue:** Entrega o integración del modelo o *pipeline* reproducible en el entorno operativo.
6. **Monitoreo y mantenimiento:** Control continuo para mitigar la degradación de las predicciones en el tiempo y aplicar las correspondientes medidas de mantenimiento.

Además, el proyecto también cuenta con un componente metodológico iterativo incremental, descrito a continuación.

El **desarrollo iterativo incremental** se basa en la capacidad de poder dividir un proyecto en diversos bloques reducidos y asignarlos a flancas temporales generalmente fijos, los cuales pueden ser adaptados a futuro. Estos bloques reducidos se denominan como **iteraciones**.

De forma más específica, las iteraciones se pueden definir como miniproyectos o como un set de tareas en las cuales «se repite un proceso de trabajo similar para proporcionar un resultado completo sobre el producto final» [16]. Cada iteración tiene como resultado una parte del producto final funcional, el cual se va integrando a lo largo de las iteraciones hasta crear el producto final.

Todos los detalles relacionados con las fases, hitos y los tiempos de realización de todo el proyecto se pueden encontrar en **Planificación y seguimiento**.

La aplicación conjunta de ambas metodologías (CRISP-ML(Q), junto con desarrollo iterativo incremental) se justifica por los siguientes motivos:

1. **Permite acortar en el tiempo las fases de mayor incertidumbre de CRISP-ML(Q).**

- Las fases de Ingeniería del modelo y Evaluación del modelo son, por su propia naturaleza, las que más ciclos de prueba-error requieren en un proyecto de aprendizaje automático. Tratarlas como un bloque monolítico dificultaría la estimación de tiempos y la detección de desviaciones en el tiempo; al dividir las iteraciones concretas, cada una entrega un resultado evaluable en el plazo acotado.

2. **Cada iteración produce un incremento funcional y evaluable.**

- En este caso, un modelo entrenado junto con sus métricas de rendimiento, en lugar de posponer cualquier resultado tangible hasta el final del proyecto. Esto permite detectar pronto problemas transversales, como el rendimiento inesperado de un modelo o la necesidad de revisar alguna variable predictora, sin tener que esperar a que todo el pipeline esté terminado.

3. **Facilita adaptar el alcance y las decisiones de modelado a medida que se dispone de más información.**

- En lugar de fijar de antemano qué algoritmos, variables o hiperparámetros se van a emplear. Esto es especialmente relevante en un trabajo que parte de datos ambientales heterogéneos y en evolución dentro de un proyecto europeo en curso (SOB4ES), donde el conocimiento sobre qué variables resultan realmente predictivas se afina en cada iteración.

4. **Mantiene el control de calidad transversal propio de CRISP-ML(Q).**

- Mantener dicho control en cada iteración nos permite evitar que la naturaleza iterativa del desarrollo derive en una pérdida de rigor a nivel metodológico: cada incremento pasa por las mismas comprobaciones de validación y evaluación antes de darse por válido, en lugar de acumular deuda técnica o metodológica de una iteración a la siguiente.

A continuación se da un pequeño desglose de las partes en las que se divide esta solución que se propone. La división detallada, como también los elementos que se van a emplear, se encuentran en la siguiente sección (véase **Marco teórico o práctico**).

1. **Adquisición e Integración de Datos:** Para alimentar los modelos, no solo se utilizarán los datos propios del proyecto SOB4ES (disponibles en archivos y carpetas como EARTHWORKS_RAW/), sino que se complementarán y completarán con bases de datos y mapas europeos de resolución variable. Las fuentes externas empleadas son las siguientes:

- Bases de datos geoespaciales como la *European Soil Database* (ESDAC) y *CORINE Land Cover* para clasificar el tipo de suelo, uso de la tierra, propiedades físicas (arena, arcilla, densidad) y propiedades químicas (pH, metales pesados, carbono orgánico) de las parcelas.
- Uso de APIs como *Google Earth Engine* (earthengine-api) para extraer medias de temperatura diaria, humedad relativa e índices de vegetación (NDVI).
- Uso de *Copernicus Climate Data Store* (cdsapi) para registrar precipitaciones mensuales acumuladas y modelos de elevación digital (DEM).

2. **Procesamiento de la Información:** El tratamiento masivo de estos datos ambientales se realizará utilizando diversas bibliotecas de Python. Se utilizarán pandas, numpy y otras librerías para la limpieza, manipulación eficiente de variables tabulares y escritura de archivos CSV/XLSX. Para el manejo de datos geoespaciales y archivos multidimensionales provistos por Copernicus y bases europeas, se hará uso herramientas específicas como rasterio, xarray y dbfread. Dichas librerías se mencionan con más detalle en **Arquitectura empleada**.

3. **Modelado y Evaluación:** Una vez consolidado el conjunto de datos definitivo, la solución final consistirá en el diseño, entrenamiento e implementación de diferentes modelos de aprendizaje automático, aplicando en cada uno de los diferentes modelos la metodología explicada previamente y cuyas fases/etapas se encuentran en las secciones de Planificación y seguimiento y Arquitectura. Estos modelos mapearán las complejas relaciones entre las variables ambientales y la biodiversidad del suelo.

Finalmente, los modelos serán evaluados y comparados métricamente para aislar la solución algorítmica más robusta y generalizable para futuras predicciones geográficas, siguiendo el ciclo de evaluación-mejora descrito en las fases de **Ingeniería del modelo** y **Evaluación del modelo** de CRISP-ML(Q).

5. Marco teórico o práctico

En esta sección se describe el marco teórico y práctico en el que se apoya y fundamenta nuestras conclusiones: la arquitectura del pipeline de datos y modelado, las tecnologías y herramientas de tercero empleadas, como también los fundamentos teóricos que nos permite crear, desarrollar, entender e interpretar los resultados que nos proporcionan los diferentes modelos seleccionados.

5.1. Fundamentos del aprendizaje automático

El término de aprendizaje automático, también conocido como *Machine Learning* (ML) fue acuñado por primera vez por el informático **Arthur Samuel**^{[18],[19]} en 1959, conocido por su trabajo en la implementación de un damas que aprendiera de forma autónoma y por dar soporte y colaborar en lo que hoy se conoce como LaTeX^[20].

Él acuñó el término de ML por primera vez en el artículo “*Some Studies in Machine Learning Using the Game of Checkers*” publicado en el IBM Journal of research and development^[21], definiéndolo de la siguiente forma:

“Machine Learning es el campo de estudio que permite le otorga a las computadoras la capacidad de aprender sin explícitamente programarlo.”

Otro personaje relevante dentro del campo de la inteligencia artificial es **Tom Mitchell**^{[19], [22]}, creador del ordenador que es capaz de “leer la mente humana” a partir de imágenes del cerebro humano^[23]. Él lo define de una forma más técnica, siendo esta una de las definiciones más citadas dentro de la literatura académica^[24], la cual es la siguiente:

“Se dice que un programa de ordenador aprende de una experiencia E con respecto a una tarea T y una media de rendimiento P, si su rendimiento en T, medido por P, mejora con la experiencia E”

Esta definición es la adoptada como referencia en este trabajo, al descomponer con claridad los tres elementos presentes del proyecto:

- **Tarea (T):** predecir los indicadores de biodiversidad del suelo a partir de las diferentes variables ambientales.
- **Experiencia (E):** El conjunto de datos de campo del proyecto SOB4ES, combinado con variables climáticas y geoespaciales de fuentes remotas.
- **Medida de rendimiento (P):** las métricas descritas en la Sección 5.4, la cual se encuentra un poco más adelante en este documento.

Estas dos definiciones se pueden resumir de la siguiente forma ^[25]:

“El Machine Learning es una rama o subconjunto de la Inteligencia Artificial que se centra en dotar a los algoritmos la capacidad de “aprender”. Este aprendizaje es a partir de la detección de patrones en los datos de entrenamiento para, posteriormente, hacer predicciones o inferencias sobre nuevos datos.”

Esta capacidad de aprender, permite a los modelos de ML realizar predicciones, tomar decisiones y hacer acciones variadas sin que lo tengan estrictamente indicado dentro de su código. También lo permite diferenciarse de los sistemas expertos tradicionales, en los que el conocimiento se codifica explícitamente, y lo acerca a la estadística de la que toma buena parte de su base matemática.

5.1.1. Tipos de aprendizaje automático

Dentro del aprendizaje automático, existen muchos algoritmos y múltiples formas de aprendizaje en base al tipo de datos o experiencias de las que aprenden.

Dichas formas de aprendizaje se clasifican en cuatro tipos generales, los cuales se explican a continuación.

5.1.1.1. Aprendizaje supervisado

Dentro del **aprendizaje supervisado**, el algoritmo aprende a partir de un conjunto de datos en el que cada observación de entrada (las variables predictoras) tiene asociada una salida conocida (la variable a predecir, o *target*). Este tipo de modelos se entrenan haciendo uso de un conjunto de datos etiquetado^{[24], [19]}, es decir, un conjunto de pares entrada-salida (x_i, y_i) donde y_i es el valor real que se desea predecir.

El objetivo del algoritmo de aprendizaje es encontrar una función f tal que $f(x_i) \approx y_i$, para las muestras de entrenamiento, y que generalice razonablemente las muestras nuevas no vistas durante el entrenamiento:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \Omega(f)$$

donde:

- $\mathcal{H}$ es el espacio de funciones que el algoritmo puede representar (Hipótesis).
- L es la función de pérdida que mide el error de predicción.
- $\Omega(f)$ es un término de regularización opcional que penaliza la complejidad de f para evitar el sobreajuste.

Dentro del aprendizaje supervisado se distinguen dos subtipos según la naturaleza de la salida (y):

- **Regresión:** La salida es una variable numérica continua, como lo puede ser, por ejemplo, un índice de biodiversidad expresado como un número real.
- **Clasificación:** La salida es una categoría discreta, como podría ser los índices de biodiversidad pero, en vez de usar el valor real, usando un criterio de clasificación como «bajo/medio/alto».

Como se menciona previamente, el aprendizaje supervisado necesita, casi por definición, un conjunto de datos ya etiquetado, lo que suele implicar un coste de recolección considerable. En este caso, el muestreo de campo y el análisis de laboratorio descritos en la **Introducción** de este TFG.

A cambio, es el paradigma que permite una evaluación más objetiva del modelo, al poder comparar directamente la predicción con un valor real conocido, algo que no es posible, o no de forma tan directa, en los otros tres paradigmas.

Algunos algoritmos representativos de este paradigma son la regresión lineal, los árboles de decisión, los métodos de ensamblado (Random Forest, XGBoost) y las redes neuronales entrenadas con ejemplos etiquetados, varios de los cuales se emplean en este propio trabajo (véase **Modelos empleados**).

5.1.1.1.1. Minimización del riesgo empírico y capacidad del modelo

La expresión anterior es un caso particular del **principio de minimización del riesgo empírico** (*Empirical Risk Minimization*, ERM)^[24]: como no se conoce la distribución real que generan los datos se aproxima el riesgo empírico.

Esta aproximación introduce un problema central dentro del aprendizaje automático: la minimización del riesgo empírico sin ningún control sobre la complejidad de $\mathcal{H}$ puede producir una función que memorice el ruido específico del conjunto de entrenamiento en lugar de aprender el patrón subyacente; este es el fenómeno de **sobreajuste** o **overfitting**.

El término $\Omega(f)$ existe precisamente para mitigar el riesgo de sobreajuste, y es la razón teórica que justifica, por ejemplo, la penalización $\alpha \|w\|^2$ de Ridge o el término

$$\Omega(h_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

de XGBoost (véase **Regresión Ridge** y **XGBoost**).

Compromiso sesgo-varianza: El error de generalización de un modelo puede descomponerse, para un punto x cualquiera, en tres términos:

$$\mathbb{E}[(y - \hat{f}(x))^2] = (\mathbb{E}[\hat{f}(x)] - f(x))^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

siendo:

- $(\mathbb{E}[\hat{f}(x)] - f(x))^2$ el sesgo al cuadrado.
- $\text{Var}[\hat{f}(x)]$ la varianza.
- σ^2 el ruido irreducible del error/punto.

Esto implica las siguientes deducciones/afirmaciones:

- Un modelo de **poca capacidad** tiende generalmente a presentar un **sesgo elevado y baja varianza**, haciéndolo incapaz de capturar relaciones reales entre los datos a pesar de ser estable dentro de las diferentes muestras de entrenamiento. A este fenómeno se lo denomina **underfitting**.
- Un modelo de **alta capacidad** tiende a justamente lo contrario: **bajo sesgo, elevada varianza**; esto se debe a que cualquier cambio pequeño en los datos de entrenamiento provoca que se generen modelos muy distintos entre sí, provocando **overfitting**.

Capacidad y dimensión Vapnik-Chervonenkis (VC): De forma más formal, la teoría del aprendizaje estadístico acota el riesgo teórico en función del riesgo empírico y de la capacidad del espacio de hipótesis $\mathcal{H}$: a mayor capacidad, mayor es la diferencia potencial entre el error de entrenamiento y el error real, y por tanto mayor es el número de muestras necesario para que el error empírico sea un estimador fiable del error real.

Parte de las decisiones tomadas dentro de este TFG en relación con el control de capacidad y el control de sesgo-varianza, radican en las nociones teóricas dispuestas dentro de este apartado.

5.1.1.1.2. Caso particular: regresión de salida múltiple

Un elemento importante de este TFG, el cual se hace uso, es hacer que y no sea un único valor, sino que sea un **vector** de varios indicadores de biodiversidad de forma simultánea:

$$y = (y_1, y_2, \dots, y_K)$$

Esto sitúa el problema dentro del subcampo de la **regresión multisalida**, en el que caben, a grandes rasgos, tres estrategias principales:

- **Modelo por target:** Consiste en entrenar K modelos de regresión de salida única, una por cada y_k , ignorando cualquier relación entre los diferentes targets.
- **Multisalida simétrica o nativa:** Consiste en entrenar un único modelo que predice el vector completo y a la vez, compartiendo parámetros o estructura interna entre todos los targets.
- **Multisalida encadenada o direccional:** Consiste en entrenar una secuencia de modelos donde cada uno usa las predicciones de los modelos anteriores como variables adicionales a las ya existentes como variables predictoras o *features*.

Desde la perspectiva del compromiso sesgo-varianza, comentado en el apartado anterior, compartir parámetros entre los targets (estrategias 2 y 3) puede ser visto como una forma adicional de regularización: si los K targets están correlacionados entre sí, forzar al modelo a explicarlos con una representación compartida reduce la varianza efectiva del conjunto de modelos, a costa de introducir un sesgo si algún target concreto se comporta de forma distinta al resto.

Estas tres estrategias, junto con la comparación empírica entre ellas, son precisamente el objeto de estudio de la *Comparación entre los modelos empleados* (véase) y los resultados que se obtuvieron en las diferentes pruebas adicionales, las cuales se pueden ver en el *Anexo V*.

5.1.1.2. Aprendizaje no supervisado

Dentro del **aprendizaje no supervisado**, el algoritmo recibe únicamente datos de entrada, sin ninguna salida asociada, y su objetivo es encontrar estructura o patrones en los propios datos.

Los datos que se proporcionan en este tipo de aprendizajes son datos no etiquetados [19], [25]. Solo se dispone de las entradas x_i , sin ningún valor y_i asociado.

Al no existir una variable objetivo, no hay una noción o una idea directa de lo que puede ser considerado un «error de predicción» que minimizar; por lo tanto, en su lugar, cada familia de métodos define su propio criterio de qué constituye una buena estructura descubierta.

Dentro de este paradigma destacan las siguientes familias de métodos:

1. **Clustering o agrupamiento:** Se agrupan las observaciones que se determinan como similares entre sí, aunque no exista una categoría predefinida. Esto es común de algoritmos como k-means o el clustering jerárquico.
2. **Reducción de la dimensionalidad:** Busca representar los datos originales con el menor número de variables posibles, conservando la mayor parte posible de la información o varianza original. Esto es común del Análisis de Componentes Principales o PCA.
3. **Estimación de densidad y detección de anomalías:** Modelos de distribución de probabilidad $p(x)$ subyacente a los datos, de forma que las muestras con baja probabilidad bajo el modelo aprendido puedan señalarse como atípicas o anómalas.
4. **Modelado generativo:** Aprender a generar nuevas muestras sintéticas que sigan la misma distribución que los datos de entrenamiento, elemento base, entre otras cosas, de los modelos de fundación, los cuales se mencionan en el apartado siguiente. Común de los autoencoders o de los modelos de difusión.

No se ha empleado en este trabajo como técnica de modelado principal, ya que se dispone en todo momento de mediciones reales de biodiversidad que actúan como salida conocida; no obstante, algunas de sus ideas, como la **reducción de variables correlacionadas (reducción de la dimensionalidad)**, están relacionadas con la selección de variables descritas en la **Sección 5.2**.

5.1.1.2.1. Formalización mediante una función objetivo interna

A diferencia del aprendizaje supervisado, donde el objetivo $L(y_i, f(x_i))$ se define por comparación directa con una etiqueta externa y_i , dentro del aprendizaje no supervisado se sustituye la señal producida por y_i por un criterio interno, definido únicamente en función de los propios datos de x_i , que cada familia de métodos formaliza de maneras distintas:

Clustering (K-Means): dado un número de grupos k , se buscan centroides $\mu_1, \dots, \mu_k$ que minimicen la suma de distancias al cuadrado dentro de cada grupo (inercia):

$$\hat{\mu} = \arg \min_{\mu_1, \dots, \mu_k} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Reducción de dimensionalidad (PCA): se buscan las direcciones $w_1, \dots, w_d$, con d menor a la dimensión original, que maximizan la varianza de los datos proyectados, lo que equivale a resolver un problema de autovalores sobre la matriz de covarianza Σ de los datos:

$$\hat{w} = \arg \max_{\|w\|=1} w^T \Sigma w \rightarrow \Sigma w = \lambda w$$

Estimación de densidad: se busca la distribución de probabilidad $p_\theta(x)$, dentro de una familia paramétrica, que maximiza la verosimilitud de los datos observados, para los cuales las muestras con $p_\theta(x)$ muy baja se señalan como anómalas.

$$\hat{\theta} = \arg \max_{\theta} \sum_i \log p_\theta(x_i)$$

En los tres casos, la ausencia de una variable y externa obliga a sustituir la noción de «error de predicción» por una noción de «calidad de la estructura descubierta», lo cual implica que no existe una forma tan directa como en el aprendizaje supervisado de validar el resultado frente a un conjunto de test independiente.

5.1.1.3. Aprendizaje semi-supervisado

El **aprendizaje semi-supervisado**^[19] es un punto intermedio entre los dos anteriores: se dispone de una pequeña cantidad de datos etiquetados (con salida conocida) y una cantidad mucho mayor de datos sin etiquetar, y el algoritmo aprovecha ambos para mejorar el modelo, por ejemplo mediante técnicas de **self-training**, en las que el propio modelo etiqueta provisionalmente los datos no etiquetados y los incorpora al entrenamiento si su confianza es suficientemente alta.

Es habitual en dominios donde etiquetar datos es costoso, pero recolectar datos sin etiquetar es barato, como ocurre por ejemplo en visión por ordenador.

Los modelos semi-supervisados no funcionan de forma automática por el mero hecho de añadir datos sin etiquetar: su validez depende de que se cumpla alguno de estos supuestos sobre la estructura de los datos^[19]:

- **Supuesto de suavidad:** Dos puntos cercanos en el espacio de entrada tienden a tener la misma etiqueta o un valor similar, en regresión.
- **Supuesto de agrupamiento:** Los puntos tienden a formar agrupaciones naturales, y los puntos dentro de un mismo cluster comparten etiqueta.
- **Supuesto de variedad:** Los datos de alta dimensión se concentran en realidad sobre una variedad de dimensión mucho menor, y es sobre esa variedad donde tiene sentido medir la similitud entre puntos.

Dentro de este paradigma destacan las siguientes familias de métodos:

- **Self-training:** Un modelo se entrena con los datos etiquetados, etiqueta con sus propias predicciones más confiables los datos sin etiquetar, y se reentrena incluyéndolos.
- **Co-training:** Dos modelos entrenados sobre vistas distintas de los datos se etiquetan mutuamente los ejemplos sin etiquetar.
- **Métodos basados en grafos:** Propagan las etiquetas conocidas a través de un grafo de similitud construido sobre todos los puntos, independientemente de que estén etiquetados o sin etiquetar.

No se ha empleado en este trabajo, dado que el conjunto de datos del proyecto SOB4ES está íntegramente etiquetado: no existe un excedente de observaciones sin medición de biodiversidad que pudiera aprovecharse de esta forma.

5.1.1.3.1. Formalización del objetivo combinado

De forma general, el aprendizaje semi-supervisado optimiza una función objetivo que combina un término supervisado, calculado sólo sobre las n_l muestras etiquetadas, con un término no supervisado o de regularización, calculado sobre las n_u muestras sin etiquetar:

$$\hat{f} = \arg \min_f \frac{1}{n_l} \sum_{i=1}^{n_l} L(y_i, f(x_i)) + \lambda \frac{1}{n_u} \sum_{j=1}^{n_u} R(f, x_j)$$

donde:

- $\frac{1}{n_l} \sum_{i=1}^{n_l} L(y_i, f(x_i))$ es el término supervisado.
- $\frac{1}{n_u} \sum_{j=1}^{n_u} R(f, x_j)$ es el término no supervisado.
- λ controla el peso relativo del término no supervisado.
- R es la instancia de alguno de los siguientes supuestos:
 - **Regularización por consistencia:** penaliza cuando el modelo hace cambios en las predicciones ante pequeñas perturbaciones de una entrada sin etiquetas, formalizando, en consecuencia, el principio de suavidad.
 - **Minimización de entropía:** empuja al modelo a hacer predicciones más seguras sobre los datos sin etiquetar, cuando se trabaja con modelos de clasificación, formalizando el principio de agrupamiento.

5.1.1.4. Aprendizaje por refuerzo

En el aprendizaje por refuerzo^{[26], [27]}, el algoritmo/agente aprende interactuando con un entorno, recibiendo una recompensa o penalización según las acciones que toma, con el objetivo de maximizar la recompensa acumulada a lo largo del tiempo.

Formalmente se suele modelar como un Proceso de Decisión de Markov (MDP), y entre sus algoritmos más conocidos están Q-learning y sus variantes basadas en redes neuronales profundas (*Deep Reinforcement Learning*), popularizadas por sistemas como AlphaGo.

El MDP se define como una tupla de la siguiente forma:

$$(S, A, P, R, \gamma)$$

Donde:

- S es un conjunto de estados.
- A es un conjunto de acciones.
- $P(s'|s, a)$ es una función de transición que da la probabilidad de pasar al estado s' tras ejecutar la acción a en estado s .
- $R(s, a)$ es una función de recompensa.
- $\gamma \in [0, 1)$ es un factor de descuento que pondera las recompensas futuras frente a las inmediatas.

El agente busca una política $\pi(a|s)$ que maximice el retorno esperado:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

A diferencia del aprendizaje supervisado, no existe un conjunto fijo de pares entrada-salida que sean totalmente correctos proporcionado de antemano; es decir, el propio agente genera su experiencia de entrenamiento a través de la interacción con el entorno, y debe resolver el llamado **dilema exploración-explotación**.

Dicho dilema consiste en decidir en cada paso entre explotar la acción que, según lo aprendido hasta el momento, parece mejor, o explorar acciones distintas que podrían resultar en mejores recompensas a largo plazo pero cuyo valor aún no se conoce con certeza.

Algoritmos como Q-learning aprenden directamente una función de valor $Q(s, a)$ que estima el retorno esperado de tomar la acción a en el estado s y seguir después la política óptima, sin necesidad de conocer explícitamente P ni R .

Es el paradigma más cercano al programa original de Samuel de 1959 que dio nombre al campo^[27]. No se ha empleado en este trabajo, ya que no existe un entorno con el que el modelo interactúe de forma secuencial ni una noción de recompensa: se trata de un problema de predicción sobre datos ya recogidos, no de una secuencia de decisiones.

5.1.1.4.1. Formalización mediante una función objetivo interna

En lugar de optimizar la política directamente, muchos algoritmos de aprendizaje por refuerzo aprenden primero una **función de valor**, que estima el retorno esperado desde un estado (o un par estado-acción) siguiendo una política dada.

La función de valor de un estado bajo la política π se define como:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

y satisface la **ecuación de Bellman**, que expresa el valor de un estado de forma recursiva en función del valor de sus posibles estados siguientes:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')]$$

Esta recursividad es la base de los algoritmos *value-based* como Q-learning, que aprenden iterativamente $Q(s, a)$ sin necesidad de conocer P ni R de antemano, actualizando las estimaciones a partir de la experiencia observada.

Existe una familia alternativa de algoritmos, los métodos *policy-based*, que en lugar de aprender una función de valor y derivar la política a partir de ella, parametrizan y optimizan la política $\pi(a|s)$ directamente mediante ascenso de gradiente sobre el retorno esperado (teorema del gradiente de la política).

La distinción *value-based* frente a *policy-based*, dentro del aprendizaje por refuerzo, es un paralelismo lejano de la distinción entre modelos discriminativos y modelos con una función objetivo optimizada de forma directa.

5.2. Selección de variables

Antes de entrenar los modelos descritos en la sección anterior, es habitual reducir el conjunto de variables predictoras candidatas a un subconjunto más manejable y menos redundante. Esto no solo simplifica el modelo, sino que en modelos como Ridge resulta prácticamente necesario cuando existe colinealidad severa (véase **Regresión Ridge**).

En la literatura de aprendizaje automático, los métodos de selección de variables se agrupan tradicionalmente en tres familias, según su relación con el modelo predictivo final.

5.2.1. Métodos de filtrado

Los **métodos de filtrado o métodos de filtro** evalúan la relevancia de cada variable, o de cada par de ellas, de forma independiente del modelo que se vaya a entrenar después, basándose únicamente en las propiedades estadísticas de los datos. Son computacionalmente baratos y suelen aplicarse como primer paso, antes de cualquier tipo de entrenamiento.

Entre los métodos de filtro se incluyen, entre otros, los siguientes:

- **Correlación entre pares de variables predictoras:** permite detectar redundancia directa entre dos variables y descartar una de cada par altamente correlacionado. Dependiendo del tipo de relación se usa Pearson para las correlaciones lineales y Spearman para las relaciones monótonas lineales o no lineales.
- **Factor de Inflación de la Varianza (VIF):** a diferencia de la correlación por pares, detecta colinealidad multivariante, esto es que una variable puede no estar muy correlacionada con ninguna otra individualmente, pero esta misma puede ser una combinación lineal aproximada de varias de ellas al mismo tiempo.
- **Correlación entre cada variable predictora y el target:** permite descartar variables con relación prácticamente nula con lo que se quiere predecir con independencia de su relación con el resto de variables predictoras.
- **Varianza mínima:** permite descartar variables casi constantes, que aportan poca o nula información discriminativa.

La principal limitación de estos métodos de filtrado es que, al ignorar el modelo final, pueden descartar una variable individualmente poco informativa pero que resulte útil en combinación con otras, o conservar variables redundantes desde el punto de vista de un modelo concreto aunque no lo sean estadísticamente en general.

5.2.2. Métodos de envoltura

A diferencia de los métodos de filtro, los **métodos de envoltura** evalúan subconjuntos de variables entrenando y validando el modelo real con cada subconjunto candidato, y usando el rendimiento obtenido como criterio de selección.

Son más costosos computacionalmente que los métodos de filtro, pero tienen en cuenta las interacciones entre variables y son específicos del modelo empleado.

Algunos de los métodos de envoltura son los siguientes:

- **Eliminación recursiva de variables (RFE):** entrena el modelo con todas las variables, elimina la menos importante según algún criterio y se repite el proceso hasta llegar al número deseado de variables.
- **Selección secuencial hacia adelante/atrás:** parte de un conjunto vacío o completo de variables y va añadiendo/eliminando una variable en cada paso en función de cuál mejora más o empeora menos el rendimiento del modelo.

5.2.3. Métodos de embebido

En los **métodos embebidos** la selección de variables ocurre como parte del propio proceso de entrenamiento, sin necesitar de una fase separada antes o después. Son los más eficientes computacionalmente de los tres tipos de métodos que se han mencionado, porque no requieren entrenar el modelo repetidamente con distintos subconjuntos de variables.

Los tipos de modelos embebidos más relevantes son los siguientes:

- **Regularización L1 (Lasso):** a diferencia de la penalización L2 de Ridge (véase **Regresión Ridge**), la penalización L1 puede llevar coeficientes exactamente a cero, realizando así una selección de variables implícita durante el propio ajuste del modelo lineal.

Este tipo de regularización se hace uso en una de las pruebas realizadas con los modelos, las cuales se pueden ver en el Anexo V.

- **Importancia de variables de modelos basados en árboles:** Random Forest y XGBoost proporcionan de forma nativa una medida de importancia (impureza media/MDI, o por permutation importance) que, aunque se calcula típicamente después de entrenar el modelo completo, puede usarse de forma iterativa como criterio embebido de selección. Véase **Random Forest** y **XGBoost**.

5.2.4. Variables empleadas

A lo largo de la elaboración de los diferentes modelos, los cuales se pueden ver en el repositorio de GitHub^[28] y también se pueden ver en el Anexo III.

Dentro de todas las posibles variables dentro del dataset final, las variables seleccionadas como variables predictoras han sido las siguientes, las cuales se pueden ver en la siguiente tabla, el resto de las variables existentes dentro del dataset empleado en la **Sección 5.5.3** se encuentran indicadas en el Anexo II.

Categoría	Variables	
Cobertura y estructura del suelo	total_plant_cover_z clay_content_z silt_content_z sand_content_z	aggregate_stability_z bulk_density_z soil_moisture_z
Química y metales del suelo	as_z cu_z k_z mo_z ni_z	p_z pb_z zn_z soil_ph_z plot_total_organic_c_z plot_total_n_z
Clima y teledetección (de Google Earth Engine)	gee_temp_media_C_z gee_humedad_rel_pct_z	gee_nvdi_verano_z
Topografía (de DEM)	dem_elevacion_m_z dem_pendiente_deg_z	dem_orientacion_deg_z
Variables europeas de referencia (capas EU)	eu_clay_content_z eu_sand_content_z eu_silt_content_z eu_water_holding_capacity_z eu_cn_ratio_z	eu_p_z eu_ph_z eu_as_z eu_organic_carbon_octop_z eu_zn_z

Tabla 3: Variables predictoras empleadas

Además de la tabla de variables predictoras, en la siguiente tabla se muestran las variables seleccionadas como variables objetivo (*targets*) a predecir en los modelos desarrollados.

Categoría	Variables
nematode_shannon_z	Índice de biodiversidad de Shannon de nematodos del suelo.
macro_shannon_z	Índice de biodiversidad de Shannon de la macrofauna del suelo.
earthworm_shannon_z	Índice de biodiversidad de Shannon de lombrices de tierra.
orib_shannon_z	Índice de biodiversidad de Shannon de ácaros oribáticos.
meso_shannon_z	Índice de biodiversidad de Shannon de la mesofauna del suelo.
coll_shannon_z	Índice de biodiversidad de Shannon de colémbolos.
bac_shannon_z	Índice de biodiversidad de Shannon de la comunidad bacteriana.
fun_shannon_z	Índice de biodiversidad de Shannon de la comunidad fúngica.
euk_shannon_z	Índice de biodiversidad de Shannon de la comunidad eucariota.
oomy_shannon_z	Índice de biodiversidad de Shannon de oomicetes (Oomycota).
cerc_shannon_z	Índice de biodiversidad de Shannon de cercozoos (Cercozoa, protistas).
macro_order_richness_z	Riqueza de órdenes de macrofauna del suelo.
earthworm_richness_z	Riqueza de especies de lombrices de tierra.
orib_species_richness_z	Riqueza de especies de mesofauna del suelo.
meso_species_richness_z	Riqueza de especies de colémbolos.
coll_species_richness_z	Riqueza de especies de colémbolos.
bac_asv_richness_z	Riqueza de ASV bacterianos.
fun_asv_richness_z	Riqueza de ASV fúngicos.
euk_asv_richness_z	Riqueza de ASV de eucariotas del suelo.
oomy_asv_richness_z	Riqueza de ASV de oomicetos.
cerc_asv_richness_z	Riqueza de ASV de cercozoos.

Tabla 4: Variables objetivo empleadas

5.3. Modelos de aprendizaje automático empleados

En las secciones de **Herramientas empleadas** y **Arquitectura general**, un poco más adelante en este documento, se describirán, tanto a nivel teórico como práctico, las tecnologías y herramientas empleadas a lo largo de todo este trabajo, tanto para el desarrollo de los modelos, como también para su evaluación y realización de pruebas y actividades, las cuales se pueden ver en los correspondientes anexos.

En esta sección se va a complementar la información, ya proporcionada por las secciones ya mencionadas previamente, con los fundamentos teóricos de cada modelo con el que se ha probado, de forma general, su funcionamiento base y la correspondiente justificación sobre por qué se han considerado estos modelos y no otros para este trabajo en concreto.

También se hablará de las variantes empleadas en caso de haberse usado variantes de los modelos base.

Además, en el [Anexo III], se podrá ver el código base de todos los modelos empleados.

5.3.1. Regresión Ridge

Los modelos de regresión ridge, es una variante de los modelos de regresión base, pero que cuenta con regularización para corregir el sobreajuste de los datos de entrenamiento cuando se están desarrollando modelos de aprendizaje automático o machine learning.

Esta variante de modelo fue propuesta originalmente por Hoerl y Kennard en 1970^[29] como respuesta a los problemas relacionados con la inestabilidad que aparecían cuando las variables predictoras presentaban niveles elevados de colinealidad^[30].

Mientras que la regresión lineal ordinaria busca los coeficientes w que minimizan únicamente el error cuadrático:

$$\hat{w} = \arg \min_w \|y - Xw\|^2$$

Ridge añade una penalización proporcional al cuadrado de la magnitud de los coeficientes, controlada por el hiperparámetro 0:

$$\hat{w}_{\text{ridge}} = \arg \min_w \|y - Xw\|^2 + \alpha \|w\|^2$$

Cuando $\alpha = 0$, Ridge se reduce a la regresión lineal ordinaria, pero a medida que α aumenta, los coeficientes se “encogen” hacia cero sin llegar nunca a anularse por completo en comparación con Lasso, reduciendo la varianza del modelo a costa de introducir un pequeño sesgo.

A diferencia de la regresión lineal ordinaria, que no tiene una solución estable cuando existe colinealidad perfecta o casi perfecta entre variables, Ridge sí tiene siempre una solución cerrada única, la cual es la siguiente:

$$\hat{w} = (X^T X + \alpha I)^{-1} X^T y$$

Nota: La colinealidad entre variables en Ridge, lo que provoca es que la matriz $X^T X$ deja de ser invertible o queda mal condicionada.

Esto permite que al sumar αI a $X^T X$ se garantice que la matriz sea invertible independientemente del grado de colinealidad entre las diferentes variables predictoras, propiedad relevante para este TFG, donde varias de las variables edáficas y climáticas están correlacionadas entre sí debido a que proceden de fuentes relacionadas entre sí (véase Anexo II).

Grafos de libertad efectivos y elección de α : Una forma útil de entender el efecto de la regularización es a través de los llamados grafos de libertad efectivos del modelo:

$$\text{df}(\alpha) = \sum_{j=1}^p \frac{d_j^2}{d_j^2 + \alpha}$$

donde d_j son los valores singulares de X .

Cuando $\alpha = 0$, $\text{df}(\alpha) = p$, es decir, se usan todos los grados de libertad del modelo lineal completo. A medida que crece α , $\text{df}(\alpha)$ disminuye, reflejando que el modelo efectivamente emplea menos parámetros libres.

Requisito de estandarización:

- La interpretación de los coeficientes de Ridge en términos de importancia relativa sólo es válida si las variables predictoras están en la misma escala, ya que la penalización $\|w\|_2$ trata todos los coeficientes por igual independientemente de la escala original de la variable a la que multiplican.

Por este motivo, todas las variables predictoras de este TFG se han normalizado (media 0, varianza 1) antes del entrenamiento.

Los motivos de su uso en este TFG como uno de los 8 modelos a desarrollar y probar son los siguientes:

1. Los coeficientes son directamente interpretables en términos de la dirección y magnitud del efecto o relevancia que tiene cada variable predictora sobre el índice de diversidad de Shannon H' o la riqueza de los diferentes grupos biológicos, siempre que dichas variables se encuentren normalizadas, el cual es el caso en este trabajo.
2. Al ser un modelo sencillo y computacionalmente barato sirve como línea base para comparar modelos que requieren de más recursos computacionales y son más complejos, pudiendo probar de esta forma si los datos se predicen bien con modelos sencillos en vez de recurrir a modelos más complejos.

5.3.2. Random Forest

Random Forest, propuesto por Breiman en 2001^[31], es un método de ensamblado de modelos basado en árboles de decisión que combina las predicciones de múltiples árboles entrenados de forma independiente para obtener una predicción más robusta que la de un único árbol. Se apoya en dos fuentes de aleatoriedad, de las que toma su nombre de Random Forest:

- **Bagging (bootstrap aggregating):** cada árbol se entrena sobre una muestra aleatoria con reemplazo del conjunto de entrenamiento original, de forma que cada árbol ve una versión ligeramente distinta de los datos.
- **Selección aleatoria de variables:** En cada división de cada árbol, sólo se considera un subconjunto aleatorio de las variables predictoras disponibles, en lugar de todas.

Para una tarea de regresión como la de este TFG, la predicción final es la media de las predicciones individuales de los N árboles del bosque:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N \hat{y}_i$$

Al promediar los árboles que comenten errores distintos entre sí, gracias a la doble aleatoriedad que se introdujo, la varianza del conjunto se reduce sin apenas aumentar el sesgo, lo que hace Random Forest especialmente robusto frente al sobreajuste en comparación con un árbol de decisión individual.

Reducción de varianza y correlación entre árboles: Si cada árbol individual tiene varianza σ^2 y los árboles del bosque están correlacionados entre sí con coeficiente medio ρ , la varianza del promedio de N árboles es:

$$\text{Var}(\hat{y}) = \rho\sigma^2 + \frac{1-\rho}{N}\sigma^2$$

Esta expresión es clave para entender por qué Random Forest introduce la selección aleatoria de variables además del **bagging**: el *bagging* por sí solo ya reduce el segundo término (el que decrece con N), pero los árboles resultantes tienden a estar muy correlacionados entre sí (comparten las variables más predictivas en los primeros splits); al forzar que cada división considere sólo un subconjunto aleatorio de variables, se reduce ρ , y por tanto el primer término, que es el que no desaparece por mucho que se aumente N .

Importancia de variables: Random Forest, en su implementación de scikit-learn^[32] proporciona de forma nativa dos formas de estimar la importancia de cada variable predictora: la **importancia por impureza media (MDI)**, calculada como la reducción media de varianza (en regresión) que aporta cada variable en los nodos donde se ha usado para dividir, ponderada por la proporción de muestras que llegan a cada nodo; y la **importancia por permutación (*permutation importance*)**, que mide la caída de rendimiento del modelo al barajar aleatoriamente los valores de una variable en el conjunto de validación, manteniendo el resto intactas. La segunda es más robusta frente a variables con muchos niveles o alta cardinalidad, aunque computacionalmente más costosa.

Ambas medidas se han empleado en este TFG como complemento a SHAP^[33] (véase).

Se ha considerado este modelo, junto con su variante multisalida, por su buen comportamiento con conjuntos de datos de tamaño reducido como el de este TFG, su tolerancia a variables predictoras correlacionadas entre sí, y por proporcionar de forma nativa una medida de importancia de variables, complementando a SHAP (véase la).

5.3.2.1. Random Forest multisalida

El módulo RandomForestRegressor de scikit-learn, permite de forma nativa una matriz y de salida múltiple permitiendo entrenar todos los árboles para que sean capaces de predecir múltiples indicadores a la vez y a partir del criterio de división de cada nodo se calcule de forma conjunta sobre todas las salidas.

Esto significa que, a diferencia de entrenar un único modelos independiente por indicador/target, la variante multisalida permite que un mismo árbol capture patrones de división que sean simultáneamente informativos para varios targets correlacionados entre sí sin necesidad de un mecanismo explícito de encadenamiento como es RegressorChain, el cual se explica con más detalle en las siguientes secciones.

La predicción para el target k -ésimo en la variante multisalida sigue siendo el promedio de las predicciones de los N árboles del bosque para ese target concreto:

$$\hat{y}_k = \frac{1}{N} \sum_{i=1}^N \hat{y}_{i,k}$$

pero, a diferencia de entrenar K bosques independientes (uno por target), aquí los N árboles son compartidos entre todos los targets, lo que reduce el coste computacional total y permite comparar directamente si el aprendizaje conjunto aporta ventaja frente a la variante independiente (véase).

5.3.3. XGBoost

En **XGBoost** o también conocido como **Gradient Boosting**, lo que hace es crear y construir los árboles de forma secuencial (a diferencia de Random Forest que los entrena y construye de forma independiente y en paralelo) para que cada árbol nuevo sea entrenado para corregir los errores o residuos cometidos por su conjunto de árboles predecesor ya entrenados.

La predicción tras añadir el árbol m -ésimo se define de forma aditiva:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

donde:

- $F_{m-1}(x)$ es la predicción acumulada de los árboles anteriores.
- $h_m(x)$ es el nuevo árbol entrenado para aproximar el gradiente negativo de la función de pérdida.
- η es la tasa de aprendizaje o *learning rate*, que controla cuánto contribuye cada nuevo árbol a la predicción final.

Este enfoque secuencial permite habitualmente alcanzar un rendimiento superior al de Random Forest con menos árboles, pero también lo hace más sensible al sobreajuste si el número de árboles o la tasa de aprendizaje no se ajustan correctamente.

Objetivo regularizado: A diferencia del gradient boosting clásico, XGBoost optimiza en cada iteración una aproximación de segundo orden (expansión de Taylor) de la función de pérdida, e incluye un término de regularización explícito sobre la complejidad del árbol añadido^[34]:

$$L^{(m)} \approx \sum_{i=1}^n \left[g_i h_m(x_i) + \frac{1}{2} l_i h_m(x_i)^2 \right] + \Omega(h_m), \quad \Omega(h_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde:

- g_i es la primera derivada de la función de pérdida respecto a la predicción actual.
- l_i es la segunda derivada de la función de pérdida respecto a la predicción actual.
- T es el número de hojas del árbol h_m .
- w_j es el valor de predicción de la hoja j .
- λ, γ son hiperparámetros que penalizan a los árboles con demasiadas hojas o con pesos demasiado grandes.

Esta regularización explícita, junto con el subsampling de filas y columnas (parámetros `subsample` y `colsample_bytree`), es lo que hace a XGBoost más resistente al sobreajuste que una implementación básica de *gradient boosting*, aun siendo más sensible que Random Forest al número de árboles y a la tasa de aprendizaje si no se ajustan mediante validación cruzada.

Se ha considerado este modelo, junto con su variante multisalida, por ser uno de los modelos que presenta mejor rendimiento en general en base a los resultados reportados por la literatura revisada en contexto a los problemas de biodiversidad del suelo con conjunto de datos reducidos (véase **Antecedentes y contexto**).

Este hecho se prueba una vez más con la variante multisalida como se podrá ver más adelante en la sección de **Conclusiones**.

5.3.3.1. XGBoost multisalida

XGBoost no soporta la salida múltiple de forma nativa en todas las versiones de su librería. Para ello es necesario indicar explícitamente parámetro `multi_strategy` del módulo `XGBRegressor`, que admite las siguientes estrategias:

1. **one_output_per_tree**: Opción por defecto, se entrena un conjunto de árboles independiente por cada target, de forma equivalente a entrenar K modelos XGBoost por separado, aunque compartiendo la misma llamada de entrenamiento.
2. **multi_output_tree**: Cada árbol se entrena para predecir todos los targets a la vez de forma similar a cómo lo realiza Random Forest en su variante multisalida de forma nativa, permitiendo que las divisiones del árbol capturen relaciones compartidas entre targets correlacionados.

En este TFG se ha empleado la estrategia **multi_output_tree**, precisamente para poder comparar, en igualdad de condiciones con la variante multisalida de Random Forest, si el aprendizaje conjunto de los distintos indicadores de biodiversidad aporta una ventaja real frente al modelado independiente (véase), comparación que en las Conclusiones de este TFG resulta favorable a XGBoost multisalida frente al resto de modelos.

5.3.4. RegressorChain

Los modelos anteriores, en su forma multisalida, tratan todos los targets de forma simétrica dentro del mismo árbol o coeficientes. **RegressorChain**^[32] basado en el concepto de cadena de clasificadores propuesto por Read et al.^[35] aborda la regresión multi-salida de una forma distinta: encadenando un modelo base por cada target, de forma que cada modelo de la cadena recibe como entrada las variables predictoras originales más las predicciones ya generadas por los modelos anteriores de la cadena:

$$\hat{y}_k = f_k(X, \hat{y}_1, \hat{y}_2, \dots, \hat{y}_{k-1})$$

De esta forma, si los indicadores de biodiversidad están relacionados entre sí, el modelo puede aprovechar esa relación de forma explícita y direccional para mejorar la predicción de los targets posteriores de la cadena.

Dentro de este TFG se hace uso de **Random Forest** como modelo base del **RegressorChain** y los hiperparámetros se ajustan una única vez dentro del modelo base, no se calculan por target ni por posición, sino sobre el conjunto de entrenamiento completo.

El orden en el que se recorren los targets y las cadenas anteriores es un detalle que determina la salida final del modelo, es decir, el orden de las cadenas tienen un impacto directo sobre el resultado final. Esto se debe a que un target situado al principio de la cadena nunca recibe como entrada las predicciones de los demás, mientras que uno situado al final se beneficia de todos los anteriores.

Para decidir el orden de las cadenas, existen diferentes formas de hacerlo:

- **Manual:** Consiste en escoger un criterio, como puede ser la varianza de cada target, y ordenar a partir de ese criterio. Es la opción más interpretable, ya que el orden tiene un significado defendible de antemano, pero depende de que el motivo y el conocimiento previo sean correctos.
- **Basado en los propios datos:** Consiste en calcular algún estadístico de asociación entre targets y el encadenando primero los targets más relacionados entre sí. Dentro de la literatura revisada (véase Referencias) esta estrategia se considera como una estrategia de segundo orden, por modelar relaciones entre targets, frente a las estrategia de primer orden y las de nivel superior, que intentan capturar relaciones conjuntas entre dos o más targets a la vez^[36].
- **Como problema de búsqueda:** Consiste en hacer uso de técnicas como beam search, métodos de Monte Carlo^[37] o algoritmos genéticos^[36] dado que evaluar todas las posibles combinaciones de un set de targets puede volverse intratable. Estos métodos suelen mejorar el resultado de un orden manual o de uno que sea puramente aleatorio, pero exige entrenar y evaluar la cadena completa muchas veces antes de llegar al orden final, con el consecuente coste computacional.
- **A partir de un modelo probabilístico de dependencia:** Consiste en hacer uso de algún modelo probabilístico, como por ejemplo, las redes bayesianas^[38] aprendido sobre los propios targets. Los targets sin otro del que dependan irán de primeros en la cadena y los que dependan de más targets irán más tarde.

Cabe indicar que, ninguna de estas estrategias garantizan encontrar un orden verdaderamente óptimo, y todas comparten el mismo riesgo común: que el orden elegido perjudique de forma considerable a los targets que queden peor posicionados en la cadena.

5.3.5. Redes neuronales

Como alternativa de aprendizaje profundo a los modelos anteriores, se ha entrenado también un **perceptrón multicapa** (Multi-Layer Perceptron, MLP), implementado con PyTorch^[39]. Un MLP está formado por una capa de entrada, una o varias capas ocultas y una capa de salida, donde cada capa transforma la salida de la anterior:

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

siendo $a^{(l)}$ la salida de la capa l , $W^{(l)}$ y $b^{(l)}$ los pesos y sesgos aprendidos de esa capa, y f una función de activación no lineal, que permite al modelo aprender relaciones no lineales entre las variables ambientales y la biodiversidad del suelo.

Dentro de los modelos MLP, se han desarrollado dos variantes, las cuales se van a explicar a continuación en las siguientes subsecciones, pero a nivel común, ambos modelos, dentro de la capa oculta se apila un bloque con la siguiente estructura, la cual termina en una capa lineal de salida con tantas neuronas como targets se tienen seleccionados, sin función de activación:

$$a^{(l)} = \text{Dropout}(\text{ReLU}(W^{(l)}a^{(l-1)} + b^{(l)})), \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$

El entrenamiento, haciendo uso de la función **ReLU**, que introduce la no-linealidad, ajusta los pesos de la red minimizando una función de pérdida mediante retropropagación del gradiente (*backpropagation*)^[40] y el algoritmo de optimización Adam^[41] que combina momento y una tasa de aprendizaje adaptativa por parámetro para acelerar y estabilizar la convergencia frente al descenso de gradiente estocástico estándar.

A pesar de que la literatura revisada en **Antecedentes y contexto** indica y sugiere que los modelos basados en redes neuronales por regla general no superan a los modelos basados en árboles cuando se trata de entrenar con conjuntos de datos reducidos, se ha incluido este modelo para probar y contrastar esa hipótesis de forma empírica.

5.3.5.1. MLP multisalida

En esta variante, la capa de salida de la red tiene tantas neuronas como targets a predecir, y la red se entrena minimizando el **error cuadrático medio (MSE) estándar**, calculado y promediado conjuntamente sobre todos los targets a la vez:

$$L(y, \hat{y}) = \frac{1}{K} \sum_{k=1}^K (y_k - \hat{y}_k)^2$$

Al compartir las capas ocultas entre todos los targets, esta arquitectura fuerza a la red a aprender una representación interna común de las variables ambientales que sea útil para predecir todos los indicadores de biodiversidad simultáneamente, un enfoque de *multi-task learning* que puede mejorar la generalización, si los targets comparten estructura, pero que también puede perjudicar a un target concreto si sus patrones difieren mucho de los del resto (efecto conocido como *negative transfer*).

5.3.5.2. MLP con función de pérdida personalizada

En esta segunda variante, en vez de hacer las relaciones de forma implícita, se hacen de forma explícita, incorporando en esa misma relación una función de pérdida que se minimiza durante el entrenamiento.

La función de pérdida implementada para este caso combina MSE estándar con un término que penaliza que la matriz de correlación entre los 21 targets predichos se aleje de la matriz de correlación entre los targets reales del lote:

$$L(y, \hat{y}) = \text{MSE}(y, \hat{y}) + \lambda_{\text{corr}} \| \text{Corr}(y) - \text{Corr}(\hat{y}) \|_F$$

donde:

- $\text{Corr}(x)$ es la matriz de correlación de Pearson entre los 21 targets calculada sobre el lote.
- $\|x\|_F$ es la norma de Frobenius.
- λ_{corr} controla el peso de la penalización, si es igual a 0, el modelo se comportará como la variante multisalida.

El objetivo de esta función de pérdida es el de forzar a la red a preservar, además de la precisión punto a punto, la estructura de covariación entre los distintos indicadores de biodiversidad, es decir, que si dos índices tienden a subir o bajar juntos en datos reales, el modelos no rompa esa relación en sus predicciones, aunque prediga cada valor individual con cierto margen de error.

5.3.6. Comparativa

En la siguiente tabla se muestra una comparativa rápida de los modelos descritos en las secciones anteriores.

Criterio	Ridge	Random Forest	XGBoost	Regressor Chain	MLP
Relaciones no lineales	No	Sí	Sí	Sí (según modelo base)	Sí
Interpretabilidad nativa	Alta	Media	Media	Media (heredada del modelo base)	Baja
Robustez a colinealidad	Requiere regularización	Alta	Alta	Alta (según modelo base)	Media
Relaciona los targets entre sí	No	Sí (simétrico)	Sí (simétrico)	Sí (direccional)	Sí (representación compartida)
Sensibilidad a hiperparámetros	Baja	Baja-media	Media-alta	Media (según modelo base)	Alta
Riesgo de sobreajuste	Bajo	Bajo	Medio	Medio (según modelo base)	Alto

Tabla 5: Comparativa de modelos base

5.4. Métricas de evaluación y validación

Evaluar un modelo de aprendizaje automático consiste en resumir, mediante una o varias cantidades numéricas, en qué medida sus predicciones se aproximan a los valores reales sobre datos que el modelo no ha visto durante el entrenamiento.

Este apartado organiza las métricas relevantes según el tipo de problema al que estos responden. Estos se pueden clasificar en dos categorías generales, las cuales se explican a continuación en las siguientes secciones:

1. Métricas de regresión.
2. Métricas de clasificación.

5.4.1. Métricas de regresión

Las **métricas de regresión** evalúan la distancia entre un valor real y un valor predicho, ambos continuos, sin necesidad de traducirlos previamente a categorías, como se hace en las métricas de clasificación.

Son el tipo de métrica natural cuando el objetivo de los modelos es aproximarse con la mayor fidelidad posible de una magnitud numérica.

5.4.1.1. Coeficiente de determinación

El coeficiente de determinación mide con qué proporción de la varianza de la variable objetivo es explicada por el modelo, en relación con un modelos de referencia trivial que siempre predice la media:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

donde:

- SS_{res} es la suma de cuadrados residual, es decir, el error del modelo.
- SS_{tot} es la suma de cuadrados total, es decir, la varianza de los datos respecto a su propia media.

El valor de R^2 oscila entre 1 y 0, pero puede llegar a números negativos, indicando que predice pero que el modelo trivial de referencia que sería predecir la media. Si el valor es 0 es que predice la media, mientras que si el valor de R^2 es un 0 muy cerca de él eso implica que el modelo predice mucho mejor que haciendo simplemente medias.

Su limitación principal, es el hecho de que no puede distinguir si su modelo mejora el rendimiento estando dentro del rango central o en los casos extremos y puede ser engañoso en casos en los que se entrena con un dataset reducido, a pesar de eso, es de gran utilidad, debido al ser una métrica relativa, nos permite ver un rendimiento a nivel general de modelos y nos facilita la comparación entre los diferentes modelos desarrollados para este TFG.

5.4.1.2. Raíz del error cuadrático medio

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

A diferencia del R^2 , el RMSE se expresa en las mismas unidades en las que se representan a las variables objetivo que se empleen, lo que facilita su interpretación directa como “magnitud típica del error”. Al elevar al cuadrado cada residuo antes de promediar, el RMSE suele acabar penalizando de forma desproporcionada los errores grandes frente a los errores pequeños.

Esto quiere decir que un único punto predicho de una forma nefasta puede provocar que el valor de RMSE sea mucho más grande que lo que vendría siendo otras métricas más relativas, pudiendo dificultar la justificación. Esto hace que el RMSE sea muy sensible a los valores atípicos, lo cual puede ser relevante para los valores de biodiversidad.

5.4.1.3. Error absoluto medio

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

El MAE también se expresa en las unidades originales de la variable objetivo, pero, al no elevar al cuadrado los errores, trata todos los errores de forma proporcional a su magnitud, sin tener la penalización extra que presenta el RMSE sobre los errores grandes.

Esto lo hace más robusto ante los valores atípicos, pero también lo hace menos sensible a errores grandes que pueden ser relevantes para la fiabilidad práctica del modelo.

Generalmente si se comparan MAE con RMSE, se puede obtener una vista informativa de con cuánta frecuencia hay errores grandes puntuales en base a la diferencia que hay entre ambos valores. Si se mantienen más o menos iguales que los errores suelen ser pequeños y homogéneos, en caso contrario, eso implica que hay muchos errores grandes dentro de las predicciones. Por lo tanto, reportar ambas métricas juntas suele dar más información que reportándose por separado.

5.4.2. Métricas de clasificación

Las **métricas de clasificación** evalúan la correspondencia entre una etiqueta real y una etiqueta predicha, ambas categóricas y pertenecientes a un conjunto finito de clases.

Cuando el problema original es de regresión, se puede optar a hacer una discretización a posteriori para poder aplicar este tipo de métricas y tener métricas también desde un punto de vista categórico, además del numérico.

5.4.2.1. Exactitud (*Accuracy*)

La exactitud o el accuracy mide la proporción de muestras cuya clase predicha coincide con la clase real:

$$\text{Acc} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(\hat{y}_i^{\text{cls}} = y_i^{\text{cls}})$$

donde:

- $\mathbb{1}(\cdot)$ es la función indicadora_[32].

La principal limitación de la misma, es que se vuelve engañosa cuando las clases están muy desbalanceadas, ya que un modelo trivial que siempre prediga la clase mayoritaria puede obtener una exactitud alta sin haber aprendido nada, lo que hace que quede en una buena medida mitigada por el propio diseño de la discretización que se emplee en cada uso.

Aún así, la exactitud no distingue el tipo de error cometido, por lo tanto se suele recurrir al **Coefficiente de Kappa de Cohen**.

5.4.2.2. Precisión, recall y F1

Dentro de un problema multiclase, la **precisión** y el **recall** se definen primero por clase, tratando cada clase c como un problema binario:

$$\text{Precisión}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \quad \text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}$$

donde TP_c , FP_c y FN_c son, respectivamente, los verdaderos positivos, los falsos positivos y los falsos negativos de la clase c _[32]. La precisión responde a “de las muestras que el modelo etiquetó como c , cuántas lo eran realmente”, mientras que el recall responde a “de las muestras que realmente eran c , cuántos detectó el modelo”.

El **F1** combina ambas en su media armónica, la cual penaliza con más fuerza que una media aritmética en el caso de que en una de las dos sea muy baja, aunque la otra sea alta.

$$F1_c = 2 \cdot \frac{\text{Precisión}_c \cdot \text{Recall}_c}{\text{Precisión}_c + \text{Recall}_c}$$

5.4.2.3. Coeficiente de Kappa de Cohen

El coeficiente de Kappa de Cohen^[42] corrige la limitación que presenta el **Accuracy** comparando el acuerdo observado con el acuerdo esperado por el azar:

$$\kappa = \frac{p_0 - p_e}{1 - p_e}$$

Un $k = 1$ indica un acuerdo perfecto, mientras que un $k = 0$ indica que el acuerdo es peor que uno escogido al azar. En este TFG al tratarse de clases con un orden natural (Bajo, Medio y Alto) y no meras etiquetas nominales sin relación aparente entre sí, resulta más útil e informativo hacer uso de la variante ponderada del Coeficiente de Kappa, la cual fue propuesta por Cohen^[43] que penaliza el descuento de forma proporcional a la distancia entre las clases implicadas, es decir, confundir Bajo con Alto, por ejemplo, penalizará más que si se confunde Bajo por Medio, indicando así, de forma cualitativa, que el primer error es mucho más grave que el segundo.

$$\kappa_w = 1 - \frac{\sum_{i,j} w_{i,j} O_{i,j}}{\sum_{i,j} w_{i,j} E_{i,j}}$$

donde:

- O_{ij} es la matriz de acuerdo observado entre la clase real i y la predicha j .
- E_{ij} es la matriz de acuerdo esperado por azar entre la clase real i y la predicha j .
- w_{ij} son los pesos de penalización, en su variante lineal, de forma que la diagonal principal ($i = j$) no penaliza y la penalización crece linealmente con la distancia ordinal entre clases.

El Kappa ponderado ofrece una lectura más adecuada al carácter ordinal de la discretización que la exactitud simple, es decir, dos modelos pueden presentar la misma exactitud, pero pueden presentar coeficientes de Kappa ponderados distintos, dependiendo de cómo se concentren los errores.

5.4.2.4. Matrices de confusión

La matriz de confusión^[32] no es en sí una métrica escalar, sino una tabla de contingencia que desglosa, para cada combinación de clase real y clase predicha, el número de muestras correspondiente. A diferencia de la exactitud, el F1 o el Kappa, que resumen el rendimiento del clasificador derivado en un único número, la matriz de confusión permite identificar el patrón de los errores, es decir, permite ver de forma gráfica las tendencias de confusión que puede presentar un modelo.

Normalizada por filas, la matriz de confusión se puede leer directamente como una tasa de aciertos y de tipos de error por clase, lo que facilita la comparación entre targets con distinto número de muestras y entre modelos con distinta exactitud global.

5.5. Arquitectura general

Para proporcionar una respuesta a los objetivos planteados y permitir la predicción de la biodiversidad del suelo a partir de variables ambientales, se ha diseñado una arquitectura de software basada en un flujo de datos modular y desacoplado, permitiendo de esta forma que cualquier cambio en alguna de las capas no provoque tener que realizar cambios en el resto de las capas.

El uso de este enfoque garantiza la reproducibilidad del sistema ante la posibilidad de incluir nuevas fuentes de datos o nuevos algoritmos de aprendizaje automático.

La arquitectura explicada previamente se compone de cuatro capas, las cuales transforman los datos brutos en uno o varios modelos predictivos para su futura evaluación y comprobación. Dicha estructura de capas se puede ver en el siguiente diagrama.

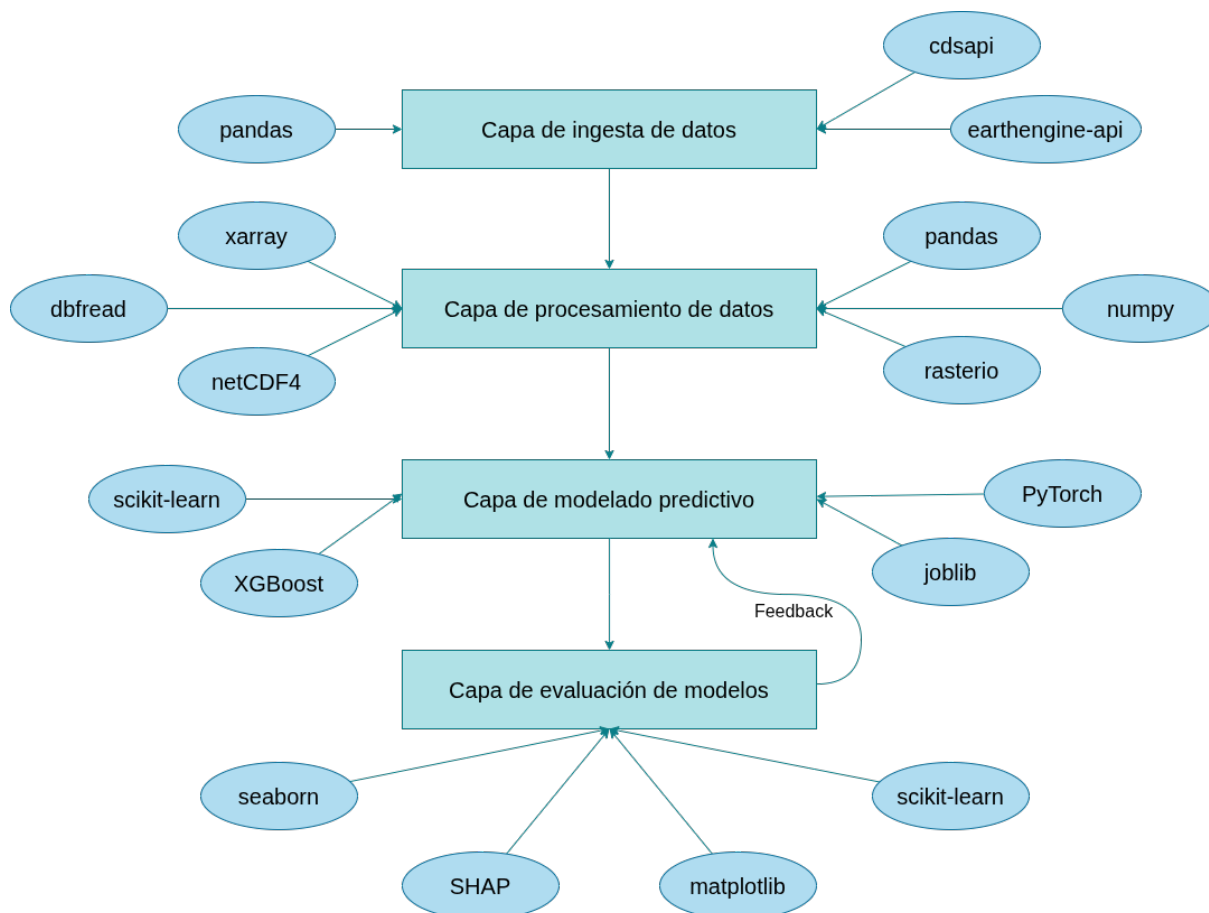


Figura 2: Arquitectura de software del proyecto

5.5.1. Capa de ingesta de datos

Esta capa se encarga de la adquisición de datos con las diferentes APIs a emplear, como también cargar los diferentes archivos de los datasets que se tengan en local.

La adquisición de datos se realiza en dos fases, las cuales están basadas en el origen de los datos que se van a recolectar:

1. Ingesta de datos locales:

- Se hace la lectura de los datos locales recolectados por el grupo europeo SOB4ES, como otras fuentes de datos Europeas almacenados en local, los cuales contienen las capas raster de la ESDAC, CORINE Land Cover, entre otras fuentes de datos que nos proporcionan información fisicoquímica de las zonas en las que se hicieron las diferentes pruebas.

2. Ingesta de datos remotos (APIs):

- Se implementa mediante el uso de APIs para obtener otros datos no recogidos dentro de los datos locales como viene siendo datos climatológicos y otros de interés como la inclinación del terreno.
- Para ello se hacen uso de los siguientes servicios:
 1. **Google Earth Engine (GEE)**^[4]: A través del módulo earthengine-api de python, para la extracción automatizada de datos como las imágenes satelitales de Sentinel-2 usadas para el cálculo de índices de vegetación y medias de temperatura y humedad.
 2. **Copernicus Climate Data Store (CDS)**^{[5], [11]}: Mediante el módulo cdsapi de python para la descarga de datos de reanálisis climático, en los que se incluyen datos sobre las precipitaciones mensuales acumuladas.
 3. **Copernicus DEM (vía AWS)**^[44]: Descarga del DEM (*Digital Elevation Model*), empleado para la obtención de altitudes, pendientes y orientaciones del terreno.

Además de las fases de la adquisición de datos, destacamos las siguientes librerías empleadas para ello:

- **pandas**^[45]: Librería empleada para la manipulación e ingeniería de datos tabulares. Permite la lectura y la escritura de archivos .csv y .xlsx (excel). También puede gestionar uniones, filtrados y operaciones vectorizadas sobre múltiples filas de forma no muy compleja y eficaz.
- **earthengine-api**^[4]: Librería de desarrollo oficial de Google que permite la conexión remota, la autenticación y la orquestación de scripts de procesamiento geoespacial sobre la API de Google Earth Engine.
- **cdsapi**^[5]: Librería que habilita un enlace directo y oficial a la API de CCDS, para el envío de solicitudes de extracción de datos parametrizadas y para la descarga automática de los datos meteorológicos solicitados.

Al final de todo el procesamiento de esta capa se crea un archivo requirements.txt cuyo objetivo es almacenar todas las librerías necesarias para que lo realizado pueda ser reproducible.

Nota: A la hora de realizar la ingesta de datos remotos, tomamos como referencia las coordenadas obtenidas a través de la ingesta de datos locales. Esto permite la obtención de los datos necesarios de forma eficiente y evitar obtener datos innecesarios o crear muchos más datos faltantes.

5.5.2. Capa de procesamiento de datos

Esta capa tiene como objetivo unificar las diferentes fuentes de datos bajo un marco de coordenadas común y resolver las diferentes discrepancias entre los datos, estandarizándolos y armonizándolos para poder emplearlos en futuras capas.

Este proceso se hace dividiéndolo en tres subprocesos o fases:

1. **Procesado de datos de ficheros:** En este subproceso nos centramos en los datos que se tienen ya tanto del proyecto SOB4ES, como de otros datasets ya existentes, tanto a nivel UVIGO, como a nivel europeo. Este procesamiento de datos da como resultado varios elementos, que será usado en el tercer subproceso. Dichos elementos son los siguientes:
 1. **Datasets:**
 - **model_ready.csv:** Dataset optimizado ya para su uso e introducción en los diferentes modelos que se desarrollen a futuro.
 - **clean.csv:** Dataset intermedio con todos los datos limpios, empleado en las siguientes fases del procesamiento de datos como referencia y para la armonización de sus datos con los datos obtenidos por consultas remotas a bases de datos en la red.
 - **imputation_flags.csv:** Dataset que contiene únicamente flags que indican en qué celdas faltan datos de cada tipo de dato introducido en clean.csv.
 2. **scaler.pkl:** Archivo pickle que guarda la media y la desviación estándar calculadas en la preparación de los datos y usadas para la normalización de cualquier otro dato nuevo de forma idéntica a como se normalizaron los datos originales.
 3. **label_encoders.pkl:** Archivo pickle que guarda el mapeo de categorías a números aprendido durante la preparación de datos, para codificar de la misma forma cualquier otro dato nuevo.
2. **Procesado de datos de consultas remotas a bases de datos:** En este subproceso nos centramos en los datos obtenidos a partir de la realización de solicitudes en remoto para obtener datasets o datos provenientes de los diferentes servicios web que se emplean dentro de este trabajo. Como consecuencia, este procesamiento de datos da como resultado un segundo dataset y un scaler.pkl intermedios con todos los datos relevantes obtenidos de datasets online, que será usado en el tercer subproceso.
3. **Combinación y armonización de ambas fuentes:** En este último subproceso, hacemos uso de los datasets intermedios comentados previamente para combinarlos y armonizar sus datos. Como resultado de este último subproceso, obtendremos un dataset final con todos los datos preparados para ser usados en los diferentes modelos que se desarrollen en la siguiente capa.

Además de los subprocesos, a lo largo de procesamiento de esta datos se hace uso de las siguientes librerías:

- **pandas**^[45]: véase el punto anterior.
- **numpy**^[46]: Librería especializada en computación numérica que es generalmente empleada para el manejo de datos numéricos complejos, así como para dar soporte matemático a otras librerías como pandas.
- **rasterio**^[47]: Herramienta geoespacial destinada a la lectura de archivos matriciales con formato GeoTIFF, procedentes de las diferentes fuentes de datos europeas. También sirven para la realización de consultas directamente a las URLs correspondientes al DEM de Copernicus.
- **xarray**^[48]: Librería diseñada para la manipulación de conjuntos de datos multidimensionales, los cuales se encuentran distribuidos en múltiples archivos .nc (NetCDF).
- **dbfread**^[49]: Librería ligera de bajo nivel capaz de leer de forma nativa y eficiente archivos con formato .bdf, facilitando la extracción de la información tabular contenida en ciertos conjuntos de datos vectoriales del catálogo europeo.
- **netCDF4**^[50]: Librería que permite la lectura, escritura y manipulación de archivos con formatos netCDF y HDF5.

5.5.3. Capa de modelado predictivo

El objetivo principal de esta capa es el desarrollo de los diferentes modelos predictivos como también la preparación de los sets de aprendizaje y pruebas y su consecuente entrenamiento.

Antes de comenzar a trabajar en esta capa es recomendable tener el dataset con los datos finales preparado y armonizado para que el proceso pueda, de esta forma, ser mucho más lineal y sencillo de seguir.

Esta capa y la siguiente, van a estar comunicadas entre sí debido a que el feedback que se reciba de la **Capa de evaluación de modelos** va a ser usada para mejorar los modelos que se desarrollen en esta capa. De esta forma se pueden obtener múltiples modelos predictivos de mayor calidad, el cual es el quinto sub objetivo de este trabajo de fin de grado.

De la misma forma que las capas anteriores, esta capa va a estar dividida en varias fases o subprocesos:

1. **Fase de partición de datos:** Esta fase consiste en coger el dataset final con los datos ya normalizados y armonizarlos y dividirlos en tres sets de menor tamaño para emplearlos cada uno para una de las siguientes funciones^{[51], [52]}.

1. **Set de Entrenamiento:** Conjunto inicial, el cual será empleado para que los modelos que se desarrollen aprendan los patrones que haya ocultos entre los datos mediante el ajuste de los parámetros de los modelos.
2. **Set de Validación:** Conjunto usado para el ajuste de los hiperparámetros de los modelos y seleccionar cuáles funcionan mejor de forma objetiva sin contaminar en proceso. En esta parte también se hará uso del feedback obtenido de la **Capa de evaluación de modelos**.
3. **Set para la evaluación:** Conjunto de datos no empleados en ninguno de los sets mencionados previamente, su uso es para evaluar la eficacia del modelo en condiciones reales y con datos con los cuales nunca ha trabajado.

La división de los datos se ha hecho teniendo en cuenta el origen de las muestras tomadas y a partir de la realización de esta fase obtenemos los datasets indicados en la siguiente tabla:

Nombre CSV	Porcentaje de datos	Nº de entradas
train.csv	70%	299
test.csv	15%	64
eval.csv	15%	65

Tabla 6: Tabla de los datasets resultantes de la fase 1

2. **Fase de investigación de posibles modelos predictivos:** El objetivo de esta fase es tener una lista de modelos predictivos que se puedan desarrollar y emplear teniendo en cuenta los requisitos de este trabajo, como también condicionantes como el formato de datos y el número de estos mismos. También en esta fase, además del listado de modelos predictivos a desarrollar, se elaborará un listado de parámetros a utilizar como variables para que el modelo tenga ya cierta calidad a la hora de hacer el primer aprendizaje y evaluación.
3. **Fase de configuración del entorno:** El objetivo de esta fase es tener el entorno con las librerías y todos los componentes necesarios para poder ejecutar sin problemas los diferentes modelos que se desarrollen a lo largo de la siguiente fase.

El resultado de esta fase es una versión actualizada del archivo requirements.txt con todas las librerías empleadas en esta capa.

4. **Fase de desarrollo de modelos:** El objetivo de esta fase es tener un par de modelos desarrollados y contenidos dentro de múltiples jupyter notebooks para poder así, de esa forma, se pueden efectuar dentro de un entorno cerrado y seguro.

El resultado de esta fase son múltiples notebooks con cada uno de los modelos desarrollados para su futuro entrenamiento.

5. **Fase de entrenamiento de los modelos:** El objetivo de esta fase es una vez creados los diferentes modelos a emplear, estos serán ejecutados uno a uno y serán entrenados para, posteriormente evaluarlos y realizar las correcciones correspondientes.

El resultado de la fase es múltiples archivos **.pkl** con todos los modelos entrenados para su futuro uso.

A lo largo del desarrollo de esta capa, se hacen uso de las siguientes librerías:

- **sklearn/scikit-learn**_[32]: Librería principalmente creada para machine learning y para análisis de datos. Dentro de la propia librería hacemos uso de las funciones/submódulos que se muestran en la siguiente tabla:

Submódulo	Descripción
linear_model.Ridge	Implementa la regresión lineal con regularización L2, siendo este el algoritmo base para Ridge Regression.
ensemble.RandomForest	Implementación del algoritmo de Random Forest para regresión.
multioutput. Regressor-Chain	Permite envolver un modelo base (en nuestro caso Random Forest) para, durante el entrenamiento, replicar el modelo base una vez por cada target. De esta forma se pueden encadenar las predicciones de los targets anteriores como entrada adicional para poder predecir los siguientes valores.
model_selection. RandomizedSearchCV	Realiza la búsqueda de hiperparámetros, probando todas las posibles combinaciones del grid de parámetros establecido por el usuario/desarrollador. Se escoge el mejor set de hiperparámetros haciendo uso de la validación cruzada (CV).
model_selection. cross_validate	Ejecuta la validación cruzada repetida para medir la robustez del modelo y poder detectar sobreajustes.
model_selection. RepeatKfold	Permite aplicar la estrategia de partición en pliegues para la validación, obteniendo así lo siguiente: $x \text{ pliegues } * y \text{ repeticiones } = xy \text{ evaluaciones}$
model_selection.KFold	Variante simple de RepeatKfold .

Tabla 7: Tabla de submódulos empleados de la librería scikit-learn

- **XGBoost**_{[53], [54]}: Librería que implementa algoritmos de ML bajo el framework de *Gradient Boosting*. Su aprendizaje se basa en árboles de decisión potenciados por el framework mencionado previamente. Además de hacer uso de la propia librería destacamos el siguiente submódulo o función:
 - **XGBRegressor**: Implementa el algoritmo de gradient boosting sobre los árboles de decisión. Además, de que permite trabajar con múltiples salidas haciendo uso del argumento `multi_strategy=multi_output_tree`.

- **PyTorch**_[39]: Librería que se emplea principalmente para actividades de ML, y para la creación de redes neuronales. Actualmente es una de las más empleadas dentro del entorno académico e investigador. De esta librería destacamos los siguientes elementos:

Submódulo/Módulo	Descripción
torch	Librería principal de PyTorch, proporcionando los tensores, el cálculo de gradientes (autograd) y el optimizador Adam usado para entrenar a las redes neuronales.
torch.nn	Módulo de torch encargado de la definición y construcción de redes neuronales. En términos de definición se pueden definir las capas, el modo de activación, la regularización y la función de pérdida, entre otros elementos.
torch.utils.data. DataLoader	Se encargan de gestionar la división del conjunto de entrenamiento en lotes de trabajo (batches) y su iteración aleatoria durante cada período de entrenamiento.
torch.utils.data. TensorDataset	

Tabla 8: Tabla de submódulos empleados de la librería PyTorch

- **joblib**_[55]: Librería que permite la serialización de las funciones como también la computación paralela. Esto es de gran utilidad a la hora de entrenar y trabajar con los diferentes modelos, especialmente para manejar la concurrencia y la persistencia de los modelos creados haciendo uso de scikit-learn y XGBoost.

5.5.4. Capa de evaluación de modelos

El objetivo principal de esta capa final es evaluar los diferentes modelos desarrollados en la **Capa de modelado predictivo** y a partir de las evaluaciones realizadas, hacer las siguientes acciones:

1. Escoger los modelos que presentan una mayor utilidad y funcionalidad para el proyecto.
2. Mejorar los modelos escogidos en múltiples iteraciones.

Una vez más, de forma similar a las capas anteriores, esta capa se divide en las siguientes fases:

1. **Fase de validación individual:** Durante el desarrollo de los modelos, en cada notebook se han integrado varias celdas que tienen como objetivo realizar la validación cruzada para cada modelo antes de generar todos los modelos resultantes.

Esto se realiza para comprobar ya en la construcción de los modelos si los parámetros escogidos son los más efectivos o si estos, por otro lado, están provocando un sobreajuste en el modelo. También permite prevenir la fuga de datos originado por un entrenamiento deficiente o mal preparado.

De esta fase se obtienen datos que figuran en los outputs de los notebooks de cada uno de los modelos diseñados, además de los modelos resultantes que serán usados en la siguiente fase.

2. **Fase de validación en conjunto de los modelos generados:** Para el correcto desarrollo de esta fase se van a crear dos notebooks más que se van a encargar de realizar la evaluación y validación de los modelos con dos puntos de vista distinto, los cuales son los siguientes:

- **Punto de vista numérico:** En este notebook nos centramos en cargar todos los modelos desarrollados y evaluar el rendimiento de cada uno de los modelos contra el dataset creado para las pruebas. Dentro de este punto de vista nos centramos en las siguientes métricas, cuyas definiciones se pueden encontrar en **Métricas de evaluación y validación:**

- **R2 global.**
- **R2 por target.**
- **RMSE.**
- **MAE.**

A partir de la ejecución del notebook se podrán ver los resultados de las pruebas tanto en las celdas de salida del notebook, como en los archivos de resultado que se exportan en la última fase de ejecución. Estos archivos contienen los resultados de la comparación entre modelos y el R2 por cada target de todos los modelos probados.

- **Punto de vista discreto:** En este notebook sigue una metodología similar de realización de pruebas en comparación con el notebook anterior, pero para este se realiza una clasificación para poder ver los resultados, no tanto como números, sino como aciertos y fallos. Dentro de este punto de vista nos podemos centrar en otras métricas, cuyas definiciones también se pueden encontrar en **Métricas de evaluación y validación:**

- **Matrices de confusión.**
- **Kappa de Cohen.**
- **F1.**

3. **Análisis de resultados y aplicación de mejoras:** El objetivo principal de esta fase es analizar los resultados obtenidos en la fase anterior, en la primera vuelta escoger los modelos que presentan el mejor funcionamiento y, a partir de la segunda vuelta, aplicar mejoras para intentar mejorar aún más el rendimiento.

Esta fase no requiere de desarrollo de código, es únicamente ver los resultados obtenidos, analizarlos y mejorarlos en la medida de lo posible en base a las capacidades y limitaciones que se tienen.

A lo largo del desarrollo de esta capa, se hacen uso de las siguientes librerías:

- **scikit-learn**: véase definición anterior. Para esta capa hacemos uso de los siguientes submódulos de scikit-learn:

Submódulo	Descripción
metrics.r2_score	Calcula el coeficiente de determinación R2, la métrica principal para medir qué proporción de la varianza real da explicado el modelo.
metrics.mean_squared_error	Calcula el error cuadrático medio, empleado para medir el error de predicción penalizando más de esa forma los errores grandes.
metric.mean_absolute_error	Calcula el error absoluto medio (MAE).
inspection.permutation_importance	Mide la importancia de cada variable predictora mediante el barajeo de sus valores y observando en cuánto cae el rendimiento del modelo. Se emplea en los modelos los cuales presentan compatibilidad con SHAP (modelos que no son de árbol) y también para complementar a SHAP.
base.BaseEstimator	Clases empleadas para envolver un modelo creado con PyTorch en una interfaz compatible con scikit-learn. Es necesario para poder hacer uso de permutation_importance sobre redes neuronales (son incompatibles de base).
base.RegressorMixin	

Tabla 9: Tabla de submódulos empleados de la librería scikit-learn (para evaluación)

También se hacen uso de los submódulos **RepeatedKFold** y **cross_validate**, los cuales ya se han explicado en la parte de librerías empleadas de la **Capa de modelado predictivo**.

- **SHAP**_[33]: Librería que calcula los valores shapley haciendo uso de TreeExplainer para determinar la contribución exacta de cada variable a cada predicción realizada. Esto nos permite ver de forma gráfica cómo funciona el modelo y también ver cuánto impacto tiene cada una de las variables en los diferentes targets establecidos.
- **matplotlib**_[56]: Librería creada para la generación de visualizaciones de diferentes tipos en Python, permitiendo mostrar datos de forma gráfica y crear ilustraciones gráficas complejas de forma intuitiva y sencilla. También funciona como motor subyacente para otras librerías como shap y seaborn.
- **seaborn**_{[57], [58]}: Librería basada en matplotlib, permitiendo la creación de gráficas de datos con una interfaz de mayor nivel en comparación con matplotlib y está integrado de forma más cercana con otras librerías empleadas en este trabajo como pandas.

5.6. Herramientas empleadas

En esta sección se justifican las herramientas, servicios y programas de terceros empleados a lo largo de este trabajo. Parte de estos elementos se encuentran mencionados de nuevo en la sección de **Arquitectura general** más adelante en el documento, donde se describe cómo se integran dentro del flujo de datos.

Aquí se explica, para cada uno, por qué se ha elegido frente a las alternativas disponibles.

5.6.1. Lenguaje de programación, control de versiones y librerías

Lenguaje de programación:

- Como lenguaje principal de desarrollo de los modelos se ha empleado **Python**^[59]. Su elección se justifica por su amplia adopción dentro de los campos de la ciencia de datos y del aprendizaje automático, así como por la disponibilidad de un ecosistema maduro y consolidado de librerías específicas y especializadas en estas tareas, lo que la ha permitido cubrir todas las necesidades del proyecto sin recurrir a herramientas externas al lenguaje.

Dentro de este mismo lenguaje, el desarrollo de todos los modelos y los distintos notebooks de pruebas se han realizado principalmente sobre **Jupyter Notebook**^[60], dado que formato que estos proporcionan (celdas ejecutables de forma independiente con salida, persistente de tablas y gráficas) encaja de forma natural con el flujo de trabajo experimental e iterativo descrito en **Resumen de la solución propuesta**. El entrenamiento, incluido el de los modelos que se benefician y pueden hacer uso de aceleración por GPU, se ha realizado en local, en un equipo con GPU dedicado facilitado por el tutor de este trabajo.

Control de versiones:

- Para el control de versiones, como también el almacenamiento de todo el trabajo, se ha hecho uso de **Git**^[61] y de **GitHub**^[62]. Esta elección permite mantener un historial completo de cambios, poder trabajar de forma aislada en cada prueba mediante ramas independientes (véase Anexo algo), y facilitar la trazabilidad entre el código y los resultados obtenidos en cada iteración del trabajo.

Estas herramientas son de especial utilidad dado el enfoque exploratorio de este TFG, en el que distintas configuraciones de un mismo modelo se han probado en paralelo y pueden proporcionar resultados variados.

Librerías:

- Todas las librerías empleadas, junto a los correspondientes módulos/funciones empleadas se encuentran todas mencionadas en la sección anterior, cada una de ellas dentro de la capa de arquitectura en la que se usara.

5.6.2. Servicios de obtención de datos remotos

A pesar de existir una gran cantidad de fuentes de datos abiertas disponibles, se han escogido las siguientes por su mayor facilidad de integración con Python y por estar centradas a nivel europeo, coherente con el ámbito del proyecto SOB4ES:

- **Google Earth Engine (GEE)**^[4]: elegido como fuente de imágenes satelitales (Sentinel-2) frente a otras alternativas por los siguientes motivos:
 - Su acceso es gratuito para actividades de investigación y uso no comercial.
 - Presenta un catálogo de más de 900 datasets públicos, entre ellos Sentinel-2 ya en formato Level-2A (reflectancia de superficie, con corrección atmosférica aplicada), listo para el análisis sin necesidad de descargar los archivos ni de realizar preprocesamiento en local.
 - Tiene integración con la API oficial de Python a través de la librería `earthengine-api`, que permite ejecutar el procesamiento geoespacial directamente sobre la infraestructura de Google en vez de tener que hacerlo en local.

- **Copernicus Climate Data Store (CDS)**_[63]: elegido como fuente de datos de reanálisis climático (ERA5) por los siguientes motivos:
 - Su cobertura europea es consistente con el ámbito del proyecto SOB4ES.
 - Dispone de un cliente oficial en Python mediante la librería cdsapi, que se configura con un token de acceso personal y permite la automatización de las descargas directamente desde los scripts de Python.
 - Es un servicio gratuito tras el registro en la plataforma y proporciona los datos en formatos estándar (NetCDF/GRIB), que cuentan con librerías maduras para procesarlos con facilidad dentro de Python.
- **Copernicus DEM**_[44]: elegido para la obtención del modelo de elevación digital por los siguientes motivos:
 - Ofrece resolución de 30 metros a escala global (GLO-30), distribuida como *Cloud Optimized GeoTIFF* y disponible de forma gratuita al público general.
 - Conexión mediante un bucket público de AWS S3, sin necesidad de autenticación ni registro previo.
 - La estructura del servidor de AWS S3 simplifica notablemente la integración en el flujo de trabajo, en comparación con otras fuentes de DEM que exigen credenciales o portales de descarga manual.

5.6.3. Programas de redacción y documentación

Para la redacción de la memoria y la gestión de las referencias bibliográficas de este TFG se han empleado las siguientes herramientas:

- **Google Docs**_[64]: programa de redacción de documentos de texto online, empleado para redactar una versión preliminar (borrador) de la memoria. Se eligió por facilitar el trabajo colaborativo y las revisiones del borrador en las reuniones con los tutores (véase **Seguimiento**): la herramienta de comentarios permite señalar directamente sobre el texto qué partes corregir y en qué aspectos mejorar.
- **Typst**_[65]: herramienta de redacción de documentos técnicos, similar y compatible con LaTeX. Existe tanto en versión web como mediante un plugin de compilación local, empleado en este trabajo para la maquetación de la versión final del documento a partir del borrador redactado en Google Docs.
- **Scribbr**_[66]: empleado para la generación del formato de citas y referencias bibliográficas siguiendo el estándar IEEE. Se eligió por su facilidad de uso, al disponer de un generador de citas accesible desde el navegador que agiliza la organización de las fuentes consultadas mientras se redacta el borrador de la memoria.

5.6.4. Programas de desarrollo de código

Para el desarrollo de los notebooks, como también de script de automatización para la ejecución de los mismos (véase) se ha trabajado alternando entre **Visual Studio Code (VSCode)**_[67] y **OpenCode**_[68] según la máquina empleada.

Esta alternancia responde a haber trabajo desde distintos equipos a lo largo del desarrollo del TFG, para lo cual ambos IDEs ofrecen una experiencia equivalente y compatible con el resto del entorno (python, terminal y git), permitiendo continuar el trabajo indistintamente desde cualquiera de las máquinas empleadas.

5.6.5. Programas de creación de diagramas e ilustraciones

Diagramas de Gantt:

- **gantt-web**^[69]: Herramienta ligera disponible en GitHub, desarrollada específicamente para la creación del diagramas de Gantt, se optó por el desarrollo de una herramienta externa a emplear las ya existentes al no ajustarse a los estilos y diseños deseados para la representación de la planificación de este TFG.

El resto de diagramas/gráficas se han hecho con las siguientes herramientas:

- **draw.io**^[70]: Herramienta de acceso gratuito y online de creación de diagramas. Ofrece soporte nativo para la exportación de imágenes y fue seleccionado por su facilidad de acceso e intuitividad a la hora de crear todos los diagramas de este TFG.
- Las gráficas de carácter analítico se han generado de forma programática con **Matplotlib**^[56] y **seaborn**^[57],^[58] las mismas librerías empleadas dentro de la **Capa de evaluación de modelos**. Frente a una gráfica de creación de diagramas manuales, el uso de estas librerías permite generar gráficas directamente a partir de los resultados numéricos que se obtienen en los modelos, garantizando la reproducibilidad y evitando errores de transcripción al representarlos.

6. Planificación y seguimiento

6.1. Planificación

La planificación de este trabajo ha empezado a partir del inicio del mismo, en mayo de 2026, a partir de ahí se planifica su desarrollo de la forma en la que se indica en la siguiente tabla:

Periodo	Fase CRISP-ML(Q)	Actividades Planificadas	Hitos
Mayo	Comprensión y preparación inicial de los datos.	Ingesta, limpieza, armonización e integración de los datos (ficheros y fuentes remotas).	Datasets intermedios de los datos en ficheros como de los datos de solicitudes remotas.
1 jun - 15 jun	Cierre de la preparación de los datos.	Correcciones de los notebooks de ingesta y procesamiento de datos. Búsquedas iniciales sobre modelos predictivos.	Datasets de datos finales. Lista de posibles modelos predictivos a probar.
16 jun - 1 jul	Modelado de modelos	Desarrollo de los modelos para entrenar. Entrenar los diferentes modelos y experimentar con las diferentes variables posibles.	Modelos entrenados. Documentación aparte sobre los diferentes modelos desarrollados.
2 jul - 15 ago	Evaluación de modelos y mejoras	Evaluar los modelos teniendo en cuenta las métricas resultantes de todas las pruebas. Mejorar los diferentes modelos en base a los resultados obtenidos en las pruebas.	Informes sobre las pruebas realizadas en los diferentes modelos y sus correspondientes resultados. Modelos predictivos mejorados (múltiples iteraciones de ellos) en base a lo obtenido en las pruebas anteriores.
16 ago - sept	Documentación final, revisión y entrega	Completar la limpieza y el refinamiento de la documentación final. Comprobar que todo lo elaborado funciona correctamente. Preparar el material de soporte necesario para la defensa.	Documentación revisada y pulida. Materiales de apoyo para la defensa (i.e. presentación, documentación adicional). Todos los archivos requeridos para la entrega.

Tabla 10: Planificación del TFG.

Dentro de la planificación no se tiene en cuenta los elementos relacionados con la documentación, debido a que este es un elemento que se ha planificado realizar a lo largo de toda la duración de este trabajo de fin de grado. Esto permite tener todos datos y todos los progresos realizados ya redactados para facilitar las actividades de pulido y mejora de la documentación de la memoria del TFG.

También como se puede ver en la planificación, la aplicación de CRISP-ML(Q) se ha adaptado a las necesidades de este trabajo, motivo por el cual los nombres de las fases no coinciden completamente con lo indicado en la correspondiente documentación sobre la metodología de desarrollo.

En el siguiente Diagrama de Gantt, podemos ver de forma más gráfica la planificación que se ha realizado para la elaboración de este trabajo.

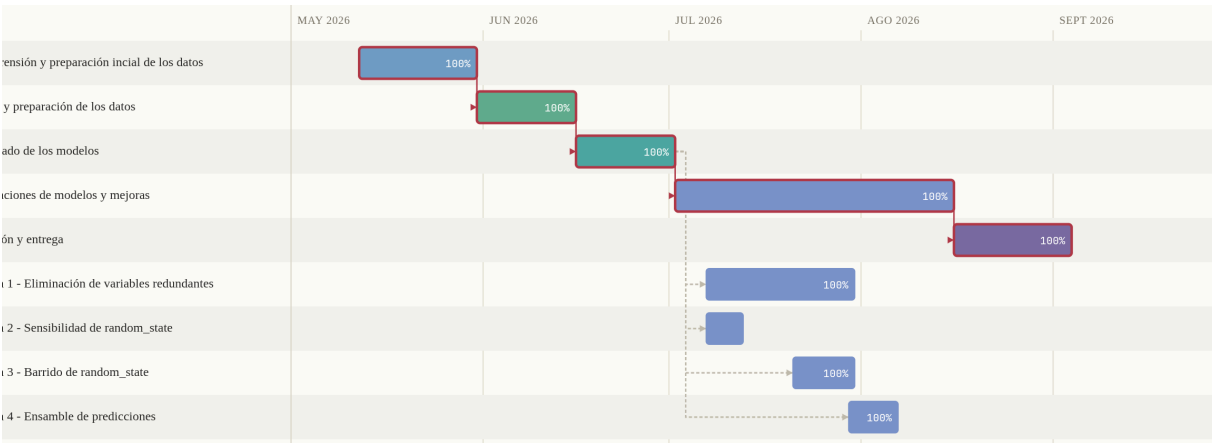


Figura 3: Diagrama de Gantt con la planificación del TFG.

Además del diagrama de Gantt mostrado en el diagrama anterior, en la siguiente tabla se muestra la distribución de horas y porcentaje de esfuerzo estimado para cada una de las correspondientes fases de desarrollo de este trabajo.

Fase CRISP-ML(Q)	Duración estimada (en horas)	Porcentaje de esfuerzo estimado (%)
Comprensión y preparación inicial de los datos	50	18.3%
Cierre de la preparación de los datos	40	13.3%
Modelado de los modelos	70	23.3%
Evaluación de modelos y mejoras	110	36.7%
Documentación final, revisión y entrega	30	10%
Total	300	100%

Tabla 11: Distribución de horas y porcentajes de esfuerzo estimados.

6.2. Puntos críticos

Dentro de un proyecto, especialmente dentro del ciclo de vida del mismo, los **puntos críticos** se definen como aquellos componentes, fases o tareas, que debido a la complejidad técnica o al desconocimiento inicial al empezar el proyecto, presentan un riesgo más elevado de poder provocar desviaciones en el cronograma establecido inicialmente. También pueden afectar en mayor o menor medida a la calidad del producto final.

Dentro del marco de este TFG, los principales puntos críticos se concentran en el desarrollo de modelos y la evaluación formal de los mismos. A continuación, se describirán los diferentes puntos críticos y sus riesgos de forma más detallada y las estrategias de mitigación aplicadas.

6.2.1. Desarrollo y parametrización de los modelos predictivos

El desarrollo y parametrización de los modelos predictivos se considera como un punto crítico debido a que el rendimiento final del sistema va a depender de las decisiones tomadas al inicio del desarrollo. Entre esas decisiones se destacan las siguientes:

1. Número y tipo de modelos a entrenar y desarrollar.
2. Selección del espacio y tiempo de búsqueda/cómputo disponible.

La selección de modelos debe de contar con el hecho de que se dispone de un set reducido de datos. Si se seleccionan modelos, los cuales no operan bien con pocos datos, entonces como consecuencia los rendimientos van a ser mucho peores de lo esperable.

Además, aún escogiendo modelos capaces de operar con pocos datos, siempre va a haber cierta cantidad de modelos que presenten sobreajuste ya por defecto. Esto mismo se podrá ver a lo largo del desarrollo de la memoria y en el **Anexo III**.

Un ajuste deficiente de hiperparámetros o una mala elección del espacio de búsqueda inicial puede provocar un sobreajuste mucho mayor al ya predispuesto por el hecho de la falta de datos.

6.2.2. Evaluación de modelos y control de calidad (QA)

La evaluación formal de los modelos es muy importante, porque de ella depende la selección final de los algoritmos a emplear. Un error en esta fase podría invalidar todas las conclusiones de este trabajo.

Además, la calidad de los datos de entrada, condiciona directamente a la fiabilidad de cualquier métrica obtenida, por lo que un fallo no detectado en las capas inferiores del flujo de trabajo puede propagarse silenciosamente hasta la evaluación final.

6.2.3. Estrategias de mitigación

Para reducir, en la medida de lo posible, los riesgos provenientes de los puntos anteriores se han aplicado las siguientes medidas de mitigación:

- Uso sistemático de validación cruzada repetida en todos los modelos desarrollados, en lugar de hacer uso de una única partición de validación, para obtener estimaciones más robustas y precisas sobre el rendimiento real.
- Separación estricta del dataset original en tres subconjuntos (entrenamiento, validación y evaluación) desde la fase inicial de modelos de modelos. Esto permite evitar que el set de evaluación se use en ningún momento en otros procesos para los cuales no fue creado, como puede ser la búsqueda de hiperparámetros.
- Registro de los campos sin datos de los datasets originales mediante un archivo de banderas de imputación. Esto permite tener un mayor conocimiento y control sobre los datos que se poseen o que siguen carentes en todo momento, además, permite la determinación sobre qué datos son mediciones reales y cuáles son estimaciones.
- Revisiones periódicas del proceso de desarrollo en forma de reuniones semanales (véase **Seguimiento**), permitiendo así detectar errores o desviaciones en el desarrollo y parametrización de los modelos antes de que afecten a fases posteriores.
- Realización de pruebas complementarias para la simplificación y mejor selección de los modelos finales (véase **Anexos**). Dichas pruebas también sirven para medir la robustez actual de los modelos.

6.3. Seguimiento

El seguimiento del progreso del presente TFG se realiza mediante la celebración de reuniones periódicas, generalmente semanales, entre el alumnado, el tutor y el co-tutor del proyecto. Estas reuniones constituyen el mecanismo principal de control y coordinación a lo largo de todo el desarrollo del trabajo.

Estas reuniones tienen los siguientes objetivos:

1. **Comunicación de avances:** exponer el trabajo realizado durante el período transcurrido desde la anterior reunión, incluyendo los resultados obtenidos así como las dificultades o problemas encontrados.
2. **Resolución de dudas:** plantear y aclarar todas las cuestiones técnicas, metodológicas o de alcance que hayan surgido durante el desarrollo de las tareas.
3. **Definición de próximos pasos:** Establecer y acordar las tareas a abordar durante el siguiente intervalo temporal, ajustando la planificación en función del estado real del proyecto.

Este esquema de seguimiento continuo permite mantener una supervisión constante sobre la evolución del proyecto, facilitando la detección temprana de desviaciones respecto a la planificación inicial y posibilitando la adopción de medidas correctoras cuando resulta necesario. Asimismo, la periodicidad semanal proporciona un marco de trabajo estructurado que favorece la organización del esfuerzo en tiempos cortos y bien definidos, contribuyendo a un desarrollo incremental y controlado del trabajo.

En casos en los que las circunstancias del proyecto lo requiera, por ejemplo, antes hitos relevantes o la necesidad de abordar decisiones de mayor importancia, la frecuencia de las reuniones puede ajustarse, intensificando o espaciando su frecuencia en función de las necesidades puntuales del desarrollo.

6.4. Justificación de desviaciones

A fecha de 25/08/2026, no existen diferencias con la planificación creada.

7. Principales aportaciones

Teniendo en cuenta los objetivos que se han establecido al principio de este TFG, los cuales se encuentran en la sección de Objetivos, las principales aportaciones que se proporcionan son las siguientes:

1. **Evidencia práctica sobre la compatibilidad entre datos in situ y datos remotos de menor resolución para la estimación de biodiversidad del suelo:** la integración y armonización de las mediciones de campo de alta calidad del proyecto SOB4ES con fuentes remotas de acceso abierto (Google Earth Engine, Copernicus) aporta un punto de comparación adicional a la evidencia ya existente sobre si los datos in situ siguen ofreciendo una ventaja informativa frente a los derivados exclusivamente de teledetección (véase Antecedentes y contexto), además de un proceso de armonización documentado que podría reutilizarse con otros grupos de especies o campañas de muestreo del propio proyecto.
2. **Justificación metodológica de qué técnicas de aprendizaje automático son adecuadas para las características concretas de este problema:** la selección de algoritmos (regresión regularizada, ensamblados basados en árboles y redes neuronales) se ha justificado explícitamente en función de las limitaciones del problema, tamaño muestral reducido e indicadores de salida potencialmente correlacionados entre sí, en lugar de partir de una elección arbitraria o exhaustiva de algoritmos.
3. **Evidencia empírica sobre qué familia de algoritmos generaliza mejor en la predicción de biodiversidad del suelo con un conjunto de datos reducido:** la comparación sistemática de regresión Ridge, Random Forest, XGBoost, RegressorChain y redes neuronales MLP bajo un mismo marco de validación cruzada repetida y una misma partición estricta de entrenamiento, validación y evaluación permite contrastar empíricamente, con datos propios, la hipótesis recurrente en la literatura revisada de que los modelos basados en árboles igualan o superan a las redes neuronales cuando el conjunto de datos es pequeño.
4. **Evaluación de si el modelado conjunto (multi-salida) aporta ventaja frente al modelado independiente por indicador:** al entrenar variantes multi-salida (rf_multisalida, xgb_multisalida, mlp_multisalida, regressorchain) junto con sus equivalentes de salida única bajo las mismas condiciones, este trabajo aporta evidencia sobre si aprovechar las relaciones entre distintos indicadores de biodiversidad del suelo mejora o no las predicciones frente a tratarlos de forma independiente, una pregunta que la literatura revisada no responde de forma directa para este dominio.

A esto se suma el resultado de la prueba de eliminación de variables sobre la relevancia real de los micronutrientes del suelo: la evaluación sistemática del efecto de eliminar `cu_z`, `ni_z` y `mo_z`, de forma individual y combinada, aporta evidencia sobre si estas variables son realmente necesarias para la predicción o si su contribución es redundante frente al resto de variables climáticas y edáficas empleadas. Dichos resultados se pueden ver con más detalle en el ANEXO (sabedios).

5. **Aplicación de una metodología de gestión de proyectos de ciencia de datos como marco de rigor metodológico:** el uso sistemático de CRISP-ML(Q) a lo largo de este trabajo ha permitido anticipar y mitigar riesgos habituales en estudios de este tipo, como por ejemplo, el sobreajuste con conjuntos de datos reducidos, fuga de información entre las fases de entrenamiento y evaluación, entre otros, mediante decisiones metodológicas concretas (partición estricta de datos, validación cruzada repetida), que quedan documentadas y podrían servir de referencia para futuros trabajos de investigación dentro del propio proyecto SOB4ES.

8. Conclusiones

9. Vías de trabajo futuro

A partir del trabajo realizado, se han identificado las siguientes líneas de continuación:

1. **Ampliar la cobertura geográfica y temporal del conjunto de datos** mediante la incorporación de nuevas campañas de muestreo dentro del proyecto SOB4ES a medida que estén disponibles, para reducir la dependencia del modelo de un conjunto de datos reducido.
2. **Extender la comparación a otros grupos de biodiversidad del suelo** más allá de los ya integrados en los diferentes modelos creados para este trabajo, replicando el mismo pipeline y metodología.
3. **Explorar posibles arquitecturas de aprendizaje profundo adicionales** como, por ejemplo, los modelos basados en embeddings de imágenes satelitales descritos previamente en Antecedentes y contexto, una vez que se dispongan de un mayor volumen de datos que permita mitigar de forma más eficiente el riesgo de sobreajuste ya existente a causa de la falta de datos.
4. **Automatizar el reentrenamiento periódico de los modelos**, cerrando de esta forma la fase de “Monitoreo y mantenimiento” de CRISP-ML(Q), la cual se encuentra descrita en Sección 4, de forma que el sistema pueda incorporar nuevas mediciones sin tener que repetir todo el proceso de forma manual.
5. **Exponer los modelos finales a través de un servicio API** que permita obtener una predicción de biodiversidad para un nuevo punto geográfico a partir de sus variables ambientales, facilitando así el uso por parte de terceros ajenos al desarrollo de este TFG.
6. **Crear un sistema multi-modelo** que permita realizar las predicciones de todos los targets teniendo en cuenta qué modelos funcionan mejor para cada target. Esto permite que todos los targets que se empleen en un futuro puedan ser empleados sin que un modelo en específico los haga inutilizables como target.
7. **Desarrollar un índice** que facilite el entrenamiento de los modelos teniendo en cuenta el coste de obtención/procesamiento del dato. Esto permite tener un índice que permita una mayor facilidad a la hora de entrenar modelos similares y tener como resultado mejores predicciones.

10. Referencias

- [1] M. A. Anthony, S. F. Bender, y M. G. A. van der Heijden, «Enumerating soil biodiversity», 2023. [En línea]. Disponible en: <https://doi.org/10.1073/pnas.2304663120>
- [2] FAO, *State of Knowledge of Soil Biodiversity: Status, Challenges and Potentialities*. 2020. [En línea]. Disponible en: <https://doi.org/10.4060/cb1928en>
- [3] SOB4ES, «Integrating Soil Biodiversity into Ecosystem Services». [En línea]. Disponible en: <https://www.sob4es.eu/>
- [4] «Google Earth Engine». [En línea]. Disponible en: <https://earthengine.google.com/>
- [5] «Homepage | Copernicus». [En línea]. Disponible en: <https://www.copernicus.eu/en>
- [6] A. M. J.-C. Wadoux, «Artificial intelligence in soil science», *European Journal of Soil Science*, vol. 76, n.º 2, p. e70080, 2025.
- [7] G. Salako, A. Zaitsev, B. Betancur-Corredor, y D. J. Russell, «Modelling and Spatial Prediction of Earthworms Ecological-Groups Distribution Reveal Their Habitat and Environmental Preferences», *SSRN*, 2024, [En línea]. Disponible en: <https://ssrn.com/abstract=4925999>
- [8] H. R. P. Phillips y et al., «Global distribution of earthworm diversity», *Science*, vol. 366, n.º 6464, pp. 480-485, 2019.
- [9] «Digging deeper: deep joint species distribution modeling reveals environmental drivers of Earthworm Communities», *ResearchGate*, 2025, [En línea]. Disponible en: <https://www.researchgate.net/publication/392737042>
- [10] «Limits and promises of Earth Observation Foundation Models in Predicting Multi-Trophic Soil Biodiversity», *bioRxiv*, 2025, [En línea]. Disponible en: <https://www.biorxiv.org/content/10.1101/2025.05.16.654504>
- [11] «Soil microbiome prediction using traditional machine learning and deep learning models», *Scientific Reports*, 2026, [En línea]. Disponible en: <https://www.nature.com/articles/s41598-026-39537-w>
- [12] X. Tian *et al.*, «Spatiotemporal prediction of soil organic carbon density in Europe (2000–2022) using earth observation and machine learning», *PeerJ*, vol. 13, n.º e19605, 2025.
- [13] V. F. Rodríguez-Galiano, B. Ghimire, J. Rogan, M. Chica-Olmo, y J. P. Rigol-Sánchez, «An assessment of the effectiveness of a random forest classifier for land-cover classification», *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 67, pp. 93-104, 2012.
- [14] S. M. Shawon, F. B. Ema, A. K. Mahi, F. L. Niha, y H. T. Zubair, «Crop yield prediction using machine learning: An extensive and systematic literature review», *Smart Agricultural Technology*, vol. 10, n.º 100718, 2025.
- [15] «Ml-ops.org». [En línea]. Disponible en: <https://ml-ops.org/content/crisp-ml>
- [16] «Desarrollo iterativo e incremental», *Proyectos Ágiles*. [En línea]. Disponible en: <https://proyectosagiles.org/desarrollo-iterativo-incremental/>
- [17] N. Hotz, «What is CRISP DM?», *Data Science PM*. [En línea]. Disponible en: <https://www.datascience-pm.com/crisp-dm-2/>
- [18] Wikipedia contributors, «Arthur Samuel (computer scientist)», *Wikipedia*. [En línea]. Disponible en: [https://en.wikipedia.org/wiki/Arthur_Samuel_\(computer_scientist\)](https://en.wikipedia.org/wiki/Arthur_Samuel_(computer_scientist))
- [19] Aurélien Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*, 3.ª ed. O'Reilly Media, Inc., 2022.
- [20] «LaTeX - A document preparation system». [En línea]. Disponible en: <https://www.latex-project.org/>
- [21] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers», *IBM Journal of Research and Development*, vol. 3, n.º 3, pp. 210-229, 1959.
- [22] Wikipedia contributors, «Tom M. Mitchell», *Wikipedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Tom_M._Mitchell

- [23] Reuters, «Computer trained to 'read' mind images of words», Reuters. [En línea]. Disponible en: <https://www.reuters.com/article/us-computer-mind-idUSN2939892820080529/>
- [24] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [25] D. Bergmann, «Machine learning». [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/machine-learning>
- [26] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 2.^a ed. MIT Press, 2018.
- [27] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.^a ed. Pearson, 2020.
- [28] Akenenosketch, «TFG-SOB4ES», GitHub.
- [29] A. E. Hoerl y R. W. Kennard, «Ridge Regression: Biased Estimation for Nonorthogonal Problems», *Technometrics*, vol. 12, n.º 1, pp. 55-67, 1970.
- [30] «Ridge Regression», IBM. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/ridge-regression>
- [31] L. Breiman, «Random Forests», *Machine Learning*, vol. 45, n.º 1, pp. 5-32, 2001.
- [32] «scikit-learn: machine learning in Python — scikit-learn 1.9.0 documentation». [En línea]. Disponible en: <https://scikit-learn.org/stable/>
- [33] «Welcome to the SHAP documentation — SHAP latest documentation». [En línea]. Disponible en: <https://shap.readthedocs.io/en/latest/>
- [34] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System», *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785-794, 2016.
- [35] J. Read, B. Pfahringer, G. Holmes, y E. Frank, «Classifier Chains for Multi-label Classification», *Machine Learning*, vol. 85, n.º 3, pp. 333-359, 2011.
- [36] J. Read, B. Pfahringer, G. Holmes, y E. Frank, «Classifier chains: A review and perspectives», *Journal of Artificial Intelligence Research*, vol. 70, pp. 683-718, 2021.
- [37] J. Read y L. Martino, «Probabilistic regressor chains with Monte-Carlo methods», *Neurocomputing*, vol. 413, pp. 471-486, 2020.
- [38] L. E. Sucar, C. Bielza, E. F. Morales, P. Hernandez-Leal, J. H. Zaragoza, y P. Larrañaga, «Multi-label classification with Bayesian network-based chain classifiers», *Pattern Recognition Letters*, vol. 41, pp. 14-22, 2014.
- [39] PyTorch Contributors, «PyTorch documentation». [En línea]. Disponible en: <https://docs.pytorch.org/docs/2.12/index.html>
- [40] D. E. Rumelhart, G. E. Hinton, y R. J. Williams, «Learning representations by back-propagating errors», *Nature*, vol. 323, n.º 6088, pp. 533-536, 1986.
- [41] D. P. Kingma y J. Ba, «Adam: A Method for Stochastic Optimization», *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.
- [42] J. Cohen, «A Coefficient of Agreement for Nominal Scales», *Educational and Psychological Measurement*, vol. 20, n.º 1, pp. 37-46, 1960.
- [43] J. Cohen, «Weighted Kappa: Nominal Scale Agreement with Provision for Scaled Disagreement or Partial Credit», *Psychological Bulletin*, vol. 70, n.º 4, pp. 213-220, 1968.
- [44] Copernicus Data Space Ecosystem, «Copernicus DEM - Global and European Digital Elevation Model», Copernicus Data Space Ecosystem. [En línea]. Disponible en: <https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM>
- [45] «pandas - Python Data Analysis Library». [En línea]. Disponible en: <https://pandas.pydata.org/>
- [46] «NumPy». [En línea]. Disponible en: <https://numpy.org/es/>
- [47] «Rasterio: access to geospatial raster data — rasterio 1.4.4 documentation». [En línea]. Disponible en: <https://rasterio.readthedocs.io/en/stable/>

- [48] «Xarray documentation», Xarray. [En línea]. Disponible en: <https://docs.xarray.dev/en/stable/>
- [49] «dbfread - Read DBF Files with Python — dbfread 2.0.7 documentation». [En línea]. Disponible en: <https://dbfread.readthedocs.io/en/latest/>
- [50] «netCDF4 API documentation». [En línea]. Disponible en: <https://unidata.github.io/netcdf4-python/>
- [51] Codificando Bits, *Los sets de entrenamiento, validación y prueba*, (16 de junio de 2023). [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=79K93XBOSlg>
- [52] Na y Na, «Sets de Entrenamiento, Test y Validación», Aprende Machine Learning. [En línea]. Disponible en: <https://www.aprendemachinelearning.com/sets-de-entrenamiento-test-validacion-cruzada/>
- [53] «XGBoost Documentation — xgboost 3.3.0 documentation». [En línea]. Disponible en: <https://xgboost.readthedocs.io/en/stable/>
- [54] «XGBoost», IBM. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/xgboost>
- [55] «Joblib: running Python functions as pipeline jobs — joblib 1.5.3 documentation». [En línea]. Disponible en: <https://joblib.readthedocs.io/en/stable/>
- [56] «Matplotlib — Visualization with Python». [En línea]. Disponible en: <https://matplotlib.org/>
- [57] «seaborn: statistical data visualization — seaborn 0.13.2 documentation». [En línea]. Disponible en: <https://seaborn.pydata.org/>
- [58] Michael Waskom, «seaborn: statistical data visualization», *The Journal of Open Source Software*, vol. 6, n.º 60, p. 3021, abr. 2021.
- [59] «Welcome to Python.org», Python.org. [En línea]. Disponible en: <https://www.python.org/>
- [60] «Project Jupyter», Home. [En línea]. Disponible en: <https://jupyter.org/>
- [61] Software Freedom Conservancy, «Git».
- [62] GitHub, Inc., «GitHub · Build and ship software on a single, collaborative platform».
- [63] «Climate Data Store». [En línea]. Disponible en: <https://cds.climate.copernicus.eu/>
- [64] Wikipedia contributors, «Google Docs», Wikipedia. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Google_Docs
- [65] «Typst: The new foundation for documents», Typst. [En línea]. Disponible en: <https://typst.app/>
- [66] «Free Citation Generator | APA, MLA, Chicago | Scribbr», Scribbr. [En línea]. Disponible en: <https://www.scribbr.com/citation/generator/>
- [67] Microsoft, «Visual Studio Code - Code Editing. Redefined.».
- [68] Microsoft, «Code - OSS», GitHub repository.
- [69] Akenosketch, «gantt-web», GitHub repository. [En línea]. Disponible en: <https://github.com/Akenosketch/gantt-web>
- [70] diagrams.net, «draw.io - Free Flowchart Maker and Diagrams Online».
- [71] S. Studer *et al.*, «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology», 2021. [En línea]. Disponible en: <https://doi.org/10.3390/make3020020>
- [72] «CoLAB», Google for Developers. [En línea]. Disponible en: <https://developers.google.com/colab>
- [73] I. Guyon y A. Elisseeff, «An Introduction to Variable and Feature Selection», *Journal of Machine Learning Research*, vol. 3, pp. 1157-1182, 2003.
- [74] R. Tibshirani, «Regression Shrinkage and Selection via the Lasso», *Journal of the Royal Statistical Society: Series B*, vol. 58, n.º 1, pp. 267-288, 1996.
- [75] M. Sokolova y G. Lapalme, «A Systematic Analysis of Performance Measures for Classification Tasks», *Information Processing & Management*, vol. 45, n.º 4, pp. 427-437, 2009.

11. Anexos

En los siguientes apartados se introducirán elementos que serán de utilidad par tener un mayor entendimiento sobre las diferentes partes de este TFG.

11.1. Informe EDA

En este anexo se presenta el análisis exploratorio de datos (EDA, Exploratory Data Analysis) realizado sobre el dataset final obtenido tras la Capa de procesamiento de datos, previo al desarrollo de los modelos predictivos descritos en la Capa de modelado predictivo. El objetivo de este análisis es doble: por un lado, caracterizar la calidad y la estructura de los datos disponibles (valores faltantes, distribución de las variables, escalas); por otro, justificar empíricamente algunas de las decisiones metodológicas tomadas en la sección Marco teórico o práctico, en particular la normalización de las variables predictoras (véase Regresión Ridge) y el uso de modelos robustos frente a la colinealidad (véase Selección de variables).

Antes de entrenar un modelo predictivo conviene entender qué hay realmente en los datos: cuántas observaciones hay, cómo se distribuyen entre países y usos de suelo, si faltan valores, si hay variables que se mueven juntas (colinealidad), si existen valores atípicos y si las relaciones entre variables predictoras y variables objetivo son lineales o no. Estas preguntas requieren calcular estadísticos y visualizar los datos directamente, que es lo que se hace en este anexo. Las conclusiones aquí obtenidas condicionan decisiones concretas del cuerpo principal del TFG, como qué algoritmo de regularización usar (sección Regresión Ridge), si tiene sentido un enfoque de modelado multisalida (Anexo V) o qué limitaciones de generalización hay que declarar en las conclusiones.

11.2. Modelos empleados

En esta sección de los anexos se detalla, para cada uno de los ocho modelos desarrollados en este TFG, su configuración concreta y los resultados numéricos obtenidos sobre `eval.csv`, el subconjunto de evaluación final que ningún modelo ha visto durante el *tuning* ni durante la validación cruzada (véase **Capa de modelado predictivo**).

Los 21 targets predichos corresponden a los índices de biodiversidad (Shannon) y de riqueza de especies de 11 grupos taxonómicos distintos, los cuales se encuentran mencionados en el **Anexo II** y en **Variables empleadas**, todos ellos normalizados a la **escala z** antes del entrenamiento. Por ello un R^2 de 0 no implica que no prediga nada, sino que su rendimiento es equivalente al de predecir siempre la media del conjunto de entrenamiento, mientras que un valor negativo indica que el modelo predice peor que dicha media.

A lo largo de todos los modelos, `earthworm_shannon_z` y `earthworm_richness_z` se tratan como los dos targets prioritarios, por lo que en los siguientes subapartados se hará referencia explícita a los rendimientos obtenidos a la hora de predecir dichos targets.

También a lo largo de los siguientes subapartados, se tendrá una explicación de los modelos y de los resultados obtenidos, como también el notebook base de dicho modelo exportado.

11.2.1. Modelo de Regresión Ridge

11.2.1.1. Fundamento y configuración

Ridge (véase **Modelo de Regresión Ridge**) se emplea como modelo baseline del trabajo: al ser un modelo lineal, permite establecer un suelo mínimo de rendimiento frente al que comparar el resto de modelos, más complejos y con mayor capacidad de capturar relaciones no lineales.

Al tratarse de un modelo determinista (no depende de una semilla aleatoria de entrenamiento), se entrena un único modelo por target, sin necesidad de recurrir a un ensamblado de varias semillas para reducir la varianza.

La búsqueda de hiperparámetros se realiza mediante `GridSearchCV` (validación cruzada de 5 pliegues, `scoring='r2'`) sobre `X_train`, para cada uno de los 21 targets de forma independiente, explorando:

- **alpha**: 120 valores en escala logarítmica entre 10^{-1} y 10^8 , para cubrir desde una regularización casi nula hasta una regularización extrema (necesaria para los targets con menor señal predictiva, donde el modelo tiende a sobreajustar si `alpha` es bajo).
- **fit_intercept**: True / False.
- **solver**: auto, cholesky, lsqr.

Para evitar seleccionar una combinación que sobreajuste, se aplica además un filtro de estabilidad: de entre todas las combinaciones evaluadas, solo se consideran válidas aquellas cuya diferencia entre el R^2 de entrenamiento y el R^2 de validación cruzada (gap) sea igual o inferior a 0.10; si ninguna combinación cumple ese criterio para un target concreto, se selecciona la de menor gap con seguridad. El `alpha` óptimo encontrado varía considerablemente entre targets (de 105.98 a 719.69), lo que confirma que la señal predictiva disponible es distinta para cada grupo taxonómico y que un único valor de regularización global no sería adecuado.

11.2.1.2. Entrenamiento

Al tratarse de un modelo determinista, no se recurre a un ensamblado de semillas: se entrena un único modelo por target con los hiperparámetros seleccionados en el *tuning*.

La validación cruzada repetida (`RepeatedKfold`, 5 pliegues $\times$ 3 repeticiones = 15 evaluaciones) sobre `X_train` no mostró señales relevantes de sobreajuste: de los 21 targets, únicamente `bac_shannon_z` superó el umbral de aviso (diferencia Train-CV > 0.15, concretamente 0.161).

El resto se mantuvo en un rango de diferencia razonable (entre 0.054 y 0.161), coherente con la baja capacidad de un modelo lineal para memorizar el conjunto de entrenamiento.

11.2.1.3. Explicabilidad y variables más relevantes

A partir del valor absoluto de los coeficientes del modelo (interpretables directamente al estar las variables normalizadas), las variables más relevantes a nivel global fueron, por este orden: soil_ph_z, gee_temp_media_C_z, eu_p_z, dem_elevacion_m_z, zn_z, gee_humedad_rel_pct_z, clay_content_z, eu_ph_z, pb_z y soil_moisture_z.

El pH del suelo y la temperatura media (variable climática remota de GEE) destacan de forma consistente como los predictores más influyentes del conjunto.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Importancia de variables.**

11.2.1.4. Resultados sobre eval.csv

El R^2 medio sobre los 21 targets es de **-0.016**, es decir, en promedio Ridge se comporta ligeramente peor que predecir la media del conjunto de entrenamiento.

Sobre los dos targets prioritarios obtiene $R^2=0.2395$ (earthworm_shannon_z) y $R^2=0.2789$ (earthworm_richness_z), sus dos mejores resultados junto con macro_shannon_z (0.1257).

En el extremo opuesto, coll_species_richness_z (-0.7341) y meso_shannon_z (-0.2499) muestran el peor ajuste, indicando que la relación lineal simple no captura la variabilidad de estos grupos.

Variable	Target	R^2	RMSE	MAE
Shannon nemátodos	nematode_shannon_z	0.0112	0.9821	0.7929
Shannon macrofauna	macro_shannon_z	0.1257	0.9391	0.8177
Shannon lombrices	earthworm_shannon_z	0.2395	0.8704	0.7094
Shannon oribátidos	orib_shannon_z	0.1137	1.0766	0.9075
Shannon mesostigmátidos	meso_shannon_z	-0.2499	1.0918	0.9023
Shannon colémbolos	coll_shannon_z	-0.4998	0.7758	0.6426
Shannon bacterias	bac_shannon_z	-0.0812	1.0413	0.7158
Shannon hongos	fun_shannon_z	-0.0161	1.1533	0.9503
Shannon eucariotas	euk_shannon_z	0.0407	0.9499	0.7011
Shannon oomicetos	oomy_shannon_z	0.0857	0.9996	0.7418
Shannon cercozoos	cerc_shannon_z	0.0155	0.8886	0.6332
Riqueza macrofauna	macro_order_richness_z	0.1521	0.8433	0.7105
Riqueza lombrices	earthworm_richness_z	0.2789	0.7754	0.6143
Riqueza oribátidos	orib_species_richness_z	0.0921	1.1400	0.7670
Riqueza mesostigmátidos	meso_species_richness_z	-0.0959	1.0097	0.7693
Riqueza colémbolos	coll_species_richness_z	-0.7341	0.6780	0.5409
Riqueza bacterias (ASV)	bac_asv_richness_z	-0.0761	1.1398	0.8335
Riqueza hongos (ASV)	fun_asv_richness_z	0.0087	1.0731	0.8114
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.0400	0.7955	0.5775
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.1035	0.9510	0.7711
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.1019	0.9350	0.7211

Tabla 12: Resultados de Ridge sobre eval.csv, por target.

SOB4ES - Modelo de Regresión (Ridge)

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook recorre el ciclo de vida completo del modelo de regresión lineal regularizada (Ridge Regression). Se usa Ridge en lugar de regresión lineal simple porque regulariza los coeficientes (penalización L2), lo que mejora la estabilidad cuando hay variables correlacionadas o cuando el número de predictores es alto.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning:** búsqueda del valor óptimo de regularización α .
4. **Validación cruzada:** evaluación de robustez.
5. **Importancia de variables:** coeficientes y permutation importance.
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
import itertools
from pathlib import Path

from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, cross_validate, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

# ignoramos los warnings
warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/reg/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
```

```

'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_c_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas correctamente...

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)

def get_n_jobs(fracccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos 'reserva' nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fracccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimacion imparcial del rendimiento.

```

In [3]: def cargar_csv(ruta, nombre):
    """Carga un CSV (sep=', ' o ';') y valida las columnas necesarias."""
    if not os.path.exists(ruta):
        raise FileNotFoundError(f"No se encontro el archivo: {ruta}")
    try:
        df = pd.read_csv(ruta, sep=',')
        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
    except Exception:
        df = pd.read_csv(ruta, sep=';')
    print(f" {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas")

```

```

return df

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados...")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

```

```

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

```

```

Datasets cargados...
X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)

```

3.- Búsqueda de hiperparámetros (Tuning)

En Ridge Regression el principal hiperparametro es `alpha`, que controla la intensidad de la regularización:

- **alpha cercano a 0:** comportamiento similar a la regresión lineal ordinaria (sin regularización).
- **alpha alto:** coeficientes más penalizados, modelo más simple y resistente al sobreajuste.

También se exploran `fit_intercept` y el `solver` numérico. La búsqueda se realiza exclusivamente sobre `X_train`.

```

In [4]: # Tuning de hiperparámetros para cada target
print("Iniciando búsqueda de hiperparámetros para cada target...")
print("Aplicando GridSearchCV y filtro estricto de estabilidad (gap Train-CV ≤ 0.10)...")

idx = 0 # Contador auxiliar

global_start = time.time()

REG_OPTIMAL_PARAMS_PER_TARGET = {}

# Definimos un espacio exhaustivo con alphas masivos para targets sin señal predictiva
param_grid = {
    'alpha': np.logspace(-1, 8, 120), # Exploración milimétrica hasta 10^8
    'fit_intercept': [True, False],
    'solver': ['auto', 'cholesky', 'lsqr'] # Solvers altamente estables para matrices con alphas grandes
}

for target_col in TARGETS:
    t0 = time.time()
    y = y_train[target_col].values
    idx += 1

    search = GridSearchCV(
        estimator=Ridge(random_state=RANDOM_STATE),
        param_grid=param_grid,
        cv=5,
        scoring='r2',
        n_jobs=N_JOBS,
        return_train_score=True
    )

    search.fit(X_train, y)

    df_res = pd.DataFrame(search.cv_results_)
    df_res['gap'] = df_res['mean_train_score'] - df_res['mean_test_score']

    # Ajustamos el umbral a 0.10 para asegurar que quede bien lejos del límite de sobreajuste
    UMBRAL_GAP = 0.10
    soluciones_estables = df_res[df_res['gap'] ≤ UMBRAL_GAP]

    if not soluciones_estables.empty:
        # Entre las que no sobreajustan, elegimos la que tenga mejor rendimiento en validación
        idx_optimo = soluciones_estables['mean_test_score'].idxmax()
    else:
        # Si el target es crítico y todo sobreajusta, forzamos el alpha más alto para desplazar el gap a 0
        idx_optimo = df_res['gap'].idxmin()

    mejores_params = df_res.loc[idx_optimo, 'params']
    mejor_r2_cv = df_res.loc[idx_optimo, 'mean_test_score']
    gap_elegido = df_res.loc[idx_optimo, 'gap']

    REG_OPTIMAL_PARAMS_PER_TARGET[target_col] = mejores_params

    print(f"[{idx:2d}/{len(TARGETS)}] | [{target_col:<25}] | R2 CV: {mejor_r2_cv:.4f} | Gap Tuning: {gap_elegido:.4f} | Alpha: {mejor")

print(f"\nTuning completado en {(time.time() - global_start) / 60:.1f} minutos.\n")

```

Iniciando búsqueda de hiperparámetros para cada target...

Aplicando GridSearchCV y filtro estricto de estabilidad (gap Train-CV ≤ 0.10)...

```
[ 1/21] | [nematode_shannon_z] | R2 CV: 0.0939 | Gap Tuning: 0.0932 | Alpha: 358.61 | Tiempo: 5.4s
```

```
[ 2/21] | [macro_shannon_z] ] | R2 CV: 0.0954 | Gap Tuning: 0.0933 | Alpha: 358.61 | Tiempo: 4.6s
[ 3/21] | [earthworm_shannon_z] ] | R2 CV: 0.1935 | Gap Tuning: 0.0945 | Alpha: 253.14 | Tiempo: 4.2s
[ 4/21] | [orib_shannon_z] ] | R2 CV: 0.0531 | Gap Tuning: 0.0974 | Alpha: 150.13 | Tiempo: 4.4s
[ 5/21] | [meso_shannon_z] ] | R2 CV: 0.0701 | Gap Tuning: 0.0966 | Alpha: 126.14 | Tiempo: 3.1s
[ 6/21] | [coll_shannon_z] ] | R2 CV: 0.1652 | Gap Tuning: 0.0957 | Alpha: 150.13 | Tiempo: 4.2s
[ 7/21] | [bac_shannon_z] ] | R2 CV: 0.0663 | Gap Tuning: 0.0976 | Alpha: 105.98 | Tiempo: 4.7s
[ 8/21] | [fun_shannon_z] ] | R2 CV: -0.0026 | Gap Tuning: 0.0748 | Alpha: 604.66 | Tiempo: 4.6s
[ 9/21] | [euk_shannon_z] ] | R2 CV: 0.1012 | Gap Tuning: 0.0959 | Alpha: 126.14 | Tiempo: 4.9s
[10/21] | [oomy_shannon_z] ] | R2 CV: 0.0920 | Gap Tuning: 0.0982 | Alpha: 358.61 | Tiempo: 4.6s
[11/21] | [cerc_shannon_z] ] | R2 CV: 0.0204 | Gap Tuning: 0.0615 | Alpha: 719.69 | Tiempo: 4.5s
[12/21] | [macro_order_richness_z] ] | R2 CV: 0.0668 | Gap Tuning: 0.0948 | Alpha: 426.83 | Tiempo: 4.2s
[13/21] | [earthworm_richness_z] ] | R2 CV: 0.3010 | Gap Tuning: 0.0967 | Alpha: 150.13 | Tiempo: 3.1s
[14/21] | [orib_species_richness_z] ] | R2 CV: 0.0203 | Gap Tuning: 0.0858 | Alpha: 508.02 | Tiempo: 4.3s
[15/21] | [meso_species_richness_z] ] | R2 CV: 0.0618 | Gap Tuning: 0.0848 | Alpha: 126.14 | Tiempo: 4.6s
[16/21] | [coll_species_richness_z] ] | R2 CV: 0.1840 | Gap Tuning: 0.0941 | Alpha: 126.14 | Tiempo: 4.2s
[17/21] | [bac_asv_richness_z] ] | R2 CV: 0.0904 | Gap Tuning: 0.0946 | Alpha: 105.98 | Tiempo: 4.3s
[18/21] | [fun_asv_richness_z] ] | R2 CV: -0.0091 | Gap Tuning: 0.0528 | Alpha: 604.66 | Tiempo: 4.2s
[19/21] | [euk_asv_richness_z] ] | R2 CV: 0.1332 | Gap Tuning: 0.0944 | Alpha: 126.14 | Tiempo: 4.5s
[20/21] | [oomy_asv_richness_z] ] | R2 CV: 0.1445 | Gap Tuning: 0.0952 | Alpha: 358.61 | Tiempo: 4.2s
[21/21] | [cerc_asv_richness_z] ] | R2 CV: 0.1753 | Gap Tuning: 0.0966 | Alpha: 212.68 | Tiempo: 4.7s
```

Tuning completado en 1.5 minutos.

4.- Validación cruzada

Se evalúa la robustez del modelo con **validación cruzada repetida** (5 pliegues x 3 repeticiones = 15 evaluaciones) sobre `X_train`.

En regresión lineal la diferencia entre train y validation suele ser pequeña; una diferencia grande indicaría que el modelo no captura bien la relación entre variables.

```
In [5]: # Validación Cruzada usando hiperparámetros específicos
print('Validacion cruzada repetida sobre X_train...\n')

cv_splitter = RepeatedKfold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
metricas = ['r2', 'neg_root_mean_squared_error', 'neg_mean_absolute_error']

print(f"{'TARGET':<25} | {'TRAIN R2':<10} | {'CV R2':<10} | {'DIFF':<10} | {'ESTADO'}")
print("-" * 75)

for nombre_target in TARGETS:
    y_prueba = y_train[nombre_target].values
    mejores_params = REG_OPTIMAL_PARAMS_PER_TARGET[nombre_target]

    modelo_cv = Ridge(**mejores_params, random_state=RANDOM_STATE)

    resultados = cross_validate(
        estimator=modelo_cv,
        X=X_train,
        y=y_prueba,
        cv=cv_splitter,
        scoring=metricas,
        return_train_score=True
    )

    r2_train = np.mean(resultados['train_r2'])
    r2_cv = np.mean(resultados['test_r2'])
    diferencia = r2_train - r2_cv

    estado = 'Aviso: Posible sobreajuste' if diferencia > 0.15 else 'OK'

    print(f"{nombre_target:<25} | {r2_train:<10.3f} | {r2_cv:<10.3f} | {diferencia:<10.3f} | {estado}")
```

Validacion cruzada repetida sobre X_train...

TARGET	TRAIN R2	CV R2	DIFF	ESTADO
nematode_shannon_z	0.188	0.113	0.075	OK
macro_shannon_z	0.189	0.101	0.088	OK
earthworm_shannon_z	0.288	0.188	0.100	OK
orib_shannon_z	0.154	0.029	0.125	OK
meso_shannon_z	0.169	0.061	0.108	OK
coll_shannon_z	0.261	0.147	0.114	OK
bac_shannon_z	0.168	0.007	0.161	Aviso: Posible sobreajuste
fun_shannon_z	0.072	0.009	0.064	OK
euk_shannon_z	0.200	0.077	0.124	OK
oomy_shannon_z	0.189	0.088	0.101	OK
cerc_shannon_z	0.083	0.029	0.054	OK
macro_order_richness_z	0.162	0.075	0.086	OK
earthworm_richness_z	0.396	0.322	0.074	OK
orib_species_richness_z	0.106	0.032	0.075	OK
meso_species_richness_z	0.150	0.038	0.111	OK
coll_species_richness_z	0.278	0.159	0.119	OK
bac_asv_richness_z	0.187	0.061	0.126	OK
fun_asv_richness_z	0.046	-0.020	0.066	OK
euk_asv_richness_z	0.229	0.127	0.103	OK
oomy_asv_richness_z	0.240	0.161	0.079	OK
cerc_asv_richness_z	0.273	0.195	0.078	OK

5.- Importancia de variables

Se utilizan los dos siguientes enfoques:

- **Permutation importance:** mide cuánto cae el R2 cuando se baraja aleatoriamente cada variable. Es independiente del tipo de modelo.
- **Coefficientes del modelo:** directamente interpretables en Ridge, ya que las variables están normalizadas (escala z).

```
In [6]: # Cálculo de las variables mas relevantes
print("Calculando variables más relevantes...\n")

mas_relevantes_dict = {}
importancias_todas = pd.DataFrame(index=FEATURES_AUTORIZADAS)

for target_col in TARGETS:
    ruta = os.path.join(MODELS_DIR, f'ridge_prod_{target_col}.pkl')
    modelo = joblib.load(ruta)

    coefs_abs = pd.Series(np.abs(modelo.coef_), index=FEATURES_AUTORIZADAS)
    importancias_todas[target_col] = coefs_abs
    mas_relevantes_dict[target_col] = coefs_abs.nlargest(10).index.tolist()

df_mas_relevantes = pd.DataFrame(mas_relevantes_dict)

# Métrica Global
importancia_global = importancias_todas.mean(axis=1)
global_mas_relevantes = importancia_global.nlargest(10).index.tolist()
df_mas_relevantes.insert(0, 'GLOBAL_mas_RELEVANTES', global_mas_relevantes)

# IMPRESIÓN EN TEXTO PLANO
print("Top 10 variables más relevantes a nivel global:")
for i, var in enumerate(global_mas_relevantes, 1):
    print(f"{i:2d}. {var}")

print("\n" + "=" * 50)
print("Top 10 variables más relevantes por target:")
for target_col in TARGETS:
    print(f"\n--- Target: {target_col} ---")
    for i, var in enumerate(mas_relevantes_dict[target_col], 1):
        print(f" {i:2d}. {var}")

# Grafico: barplot importancia de variables (top 10 global, por valor de coeficiente absoluto)
fig, ax = plt.subplots(figsize=(9, 5))
importancia_global.loc[global_mas_relevantes].sort_values().plot(
    kind='barh', ax=ax, color=PALETTE['blue'], edgecolor='white'
)
ax.set_title('Top 10 variables - Regresión (Coeficientes |Ridge|)', fontsize=13)
ax.set_xlabel('Importancia media (|coeficiente|)')
ax.set_ylabel('')
plt.tight_layout()
plt.show()
```

Calculando variables más relevantes...

Top 10 variables más relevantes a nivel global:

1. soil_ph_z
2. gee_temp_media_C_z
3. eu_p_z
4. dem_elevacion_m_z
5. zn_z
6. gee_humedad_rel_pct_z
7. clay_content_z
8. eu_ph_z
9. pb_z
10. soil_moisture_z

=====

Top 10 variables más relevantes por target:

--- Target: nematode_shannon_z ---

1. gee_temp_media_C_z
2. silt_content_z
3. eu_cn_ratio_z
4. gee_humedad_rel_pct_z
5. soil_ph_z
6. aggregate_stability_z
7. k_z
8. total_plant_cover_z
9. eu_p_z
10. eu_zn_z

--- Target: macro_shannon_z ---

1. dem_elevacion_m_z
2. eu_as_z
3. eu_organic_carbon_octop_z
4. eu_clay_content_z
5. soil_moisture_z
6. eu_cn_ratio_z
7. eu_silt_content_z
8. p_z
9. dem_pendiente_deg_z
10. eu_water_holding_capacity_z

--- Target: earthworm_shannon_z ---

1. soil_ph_z
2. eu_p_z
3. silt_content_z
4. zn_z
5. aggregate_stability_z
6. eu_clay_content_z
7. gee_humedad_rel_pct_z
8. clay_content_z
9. p_z
10. soil_moisture_z

--- Target: orib_shannon_z ---

1. soil_ph_z
2. p_z
3. eu_p_z
4. eu_ph_z
5. plot_total_n_z
6. gee_ndvi_verano_z
7. eu_clay_content_z
8. zn_z
9. sand_content_z
10. mo_z

--- Target: meso_shannon_z ---

1. zn_z
2. gee_ndvi_verano_z
3. dem_elevacion_m_z
4. eu_cn_ratio_z
5. pb_z
6. eu_zn_z
7. soil_moisture_z
8. dem_pendiente_deg_z
9. eu_water_holding_capacity_z
10. bulk_density_z

--- Target: coll_shannon_z ---

1. clay_content_z
2. zn_z
3. gee_temp_media_C_z
4. dem_elevacion_m_z
5. eu_zn_z
6. bulk_density_z
7. pb_z
8. total_plant_cover_z
9. eu_as_z
10. eu_cn_ratio_z

--- Target: bac_shannon_z ---

1. gee_temp_media_C_z
2. pb_z
3. eu_organic_carbon_octop_z
4. soil_moisture_z
5. k_z
6. dem_elevacion_m_z
7. aggregate_stability_z
8. dem_pendiente_deg_z
9. plot_total_n_z
10. eu_as_z

```
--- Target: fun_shannon_z ---
1. eu_clay_content_z
2. eu_organic_carbon_octop_z
3. dem_orientacion_deg_z
4. total_plant_cover_z
5. eu_water_holding_capacity_z
6. zn_z
7. ni_z
8. silt_content_z
9. p_z
10. dem_elevacion_m_z

--- Target: euk_shannon_z ---
1. soil_ph_z
2. gee_humedad_rel_pct_z
3. bulk_density_z
4. eu_p_z
5. eu_ph_z
6. soil_moisture_z
7. clay_content_z
8. gee_temp_media_C_z
9. total_plant_cover_z
10. eu_silt_content_z

--- Target: oomy_shannon_z ---
1. gee_temp_media_C_z
2. soil_ph_z
3. p_z
4. silt_content_z
5. eu_ph_z
6. eu_silt_content_z
7. gee_ndvi_verano_z
8. dem_pendiente_deg_z
9. bulk_density_z
10. eu_sand_content_z

--- Target: cerc_shannon_z ---
1. dem_elevacion_m_z
2. gee_temp_media_C_z
3. gee_ndvi_verano_z
4. plot_total_organic_c_z
5. dem_orientacion_deg_z
6. gee_humedad_rel_pct_z
7. mo_z
8. p_z
9. eu_p_z
10. plot_total_n_z

--- Target: macro_order_richness_z ---
1. dem_elevacion_m_z
2. eu_organic_carbon_octop_z
3. eu_as_z
4. eu_clay_content_z
5. total_plant_cover_z
6. eu_silt_content_z
7. eu_water_holding_capacity_z
8. eu_cn_ratio_z
9. soil_moisture_z
10. dem_pendiente_deg_z

--- Target: earthworm_richness_z ---
1. soil_ph_z
2. eu_p_z
3. silt_content_z
4. gee_humedad_rel_pct_z
5. clay_content_z
6. zn_z
7. aggregate_stability_z
8. eu_organic_carbon_octop_z
9. plot_total_organic_c_z
10. eu_clay_content_z

--- Target: orib_species_richness_z ---
1. soil_ph_z
2. eu_ph_z
3. gee_ndvi_verano_z
4. p_z
5. eu_p_z
6. mo_z
7. eu_as_z
8. eu_clay_content_z
9. dem_elevacion_m_z
10. plot_total_n_z

--- Target: meso_species_richness_z ---
1. zn_z
2. gee_ndvi_verano_z
3. dem_elevacion_m_z
4. eu_cn_ratio_z
5. pb_z
6. aggregate_stability_z
7. eu_zn_z
8. soil_moisture_z
9. mo_z
10. eu_water_holding_capacity_z

--- Target: coll_species_richness_z ---
1. zn_z
2. clay_content_z
```

```

3. gee_temp_media_C_z
4. dem_elevacion_m_z
5. total_plant_cover_z
6. pb_z
7. bulk_density_z
8. eu_zn_z
9. eu_as_z
10. eu_cn_ratio_z

--- Target: bac_asv_richness_z ---
1. gee_temp_media_C_z
2. eu_organic_carbon_octop_z
3. eu_as_z
4. pb_z
5. plot_total_n_z
6. aggregate_stability_z
7. clay_content_z
8. eu_water_holding_capacity_z
9. soil_moisture_z
10. dem_pendiente_deg_z

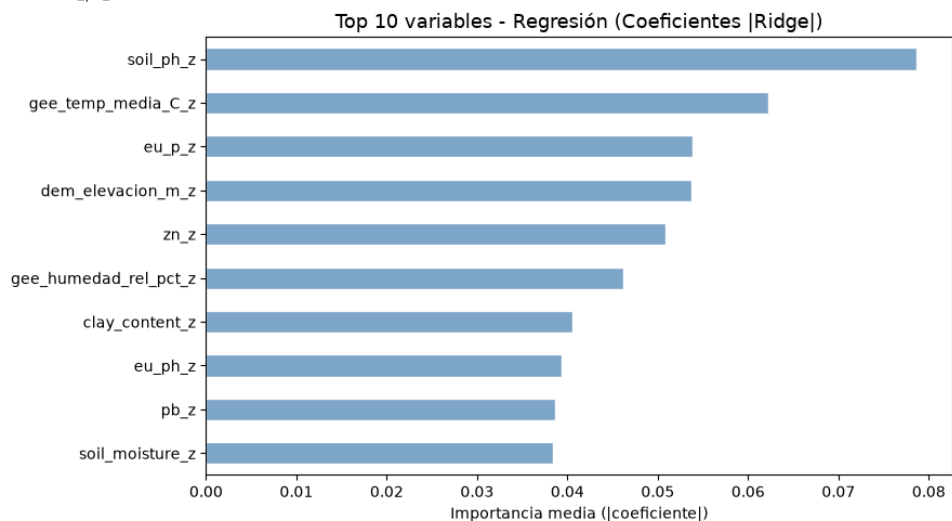
--- Target: fun_asv_richness_z ---
1. eu_as_z
2. soil_ph_z
3. eu_cn_ratio_z
4. clay_content_z
5. gee_temp_media_C_z
6. bulk_density_z
7. total_plant_cover_z
8. eu_clay_content_z
9. eu_zn_z
10. k_z

--- Target: euk_asv_richness_z ---
1. soil_ph_z
2. gee_humedad_rel_pct_z
3. zn_z
4. eu_silt_content_z
5. eu_ph_z
6. p_z
7. bulk_density_z
8. eu_sand_content_z
9. silt_content_z
10. eu_p_z

--- Target: oomy_asv_richness_z ---
1. gee_temp_media_C_z
2. eu_ph_z
3. dem_pendiente_deg_z
4. dem_orientacion_deg_z
5. eu_clay_content_z
6. gee_humedad_rel_pct_z
7. soil_ph_z
8. ni_z
9. p_z
10. soil_moisture_z

--- Target: cerc_asv_richness_z ---
1. dem_elevacion_m_z
2. gee_temp_media_C_z
3. p_z
4. eu_p_z
5. total_plant_cover_z
6. plot_total_organic_c_z
7. mo_z
8. gee_humedad_rel_pct_z
9. plot_total_n_z
10. eu_ph_z

```



6.- Entrenamiento de producción

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

La regresión lineal es determinista (no depende de semillas aleatorias), por lo que se entrena un único modelo por variable objetivo.

Cada modelo se guarda en disco en formato `.pkl`.

```
In [7]: print('Entrenamiento de produccion (un modelo por target con sus propios hiperparametros)... \n')

global_start = time.time()

for target_col in TARGETS:
    y = y_train[target_col].values

    mejores_params = REG_OPTIMAL_PARAMS_PER_TARGET[target_col]
    model = Ridge(**mejores_params, random_state=RANDOM_STATE)
    model.fit(X_train, y)

    ruta = os.path.join(MODELS_DIR, f'ridge_prod_{target_col}.pkl')
    joblib.dump(model, ruta)

print(f'Produccion completada. Modelos guardados: {len(TARGETS)}')
```

Entrenamiento de produccion (un modelo por target con sus propios hiperparametros)...

Produccion completada. Modelos guardados: 21

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretación del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biológica).

```
In [8]: # Evaluacion final sobre todos los targets

print('Evaluacion final sobre eval.csv\n')

print(f' {"Target":30} {"R2":>8} {"RMSE":>8} {"MAE":>8}')
print('-' * 65)

for test_target in TARGETS:
    ruta = os.path.join(MODELS_DIR, f'ridge_prod_{test_target}.pkl')
    modelo_eval = joblib.load(ruta)
    y_pred_eval = modelo_eval.predict(X_eval)
    y_true_eval = y_eval[test_target].values

    r2 = r2_score(y_true_eval, y_pred_eval)
    rmse = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval))
    mae = mean_absolute_error(y_true_eval, y_pred_eval)
    print(f' {test_target:30} {r2:8.4f} {rmse:8.4f} {mae:8.4f}')
```

print('-' * 65)

Smoke test con datos reales
Se prueban dos targets representativos: uno shannon y uno richness

print('\nSmoke test - Inferencia con datos reales')

print('-' * 65)

try:

df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
indices_elegidos = df_new_data.index

for test_target in ['earthworm_shannon_z', 'earthworm_richness_z']:

print(f'\n Target: {test_target}')

print(' ' + '-' * 63)

valores_reales = y_eval.loc[indices_elegidos, test_target].values

ruta = os.path.join(MODELS_DIR, f'ridge_prod_{test_target}.pkl')

modelo_eval = joblib.load(ruta)

pred = modelo_eval.predict(df_new_data)

for i, idx in enumerate(indices_elegidos):

real_val = valores_reales[i]

print(f' Fila {idx:<4} | Prediccion: {pred[i]:.4f} | ')

f'Valor Real: {real_val:.4f} | Error: {abs(pred[i]-real_val):.4f}')

print('\nSmoke test superado. El pipeline funciona correctamente...')

except Exception as e:

print(f'Error en el smoke test: {str(e)}')

Grafico: scatter prediccion vs valor real

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, tgt in zip(axes, ['earthworm_shannon_z', 'earthworm_richness_z']):

m = joblib.load(os.path.join(MODELS_DIR, f'ridge_prod_{tgt}.pkl'))

yp = m.predict(X_eval)

yt = y_eval[tgt].values

ax.scatter(yt, yp, alpha=0.6, s=25, color=PALETTE['blue'])

lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]

ax.plot(lim, lim, 'r--', linewidth=1)

ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)

ax.set_xlabel('Valor real', fontsize=8)

ax.set_ylabel('Prediccion', fontsize=8)

plt.suptitle('Prediccion vs Valor Real - Regresión (eval.csv)', fontsize=12)

plt.tight_layout()

plt.show()

Evaluacion final sobre eval.csv

Target	R2	RMSE	MAE
nematode_shannon_z	0.0112	0.9821	0.7929
macro_shannon_z	0.1257	0.9391	0.8177
earthworm_shannon_z	0.2395	0.8704	0.7094
orib_shannon_z	0.1137	1.0766	0.9075
meso_shannon_z	-0.2499	1.0918	0.9023
coll_shannon_z	-0.4998	0.7758	0.6426
bac_shannon_z	-0.0812	1.0413	0.7158
fun_shannon_z	-0.0161	1.1533	0.9503
euk_shannon_z	0.0407	0.9499	0.7011
oomy_shannon_z	0.0857	0.9996	0.7418
cerc_shannon_z	0.0155	0.8886	0.6332
macro_order_richness_z	0.1521	0.8433	0.7105
earthworm_richness_z	0.2789	0.7754	0.6143
orib_species_richness_z	0.0921	1.1400	0.7670
meso_species_richness_z	-0.0959	1.0097	0.7693
coll_species_richness_z	-0.7341	0.6780	0.5409
bac_asv_richness_z	-0.0761	1.1398	0.8335
fun_asv_richness_z	0.0087	1.0731	0.8114
euk_asv_richness_z	0.0400	0.7955	0.5775
oomy_asv_richness_z	0.1035	0.9510	0.7711
cerc_asv_richness_z	0.1019	0.9350	0.7211

Smoke test - Inferencia con datos reales

Target: earthworm_shannon_z

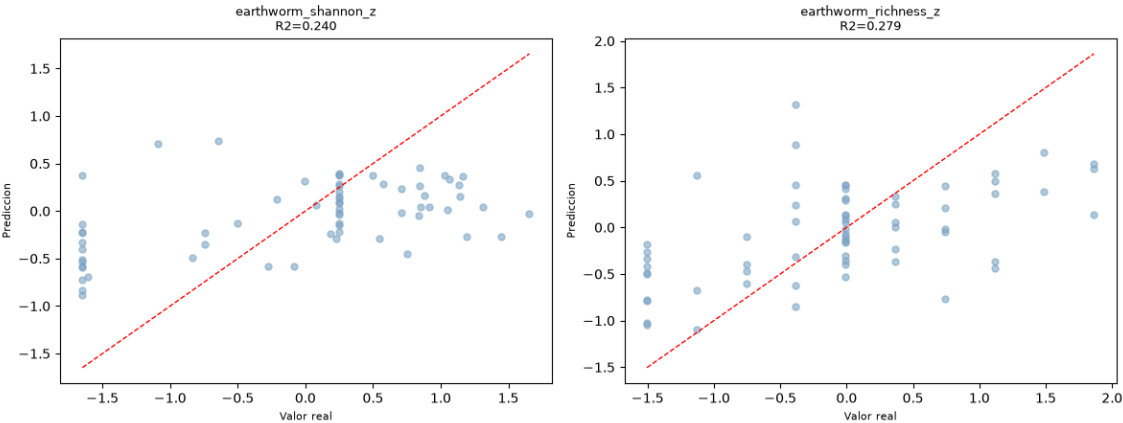
Fila 53 | Prediccion: 0.2759 | Valor Real: 1.1347 | Error: 0.8588
Fila 60 | Prediccion: 0.1580 | Valor Real: 1.1388 | Error: 0.9808
Fila 0 | Prediccion: -0.3476 | Valor Real: -0.7453 | Error: 0.3976

Target: earthworm_richness_z

Fila 53 | Prediccion: 0.3828 | Valor Real: 1.4889 | Error: 1.1060
Fila 60 | Prediccion: 0.3665 | Valor Real: 1.1147 | Error: 0.7482
Fila 0 | Prediccion: -0.4664 | Valor Real: -0.7562 | Error: 0.2899

Smoke test superado. El pipeline funciona correctamente...

Prediccion vs Valor Real - Regresión (eval.csv)



11.2.2. Modelo Random Forest

11.2.2.1. Fundamento y configuración

Random Forest (véase **Modelo Random Forest**) se entrena, en esta primera variante, en su forma de salida única: un conjunto de árboles independiente por cada uno de los 21 targets, cada uno con su propia búsqueda de hiperparámetros.

La búsqueda se realiza mediante RandomizedSearchCV (50 combinaciones, validación cruzada de 5 pliegues) sobre X_{train} , explorando:

- **n_estimators:** 100, 150, 200.
- **max_depth:** 3, 4, 5, 10, 15, 20.
- **min_samples_leaf:** 2, 4, 8, 12, 16.
- **min_samples_split:** 15, 20, 25.
- **max_features:** sqrt, 0.2, 0.3.
- **ccp_alpha:** 0.005, 0.01, 0.02.
- **bootstrap:** True.

Algunos de dichos parámetros se han capado a un mínimo o máximo específico por los siguientes motivos:

1. **max_depth:** Limitado a un máximo de 20 para reducir el riesgo de sobreajuste a causa del tamaño reducido de los datos.
2. **min_samples_leaf:** Limitado a un mínimo de 2 hojas para evitar la creación de hojas triviales.

11.2.2.2. Entrenamiento

La validación cruzada repetida sobre X_{train} muestra señales de sobreajuste (diferencia Train-CV > 0.15) en la práctica totalidad de los 21 targets, algo esperable dada la combinación de un modelo con alta capacidad (max_depth hasta 20) y un conjunto de entrenamiento reducido (300 muestras).

Este comportamiento es muy distinto al observado en Ridge, y es coherente con la mayor capacidad de un ensamblado de árboles para memorizar el conjunto de entrenamiento frente a un modelo lineal.

Además, a diferencia de Ridge, Random Forest sí depende de la semilla aleatoria (tanto en el *bootstrap* de las muestras como en la selección aleatoria de variables en cada división). Para reducir la varianza de las predicciones finales, se entrena un ensamblado de 5 modelos por target (semillas RANDOM_STATE, RANDOM_STATE+1, ..., RANDOM_STATE+4), cuyas predicciones se promedian en el momento de la inferencia.

En total se generan y almacenan $21 \times 5 = 105$ **modelos .pkl**.

11.2.2.3. Explicabilidad y variables más relevantes

Se calculan valores **SHAP** sobre X_{train} para todos los targets conjuntamente.

El top-10 de variables más relevantes a nivel global por importancia SHAP media es: soil_ph_z, gee_temp_media_C_z, eu_ph_z, eu_as_z, eu_cn_ratio_z, gee_humedad_rel_pct_z, eu_p_z, eu_clay_content_z, silt_content_z y eu_silt_content_z.

Este resultado es consistente con el obtenido por los coeficientes de Ridge (soil_ph_z y gee_temp_media_C_z en primer y segundo lugar en ambos casos), lo que refuerza la fiabilidad de ambas variables como predictores robustos, independientemente del tipo de modelo empleado.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Explicabilidad del modelo (SHAP)**.

11.2.2.4. Resultados sobre eval.csv

El R^2 medio sobre los 21 targets es de **0.090**, una mejora sustancial frente al -0.016 de Ridge.

Random Forest es, de hecho, el modelo de salida única con mejor rendimiento global de todo el trabajo.

Sobre los targets prioritarios: $R^2=0.5041$ (earthworm_shannon_z) y $R^2=0.5722$ (earthworm_richness_z), los mejores resultados obtenidos para ambos targets entre todos los modelos de salida única evaluados.

Por otro lado, coll_species_richness_z (-0.5484) vuelve a ser, como en Ridge, el target con peor ajuste.

Variable	Target	R^2	RMSE	MAE
Shannon nemátodos	nematode_shannon_z	0.1693	0.9001	0.7070
Shannon macrofauna	macro_shannon_z	0.3199	0.8283	0.6935
Shannon lombrices	earthworm_shannon_z	0.5041	0.7029	0.5055
Shannon oribátidos	orib_shannon_z	0.1794	1.0359	0.8803
Shannon mesostigmátidos	meso_shannon_z	-0.1731	1.0577	0.8816
Shannon colémbolos	coll_shannon_z	-0.3489	0.7358	0.6129
Shannon bacterias	bac_shannon_z	0.0077	0.9975	0.6701
Shannon hongos	fun_shannon_z	-0.0455	1.1698	0.9399
Shannon eucariotas	euk_shannon_z	0.1683	0.8844	0.6227
Shannon oomicetos	oomy_shannon_z	0.0832	1.0010	0.7279
Shannon cercozoos	cerc_shannon_z	-0.0318	0.9096	0.6274
Riqueza macrofauna	macro_order_richness_z	0.3417	0.7431	0.6316
Riqueza lombrices	earthworm_richness_z	0.5722	0.5973	0.4324
Riqueza oribátidos	orib_species_richness_z	0.2034	1.0679	0.7166
Riqueza mesostigmátidos	meso_species_richness_z	-0.0438	0.9854	0.7472
Riqueza colémbolos	coll_species_richness_z	-0.5484	0.6407	0.5033
Riqueza bacterias (ASV)	bac_asv_richness_z	0.0042	1.0965	0.8126
Riqueza hongos (ASV)	fun_asv_richness_z	-0.0256	1.0915	0.8189
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.2520	0.7022	0.5142
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.1843	0.9071	0.7446
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.1250	0.9229	0.7550

Tabla 13: Resultados de Random Forest (salida única) sobre eval.csv, por target.

SOB4ES - Modelo Random Forest

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook recorre el ciclo de vida completo del modelo Random Forest.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning:** búsqueda del valor óptimo de los hiperparámetros.
4. **Validación cruzada:** evaluación de robustez.
5. **Explicabilidad (SHAP):** análisis de importancia de variables.
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import math
import pandas as pd
import numpy as np
from pathlib import Path

import itertools
from matplotlib.colors import LinearSegmentedColormap

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV, cross_validate, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

import shap
import matplotlib.pyplot as plt

# ignoramos los warnings
warnings.filterwarnings('ignore')
os.environ["PYTHONWARNINGS"] = "ignore"

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/rf/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

TARGETS = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',

```

```

'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}
# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas correctamente...

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos 'reserva' nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```

In [3]: def cargar_csv(ruta, nombre):
    """Carga un CSV (sep=', ' o ';') y valida las columnas necesarias."""
    if not os.path.exists(ruta):
        raise FileNotFoundError(f"No se encontro el archivo: {ruta}")
    try:
        df = pd.read_csv(ruta, sep=',')
        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
    except Exception:
        df = pd.read_csv(ruta, sep=';')
    print(f" {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas")
    return df

```

```

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados.")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

```

Datasets cargados.

```

X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)

```

3.- Búsqueda de hiperparámetros (Tuning)

Se realiza una búsqueda aleatoria (`RandomizedSearchCV`) sobre el espacio de hiperparámetros de `RandomForestRegressor`. La búsqueda se ejecuta exclusivamente sobre `X_train`. Como variable objetivo de referencia se usa `earthworm_shannon_z`.

Principales hiperparametros explorados:

- `n_estimators` : número de árboles en el bosque.
- `max_depth` : profundidad máxima de cada árbol.
- `min_samples_split` / `min_samples_leaf` : criterios de parada para la división de nodos.
- `max_features` : número de variables consideradas en cada split.
- `bootstrap` : si se usa muestreo con reemplazamiento.

Para reducir el riesgo de sobreajuste con un dataset pequeño (~300 muestras), el grid se restringe de la siguiente forma:

- `max_depth` : maximo 20.
- `min_samples_leaf` : minimo 2.
- `bootstrap=True` : para mayor variabilidad entre árboles.

```

In [4]: print('Iniciando busqueda de hiperparametros (Tuning) para cada target...')
print('Este proceso puede tardar varios minutos...')
param_grid = {
    'n_estimators': [100, 150, 200],
    'max_depth': [3, 4, 5, 10, 15, 20],
    'min_samples_leaf': [2, 4, 8, 12, 16],
    'min_samples_split': [15, 20, 25],
    'max_features': ['sqrt', 0.2, 0.3],
    'ccp_alpha': [0.005, 0.01, 0.02],
    'bootstrap': [True],
    'random_state': [RANDOM_STATE]
}

resultados_tuning = {}
RF_OPTIMAL_PARAMS_PER_TARGET = {}
start_total = time.time()

for idx, target_col in enumerate(TARGETS, 1):
    y_prueba = y_train[target_col].values
    t0 = time.time()

    buscador = RandomizedSearchCV(
        estimator=RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1),
        param_distributions=param_grid,
        n_iter=50,
        scoring='r2',
        cv=5,
        verbose=0,
        random_state=RANDOM_STATE,
        n_jobs=N_JOBS
    )
    buscador.fit(X_train, y_prueba)
    elapsed = time.time() - t0

    mejores_params = dict(buscador.best_params_)
    mejores_params['n_jobs'] = -1

    resultados_tuning[target_col] = {
        'params': mejores_params,
        'r2': buscador.best_score_,
    }
    RF_OPTIMAL_PARAMS_PER_TARGET[target_col] = mejores_params

    print(f"[{idx:2d}/{len(TARGETS)}] R2 CV: {buscador.best_score_:0.4f} | Tiempo: {time.time()-t0:>4.1f}s")

print(f'\nBusqueda de hiperparametros completada en {(time.time() - start_total)/60:.1f} minutos...')

# Grafico: mejor R2 de tuning obtenido para cada uno de los 21 targets
r2_tuning_por_target = pd.Series({t: v['r2'] for t, v in resultados_tuning.items()}).sort_values()

```

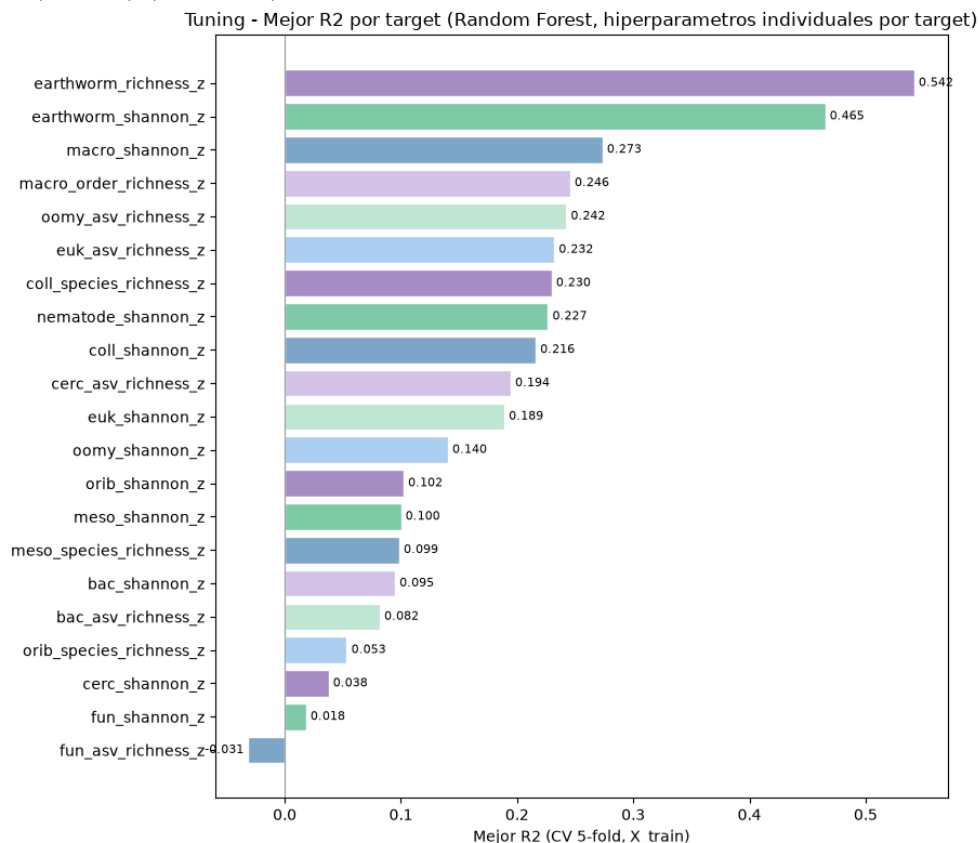
```
fig, ax = plt.subplots(figsize=(9, 8))
_bars = ax.barh(r2_tuning_por_target.index, r2_tuning_por_target.values,
                color=palette_cycle(len(r2_tuning_por_target)), edgecolor='white')
ax.bar_label(_bars, fmt='{:.3f}', padding=3, fontsize=8)
ax.set_xlabel('Mejor R2 (CV 5-fold, X_train)')
ax.set_title('Tuning - Mejor R2 por target (Random Forest, hiperparametros individuales por target)', fontsize=12)
ax.axvline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()
```

Iniciando búsqueda de hiperparametros (Tuning) para cada target...

Este proceso puede tardar varios minutos...

[1/21]	[nematode_shannon_z]	R2 CV: 0.2266	Tiempo: 4.7s
[2/21]	[macro_shannon_z]	R2 CV: 0.2735	Tiempo: 3.4s
[3/21]	[earthworm_shannon_z]	R2 CV: 0.4652	Tiempo: 3.3s
[4/21]	[orib_shannon_z]	R2 CV: 0.1023	Tiempo: 3.3s
[5/21]	[meso_shannon_z]	R2 CV: 0.1000	Tiempo: 3.3s
[6/21]	[coll_shannon_z]	R2 CV: 0.2160	Tiempo: 3.3s
[7/21]	[bac_shannon_z]	R2 CV: 0.0949	Tiempo: 3.4s
[8/21]	[fun_shannon_z]	R2 CV: 0.0183	Tiempo: 3.3s
[9/21]	[euk_shannon_z]	R2 CV: 0.1890	Tiempo: 3.3s
[10/21]	[oomy_shannon_z]	R2 CV: 0.1404	Tiempo: 3.4s
[11/21]	[cerc_shannon_z]	R2 CV: 0.0377	Tiempo: 3.4s
[12/21]	[macro_order_richness_z]	R2 CV: 0.2456	Tiempo: 3.4s
[13/21]	[earthworm_richness_z]	R2 CV: 0.5418	Tiempo: 3.4s
[14/21]	[orib_species_richness_z]	R2 CV: 0.0534	Tiempo: 3.3s
[15/21]	[meso_species_richness_z]	R2 CV: 0.0985	Tiempo: 3.4s
[16/21]	[coll_species_richness_z]	R2 CV: 0.2297	Tiempo: 3.4s
[17/21]	[bac_asv_richness_z]	R2 CV: 0.0818	Tiempo: 3.5s
[18/21]	[fun_asv_richness_z]	R2 CV: -0.0313	Tiempo: 3.5s
[19/21]	[euk_asv_richness_z]	R2 CV: 0.2320	Tiempo: 3.5s
[20/21]	[oomy_asv_richness_z]	R2 CV: 0.2423	Tiempo: 3.5s
[21/21]	[cerc_asv_richness_z]	R2 CV: 0.1944	Tiempo: 3.5s

Busqueda de hiperparametros completada en 1.2 minutos...



4.- Validación cruzada

Se evalúa la robustez del modelo mediante **validación cruzada repetida** (5 pliegues x 3 repeticiones = 15 evaluaciones) sobre `X_train`.

Comparando el rendimiento en entrenamiento y en validación se puede detectar sobreajuste, algo especialmente relevante en Random Forest dado que puede memorizar los datos de entrenamiento con facilidad.

```
In [5]: print('Validacion cruzada repetida sobre X_train...\n')
cv_splitter = RepeatedKfold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
metricas = ['r2', 'neg_root_mean_squared_error', 'neg_mean_absolute_error']

resultados_cv_por_target = {}

# Impresión del encabezado de la tabla
header = f"{'TARGET':<26} | {'TRAIN R2':<10} | {'CV R2':<10} | {'DIFF':<10} | {'ESTADO'}"
```

```

print(header)
print('-' * len(header))

for target_col in TARGETS:
    y_prueba = y_train[target_col].values

    modelo_cv = RandomForestRegressor(
        **RF_OPTIMAL_PARAMS_PER_TARGET[target_col],
    )

    resultados = cross_validate(
        estimator=modelo_cv,
        X=X_train,
        y=y_prueba,
        cv=cv_splitter,
        scoring=metricas,
        return_train_score=True,
        n_jobs=N_JOBS
    )

    r2_train = np.mean(resultados['train_r2'])
    r2_cv = np.mean(resultados['test_r2'])
    diferencia = r2_train - r2_cv

    resultados_cv_por_target[target_col] = {'train_r2': r2_train, 'cv_r2': r2_cv}

    estado = "Aviso: Posible sobreajuste" if diferencia > 0.15 else "OK"

    print(f"{target_col:<26} | {r2_train:<10.3f} | {r2_cv:<10.3f} | {diferencia:<10.3f} | {estado}")

```

Validación cruzada repetida sobre X_train...

TARGET	TRAIN R2	CV R2	DIFF	ESTADO
nematode_shannon_z	0.539	0.261	0.278	Aviso: Posible sobreajuste
macro_shannon_z	0.611	0.282	0.329	Aviso: Posible sobreajuste
earthworm_shannon_z	0.694	0.434	0.261	Aviso: Posible sobreajuste
orib_shannon_z	0.441	0.106	0.335	Aviso: Posible sobreajuste
meso_shannon_z	0.374	0.090	0.284	Aviso: Posible sobreajuste
coll_shannon_z	0.572	0.190	0.383	Aviso: Posible sobreajuste
bac_shannon_z	0.327	0.037	0.290	Aviso: Posible sobreajuste
fun_shannon_z	0.471	0.035	0.435	Aviso: Posible sobreajuste
euk_shannon_z	0.536	0.168	0.369	Aviso: Posible sobreajuste
oomy_shannon_z	0.515	0.115	0.400	Aviso: Posible sobreajuste
cerc_shannon_z	0.345	0.023	0.322	Aviso: Posible sobreajuste
macro_order_richness_z	0.597	0.254	0.343	Aviso: Posible sobreajuste
earthworm_richness_z	0.772	0.543	0.228	Aviso: Posible sobreajuste
orib_species_richness_z	0.416	0.091	0.325	Aviso: Posible sobreajuste
meso_species_richness_z	0.413	0.089	0.325	Aviso: Posible sobreajuste
coll_species_richness_z	0.555	0.194	0.361	Aviso: Posible sobreajuste
bac_asv_richness_z	0.403	0.048	0.355	Aviso: Posible sobreajuste
fun_asv_richness_z	0.251	-0.022	0.274	Aviso: Posible sobreajuste
euk_asv_richness_z	0.576	0.219	0.358	Aviso: Posible sobreajuste
oomy_asv_richness_z	0.579	0.248	0.331	Aviso: Posible sobreajuste
cerc_asv_richness_z	0.519	0.233	0.286	Aviso: Posible sobreajuste

5.- Explicabilidad del modelo (SHAP)

Se utiliza SHAP (`TreeExplainer`) para analizar la importancia de las variables predictoras. `TreeExplainer` es compatible con `RandomForestRegressor` y calcula los valores de Shapley de forma exacta.

Interpretación del gráfico:

- Las variables se ordenan de mayor a menor importancia global.
- El color indica el valor de la variable (rojo = valor alto, azul = valor bajo).
- La posición horizontal indica el efecto sobre la predicción.

Posteriormente, se agrupan los datos de SHAP para obtener las variables más relevantes ordenadas de más relevantes a menos dentro de un top 10.

```

In [6]: print('Calculando valores SHAP sobre X_train para TODOS los targets...\n')

all_shap_means = []
n_targets = len(TARGETS)
n_cols = 3
n_rows = math.ceil(n_targets / n_cols)

# Crear el grid de subplots (5 columnas x N filas)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(25, 4.5 * n_rows))
axes_flat = axes.flatten()

for i, tgt in enumerate(TARGETS):
    # Modelo específico para cada target
    modelo_shap = RandomForestRegressor(
        **RF_OPTIMAL_PARAMS_PER_TARGET[tgt],
    )
    modelo_shap.fit(X_train, y_train[tgt].values)

    # Cálculo de SHAP
    explainer = shap.TreeExplainer(modelo_shap)
    shap_values = explainer.shap_values(X_train)

    # Renderizado en el subplot asignado dentro del grid
    plt.sca(axes_flat[i])
    shap.summary_plot(
        shap_values,
        X_train,
        plot_type='dot',

```

```
        max_display=10,
        show=False,
        plot_size=None
    )
    axes_flat[i].set_title(tgt, fontsize=11, fontweight='bold')

    # Acumular la importancia absoluta media
    mean_abs_tgt = np.abs(shap_values).mean(axis=0)
    all_shap_means.append(mean_abs_tgt)

# Ocultar los subplots sobrantes en el grid
for j in range(n_targets, len(axes_flat)):
    fig.delaxes(axes_flat[j])

plt.tight_layout()
plt.show()

# Promedio global integrando todos los targets
global_shap_mean = np.mean(all_shap_means, axis=0)
shap_importance_series = pd.Series(global_shap_mean, index=FEATURES_AUTORIZADAS)

# Extraer Top 10 variables más y menos importantes a nivel global
top10_shap = shap_importance_series.nlargest(10)

print('\nTop 10 variables MÁS importantes por SHAP (Promedio Global de Targets):')
print(top10_shap.round(4).to_string())

# Gráficos comparativos globales de barras
fig, axes_bar = plt.subplots(1, 1, figsize=(16, 5))

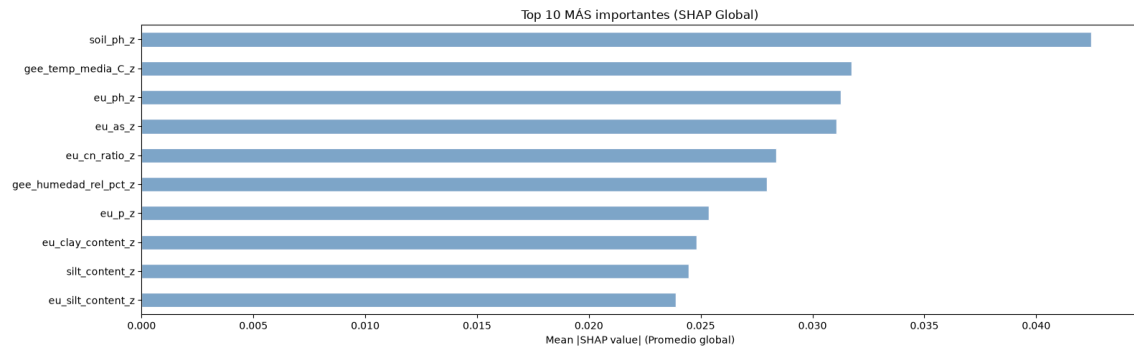
# Gráfico A: Más importantes
top10_shap.sort_values().plot(kind='barh', ax=axes_bar, color=PALETTE['blue'], edgecolor='white')
axes_bar.set_title('Top 10 MÁS importantes (SHAP Global)', fontsize=12)
axes_bar.set_xlabel('Mean |SHAP value| (Promedio global)')
axes_bar.set_ylabel('')

plt.tight_layout()
plt.show()
```

Calculando valores SHAP sobre X_train para TODOS los targets...

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo





6.- Entrenamiento

Se entrena un ensamble de 5 modelos por cada variable objetivo, variando la semilla aleatoria (semilla + n_modelo).

Esto complementa la aleatoriedad intrínseca de Random Forest y reduce la varianza de las predicciones finales.

Todos los modelos se guardan en disco en formato `.pkl`.

```
In [7]: print('Entrenamiento de produccion (ensamble de 5 modelos por target)...')
print('-' * 70)

global_start = time.time()
NUM_MODELOS = 5

for idx, target_col in enumerate(TARGETS, 1):
    t0 = time.time()
    y = y_train[target_col].values

    for seed_id in range(NUM_MODELOS):
        params = RF_OPTIMAL_PARAMS_PER_TARGET[target_col].copy()
        params['random_state'] = RANDOM_STATE + seed_id

        model = RandomForestRegressor(**params)
        model.fit(X_train, y)

        joblib.dump(model, os.path.join(MODELS_DIR, f'rf_prod_{target_col}_m{seed_id}.pkl'))

    print(f' [{idx}/{len(TARGETS)}] {target_col:30} completado ({time.time()-t0:.1f}s)')

print('-' * 70)
print(f'Produccion completada en {(time.time() - global_start)/60:.1f} minutos...')
print(f'Modelos guardados: {len(TARGETS) * NUM_MODELOS} ({NUM_MODELOS} por target)...')
print('Cada target fue entrenado con su propia configuración de hiperparametros...')
```

Entrenamiento de produccion (ensamble de 5 modelos por target)...

```
[1/21] nematode_shannon_z      completado (0.6s)
[2/21] macro_shannon_z         completado (0.6s)
[3/21] earthworm_shannon_z     completado (0.6s)
[4/21] orib_shannon_z          completado (0.4s)
[5/21] meso_shannon_z          completado (0.8s)
[6/21] coll_shannon_z          completado (0.6s)
[7/21] bac_shannon_z           completado (0.8s)
[8/21] fun_shannon_z           completado (0.6s)
[9/21] euk_shannon_z           completado (0.4s)
[10/21] oomy_shannon_z         completado (0.4s)
[11/21] cerc_shannon_z         completado (0.8s)
[12/21] macro_order_richness_z completado (0.6s)
[13/21] earthworm_richness_z   completado (0.6s)
[14/21] orib_species_richness_z completado (0.4s)
[15/21] meso_species_richness_z completado (0.8s)
[16/21] coll_species_richness_z completado (0.6s)
[17/21] bac_asv_richness_z     completado (0.6s)
[18/21] fun_asv_richness_z     completado (0.5s)
[19/21] euk_asv_richness_z     completado (0.4s)
[20/21] oomy_asv_richness_z    completado (0.4s)
[21/21] cerc_asv_richness_z    completado (0.8s)
```

Produccion completada en 0.2 minutos...

Modelos guardados: 105 (5 por target)...

Cada target fue entrenado con su propia configuración de hiperparametros...

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretacion del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biológica).

```
In [8]: # Evaluacion final sobre todos los targets

print('Evaluacion final sobre eval.csv')
print('-' * 65)
print(f' {"Target":30} {"R2">8} {"RMSE">8} {"MAE">8}')
```

```

print('-' * 65)

for test_target in TARGETS:
    preds_eval = []
    for seed_id in range(5):
        ruta = os.path.join(MODELS_DIR, f'rf_prod_{test_target}_m{seed_id}.pkl')
        model = joblib.load(ruta)
        preds_eval.append(model.predict(X_eval))

    y_pred_eval = np.mean(preds_eval, axis=0)
    y_true_eval = y_eval[test_target].values

    r2 = r2_score(y_true_eval, y_pred_eval)
    rmse = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval))
    mae = mean_absolute_error(y_true_eval, y_pred_eval)
    print(f' {test_target:30} {r2:8.4f} {rmse:8.4f} {mae:8.4f}')

print('-' * 65)

# Smoke test con datos reales
# Se prueban los targets representativos

print('\nSmoke test - Inferencia con datos reales')
print('-' * 65)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    for test_target in ['earthworm_shannon_z', 'earthworm_richness_z']:
        print(f'\n Target: {test_target}')
        print(' ' + '-' * 63)
        valores_reales = y_eval.loc[indices_elegidos, test_target].values

        votos = []
        for seed_id in range(5):
            ruta = os.path.join(MODELS_DIR, f'rf_prod_{test_target}_m{seed_id}.pkl')
            m = joblib.load(ruta)
            pred = m.predict(df_new_data)
            votos.append(pred)
            print(f' Modelo {seed_id+1} → '
                  f'Fila {indices_elegidos[0]}: {pred[0]:.4f} | '
                  f'Fila {indices_elegidos[1]}: {pred[1]:.4f} | '
                  f'Fila {indices_elegidos[2]}: {pred[2]:.4f}')

        consenso = np.mean(votos, axis=0)
        print(' ' + '-' * 63)
        print(' Resultados detallados:')
        print(' ' + '-' * 63)
        for i, idx in enumerate(indices_elegidos):
            pred_val = consenso[i]
            real_val = valores_reales[i]
            print(f' Fila {idx:<4} | Prediccion: {pred_val:.4f} | '
                  f'Valor Real: {real_val:.4f} | Error: {abs(pred_val-real_val):.4f}')

        print('\nSmoke test superado. El pipeline funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: scatter prediccion vs valor real
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, tgt in zip(axes, ['earthworm_shannon_z', 'earthworm_richness_z']):
    preds = [joblib.load(os.path.join(MODELS_DIR, f'rf_prod_{tgt}_m{i}.pkl')).predict(X_eval)
              for i in range(NUM_MODELOS)]
    yp = np.mean(preds, axis=0)
    yt = y_eval[tgt].values
    ax.scatter(yt, yp, alpha=0.6, s=25, color=PALETTE['blue'])
    lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]
    ax.plot(lim, lim, 'r--', linewidth=1)
    ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)
    ax.set_xlabel('Valor real', fontsize=8)
    ax.set_ylabel('Prediccion', fontsize=8)
plt.suptitle('Prediccion vs Valor Real - Random Forest (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()

```

Evaluación final sobre eval.csv

Target	R2	RMSE	MAE
nematode_shannon_z	0.1693	0.9001	0.7070
macro_shannon_z	0.3199	0.8283	0.6935
earthworm_shannon_z	0.5041	0.7029	0.5055
orib_shannon_z	0.1794	1.0359	0.8803
meso_shannon_z	-0.1731	1.0577	0.8816
coll_shannon_z	-0.3489	0.7358	0.6129
bac_shannon_z	0.0077	0.9975	0.6701
fun_shannon_z	-0.0455	1.1698	0.9399
euk_shannon_z	0.1683	0.8844	0.6227
oomy_shannon_z	0.0832	1.0010	0.7279
cerc_shannon_z	-0.0318	0.9096	0.6274
macro_order_richness_z	0.3417	0.7431	0.6316
earthworm_richness_z	0.5722	0.5973	0.4324
orib_species_richness_z	0.2034	1.0679	0.7166
meso_species_richness_z	-0.0438	0.9854	0.7472
coll_species_richness_z	-0.5484	0.6407	0.5033
bac_asv_richness_z	0.0042	1.0965	0.8126
fun_asv_richness_z	-0.0256	1.0915	0.8189
euk_asv_richness_z	0.2520	0.7022	0.5142
oomy_asv_richness_z	0.1843	0.9071	0.7446

cerc_asv_richness_z 0.1250 0.9229 0.7550

Smoke test - Inferencia con datos reales

Target: earthworm_shannon_z

Modelo 1 → Fila 53: 0.5283 | Fila 60: 0.8817 | Fila 0: -0.4357
Modelo 2 → Fila 53: 0.5831 | Fila 60: 0.8579 | Fila 0: -0.4620
Modelo 3 → Fila 53: 0.5598 | Fila 60: 0.9005 | Fila 0: -0.5326
Modelo 4 → Fila 53: 0.4980 | Fila 60: 0.8424 | Fila 0: -0.4882
Modelo 5 → Fila 53: 0.5542 | Fila 60: 0.8902 | Fila 0: -0.4091

Resultados detallados:

Fila 53	Prediccion: 0.5447	Valor Real: 1.1347	Error: 0.5900
Fila 60	Prediccion: 0.8745	Valor Real: 1.1388	Error: 0.2643
Fila 0	Prediccion: -0.4655	Valor Real: -0.7453	Error: 0.2797

Target: earthworm_richness_z

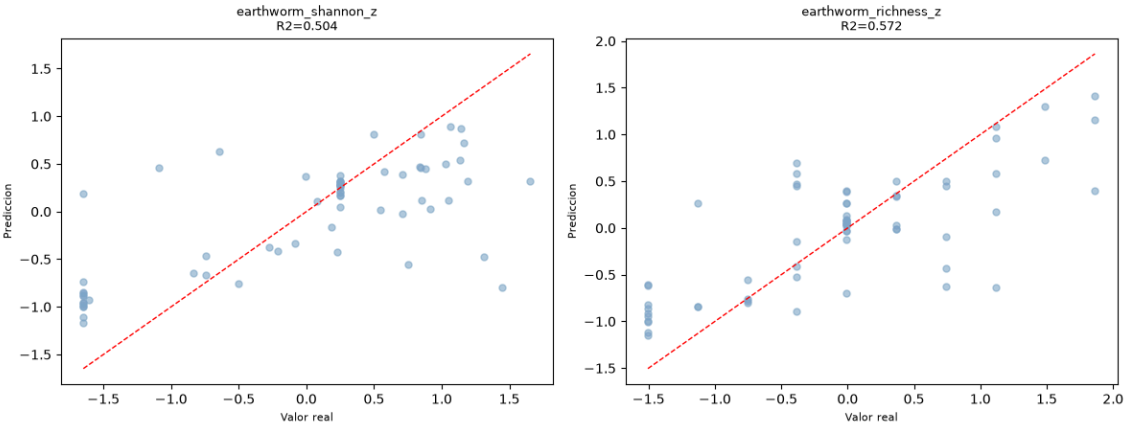
Modelo 1 → Fila 53: 0.7565 | Fila 60: 1.0116 | Fila 0: -0.5675
Modelo 2 → Fila 53: 0.7771 | Fila 60: 1.0948 | Fila 0: -0.5450
Modelo 3 → Fila 53: 0.6814 | Fila 60: 1.1291 | Fila 0: -0.6300
Modelo 4 → Fila 53: 0.6546 | Fila 60: 1.0601 | Fila 0: -0.4509
Modelo 5 → Fila 53: 0.7429 | Fila 60: 1.1116 | Fila 0: -0.5564

Resultados detallados:

Fila 53	Prediccion: 0.7225	Valor Real: 1.4889	Error: 0.7664
Fila 60	Prediccion: 1.0815	Valor Real: 1.1147	Error: 0.0332
Fila 0	Prediccion: -0.5500	Valor Real: -0.7562	Error: 0.2063

Smoke test superado. El pipeline funciona correctamente.

Prediccion vs Valor Real - Random Forest (eval.csv)



11.2.3. Modelo Random Forest Multisalida

11.2.3.1. Fundamento y configuración

Esta variante entrena un único RandomForestRegressor que predice los 21 targets de forma simultánea, aprovechando el soporte nativo de `scikit-learn` para y multivariante: cada división de cada árbol se decide considerando conjuntamente los 21 targets, lo que permite capturar correlaciones entre grupos biológicos directamente en la estructura del árbol, sin necesidad de un mecanismo explícito de encadenamiento.

Al no poder tener hiperparámetros distintos por target (un único bosque sirve a los 21 a la vez), la búsqueda de hiperparámetros se realiza igualmente por target mediante `RandomizedSearchCV`, pero el resultado final se resuelve mediante un consenso ponderado por R^2 : cada target «vota» su combinación óptima con un peso proporcional a su propio R^2 de validación cruzada, esto implica que los targets con mayor R^2 influirán más que los que tenga un valor más pequeño.

El consenso resultante se puede ver en la siguientes tabla:

Hiperparámetro	Valor de consenso
n_estimators	292
max_depth	6
min_samples_split	14
min_samples_leaf	5
max_features	log2
ccp_alpha	≈ 0.00125

Tabla 14: Consenso de hiperparámetros en Random Forest multisalida.

Resulta interesante que el consenso limite la profundidad a solo 6 niveles (frente al máximo de 20 permitido en la búsqueda), lo que sugiere que, al tener que servir a los 21 targets simultáneamente, el modelo prioriza una estructura de árbol menos profunda y más generalista, en vez de sobreajustar a las particularidades de un target concreto.

Cabe destacar que la matriz de parámetros empleado para realizar la búsqueda de hiperparámetros es idéntico al empleado en **Random Forest**.

11.2.3.2. Entrenamiento

Al igual que Random Forest de salida única, esta variante depende de la semilla aleatoria del *bootstrap* y de la selección de variables en cada división. Se entrena un ensamblado de 5 modelos con semillas `RANDOM_STATE` a `RANDOM_STATE+4`, cuyas predicciones se promedian en la inferencia. Al tratarse de un único bosque multisalida por semilla (no uno por target), el ensamblado completo requiere solo 5 modelos .pkl, frente a los 105 de la variante de salida única.

La validación cruzada repetida sobre X_{train} , evaluada de forma global sobre los 21 targets conjuntamente, muestra una diferencia Train-CV de 0.166 (R^2 train 0.307 frente a R^2 CV 0.142), por encima del umbral de aviso de 0.15.

A diferencia de Random Forest de salida única, donde prácticamente todos los targets mostraban aviso individual, aquí el consenso de hiperparámetros (con una profundidad más conservadora) modera algo el sobreajuste, aunque no lo elimina del todo.

11.2.3.3. Explicabilidad y variables más relevantes

Se calculan valores **SHAP** sobre `X_train`, promediando el valor absoluto sobre los 21 targets. El top-10 de variables más relevantes es: `soil_ph_z`, `gee_temp_media_C_z`, `eu_as_z`, `eu_ph_z`, `p_z`, `gee_humedad_rel_pct_z`, `eu_cn_ratio_z`, `eu_p_z`, `eu_clay_content_z` y `dem_elevacion_m_z`.

Este ranking es prácticamente idéntico al obtenido por Random Forest de salida única, con las dos mismas variables en primer y segundo lugar, lo que confirma que la relevancia de `soil_ph_z` y `gee_temp_media_C_z` es una propiedad del conjunto de datos y no un artefacto de una configuración de modelo concreta.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Explicabilidad del modelo (SHAP)**.

11.2.3.4. Resultados sobre eval.csv

Con un R^2 medio de **0.0964**, Random Forest multisalida es, de los ocho modelos evaluados en este trabajo, el que obtiene el **mejor promedio global** de R^2 sobre `eval.csv`, ligeramente por encima incluso de su propia variante de salida única (0.090).

Sobre los targets prioritarios obtiene **$R^2=0.4160$** (`earthworm_shannon_z`) y **$R^2=0.4509$** (`earthworm_richness_z`), algo por debajo de lo logrado por Random Forest de salida única para estos dos targets concretos.

Esto sugiere que, si bien compartir la estructura del árbol entre los 21 targets mejora el promedio global, probablemente porque ayuda a los targets con menos señal individual, puede hacerlo a costa de una ligera pérdida de precisión específica en los dos targets con más señal predictiva propia del conjunto de datos.

Variable	Target	R^2
Shannon nemátodos	nematode_shannon_z	0.1545
Shannon macrofauna	macro_shannon_z	0.2000
Shannon lombrices	earthworm_shannon_z	0.4160
Shannon oribátidos	orib_shannon_z	0.1693
Shannon mesostigmátidos	meso_shannon_z	-0.1178
Shannon colémbolos	coll_shannon_z	-0.1953
Shannon bacterias	bac_shannon_z	0.0290
Shannon hongos	fun_shannon_z	-0.0127
Shannon eucariotas	euk_shannon_z	0.1378
Shannon oomicetos	oomy_shannon_z	0.1252
Shannon cercozoos	cerc_shannon_z	-0.0041
Riqueza macrofauna	macro_order_richness_z	0.2532
Riqueza lombrices	earthworm_richness_z	0.4509
Riqueza oribátidos	orib_species_richness_z	0.2386
Riqueza mesostigmátidos	meso_species_richness_z	-0.0255
Riqueza colémbolos	coll_species_richness_z	-0.2920
Riqueza bacterias (ASV)	bac_asv_richness_z	0.0044
Riqueza hongos (ASV)	fun_asv_richness_z	-0.0068
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.1903
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.1917
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.1171

Tabla 15: R^2 de Random Forest multisalida sobre `eval.csv`, por target.

SOB4ES - Random Forest Multisalida

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook implementa un **Random Forest con salida vectorial**: en lugar de entrenar un modelo independiente por cada variable objetivo, se entrena un único modelo que predice los 21 targets simultáneamente.

La clave de este enfoque es que cada nodo de cada árbol toma sus decisiones considerando todos los targets a la vez, lo que permite capturar correlaciones entre grupos biológicos. Por ejemplo, si la diversidad de lombrices y la de colémbolos tienden a co-variar, el modelo puede aprender esa relación.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración**: importaciones y parámetros globales.
2. **Carga de datos**: se cargan los tres splits pre-generados.
3. **Tuning**: búsqueda del valor óptimo de los hiperparámetros.
4. **Validación cruzada**: evaluación de robustez.
5. **Explicabilidad (SHAP)**: análisis de importancia de variables.
6. **Producción**: entrenamiento final y exportación de modelos.
7. **Evaluación final**: rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
from pathlib import Path
import itertools

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

import shap

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV, cross_validate, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance

# Ignorar los warnings
warnings.filterwarnings('ignore')
os.environ["PYTHONWARNINGS"] = "ignore"

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/rf_multi/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
```

```

'silt_content_z',
'sand_content_z',
'aggregate_stability_z',
'bulk_density_z',
'soil_moisture_z',
'as_z',
'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas correctamente...

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos `reserva` nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```

In [3]: def cargar_csv(ruta, nombre):
    if not os.path.exists(ruta):
        raise FileNotFoundError(f'No se encontro: {ruta}')
    try:
        df = pd.read_csv(ruta, sep=',')
    
```

```

        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
        except Exception:
            df = pd.read_csv(ruta, sep=';')
        print(f' {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas')
        return df
print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados.")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

```

Datasets cargados.

```

X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)

```

3.- Búsqueda de hiperparámetros (Tuning)

La búsqueda de hiperparámetros es idéntica a la del Random Forest individual.

El modelo acepta `y` como matriz 2D de forma transparente, por lo que el espacio de búsqueda no cambia.

La búsqueda se realiza sobre `X_train` y `y_train` (todos los targets a la vez).

```

In [4]: print('Iniciando busqueda de hiperparametros...')
        print('Este proceso puede tardar bastante...\n')

        param_grid = {
            'n_estimators': [150, 250, 350],
            'max_depth': [3, 5, 7, 10],
            'min_samples_split': [10, 20, 30, 50],
            'min_samples_leaf': [4, 8, 12, 16],
            'max_features': ['sqrt', 'log2', 0.2, 0.3, 0.4],
            'criterion': ['squared_error', 'friedman_mse'],
            'ccp_alpha': [0.0, 0.001, 0.01, 0.05],
            'bootstrap': [True],
            'max_samples': [0.5, 0.7, 0.8],
            'random_state': [RANDOM_STATE],
        }
        resultados_tuning = {}

        for idx, nombre_target in enumerate(TARGETS, 1):
            buscador = RandomizedSearchCV(
                estimator=RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=N_JOBS),
                param_distributions=param_grid,
                n_iter=150,
                scoring='r2',
                cv=5,
                verbose=0,
                random_state=RANDOM_STATE,
                n_jobs=N_JOBS
            )
            start = time.time()
            buscador.fit(X_train, y_train[nombre_target].values)
            elapsed = (time.time() - start) / 60

            resultados_tuning[nombre_target] = {
                'params': buscador.best_params_,
                'r2': buscador.best_score_,
            }
            print(f'[{idx:2d}/{len(TARGETS)}] | [{nombre_target:<25}] R2: {buscador.best_score_:.4f} | tiempo: {elapsed:.1f} min')

        # Consenso de hiperparametros ponderado por R2 (ahora sobre los 21 targets)
        r2_por_target = {t: max(resultados_tuning[t]['r2'], 0.0) for t in TARGETS}
        suma_r2 = sum(r2_por_target.values())
        if suma_r2 <= 0:
            pesos = {t: 1 / len(TARGETS) for t in TARGETS}
        else:
            pesos = {t: v / suma_r2 for t, v in r2_por_target.items()}

        print('\nCalculando consenso de hiperparametros (ponderado por R2 de los 21 targets)...')
        for t in sorted(pesos, key=pesos.get, reverse=True)[:5]:
            print(f' Peso {t:30}: {pesos[t]:.3f}')
        print(' ...')

        def consenso_int(key, default=0):
            return int(round(sum(pesos[t] * resultados_tuning[t]['params'].get(key, default) for t in TARGETS)))

        def consenso_float(key, default=0.0):
            return sum(pesos[t] * resultados_tuning[t]['params'].get(key, default) for t in TARGETS)

        def consenso_cat(key):

```

```

mejor_target = max(pesos, key=pesos.get)
return resultados_tuning[mejor_target]['params'][key]

def consenso_none_or_int(key):
    items = {t: resultados_tuning[t]['params'].get(key) for t in TARGETS}
    items = {t: v for t, v in items.items() if v is not None}
    if not items:
        return None
    suma_sub = sum(pesos[t] for t in items)
    sub_pesos = {t: (pesos[t] / suma_sub if suma_sub > 0 else 1 / len(items)) for t in items}
    return int(round(sum(sub_pesos[t] * items[t] for t in items)))

def consenso_none_or_float(key):
    items = {t: resultados_tuning[t]['params'].get(key) for t in TARGETS}
    items = {t: v for t, v in items.items() if v is not None}
    if not items:
        return None
    suma_sub = sum(pesos[t] for t in items)
    sub_pesos = {t: (pesos[t] / suma_sub if suma_sub > 0 else 1 / len(items)) for t in items}
    return sum(sub_pesos[t] * items[t] for t in items)

RF_MULTI_PARAMS = {
    'n_estimators':      consenso_int('n_estimators'),
    'max_depth':         consenso_none_or_int('max_depth'),
    'min_samples_split': consenso_int('min_samples_split'),
    'min_samples_leaf':  consenso_int('min_samples_leaf'),
    'max_features':      consenso_cat('max_features'),
    'ccp_alpha':         consenso_float('ccp_alpha'),
    'bootstrap':         consenso_cat('bootstrap'),
    'max_samples':       consenso_none_or_float('max_samples'),
    'max_leaf_nodes':    consenso_none_or_int('max_leaf_nodes'),
    'random_state':      RANDOM_STATE,
    'n_jobs':            N_JOBS,
}

print('\nHiperparametros consenso finales:')
for k, v in RF_MULTI_PARAMS.items():
    print(f' {k}: {v}')

# Grafico: R2 de tuning por target (ordenado)
_orden = sorted(r2_por_target, key=r2_por_target.get)
fig, ax = plt.subplots(figsize=(7, 0.35 * len(_orden) + 1))
_bars = ax.barh([t for t in _orden], [r2_por_target[t] for t in _orden], color=PALETTE['blue'], edgecolor='white', height=0.6)
ax.bar_label(_bars, fmt='{:.3f}', padding=4, fontsize=8)
ax.set_xlabel('R2 (CV 5-fold, X_train)')
ax.set_title('Tuning - Mejor R2 por target (referencia para consenso)', fontsize=12)
ax.axvline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()

```

Iniciando búsqueda de hiperparametros...

Este proceso puede tardar bastante...

[1/21]	[nematode_shannon_z]	R2: 0.2134	tiempo: 0.2 min
[2/21]	[macro_shannon_z]	R2: 0.2450	tiempo: 0.1 min
[3/21]	[earthworm_shannon_z]	R2: 0.4419	tiempo: 0.1 min
[4/21]	[orib_shannon_z]	R2: 0.1032	tiempo: 0.1 min
[5/21]	[meso_shannon_z]	R2: 0.1030	tiempo: 0.1 min
[6/21]	[coll_shannon_z]	R2: 0.2075	tiempo: 0.1 min
[7/21]	[bac_shannon_z]	R2: 0.0901	tiempo: 0.1 min
[8/21]	[fun_shannon_z]	R2: 0.0245	tiempo: 0.1 min
[9/21]	[euk_shannon_z]	R2: 0.1842	tiempo: 0.1 min
[10/21]	[oomy_shannon_z]	R2: 0.1277	tiempo: 0.1 min
[11/21]	[cerc_shannon_z]	R2: 0.0338	tiempo: 0.1 min
[12/21]	[macro_order_richness_z]	R2: 0.2386	tiempo: 0.1 min
[13/21]	[earthworm_richness_z]	R2: 0.4919	tiempo: 0.1 min
[14/21]	[orib_species_richness_z]	R2: 0.0500	tiempo: 0.1 min
[15/21]	[meso_species_richness_z]	R2: 0.1013	tiempo: 0.1 min
[16/21]	[coll_species_richness_z]	R2: 0.2276	tiempo: 0.1 min
[17/21]	[bac_asv_richness_z]	R2: 0.0940	tiempo: 0.1 min
[18/21]	[fun_asv_richness_z]	R2: -0.0251	tiempo: 0.1 min
[19/21]	[euk_asv_richness_z]	R2: 0.2181	tiempo: 0.1 min
[20/21]	[oomy_asv_richness_z]	R2: 0.2355	tiempo: 0.1 min
[21/21]	[cerc_asv_richness_z]	R2: 0.1944	tiempo: 0.1 min

Calculando consenso de hiperparametros (ponderado por R2 de los 21 targets)...

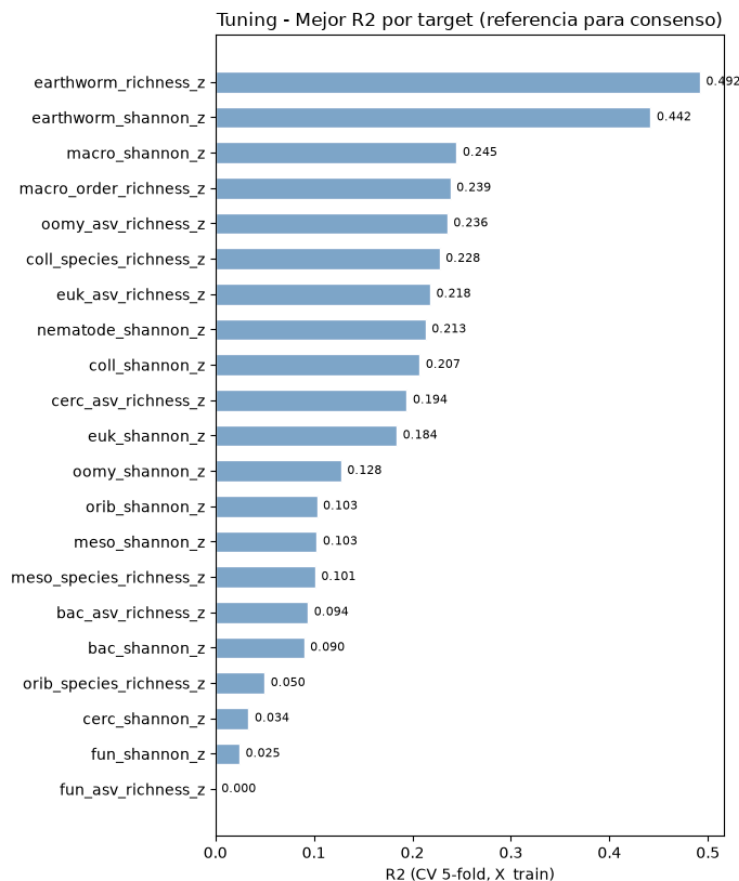
Peso	earthworm_richness_z	: 0.136
Peso	earthworm_shannon_z	: 0.122
Peso	macro_shannon_z	: 0.068
Peso	macro_order_richness_z	: 0.066
Peso	oomy_asv_richness_z	: 0.065
...		

Hiperparametros consenso finales:

```

n_estimators: 292
max_depth: 6
min_samples_split: 14
min_samples_leaf: 5
max_features: log2
ccp_alpha: 0.0012528419146331847
bootstrap: True
max_samples: 0.7205487907512752
max_leaf_nodes: None
random_state: 42
n_jobs: 15

```



4.- Validación cruzada

Se evalúa la robustez del modelo con **validación cruzada repetida** (5 pliegues x 3 repeticiones).

Al ser multisalida, el R2 que reporta sklearn es el promedio sobre todos los targets.

Se muestra también el R2 individual por target para identificar cuáles aprende mejor el modelo.

```
In [5]: print('Validacion cruzada repetida sobre X_train (todos los targets)... \n')

modelo_cv = RandomForestRegressor(**RF_MULTI_PARAMS)
cv_splitter = RepeatedKfold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)

resultados = cross_validate(
    estimator=modelo_cv,
    X=X_train,
    y=y_train,
    cv=cv_splitter,
    return_train_score=True,
    n_jobs=N_JOBS
)

r2_train = np.mean(resultados['train_score'])
r2_cv = np.mean(resultados['test_score'])
r2_std = np.std(resultados['test_score'])

print(f' Train R2 (promedio targets): {r2_train:.4f}')
print(f' Val R2 (promedio targets): {r2_cv:.4f} +/- {r2_std:.4f}')

# R2 individual por target (entrenando sobre todo X_train)
modelo_cv.fit(X_train, y_train)
y_pred_train = modelo_cv.predict(X_train)
print('\n R2 por target (train):')
for i, t in enumerate(TARGETS):
    r2 = r2_score(y_train.iloc[:, i], y_pred_train[:, i])
    print(f' {t:30} {r2:.4f}')

diferencia = r2_train - r2_cv
estado = 'Aviso: Posible sobreajuste' if diferencia > 0.15 else 'OK'

print("Resultados globales (promedio targets):")
print(f"{'R2 global':<25} | {r2_train:<10.3f} | {r2_cv:<10.3f} | {diferencia:<10.3f} | {estado}")

Validacion cruzada repetida sobre X_train (todos los targets)...

Train R2 (promedio targets): 0.3074
Val R2 (promedio targets): 0.1417 +/- 0.0174
```

```
R2 por target (train):
nematode_shannon_z      0.3602
macro_shannon_z         0.3669
earthworm_shannon_z     0.5328
orib_shannon_z          0.2900
meso_shannon_z          0.2695
coll_shannon_z          0.4025
bac_shannon_z           0.2240
fun_shannon_z           0.1777
euk_shannon_z           0.2941
oomy_shannon_z          0.2848
cerc_shannon_z          0.1757
macro_order_richness_z  0.3446
earthworm_richness_z    0.5685
orib_species_richness_z 0.3220
meso_species_richness_z 0.2546
coll_species_richness_z 0.4176
bac_asv_richness_z      0.2167
fun_asv_richness_z      0.1202
euk_asv_richness_z      0.3310
oomy_asv_richness_z     0.3169
cerc_asv_richness_z     0.3356
Resultados globales (promedio targets):
R2 global      | 0.307      | 0.142      | 0.166      | Aviso: Posible sobreajuste
```

5.- Importancia de variables

Se calcula la importancia de variables mediante permutación sobre `X_train`.

Al ser multisalida, la importancia refleja el impacto de cada variable sobre el conjunto de todos los targets simultaneamente.

También se calcula las variables SHAPley para una mejor interpretabilidad del modelo en conjunto.

```
In [6]: # El modelo ya esta ajustado sobre y_train completo desde la celda anterior
perm = permutation_importance(
    modelo_cv, X_train, y_train,
    n_repeats=10, random_state=RANDOM_STATE, n_jobs=-1
)

importancias = pd.Series(perm.importances_mean, index=FEATURES_AUTORIZADAS)
top10 = importancias.nlargest(10)

print('Top 10 variables por importancia global (todos los targets):')
print(top10.round(4).to_string())

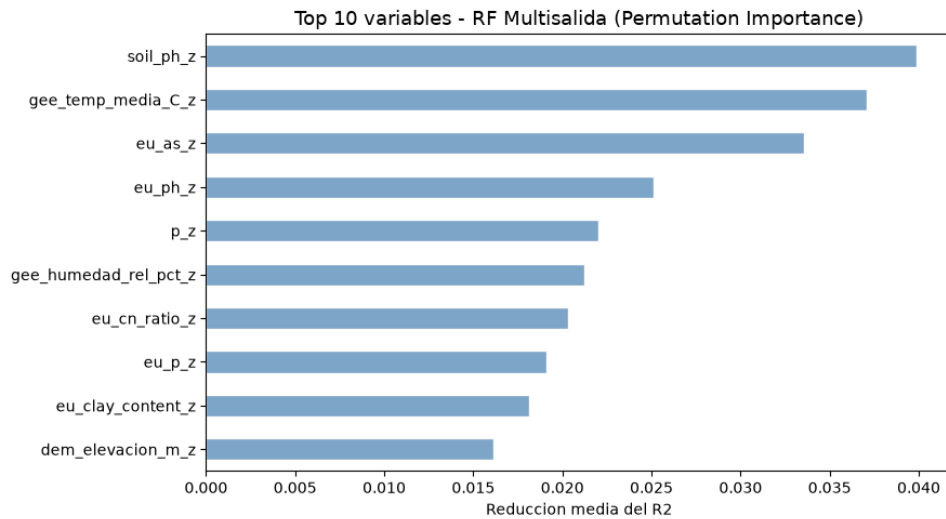
fig, ax = plt.subplots(figsize=(9, 5))
importancias.nlargest(10).sort_values().plot(kind='barh', ax=ax, color=PALETTE['blue'], edgecolor='white')
ax.set_title('Top 10 variables - RF Multisalida (Permutation Importance)', fontsize=13)
ax.set_xlabel('Reduccion media del R2')
ax.set_ylabel('')
plt.tight_layout()
plt.show()

print('Calculando valores SHAP (RF Multisalida)...')
explainer = shap.TreeExplainer(modelo_cv)
shap_values = explainer.shap_values(X_train)

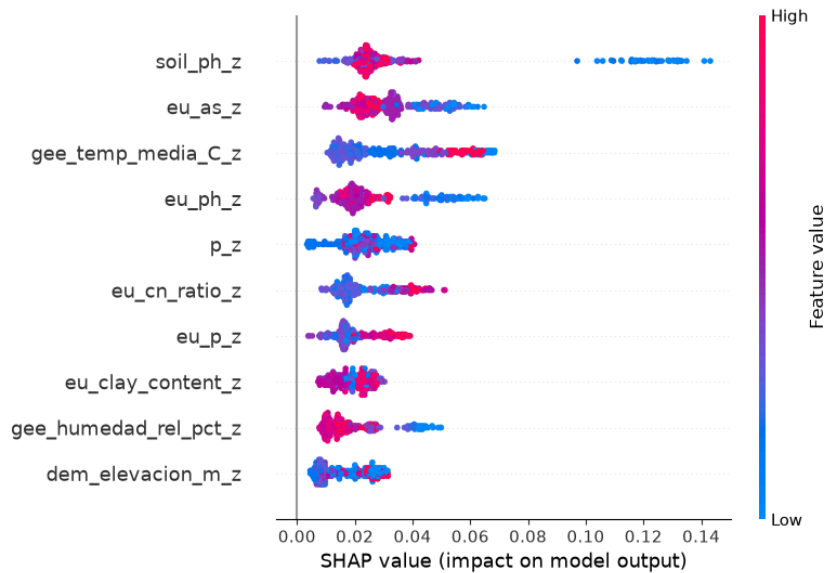
# Si shap_values es 3D (multisalida), promediamos el valor absoluto sobre todos los targets
if isinstance(shap_values, list):
    shap_mean = np.mean([np.abs(sv) for sv in shap_values], axis=0)
elif shap_values.ndim == 3:
    shap_mean = np.abs(shap_values).mean(axis=2)
else:
    shap_mean = shap_values

shap.summary_plot(shap_mean, X_train, plot_type='dot', max_display=10, show=True)

Top 10 variables por importancia global (todos los targets):
soil_ph_z      0.0399
gee_temp_media_C_z  0.0371
eu_as_z        0.0336
eu_ph_z        0.0251
p_z            0.0220
gee_humedad_rel_pct_z 0.0212
eu_cn_ratio_z   0.0203
eu_p_z         0.0191
eu_clay_content_z 0.0182
dem_elevacion_m_z 0.0161
```



Calculando valores SHAP (RF Multisalida)...



6.- Entrenamiento de producción

Se entrena un ensamble de 5 modelos multisalida, variando la semilla aleatoria (semilla + n_modelo).

Cada modelo predice los 21 targets a la vez. Promediar el ensamble reduce la varianza de las predicciones finales.

```
In [7]: print('Entrenamiento de produccion (ensamble de 5 modelos multisalida)...')
print('-' * 70)

global_start = time.time()
NUM_MODELOS = 5

for seed_id in range(NUM_MODELOS):
    t0 = time.time()
    params = RF_MULTI_PARAMS.copy()
    params['random_state'] = RANDOM_STATE + seed_id

    # Entrenamos sobre y_train completo (matriz 11 targets)
    model = RandomForestRegressor(**params)
    model.fit(X_train, y_train)

    joblib.dump(model, os.path.join(MODELS_DIR, f'rf_multi_m{seed_id}.pkl'))
    print(f' Modelo {seed_id+1}/5 completado ({time.time()-t0:.1f}s)')

print('-' * 70)
print(f'Produccion completada en {(time.time()-global_start)/60:.1f} minutos.')
```

Entrenamiento de produccion (ensamble de 5 modelos multisalida)...

Modelo 1/5 completado (0.2s)
 Modelo 2/5 completado (0.2s)
 Modelo 3/5 completado (0.2s)
 Modelo 4/5 completado (0.2s)

Modelo 5/5 completado (0.2s)

Producción completada en 0.0 minutos.

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretación del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biológica).

```
In [8]: NUM_MODELOS = 5

# Evaluación final
# Cargamos los 5 modelos y promediamos sus predicciones sobre eval.csv.

print('Evaluación final sobre eval.csv')
print('-' * 60)

preds_eval = []
for seed_id in range(NUM_MODELOS):
    ruta = os.path.join(MODELS_DIR, f'rf_multi_m{seed_id}.pkl')
    model = joblib.load(ruta)
    preds_eval.append(model.predict(X_eval))

# y_pred_eval es una matriz (n_muestras x len(TARGETS) targets)
y_pred_eval = np.mean(preds_eval, axis=0)
y_true_eval = y_eval.values

# Métricas globales (promedio sobre todos los targets)
r2_global = r2_score(y_true_eval, y_pred_eval, multioutput='uniform_average')
rmse_global = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval, multioutput='uniform_average'))
mae_global = mean_absolute_error(y_true_eval, y_pred_eval, multioutput='uniform_average')

print(f' R2 global: {r2_global:.4f}')
print(f' RMSE global: {rmse_global:.4f} (Z-score)')
print(f' MAE global: {mae_global:.4f} (Z-score)')

# R2 individual por target
print('\n R2 por target:')
r2_por_target = r2_score(y_true_eval, y_pred_eval, multioutput='raw_values')
for t, r2 in zip(TARGETS, r2_por_target):
    print(f' {t:30} {r2:.4f}')

# Smoke test con datos reales
print('\nSmoke test - Inferencia con datos reales')
print('-' * 60)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    # Generamos las predicciones de todos los modelos del ensemble
    votos = []
    for seed_id in range(NUM_MODELOS):
        ruta = os.path.join(MODELS_DIR, f'rf_multi_m{seed_id}.pkl')
        m = joblib.load(ruta)
        pred = m.predict(df_new_data)
        votos.append(pred)

    # Promediamos para obtener el consenso final del ensemble
    consenso = np.mean(votos, axis=0)

    # Lista de targets a inspeccionar en el smoke test
    targets_smoke = ['earthworm_shannon_z', 'earthworm_richness_z']

    for tgt in targets_smoke:
        tgt_idx = TARGETS.index(tgt)
        print(f'\n Target: {tgt}')
        print(' ' + '-' * 58)

        for i, idx in enumerate(indices_elegidos):
            pred_val = consenso[i, tgt_idx]
            real_val = y_eval.loc[idx, tgt]
            error = abs(pred_val - real_val)
            print(f' Fila {idx:<4} | Predicción: {pred_val:>7.4f} | '
                  f'Valor Real: {real_val:>7.4f} | Error: {error:>6.4f}')

        print('\n' + '-' * 60)
        print('Smoke test superado. El pipeline multisalida funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Gráfico: scatter predicción vs valor real (earthworm_shannon_z y earthworm_richness_z)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, tgt, col in zip(axes, _targets_plot, _colors_plot):
    preds = [joblib.load(os.path.join(MODELS_DIR, f'rf_multi_m{i}.pkl')).predict(X_eval)
              for i in range(NUM_MODELOS)]
    yp = np.mean(preds, axis=0)[ :, TARGETS.index(tgt)]
    yt = y_eval[tgt].values

    ax.scatter(yt, yp, alpha=0.7, s=30, color=col, edgecolors='white', linewidths=0.4)
```

```
lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]
ax.plot(lim, lim, 'r--', linewidth=1)

ax.set_title(f'{{tgt}}\nR2={{r2_score(yt, yp):.3f}}', fontsize=9)
ax.set_xlabel('Valor real', fontsize=8)
ax.set_ylabel('Prediccion', fontsize=8)

plt.suptitle('Prediccion vs Valor Real - RF Multisalida Ensemble (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()
```

Evaluacion final sobre eval.csv

R2 global: 0.0964
RMSE global: 0.9198 (Z-score)
MAE global: 0.7026 (Z-score)

R2 por target:

nematode_shannon_z	0.1545
macro_shannon_z	0.2000
earthworm_shannon_z	0.4160
orib_shannon_z	0.1693
meso_shannon_z	-0.1178
coll_shannon_z	-0.1953
bac_shannon_z	0.0290
fun_shannon_z	-0.0127
euk_shannon_z	0.1378
oomy_shannon_z	0.1252
cerc_shannon_z	-0.0041
macro_order_richness_z	0.2532
earthworm_richness_z	0.4509
orib_species_richness_z	0.2386
meso_species_richness_z	-0.0255
coll_species_richness_z	-0.2920
bac_asv_richness_z	0.0044
fun_asv_richness_z	-0.0068
euk_asv_richness_z	0.1903
oomy_asv_richness_z	0.1917
cerc_asv_richness_z	0.1171

Smoke test - Inferencia con datos reales

Target: earthworm_shannon_z

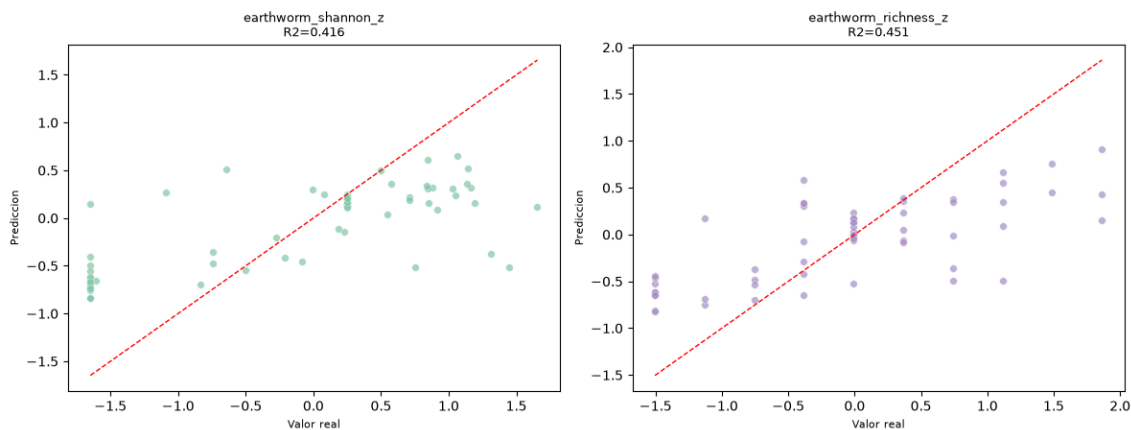
Fila 53	Prediccion: 0.3637	Valor Real: 1.1347	Error: 0.7710
Fila 60	Prediccion: 0.5220	Valor Real: 1.1388	Error: 0.6168
Fila 0	Prediccion: -0.3537	Valor Real: -0.7453	Error: 0.3916

Target: earthworm_richness_z

Fila 53	Prediccion: 0.4514	Valor Real: 1.4889	Error: 1.0375
Fila 60	Prediccion: 0.6655	Valor Real: 1.1147	Error: 0.4492
Fila 0	Prediccion: -0.3746	Valor Real: -0.7562	Error: 0.3817

Smoke test superado. El pipeline multisalida funciona correctamente.

Prediccion vs Valor Real - RF Multisalida Ensemble (eval.csv)



11.2.4. Modelo RegressorChain con Random Forest

11.2.4.1. Fundamento y configuración

RegressorChain (véase **RegressorChain**) encadena las predicciones de los 21 targets: el modelo predice primero un target, y usa esa predicción, junto con las variables originales, como entrada adicional para predecir el siguiente, y así sucesivamente.

A diferencia de Random Forest multisalida, donde la relación entre targets se aprende de forma implícita dentro de la estructura del árbol, aquí la correlación entre targets se introduce de forma explícita, como una variable de entrada más para los targets posteriores de la cadena.

El modelo base de cada eslabón de la cadena es un RandomForestRegressor. Sus hiperparámetros se afinan una única vez mediante RandomizedSearchCV, empleando como métrica el R^2 promedio de los 21 targets simultáneamente.

El mejor R^2 promedio de validación cruzada obtenido en esta búsqueda fue de 0.1521, con los siguientes hiperparámetros:

Hiperparámetro	Valor
n_estimators	200
min_samples_split	15
min_samples_leaf	4
max_samples	0.7
max_features	0.2

Tabla 16: Consenso de hiperparámetros en RegressorChain.

11.2.4.2. Ensemble of Chains (ECC)

El principal problema del encadenamiento simple es que los targets al principio de la cadena disponen de menos información (solo las variables originales) que los del final, los que además cuentan con las predicciones de todos los targets anteriores, por lo que el orden elegido condiciona el rendimiento de cada target concreto.

Para compensar este efecto, se entrena un ensamble de 10 cadenas ($N_CHAINS=10$), cada una con un orden aleatorio distinto de los 21 targets (permutación generada con semilla $RANDOM_STATE + chain_id$) y su propio modelo base con semilla también distinta.

Las predicciones finales se obtienen promediando las 10 cadenas, de forma que los efectos de orden favorables y desfavorables para cada target tienden a cancelarse entre sí.

11.2.4.3. Entrenamiento

La validación cruzada del modelo base muestra avisos de sobreajuste ($diferencia\ Train-CV > 0.15$) en varios de los targets situados en la segunda mitad de la cadena por defecto, por ejemplo, coll_species_richness_z (0.416), cerc_asv_richness_z (0.433) u oomy_asv_richness_z (0.421), un patrón distinto al de Random Forest de salida única, donde los avisos aparecían de forma más homogénea en casi todos los targets.

En una cadena de referencia con orden por defecto, el R^2 de entrenamiento del primer target de la cadena (nematode_shannon_z, 0.5475) es notablemente inferior al de un target situado en una posición más avanzada como earthworm_richness_z en la posición 13 (0.7338).

11.2.4.4. Explicabilidad y variables más relevantes

Se calcula la importancia de variables (SHAP y MDI) del modelo base únicamente para el primer target de la cadena, al ser la posición de mayor dificultad. El top-10 por MDI para esta posición es: sand_content_z, silt_content_z, soil_ph_z, gee_humedad_rel_pct_z, gee_temp_media_C_z, aggregate_stability_z, pb_z, eu_water_holding_capacity_z, k_z y ni_z.

A diferencia del resto de modelos, aquí sand_content_z (contenido de arena) desplaza a soil_ph_z al tercer puesto, lo que resulta coherente con la ausencia de cualquier información de targets relacionados en esta posición de la cadena: el modelo depende en mayor medida de las variables edáficas más básicas.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Análisis de la cadena.**

11.2.4.5. Resultados sobre eval.csv

Con un R^2 medio de **0.0809**, RegressorChain queda por debajo de Random Forest multisalida (0.0964) pero, en cambio, obtiene los mejores resultados de todo el trabajo sobre los dos targets prioritarios: $R^2=0.4876$ (earthworm_shannon_z) y $R^2=0.5359$ (earthworm_richness_z), superando incluso a Random Forest de salida única.

Esto es coherente con la hipótesis de partida del modelo: al encadenar explícitamente las predicciones, los targets con mayor señal individual (como los de lombrices) pueden beneficiarse de la información aportada por targets relacionados situados antes en la cadena, algo que Random Forest multisalida solo captura de forma indirecta. Esto se ve reflejado en las **Conclusiones.**

Variable	Target	R^2
Shannon nemátodos	nematode_shannon_z	0.1467
Shannon macrofauna	macro_shannon_z	0.2804
Shannon lombrices	earthworm_shannon_z	0.4876
Shannon oribátidos	orib_shannon_z	0.2069
Shannon mesostigmátidos	meso_shannon_z	-0.1640
Shannon colémbolos	coll_shannon_z	-0.6087
Shannon bacterias	bac_shannon_z	0.0288
Shannon hongos	fun_shannon_z	-0.0219
Shannon eucariotas	euk_shannon_z	0.1624
Shannon oomicetos	oomy_shannon_z	0.0913
Shannon cercozoos	cerc_shannon_z	-0.0006
Riqueza macrofauna	macro_order_richness_z	0.4069
Riqueza lombrices	earthworm_richness_z	0.5359
Riqueza oribátidos	orib_species_richness_z	0.2026
Riqueza mesostigmátidos	meso_species_richness_z	-0.0400
Riqueza colémbolos	coll_species_richness_z	-0.5799
Riqueza bacterias (ASV)	bac_asv_richness_z	-0.0052
Riqueza hongos (ASV)	fun_asv_richness_z	-0.0186
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.2690
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.2019
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.1171

Tabla 17: R^2 de RegressorChain (ensamble de 10 cadenas) sobre eval.csv, por target.

SOB4ES - RegressorChain (Random Forest base)

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook implementa un **RegressorChain** usando Random Forest como modelo base.

La idea es encadenar las predicciones: el modelo primero predice el target 1, luego usa esa predicción junto a las variables originales para predecir el target 2, y así sucesivamente.

Esto introduce correlación real entre targets: cada predicción tiene acceso a las predicciones anteriores en la cadena.

El problema del encadenamiento simple es que los targets al principio de la cadena tienen menos información que los del final. Para compensarlo se usa un **ensemble de cadenas** donde cada cadena usa un orden diferente y aleatorio de los targets.

Al promediar los resultados, los errores acumulados por el orden se cancelan entre sí.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning del modelo base:** búsqueda del valor óptimo de los hiperparámetros.
4. **Validación cruzada:** evaluación de robustez.
5. **Análisis de la cadena:** importancia de variables y orden
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
from pathlib import Path
import itertools

from sklearn.ensemble import RandomForestRegressor
from sklearn.multioutput import RegressorChain
from sklearn.model_selection import RandomizedSearchCV, cross_validate, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
import shap

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/chain/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

N_CHAINS = 10 # numero de cadenas en el ensemble; mas cadenas = mas estable pero mas lento
RANDOM_STATE = 42

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]
```

```

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
    'zn_z',
    'soil_ph_z',
    'plot_total_organic_c_z',
    'plot_total_n_z',
    'gee_temp_media_C_z',
    'gee_humedad_rel_pct_z',
    'gee_ndvi_verano_z',
    'dem_elevacion_m_z',
    'dem_pendiente_deg_z',
    'dem_orientacion_deg_z',
    'eu_clay_content_z',
    'eu_sand_content_z',
    'eu_silt_content_z',
    'eu_water_holding_capacity_z',
    'eu_cn_ratio_z',
    'eu_p_z',
    'eu_ph_z',
    'eu_as_z',
    'eu_organic_carbon_octop_z',
    'eu_zn_z'
]

print('Librerías y variables cargadas.')

# Paleta de colores unificada
PALETTE = {
    'blue':      '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green':      '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple':      '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray':        '#9B9B9B',
    'gray_dark':   '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas.

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)
import os

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos `reserva` nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 16 | n_jobs asignado: 12

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```
In [3]: def cargar_csv(ruta, nombre):
    if not os.path.exists(ruta):
        raise FileNotFoundError(f'No se encontro: {ruta}')
    try:
        df = pd.read_csv(ruta, sep=',')
        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
    except Exception:
        df = pd.read_csv(ruta, sep=';')
    print(f' {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas')
    return df

print('Cargando datasets...')
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
X_test = df_test[FEATURES_AUTORIZADAS]
X_eval = df_eval[FEATURES_AUTORIZADAS]

# y es una matriz 2D (n_muestras x n_targets) para todos los modelos de esta fase
y_train = df_train[TARGETS]
y_test = df_test[TARGETS]
y_eval = df_eval[TARGETS]

print(f"\nDatasets cargados...")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

Datasets cargados...
X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)
```

3.- Tuning del modelo base

`RegressorChain` encapsula un modelo base que se replica una vez por target.

El tuning se hace sobre ese modelo base (Random Forest individual) usando un único target de referencia. Los parámetros óptimos se usarán para construir cada cadena del ensemble.

Nota: el tuning del modelo base es suficiente porque `RegressorChain` no tiene hiperparámetros propios más allá del orden de la cadena, que se aleatoriza en el entrenamiento.

```
In [4]: print('Iniciando búsqueda de hiperparametros del modelo base...')
print('La búsqueda evalua el R2 promedio sobre los 21 targets simultaneamente (no solo 2 de referencia).\n')

# Espacio de búsqueda ampliado
param_grid_frenado = {
    'n_estimators': [100, 150, 200],
    'max_depth': [3, 4, 5, 10, 15, 20],
    'min_samples_leaf': [2, 4, 8, 12, 16],
    'min_samples_split': [15, 20, 25],
    'max_features': ['sqrt', 0.2, 0.3],
    'max_samples': [0.7, 0.8],
    'ccp_alpha': [0.005, 0.01, 0.02],
    'bootstrap': [True],
    'random_state': [RANDOM_STATE]
}

start_total = time.time()

# RandomForestRegressor admite y multivariante de forma nativa: al pasarle
# y_train completo (21 columnas), el scoring 'r2' de sklearn calcula el
# R2 de cada target y devuelve el promedio (multioutput='uniform_average').
buscador = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1),
    param_distributions=param_grid_frenado,
    n_iter=50,
    scoring='r2',
    cv=5,
    verbose=0,
    random_state=RANDOM_STATE,
    n_jobs=N_JOBS
)
buscador.fit(X_train, y_train)
elapsed = (time.time() - start_total) / 60

BASE_PARAMS = dict(buscador.best_params_)
BASE_PARAMS['random_state'] = RANDOM_STATE
BASE_PARAMS['n_jobs'] = N_JOBS

print(f'Busqueda completada en {elapsed:.1f} minutos.')
print(f'Mejor R2 promedio (21 targets, CV 5-fold): {buscador.best_score_:.4f}\n')
print('Hiperparametros optimos del modelo base:')
for k, v in BASE_PARAMS.items():
    print(f' {k}: {v}')

# R2 individual por target del mejor modelo encontrado, para ver que tan
# representativo es el consenso de hiperparametros para cada variable objetivo
print('\nR2 individual por target (mismo modelo base, CV 5-fold):')
r2_por_target_base = {}
```

```
for target_col in TARGETS:
    scores = cross_validate(
        RandomForestRegressor(**BASE_PARAMS),
        X_train, y_train[target_col].values,
        cv=5, scoring='r2'
    )['test_score']
    r2_por_target_base[target_col] = np.mean(scores)
    print(f' {target_col:30} {r2_por_target_base[target_col]:.4f}')

# Grafico: R2 por target del modelo base ya afinado
r2_series = pd.Series(r2_por_target_base).sort_values()
fig, ax = plt.subplots(figsize=(9, 8))
_bars = ax.barh(r2_series.index, r2_series.values,
                color=palette_cycle(len(r2_series)), edgecolor='white')
ax.bar_label(_bars, fmt='{:.3f}', padding=3, fontsize=8)
ax.set_xlabel('R2 (CV 5-fold, X_train)')
ax.set_title('Tuning - R2 por target (Modelo Base Cadena, hiperparametros afinados sobre los 21 targets)', fontsize=12)
ax.axvline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()
```

Iniciando búsqueda de hiperparametros del modelo base...

La búsqueda evalúa el R2 promedio sobre los 21 targets simultaneamente (no solo 2 de referencia).

Busqueda completada en 0.1 minutos.

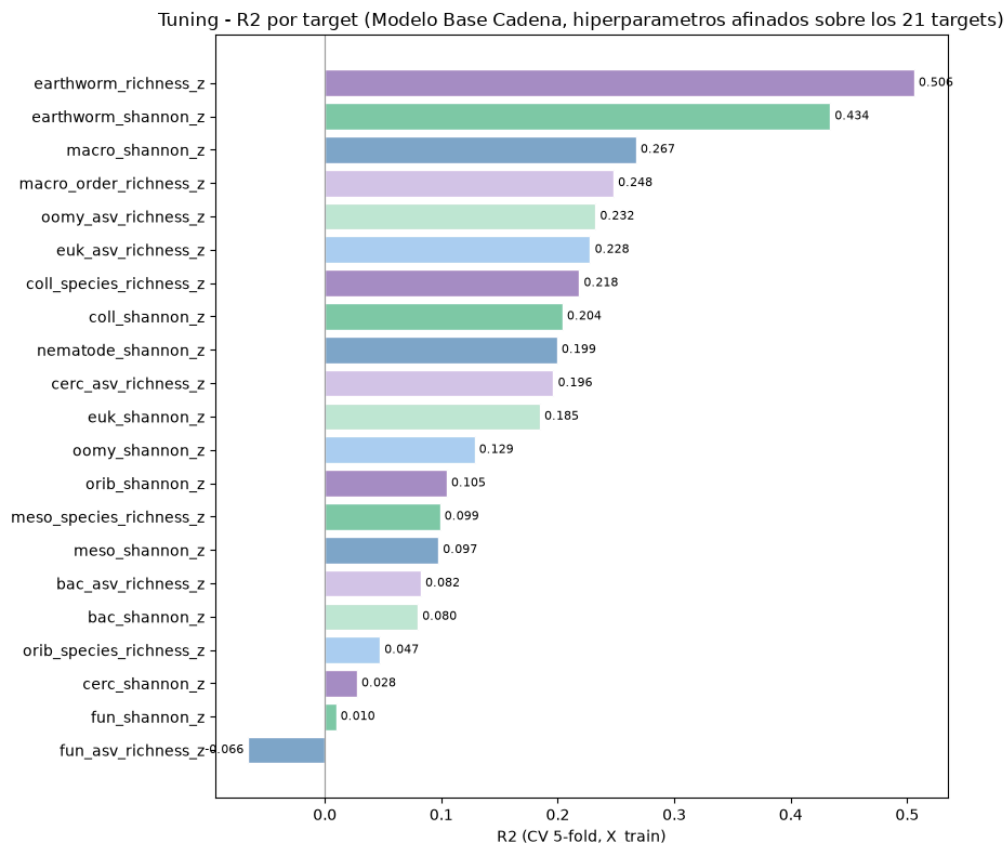
Mejor R2 promedio (21 targets, CV 5-fold): 0.1521

Hiperparametros optimos del modelo base:

```
random_state: 42
n_estimators: 200
min_samples_split: 15
min_samples_leaf: 4
max_samples: 0.7
max_features: 0.2
max_depth: 15
ccp_alpha: 0.005
bootstrap: True
n_jobs: 12
```

R2 individual por target (mismo modelo base, CV 5-fold):

nematode_shannon_z	0.1993
macro_shannon_z	0.2672
earthworm_shannon_z	0.4338
orib_shannon_z	0.1046
meso_shannon_z	0.0975
coll_shannon_z	0.2040
bac_shannon_z	0.0797
fun_shannon_z	0.0096
euk_shannon_z	0.1846
oomy_shannon_z	0.1285
cerc_shannon_z	0.0275
macro_order_richness_z	0.2478
earthworm_richness_z	0.5059
orib_species_richness_z	0.0472
meso_species_richness_z	0.0988
coll_species_richness_z	0.2184
bac_asv_richness_z	0.0824
fun_asv_richness_z	-0.0657
euk_asv_richness_z	0.2278
oomy_asv_richness_z	0.2323
cerc_asv_richness_z	0.1962



4.- Validación cruzada

Se evalúa la robustez de una cadena única (orden por defecto) con **validación cruzada repetida** (5 pliegues x 3 repeticiones) sobre `X_train`.

El ensemble de cadenas aleatorias que se entrena en producción será más robusto que esta cadena única, por lo que estos resultados son una estimación conservadora del rendimiento final.

```
In [5]: import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import RepeatedKFold, cross_validate

print('Validación cruzada repetida (Modelos Independientes + Selección de Variables)... \n')

# Número de mejores predictores a conservar individualmente por target
K_BEST_FEATURES = 8

cv_splitter = RepeatedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
resultados_cv_por_target = {}

header = f"{'TARGET':<26} | {'TRAIN R2':<10} | {'CV R2':<10} | {'DIFF':<10} | {'ESTADO'}"
print(header)
print('-' * len(header))

for target_col in TARGETS:
    y_target = y_train[target_col].values

    # Opción B: Obtener hiperparámetros específicos del target de forma segura
    params_target = BASE_PARAMS.get(target_col, BASE_PARAMS)

    # Pipeline individual: Filtrado de características + Random Forest regularizado
    pipeline_modelo = Pipeline([
        ('selector', SelectKBest(score_func=f_regression, k=K_BEST_FEATURES)),
        ('rf', RandomForestRegressor(
            **params_target
        ))
    ])

    resultados = cross_validate(
        estimator=pipeline_modelo,
        X=X_train,
        y=y_target,
        cv=cv_splitter,
        scoring=['r2'],
        return_train_score=True,
        n_jobs=N_JOBS
    )
```

```

r2_train = np.mean(resultados['train_r2'])
r2_cv = np.mean(resultados['test_r2'])
diferencia = r2_train - r2_cv

resultados_cv_por_target[target_col] = {'train_r2': r2_train, 'cv_r2': r2_cv}

estado = "Aviso: Posible sobreajuste" if diferencia > 0.15 else "OK"

print(f"{target_col:<26} | {r2_train:<10.3f} | {r2_cv:<10.3f} | {diferencia:<10.3f} | {estado}")

```

Validación cruzada repetida (Modelos Independientes + Selección de Variables)...

TARGET	TRAIN R2	CV R2	DIFF	ESTADO
nematode_shannon_z	0.422	0.211	0.211	Aviso: Posible sobreajuste
macro_shannon_z	0.397	0.143	0.254	Aviso: Posible sobreajuste
earthworm_shannon_z	0.516	0.264	0.252	Aviso: Posible sobreajuste
orib_shannon_z	0.363	0.086	0.277	Aviso: Posible sobreajuste
meso_shannon_z	0.334	0.055	0.279	Aviso: Posible sobreajuste
coll_shannon_z	0.432	0.179	0.254	Aviso: Posible sobreajuste
bac_shannon_z	0.309	0.004	0.305	Aviso: Posible sobreajuste
fun_shannon_z	0.309	0.003	0.307	Aviso: Posible sobreajuste
euk_shannon_z	0.369	0.097	0.272	Aviso: Posible sobreajuste
oomy_shannon_z	0.358	0.060	0.297	Aviso: Posible sobreajuste
cerc_shannon_z	0.278	-0.017	0.295	Aviso: Posible sobreajuste
macro_order_richness_z	0.384	0.161	0.223	Aviso: Posible sobreajuste
earthworm_richness_z	0.570	0.368	0.202	Aviso: Posible sobreajuste
orib_species_richness_z	0.340	0.081	0.258	Aviso: Posible sobreajuste
meso_species_richness_z	0.320	0.041	0.278	Aviso: Posible sobreajuste
coll_species_richness_z	0.416	0.179	0.237	Aviso: Posible sobreajuste
bac_asv_richness_z	0.320	0.009	0.312	Aviso: Posible sobreajuste
fun_asv_richness_z	0.270	-0.031	0.301	Aviso: Posible sobreajuste
euk_asv_richness_z	0.415	0.140	0.275	Aviso: Posible sobreajuste
oomy_asv_richness_z	0.421	0.179	0.242	Aviso: Posible sobreajuste
cerc_asv_richness_z	0.433	0.194	0.239	Aviso: Posible sobreajuste

5.- Análisis de la cadena

Se analiza el efecto del orden de la cadena en el rendimiento por target. Los targets al principio de la cadena solo tienen acceso a las variables originales, mientras que los del final tienen además las predicciones de todos los anteriores.

Se muestra también la importancia de variables del modelo base para el primer target de la cadena, donde no hay predicciones previas disponibles y la dificultad es máxima.

```

In [6]: print('Análisis del efecto del orden en la cadena...\n')

print('Entrenando cadena de referencia (orden por defecto) sobre X_train...')
cadena_cv = RegressorChain(
    RandomForestRegressor(**BASE_PARAMS),
    order=list(range(len(TARGETS))),
    random_state=RANDOM_STATE
)
cadena_cv.fit(X_train, y_train)
y_pred_train = cadena_cv.predict(X_train)
print('Cadena de referencia entrenada.\n')

print(' Posicion en cadena → target → R2 (train):')
for i, t in enumerate(TARGETS):
    r2 = r2_score(y_train.iloc[:, i], y_pred_train[:, i])
    print(f' Posicion {i+1:2d} | {t:30} R2: {r2:.4f}')

primer_modelo = cadena_cv.estimators_[0]
imp_mdi = pd.Series(primer_modelo.feature_importances_, index=FEATURES_AUTORIZADAS)

top10_mdi = imp_mdi.nlargest(10)

print('\n Top 10 variables más relevantes (MDI - Primer Target):')
print(top10_mdi.round(4).to_string())

fig, axes_mdi = plt.subplots(1, 1, figsize=(16, 5))

top10_mdi.sort_values().plot(kind='barh', ax=axes_mdi, color=PALETTE['blue'], edgecolor='white')
axes_mdi.set_title('Top 10 más relevantes - Modelo base (MDI)', fontsize=12)
axes_mdi.set_xlabel('Importancia MDI')

plt.tight_layout()
plt.show()

explainer = shap.TreeExplainer(primer_modelo)
shap_values = explainer.shap_values(X_train)

if isinstance(shap_values, list):
    shap_arr = np.mean([np.abs(sv) for sv in shap_values], axis=0)
elif hasattr(shap_values, 'ndim') and shap_values.ndim == 3:
    shap_arr = np.abs(shap_values).mean(axis=2)
else:
    shap_arr = shap_values

print('\nGráfico SHAP summary plot (primer target):')
shap.summary_plot(shap_arr, X_train, plot_type='dot', max_display=10)

mean_abs_shap = np.abs(shap_arr).mean(axis=0)
imp_shap = pd.Series(mean_abs_shap, index=FEATURES_AUTORIZADAS)

top10_shap = imp_shap.nlargest(10)

fig, axes_shap = plt.subplots(1, 1, figsize=(16, 5))

```

```
top10_shap.sort_values().plot(kind='barh', ax=axes_shap, color=PALETTE['blue'], edgecolor='white')
axes_shap.set_title('Top 10 MÁS relevantes - Modelo base (SHAP)', fontsize=12)
axes_shap.set_xlabel('Mean [SHAP value]')

plt.tight_layout()
plt.show()
```

Análisis del efecto del orden en la cadena...

Entrenando cadena de referencia (orden por defecto) sobre X_train...
Cadena de referencia entrenada.

Posicion en cadena → target → R2 (train):

Posicion 1		nematode_shannon_z	R2: 0.5475
Posicion 2		macro_shannon_z	R2: 0.5785
Posicion 3		earthworm_shannon_z	R2: 0.6811
Posicion 4		orib_shannon_z	R2: 0.4634
Posicion 5		meso_shannon_z	R2: 0.3865
Posicion 6		coll_shannon_z	R2: 0.4797
Posicion 7		bac_shannon_z	R2: 0.3676
Posicion 8		fun_shannon_z	R2: 0.4135
Posicion 9		euk_shannon_z	R2: 0.4724
Posicion 10		oomy_shannon_z	R2: 0.4822
Posicion 11		cerc_shannon_z	R2: 0.2991
Posicion 12		macro_order_richness_z	R2: 0.5504
Posicion 13		earthworm_richness_z	R2: 0.7338
Posicion 14		orib_species_richness_z	R2: 0.3623
Posicion 15		meso_species_richness_z	R2: 0.2939
Posicion 16		coll_species_richness_z	R2: 0.4368
Posicion 17		bac_asv_richness_z	R2: 0.4276
Posicion 18		fun_asv_richness_z	R2: 0.2744
Posicion 19		euk_asv_richness_z	R2: 0.4726
Posicion 20		oomy_asv_richness_z	R2: 0.4792
Posicion 21		cerc_asv_richness_z	R2: 0.4287

Top 10 variables más relevantes (MDI - Primer Target):

sand_content_z	0.1228
silt_content_z	0.0824
soil_ph_z	0.0736
gee_humedad_rel_pct_z	0.0599
gee_temp_media_C_z	0.0578
aggregate_stability_z	0.0507
pb_z	0.0401
eu_water_holding_capacity_z	0.0296
k_z	0.0296
ni_z	0.0295

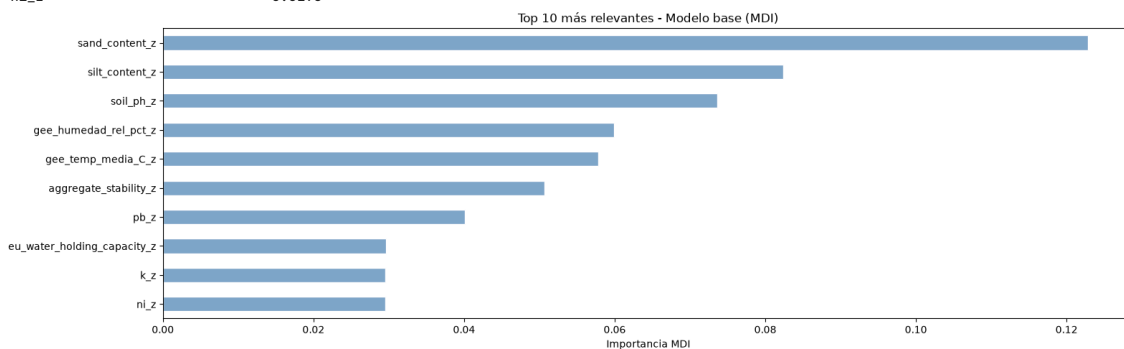
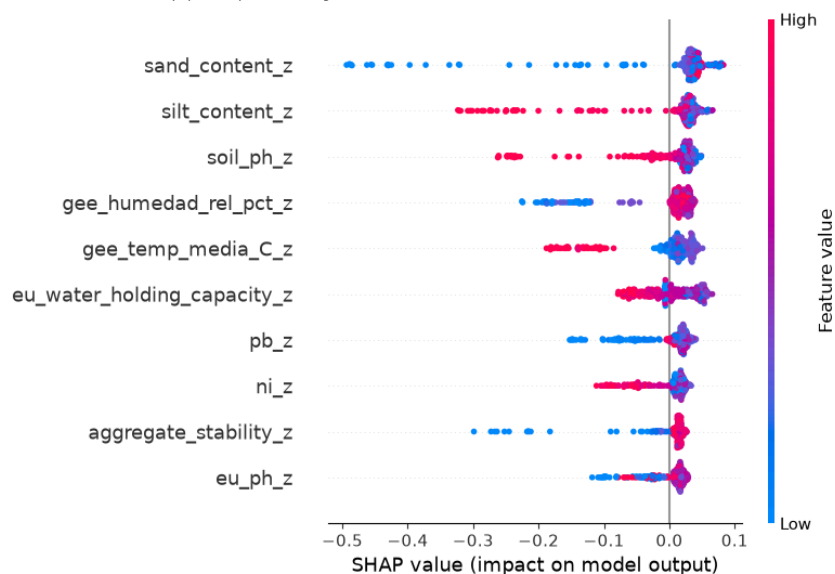
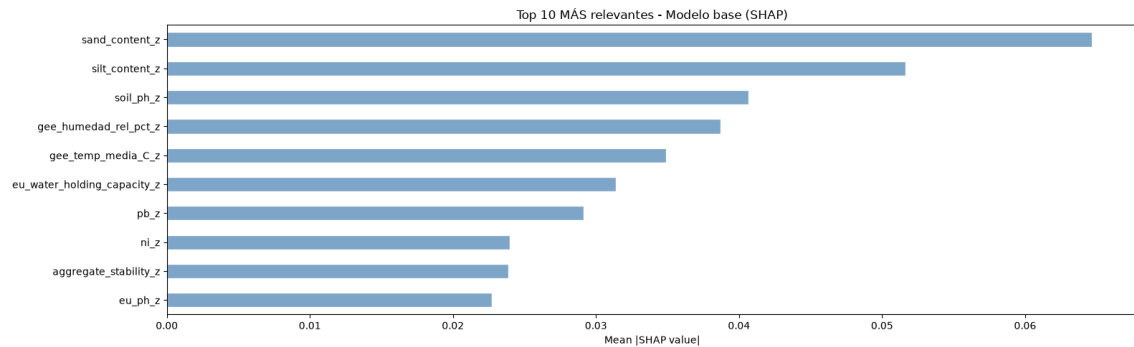


Gráfico SHAP summary plot (primer target):





6.- Entrenamiento

Se entrenan 10 cadenas con ordenes aleatorios distintos.

Cada cadena usa el mismo modelo base (Random Forest con los hiperparámetros óptimos) pero un orden diferente de los targets.

Al promediar las predicciones de todas las cadenas, se compensan los errores de acumulación que afectan a los targets al principio de cada cadena.

```
In [7]: print(f'Entrenamiento de produccion ({N_CHAINS} cadenas con ordenes aleatorios)...')
print('-' * 70)

global_start = time.time()
ordenes_usados = []

for chain_id in range(N_CHAINS):
    t0 = time.time()

    # Cada cadena usa un orden aleatorio distinto de los 11 targets
    orden = list(np.random.RandomState(RANDOM_STATE + chain_id).permutation(len(TARGETS)))
    ordenes_usados.append(orden)

    base_model = RandomForestRegressor(**BASE_PARAMS, 'random_state': RANDOM_STATE + chain_id)
    cadena = RegressorChain(base_model, order=orden)
    cadena.fit(X_train, y_train)

    joblib.dump(cadena, os.path.join(MODELS_DIR, f'chain_{chain_id}.pkl'))
    print(f' Cadena {chain_id+1:2d}/{N_CHAINS} | orden: {orden} | {(time.time()-t0:.1f)s}')

# Guardamos los ordenes para referencia futura
joblib.dump(ordenes_usados, os.path.join(MODELS_DIR, 'ordenes.pkl'))

print('-' * 70)
print(f'Produccion completada en {(time.time()-global_start)/60:.1f} minutos.')
```

Entrenamiento de produccion (10 cadenas con ordenes aleatorios)...

```
-----
Cadena 1/10 | orden: [np.int64(0), np.int64(17), np.int64(15), np.int64(1), np.int64(8), np.int64(5), np.int64(11), np.int64(3), n
p.int64(18), np.int64(16), np.int64(13), np.int64(2), np.int64(9), np.int64(20), np.int64(4), np.int64(12), np.int64(7), np.int64(10),
np.int64(14), np.int64(19), np.int64(6)] | (3.5s)
Cadena 2/10 | orden: [np.int64(20), np.int64(12), np.int64(8), np.int64(5), np.int64(6), np.int64(11), np.int64(10), np.int64(13),
np.int64(1), np.int64(9), np.int64(18), np.int64(3), np.int64(19), np.int64(7), np.int64(15), np.int64(14), np.int64(2), np.int64(16),
np.int64(17), np.int64(0), np.int64(4)] | (3.3s)
Cadena 3/10 | orden: [np.int64(15), np.int64(14), np.int64(9), np.int64(2), np.int64(5), np.int64(11), np.int64(18), np.int64(1), n
p.int64(6), np.int64(12), np.int64(7), np.int64(8), np.int64(10), np.int64(16), np.int64(4), np.int64(19), np.int64(13),
np.int64(17), np.int64(3), np.int64(20)] | (3.4s)
Cadena 4/10 | orden: [np.int64(13), np.int64(17), np.int64(20), np.int64(2), np.int64(16), np.int64(12), np.int64(18), np.int64(1
0), np.int64(7), np.int64(14), np.int64(6), np.int64(15), np.int64(5), np.int64(1), np.int64(8), np.int64(9), np.int64(4), np.int64(1
9), np.int64(0), np.int64(3), np.int64(11)] | (3.3s)
Cadena 5/10 | orden: [np.int64(1), np.int64(3), np.int64(4), np.int64(10), np.int64(20), np.int64(12), np.int64(15), np.int64(13),
np.int64(9), np.int64(14), np.int64(6), np.int64(7), np.int64(0), np.int64(19), np.int64(2), np.int64(17), np.int64(16), np.int64(11),
np.int64(18), np.int64(8), np.int64(5)] | (3.4s)
Cadena 6/10 | orden: [np.int64(12), np.int64(4), np.int64(16), np.int64(15), np.int64(5), np.int64(19), np.int64(2), np.int64(9), n
p.int64(1), np.int64(10), np.int64(13), np.int64(14), np.int64(11), np.int64(18), np.int64(0), np.int64(3), np.int64(17), np.int64(8),
np.int64(20), np.int64(6), np.int64(7)] | (3.4s)
Cadena 7/10 | orden: [np.int64(1), np.int64(9), np.int64(5), np.int64(7), np.int64(8), np.int64(12), np.int64(18), np.int64(3), np.
int64(11), np.int64(10), np.int64(13), np.int64(15), np.int64(2), np.int64(14), np.int64(6), np.int64(20), np.int64(16), np.int64(4),
np.int64(17), np.int64(19), np.int64(0)] | (3.4s)
Cadena 8/10 | orden: [np.int64(15), np.int64(20), np.int64(9), np.int64(6), np.int64(19), np.int64(7), np.int64(18), np.int64(3), n
p.int64(11), np.int64(0), np.int64(4), np.int64(2), np.int64(1), np.int64(17), np.int64(14), np.int64(16), np.int64(5), np.int64(12),
np.int64(8), np.int64(13), np.int64(10)] | (3.4s)
Cadena 9/10 | orden: [np.int64(20), np.int64(9), np.int64(10), np.int64(19), np.int64(3), np.int64(15), np.int64(8), np.int64(7), n
p.int64(2), np.int64(12), np.int64(18), np.int64(17), np.int64(5), np.int64(6), np.int64(4), np.int64(14), np.int64(1), np.int64(13),
np.int64(11), np.int64(0), np.int64(16)] | (3.4s)
Cadena 10/10 | orden: [np.int64(1), np.int64(12), np.int64(19), np.int64(2), np.int64(7), np.int64(6), np.int64(3), np.int64(8), np.
int64(13), np.int64(11), np.int64(10), np.int64(15), np.int64(4), np.int64(14), np.int64(17), np.int64(16), np.int64(2
0), np.int64(9), np.int64(0), np.int64(5)] | (3.4s)
-----
Produccion completada en 0.6 minutos.
```

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretación del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de 0.0 (sin diversidad) a 4.0-5.0 (alta diversidad biológica).

```
In [8]: # Evaluacion final
# Cargamos las N cadenas y promediamos sus predicciones sobre eval.csv.

print('Evaluacion final sobre eval.csv')
print('-' * 60)

preds_eval = []
for chain_id in range(N_CHAINS):
    ruta = os.path.join(MODELS_DIR, f'chain_{chain_id}.pkl')
    cadena = joblib.load(ruta)
    preds_eval.append(cadena.predict(X_eval))

# Promediamos sobre todas las cadenas: shape (N_CHAINS, n_eval, 11) -> (n_eval, 11)
y_pred_eval = np.mean(preds_eval, axis=0)
y_true_eval = y_eval.values

r2_global = r2_score(y_true_eval, y_pred_eval, multioutput='uniform_average')
rmse_global = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval, multioutput='uniform_average'))
mae_global = mean_absolute_error(y_true_eval, y_pred_eval, multioutput='uniform_average')

print(f' R2 global: {r2_global:.4f}')
print(f' RMSE global: {rmse_global:.4f} puntos de Shannon H')
print(f' MAE global: {mae_global:.4f} puntos de Shannon H')

print('\n R2 por target:')
r2_por_target = r2_score(y_true_eval, y_pred_eval, multioutput='raw_values')
for t, r2 in zip(TARGETS, r2_por_target):
    print(f' {t:30} {r2:.4f}')

# Smoke test con datos reales

print('\nSmoke test - Inferencia con datos reales')
print('-' * 60)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index
    valores_reales = y_eval.loc[indices_elegidos].values

    votos = []
    for chain_id in range(N_CHAINS):
        ruta = os.path.join(MODELS_DIR, f'chain_{chain_id}.pkl')
        cadena = joblib.load(ruta)
        pred = cadena.predict(df_new_data)
        votos.append(pred)
    # pred es una matriz (3 muestras x 11 targets)
    print(f' Cadena {chain_id*1:2d} | '
          f'Fila {indices_elegidos[0]}: {pred[0, 2]:.4f} | '
          f'Fila {indices_elegidos[1]}: {pred[1, 2]:.4f} | '
          f'Fila {indices_elegidos[2]}: {pred[2, 2]:.4f} (earthworm_shannon_z)')

    consenso = np.mean(votos, axis=0)
    print('-' * 60)
    print(' Resultados detallados por muestra (earthworm_shannon_z):')
    print('-' * 60)

    idx_earth = TARGETS.index('earthworm_shannon_z')
    for i, idx in enumerate(indices_elegidos):
        pred_val = consenso[i, idx_earth]
        real_val = y_eval.loc[idx, 'earthworm_shannon_z']
        error = abs(pred_val - real_val)
        print(f' Fila {idx:<4} | Prediccion: {pred_val:.4f} | '
              f'Valor Real: {real_val:.4f} | Error absoluto: {error:.4f}')

    print('-' * 60)
    print('\nSmoke test superado. El ensemble de cadenas funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: scatter prediccion vs valor real (targets principales)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, tgt, col in zip(axes, _targets_plot, _colors_plot):
    idx_col = TARGETS.index(tgt)
    yp = y_pred_eval[:, idx_col]
    yt = y_true_eval[:, idx_col]
    ax.scatter(yt, yp, alpha=0.7, s=30, color=col, edgecolors='white', linewidths=0.4)
    lim = (min(yt.min(), yp.min()), max(yt.max(), yp.max()))
    ax.plot(lim, lim, 'r--', linewidth=1)
    ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)
    ax.set_xlabel('Valor real', fontsize=8)
    ax.set_ylabel('Prediccion', fontsize=8)
plt.suptitle('Prediccion vs Valor Real - RegressorChain (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()
```

Evaluacion final sobre eval.csv

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

R2 global: 0.0809
 RMSE global: 0.9184 puntos de Shannon H'
 MAE global: 0.6860 puntos de Shannon H'

R2 por target:

nematode_shannon_z	0.1467
macro_shannon_z	0.2804
earthworm_shannon_z	0.4876
orib_shannon_z	0.2069
meso_shannon_z	-0.1640
coll_shannon_z	-0.6087
bac_shannon_z	0.0288
fun_shannon_z	-0.0219
euk_shannon_z	0.1624
oomy_shannon_z	0.0913
cerc_shannon_z	-0.0006
macro_order_richness_z	0.4069
earthworm_richness_z	0.5359
orib_species_richness_z	0.2026
meso_species_richness_z	-0.0400
coll_species_richness_z	-0.5799
bac_asv_richness_z	-0.0052
fun_asv_richness_z	-0.0186
euk_asv_richness_z	0.2690
oomy_asv_richness_z	0.2019
cerc_asv_richness_z	0.1171

Smoke test - Inferencia con datos reales

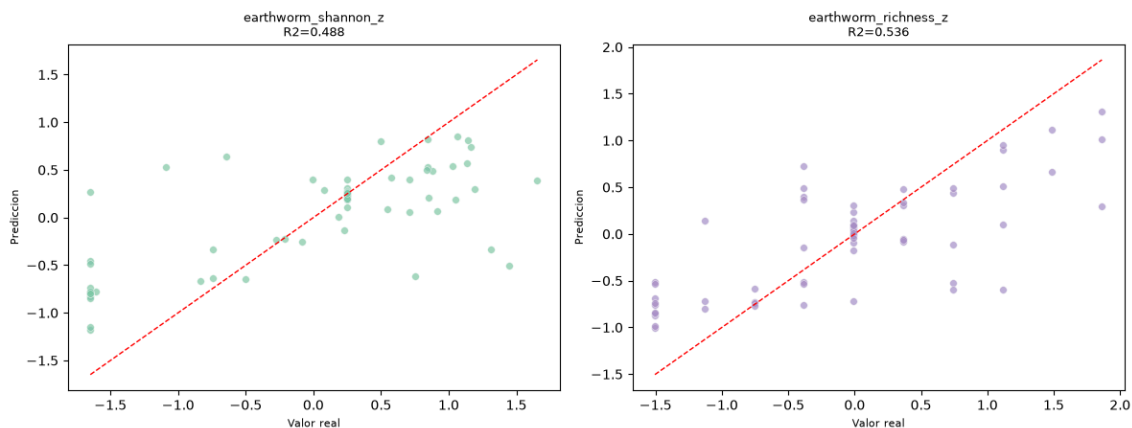
Cadena 1		Fila 53: 0.5193		Fila 60: 0.8298		Fila 0: -0.4131	(earthworm_shannon_z)
Cadena 2		Fila 53: 0.6852		Fila 60: 0.8915		Fila 0: -0.1392	(earthworm_shannon_z)
Cadena 3		Fila 53: 0.4757		Fila 60: 0.8262		Fila 0: -0.4774	(earthworm_shannon_z)
Cadena 4		Fila 53: 0.4903		Fila 60: 0.7401		Fila 0: -0.4619	(earthworm_shannon_z)
Cadena 5		Fila 53: 0.6739		Fila 60: 0.8199		Fila 0: -0.1565	(earthworm_shannon_z)
Cadena 6		Fila 53: 0.6066		Fila 60: 0.7660		Fila 0: -0.1565	(earthworm_shannon_z)
Cadena 7		Fila 53: 0.5839		Fila 60: 0.8394		Fila 0: -0.1110	(earthworm_shannon_z)
Cadena 8		Fila 53: 0.5103		Fila 60: 0.7198		Fila 0: -0.4841	(earthworm_shannon_z)
Cadena 9		Fila 53: 0.5261		Fila 60: 0.7943		Fila 0: -0.4297	(earthworm_shannon_z)
Cadena 10		Fila 53: 0.6556		Fila 60: 0.8574		Fila 0: -0.5372	(earthworm_shannon_z)

Resultados detallados por muestra (earthworm_shannon_z):

Fila 53		Prediccion: 0.5727		Valor Real: 1.1347		Error absoluto: 0.5620
Fila 60		Prediccion: 0.8084		Valor Real: 1.1388		Error absoluto: 0.3304
Fila 0		Prediccion: -0.3367		Valor Real: -0.7453		Error absoluto: 0.4086

Smoke test superado. El ensemble de cadenas funciona correctamente.

Prediccion vs Valor Real - RegressorChain (eval.csv)



11.2.5. Modelo XGBoost

11.2.5.1. Fundamento y configuración

XGBoost (véase **Modelo XGBoost**) se entrena, igual que en el caso de Random Forest, en su variante de salida única: un modelo independiente por target, cada uno con su propia búsqueda de hiperparámetros mediante RandomizedSearchCV sobre X_{train} .

El notebook incorpora una celda de detección automática de **GPU/CUDA**: si el sistema dispone de una GPU NVIDIA con drivers CUDA, XGBoost se configura para usar **tree_method='hist'** junto con **device='cuda'**. En caso contrario, se utiliza **tree_method='hist'** sobre CPU.

La versión del notebook exportado se puede ver que se detectó CUDA sin problemas.

El espacio de búsqueda de hiperparámetros se restringe deliberadamente hacia modelos conservadores, dado el tamaño reducido del dataset:

- **n_estimators**: 50, 100, 150.
- **max_depth**: 2, 3.
- **learning_rate**: 0.01, 0.05.
- **subsample** / **colsample_bytree**: 0.4-0.6 / 0.3-0.5.
- **gamma**: 1.0, 5.0, 10.0.
- **min_child_weight**: 10, 20, 30.
- **reg_alpha** / **reg_lambda**: regularización L1 y L2, con valores 5-20 y 10-50 respectivamente.
- **grow_policy**: depthwise.

Esta combinación de árboles poco profundos, submuestreo agresivo y regularización fuerte responde directamente a la limitación de tener solo 300 muestras de entrenamiento: sin estas restricciones, XGBoost tiende a sobreajustar con rapidez.

11.2.5.2. Entrenamiento

Al igual que Random Forest, XGBoost depende de la semilla aleatoria, por lo que se entrena un ensamblado de 5 modelos por target (**NUM_EXPERTS=5**), variando únicamente la semilla, y promediando sus predicciones en la inferencia.

La validación cruzada repetida sobre X_{train} muestra avisos de sobreajuste en 19 de los 21 targets, un patrón similar al de Random Forest de salida única aunque con diferencias Train-CV algo menores en varios targets, lo que sugiere que la regularización fuerte aplicada en el espacio de búsqueda modera ligeramente el sobreajuste sin llegar a eliminarlo.

11.2.5.3. Explicabilidad y variables más relevantes

A diferencia de Random Forest, en este notebook SHAP se calcula únicamente sobre los dos targets prioritarios (**earthworm_shannon_z** y **earthworm_richness_z**), en vez de sobre el conjunto completo de 21 targets, por lo que no se dispone de un ranking de variables a nivel global comparable al del resto de modelos.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Explicabilidad del modelo (SHAP)**.

11.2.5.4. Resultados sobre eval.csv

El R^2 medio sobre los 21 targets es de **0.067**, ligeramente por debajo de Random Forest de salida única (0.090) pero muy por encima de Ridge.

Sobre los targets prioritarios: $R^2=0.4946$ (earthworm_shannon_z) y $R^2=0.5171$ (earthworm_richness_z).

Variable	Target	R^2	RMSE	MAE
Shannon nemátodos	nematode_shannon_z	0.1637	0.9032	0.7152
Shannon macrofauna	macro_shannon_z	0.2601	0.8639	0.7383
Shannon lombrices	earthworm_shannon_z	0.4946	0.7096	0.5336
Shannon oribátidos	orib_shannon_z	0.1147	1.0760	0.9105
Shannon mesostigmátidos	meso_shannon_z	-0.1868	1.0639	0.8782
Shannon colémbolos	coll_shannon_z	-0.3312	0.7309	0.5941
Shannon bacterias	bac_shannon_z	0.0092	0.9968	0.6707
Shannon hongos	fun_shannon_z	-0.0206	1.1559	0.9385
Shannon eucariotas	euk_shannon_z	0.0912	0.9245	0.6622
Shannon oomicetos	oomy_shannon_z	0.0965	0.9937	0.7411
Shannon cercozoos	cerc_shannon_z	-0.0414	0.9138	0.6353
Riqueza macrofauna	macro_order_richness_z	0.3082	0.7617	0.6447
Riqueza lombrices	earthworm_richness_z	0.5171	0.6346	0.4842
Riqueza oribátidos	orib_species_richness_z	0.1259	1.1186	0.7501
Riqueza mesostigmátidos	meso_species_richness_z	-0.0592	0.9927	0.7503
Riqueza colémbolos	coll_species_richness_z	-0.5803	0.6472	0.5071
Riqueza bacterias (ASV)	bac_asv_richness_z	0.0014	1.0980	0.8196
Riqueza hongos (ASV)	fun_asv_richness_z	-0.0242	1.0907	0.8221
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.1671	0.7409	0.5285
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.1776	0.9108	0.7459
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.1227	0.9241	0.7506

Tabla 18: Resultados de XGBoost (salida única, ensamble de 5 modelos) sobre eval.csv, por target.

SOB4ES - Modelo XGBoost

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook recorre el ciclo de vida completo del modelo XGBoost.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning:** búsqueda del valor óptimo de los hiperparámetros.
4. **Validación cruzada:** evaluación de robustez.
5. **Explicabilidad (SHAP):** análisis de importancia de variables.
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
from pathlib import Path
from datetime import datetime

import itertools

from matplotlib.colors import LinearSegmentedColormap
import matplotlib.pyplot as plt

import shap
import xgboost
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV, cross_validate, KFold, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# ignoramos los warnings
warnings.filterwarnings('ignore')
os.environ["PYTHONWARNINGS"] = "ignore"

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/xgb'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
```

```

'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue':      '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green':     '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple':    '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray':      '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusión
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas correctamente...

```

In [2]: # Deteccion de GPU / CUDA

# Comprueba si hay una GPU NVIDIA disponible y, en ese caso, configura
# XGBoost para usarla automaticamente. Si no hay GPU (o XGBoost no fue
# compilado con soporte CUDA), se usa la CPU sin que falle nada.

import subprocess

def _detectar_cuda():
    """Comprueba si hay una GPU NVIDIA con drivers CUDA disponibles en el sistema."""
    try:
        resultado = subprocess.run(
            ['nvidia-smi'],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            timeout=5,
        )
        return resultado.returncode == 0
    except (FileNotFoundError, subprocess.TimeoutExpired):
        return False

def _xgb_soporta_device():
    """XGBoost ≥ 2.0 usa el parametro 'device'; las versiones anteriores
    activan la GPU unicamente con tree_method='gpu_hist'."""
    try:
        return int(xgboost.__version__.split('.')[0]) ≥ 2
    except Exception:
        return False

CUDA_DISPONIBLE = _detectar_cuda()

if CUDA_DISPONIBLE and _xgb_soporta_device():
    # XGBoost ≥ 2.0: GPU via tree_method='hist' + device='cuda'
    XGB_DEVICE_PARAMS = {'tree_method': 'hist', 'device': 'cuda'}
elif CUDA_DISPONIBLE:
    # XGBoost < 2.0: GPU via tree_method='gpu_hist'
    XGB_DEVICE_PARAMS = {'tree_method': 'gpu_hist'}
else:
    XGB_DEVICE_PARAMS = {'tree_method': 'hist'}

print(f'GPU NVIDIA / CUDA detectada : {CUDA_DISPONIBLE}')
print(f'Version de XGBoost       : {xgboost.__version__}')
print(f'Parametros de dispositivo   : {XGB_DEVICE_PARAMS}')
if CUDA_DISPONIBLE:
    print('XGBoost usará la GPU para el tuning, la validacion y el entrenamiento de produccion...')
else:
    print('No se detecto GPU. XGBoost usara CPU (tree_method=\'hist\')...')

# Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)
import os

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos 'reserva' nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""

```

```

n_cpu = os.cpu_count() or 1
n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
return n_usar

N_JOBS = get_n_jobs()
print(f'Núcleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

GPU NVIDIA / CUDA detectada : True
 Version de XGBoost : 3.3.0
 Parametros de dispositivo : {'tree_method': 'hist', 'device': 'cuda'}
 XGBoost usará la GPU para el tuning, la validación y el entrenamiento de producción...
 Núcleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparámetros / Val. cruzada	64
eval.csv	Evaluación final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```

In [3]: def cargar_csv(ruta, nombre):
        """Carga un CSV (sep=', ' o ';') y valida las columnas necesarias."""
        if not os.path.exists(ruta):
            raise FileNotFoundError(f"No se encontro el archivo: {ruta}")
        try:
            df = pd.read_csv(ruta, sep=',')
            if len(df.columns) < 5:
                df = pd.read_csv(ruta, sep=';')
        except Exception:
            df = pd.read_csv(ruta, sep=';')
        print(f" {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas")
        return df

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados.")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

```

Cargando datasets...
 train.csv: 299 filas x 71 columnas
 test.csv: 64 filas x 71 columnas
 eval.csv: 65 filas x 71 columnas

Datasets cargados.
 X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
 y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)

3.- Búsqueda de hiperparámetros (Tuning)

Se realiza una búsqueda aleatoria (`RandomizedSearchCV`) sobre el espacio de hiperparámetros del modelo XGBoost.

La búsqueda se ejecuta exclusivamente sobre `X_train` para evitar cualquier fuga de información hacia los splits de validación y evaluación.

Como variable objetivo de referencia se usa `earthworm_shannon_z` (lombrices de tierra).

Se hacen uso de las siguientes medidas para reducir el sobreajuste:

- **Grid mas conservador:** `max_depth` maximo 4, `n_estimators` maximo 300.
- **Regularización L1/L2:** se añaden `reg_alpha` y `reg_lambda` al grid.
- **Early stopping:** el entrenamiento para cuando el rendimiento en `X_test` deja de mejorar durante 20 iteraciones consecutivas.

```

In [4]: print("Iniciando búsqueda de hiperparámetros (Tuning) para cada target...")
        print("Este proceso puede tardar bastante...\n")

        param_grid = {
            'n_estimators': [50, 100, 150],
            'max_depth': [2, 3],
            'learning_rate': [0.01, 0.05],
            'subsample': [0.4, 0.5, 0.6],
            'colsample_bytree': [0.3, 0.4, 0.5],
            'gamma': [1.0, 5.0, 10.0],
            'min_child_weight': [10, 20, 30],
            'reg_alpha': [5.0, 10.0, 20.0],
            'reg_lambda': [10.0, 20.0, 50.0],
            'grow_policy': ['depthwise'],
            'tree_method': [XGB_DEVICE_PARAMS['tree_method']],
            'random_state': [RANDOM_STATE],
        }

        if 'device' in XGB_DEVICE_PARAMS:
            param_grid['device'] = [XGB_DEVICE_PARAMS['device']]

        XGB_OPTIMAL_PARAMS_PER_TARGET = {}
        resultados_tuning = {}

```

```

global_start = time.time()

for idx, target_col in enumerate(TARGETS, 1):
    t0 = time.time()
    y_prueba = y_train[target_col].values

    buscador = RandomizedSearchCV(
        estimator=XGBRegressor(**XGB_DEVICE_PARAMS, random_state=RANDOM_STATE),
        param_distributions=param_grid,
        n_iter=150,
        scoring='r2',
        cv=5,
        verbose=0,
        random_state=RANDOM_STATE,
        n_jobs=N_JOBS
    )
    buscador.fit(X_train, y_prueba)

    params_target = dict(buscador.best_params_)
    params_target.update(XGB_DEVICE_PARAMS)
    params_target['random_state'] = RANDOM_STATE

    XGB_OPTIMAL_PARAMS_PER_TARGET[target_col] = params_target
    resultados_tuning[target_col] = {'params': params_target, 'r2': buscador.best_score_}

    print(f" [{idx:2d}/{len(TARGETS)}] {target_col:30} R2 CV: {buscador.best_score_:0.4f} | Tiempo: {time.time()-t0:>5.1f}s")

print(f"\nTuning por target completado en {(time.time() - global_start) / 60:.1f} minutos.\n")

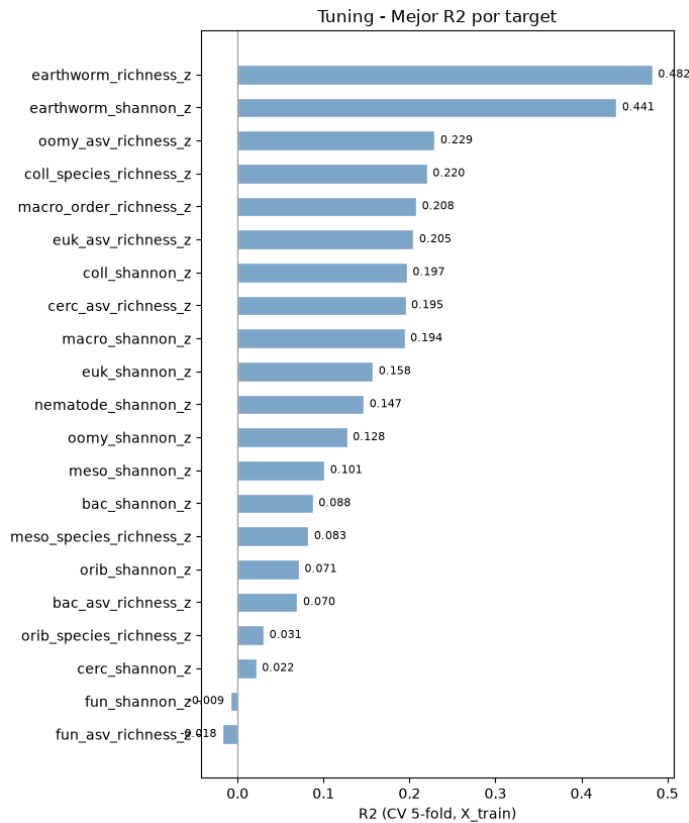
# Grafico: R2 de tuning por target (ordenado)
_tuning_r2 = {t: resultados_tuning[t]['r2'] for t in TARGETS}
_orden = sorted(_tuning_r2, key=_tuning_r2.get)
fig, ax = plt.subplots(figsize=(7, 0.35 * len(_orden) + 1))
_bars = ax.barh([t for t in _orden], [_tuning_r2[t] for t in _orden], color=PALETTE['blue'], edgecolor='white', height=0.6)
ax.bar_label(_bars, fmt='{:0.3f}', padding=4, fontsize=8)
ax.set_xlabel('R2 (CV 5-fold, X_train)')
ax.set_title('Tuning - Mejor R2 por target', fontsize=12)
ax.axvline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()

```

Iniciando búsqueda de hiperparametros (Tuning) para cada target...
 Este proceso puede tardar bastante...

[1/21]	nematode_shannon_z	R2 CV: 0.1469	Tiempo: 45.8s
[2/21]	macro_shannon_z	R2 CV: 0.1944	Tiempo: 43.0s
[3/21]	earthworm_shannon_z	R2 CV: 0.4408	Tiempo: 45.0s
[4/21]	orib_shannon_z	R2 CV: 0.0714	Tiempo: 43.0s
[5/21]	meso_shannon_z	R2 CV: 0.1011	Tiempo: 43.0s
[6/21]	coll_shannon_z	R2 CV: 0.1971	Tiempo: 44.3s
[7/21]	bac_shannon_z	R2 CV: 0.0877	Tiempo: 42.4s
[8/21]	fun_shannon_z	R2 CV: -0.0090	Tiempo: 42.1s
[9/21]	euk_shannon_z	R2 CV: 0.1576	Tiempo: 42.6s
[10/21]	oomy_shannon_z	R2 CV: 0.1275	Tiempo: 43.6s
[11/21]	cerc_shannon_z	R2 CV: 0.0218	Tiempo: 42.3s
[12/21]	macro_order_richness_z	R2 CV: 0.2080	Tiempo: 43.1s
[13/21]	earthworm_richness_z	R2 CV: 0.4825	Tiempo: 45.5s
[14/21]	orib_species_richness_z	R2 CV: 0.0305	Tiempo: 43.0s
[15/21]	meso_species_richness_z	R2 CV: 0.0825	Tiempo: 42.5s
[16/21]	coll_species_richness_z	R2 CV: 0.2204	Tiempo: 44.5s
[17/21]	bac_asv_richness_z	R2 CV: 0.0697	Tiempo: 42.4s
[18/21]	fun_asv_richness_z	R2 CV: -0.0178	Tiempo: 42.0s
[19/21]	euk_asv_richness_z	R2 CV: 0.2047	Tiempo: 43.4s
[20/21]	oomy_asv_richness_z	R2 CV: 0.2295	Tiempo: 44.5s
[21/21]	cerc_asv_richness_z	R2 CV: 0.1954	Tiempo: 44.1s

Tuning por target completado en 15.2 minutos.



4.- Validación cruzada

Se evalúa la robustez del modelo mediante **validación cruzada repetida** (5 pliegues x 3 repeticiones = 15 evaluaciones) aplicada sobre `X_train`.

Esto permite detectar sobreajuste comparando el rendimiento en entrenamiento y en los pliegues de validación.

```
In [5]: print('Validacion cruzada repetida (15 evaluaciones sobre X_train, por target)... \n')

cv_splitter = RepeatedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
metricas = ['r2', 'neg_root_mean_squared_error', 'neg_mean_absolute_error']

print(f'{{"TARGET":<25}} | {{"TRAIN R2":<10}} | {{"CV R2":<10}} | {{"DIFF":<10}} | {{"ESTADO"}}}')
print("-" * 75)

for nombre_target in TARGETS:
    y_prueba = y_train[nombre_target].values
    modelo_evaluacion = XGBRegressor(**XGB_OPTIMAL_PARAMS_PER_TARGET[nombre_target])

    resultados_examen = cross_validate(
        estimator=modelo_evaluacion,
        X=X_train,
        y=y_prueba,
        cv=cv_splitter,
        scoring=metricas,
        return_train_score=True
    )

    r2_entrenamiento = np.mean(resultados_examen['train_r2'])
    r2_examen_real = np.mean(resultados_examen['test_r2'])
    diferencia = r2_entrenamiento - r2_examen_real

    estado = 'Aviso: Posible sobreajuste' if diferencia > 0.15 else 'OK'

    print(f'{{nombre_target:<25}} | {{r2_entrenamiento:<10.3f}} | {{r2_examen_real:<10.3f}} | {{diferencia:<10.3f}} | {{estado}}')
```

Validacion cruzada repetida (15 evaluaciones sobre X_train, por target)...

TARGET	TRAIN R2	CV R2	DIFF	ESTADO
nematode_shannon_z	0.395	0.175	0.221	Aviso: Posible sobreajuste
macro_shannon_z	0.412	0.193	0.219	Aviso: Posible sobreajuste
earthworm_shannon_z	0.588	0.398	0.190	Aviso: Posible sobreajuste
orib_shannon_z	0.268	0.079	0.189	Aviso: Posible sobreajuste
meso_shannon_z	0.308	0.092	0.216	Aviso: Posible sobreajuste
coll_shannon_z	0.459	0.179	0.281	Aviso: Posible sobreajuste
bac_shannon_z	0.315	0.007	0.308	Aviso: Posible sobreajuste
fun_shannon_z	0.140	0.003	0.138	OK
euk_shannon_z	0.365	0.139	0.226	Aviso: Posible sobreajuste
oomy_shannon_z	0.392	0.121	0.271	Aviso: Posible sobreajuste
cerc_shannon_z	0.236	0.016	0.219	Aviso: Posible sobreajuste
macro_order_richness_z	0.401	0.194	0.206	Aviso: Posible sobreajuste
earthworm_richness_z	0.621	0.471	0.150	Aviso: Posible sobreajuste
orib_species_richness_z	0.179	0.060	0.118	OK
meso_species_richness_z	0.270	0.067	0.204	Aviso: Posible sobreajuste
coll_species_richness_z	0.472	0.207	0.265	Aviso: Posible sobreajuste
bac_asv_richness_z	0.299	0.030	0.269	Aviso: Posible sobreajuste

fun_asv_richness_z	0.070	-0.017	0.086	OK
euk_asv_richness_z	0.434	0.213	0.221	Aviso: Posible sobreajuste
oomy_asv_richness_z	0.440	0.220	0.220	Aviso: Posible sobreajuste
cerc_asv_richness_z	0.432	0.223	0.209	Aviso: Posible sobreajuste

5.- Explicabilidad del modelo (SHAP)

Se utiliza SHAP (SHapley Additive exPlanations) para identificar que variables influyen más en las predicciones del modelo y en que dirección lo hacen.

El gráfico de puntos muestra las 10 variables más importantes:

- Las variables se ordenan de mayor a menor importancia global (de arriba a abajo).
- El color indica el valor de la variable (rojo = valor alto, azul = valor bajo).
- La posición horizontal indica el efecto sobre la predicción (derecha = aumenta el índice, izquierda = lo reduce).

```
In [6]: %matplotlib inline

print("Calculando valores SHAP sobre X_train...")

# Shannon
modelo_explicable_shannon = XGBRegressor(**XGB_OPTIMAL_PARAMS_PER_TARGET['earthworm_shannon_z'])
modelo_explicable_shannon.fit(X_train, y_train['earthworm_shannon_z'].values)

explainer_shannon = shap.TreeExplainer(modelo_explicable_shannon)
shap_values_shannon = explainer_shannon.shap_values(X_train)

print("Top 10 variables por importancia SHAP (earthworm_shannon_z):")
shap.summary_plot(shap_values_shannon, X_train, plot_type="dot", max_display=10)

# Richness
modelo_explicable_richness = XGBRegressor(**XGB_OPTIMAL_PARAMS_PER_TARGET['earthworm_richness_z'])
modelo_explicable_richness.fit(X_train, y_train['earthworm_richness_z'].values)

explainer_richness = shap.TreeExplainer(modelo_explicable_richness)
shap_values_richness = explainer_richness.shap_values(X_train)

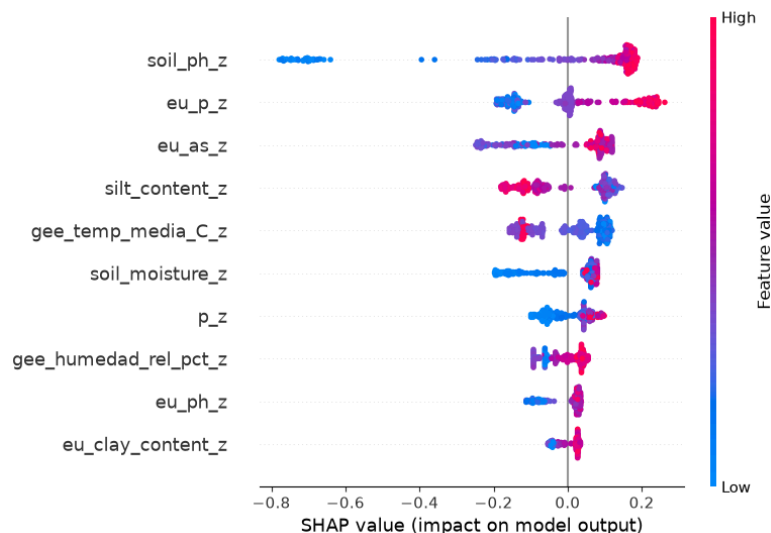
print("Top 10 variables por importancia SHAP (earthworm_richness_z):")
shap.summary_plot(shap_values_richness, X_train, plot_type="dot", max_display=10)
# Grafico: barplot importancia SHAP media (shannon + richness)
shap_mean = np.mean([
    np.abs(shap_values_shannon),
    np.abs(shap_values_richness)
], axis=0).mean(axis=0)

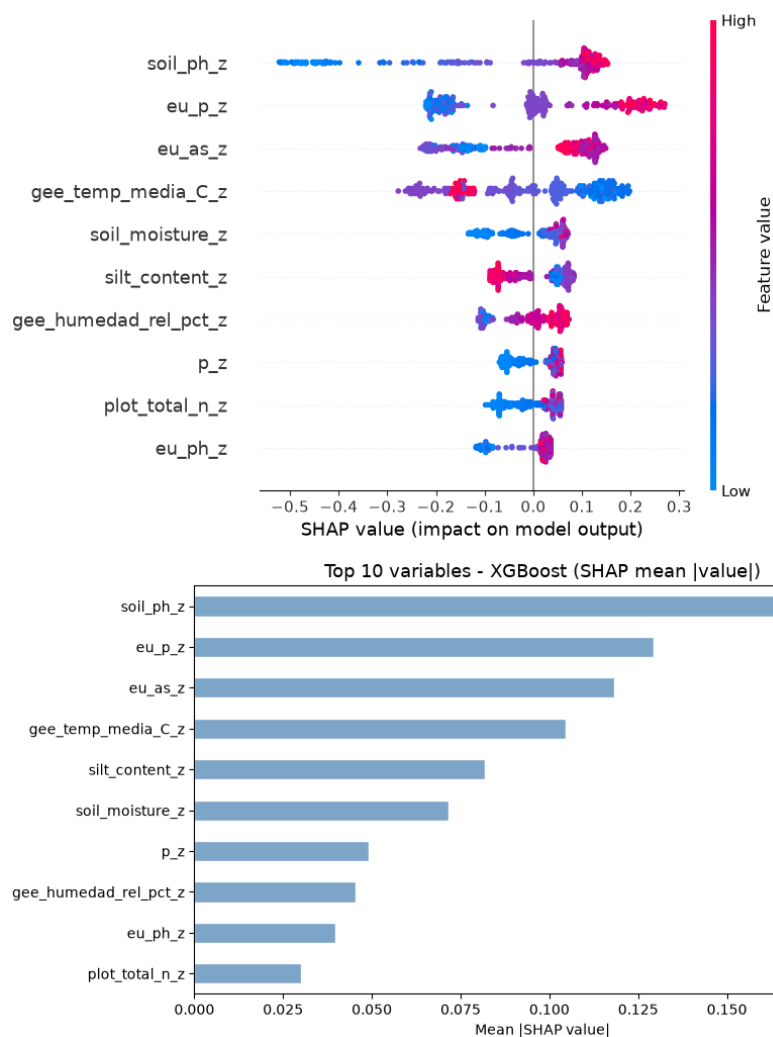
shap_importance = pd.Series(shap_mean, index=FEATURES_AUTORIZADAS).nlargest(10).sort_values()

fig, ax = plt.subplots(figsize=(9, 5))
shap_importance.plot(kind='barh', ax=ax, color=PALETTE['blue'], edgecolor='white')
ax.set_title('Top 10 variables - XGBoost (SHAP mean |value|)', fontsize=13)
ax.set_xlabel('Mean |SHAP value|')
plt.tight_layout()
plt.show()
```

Calculando valores SHAP sobre X_train...

Top 10 variables por importancia SHAP (earthworm_shannon_z):





6.- Entrenamiento

Se entrena un ensamble de 5 modelos por cada variable objetivo, variando la semilla aleatoria (semilla + n_modelo).

Esto complementa la aleatoridad intrínseca de Random Forest y reduce la varianza de las predicciones finales.

Todos los modelos se guardan en disco en formato .pkl.

```
In [7]: print("Entrenamiento de produccion (ensamble de 5 modelos por target)...")
print("-" * 70)

global_start_time = time.time()
NUM_EXPERTOS = 5

for idx, target_col in enumerate(TARGETS, 1):
    target_start_time = time.time()
    y = y_train[target_col].values

    for experto_id in range(NUM_EXPERTOS):
        parametros_experto = XGB_OPTIMAL_PARAMS_PER_TARGET[target_col].copy()
        parametros_experto['random_state'] = RANDOM_STATE + experto_id

        model = XGBRegressor(**parametros_experto)
        model.fit(X_train, y)

        pkl_path = os.path.join(MODELS_DIR, f"xgb_prod_{target_col}_exp{experto_id}.pkl")
        joblib.dump(model, pkl_path)

    elapsed = time.time() - target_start_time
    print(f" [{idx}/{len(TARGETS)}] {target_col:30} completado ({elapsed:.1f}s)")

print("-" * 70)
print(f"Produccion completada en {(time.time() - global_start_time)/60:.1f} minutos.")
print(f"Modelos guardados: {len(TARGETS) * NUM_EXPERTOS} ({NUM_EXPERTOS} por target).")

Entrenamiento de produccion (ensamble de 5 modelos por target)...
-----
[1/21] nematode_shannon_z      completado (0.3s)
[2/21] macro_shannon_z        completado (0.3s)
[3/21] earthworm_shannon_z    completado (0.3s)
[4/21] orib_shannon_z         completado (0.2s)
[5/21] meso_shannon_z         completado (0.3s)
[6/21] coll_shannon_z         completado (0.3s)
[7/21] bac_shannon_z          completado (0.3s)
```

```
[8/21] fun_shannon_z      completado (0.3s)
[9/21] euk_shannon_z      completado (0.3s)
[10/21] oomy_shannon_z    completado (0.2s)
[11/21] cerc_shannon_z    completado (0.3s)
[12/21] macro_order_richness_z completado (0.3s)
[13/21] earthworm_richness_z completado (0.3s)
[14/21] orib_species_richness_z completado (0.2s)
[15/21] meso_species_richness_z completado (0.3s)
[16/21] coll_species_richness_z completado (0.3s)
[17/21] bac_asv_richness_z completado (0.3s)
[18/21] fun_asv_richness_z completado (0.1s)
[19/21] euk_asv_richness_z completado (0.3s)
[20/21] oomy_asv_richness_z completado (0.2s)
[21/21] cerc_asv_richness_z completado (0.2s)
```

 Produccion completada en 0.1 minutos.
 Modelos guardados: 105 (5 por target).

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretacion del indice Shannon H':

El modelo predice el indice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biologica).

```
In [8]: NUM_EXPERTOS = 5

# Evaluacion final sobre todos los targets
# Usamos eval.csv, que el modelo no ha visto hasta ahora.

print('Evaluacion final sobre eval.csv')
print('-' * 65)
print(f' {"Target":30} {"R2":>8} {"RMSE":>8} {"MAE":>8}')
print('-' * 65)

for test_target in TARGETS:
    # Cargamos los 5 modelos del target actual y promediamos sus predicciones
    preds_eval = []
    for experto_id in range(NUM_EXPERTOS):
        ruta = os.path.join(MODELS_DIR, f'xgb_prod_{test_target}_exp{experto_id}.pkl')
        model = joblib.load(ruta)
        preds_eval.append(model.predict(X_eval))

    y_pred_eval = np.mean(preds_eval, axis=0)
    y_true_eval = y_eval[test_target].values

    r2 = r2_score(y_true_eval, y_pred_eval)
    rmse = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval))
    mae = mean_absolute_error(y_true_eval, y_pred_eval)
    print(f' {"test_target":30} {"r2":8.4f} {"rmse":8.4f} {"mae":8.4f}')

print('-' * 65)

# Smoke test con datos reales
# Se prueba sobre dos targets representativos: uno shannon y uno richness.

print('Smoke test - Inferencia con datos reales')
print('-' * 65)

try:
    # Tomamos 3 filas reales al azar del dataset de evaluacion
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    for test_target in ['earthworm_shannon_z', 'earthworm_richness_z']:
        print(f'\n Target: {test_target}')
        print(' ' + '-' * 63)

        valores_reales = y_eval.loc[indices_elegidos, test_target].values

        # Cada uno de los 5 modelos vota con su propia predicción para las 3 parcelas
        votos = []
        for experto_id in range(NUM_EXPERTOS):
            ruta = os.path.join(MODELS_DIR, f'xgb_prod_{test_target}_exp{experto_id}.pkl')
            m = joblib.load(ruta)
            pred = m.predict(df_new_data)
            votos.append(pred)
            print(f' Modelo {experto_id+1} -> '
                  f'Fila {indices_elegidos[0]}: {pred[0]:.4f} | '
                  f'Fila {indices_elegidos[1]}: {pred[1]:.4f} | '
                  f'Fila {indices_elegidos[2]}: {pred[2]:.4f}')

        # Promediamos los 5 votos para obtener la predicción final de consenso
        consenso = np.mean(votos, axis=0)

        print(' ' + '-' * 63)
        print(' Resultados detallados por muestra evaluada:')
        print(' ' + '-' * 63)

        for i, idx_muestra in enumerate(indices_elegidos):
            pred_val = consenso[i]
            real_val = valores_reales[i]
            error = abs(pred_val - real_val)
            print(f' Fila {idx_muestra}<4} | Predicción: {pred_val:.4f} | '
                  f'Valor Real: {real_val:.4f} | Error absoluto: {error:.4f}')

        print('\nSmoke test superado. El pipeline de XGBoost funciona y predice con coherencia.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: scatter predicción vs valor real (targets principales)
```

```

_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, tgt, col in zip(axes, _targets_plot, _colors_plot):
    preds = []
    for experto_id in range(NUM_EXPERTOS):
        ruta = os.path.join(MODELS_DIR, f'xgb_prod_{tgt}_exp{experto_id}.pkl')
        preds.append(joblib.load(ruta).predict(X_eval))
    yp = np.mean(preds, axis=0)
    yt = y_eval[tgt].values
    ax.scatter(yt, yp, alpha=0.7, s=30, color=col, edgecolors='white', linewidths=0.4)
    lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]
    ax.plot(lim, lim, 'r--', linewidth=1)
    ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)
    ax.set_xlabel('Valor real', fontsize=8)
    ax.set_ylabel('Prediccion', fontsize=8)
plt.suptitle('Prediccion vs Valor Real - XGBoost (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()

```

Evaluacion final sobre eval.csv

Target	R2	RMSE	MAE
nematode_shannon_z	0.1637	0.9032	0.7152
macro_shannon_z	0.2601	0.8639	0.7383
earthworm_shannon_z	0.4946	0.7096	0.5336
orib_shannon_z	0.1147	1.0760	0.9105
meso_shannon_z	-0.1868	1.0639	0.8782
coll_shannon_z	-0.3312	0.7309	0.5941
bac_shannon_z	0.0092	0.9968	0.6707
fun_shannon_z	-0.0206	1.1559	0.9385
euk_shannon_z	0.0912	0.9245	0.6622
oomy_shannon_z	0.0965	0.9937	0.7411
cerc_shannon_z	-0.0414	0.9138	0.6353
macro_order_richness_z	0.3082	0.7617	0.6447
earthworm_richness_z	0.5171	0.6346	0.4842
orib_species_richness_z	0.1259	1.1186	0.7501
meso_species_richness_z	-0.0592	0.9927	0.7503
coll_species_richness_z	-0.5803	0.6472	0.5071
bac_asv_richness_z	0.0014	1.0980	0.8196
fun_asv_richness_z	-0.0242	1.0907	0.8221
euk_asv_richness_z	0.1671	0.7409	0.5285
oomy_asv_richness_z	0.1776	0.9108	0.7459
cerc_asv_richness_z	0.1227	0.9241	0.7506

Smoke test - Inferencia con datos reales

Target: earthworm_shannon_z

Modelo 1 → Fila 53: 0.5515 | Fila 60: 0.7437 | Fila 0: -0.3029
 Modelo 2 → Fila 53: 0.6079 | Fila 60: 0.7023 | Fila 0: -0.3698
 Modelo 3 → Fila 53: 0.5115 | Fila 60: 0.6923 | Fila 0: -0.3004
 Modelo 4 → Fila 53: 0.5058 | Fila 60: 0.6708 | Fila 0: -0.3637
 Modelo 5 → Fila 53: 0.5695 | Fila 60: 0.7009 | Fila 0: -0.3502

Resultados detallados por muestra evaluada:

Fila 53 | Prediccion: 0.5492 | Valor Real: 1.1347 | Error absoluto: 0.5854
 Fila 60 | Prediccion: 0.7020 | Valor Real: 1.1388 | Error absoluto: 0.4368
 Fila 0 | Prediccion: -0.3374 | Valor Real: -0.7453 | Error absoluto: 0.4079

Target: earthworm_richness_z

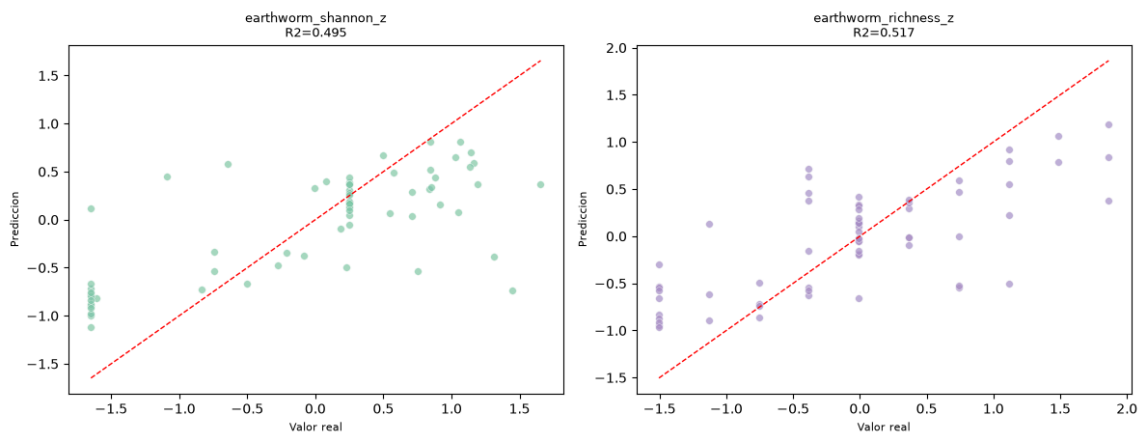
Modelo 1 → Fila 53: 0.8415 | Fila 60: 1.0071 | Fila 0: -0.4681
 Modelo 2 → Fila 53: 0.6864 | Fila 60: 0.9208 | Fila 0: -0.5674
 Modelo 3 → Fila 53: 0.8310 | Fila 60: 0.9784 | Fila 0: -0.4710
 Modelo 4 → Fila 53: 0.8203 | Fila 60: 0.8421 | Fila 0: -0.5175
 Modelo 5 → Fila 53: 0.7490 | Fila 60: 0.8783 | Fila 0: -0.4302

Resultados detallados por muestra evaluada:

Fila 53 | Prediccion: 0.7856 | Valor Real: 1.4889 | Error absoluto: 0.7032
 Fila 60 | Prediccion: 0.9254 | Valor Real: 1.1147 | Error absoluto: 0.1893
 Fila 0 | Prediccion: -0.4909 | Valor Real: -0.7562 | Error absoluto: 0.2654

Smoke test superado. El pipeline de XGBoost funciona y predice con coherencia.

Prediccion vs Valor Real - XGBoost (eval.csv)



11.2.6. Modelos XGBoost multisalida

11.2.6.1. Fundamento y configuración

Disponible desde la versión 1.7 de XGBoost mediante el parámetro `multi_strategy='multi_output_tree'`, esta variante construye, en cada iteración del *boosting*, un único árbol que predice los 21 targets a la vez, en lugar del comportamiento por defecto de XGBoost. Al igual que en Random Forest multisalida, esto permite que las divisiones del árbol capturen relaciones compartidas entre targets directamente en su estructura.

Comparte la misma celda de detección de GPU/CUDA que la variante de salida única, y al no poder tener un espacio de hiperparámetros distinto por target, sigue la misma estrategia de consenso descrita para Random Forest multisalida: búsqueda por target seguida de una combinación ponderada por R^2 .

Al igual que en las variantes de salida única, se entrena un ensamblado de varios modelos (5 expertos) para reducir la varianza de las predicciones.

11.2.6.2. Entrenamiento

La validación cruzada global sobre `X_train` muestra una diferencia Train-CV de 0.5423, muy por encima del umbral de aviso de 0.15 y notablemente mayor que la obtenida por Random Forest multisalida (0.166) para el mismo tipo de evaluación global.

Esto indica que, a pesar de la regularización aplicada en el espacio de búsqueda, el mecanismo de boosting de XGBoost combinado con la estrategia `multi_output_tree` tiende a memorizar con más facilidad el conjunto de entrenamiento cuando debe servir a los 21 targets a la vez, en comparación con un bosque de Random Forest equivalente.

11.2.6.3. Explicabilidad y variables más relevantes

A diferencia de la variante de salida única, donde SHAP solo se calculaba sobre los dos targets prioritarios, aquí sí se dispone de un ranking de importancia global.

El top-10 por importancia de permutación es: `soil_ph_z`, `p_z`, `gee_temp_media_C_z`, `dem_elevacion_m_z`, `soil_moisture_z`, `eu_as_z`, `eu_p_z`, `k_z`, `gee_humedad_rel_pct_z` y `dem_orientacion_deg_z`.

El ranking por ganancia intrínseca de XGBoost (XGBoost gain) coincide en gran medida con el de permutación para las primeras posiciones, situando también a `soil_ph_z` en primer lugar, lo que refuerza, junto con el resto de modelos de este anexo, la robustez de esta variable como el predictor individual más influyente de todo el trabajo.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Importancia de variables (SHAP)**.

11.2.6.4. Resultados sobre eval.csv

Con un R^2 medio de **0.0835**, XGBoost multisalida queda en segunda posición del ranking global de R^2 medio de este trabajo, por detrás de Random Forest multisalida (0.0964) pero por delante de RegressorChain (0.0809) y de XGBoost de salida única (0.067).

Sobre los targets prioritarios obtiene sus mejores resultados dentro de la familia XGBoost: $R^2=0.5375$ (earthworm_shannon_z) y $R^2=0.5893$ (earthworm_richness_z), este último el segundo mejor resultado de todo el trabajo para earthworm_richness_z, solo por detrás de RegressorChain (0.5359).

Variable	Target	R^2	RMSE	MAE
Shannon nemátodos	nematode_shannon_z	0.2228	0.8707	0.6737
Shannon macrofauna	macro_shannon_z	0.3771	0.7927	0.6529
Shannon lombrices	earthworm_shannon_z	0.5375	0.6788	0.4725
Shannon oribátidos	orib_shannon_z	0.2275	1.0051	0.8385
Shannon mesostigmátidos	meso_shannon_z	-0.2861	1.1075	0.8883
Shannon colémbolos	coll_shannon_z	-0.3229	0.7286	0.5647
Shannon bacterias	bac_shannon_z	-0.0151	1.0089	0.6857
Shannon hongos	fun_shannon_z	-0.0502	1.1725	0.9306
Shannon eucariotas	euk_shannon_z	0.0886	0.9259	0.6549
Shannon oomicetos	oomy_shannon_z	0.0620	1.0124	0.7385
Shannon cercozoos	cerc_shannon_z	-0.0330	0.9101	0.6371
Riqueza macrofauna	macro_order_richness_z	0.4013	0.7086	0.5880
Riqueza lombrices	earthworm_richness_z	0.5893	0.5852	0.4240
Riqueza oribátidos	orib_species_richness_z	0.2800	1.0152	0.6813
Riqueza mesostigmátidos	meso_species_richness_z	-0.1146	1.0183	0.7689
Riqueza colémbolos	coll_species_richness_z	-0.6465	0.6606	0.4997
Riqueza bacterias (ASV)	bac_asv_richness_z	0.0155	1.0902	0.8042
Riqueza hongos (ASV)	fun_asv_richness_z	-0.0662	1.1129	0.8448
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.2233	0.7155	0.5091
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.1650	0.9178	0.7293
Riqueza cercozoos (ASV)	cerc_asv_richness_z	0.0983	0.9369	0.7733

Tabla 19: Resultados de XGBoost multisalida (ensamble) sobre eval.csv, por target.

SOB4ES - XGBoost Multisalida

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook implementa **XGBoost con salida vectorial**, disponible desde la version 1.7 mediante el parámetro `multi_strategy='multi_output_tree'`.

A diferencia del enfoque individual (un modelo por target), aquí un único árbol de decisión en cada iteración del *boosting* aprende a predecir los 21 targets simultáneamente.

Esto permite capturar correlaciones entre grupos biológicos de forma más directa que entrenar modelos separados.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning:** búsqueda del valor óptimo de los hiperparámetros.
4. **Validación cruzada:** evaluación de robustez.
5. **Explicabilidad (SHAP) / Importancia de variables:** análisis de importancia de variables.
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
import seaborn as sns
from pathlib import Path

import xgboost
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV, cross_validate, RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance

import itertools
from matplotlib.colors import LinearSegmentedColormap

import matplotlib.pyplot as plt
import shap

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/xgb_multi/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
```

```

'sand_content_z',
'aggregate_stability_z',
'bulk_density_z',
'soil_moisture_z',
'as_z',
'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevation_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD8F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#989898',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Librerías y variables cargadas correctamente...

```

In [2]: # Deteccion de GPU / CUDA

# Comprueba si hay una GPU NVIDIA disponible y, en ese caso, configura
# XGBoost para usarla automaticamente. Si no hay GPU (o XGBoost no fue
# compilado con soporte CUDA), se usa la CPU sin que falle nada.

import subprocess

def _detectar_cuda():
    """Comprueba si hay una GPU NVIDIA con drivers CUDA disponibles en el sistema."""
    try:
        resultado = subprocess.run(
            ['nvidia-smi'],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            timeout=5,
        )
        return resultado.returncode == 0
    except (FileNotFoundError, subprocess.TimeoutExpired):
        return False

def _xgb_soporta_device():
    """XGBoost ≥ 2.0 usa el parametro 'device'; las versiones anteriores
    activan la GPU unicamente con tree_method='gpu_hist'."""
    try:
        return int(xgboost.__version__.split('.')[0]) ≥ 2
    except Exception:
        return False

CUDA_DISPONIBLE = _detectar_cuda()

if CUDA_DISPONIBLE and _xgb_soporta_device():
    # XGBoost ≥ 2.0: GPU via tree_method='hist' + device='cuda'
    XGB_DEVICE_PARAMS = {'tree_method': 'hist', 'device': 'cuda'}
elif CUDA_DISPONIBLE:
    # XGBoost < 2.0: GPU via tree_method='gpu_hist'
    XGB_DEVICE_PARAMS = {'tree_method': 'gpu_hist'}
else:
    XGB_DEVICE_PARAMS = {'tree_method': 'hist'}

```

```

print(f'GPU NVIDIA / CUDA detectada : {CUDA_DISPONIBLE}')
print(f'Version de XGBoost : {xgboost.__version__}')
print(f'Parametros de dispositivo : {XGB_DEVICE_PARAMS}')
if CUDA_DISPONIBLE:
    print('XGBoost usara la GPU para el tuning, la validacion y el entrenamiento de produccion.')
else:
    print('No se detecto GPU. XGBoost usara CPU (tree_method=\'hist\').')

# Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos `reserva` nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

GPU NVIDIA / CUDA detectada : True
Version de XGBoost : 3.3.0
Parametros de dispositivo : {'tree_method': 'hist', 'device': 'cuda'}
XGBoost usara la GPU para el tuning, la validacion y el entrenamiento de produccion.
Nucleos disponibles: 20 | n_jobs asignado: 15

```

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```

In [3]: def cargar_csv(ruta, nombre):
        if not os.path.exists(ruta):
            raise FileNotFoundError(f'No se encontro: {ruta}')
        try:
            df = pd.read_csv(ruta, sep=',')
            if len(df.columns) < 5:
                df = pd.read_csv(ruta, sep=';')
        except Exception:
            df = pd.read_csv(ruta, sep=';')
        print(f' {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas')
        return df

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados.")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

```

```

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

```

```

Datasets cargados.
X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)

```

3.- Búsqueda de hiperparámetros (Tuning)

El espacio de búsqueda es idéntico al del XGBoost individual.

El parámetro `multi_strategy='multi_output_tree'` activa la salida vectorial: en lugar de construir un árbol por target (comportamiento por defecto), construye un único árbol que predice todos los targets a la vez en cada iteración del *boosting*.

Al igual que con RF multisalida, el tuning usa un único target de referencia para el scoring y los parametros óptimos se aplican después al modelo completo multisalida.

```

In [4]: print("Iniciando busqueda de hiperparametros (Tuning)...")
        print("Este proceso puede tardar bastante...\n")

        param_grid = {
            'n_estimators': [50, 100, 150, 200, 300],
            'learning_rate': [0.01, 0.02, 0.03, 0.05],
            'max_depth': [2, 3, 4],
            'subsample': [0.5, 0.6, 0.7, 0.8],

```

```

'colsample_bytree': [0.5, 0.6, 0.7, 0.8],
'min_child_weight': [5, 10, 15, 20],
'gamma': [0.2, 0.5, 1.0, 2.0],
'reg_alpha': [0.5, 1.0, 2.0, 5.0, 10.0],
'reg_lambda': [1.0, 2.0, 5.0, 10.0, 20.0],
'scale_pos_weight': [1.0],
'tree_method': [XGB_DEVICE_PARAMS['tree_method']],
'random_state': [RANDOM_STATE],
'multi_strategy': ['multi_output_tree'],
}
if 'device' in XGB_DEVICE_PARAMS:
    param_grid['device'] = [XGB_DEVICE_PARAMS['device']]

resultados_tuning = {}

for idx, nombre_target in enumerate(TARGETS, 1):
    print(f"[{idx:2d}/{len(TARGETS)}] Buscando hiperparametros para: {nombre_target}")
    buscador = RandomizedSearchCV(
        estimator=XGBRegressor(**XGB_DEVICE_PARAMS, random_state=RANDOM_STATE),
        param_distributions=param_grid,
        n_iter=150,
        scoring='r2',
        cv=5,
        verbose=0,
        random_state=RANDOM_STATE,
        n_jobs= N_JOBS
    )
    start = time.time()
    buscador.fit(X_train, y_train[nombre_target].values)
    elapsed = (time.time() - start) / 60

    resultados_tuning[nombre_target] = {
        'params': buscador.best_params_,
        'r2': buscador.best_score_,
    }
    print(f" R2: {buscador.best_score_:.4f} | tiempo: {elapsed:.1f} min")

# Consenso de hiperparametros ponderado por R2 (ahora sobre los 21 targets)
r2_por_target = {t: max(resultados_tuning[t]['r2'], 0.0) for t in TARGETS}
suma_r2 = sum(r2_por_target.values())
if suma_r2 <= 0:
    pesos = {t: 1 / len(TARGETS) for t in TARGETS}
else:
    pesos = {t: v / suma_r2 for t, v in r2_por_target.items()}

print("\nCalculando consenso de hiperparametros (ponderado por R2 de los 21 targets)...")
for t in sorted(pesos, key=pesos.get, reverse=True)[:5]:
    print(f" Peso {t:30}: {pesos[t]:.3f}")
print(" ...")

def consenso_int(key, default=0):
    return int(round(sum(pesos[t] * resultados_tuning[t]['params'].get(key, default) for t in TARGETS)))

def consenso_float(key, default=0.0):
    return sum(pesos[t] * resultados_tuning[t]['params'].get(key, default) for t in TARGETS)

def consenso_cat(key):
    mejor_target = max(pesos, key=pesos.get)
    return resultados_tuning[mejor_target]['params'][key]

def consenso_log(key):
    return float(np.exp(sum(pesos[t] * np.log(resultados_tuning[t]['params'][key]) for t in TARGETS)))

XGB_MULTI_PARAMS = {
    'n_estimators': consenso_int('n_estimators'),
    'max_depth': consenso_int('max_depth'),
    'learning_rate': consenso_log('learning_rate'),
    'subsample': consenso_float('subsample'),
    'colsample_bytree': consenso_float('colsample_bytree'),
    'gamma': consenso_float('gamma', 0.0),
    'min_child_weight': consenso_int('min_child_weight'),
    'reg_alpha': consenso_float('reg_alpha', 0.0),
    'reg_lambda': consenso_float('reg_lambda', 1.0),
    'max_leaves': consenso_int('max_leaves', 0),
    'scale_pos_weight': 1.0,
    'random_state': RANDOM_STATE,
}
XGB_MULTI_PARAMS.update(XGB_DEVICE_PARAMS)

print("\nHiperparametros consenso finales:")
for k, v in XGB_MULTI_PARAMS.items():
    print(f" {k}: {v}")

# Grafico: R2 de tuning por target (ordenado)
_orden = sorted(r2_por_target, key=r2_por_target.get)
fig, ax = plt.subplots(figsize=(7, 0.35 * len(_orden) + 1))
_bars = ax.barh([t for t in _orden], [r2_por_target[t] for t in _orden], color=PALETTE['blue'], edgecolor='white', height=0.6)
ax.bar_label(_bars, fmt='{:.3f}', padding=4, fontsize=8)
ax.set_xlabel('R2 (CV 5-fold, X_train)')
ax.set_title('Tuning - Mejor R2 por target (referencia para consenso)', fontsize=12)
ax.axvline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show

```

Iniciando búsqueda de hiperparametros (Tuning)...

Este proceso puede tardar bastante...

```

[ 1/21] Buscando hiperparametros para: nematode_shannon_z
R2: 0.2175 | tiempo: 1.6 min
[ 2/21] Buscando hiperparametros para: macro_shannon_z
R2: 0.3285 | tiempo: 1.6 min
[ 3/21] Buscando hiperparametros para: earthworm_shannon_z

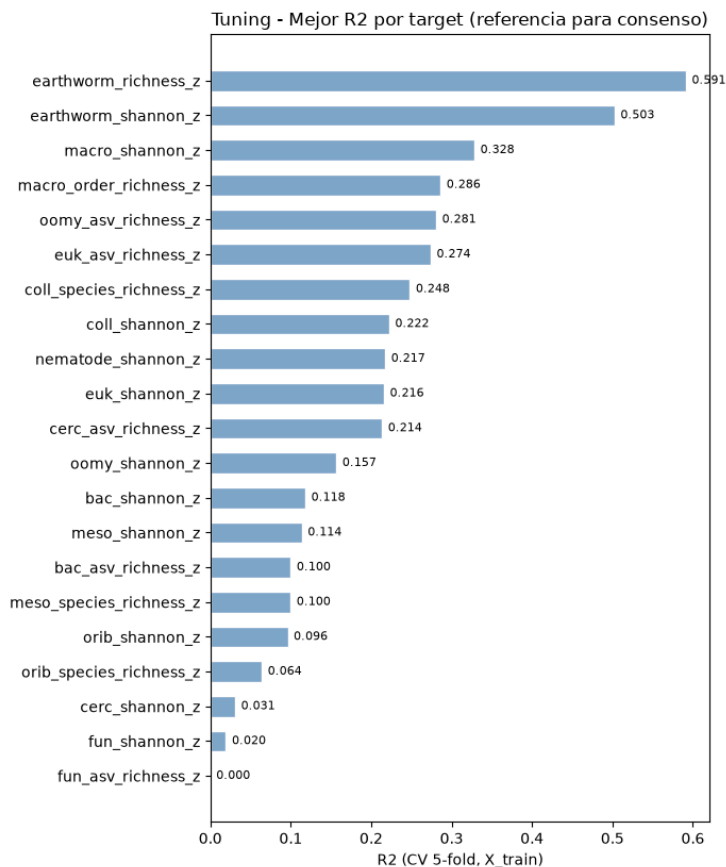
```

```
R2: 0.5027 | tiempo: 1.6 min
[ 4/21] Buscando hiperparametros para: orib_shannon_z
R2: 0.0963 | tiempo: 1.6 min
[ 5/21] Buscando hiperparametros para: meso_shannon_z
R2: 0.1138 | tiempo: 1.6 min
[ 6/21] Buscando hiperparametros para: coll_shannon_z
R2: 0.2224 | tiempo: 1.6 min
[ 7/21] Buscando hiperparametros para: bac_shannon_z
R2: 0.1182 | tiempo: 1.6 min
[ 8/21] Buscando hiperparametros para: fun_shannon_z
R2: 0.0195 | tiempo: 1.6 min
[ 9/21] Buscando hiperparametros para: euk_shannon_z
R2: 0.2160 | tiempo: 1.6 min
[10/21] Buscando hiperparametros para: oomy_shannon_z
R2: 0.1569 | tiempo: 1.6 min
[11/21] Buscando hiperparametros para: cerc_shannon_z
R2: 0.0309 | tiempo: 1.6 min
[12/21] Buscando hiperparametros para: macro_order_richness_z
R2: 0.2864 | tiempo: 1.6 min
[13/21] Buscando hiperparametros para: earthworm_richness_z
R2: 0.5914 | tiempo: 1.6 min
[14/21] Buscando hiperparametros para: orib_species_richness_z
R2: 0.0641 | tiempo: 1.5 min
[15/21] Buscando hiperparametros para: meso_species_richness_z
R2: 0.0999 | tiempo: 1.5 min
[16/21] Buscando hiperparametros para: coll_species_richness_z
R2: 0.2482 | tiempo: 1.6 min
[17/21] Buscando hiperparametros para: bac_asv_richness_z
R2: 0.1000 | tiempo: 1.6 min
[18/21] Buscando hiperparametros para: fun_asv_richness_z
R2: -0.0189 | tiempo: 1.6 min
[19/21] Buscando hiperparametros para: euk_asv_richness_z
R2: 0.2741 | tiempo: 1.6 min
[20/21] Buscando hiperparametros para: oomy_asv_richness_z
R2: 0.2809 | tiempo: 1.6 min
[21/21] Buscando hiperparametros para: cerc_asv_richness_z
R2: 0.2136 | tiempo: 1.6 min

Calculando consenso de hiperparametros (ponderado por R2 de los 21 targets)...
Peso earthworm_richness_z      : 0.141
Peso earthworm_shannon_z      : 0.120
Peso macro_shannon_z          : 0.079
Peso macro_order_richness_z    : 0.068
Peso oomy_asv_richness_z       : 0.067
...

Hiperparametros consenso finales:
n_estimators: 207
max_depth: 4
learning_rate: 0.03139066781580437
subsample: 0.6699267538405558
colsample_bytree: 0.7021338916961798
gamma: 0.4574244751880444
min_child_weight: 8
reg_alpha: 2.5026629693131883
reg_lambda: 5.3852575991816565
max_leaves: 0
scale_pos_weight: 1.0
random_state: 42
tree_method: hist
device: cuda

Out[4]: <function matplotlib.pyplot.show(close=None, block=None)>
```



4.- Validación cruzada

Se evalúa la robustez con **validación cruzada repetida** (5 pliegues x 3 repeticiones) sobre `X_train` con todos los targets a la vez.

Se muestra el R2 promedio global y el R2 individual por target.

```
In [5]: print('Validacion cruzada repetida sobre X_train (todos los targets)...\n')

cv_splitter = RepeatedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
metricas = ['r2', 'neg_root_mean_squared_error', 'neg_mean_absolute_error']

modelo_cv = XGBRegressor(**XGB_MULTI_PARAMS)

print('-' * 65)
print('Target: MULTI-OUTPUT (Promedio de todos los targets - XGBoost)')
print('-' * 65)

resultados = cross_validate(
    estimator=modelo_cv,
    X=X_train,
    y=y_train,
    cv=cv_splitter,
    scoring=metricas,
    return_train_score=True
)

r2_train = np.mean(resultados['train_r2'])
r2_cv = np.mean(resultados['test_r2'])
r2_std = np.std(resultados['test_r2'])
rmse_cv = np.mean(-resultados['test_neg_root_mean_squared_error'])
mae_cv = np.mean(-resultados['test_neg_mean_absolute_error'])

print(f' Train R2 : {r2_train:.4f}')
print(f' Validacion R2 (CV): {r2_cv:.4f} +/- {r2_std:.4f}')
print(f' RMSE global : {rmse_cv:.4f}')
print(f' MAE global : {mae_cv:.4f}')

diferencia = r2_train - r2_cv
if diferencia > 0.15:
    print(f' Aviso: diferencia train/val = {diferencia:.4f}. Posible sobreajuste.\n')
else:
    print(f' Diferencia train/val = {diferencia:.4f}. El modelo generaliza correctamente.\n')

# R2 individual por target (entrenando sobre todo X_train)
modelo_cv.fit(X_train, y_train)
y_pred_train = modelo_cv.predict(X_train)

print(' R2 individual por target (Evaluado en train):')
print('-' * 65)
for i, t in enumerate(TARGETS):
```

```
# Aseguramos compatibilidad si y_train es un DataFrame de Pandas
y_true_col = y_train.iloc[:, i] if hasattr(y_train, 'iloc') else y_train[:, i]
r2 = r2_score(y_true_col, y_pred_train[:, i])
print(f'      {t:30} {r2:.4f}')
print()
```

Validación cruzada repetida sobre X_train (todos los targets)...

Target: MULTI-OUTPUT (Promedio de todos los targets - XGBoost)

```
Train R2      : 0.7107
Validación R2 (CV): 0.1684 +/- 0.0353
RMSE global   : 0.8891
MAE global    : 0.6671
Aviso: diferencia train/val = 0.5423. Posible sobreajuste.
```

R2 individual por target (Evaluado en train):

```
-----
nematode_shannon_z      0.7163
macro_shannon_z         0.7496
earthworm_shannon_z     0.8076
orib_shannon_z          0.6940
meso_shannon_z          0.7039
coll_shannon_z          0.7387
bac_shannon_z           0.6569
fun_shannon_z           0.6849
euk_shannon_z           0.7066
oomy_shannon_z          0.7089
cerc_shannon_z          0.6363
macro_order_richness_z  0.7223
earthworm_richness_z    0.8368
orib_species_richness_z 0.6938
meso_species_richness_z 0.6752
coll_species_richness_z 0.7413
bac_asv_richness_z      0.6607
fun_asv_richness_z      0.6393
euk_asv_richness_z      0.7433
oomy_asv_richness_z     0.7386
cerc_asv_richness_z     0.7198
```

5.- Importancia de variables (SHAP)

XGBoost expone directamente la importancia de cada variable mediante `feature_importances_`, que en modo multisalida refleja el impacto global sobre todos los targets.

Se complementa con permutation importance y SHAP para una estimación mas fiable.

```
In [6]: print('Calculando importancia de variables...\n')

# 1.- Cálculo de importancias

# Importancia intrínseca de XGBoost (gain medio por variable)
imp_xgb = pd.Series(modelo_cv.feature_importances_, index=FEATURES_AUTORIZADAS)

# Permutation importance sobre X_train
print('Calculando Permutation Importance (esto puede tardar un poco)...\n')
perm = permutation_importance(
    modelo_cv, X_train, y_train,
    n_repeats=10, random_state=RANDOM_STATE, n_jobs=-1
)
imp_perm = pd.Series(perm.importances_mean, index=FEATURES_AUTORIZADAS)

# Unir en un DataFrame y extraer el Top 10
df_importancias = pd.DataFrame({
    'XGBoost gain': imp_xgb,
    'Permutación (R2)': imp_perm
})

TOP_N = 10
top_features = df_importancias.sort_values('Permutación (R2)', ascending=False).head(TOP_N)

print(f'Top {TOP_N} variables por importancia (Ordenado por Permutación):')
print(top_features.round(4).to_string())

# 2.- Gráficos de barras (Permutación vs XGBoost Gain)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Gráfico A: Permutación
sns.barplot(x=top_features['Permutación (R2)'], y=top_features.index, ax=axes[0], color=PALETTE['blue'])
axes[0].set_title('Importancia por Permutación (Reducción del R2)', fontsize=14)
axes[0].set_xlabel('Caída media en R2 al permutar la variable')
axes[0].set_ylabel('')

# Gráfico B: XGBoost Gain
top_features_gain = df_importancias.sort_values('XGBoost gain', ascending=False).head(TOP_N)
sns.barplot(x=top_features_gain['XGBoost gain'], y=top_features_gain.index, ax=axes[1], color=PALETTE['green'])
axes[1].set_title('Importancia Intrínseca (XGBoost Gain)', fontsize=14)
axes[1].set_xlabel('Ganancia media en los cortes de los árboles')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()

# 3.- Análisis SHAP (Shannon y Richness)
```

```
print('\nCalculando valores SHAP...')

# Lista de targets que queremos analizar con SHAP
targets_shap = ['earthworm_shannon_z', 'earthworm_richness_z']

for target in targets_shap:
    print(f"Variables más importantes para {target}")

    # Entrenamos un modelo rápido de un solo target con los mismos parámetros base
    modelo_shap = XGBRegressor(random_state=RANDOM_STATE, n_jobs=-1, **XGB_DEVICE_PARAMS)

    # Intentamos copiar los hiperparámetros de los modelos generados
    try:
        if hasattr(modelo_cv, 'get_params'):
            modelo_shap.set_params(**modelo_cv.get_params())
    except Exception:
        pass # Si falla al copiar, usamos los parámetros base

    # Entrenamos solo con el target actual
    modelo_shap.fit(X_train, y_train[target])

    # Calculamos SHAP
    explainer = shap.TreeExplainer(modelo_shap)
    shap_values = explainer.shap_values(X_train)

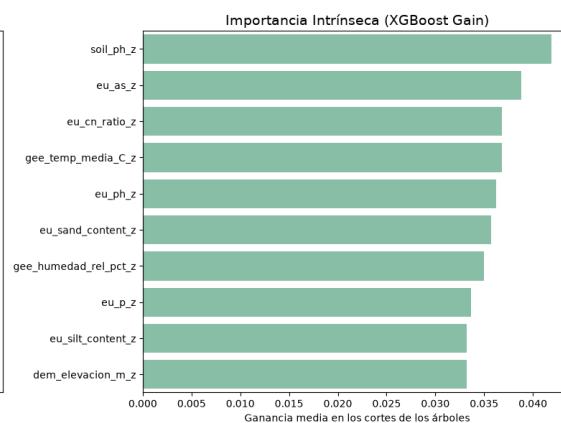
    # Generamos el gráfico
    plt.figure(figsize=(10, 6))
    shap.summary_plot(shap_values, X_train, plot_type='dot', max_display=TOP_N, show=True)
```

Calculando importancia de variables...

Calculando Permutation Importance (esto puede tardar un poco)...

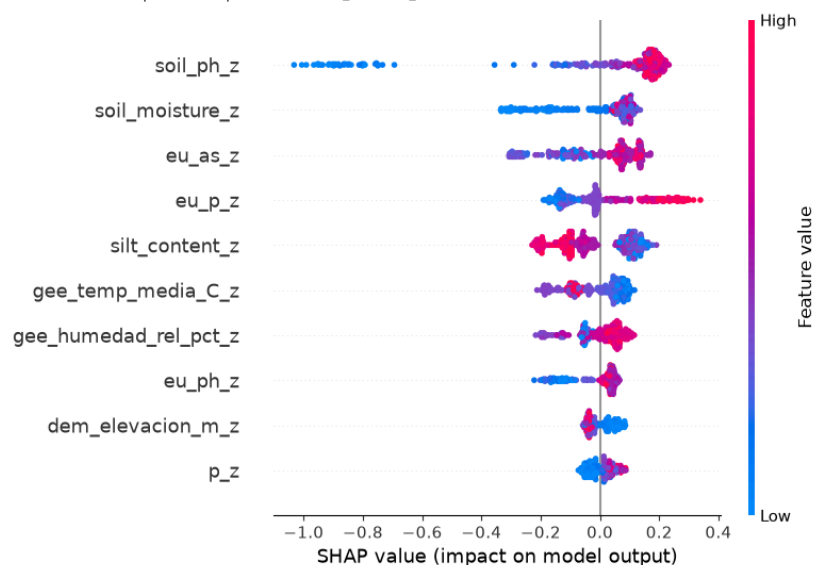
Top 10 variables por importancia (Ordenado por Permutación):

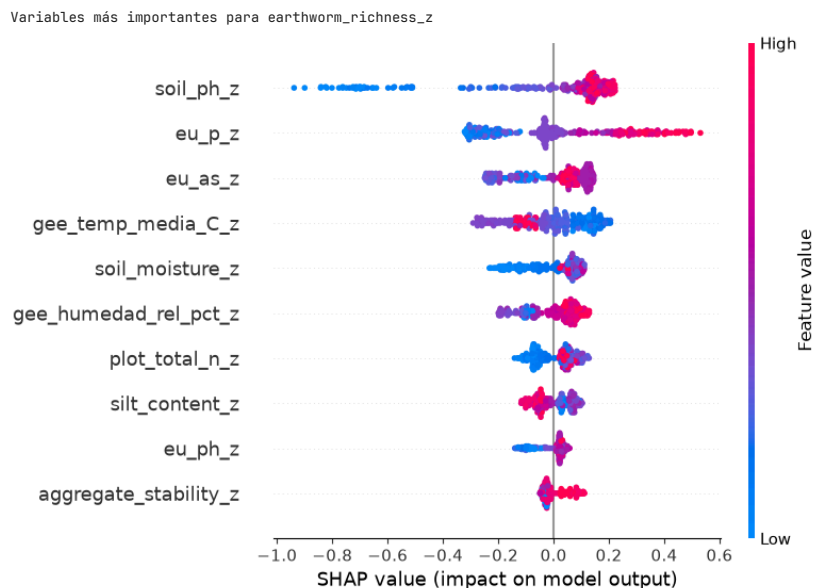
	XGBoost gain	Permutacion (R2)
soil_ph_z	0.0419	0.0717
p_z	0.0290	0.0438
gee_temp_media_C_z	0.0368	0.0412
dem_elevacion_m_z	0.0332	0.0336
soil_moisture_z	0.0285	0.0324
eu_as_z	0.0388	0.0312
eu_p_z	0.0337	0.0306
k_z	0.0268	0.0303
gee_humedad_rel_pct_z	0.0350	0.0294
dem_orientacion_deg_z	0.0231	0.0277



Calculando valores SHAP...

Variables más importantes para earthworm_shannon_z





6.- Entrenamiento

Se entrena un ensemble de 5 modelos XGBoost multisalida, variando la semilla aleatoria (semilla + n_modelo).

Cada modelo predice los 21 targets a la vez con `multi_strategy='multi_output_tree'`.

```
In [7]: print('Entrenamiento de produccion (ensemble de 5 modelos XGBoost multisalida)...')
print('-' * 70)

global_start = time.time()
NUM_EXPERTOS = 5

for exp_id in range(NUM_EXPERTOS):
    t0 = time.time()
    params = XGB_MULTI_PARAMS.copy()
    params['random_state'] = RANDOM_STATE + exp_id

    # Entrenamos sobre y_train completo (matriz 11 targets)
    model = XGBRegressor(**params)
    model.fit(X_train, y_train)

    joblib.dump(model, os.path.join(MODELS_DIR, f'xgb_multi_exp{exp_id}.pkl'))
    print(f' Modelo {exp_id+1}/5 completado ({time.time()-t0:.1f}s)')

print('-' * 70)
print(f'Produccion completada en {(time.time()-global_start)/60:.1f} minutos.')

Entrenamiento de produccion (ensemble de 5 modelos XGBoost multisalida)...
-----
Modelo 1/5 completado (1.9s)
Modelo 2/5 completado (1.9s)
Modelo 3/5 completado (1.9s)
Modelo 4/5 completado (1.9s)
Modelo 5/5 completado (1.9s)
-----
Produccion completada en 0.2 minutos.
```

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretación del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biológica).

```
In [8]: NUM_EXPERTOS = 5

# Evaluacion final
# Cargamos los 5 modelos expertos y promediamos sus predicciones sobre eval.csv.
print('Evaluacion final sobre eval.csv (Ensamble XGBoost)')
print('-' * 65)
print(f' {"Target":30} {"R2":>8} {"RMSE":>8} {"MAE":>8}')
print('-' * 65)

preds_eval = []
for exp_id in range(NUM_EXPERTOS):
    ruta = os.path.join(MODELS_DIR, f'xgb_multi_exp{exp_id}.pkl')
    model = joblib.load(ruta)
    preds_eval.append(model.predict(X_eval))

# y_pred_eval es una matriz (n_muestras x len(TARGETS) targets)
y_pred_eval = np.mean(preds_eval, axis=0)
y_true_eval = y_eval.values
```

```

# Metricas globales
r2_global = r2_score(y_true_eval, y_pred_eval, multioutput='uniform_average')
rmse_global = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval, multioutput='uniform_average'))
mae_global = mean_absolute_error(y_true_eval, y_pred_eval, multioutput='uniform_average')

print(f' {"6Global (Promedio)":30} {r2_global:8.4f} {rmse_global:8.4f} {mae_global:8.4f}')
print('-' * 65)

# Metricas individuales por target
r2_por_target = r2_score(y_true_eval, y_pred_eval, multioutput='raw_values')
for i, t in enumerate(TARGETS):
    rmse_t = np.sqrt(mean_squared_error(y_true_eval[:, i], y_pred_eval[:, i]))
    mae_t = mean_absolute_error(y_true_eval[:, i], y_pred_eval[:, i])
    print(f' {t:30} {r2_por_target[i]:8.4f} {rmse_t:8.4f} {mae_t:8.4f}')

print('-' * 65)

# Smoke test con datos reales
print('\nSmoke test - Inferencia con datos reales')
print('-' * 65)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    # Generamos y guardamos las predicciones de todos los modelos del ensemble
    votos = []
    for exp_id in range(NUM_EXPERTOS):
        ruta = os.path.join(MODELS_DIR, f'xgb_multi_exp{exp_id}.pkl')
        m = joblib.load(ruta)
        votos.append(m.predict(df_new_data))

    # Promediamos para obtener el consenso final del ensemble
    consenso = np.mean(votos, axis=0)

    targets_smoke = ['earthworm_shannon_z', 'earthworm_richness_z']

    for tgt in targets_smoke:
        tgt_idx = TARGETS.index(tgt)
        print(f'\n Target: {tgt}')
        print(' ' + '-' * 63)

        # Mostramos el voto de cada experto
        print(' Votos individuales por experto:')
        for exp_id in range(NUM_EXPERTOS):
            pred_experto = votos[exp_id]
            print(f' Modelo {exp_id+1} | '
                  f'Fila {indices_elegidos[0]}: {pred_experto[0, tgt_idx]:>7.4f} | '
                  f'Fila {indices_elegidos[1]}: {pred_experto[1, tgt_idx]:>7.4f} | '
                  f'Fila {indices_elegidos[2]}: {pred_experto[2, tgt_idx]:>7.4f}')

        print(' ' + '-' * 63)
        print(' Consenso final frente a valores reales:')
        for i, idx in enumerate(indices_elegidos):
            pred_val = consenso[i, tgt_idx]
            real_val = y_eval.loc[idx, tgt]
            error = abs(pred_val - real_val)
            print(f' Fila {idx:<4} | Consenso: {pred_val:>7.4f} | '
                  f'Valor Real: {real_val:>7.4f} | Error: {error:>6.4f}')

        print('\n' + '-' * 65)
        print('Smoke test superado. El pipeline XGBoost multisalida funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: scatter prediccion vs valor real (targets principales)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, tgt, col in zip(axes, _targets_plot, _colors_plot):
    idx_col = TARGETS.index(tgt)

    preds = []
    for exp_id in range(NUM_EXPERTOS):
        ruta = os.path.join(MODELS_DIR, f'xgb_multi_exp{exp_id}.pkl')
        preds.append(joblib.load(ruta).predict(X_eval))

    yp = np.mean(preds, axis=0)[:, idx_col]
    yt = y_eval[tgt].values

    ax.scatter(yt, yp, alpha=0.7, s=30, color=col, edgecolors='white', linewidths=0.4)
    lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]
    ax.plot(lim, lim, 'r--', linewidth=1)

    ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)
    ax.set_xlabel('Valor real', fontsize=8)
    ax.set_ylabel('Prediccion', fontsize=8)

plt.suptitle('Prediccion vs Valor Real - XGBoost Ensemble (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()

```

Evaluacion final sobre eval.csv (Ensamble XGBoost)

Target	R2	RMSE	MAE
--------	----	------	-----

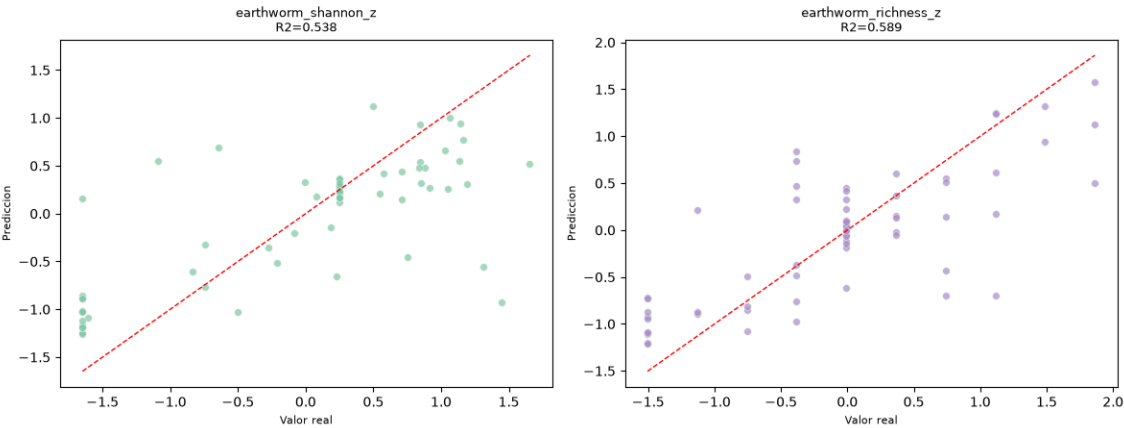
Global (Promedio)	0.0835	0.9187	0.6838
nematode_shannon_z	0.2228	0.8707	0.6737
macro_shannon_z	0.3771	0.7927	0.6529
earthworm_shannon_z	0.5375	0.6788	0.4725
orib_shannon_z	0.2275	1.0051	0.8385
meso_shannon_z	-0.2861	1.1075	0.8883
coll_shannon_z	-0.3229	0.7286	0.5647
bac_shannon_z	-0.0151	1.0089	0.6857
fun_shannon_z	-0.0502	1.1725	0.9306
euk_shannon_z	0.0886	0.9259	0.6549
oomy_shannon_z	0.0620	1.0124	0.7385
cerc_shannon_z	-0.0330	0.9101	0.6371
macro_order_richness_z	0.4013	0.7086	0.5880
earthworm_richness_z	0.5893	0.5852	0.4240
orib_species_richness_z	0.2800	1.0152	0.6813
meso_species_richness_z	-0.1146	1.0183	0.7689
coll_species_richness_z	-0.6465	0.6606	0.4997
bac_asv_richness_z	0.0155	1.0902	0.8042
fun_asv_richness_z	-0.0662	1.1129	0.8448
euk_asv_richness_z	0.2233	0.7155	0.5091
oomy_asv_richness_z	0.1650	0.9178	0.7293
cerc_asv_richness_z	0.0983	0.9369	0.7733

Smoke test - Inferencia con datos reales

Target: earthworm_shannon_z				
Votos individuales por experto:				
Modelo 1	Fila 53:	0.5139	Fila 60:	0.9741
Modelo 2	Fila 53:	0.5010	Fila 60:	0.8811
Modelo 3	Fila 53:	0.5463	Fila 60:	0.9226
Modelo 4	Fila 53:	0.5621	Fila 60:	0.9504
Modelo 5	Fila 53:	0.6387	Fila 60:	0.9726
Consenso final frente a valores reales:				
Fila 53	Consenso:	0.5524	Valor Real:	1.1347
Fila 60	Consenso:	0.9402	Valor Real:	1.1388
Fila 0	Consenso:	-0.3243	Valor Real:	-0.7453
Target: earthworm_richness_z				
Votos individuales por experto:				
Modelo 1	Fila 53:	0.8651	Fila 60:	1.1709
Modelo 2	Fila 53:	0.9415	Fila 60:	1.2170
Modelo 3	Fila 53:	0.9554	Fila 60:	1.3132
Modelo 4	Fila 53:	0.9588	Fila 60:	1.2461
Modelo 5	Fila 53:	0.9870	Fila 60:	1.2231
Consenso final frente a valores reales:				
Fila 53	Consenso:	0.9415	Valor Real:	1.4889
Fila 60	Consenso:	1.2341	Valor Real:	1.1147
Fila 0	Consenso:	-0.4974	Valor Real:	-0.7562

Smoke test superado. El pipeline XGBoost multisalida funciona correctamente.

Prediccion vs Valor Real - XGBoost Ensemble (eval.csv)



11.2.7. Modelos MLP mutlisalida

11.2.7.1. Fundamento y configuración

El perceptrón multicapa (véase **Redes neuronales**), implementado sobre PyTorch, representa el único enfoque de aprendizaje profundo de este trabajo.

La arquitectura (**MLPMultiSalida**) es una red densa totalmente conectada: cada capa oculta combina una transformación lineal, una activación no lineal y una capa de Dropout para regularización. La capa de salida es lineal y tiene 21 neuronas, una por target, compartiendo todas ellas las mismas capas ocultas.

Se entrena con el **optimizador Adam** y la **función de pérdida MSE estándar**, calculada conjuntamente sobre los 21 targets.

A diferencia de los modelos basados en árboles, aquí el tuning no busca solo hiperparámetros de entrenamiento sino también la propia arquitectura de la red.

Se evaluaron 6 configuraciones distintas, variando el tamaño de las capas ocultas (de [32, 16] a [64, 16, 8]), la tasa de dropout (0.2-0.4), la tasa de aprendizaje, el número de épocas y el `weight_decay` (regularización L2 del propio optimizador Adam).

La mejor configuración fue la siguiente:

Hiperparámetro	Valor
Capas ocultas	[128, 32]
dropout	0.3
lr	0.0005
epochs	250
batch_size	16
weight_decay	0.001

Tabla 20: Consenso de hiperparámetros en MLP Multisalida.

11.2.7.2. Entrenamiento

A diferencia de los modelos basados en árboles, este notebook no aplica una validación cruzada explícita tipo `RepeatedKFold`: dado el coste computacional de reentrenar una red neuronal por cada pliegue y cada una de las 6 configuraciones evaluadas, la selección de la arquitectura se apoya en la partición fija de `train.csv/test.csv`, monitorizando la curva de pérdida (entrenamiento frente a validación) por época para detectar sobreajuste.

Al ser un modelo determinista una vez fijada la semilla de inicialización de pesos, no se entrena un ensamblado de varias semillas como en Random Forest o XGBoost, sino un único modelo con la configuración ganadora.

11.2.7.3. Explicabilidad y variables más relevantes

Al no disponer las redes neuronales de una medida de importancia intrínseca como los árboles, se recurre a `permutation importance` sobre un modelo de referencia entrenado sobre todo `X_train`.

El top-10 resultante es el siguiente: `soil_ph_z`, `gee_temp_media_C_z`, `dem_elevacion_m_z`, `eu_p_z`, `eu_ph_z`, `gee_humedad_rel_pct_z`, `gee_ndvi_verano_z`, `silt_content_z`, `bulk_density_z`, `clay_content_z`.

Vuelve a situar a `soil_ph_z` y `gee_temp_media_C_z` en primer y segundo lugar, exactamente igual que en Ridge y en Random Forest, lo que refuerza aún más la robustez de ambas variables como predictores del problema, con independencia del tipo de modelo empleado.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Importancia de variables**.

11.2.7.4. Resultados sobre eval.csv

Con un R^2 medio de **-0.038**, el MLP multisalida se sitúa por debajo de los tres modelos basados en árboles y también por debajo del modelo Ridge.

Sobre los targets prioritarios obtiene $R^2=0.4264$ (earthworm_shannon_z) y $R^2=0.5176$ (earthworm_richness_z), resultados razonables a pesar de que el rendimiento global se ve penalizado por un desempeño muy negativo en varios targets minoritarios, en particular coll_species_richness_z ($R^2=-1.4628$), con diferencia el peor resultado individual obtenido por ningún modelo sobre ningún target en todo este trabajo.

Este comportamiento es coherente con la limitación señalada en la introducción del propio notebook: las redes neuronales necesitan, en general, más datos de entrenamiento para generalizar bien, especialmente en los targets con menos señal predictiva.

Variable	Target	R^2
Shannon nemátodos	nematode_shannon_z	0.1180
Shannon macrofauna	macro_shannon_z	0.2571
Shannon lombrices	earthworm_shannon_z	0.4264
Shannon oribátidos	orib_shannon_z	0.2356
Shannon mesostigmátidos	meso_shannon_z	-0.4898
Shannon colémbolos	coll_shannon_z	-0.8227
Shannon bacterias	bac_shannon_z	-0.0854
Shannon hongos	fun_shannon_z	-0.0757
Shannon eucariotas	euk_shannon_z	0.1453
Shannon oomicetos	oomy_shannon_z	0.1534
Shannon cercozoos	cerc_shannon_z	-0.1361
Riqueza macrofauna	macro_order_richness_z	0.3525
Riqueza lombrices	earthworm_richness_z	0.5176
Riqueza oribátidos	orib_species_richness_z	0.2596
Riqueza mesostigmátidos	meso_species_richness_z	-0.2753
Riqueza colémbolos	coll_species_richness_z	-1.4628
Riqueza bacterias (ASV)	bac_asv_richness_z	-0.1464
Riqueza hongos (ASV)	fun_asv_richness_z	0.0145
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.2164
Riqueza oomicetos (ASV)	oomy_asv_richness_z	0.0113
Riqueza cercozoos (ASV)	cerc_asv_richness_z	-0.0120

Tabla 21: R^2 del MLP multisalida sobre eval.csv, por target.

SOB4ES - Red Neuronal Multisalida (MLP)

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook implementa una **red neuronal de tipo MLP (Multilayer Perceptron)** con una capa de salida de 21 neuronas, una por cada target.

A diferencia de los modelos basados en árboles, la red neuronal aprende representaciones internas de los datos en capas sucesivas, lo que le permite capturar relaciones no lineales complejas entre las variables de entrada y los targets de salida. Las correlaciones entre targets se capturan de forma implícita al compartir las capas ocultas entre todas las salidas.

La principal limitación en este contexto es el **tamaño del dataset** (~300 muestras de entrenamiento): las redes neuronales generalmente necesitan más datos para generalizar bien. Se aplican técnicas de regularización (Dropout, L2) para mitigar el sobreajuste.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Tuning:** búsqueda del valor óptimo de los hiperparámetros, junto con la arquitectura más adecuada.
4. **Validación cruzada:** evaluación de robustez.
5. **Análisis de pesos e importancia de variables.**
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
from pathlib import Path
import itertools

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

from sklearn.model_selection import RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance
from sklearn.base import BaseEstimator, RegressorMixin

from matplotlib.colors import LinearSegmentedColormap
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/mlp/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

# Usar GPU si esta disponible, si no CPU

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo de computo: {DEVICE}')

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
```

```

'oomy_asv_richness_z',
'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
    'zn_z',
    'soil_ph_z',
    'plot_total_organic_c_z',
    'plot_total_n_z',
    'gee_temp_media_C_z',
    'gee_humedad_rel_pct_z',
    'gee_ndvi_verano_z',
    'dem_elevacion_m_z',
    'dem_pendiente_deg_z',
    'dem_orientacion_deg_z',
    'eu_clay_content_z',
    'eu_sand_content_z',
    'eu_silt_content_z',
    'eu_water_holding_capacity_z',
    'eu_cn_ratio_z',
    'eu_p_z',
    'eu_ph_z',
    'eu_as_z',
    'eu_organic_carbon_octop_z',
    'eu_zn_z'
]

print('Librerías y variables cargadas correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD0F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Dispositivo de computo: cuda
 Librerías y variables cargadas correctamente...

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
# Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)
import os

def get_n_jobs(fraccion=0.75, reserva=1):
    """Calcula un n_jobs seguro dejando al menos `reserva` nucleos libres
    para que el sistema no se vuelva inutilizable durante la ejecucion."""
    n_cpu = os.cpu_count() or 1
    n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
    return n_usar

N_JOBS = get_n_jobs()
print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```
In [3]: def cargar_csv(ruta, nombre):
    if not os.path.exists(ruta):
        raise FileNotFoundError(f'No se encontro: {ruta}')
    try:
        df = pd.read_csv(ruta, sep=',')
        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
    except Exception:
        df = pd.read_csv(ruta, sep=';')
    print(f' {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas')
    return df

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados...")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")
```

```
Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas
```

```
Datasets cargados...
X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)
```

3.- Definición de la arquitectura y tuning

Se define la arquitectura de la red neuronal y se busca la mejor combinación de hiperparámetros mediante una búsqueda manual sobre un grid reducido.

La arquitectura base es:

- **Capa de entrada:** 54 neuronas (una por variable predictora).
- **Capas ocultas:** 2-3 capas densas con activación ReLU.
- **Dropout:** para regularización y prevención de sobreajuste.
- **Capa de salida:** 21 neuronas lineales (una por target, sin activación).

Se usa **Adam** como optimizador y **MSE** como función de pérdida.

```
In [4]: class MLPMultiSalida(nn.Module):
    """
    Red neuronal densa con multiples capas ocultas y salida vectorial.
    Cada neurona de salida predice un target independiente,
    pero comparten las capas ocultas, permitiendo aprender correlaciones implícitas.
    """
    def __init__(self, n_inputs, n_outputs, hidden_sizes, dropout_rate):
        super().__init__()
        layers = []
        in_size = n_inputs
        for h in hidden_sizes:
            layers += [
                nn.Linear(in_size, h),
                nn.ReLU(),
                nn.Dropout(dropout_rate)
            ]
            in_size = h
        layers.append(nn.Linear(in_size, n_outputs)) # capa de salida lineal
        self.red = nn.Sequential(*layers)

    def forward(self, x):
        return self.red(x)

def entrenar_mlp(X_tr, y_tr, X_val, y_val, hidden_sizes, dropout, lr, epochs, batch_size):
    """Entrena un MLP y devuelve el modelo y el R2 de validación."""
    X_t = torch.tensor(X_tr.values, dtype=torch.float32).to(DEVICE)
    y_t = torch.tensor(y_tr.values, dtype=torch.float32).to(DEVICE)
    X_v = torch.tensor(X_val.values, dtype=torch.float32).to(DEVICE)
    y_v = torch.tensor(y_val.values, dtype=torch.float32).to(DEVICE)

    loader = DataLoader(TensorDataset(X_t, y_t), batch_size=batch_size, shuffle=True)

    model = MLPMultiSalida(X_tr.shape[1], y_tr.shape[1], hidden_sizes, dropout).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    criterion = nn.MSELoss()

    model.train()
    for epoch in range(epochs):
        for xb, yb in loader:
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)
```

```

        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        y_pred = model(X_v).cpu().numpy()
    r2 = r2_score(y_val.values, y_pred, multioutput='uniform_average')
    return model, r2

# Grid de hiperparametros a explorar
grid = [
    {'hidden': [32, 16], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 150, 'batch': 16, 'weight_decay': 1e-2},
    {'hidden': [64, 32], 'dropout': 0.3, 'lr': 1e-3, 'epochs': 200, 'batch': 32, 'weight_decay': 1e-2},
    {'hidden': [64, 64], 'dropout': 0.3, 'lr': 5e-4, 'epochs': 200, 'batch': 32, 'weight_decay': 5e-3},
    {'hidden': [128, 64], 'dropout': 0.4, 'lr': 1e-3, 'epochs': 200, 'batch': 32, 'weight_decay': 2e-3},
    {'hidden': [128, 32], 'dropout': 0.3, 'lr': 5e-4, 'epochs': 250, 'batch': 16, 'weight_decay': 1e-3},
    {'hidden': [64, 16, 8], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 200, 'batch': 16, 'weight_decay': 1e-3},
    {'hidden': [64, 32], 'dropout': 0.3, 'lr': 1e-4, 'epochs': 250, 'batch': 16, 'weight_decay': 1e-3},
    {'hidden': [128, 64, 32], 'dropout': 0.4, 'lr': 5e-4, 'epochs': 200, 'batch': 32, 'weight_decay': 5e-3}
]

print('Buscando mejor arquitectura sobre X_train / X_test...\n')
mejor_r2 = -np.inf
mejor_config = None

for cfg in grid:
    _, r2 = entrenar_mlp(
        X_train, y_train, X_test, y_test,
        cfg['hidden'], cfg['dropout'], cfg['lr'], cfg['epochs'], cfg['batch']
    )
    print(f'hidden={cfg["hidden"]} | dropout={cfg["dropout"]} | lr={cfg["lr"]} | epochs={cfg["epochs"]} → R2 val: {r2:.4f}')
    if r2 > mejor_r2:
        mejor_r2 = r2
        mejor_config = cfg

print(f'\nMejor configuracion: {mejor_config}')
print(f'Mejor R2 de validacion: {mejor_r2:.4f}')
MLP_CONFIG = mejor_config

Buscando mejor arquitectura sobre X_train / X_test...

hidden=[32, 16] | dropout=0.2 | lr=0.001 | epochs=150 → R2 val: 0.1307
hidden=[64, 32] | dropout=0.3 | lr=0.001 | epochs=200 → R2 val: 0.1494
hidden=[64, 64] | dropout=0.3 | lr=0.0005 | epochs=200 → R2 val: 0.1450
hidden=[128, 64] | dropout=0.4 | lr=0.001 | epochs=200 → R2 val: 0.1284
hidden=[128, 32] | dropout=0.3 | lr=0.0005 | epochs=250 → R2 val: 0.1693
hidden=[64, 16, 8] | dropout=0.2 | lr=0.001 | epochs=200 → R2 val: 0.1054
hidden=[64, 32] | dropout=0.3 | lr=0.0001 | epochs=250 → R2 val: 0.0996
hidden=[128, 64, 32] | dropout=0.4 | lr=0.0005 | epochs=200 → R2 val: 0.1650

Mejor configuracion: {'hidden': [128, 32], 'dropout': 0.3, 'lr': 0.0005, 'epochs': 250, 'batch': 16, 'weight_decay': 0.001}
Mejor R2 de validacion: 0.1693

```

4.- Validación cruzada

Se evalúa la robustez de la arquitectura óptima con **validación cruzada repetida** (5 pliegues x 3 repeticiones) sobre `X_train`.

Al ser una red neuronal, el entrenamiento es estocástico: se varían las semillas para obtener una estimación más fiable de la varianza del modelo.

```

In [5]: print('Validacion cruzada repetida (15 evaluaciones sobre X_train)...\n')

cv_splitter = RepeatedKfold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
r2_scores_cv = []

for fold, (train_idx, val_idx) in enumerate(cv_splitter.split(X_train)):
    X_tr_fold = X_train.iloc[train_idx]
    y_tr_fold = y_train.iloc[train_idx]
    X_val_fold = X_train.iloc[val_idx]
    y_val_fold = y_train.iloc[val_idx]

    _, r2 = entrenar_mlp(
        X_tr_fold, y_tr_fold, X_val_fold, y_val_fold,
        MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
        MLP_CONFIG['lr'], MLP_CONFIG['epochs'], MLP_CONFIG['batch']
    )
    r2_scores_cv.append(r2)
    if (fold + 1) % 5 == 0:
        print(f'Pliegues completados: {fold+1}/15 | R2 medio hasta ahora: {np.mean(r2_scores_cv):.4f}')

print(f'\n R2 medio CV : {np.mean(r2_scores_cv):.4f}')
print(f' Desviacion : {np.std(r2_scores_cv):.4f}')
print(f' Minimo : {np.min(r2_scores_cv):.4f}')
print(f' Maximo : {np.max(r2_scores_cv):.4f}')

# Grafico: distribucion R2 CV (boxplot)
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.boxplot(r2_scores_cv, vert=False, patch_artist=True,
            boxprops=dict(facecolor=PALETTE['blue'], alpha=0.7),
            medianprops=dict(color=PALETTE['gray_dark'], linewidth=1.5),
            whiskerprops=dict(color=PALETTE['gray']),
            capprops=dict(color=PALETTE['gray']),
            flierprops=dict(marker='o', color=PALETTE['gray'], markersize=4))
ax.set_xlabel('R2')
ax.set_title('Distribucion R2 en validacion cruzada (configuracion optima)', fontsize=11)
ax.axvline(np.median(r2_scores_cv), color=PALETTE['gray_dark'], linestyle='--', linewidth=0.8, label=f'Mediana {np.median(r2_scores_cv):.4f}')
ax.legend(fontsize=9)

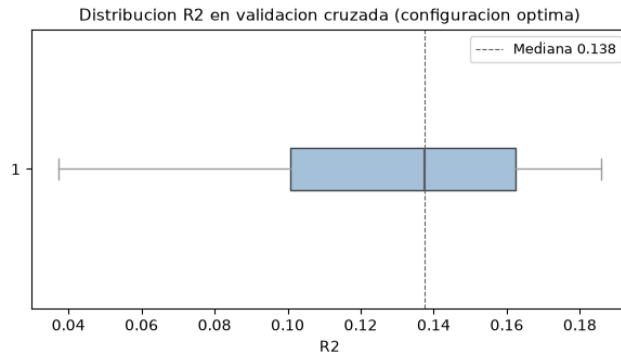
```

```
plt.tight_layout()
plt.show()
```

Validación cruzada repetida (15 evaluaciones sobre X_train)...

```
Pliegues completados: 5/15 | R2 medio hasta ahora: 0.1290
Pliegues completados: 10/15 | R2 medio hasta ahora: 0.1277
Pliegues completados: 15/15 | R2 medio hasta ahora: 0.1261
```

```
R2 medio CV : 0.1261
Desviación : 0.0454
Mínimo : 0.0372
Máximo : 0.1858
```



5.- Importancia de variables

Las redes neuronales no tienen una medida de importancia intrínseca como los árboles.

Se usa **permutation importance** sobre `X_train`: se baraja aleatoriamente cada variable y se mide cuánto cae el R2, lo que indica cuánto depende el modelo de esa variable.

```
In [6]: print('Entrenando modelo de referencia para calcular importancia...\n')

# Entrenamos un modelo sobre todo X_train para calcular la importancia
modelo_imp, _ = entrenar_mlp(
    X_train, y_train, X_test, y_test,
    MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
    MLP_CONFIG['lr'], MLP_CONFIG['epochs'], MLP_CONFIG['batch']
)

# Envolvemos el modelo PyTorch en un wrapper sklearn para usar permutation_importance
class MLPWrapper(BaseEstimator, RegressorMixin):
    def __init__(self, model):
        self.model = model
    def fit(self, X, y):
        return self
    def predict(self, X):
        X_t = torch.tensor(np.array(X), dtype=torch.float32).to(DEVICE)
        self.model.eval()
        with torch.no_grad():
            return self.model(X_t).cpu().numpy()

wrapper = MLPWrapper(modelo_imp)
perm = permutation_importance(
    wrapper, X_train, y_train,
    n_repeats=10, random_state=RANDOM_STATE
)

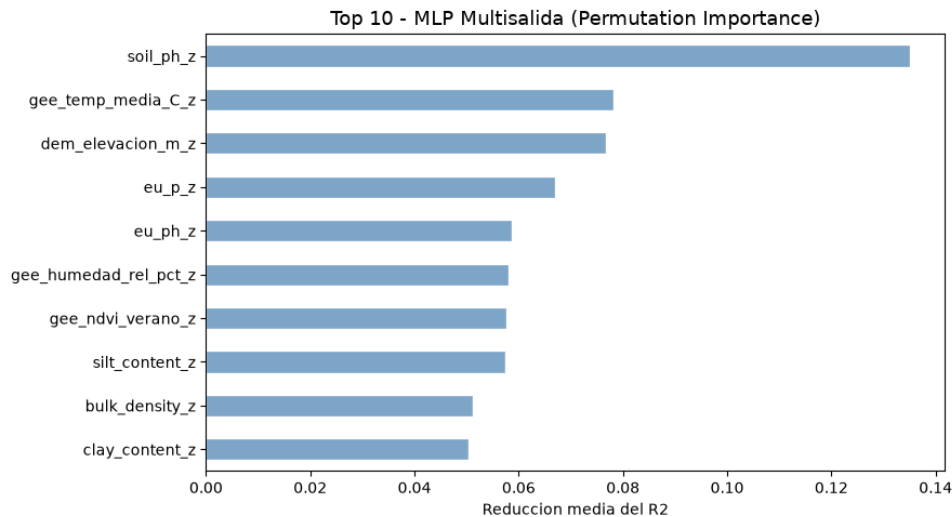
importancias = pd.Series(perm.importances_mean, index=FEATURES_AUTORIZADAS)
print('Top 10 variables por importancia de permutación (todos los targets):')
print(importancias.nlargest(10).round(4).to_string())

# Grafico: barplot importancia de variables
fig, ax = plt.subplots(figsize=(9, 5))
importancias.nlargest(10).sort_values().plot(kind='barh', ax=ax, color=PALETTE['blue'], edgecolor='white')
ax.set_title('Top 10 - MLP Multisalida (Permutation Importance)', fontsize=13)
ax.set_xlabel('Reducción media del R2')
ax.set_ylabel('')
plt.tight_layout()
plt.show()
```

Entrenando modelo de referencia para calcular importancia...

Top 10 variables por importancia de permutación (todos los targets):

```
soil_ph_z      0.1350
gee_temp_media_C_z  0.0781
dem_elevation_m_z  0.0768
eu_p_z        0.0670
eu_ph_z       0.0587
gee_humedad_rel_pct_z  0.0581
gee_ndvi_verano_z  0.0576
silt_content_z  0.0575
bulk_density_z  0.0512
clay_content_z  0.0504
```



6.- Entrenamiento

Se entrena un ensamble de 5 redes neuronales con semillas distintas.

Al ser el entrenamiento estocastico, cada red converge a una solución ligeramente diferente. Promediar las predicciones reduce la varianza y mejora la estabilidad del modelo final.

```
In [7]: print('Entrenamiento de produccion (ensamble de 5 redes neuronales)...')
print('-' * 70)

global_start = time.time()
NUM_REDES = 5

for seed_id in range(NUM_REDES):
    t0 = time.time()
    torch.manual_seed(128 + seed_id)
    np.random.seed(128 + seed_id)

    model, r2_test = entrenar_mlp(
        X_train, y_train, X_test, y_test,
        MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
        MLP_CONFIG['lr'], MLP_CONFIG['epochs'], MLP_CONFIG['batch']
    )

    ruta = os.path.join(MODELS_DIR, f'mlp_prod_{seed_id}.pt')
    torch.save(model.state_dict(), ruta)
    print(f' Red {seed_id+1}/5 | R2 test: {r2_test:.4f} | guardada en {ruta} ({time.time()-t0:.1f}s)')

# Guardamos la configuracion para poder reconstruir la arquitectura al cargar
joblib.dump(MLP_CONFIG, os.path.join(MODELS_DIR, 'mlp_config.pkl'))

print('-' * 70)
print(f'Produccion completada en {(time.time()-global_start)/60:.1f} minutos.')
```

```
Entrenamiento de produccion (ensamble de 5 redes neuronales)...
-----
Red 1/5 | R2 test: 0.1338 | guardada en output/models/mlp/mlp_prod_0.pt (2.6s)
Red 2/5 | R2 test: 0.1568 | guardada en output/models/mlp/mlp_prod_1.pt (2.8s)
Red 3/5 | R2 test: 0.1711 | guardada en output/models/mlp/mlp_prod_2.pt (2.8s)
Red 4/5 | R2 test: 0.1260 | guardada en output/models/mlp/mlp_prod_3.pt (3.0s)
Red 5/5 | R2 test: 0.1513 | guardada en output/models/mlp/mlp_prod_4.pt (3.4s)
-----
```

Produccion completada en 0.2 minutos.

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretacion del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biologica).

```
In [8]: NUM_REDES = 5

# Evaluacion final

print('Evaluacion final sobre eval.csv')
print('-' * 60)

# Cargamos la configuracion y reconstruimos cada red para cargar sus pesos
cfg_prod = joblib.load(os.path.join(MODELS_DIR, 'mlp_config.pkl'))

preds_eval = []
for seed_id in range(NUM_REDES):
```

```

model = MLPMultiSalida(
    len(FEATURES_AUTORIZADAS), len(TARGETS),
    cfg_prod['hidden'], cfg_prod['dropout']
).to(DEVICE)
model.load_state_dict(torch.load(os.path.join(MODELS_DIR, f'mlp_prod_{seed_id}.pt'), map_location=DEVICE))
model.eval()
with torch.no_grad():
    X_eval_t = torch.tensor(X_eval.values, dtype=torch.float32).to(DEVICE)
    pred = model(X_eval_t).cpu().numpy()
    preds_eval.append(pred)

y_pred_eval = np.mean(preds_eval, axis=0)
y_true_eval = y_eval.values

r2_global = r2_score(y_true_eval, y_pred_eval, multioutput='uniform_average')
rmse_global = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval, multioutput='uniform_average'))
mae_global = mean_absolute_error(y_true_eval, y_pred_eval, multioutput='uniform_average')

print(f' R2 global: {r2_global:.4f}')
print(f' RMSE global: {rmse_global:.4f} puntos de Shannon H\''')
print(f' MAE global: {mae_global:.4f} puntos de Shannon H\''')

print('\n R2 por target:')
r2_por_target = r2_score(y_true_eval, y_pred_eval, multioutput='raw_values')
for t, r2 in zip(TARGETS, r2_por_target):
    print(f' {t:30} {r2:.4f}')

# Smoke test con datos reales

print('\nSmoke test - Inferencia con datos reales (Shannon & Richness)')
print('-' * 75)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    votos = []
    for seed_id in range(NUM_REDES):
        model = MLPMultiSalida(
            len(FEATURES_AUTORIZADAS), len(TARGETS),
            cfg_prod['hidden'], cfg_prod['dropout']
        ).to(DEVICE)
        model.load_state_dict(torch.load(os.path.join(MODELS_DIR, f'mlp_prod_{seed_id}.pt'), map_location=DEVICE))
        model.eval()
        with torch.no_grad():
            X_new_t = torch.tensor(df_new_data.values, dtype=torch.float32).to(DEVICE)
            pred = model(X_new_t).cpu().numpy()
            votos.append(pred)

        idx_sha = TARGETS.index('earthworm_shannon_z')
        idx_ric = TARGETS.index('earthworm_richness_z')
        print(f' Red {seed_id+1} | Fila {indices_elegidos[0]}: Sha={pred[0, idx_sha]:.3f}, Ric={pred[0, idx_ric]:.3f} | '
              f'Fila {indices_elegidos[1]}: Sha={pred[1, idx_sha]:.3f}, Ric={pred[1, idx_ric]:.3f}')

    consenso = np.mean(votos, axis=0)
    idx_earth_sha = TARGETS.index('earthworm_shannon_z')
    idx_earth_ric = TARGETS.index('earthworm_richness_z')

    print('-' * 75)
    print(' Resultados detallados por muestra (Métrica: Shannon):')
    print('-' * 75)
    for i, idx in enumerate(indices_elegidos):
        pred_val = consenso[i, idx_earth_sha]
        real_val = y_eval.loc[idx, 'earthworm_shannon_z']
        print(f' Fila {idx:<4} | Predicción: {pred_val:.4f} | Valor Real: {real_val:.4f} | Error: {abs(pred_val - real_val):.4f}')

    print('-' * 75)
    print(' Resultados detallados por muestra (Métrica: Richness):')
    print('-' * 75)
    for i, idx in enumerate(indices_elegidos):
        pred_val = consenso[i, idx_earth_ric]
        real_val = y_eval.loc[idx, 'earthworm_richness_z']
        print(f' Fila {idx:<4} | Predicción: {pred_val:.4f} | Valor Real: {real_val:.4f} | Error: {abs(pred_val - real_val):.4f}')

    print('-' * 75)
    print('\nSmoke test superado. El pipeline MLP predictivo (Shannon + Richness) funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: prediccion vs valor real por target
targets = ['earthworm_shannon_z', 'earthworm_richness_z']
n_targets = len(targets)
n_cols = 2
n_rows = (n_targets + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(12,5))
axes = axes.flatten()

for i, t in enumerate(targets):
    ax = axes[i]
    ax.scatter(y_true_eval[:, i] if y_true_eval.ndim > 1 else y_true_eval,
               y_pred_eval[:, i] if y_pred_eval.ndim > 1 else y_pred_eval,
               alpha=0.6, s=25, color=PALETTE['blue'])
    lim = [min((y_true_eval[:, i] if y_true_eval.ndim > 1 else y_true_eval).min(),
               (y_pred_eval[:, i] if y_pred_eval.ndim > 1 else y_pred_eval).min()),
           max((y_true_eval[:, i] if y_true_eval.ndim > 1 else y_true_eval).max(),
               (y_pred_eval[:, i] if y_pred_eval.ndim > 1 else y_pred_eval).max())]
    ax.plot(lim, lim, 'r--', linewidth=1)
    r2_t = r2_score(
        y_true_eval[:, i] if y_true_eval.ndim > 1 else y_true_eval,

```

```

        y_pred_eval[:, i] if y_pred_eval.ndim > 1 else y_pred_eval
    )
    ax.set_title(f'{t}\nR2={r2_t:.3f}', fontsize=8)
    ax.set_xlabel('Valor real', fontsize=7)
    ax.set_ylabel('Predicción', fontsize=7)
    ax.tick_params(labelsize=7)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Predicción vs Valor Real por target (eval.csv)', fontsize=12, y=1.01)
plt.tight_layout()
plt.show()

```

Evaluación final sobre eval.csv

R2 global: -0.0380
 RMSE global: 0.9597 puntos de Shannon H'
 MAE global: 0.7042 puntos de Shannon H'

R2 por target:

nematode_shannon_z	0.1180
macro_shannon_z	0.2571
earthworm_shannon_z	0.4264
orib_shannon_z	0.2356
meso_shannon_z	-0.4898
coll_shannon_z	-0.8227
bac_shannon_z	-0.0854
fun_shannon_z	-0.0757
euk_shannon_z	0.1453
oomy_shannon_z	0.1534
cerc_shannon_z	-0.1361
macro_order_richness_z	0.3525
earthworm_richness_z	0.5176
orib_species_richness_z	0.2596
meso_species_richness_z	-0.2753
coll_species_richness_z	-1.4628
bac_asv_richness_z	-0.1464
fun_asv_richness_z	0.0145
euk_asv_richness_z	0.2164
oomy_asv_richness_z	0.0113
cerc_asv_richness_z	-0.0120

Smoke test - Inferencia con datos reales (Shannon & Richness)

Red 1 | Fila 53: Sha=0.567, Ric=0.722 | Fila 60: Sha=0.877, Ric=1.054
 Red 2 | Fila 53: Sha=0.298, Ric=0.382 | Fila 60: Sha=1.170, Ric=1.373
 Red 3 | Fila 53: Sha=0.514, Ric=0.681 | Fila 60: Sha=1.305, Ric=1.462
 Red 4 | Fila 53: Sha=0.415, Ric=0.466 | Fila 60: Sha=1.041, Ric=1.155
 Red 5 | Fila 53: Sha=0.446, Ric=0.611 | Fila 60: Sha=0.965, Ric=1.158

Resultados detallados por muestra (Métrica: Shannon):

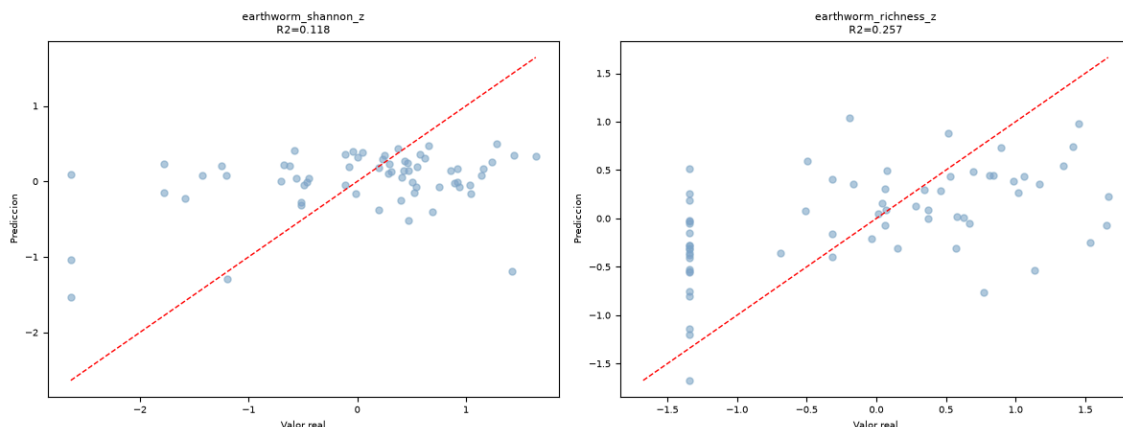
Fila 53	Predicción: 0.4484 Valor Real: 1.1347 Error: 0.6863
Fila 60	Predicción: 1.0717 Valor Real: 1.1388 Error: 0.0671
Fila 0	Predicción: -0.3981 Valor Real: -0.7453 Error: 0.3472

Resultados detallados por muestra (Métrica: Richness):

Fila 53	Predicción: 0.5725 Valor Real: 1.4889 Error: 0.9164
Fila 60	Predicción: 1.2405 Valor Real: 1.1147 Error: 0.1258
Fila 0	Predicción: -0.4892 Valor Real: -0.7562 Error: 0.2670

Smoke test superado. El pipeline MLP predictivo (Shannon + Richness) funciona correctamente.

Predicción vs Valor Real por target (eval.csv)



11.2.8. Modelos MLP con función de pérdida personalizada

11.2.8.1. Fundamento y configuración

Esta variante extiende el MLP multisalida anterior con una función de pérdida personalizada (CorrelationAwareLoss) que combina dos componentes:

$$L(y, \hat{y}) = \text{MSE}(y, \hat{y}) + \lambda_{\text{corr}} \left\| \text{Corr}(y) - \text{Corr}(\hat{y}) \right\|_F$$

El primer término es el **MSE estándar**, que minimiza el error de predicción de cada target por separado. El segundo penaliza la diferencia (**norma de Frobenius**) entre la matriz de correlación de las predicciones del lote actual y la matriz de correlación de los valores reales del mismo lote: si el modelo predice, por ejemplo, una diversidad alta de lombrices junto con una diversidad muy baja de colémbolos cuando ambas normalmente co-varían en los datos reales, este término penaliza esa incoherencia, aunque el error individual (MSE) de cada target por separado sea bajo.

El parámetro `lambda_corr` controla el peso relativo de esta penalización frente al MSE (`lambda_corr=0` equivale a MSE puro).

El tuning explora 10 configuraciones que combinan arquitectura, hiperparámetros de entrenamiento y distintos valores de `lambda_corr` (de 0.0 a 0.5). El resultado del tuning es, en sí mismo, uno de los hallazgos más relevantes de este notebook:

Capas ocultas	lambda_corr	weight_decay	R ² validación
[32, 16]	0.0	0.01	0.1151
[64, 32]	0.0	0.01	0.1495 (mejor)
[64, 32]	0.05	0.005	0.1202
[64, 32]	0.1	0.005	0.1321
[128, 64]	0.05	0.002	0.1375
[128, 64]	0.2	0.002	0.1185
[128, 32]	0.1	0.001	0.1419
[128, 32]	0.3	0.002	0.1092
[64, 16, 8]	0.1	0.001	0.1186
[64, 16, 8]	0.5	0.005	0.0071
[128, 64, 32]	0.2	0.005	0.0304

Tabla 22: Combinaciones de arquitecturas para MLP Custom Loss

La configuración ganadora del tuning fue `hidden=[64, 32]` con `lambda_corr=0.0`, es decir: de entre todas las combinaciones probadas, la que mejor R² de validación obtuvo fue la que equivale a MSE puro, sin ninguna penalización de correlación activa.

Esto indica que, con el tamaño de dataset disponible en este trabajo, el término adicional de coherencia de correlaciones no aportó una mejora medible frente al MSE estándar e incluso empeoró el resultado en la mayoría de las configuraciones donde se activó.

11.2.8.2. Entrenamiento

Al igual que en el MLP multisalida estándar, no se aplica una validación cruzada explícita: la arquitectura se selecciona mediante la comparación directa de las 10 configuraciones del tuning sobre la partición fija de entrenamiento/validación, y se entrena un único modelo final con la configuración ganadora (sin ensamblado de semillas).

Además de las métricas de error habituales, este notebook calcula la diferencia media entre la matriz de correlación real y la matriz de correlación predicha como indicador directo de si la función de pérdida personalizada está cumpliendo su objetivo. Sobre `X_train`, esta diferencia media es de **0.1131**.

11.2.8.3. Explicabilidad y variables más relevantes

Al igual que en el MLP multisalida estándar, se recurre a permutation importance sobre X_{train} , esta vez calculada de forma global sobre los 21 targets.

El top-10 resultante es: soil_ph_z, gee_humedad_rel_pct_z, eu_ph_z, dem_elevacion_m_z, eu_p_z, gee_ndvi_verano_z, gee_temp_media_C_z, eu_cn_ratio_z, eu_zn_z y silt_content_z.

soil_ph_z mantiene su primer puesto respecto al MLP multisalida estándar, aunque con una importancia relativa considerablemente mayor (0.165 frente a 0.135), lo que sugiere que, al reforzar la coherencia entre targets, el modelo termina apoyándose todavía más en la variable individual con mayor poder predictivo del conjunto.

La gráfica que muestra las variables más importantes se puede ver más adelante en el notebook exportado en la sección 5.- **Importancia de variables**.

11.2.8.4. Resultados sobre eval.csv

Con un R^2 medio de **-0.1316**, este es el modelo con **peor rendimiento global** de los ocho evaluados en este trabajo, por debajo incluso del MLP multisalida estándar (-0.038) del que parte.

Esto es consistente con el resultado del propio tuning: al haberse seleccionado $\lambda_{corr}=0.0$ como configuración óptima, el modelo final es, en la práctica, equivalente a un MLP multisalida con una arquitectura ligeramente distinta ([64,32] en vez de [128,32]) y menos épocas de entrenamiento (150 en vez de 250), sin que la penalización de correlación llegue a aplicarse de forma efectiva.

Sobre los targets prioritarios obtiene $R^2=0.3750$ (earthworm_shannon_z) y $R^2=0.4671$ (earthworm_richness_z), ambos por debajo de los conseguidos por el MLP multisalida estándar. El target coll_species_richness_z vuelve a ser el más problemático, con un $R^2=-2.2411$, el peor resultado individual de todo este trabajo.

Variable	Target	R^2
Shannon nemátodos	nematode_shannon_z	0.0903
Shannon macrofauna	macro_shannon_z	0.2676
Shannon lombrices	earthworm_shannon_z	0.3750
Shannon oribátidos	orib_shannon_z	0.2216
Shannon mesostigmátidos	meso_shannon_z	-0.6189
Shannon colémbolos	coll_shannon_z	-1.3439
Shannon bacterias	bac_shannon_z	-0.1004
Shannon hongos	fun_shannon_z	-0.0842
Shannon eucariotas	euk_shannon_z	0.0868
Shannon oomicetos	oomy_shannon_z	0.0588
Shannon cercozoos	cerc_shannon_z	-0.1194
Riqueza macrofauna	macro_order_richness_z	0.3759
Riqueza lombrices	earthworm_richness_z	0.4671
Riqueza oribátidos	orib_species_richness_z	0.2542
Riqueza mesostigmátidos	meso_species_richness_z	-0.3462
Riqueza colémbolos	coll_species_richness_z	-2.2411
Riqueza bacterias (ASV)	bac_asv_richness_z	-0.1680
Riqueza hongos (ASV)	fun_asv_richness_z	0.0027
Riqueza eucariotas (ASV)	euk_asv_richness_z	0.1902
Riqueza oomicetos (ASV)	oomy_asv_richness_z	-0.0546
Riqueza cercozoos (ASV)	cerc_asv_richness_z	-0.0770

Tabla 23: R^2 del MLP con pérdida personalizada sobre eval.csv, por target.

SOB4ES - Red Neuronal con Pérdida Personalizada

CRISP-ML(Q) - Fase 3 y 4: Desarrollo y evaluación de los modelos

Este notebook extiende el MLP multisalida estándar con una **función de pérdida personalizada** que penaliza las predicciones que no respetan la estructura de correlación entre targets observada en los datos de entrenamiento.

La pérdida tiene dos componentes:

- **MSE estándar:** minimiza el error de predicción por target individualmente.
- **Penalización de correlación:** compara la matriz de correlación de las predicciones con la matriz de correlación de los targets reales en el batch. Si el modelo predice valores que rompen las correlaciones biológicas conocidas (por ejemplo, diversidad alta de lombrices con diversidad muy baja de colémbolos cuando normalmente co-varian), la pérdida aumenta.

El peso relativo entre ambos componentes se controla con el parámetro `lambda_corr`.

Datasets utilizados:

- `train.csv` - Entrenamiento del modelo (70 % del total, estratificado por país)
- `test.csv` - Búsqueda de hiperparámetros y validación cruzada (15 %)
- `eval.csv` - Evaluación final e imparcial del rendimiento (15 %)

Estructura del notebook:

1. **Configuración:** importaciones y parámetros globales.
2. **Carga de datos:** se cargan los tres splits pre-generados.
3. **Definición de la pérdida personalizada y tuning.**
4. **Validación cruzada:** evaluación de robustez.
5. **Análisis de pesos e importancia de variables.**
6. **Producción:** entrenamiento final y exportación de modelos.
7. **Evaluación final:** rendimiento real sobre `eval.csv`.

1.- Configuración del entorno

En esta sección del notebook importaremos todas las librerías necesarias como también realizaremos las configuraciones iniciales.

```
In [1]: import os
import time
import warnings
import joblib
import pandas as pd
import numpy as np
import itertools
from pathlib import Path
import seaborn as sns

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

from sklearn.model_selection import RepeatedKFold
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import permutation_importance
from sklearn.base import BaseEstimator, RegressorMixin

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_DIR = 'output/models/mlp_custom/'
Path(MODELS_DIR).mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo de computo: {DEVICE}')

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
```

```

        'bac_asv_richness_z',
        'fun_asv_richness_z',
        'euk_asv_richness_z',
        'oomy_asv_richness_z',
        'cerc_asv_richness_z'
    ]

    FEATURES_AUTORIZADAS = [
        'total_plant_cover_z',
        'clay_content_z',
        'silt_content_z',
        'sand_content_z',
        'aggregate_stability_z',
        'bulk_density_z',
        'soil_moisture_z',
        'as_z',
        'cu_z',
        'k_z',
        'mo_z',
        'ni_z',
        'p_z',
        'pb_z',
        'zn_z',
        'soil_ph_z',
        'plot_total_organic_c_z',
        'plot_total_n_z',
        'gee_temp_media_C_z',
        'gee_humedad_rel_pct_z',
        'gee_ndvi_verano_z',
        'dem_elevacion_m_z',
        'dem_pendiente_deg_z',
        'dem_orientacion_deg_z',
        'eu_clay_content_z',
        'eu_sand_content_z',
        'eu_silt_content_z',
        'eu_water_holding_capacity_z',
        'eu_cn_ratio_z',
        'eu_p_z',
        'eu_ph_z',
        'eu_as_z',
        'eu_organic_carbon_octop_z',
        'eu_zn_z'
    ]

    print('Librerías y variables cargadas correctamente...')

    # Paleta de colores unificada
    PALETTE = {
        'blue': '#7FA6C9',
        'blue_light': '#AAD8F1',
        'green': '#7FC8A9',
        'green_light': '#BEE6D3',
        'purple': '#A98FC7',
        'purple_light': '#D6C7EA',
        'gray': '#9B9B9B',
        'gray_dark': '#5C5C5C',
    }

    # Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
    PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                     PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

    def palette_cycle(n):
        """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
        return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

    # Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
    PALETTE_CMAP = LinearSegmentedColormap.from_list(
        'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
    )

```

Dispositivo de computo: cuda
 Librerías y variables cargadas correctamente...

```

In [2]: # Ajuste automatico de n_jobs segun los nucleos disponibles
        # Evita saturar la maquina (todos los procesos a la vez con n_jobs=-1)
        import os

        def get_n_jobs(fraccion=0.75, reserva=1):
            """Calcula un n_jobs seguro dejando al menos `reserva` nucleos libres
            para que el sistema no se vuelva inutilizable durante la ejecucion."""
            n_cpu = os.cpu_count() or 1
            n_usar = max(1, min(int(n_cpu * fraccion), n_cpu - reserva))
            return n_usar

        N_JOBS = get_n_jobs()
        print(f'Nucleos disponibles: {os.cpu_count()} | n_jobs asignado: {N_JOBS}')

```

Nucleos disponibles: 20 | n_jobs asignado: 15

2.- Carga de datos

Se cargan los tres datasets pre-generados a partir de la división 70/15/15 estratificada por país.

Archivo	Uso	Filas aprox.
train.csv	Ajuste del modelo	299
test.csv	Tuning de hiperparametros / Val. cruzada	64
eval.csv	Evaluacion final imparcial	65

El dataset `eval.csv` no se utiliza hasta la sección 7 para garantizar una estimación imparcial del rendimiento.

```
In [3]: def cargar_csv(ruta, nombre):
    if not os.path.exists(ruta):
        raise FileNotFoundError(f'No se encontro: {ruta}')
    try:
        df = pd.read_csv(ruta, sep=',')
        if len(df.columns) < 5:
            df = pd.read_csv(ruta, sep=';')
    except Exception:
        df = pd.read_csv(ruta, sep=';')
    print(f' {nombre}: {df.shape[0]} filas x {df.shape[1]} columnas')
    return df

print("Cargando datasets...")
df_train = cargar_csv(os.path.join(DATA_DIR, 'train.csv'), 'train.csv')
df_test = cargar_csv(os.path.join(DATA_DIR, 'test.csv'), 'test.csv')
df_eval = cargar_csv(os.path.join(DATA_DIR, 'eval.csv'), 'eval.csv')

X_train = df_train[FEATURES_AUTORIZADAS]
y_train = df_train[TARGETS]

X_test = df_test[FEATURES_AUTORIZADAS]
y_test = df_test[TARGETS]

X_eval = df_eval[FEATURES_AUTORIZADAS] # no se usa hasta la seccion 7
y_eval = df_eval[TARGETS] # no se usa hasta la seccion 7

print(f"\nDatasets cargados...")
print(f"X_train: {X_train.shape} | X_test: {X_test.shape} | X_eval: {X_eval.shape}")
print(f"y_train: {y_train.shape} | y_test: {y_test.shape} | y_eval: {y_eval.shape}")

Cargando datasets...
train.csv: 299 filas x 71 columnas
test.csv: 64 filas x 71 columnas
eval.csv: 65 filas x 71 columnas

Datasets cargados...
X_train: (299, 34) | X_test: (64, 34) | X_eval: (65, 34)
y_train: (299, 21) | y_test: (64, 21) | y_eval: (65, 21)
```

3.- Pérdida personalizada y tuning

Función de pérdida

La pérdida combina dos terminos:

$$\text{Loss} = \text{MSE}(y_{\text{pred}}, y_{\text{real}}) + \lambda_{\text{corr}} * \text{CorrelationPenalty}(y_{\text{pred}}, y_{\text{real}})$$

La penalización de correlación se calcula como la **diferencia de Frobenius** entre la matriz de correlación de las predicciones y la de los valores reales en cada batch:

$$\text{CorrelationPenalty} = || \text{corr}(y_{\text{pred}}) - \text{corr}(y_{\text{real}}) ||_F$$

Un `lambda_corr` alto fuerza al modelo a respetar las correlaciones biológicas pero puede perjudicar la precisión individual por target. El tuning busca el valor óptimo de este parámetro junto a la arquitectura.

Arquitectura

Idéntica al MLP estándar: capas densas con ReLU, Dropout y capa de salida lineal de 21 neuronas.

```
In [4]: class MLPMultiSalida(nn.Module):
    def __init__(self, n_inputs, n_outputs, hidden_sizes, dropout_rate):
        super().__init__()
        layers = []
        in_size = n_inputs
        for h in hidden_sizes:
            layers += [nn.Linear(in_size, h), nn.ReLU(), nn.Dropout(dropout_rate)]
            in_size = h
        layers.append(nn.Linear(in_size, n_outputs))
        self.red = nn.Sequential(*layers)
    def forward(self, x):
        return self.red(x)

def correlation_matrix(y):
    """
    Calcula la matriz de correlacion de un tensor (batch x targets).
    Se usa para comparar las correlaciones entre targets predichos vs reales.
    """
    y_centered = y - y.mean(dim=0, keepdim=True)
```

```

std = y_centered.std(dim=0, keepdim=True).clamp(min=1e-8)
y_norm = y_centered / std
return torch.mm(y_norm.T, y_norm) / y.shape[0]

class CorrelationAwareLoss(nn.Module):
    """
    Perdida personalizada que combina MSE con una penalizacion
    por diferencias en la estructura de correlacion entre targets.
    lambda_corr controla el peso de la penalizacion:
    - lambda_corr = 0.0: equivale a MSE puro
    - lambda_corr = 1.0: penalizacion de correlacion con mismo peso que MSE
    """
    def __init__(self, lambda_corr=0.1):
        super().__init__()
        self.lambda_corr = lambda_corr
        self.mse = nn.MSELoss()

    def forward(self, y_pred, y_real):
        mse_loss = self.mse(y_pred, y_real)
        corr_pred = correlation_matrix(y_pred)
        corr_real = correlation_matrix(y_real)
        # Norma de Frobenius: penaliza diferencias elemento a elemento entre matrices
        corr_loss = torch.norm(corr_pred - corr_real, p='fro')
        return mse_loss + self.lambda_corr * corr_loss

def entrenar_mlp_custom(X_tr, y_tr, X_val, y_val, hidden_sizes, dropout, lr, epochs, batch_size, lambda_corr):
    X_t = torch.tensor(X_tr.values, dtype=torch.float32).to(DEVICE)
    y_t = torch.tensor(y_tr.values, dtype=torch.float32).to(DEVICE)
    X_v = torch.tensor(X_val.values, dtype=torch.float32).to(DEVICE)
    y_v = torch.tensor(y_val.values, dtype=torch.float32).to(DEVICE)

    loader = DataLoader(TensorDataset(X_t, y_t), batch_size=batch_size, shuffle=True)
    model = MLPMultiSalida(X_tr.shape[1], y_tr.shape[1], hidden_sizes, dropout).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    criterion = CorrelationAwareLoss(lambda_corr=lambda_corr)

    model.train()
    for epoch in range(epochs):
        for xb, yb in loader:
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)
            loss.backward()
            optimizer.step()

    model.eval()
    with torch.no_grad():
        y_pred = model(X_v).cpu().numpy()
        r2 = r2_score(y_val.values, y_pred, multioutput='uniform_average')
    return model, r2

# Grid de hiperparametros: arquitectura + lambda_corr
grid = [
    {'hidden': [32, 16], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 150, 'batch': 16, 'weight_decay': 1e-2, 'lambda_corr': 0.0},
    {'hidden': [64, 32], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 150, 'batch': 16, 'weight_decay': 1e-2, 'lambda_corr': 0.0},
    {'hidden': [64, 32], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 150, 'batch': 16, 'weight_decay': 5e-3, 'lambda_corr': 0.05},
    {'hidden': [64, 32], 'dropout': 0.3, 'lr': 1e-3, 'epochs': 200, 'batch': 32, 'weight_decay': 5e-3, 'lambda_corr': 0.1},
    {'hidden': [128, 64], 'dropout': 0.3, 'lr': 1e-3, 'epochs': 200, 'batch': 32, 'weight_decay': 2e-3, 'lambda_corr': 0.05},
    {'hidden': [128, 64], 'dropout': 0.3, 'lr': 1e-3, 'epochs': 200, 'batch': 32, 'weight_decay': 2e-3, 'lambda_corr': 0.2},
    {'hidden': [128, 32], 'dropout': 0.3, 'lr': 5e-4, 'epochs': 250, 'batch': 16, 'weight_decay': 1e-3, 'lambda_corr': 0.1},
    {'hidden': [128, 32], 'dropout': 0.4, 'lr': 5e-4, 'epochs': 250, 'batch': 16, 'weight_decay': 2e-3, 'lambda_corr': 0.3},
    {'hidden': [64, 16, 8], 'dropout': 0.2, 'lr': 1e-3, 'epochs': 200, 'batch': 16, 'weight_decay': 1e-3, 'lambda_corr': 0.1},
    {'hidden': [64, 16, 8], 'dropout': 0.3, 'lr': 1e-3, 'epochs': 200, 'batch': 16, 'weight_decay': 5e-3, 'lambda_corr': 0.5},
    {'hidden': [128, 64, 32], 'dropout': 0.4, 'lr': 5e-4, 'epochs': 200, 'batch': 32, 'weight_decay': 5e-3, 'lambda_corr': 0.2}
]

print('Buscando mejor configuracion (arquitectura + lambda_corr)... \n')
mejor_r2 = -np.inf
mejor_config = None

for cfg in grid:
    _, r2 = entrenar_mlp_custom(
        X_train, y_train, X_test, y_test,
        cfg['hidden'], cfg['dropout'], cfg['lr'],
        cfg['epochs'], cfg['batch'], cfg['lambda_corr']
    )
    print(f' hidden={str(cfg["hidden"]):20} lambda_corr={cfg["lambda_corr"]} → R2 val: {r2:.4f}')
    if r2 > mejor_r2:
        mejor_r2 = r2
        mejor_config = cfg

print(f'\nMejor configuracion: {mejor_config}')
print(f'Mejor R2 de validacion: {mejor_r2:.4f}')
MLP_CONFIG = mejor_config

Buscando mejor configuracion (arquitectura + lambda_corr)...

hidden=[32, 16] lambda_corr=0.0 → R2 val: 0.1151
hidden=[64, 32] lambda_corr=0.0 → R2 val: 0.1495
hidden=[64, 32] lambda_corr=0.05 → R2 val: 0.1202
hidden=[64, 32] lambda_corr=0.1 → R2 val: 0.1321
hidden=[128, 64] lambda_corr=0.05 → R2 val: 0.1375
hidden=[128, 64] lambda_corr=0.2 → R2 val: 0.1185
hidden=[128, 32] lambda_corr=0.1 → R2 val: 0.1419
hidden=[128, 32] lambda_corr=0.3 → R2 val: 0.1092
hidden=[64, 16, 8] lambda_corr=0.1 → R2 val: 0.1186
hidden=[64, 16, 8] lambda_corr=0.5 → R2 val: 0.0071

```

hidden=[128, 64, 32] lambda_corr=0.2 → R2 val: 0.0304

Mejor configuracion: {'hidden': [64, 32], 'dropout': 0.2, 'lr': 0.001, 'epochs': 150, 'batch': 16, 'weight_decay': 0.01, 'lambda_corr': 0.0}

Mejor R2 de validacion: 0.1495

4.- Validación cruzada

Se evalúa la robustez de la configuración óptima con **validación cruzada repetida** (5 pliegues x 3 repeticiones) sobre `X_train`.

Se registra tanto el R2 global como el efecto que tiene `lambda_corr` sobre la consistencia de las predicciones entre pliegues.

```
In [5]: print('Validacion cruzada repetida (15 evaluaciones sobre X_train)...\n')

cv_splitter = RepeatedKfold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
r2_scores_cv = []

for fold, (train_idx, val_idx) in enumerate(cv_splitter.split(X_train)):
    _, r2 = entrenar_mlp_custom(
        X_train.iloc[train_idx], y_train.iloc[train_idx],
        X_train.iloc[val_idx], y_train.iloc[val_idx],
        MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
        MLP_CONFIG['lr'], MLP_CONFIG['epochs'],
        MLP_CONFIG['batch'], MLP_CONFIG['lambda_corr']
    )
    r2_scores_cv.append(r2)
    if (fold + 1) % 5 == 0:
        print(f' Pliegues completados: {fold+1}/15 | R2 medio hasta ahora: {np.mean(r2_scores_cv):.4f}')

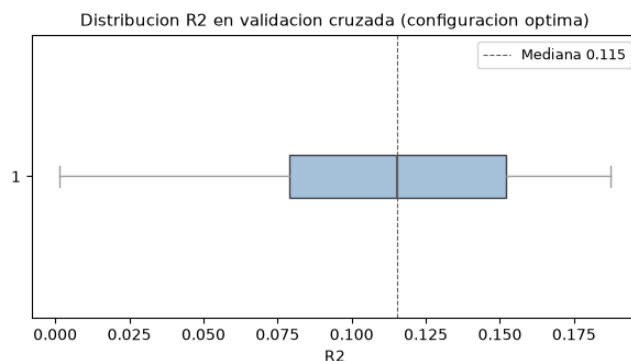
print(f'\n R2 medio CV : {np.mean(r2_scores_cv):.4f}')
print(f' Desviacion : {np.std(r2_scores_cv):.4f}')
print(f' Minimo : {np.min(r2_scores_cv):.4f}')
print(f' Maximo : {np.max(r2_scores_cv):.4f}')

# Grafico: distribucion R2 CV (boxplot)
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.boxplot(r2_scores_cv, vert=False, patch_artist=True,
           boxprops=dict(facecolor=PALETTE['blue'], alpha=0.7),
           medianprops=dict(color=PALETTE['gray_dark'], linewidth=1.5),
           whiskerprops=dict(color=PALETTE['gray']),
           capprops=dict(color=PALETTE['gray']),
           flierprops=dict(marker='o', color=PALETTE['gray'], markersize=4))
ax.set_xlabel('R2')
ax.set_title('Distribucion R2 en validacion cruzada (configuracion optima)', fontsize=11)
ax.axvline(np.median(r2_scores_cv), color=PALETTE['gray_dark'], linestyle='--', linewidth=0.8, label=f'Mediana {np.median(r2_scores_cv):.4f}')
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
```

Validacion cruzada repetida (15 evaluaciones sobre X_train)...

Pliegues completados: 5/15 | R2 medio hasta ahora: 0.1187
 Pliegues completados: 10/15 | R2 medio hasta ahora: 0.1063
 Pliegues completados: 15/15 | R2 medio hasta ahora: 0.1086

R2 medio CV : 0.1086
 Desviacion : 0.0556
 Minimo : 0.0013
 Maximo : 0.1873



5.- Importancia de variables

Se usa **permutation importance** sobre `X_train` mediante un wrapper sklearn.

Al igual que en el MLP estándar, la importancia es global sobre los 21 targets.

Adicionalmente se muestra la matriz de correlación aprendida por el modelo comparada con la matriz de correlación real de los datos de entrenamiento, para verificar que la pérdida personalizada esta funcionando.

```
In [6]: print('Entrenando modelo de referencia para analisis...\n')

# 1. Entrenamiento del modelo con pérdida personalizada
modelo_ref, _ = entrenar_mlp_custom(
    X_train, y_train, X_test, y_test,
    MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
```

```

    MLP_CONFIG['lr'], MLP_CONFIG['epochs'],
    MLP_CONFIG['batch'], MLP_CONFIG['lambda_corr']
)

# 2. Wrapper para compatibilidad con Scikit-Learn
class MLPWrapper(BaseEstimator, RegressorMixin):
    def __init__(self, model):
        self.model = model
    def fit(self, X, y): return self
    def predict(self, X):
        X_t = torch.tensor(np.array(X), dtype=torch.float32).to(DEVICE)
        self.model.eval()
        with torch.no_grad():
            return self.model(X_t).cpu().numpy()

wrapper = MLPWrapper(modelo_ref)

# 3. Cálculo de importancia por permutación
perm = permutation_importance(wrapper, X_train, y_train, n_repeats=10, random_state=RANDOM_STATE)
importancias = pd.Series(perm.importances_mean, index=FEATURES_AUTORIZADAS)

top10_imp = importancias.nlargest(10)
bottom10_imp = importancias.nsmallest(10)

print('Top 10 variables más revelantes por permutación:')
print(top10_imp.round(4).to_string())

# 4. Comparación de matrices de correlación (Real vs Predicha)
y_pred_train = wrapper.predict(X_train)
corr_real = pd.DataFrame(y_train.values, columns=TARGETS).corr()
corr_pred = pd.DataFrame(y_pred_train, columns=TARGETS).corr()
diff_corr = (corr_real - corr_pred).abs()

print(f'\nDiferencia media entre matrices de correlación (real vs predicha): {diff_corr.values.mean():.4f}')
print('(Valor cercano a 0 indica que la pérdida personalizada está preservando las correlaciones.)')

# GRÁFICA 1: Comparativa de Importancia de Variables (Más vs Menos revelantes)

fig, axes = plt.subplots(1, 1, figsize=(16, 5))

# Subplot A: Más revelantes
top10_imp.sort_values().plot(kind='barh', ax=axes, color=PALETTE.get('blue', '#2b5c8f'), edgecolor='white')
axes.set_title('Top 10 MÁS revelantes - MLP Custom', fontsize=12, fontweight='bold')
axes.set_xlabel('Reducción media del R2')
axes.set_ylabel('')

plt.tight_layout()
plt.show()

# GRÁFICA 2: Heatmaps de Preservación de Correlación entre Targets

fig, axes = plt.subplots(1, 3, figsize=(20, 6))

# Heatmap Real
sns.heatmap(corr_real, ax=axes[0], cmap='coolwarm', vmin=-1, vmax=1, cbar_kws={'shrink': 0.8}, square=True)
axes[0].set_title('A. Correlación Real (y_train)', fontsize=11, fontweight='bold')

# Heatmap Predicha
sns.heatmap(corr_pred, ax=axes[1], cmap='coolwarm', vmin=-1, vmax=1, cbar_kws={'shrink': 0.8}, square=True)
axes[1].set_title('B. Correlación Predicha (ŷ_train)', fontsize=11, fontweight='bold')

# Heatmap Diferencia
sns.heatmap(diff_corr, ax=axes[2], cmap='YlOrRd', vmin=0, vmax=1, cbar_kws={'shrink': 0.8}, square=True)
axes[2].set_title('C. Diferencia Absoluta (|Real - Pred|)', fontsize=11, fontweight='bold')

for ax in axes:
    ax.tick_params(axis='x', labelsize=7, labelrotation=90)
    ax.tick_params(axis='y', labelsize=7, labelrotation=0)

plt.tight_layout()
plt.show()

```

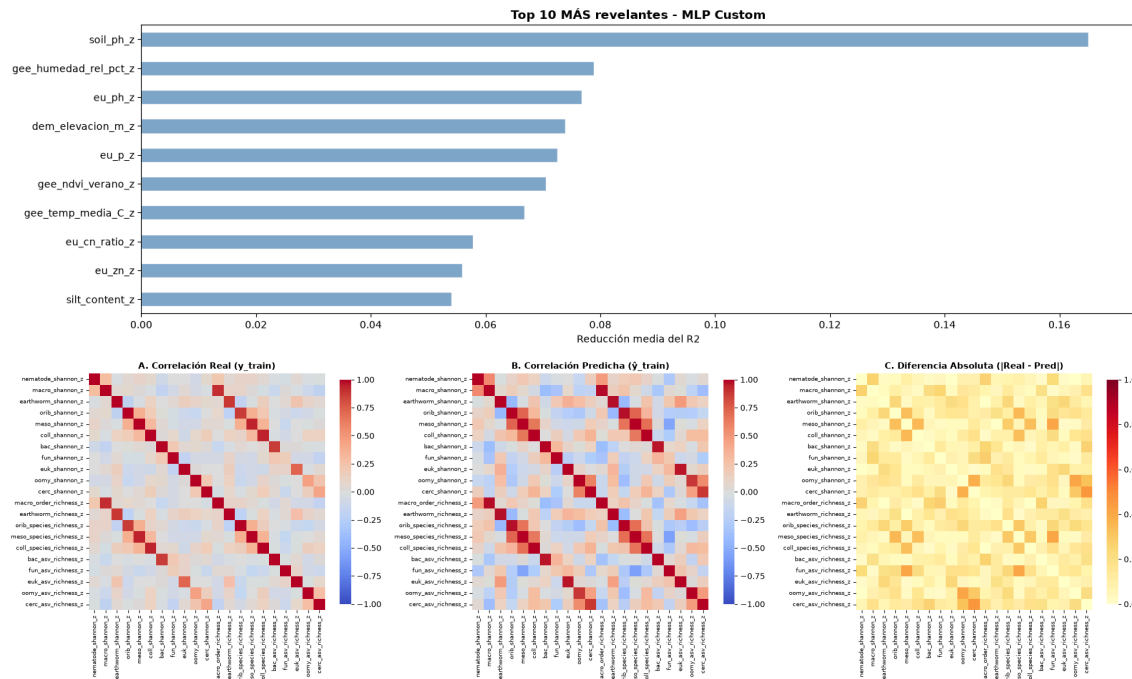
Entrenando modelo de referencia para análisis...

Top 10 variables más revelantes por permutación:

soil_ph_z	0.1650
gee_humedad_rel_pct_z	0.0788
eu_ph_z	0.0767
dem_elevacion_m_z	0.0739
eu_p_z	0.0725
gee_ndvi_verano_z	0.0705
gee_temp_media_C_z	0.0668
eu_cn_ratio_z	0.0577
eu_zn_z	0.0559
silt_content_z	0.0541

Diferencia media entre matrices de correlación (real vs predicha): 0.1131

(Valor cercano a 0 indica que la pérdida personalizada está preservando las correlaciones.)



6.- Entrenamiento

Se entrena un ensamble de 5 redes con semillas distintas usando la pérdida personalizada.

Los pesos se guardan como `.pt` y la configuración de arquitectura como `.pkl`.

```
In [7]: print('Entrenamiento de produccion (ensamble de 5 redes con perdida personalizada)...')
print('-' * 70)

global_start = time.time()
NUM_REDES = 5

for seed_id in range(NUM_REDES):
    t0 = time.time()
    torch.manual_seed(42 + seed_id)
    np.random.seed(42 + seed_id)

    model, r2_test = entrenar_mlp_custom(
        X_train, y_train, X_test, y_test,
        MLP_CONFIG['hidden'], MLP_CONFIG['dropout'],
        MLP_CONFIG['ln'], MLP_CONFIG['epochs'],
        MLP_CONFIG['batch'], MLP_CONFIG['lambda_corr']
    )
    ruta = os.path.join(MODELS_DIR, f'mlp_custom_{seed_id}.pt')
    torch.save(model.state_dict(), ruta)
    print(f' Red {seed_id+1}/5 | R2 test: {r2_test:.4f} | {(time.time()-t0:.1f)s}')

joblib.dump(MLP_CONFIG, os.path.join(MODELS_DIR, 'mlp_custom_config.pkl'))

print('-' * 70)
print(f'Produccion completada en {(time.time()-global_start)/60:.1f} minutos.')
```

Entrenamiento de produccion (ensamble de 5 redes con perdida personalizada)...

```
Red 1/5 | R2 test: 0.1084 | (2.6s)
Red 2/5 | R2 test: 0.1145 | (2.8s)
Red 3/5 | R2 test: 0.1335 | (2.6s)
Red 4/5 | R2 test: 0.1094 | (2.9s)
Red 5/5 | R2 test: 0.1290 | (2.6s)
```

Produccion completada en 0.2 minutos.

7.- Evaluación final sobre eval.csv

Se usa `eval.csv` por primera vez para obtener una estimación real del rendimiento en producción. A continuación se ejecuta un smoke test para verificar que el pipeline de carga e inferencia funciona correctamente.

Nota: Interpretacion del índice Shannon H':

El modelo predice el índice de diversidad de Shannon, que va aproximadamente de `0.0` (sin diversidad) a `4.0-5.0` (alta diversidad biológica).

```
In [8]: NUM_REDES = 5

# Evaluacion final
```

```

print('Evaluacion final sobre eval.csv')
print('-' * 60)

cfg_prod = joblib.load(os.path.join(MODELS_DIR, 'mlp_custom_config.pkl'))
preds_eval = []

for seed_id in range(NUM_REDES):
    model = MLPMultiSalida(
        len(FEATURES_AUTORIZADAS), len(TARGETS),
        cfg_prod['hidden'], cfg_prod['dropout']
    ).to(DEVICE)
    model.load_state_dict(torch.load(
        os.path.join(MODELS_DIR, f'mlp_custom_{seed_id}.pt'), map_location=DEVICE
    ))
    model.eval()
    with torch.no_grad():
        X_eval_t = torch.tensor(X_eval.values, dtype=torch.float32).to(DEVICE)
        preds_eval.append(model(X_eval_t).cpu().numpy())

y_pred_eval = np.mean(preds_eval, axis=0)
y_true_eval = y_eval.values

r2_global = r2_score(y_true_eval, y_pred_eval, multioutput='uniform_average')
rmse_global = np.sqrt(mean_squared_error(y_true_eval, y_pred_eval, multioutput='uniform_average'))
mae_global = mean_absolute_error(y_true_eval, y_pred_eval, multioutput='uniform_average')

print(f' R2 global: {r2_global:.4f}')
print(f' RMSE global: {rmse_global:.4f} puntos de Shannon H\''')
print(f' MAE global: {mae_global:.4f} puntos de Shannon H\''')

# Verificacion de correlaciones sobre eval.csv
corr_real_eval = pd.DataFrame(y_true_eval, columns=TARGETS).corr()
corr_pred_eval = pd.DataFrame(y_pred_eval, columns=TARGETS).corr()
diff_eval = (corr_real_eval - corr_pred_eval).abs().values.mean()
print(f'\n Diferencia media de correlacion en eval.csv: {diff_eval:.4f}')

print('\n R2 por target:')
for t, r2 in zip(TARGETS, r2_score(y_true_eval, y_pred_eval, multioutput='raw_values')):
    print(f' {t:30} {r2:.4f}')

# Smoke test

print('\nSmoke test - Inferencia con datos reales')
print('-' * 60)

try:
    df_new_data = X_eval.sample(n=3, random_state=RANDOM_STATE)
    indices_elegidos = df_new_data.index

    votos = []
    for seed_id in range(NUM_REDES):
        model = MLPMultiSalida(
            len(FEATURES_AUTORIZADAS), len(TARGETS),
            cfg_prod['hidden'], cfg_prod['dropout']
        ).to(DEVICE)
        model.load_state_dict(torch.load(
            os.path.join(MODELS_DIR, f'mlp_custom_{seed_id}.pt'), map_location=DEVICE
        ))
        model.eval()
        with torch.no_grad():
            X_new_t = torch.tensor(df_new_data.values, dtype=torch.float32).to(DEVICE)
            pred = model(X_new_t).cpu().numpy()
            votos.append(pred)
    idx_e = TARGETS.index('earthworm_shannon_z')
    print(f' Red {seed_id+1} | '
          f'Fila {indices_elegidos[0]}: {pred[0, idx_e]:.4f} | '
          f'Fila {indices_elegidos[1]}: {pred[1, idx_e]:.4f} | '
          f'Fila {indices_elegidos[2]}: {pred[2, idx_e]:.4f} (earthworm_shannon_z)')

    consenso = np.mean(votos, axis=0)
    idx_earth = TARGETS.index('earthworm_shannon_z')

    print('-' * 60)
    print(' Resultados detallados por muestra (earthworm_shannon_z):')
    print('-' * 60)

    for i, idx in enumerate(indices_elegidos):
        pred_val = consenso[i, idx_earth]
        real_val = y_eval.loc[idx, 'earthworm_shannon_z']
        error = abs(pred_val - real_val)
        print(f' Fila {idx:<4} | Prediccion: {pred_val:.4f} | '
              f'Valor Real: {real_val:.4f} | Error absoluto: {error:.4f}')

    print('-' * 60)
    print('\nSmoke test superado. El pipeline MLP con perdida personalizada funciona correctamente.')

except Exception as e:
    print(f'Error en el smoke test: {str(e)}')

# Grafico: scatter prediccion vs valor real (targets principales)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, tgt, col in zip(axes, _targets_plot, _colors_plot):
    idx_col = TARGETS.index(tgt)
    yp = y_pred_eval[:, idx_col]
    yt = y_true_eval[:, idx_col]

```

```
ax.scatter(yt, yp, alpha=0.7, s=30, color=col, edgecolors='white', linewidths=0.4)
lim = [min(yt.min(), yp.min()), max(yt.max(), yp.max())]
ax.plot(lim, lim, 'r--', linewidth=1)
ax.set_title(f'{tgt}\nR2={r2_score(yt, yp):.3f}', fontsize=9)
ax.set_xlabel('Valor real', fontsize=8)
ax.set_ylabel('Predicción', fontsize=8)
plt.suptitle('Predicción vs Valor Real - MLP Custom Loss (eval.csv)', fontsize=12)
plt.tight_layout()
plt.show()
```

Evaluación final sobre eval.csv

R2 global: -0.1316
RMSE global: 0.9861 puntos de Shannon H'
MAE global: 0.7187 puntos de Shannon H'

Diferencia media de correlación en eval.csv: 0.1726

R2 por target:

nematode_shannon_z	0.0903
macro_shannon_z	0.2676
earthworm_shannon_z	0.3750
orib_shannon_z	0.2216
meso_shannon_z	-0.6189
coll_shannon_z	-1.3439
bac_shannon_z	-0.1004
fun_shannon_z	-0.0842
euk_shannon_z	0.0868
oomy_shannon_z	0.0588
cerc_shannon_z	-0.1194
macro_order_richness_z	0.3759
earthworm_richness_z	0.4671
orib_species_richness_z	0.2542
meso_species_richness_z	-0.3462
coll_species_richness_z	-2.2411
bac_asv_richness_z	-0.1680
fun_asv_richness_z	0.0027
euk_asv_richness_z	0.1902
oomy_asv_richness_z	-0.0546
cerc_asv_richness_z	-0.0770

Smoke test - Inferencia con datos reales

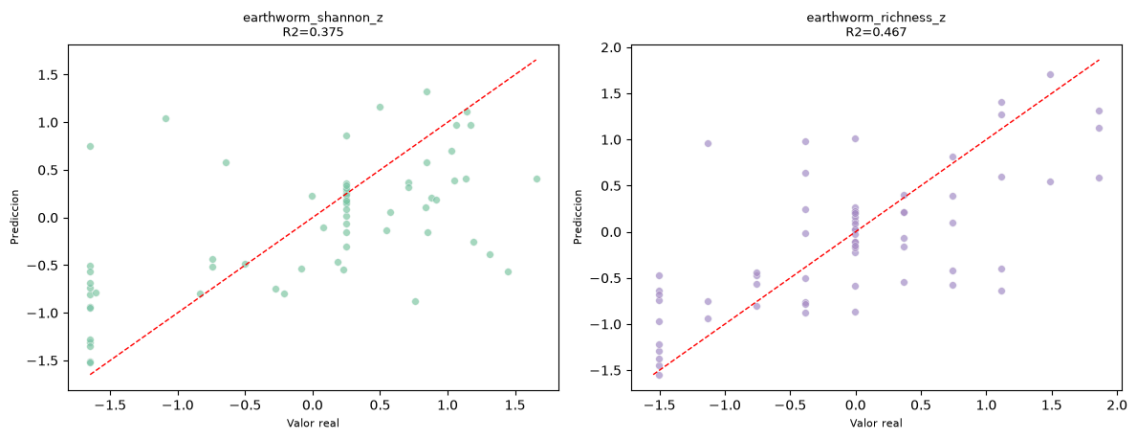
Red 1		Fila 53: 0.1138		Fila 60: 1.2291		Fila 0: -0.5026	(earthworm_shannon_z)
Red 2		Fila 53: 0.4839		Fila 60: 1.1523		Fila 0: -0.4411	(earthworm_shannon_z)
Red 3		Fila 53: 0.5471		Fila 60: 0.9066		Fila 0: -0.5035	(earthworm_shannon_z)
Red 4		Fila 53: 0.4328		Fila 60: 1.0480		Fila 0: -0.4886	(earthworm_shannon_z)
Red 5		Fila 53: 0.4635		Fila 60: 1.2240		Fila 0: -0.6305	(earthworm_shannon_z)

Resultados detallados por muestra (earthworm_shannon_z):

Fila 53		Predicción: 0.4082		Valor Real: 1.1347		Error absoluto: 0.7265
Fila 60		Predicción: 1.1120		Valor Real: 1.1388		Error absoluto: 0.0268
Fila 0		Predicción: -0.5133		Valor Real: -0.7453		Error absoluto: 0.2320

Smoke test superado. El pipeline MLP con pérdida personalizada funciona correctamente.

Predicción vs Valor Real - MLP Custom Loss (eval.csv)



11.3. Scripts y notebooks auxiliares

En este anexo se muestra el código de todos los scripts y notebooks auxiliares empleados para la facilitación de las tareas de ejecución y comparación de resultados.

11.3.1. Script de automatización de notebooks

```
"""
```

Ejecuta de principio a fin varios notebooks (todos los .ipynb de una carpeta, o los que le indiques) y deja constancia de si cada uno termino bien o fallo.

Que hace por cada notebook:

1. Ejecuta todas las celdas en orden (jupyter nbconvert --execute --inplace), sin limite de tiempo por celda.
2. Guarda el resultado sobre el propio notebook (con las tablas/graficas ya generadas dentro).
3. Si un notebook falla (una celda lanza una excepcion), no interrumpe el resto: sigue con el siguiente y lo apunta en el resumen final (output/report/resumen_ejecucion.txt).
4. Si termino bien, hace commit del notebook con git automaticamente.
Usa --no-git para desactivar esto.

Uso:

```
python ejecutar_pruebas.py
    -> ejecuta TODOS los .ipynb del directorio actual (no busca en subcarpetas)

python ejecutar_pruebas.py --recursive
    -> ejecuta TODOS los .ipynb del directorio actual y sus subcarpetas

python ejecutar_pruebas.py prueba4_ensamblado_corregido.ipynb prueba5_ensamblado_stacking.ipynb
    -> ejecuta solo los notebooks que le pases (ignora --pattern/--recursive)

python ejecutar_pruebas.py --pattern "prueba*.ipynb"
    -> vuelve al comportamiento anterior: solo los que empiecen por "prueba"
```

Requisitos: jupyter y nbconvert instalados (pip install jupyter nbconvert ipykernel), y el kernel de Python que usan los notebooks disponible (el que se ve en "kernelspec" -> "name" dentro del .ipynb; por defecto "python3").

AVISO: al ejecutar in-place, el notebook original se sobrescribe con los resultados de la ejecucion. Si quieres conservar el original sin ejecutar, haz una copia antes.

```
"""
```

```
import argparse
import subprocess
import sys
import time
from pathlib import Path

def buscar_notebooks(directorio: Path, pattern: str, recursive: bool) -> list:
    """Busca notebooks segun el patron."""
    buscador = directorio.rglob(pattern) if recursive else directorio.glob(pattern)
    return sorted(buscador)

def ejecutar_notebook(nb_path: Path) -> dict:
    """Ejecuta un notebook de principio a fin, sobrescribiendolo con el resultado,
    y devuelve un resumen. No aplica timeout: cada celda puede tardar lo que
    necesite."""
    inicio = time.time()
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "notebook",
        "--execute",
        "--inplace",
        "--ExecutePreprocessor.timeout=-1",
        str(nb_path),
    ]
```

```
]
resultado = subprocess.run(cmd, capture_output=True, text=True)
duracion = time.time() - inicio

ok = resultado.returncode == 0

return {
    "notebook": nb_path.name,
    "ok": ok,
    "duracion_s": round(duracion, 1),
    "error": None if ok else _extraer_error(resultado.stderr),
}

def _extraer_error(stderr: str) -> str:
    """Se queda con las ultimas lineas del stderr, que suelen tener el traceback util
    (el principio de la salida de nbconvert suele ser solo progreso, no el error)."""
    lineas = [l for l in stderr.strip().split("\n") if l.strip()]
    return "\n".join(lineas[-15:]) if lineas else "Error desconocido (nbconvert no genero stderr)."

def git_commit(paths: list, mensaje: str) -> bool:
    """Hace 'git add' de los paths indicados y un commit con el mensaje dado.
    Devuelve True si el commit se creo, False si fallo o no habia nada que commitear
    (por ejemplo, si el notebook ya estaba igual que en el ultimo commit)."""
    paths_str = [str(p) for p in paths if p is not None]
    if not paths_str:
        return False

    add = subprocess.run(["git", "add", *paths_str], capture_output=True, text=True)
    if add.returncode != 0:
        print(f"[git] fallo 'git add': {add.stderr.strip()}")
        return False

    commit = subprocess.run(
        ["git", "commit", "-m", mensaje], capture_output=True, text=True
    )
    if commit.returncode != 0:
        # Suele ser porque no hay cambios que commitear; no es un error grave.
        if "nothing to commit" in commit.stdout.lower():
            return False
        print(f"[git] fallo 'git commit': {commit.stdout.strip()} {commit.stderr.strip()}")
        return False

    return True

def main():
    parser = argparse.ArgumentParser(
        description="Ejecuta notebooks Jupyter de principio a fin, uno detras de otro."
    )
    parser.add_argument(
        "notebooks", nargs="*",
        help="Notebooks a ejecutar. Si no se indica ninguno, se buscan con --pattern.",
    )
    parser.add_argument(
        "--pattern", default="*.ipynb",
        help="Patron para buscar notebooks si no se pasan explicitamente (default: *.ipynb, es decir, todos)",
    )
    parser.add_argument(
        "--recursive", action="store_true",
        help="Busca tambien en subcarpetas (por defecto solo mira el directorio actual)",
    )
    parser.add_argument(
        "--no-git", action="store_true",
        help="No hace commit automatico tras cada notebook (por defecto SI se commitea)",
    )
    args = parser.parse_args()
```

```

if args.notebooks:
    notebooks = [Path(n) for n in args.notebooks]
    notebooks_existentes = [nb for nb in notebooks if nb.exists()]
    notebooks_faltantes = [nb for nb in notebooks if not nb.exists()]
    for nb in notebooks_faltantes:
        print(f"[AVISO] No existe: {nb} (se omite)")
else:
    notebooks_existentes = buscar_notebooks(Path("."), args.pattern, args.recursive)

if not notebooks_existentes:
    print(f"No se encontro ningun notebook (patron: {args.pattern}, recursive={args.recursive}).")
    print(f"Nada que ejecutar.")
    sys.exit(1)

print(f"Ejecutando {len(notebooks_existentes)} notebook(s) in-place, sin timeout...\n")

resultados = []
for nb_path in notebooks_existentes:
    print(f"-> {nb_path.name} ...", end=" ", flush=True)
    r = ejecutar_notebook(nb_path)
    resultados.append(r)
    if r["ok"]:
        print(f"OK ({r['duracion_s']}s)")
        if not args.no_git:
            mensaje = f"[executor.py] Executed {nb_path.name} without issues. Elapsed time {r['duracion_s']}s..."
            if git_commit([nb_path], mensaje):
                print(f"      [git] commit: {mensaje}")
            else:
                print(f"      [git] sin cambios que commitear")
        else:
            print(f"FALLO ({r['duracion_s']}s)")

print("\n" + "=" * 70)
print("RESUMEN")
print("=" * 70)
for r in resultados:
    estado = "OK" if r["ok"] else "FALLO"
    print(f"{estado} | {r['notebook']}:45s | {r['duracion_s']}:6.1fs")
    if not r["ok"]:
        print(f"      " + r["error"].replace("\n", "\n      "))

resumen_dir = Path("output") / "report"
resumen_dir.mkdir(parents=True, exist_ok=True)
resumen_path = resumen_dir / "resumen_ejecucion.txt"
with open(resumen_path, "w", encoding="utf-8") as f:
    for r in resultados:
        estado = "OK" if r["ok"] else "FALLO"
        f.write(f"{estado} | {r['notebook']} | {r['duracion_s']}s\n")
        if not r["ok"]:
            f.write(r["error"] + "\n")
        f.write("-" * 70 + "\n")
print(f"\nResumen guardado en {resumen_path}")

n_fallos = sum(1 for r in resultados if not r["ok"])
if n_fallos:
    print(f"\n{n_fallos} de {len(resultados)} notebook(s) fallaron.")
    sys.exit(1 if n_fallos else 0)

if __name__ == "__main__":
    main()

```

El objetivo principal de este script es el de automatizar la ejecución de todos los notebooks (o los que se indiquen) localizados dentro de una misma rama, permitiendo así no tener que ejecutarlos uno a uno.

El script también cuenta con las siguientes capacidades:

1. Capacidad de hacer commits en local al terminar de ejecutar correctamente cada notebook.
2. Al final de la ejecución de cada notebook genera un archivo de texto en el que se muestra si en alguno de los notebooks hubo algún error o no.
3. En caso de que ocurra un error en uno de los notebooks, se registra para añadirlo en el reporte final (archivo de texto) y se sigue con el siguiente notebook, es decir, la ejecución de los notebooks no se para en el caso de que uno de los notebooks a ejecutar devuelva un error.
4. Si hay un notebook que no exista dentro de la rama en la que se ejecuta, el script lo omite y ejecuta los que se haya indicado.

11.3.2. Script de automatización general

```
echo "AUTOMATED WORK STARTED DO NOT TOUCH"

# 0 nombre de la carpeta/repositorio (se ejecuta desde la carpeta anterior al repositorio)
cd TFG-SOB4ES

# Creamos un array con las ramas, aqui se ponen las ramas que se quieren ejecutar
ramas = ("model-prep" "model-prep-var" "model-prep-var-1" "model-prep-var-2" "model-prep-var-3"
"model-prep-var-1-2" "model-prep-var-1-3" "model-prep-var-2-3" "model-prep-var-1-2-3" "model-prep-
rs" "model-prep-rs-1" "model-prep-rs-group" "model-prep-rs-group-1" "model-prep-mixin" "model-prep-
mixin-1")

# Recorrer el array
for rama in "${ramas[@]}; do
    echo "In branch $rama ..."
    git stash
    git fetch
    git pull

    # Se ponen los nombres de los notebooks a ejecutar
    python executor.py reg_model.ipynb rf_model.ipynb #... y así sucesivamente

    # cuando se terminan de ejecutar los notebooks se les hace el commit y se suben a remoto
    git add *
    git commit -m "[exec.sh] Executed all models from $rama ..."
    # Esto solo funciona si se ejecuta dentro de una IDE con las credenciales de GitHub cargadas
    git push
done

git switch model-prep # o la rama en la que se quiera dejar post ejecución

echo "Trabajo terminado..."
```

Este script permite la automatización de la ejecución de todos los notebooks o de los notebooks indicados dentro de distintas ramas.

El script cuenta con las siguientes capacidades:

1. Capacidad de realizar commits en local y subirlos a remoto cuando se completa la ejecución de una rama completa. Esto es si se ejecuta dentro de un IDE con el plugin de GitHub instalado, en caso contrario subir a remoto va a ser imposible.
2. Capacidad de saltar entre ramas del mismo repositorio y actualizarlas en el caso de que haya commits previos.

Nota: Si se crean conflictos a partir de la recuperación de commits en remoto, el script no podrá hacer nada.

El script no tiene la capacidad de resolver conflictos ni hacer merges a las branches remotas debido a que es una tarea que debería de ser realizada por un humano no un script.

11.3.3. Notebook de división de datos

División de Dataset: Train / Test / Evaluación

Este notebook divide el dataset `sob4es_final_model_ready.csv` en tres subconjuntos:

Split	Proporción	Descripción
Entrenamiento	70 %	Ajuste de parámetros del modelo
Pruebas	15 %	Selección de hiperparámetros / evaluación durante desarrollo
Evaluación	15 %	Estimación final e imparcial del rendimiento

Para la división de los diferentes datasets, se hace uso del país de origen de la muestra, eso se determina a partir de las 2 letras de cada uno de los 'site_id' del dataset original. De esta forma nos podemos asegurar cierta representación geoespacial en los tres subconjuntos resultantes.

Nota: IT, al solo tener 2 entradas y ese ser el mínimo para poder realizar actividades de aprendizaje, se han anexionado a otro de los países.

1.- Importaciones y configuración del entorno

```
In [9]: import pandas as pd
import numpy as np
import itertools
import os

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from matplotlib.colors import LinearSegmentedColormap

from sklearn.model_selection import train_test_split

# Parámetros configurables
INPUT_PATH = './input/sob4es_final_model_ready.csv' # ruta al CSV original
OUTPUT_DIR = './input/' # carpeta de salida
TEST_SIZE = 0.30 # fracción para temp (test + eval juntos)
EVAL_FRAC = 0.50 # fracción del temp que irá a evaluación
RANDOM_SEED = 42

print("Entorno cargado correctamente...")

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#9BBFE0',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)
```

Entorno cargado correctamente...

2.- Carga e inspección inicial

En esta sección del notebook cargamos el dataset resultante del proceso de preparación de datos y le damos una primera inspección inicial

```
In [10]: df = pd.read_csv(INPUT_PATH)

print(f"Filas totales : {len(df)}")
print(f"Columnas      : {len(df.columns)}")
print(f"Valores nulos : {df.isnull().sum().sum()}")
df.head(3)
```

Filas totales : 428
Columnas : 71
Valores nulos : 0

```
Out[10]:
```

	site_id	latitude	longitude	outlier_flag	total_plant_cover_z	clay_content_z	silt_content_z	sand_content_z	aggregate_stability_z	bulk_der
0	BE_001	51.099	4.975	True	0.466314	-0.270211	-0.178983	0.269579	0.545059	-1.7
1	BE_002	51.102	4.978	True	0.466314	-0.606795	-0.676556	0.778451	0.315437	-0.7
2	BE_003	51.097	4.976	False	0.466314	-1.286227	-1.349829	1.596978	0.926022	-0.0

3 rows x 11 columns

```
In [11]: # Extraer país a partir del prefijo de site_id
df['country'] = df['site_id'].str[:2]

print("Distribución por país:")
print(df['country'].value_counts().to_string())
print()
print("Distribución outlier_flag:")
print(df['outlier_flag'].value_counts().to_string())
```

Distribución por país:

country	count
IL	51
RO	51
CH	50
IE	45
SI	45
BE	36
ES	36
NL	33
FR	29
SE	27
DE	23
IT	2

Distribución outlier_flag:

outlier_flag	count
True	347
False	81

3.- División de los datos

En esta sección del notebook nos centraremos en dividir los datos en base al criterio indicado al principio de este notebook (**país dónde se tomó la muestra**).

En el caso de Italia, está se agrupa con DE (Alemania)

```
In [12]: # Italia (IT) tiene solo 2 filas → no se puede estratificar directamente.
# Se agrupa con DE únicamente para el muestreo (mismo bloque geográfico central).
df['strat'] = df['country'].replace({'IT': 'DE'})

# Primera partición: train vs temp (test + eval)
train, temp = train_test_split(
    df,
    test_size=TEST_SIZE,
    random_state=RANDOM_SEED,
    stratify=df['strat']
)

# Segunda partición: temp → test y eval
test, eval_ = train_test_split(
    temp,
    test_size=EVAL_FRAC,
    random_state=RANDOM_SEED,
    stratify=temp['strat']
)

# Eliminar columnas auxiliares
for d in [train, test, eval_]:
    d.drop(columns=['country', 'strat'], inplace=True)

print(f"Train : {len(train):>4} filas ({len(train)/len(df)*100:.1f}%)")
print(f"Test : {len(test):>4} filas ({len(test)/len(df)*100:.1f}%)")
print(f"Eval : {len(eval_):>4} filas ({len(eval_)/len(df)*100:.1f}%)")
print(f"Total : {len(train)+len(test)+len(eval_):>4} filas")
```

Train : 299 filas (69.9%)
 Test : 64 filas (15.0%)
 Eval : 65 filas (15.2%)
 Total : 428 filas

4.- Verificación de la distribución

En esta sección verificamos que las distribuciones aplicadas en la división con las distribuciones deseadas.

```
In [13]: # Re-añadir country para análisis (no se guardará en los CSVs)
for d, name in [(train, 'Train'), (test, 'Test'), (eval_, 'Eval')]:
    d['_country'] = d['site_id'].str[:2]

comp = pd.DataFrame({
```

```

'Total' : df['site_id'].str[:2].value_counts(),
'Train' : train['_country'].value_counts(),
'Test'  : test['_country'].value_counts(),
'Eval'  : eval['_country'].value_counts(),
}).fillna(0).astype(int)

comp['Train %'] = (comp['Train'] / comp['Total'] * 100).round(1)
comp.sort_values('Total', ascending=False)

```

```
Out[13]:
```

	Total	Train	Test	Eval	Train %
RO	51	36	7	8	70.6
IL	51	36	7	8	70.6
CH	50	35	7	8	70.0
SI	45	31	7	7	68.9
IE	45	31	7	7	68.9
BE	36	25	6	5	69.4
ES	36	25	5	6	69.4
NL	33	23	5	5	69.7
FR	29	20	5	4	69.0
SE	27	19	4	4	70.4
DE	23	18	3	2	78.3
IT	2	0	1	1	0.0

```

In [14]: # Verificar outlier_flag en los tres splits
flag_stats = pd.DataFrame({
    'Total' : df['outlier_flag'].value_counts(normalize=True).mul(100).round(1),
    'Train' : train['outlier_flag'].value_counts(normalize=True).mul(100).round(1),
    'Test'  : test['outlier_flag'].value_counts(normalize=True).mul(100).round(1),
    'Eval'  : eval['outlier_flag'].value_counts(normalize=True).mul(100).round(1),
}).rename(index={True: 'outlier=True (%)', False: 'outlier=False (%)'})

print("Distribución de outlier_flag (%) por split:")
flag_stats

```

Distribución de outlier_flag (%) por split:

```
Out[14]:
```

	Total	Train	Test	Eval
outlier_flag				
outlier=True (%)	81.1	81.3	81.2	80.0
outlier=False (%)	18.9	18.7	18.8	20.0

5.- Visualización

En esta sección se muestra de forma visual las divisiones realizadas entre otros datos relevantes.

Para esto se hace uso de la librería matplotlib.

```

In [15]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico 1: tamaño de splits
sizes = [len(train), len(test), len(eval_)]
labels = [f'Train\n{len(train)} ({len(train)/len(df)*100:.0f}%)',
          f'Test\n{len(test)} ({len(test)/len(df)*100:.0f}%)',
          f'Eval\n{len(eval_)} ({len(eval_)/len(df)*100:.0f}%)']
colors = [PALETTE['blue'], PALETTE['purple'], PALETTE['green']]

axes[0].pie(sizes, labels=labels, colors=colors,
            autopct='', startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[0].set_title('Distribución de splits', fontsize=13, fontweight='bold')

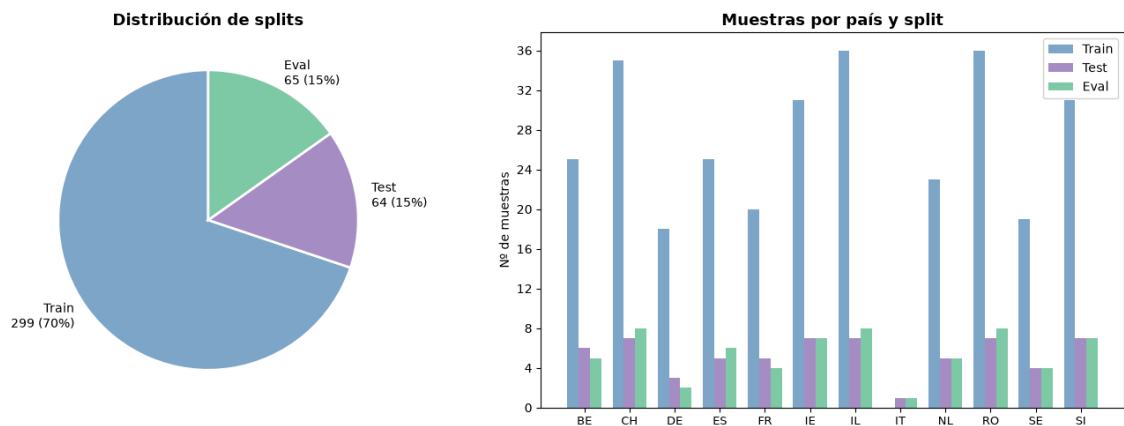
# Gráfico 2: distribución por país en cada split
countries = comp.index.tolist()
x = np.arange(len(countries))
w = 0.25

axes[1].bar(x - w, comp['Train'], w, label='Train', color=PALETTE['blue'])
axes[1].bar(x, comp['Test'], w, label='Test', color=PALETTE['purple'])
axes[1].bar(x + w, comp['Eval'], w, label='Eval', color=PALETTE['green'])

axes[1].set_xticks(x)
axes[1].set_xticklabels(countries, fontsize=9)
axes[1].set_ylabel('N° de muestras')
axes[1].set_title('Muestras por país y split', fontsize=13, fontweight='bold')
axes[1].legend()
axes[1].yaxis.set_major_locator(mticker.MaxNLocator(integer=True))

plt.tight_layout()
plt.show()

```



6.- Guardado de los datasets

Una vez realizado todo el proceso, se procede a guardar los subconjuntos resultantes en formato .csv

```
In [16]: # Quitar columna auxiliar _country antes de guardar
for d in [train, test, eval_]:
    if '_country' in d.columns:
        d.drop(columns=['_country'], inplace=True)

train.to_csv(os.path.join(OUTPUT_DIR, 'train.csv'), index=False)
test.to_csv(os.path.join(OUTPUT_DIR, 'test.csv'), index=False)
eval_.to_csv(os.path.join(OUTPUT_DIR, 'eval.csv'), index=False)

print("train.csv guardado:", len(train), "filas...")
print("test.csv guardado:", len(test), "filas...")
print("eval.csv guardado:", len(eval_), "filas...")

train.csv guardado: 299 filas...
test.csv guardado: 64 filas...
eval.csv guardado: 65 filas...
```

11.3.4. Notebooks para la comparación de modelos

11.3.4.1. Comparador de regresión

SOB4ES - Comparación de Modelos

CRISP-ML(Q) - Fase 5: Evaluación y selección del modelo final

Este notebook carga los modelos entrenados en los notebooks anteriores y compara su rendimiento sobre `eval.csv` de forma sistemática.

Modelos comparados:

1. Random Forest individual (modelos por target)
2. XGBoost individual (modelos por target)
3. Ridge Regression individual (modelos por target)
4. Random Forest multisalida
5. XGBoost multisalida
6. Ensemble de RegressorChains
7. MLP multisalida estándar
8. MLP con pérdida personalizada de correlación

Métricas de comparación:

- R2 global (promedio sobre todos los targets)
- R2 por target (para identificar grupos biológicos difíciles de predecir)
- RMSE y MAE globales

1.- Configuración

Configuración inicial del entorno, con librerías, variables globales y todo cargado.

```
In [1]: import os
import warnings
import joblib
import pandas as pd
import numpy as np
import itertools

import matplotlib.pyplot as plt

import torch
import torch.nn as nn

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.metrics import precision_score, recall_score, f1_score

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_BASE = 'output/models/'

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
```

```

'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Configuracion cargada.')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD8F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

```

Configuracion cargada.

2.- Carga de eval.csv

Se carga únicamente el dataset de evaluación final. Ninguno de los modelos ha visto estos datos durante el entrenamiento ni el tuning.

```

In [2]: df_eval = pd.read_csv(os.path.join(DATA_DIR, 'eval.csv'))
X_eval = df_eval[FEATURES_AUTORIZADAS]
y_eval = df_eval[TARGETS]

print(f'eval.csv cargado: {X_eval.shape[0]} muestras x {X_eval.shape[1]} variables')

```

eval.csv cargado: 65 muestras x 34 variables

3.- Funciones auxiliares

Se definen las funciones necesarias para cargar cada tipo de modelo y calcular sus predicciones sobre `eval.csv`.

```

In [3]: def metricas(y_true, y_pred, nombre):
    """Calcula R2, RMSE y MAE globales y devuelve un dict con los resultados."""
    r2 = r2_score(y_true, y_pred, multioutput='uniform_average')
    rmse = np.sqrt(mean_squared_error(y_true, y_pred, multioutput='uniform_average'))
    mae = mean_absolute_error(y_true, y_pred, multioutput='uniform_average')
    r2_targets = r2_score(y_true, y_pred, multioutput='raw_values')
    return {'modelo': nombre, 'R2': round(r2, 4), 'RMSE': round(rmse, 4), 'MAE': round(mae, 4),
            'r2_targets': r2_targets}

def predecir_ensemble_pkl(carpeta, prefijo, sufijo, n, X):
    """Carga n modelos .pkl y devuelve el promedio de sus predicciones."""
    preds = []
    for i in range(n):
        m = joblib.load(os.path.join(MODELS_BASE, carpeta, f'{prefijo}{i}{sufijo}.pkl'))
        preds.append(m.predict(X))
    return np.mean(preds, axis=0)

def predecir_ensemble_por_target(carpeta, prefijo_tpl, n, X):
    """
    Para modelos individuales (un modelo por target):
    carga n modelos por target y devuelve la matriz de predicciones.
    """

```

```

prefijo_tpl debe ser una funcion que recibe (target, seed_id) y devuelve el nombre del archivo.
"""
y_pred = np.zeros((len(X), len(TARGETS)))
for j, target in enumerate(TARGETS):
    preds_t = []
    for i in range(n):
        ruta = os.path.join(MODELS_BASE, carpeta, prefijo_tpl(target, i))
        m = joblib.load(ruta)
        preds_t.append(m.predict(X))
    y_pred[:, j] = np.mean(preds_t, axis=0)
return y_pred

class MLPMultiSalida(nn.Module):
    def __init__(self, n_inputs, n_outputs, hidden_sizes, dropout_rate):
        super().__init__()
        layers, in_size = [], n_inputs
        for h in hidden_sizes:
            layers += [nn.Linear(in_size, h), nn.ReLU(), nn.Dropout(dropout_rate)]
            in_size = h
        layers.append(nn.Linear(in_size, n_outputs))
        self.red = nn.Sequential(*layers)
    def forward(self, x): return self.red(x)

def predecir_ensamble_mlp(carpeta, prefijo, n, X, config_file):
    """Carga n redes PyTorch y devuelve el promedio de sus predicciones."""
    cfg = joblib.load(os.path.join(MODELS_BASE, carpeta, config_file))
    X_t = torch.tensor(X.values, dtype=torch.float32).to(DEVICE)
    preds = []
    for i in range(n):
        model = MLPMultiSalida(X.shape[1], len(TARGETS), cfg['hidden'], cfg['dropout']).to(DEVICE)
        model.load_state_dict(torch.load(
            os.path.join(MODELS_BASE, carpeta, f'{prefijo}{i}.pt'), map_location=DEVICE
        ))
        model.eval()
        with torch.no_grad():
            preds.append(model(X_t).cpu().numpy())
    return np.mean(preds, axis=0)

print('Funciones auxiliares definidas.')

```

Funciones auxiliares definidas.

4.- Predicciones de cada modelo

Se cargan y ejecutan todos los modelos sobre `eval.csv`. Este proceso puede tardar varios minutos dependiendo del número de modelos y el hardware disponible.

```

In [4]: resultados = []
y_true = y_eval.values

print('Cargando modelos y generando predicciones...\n')

# 1. Random Forest individual
print(' [1/8] Random Forest individual...')
y_pred = predecir_ensamble_por_target('rf', lambda t, i: f'rf_prod_{t}_m{i}.pkl', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'RF Individual'))

# 2. XGBoost individual
print(' [2/8] XGBoost individual...')
y_pred = predecir_ensamble_por_target('xgb', lambda t, i: f'xgb_prod_{t}_exp{i}.pkl', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'XGBoost Individual'))

# 3. Ridge individual
print(' [3/8] Ridge Regression...')
y_pred_ridge = np.zeros((len(X_eval), len(TARGETS)))
for j, target in enumerate(TARGETS):
    m = joblib.load(os.path.join(MODELS_BASE, 'reg', f'ridge_prod_{target}.pkl'))
    y_pred_ridge[:, j] = m.predict(X_eval)
resultados.append(metricas(y_true, y_pred_ridge, 'Regresión Individual'))

# 4. RF multisalida
print(' [4/8] Random Forest multisalida...')
y_pred = predecir_ensamble_pkl('rf_multi', 'rf_multi_m', '', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'RandomForest Multisalida'))

# 5. XGBoost multisalida
print(' [5/8] XGBoost multisalida...')
y_pred = predecir_ensamble_pkl('xgb_multi', 'xgb_multi_exp', '', 5, X_eval)
resultados.append(metricas(y_true, y_pred, 'XGBoost Multisalida'))

# 6. RegressorChain
print(' [6/8] Ensemble RegressorChain...')
N_CHAINS = 10
preds_chain = []
for chain_id in range(N_CHAINS):
    cadena = joblib.load(os.path.join(MODELS_BASE, 'chain', f'chain_{chain_id}.pkl'))
    preds_chain.append(cadena.predict(X_eval))
y_pred = np.mean(preds_chain, axis=0)
resultados.append(metricas(y_true, y_pred, 'Random Forest con RegressorChain'))

```

```
# 7. MLP estandar
print(' [7/8] MLP multisalida estandar...')
y_pred = predecir_ensamble_mlp('mlp', 'mlp_prod_', 5, X_eval, 'mlp_config.pkl')
resultados.append(metricas(y_true, y_pred, 'MLP Estandar'))

# 8. MLP con perdida personalizada
print(' [8/8] MLP perdida personalizada...')
y_pred = predecir_ensamble_mlp('mlp_custom', 'mlp_custom_', 5, X_eval, 'mlp_custom_config.pkl')
resultados.append(metricas(y_true, y_pred, 'MLP Perdida Custom'))

print('\nPredicciones completadas...')
```

Cargando modelos y generando predicciones...

```
[1/8] Random Forest individual...
[2/8] XGBoost individual...
[3/8] Ridge Regression...
[4/8] Random Forest multisalida...
[5/8] XGBoost multisalida...
[6/8] Ensemble RegressorChain...
[7/8] MLP multisalida estandar...
[8/8] MLP perdida personalizada...
```

Predicciones completadas...

5.- Tabla comparativa global

Se muestra la comparación ordenada por R2 descendente.

El mejor modelo es el que tiene R2 más alto y RMSE/MAE más bajos.

```
In [5]: tabla = pd.DataFrame([{'Modelo': r['modelo'],
                              'R2 global': r['R2'],
                              'RMSE': r['RMSE'],
                              'MAE': r['MAE']}
                             for r in resultados]).sort_values('R2 global', ascending=False).reset_index(drop=True)

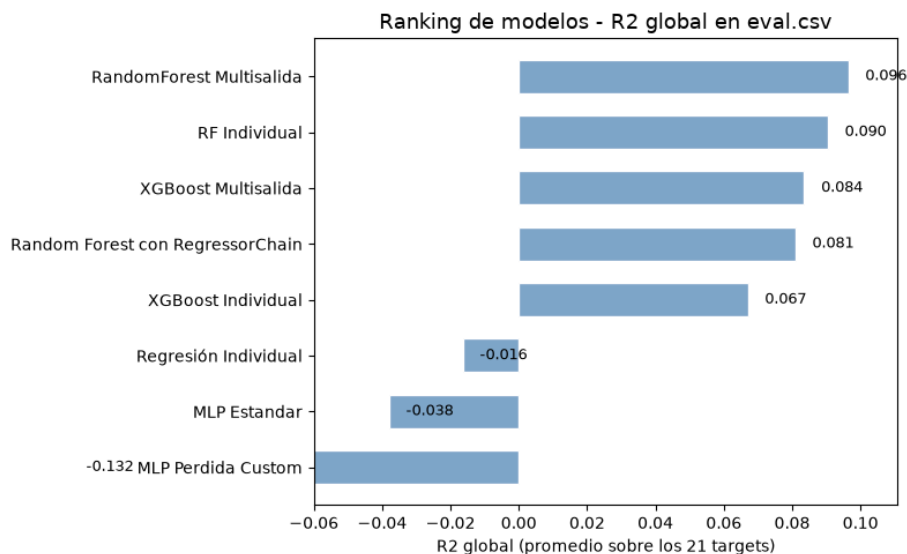
tabla.index += 1 # ranking desde 1
print('Comparacion de modelos (ordenado por R2 descendente):')
print('=' * 60)
print(tabla.to_string())
print('=' * 60)
print(f'\nMejor modelo: {tabla.iloc[0]["Modelo"]} (R2 = {tabla.iloc[0]["R2 global"]})')

# Grafico: ranking de modelos por R2 global
fig, ax = plt.subplots(figsize=(8, 5))
_orden = tabla.sort_values('R2 global')
ax.barh(_orden['Modelo'], _orden['R2 global'], color=PALETTE['blue'], edgecolor='white', height=0.6)
for i, v in enumerate(_orden['R2 global']):
    ax.text(v + 0.005, i, f'{v:.3f}', va='center', fontsize=9)
ax.set_xlabel('R2 global (promedio sobre los 21 targets)')
ax.set_title('Ranking de modelos - R2 global en eval.csv', fontsize=13)
ax.set_xlim(-0.060, _orden['R2 global'].max() * 1.15)
plt.tight_layout()
plt.show()
```

Comparacion de modelos (ordenado por R2 descendente):

```
=====
          Modelo  R2 global  RMSE  MAE
1  RandomForest Multisalida  0.0964  0.9198  0.7026
2           RF Individual  0.0904  0.9178  0.6926
3  XGBoost Multisalida  0.0835  0.9187  0.6838
4  Random Forest con RegressorChain  0.0809  0.9184  0.6860
5  XGBoost Individual  0.0670  0.9300  0.7058
6  Regresión Individual -0.0164  0.9665  0.7444
7           MLP Estandar -0.0380  0.9597  0.7042
8  MLP Perdida Custom -0.1316  0.9861  0.7187
=====
```

Mejor modelo: RandomForest Multisalida (R2 = 0.0964)



6.- Comparación por target

Se muestra el R2 de cada modelo para cada grupo biológico.

Esto permite identificar para que grupos funciona mejor cada enfoque y si hay grupos que ninguno de los modelos predice bien (posiblemente por falta de datos o por relaciones muy ruidosas con las variables predictoras).

```
In [6]: # Construimos una tabla con R2 por target para cada modelo
filas = []
for r in resultados:
    fila = {'Target': 'GLOBAL', 'Modelo': r['modelo'], 'R2': r['R2']}
    filas.append(fila)
    for t, r2_t in zip(TARGETS, r['r2_targets']):
        filas.append({'Target': t, 'Modelo': r['modelo'], 'R2': round(float(r2_t), 4)})

df_r2 = pd.DataFrame(filas).pivot(index='Target', columns='Modelo', values='R2')

# Ordenamos los targets: GLOBAL primero, luego por R2 medio descendente
orden = ['GLOBAL'] + [t for t in TARGETS]
df_r2 = df_r2.loc[orden]

# Ordenamos las columnas por R2 global descendente
orden_modelos = tabla['Modelo'].tolist()
df_r2 = df_r2[orden_modelos]

print('R2 por target y modelo:')
print('=' * 120)
print(df_r2.to_string())
print('=' * 120)
print('\nTargets con R2 medio < 0.2 en todos los modelos (posiblemente difíciles de predecir):')
media_por_target = df_r2.drop('GLOBAL').mean(axis=1)
dificiles = media_por_target[media_por_target < 0.2]
if len(dificiles) > 0:
    for t, v in dificiles.items():
        print(f' {t:30} R2 medio: {v:.4f}')
else:
    print(' Ninguno.')

# Grafico: R2 por modelo para los targets principales (Shannon y Richness de lombrices)
_targets_plot = ['earthworm_shannon_z', 'earthworm_richness_z']
_colors_plot = [PALETTE['green'], PALETTE['purple']] # colores fijos del proyecto: shannon / richness

_df_main = df_r2.loc[_targets_plot]
_x = np.arange(len(orden_modelos))
_width = 0.38

fig, ax = plt.subplots(figsize=(11, 5))
for i, (tgt, col) in enumerate(zip(_targets_plot, _colors_plot)):
    offset = (i - 0.5) * _width
    bars = ax.bar(_x + offset, _df_main.loc[tgt, orden_modelos], _width,
                  label=tgt, color=col, edgecolor='white')
    ax.bar_label(bars, fmt='%.2f', fontsize=7, padding=2)

ax.set_xticks(_x)
ax.set_xticklabels(orden_modelos, rotation=30, ha='right')
ax.set_ylabel('R2')
ax.set_title('R2 por modelo - earthworm_shannon_z vs earthworm_richness_z (eval.csv)', fontsize=13)
ax.axhline(0, color=PALETTE['gray'], linewidth=0.8)
ax.legend()
plt.tight_layout()
plt.show()
```

```
# Grafico: R2 medio (Shannon + Richness) por modelo
_media_principal = _df_main.loc[_targets_plot, orden_modelos].mean(axis=0).sort_values(ascending=False)

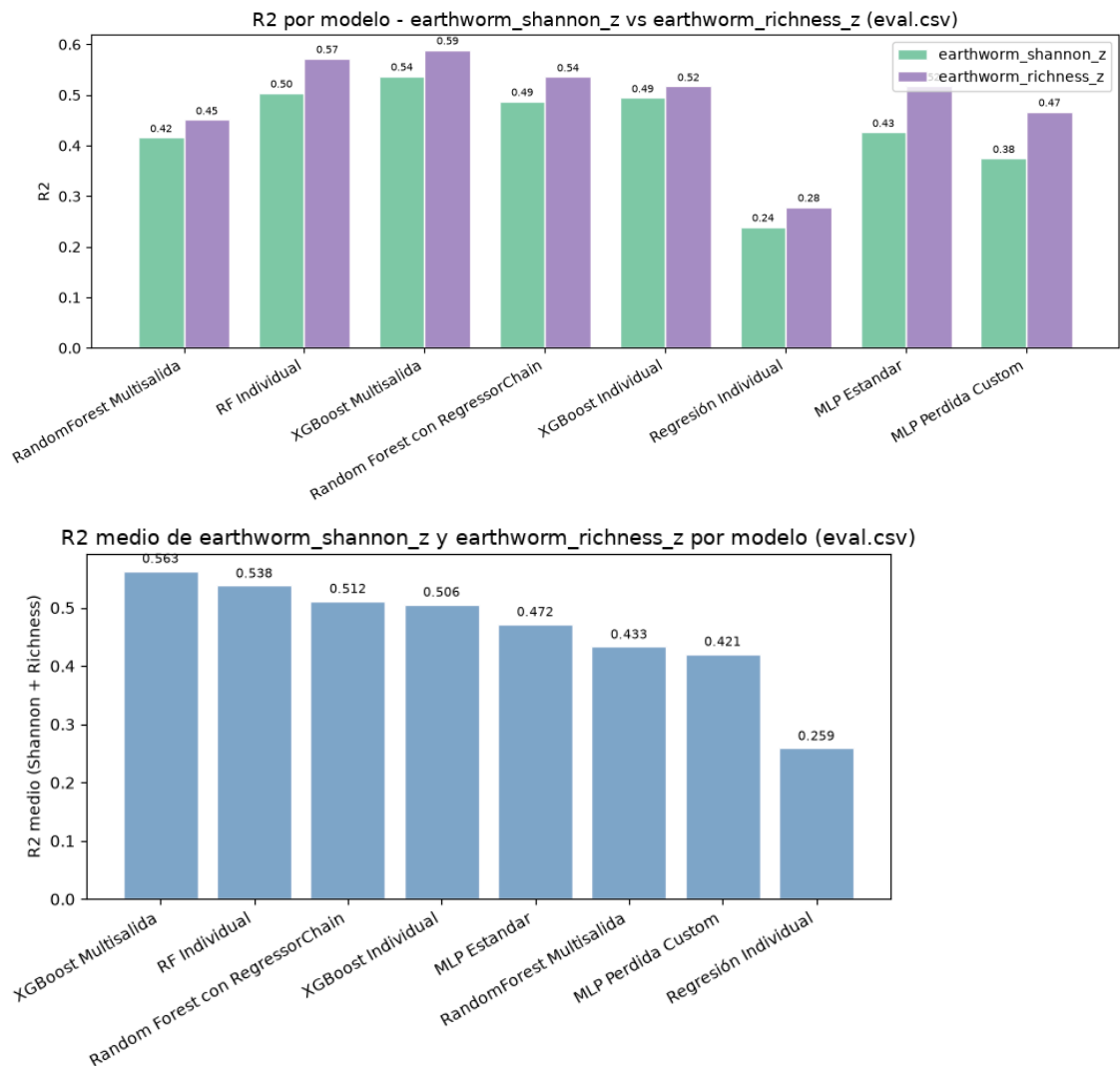
fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(_media_principal.index, _media_principal.values, color=PALETTE['blue'], edgecolor='white')
ax.bar_label(bars, fmt='%.3f', fontsize=8, padding=3)
ax.set_xticklabels(_media_principal.index, rotation=30, ha='right')
ax.set_ylabel('R2 medio (Shannon + Richness)')
ax.set_title('R2 medio de earthworm_shannon_z y earthworm_richness_z por modelo (eval.csv)', fontsize=13)
ax.axhline(0, color=PALETTE['gray'], linewidth=0.8)
plt.tight_layout()
plt.show()
```

R2 por target y modelo:

Modelo	RandomForest	Multisalida	RF Individual	XGBoost Multisalida	Random Forest con RegressorChain	XGB
oost Individual	Regresión Individual	MLP Estandar	MLP Perdida Custom			
Target						
GLOBAL		0.0964	0.0904		0.0835	0.0809
0.0670	-0.0164	-0.0380	-0.1316			
nematode_shannon_z		0.1545	0.1693		0.2228	0.1467
0.1637	0.0112	0.1180	0.0903			
macro_shannon_z		0.2000	0.3199		0.3771	0.2804
0.2601	0.1257	0.2571	0.2676			
earthworm_shannon_z		0.4160	0.5041		0.5375	0.4876
0.4946	0.2395	0.4264	0.3750			
orib_shannon_z		0.1693	0.1794		0.2275	0.2069
0.1147	0.1137	0.2356	0.2216			
meso_shannon_z		-0.1178	-0.1731		-0.2861	-0.1640
-0.1868	-0.2499	-0.4898	-0.6189			
coll_shannon_z		-0.1953	-0.3489		-0.3229	-0.6087
-0.3312	-0.4998	-0.8227	-1.3439			
bac_shannon_z		0.0290	0.0077		-0.0151	0.0288
0.0092	-0.0812	-0.0854	-0.1004			
fun_shannon_z		-0.0127	-0.0455		-0.0502	-0.0219
-0.0206	-0.0161	-0.0757	-0.0842			
euk_shannon_z		0.1378	0.1683		0.0886	0.1624
0.0912	0.0407	0.1453	0.0868			
oomy_shannon_z		0.1252	0.0832		0.0620	0.0913
0.0965	0.0857	0.1534	0.0588			
cerc_shannon_z		-0.0041	-0.0318		-0.0330	-0.0006
-0.0414	0.0155	-0.1361	-0.1194			
macro_order_richness_z		0.2532	0.3417		0.4013	0.4069
0.3082	0.1521	0.3525	0.3759			
earthworm_richness_z		0.4509	0.5722		0.5893	0.5359
0.5171	0.2789	0.5176	0.4671			
orib_species_richness_z		0.2386	0.2034		0.2800	0.2026
0.1259	0.0921	0.2596	0.2542			
meso_species_richness_z		-0.0255	-0.0438		-0.1146	-0.0400
-0.0592	-0.0959	-0.2753	-0.3462			
coll_species_richness_z		-0.2920	-0.5484		-0.6465	-0.5799
-0.5803	-0.7341	-1.4628	-2.2411			
bac_asv_richness_z		0.0044	0.0042		0.0155	-0.0052
0.0014	-0.0761	-0.1464	-0.1680			
fun_asv_richness_z		-0.0068	-0.0256		-0.0662	-0.0186
-0.0242	0.0087	0.0145	0.0027			
euk_asv_richness_z		0.1903	0.2520		0.2233	0.2690
0.1671	0.0400	0.2164	0.1902			
oomy_asv_richness_z		0.1917	0.1843		0.1650	0.2019
0.1776	0.1035	0.0113	-0.0546			
cerc_asv_richness_z		0.1171	0.1250		0.0983	0.1171
0.1227	0.1019	-0.0120	-0.0770			

Targets con R2 medio < 0.2 en todos los modelos (posiblemente difíciles de predecir):

nematode_shannon_z	R2 medio: 0.1346
orib_shannon_z	R2 medio: 0.1836
meso_shannon_z	R2 medio: -0.2858
coll_shannon_z	R2 medio: -0.5592
bac_shannon_z	R2 medio: -0.0259
fun_shannon_z	R2 medio: -0.0409
euk_shannon_z	R2 medio: 0.1151
oomy_shannon_z	R2 medio: 0.0945
cerc_shannon_z	R2 medio: -0.0439
meso_species_richness_z	R2 medio: -0.1251
coll_species_richness_z	R2 medio: -0.8856
bac_asv_richness_z	R2 medio: -0.0463
fun_asv_richness_z	R2 medio: -0.0144
euk_asv_richness_z	R2 medio: 0.1935
oomy_asv_richness_z	R2 medio: 0.1226
cerc_asv_richness_z	R2 medio: 0.0741



7.- Exportación de las comparaciones de regresión

```
In [7]: os.makedirs('output/regresion', exist_ok=True)

# Guardamos la tabla comparativa para referencia futura
tabla.to_csv('output/regresion/comparacion_modelos.csv', index=False)
df_r2.to_csv('output/regresion/r2_por_target.csv')

print('Resultados guardados en output/regresion/comparacion_modelos.csv y output/regresion/r2_por_target.csv')
```

Resultados guardados en output/regresion/comparacion_modelos.csv y output/regresion/r2_por_target.csv

8.- Prueba de clasificación

En este notebook también se realiza una pequeña prueba de clasificación para comprobar si los rankings cambian cuando se hace las pruebas teniendo en cuenta los resultados de regresión o no.

```
In [8]: # Paso 1: Carga de datos y preparación segura de percentiles
print("Iniciando Análisis Avanzado con Métricas de Clasificación Blanda...")
try:
    # Intentamos cargar train.csv usando la ruta configurada en el notebook
    df_train_tmp = pd.read_csv(os.path.join(DATA_DIR, 'train.csv'))
    df_combinado = pd.concat([df_train_tmp[TARGETS], df_eval[TARGETS]], axis=0)
    print("[INFO] Cargado train.csv con éxito. Umbrales calculados sobre train + eval.")
except Exception as e:
    df_combinado = df_eval[TARGETS]
    print("[WARN] No se pudo cargar train.csv, usando solo eval.csv para umbrales:", e)

os.makedirs('output/regresion/graphs', exist_ok=True)

y_true = y_eval.values

# Paso 2: Recalcular predicciones de todos los modelos de forma explícita y segura
```

```

dicc_preds = {}
print("\nGenerando predicciones para los 8 modelos...")

# 1. Random Forest individual
print(" [1/8] Recalculando RF Individual...")
dicc_preds['RF Individual'] = predecir_ensamble_por_target('rf', lambda t, i: f'rf_prod_{t}_m{i}.pkl', 5, X_eval)

# 2. XGBoost individual
print(" [2/8] Recalculando XGBoost Individual...")
dicc_preds['XGBoost Individual'] = predecir_ensamble_por_target('xgb', lambda t, i: f'xgb_prod_{t}_exp{i}.pkl', 5, X_eval)

# 3. Ridge individual
print(" [3/8] Recalculando Regresión Individual (Ridge)...")
y_pred_ridge = np.zeros((len(X_eval), len(TARGETS)))
for j, target in enumerate(TARGETS):
    m = joblib.load(os.path.join(MODELS_BASE, 'reg', f'ridge_prod_{target}.pkl'))
    y_pred_ridge[:, j] = m.predict(X_eval)
dicc_preds['Regresión Individual'] = y_pred_ridge

# 4. RF multisalida
print(" [4/8] Recalculando RandomForest Multisalida...")
dicc_preds['RandomForest Multisalida'] = predecir_ensamble_pkl('rf_multi', 'rf_multi_m', '', 5, X_eval)

# 5. XGBoost multisalida
print(" [5/8] Recalculando XGBoost Multisalida...")
dicc_preds['XGBoost Multisalida'] = predecir_ensamble_pkl('xgb_multi', 'xgb_multi_exp', '', 5, X_eval)

# 6. RegressorChain
print(" [6/8] Recalculando Random Forest con RegressorChain...")
N_CHAINS = 10
preds_chain = []
for chain_id in range(N_CHAINS):
    cadena = joblib.load(os.path.join(MODELS_BASE, 'chain', f'chain_{chain_id}.pkl'))
    preds_chain.append(cadena.predict(X_eval))
dicc_preds['Random Forest con RegressorChain'] = np.mean(preds_chain, axis=0)

# 7. MLP estandar
print(" [7/8] Recalculando MLP Estandar...")
dicc_preds['MLP Estandar'] = predecir_ensamble_mlp('mlp', 'mlp_prod_', 5, X_eval, 'mlp_config.pkl')

# 8. MLP con perdida personalizada
print(" [8/8] Recalculando MLP Perdida Custom...")
dicc_preds['MLP Perdida Custom'] = predecir_ensamble_mlp('mlp_custom', 'mlp_custom_', 5, X_eval, 'mlp_custom_config.pkl')

print("¡Predicciones obtenidas con éxito!")

# Paso 3: Calcular métricas combinadas (Regresión R2 + Clasificación)
filas_radar = []

for nombre_modelo, y_pred in dicc_preds.items():
    # Extraer el R2 global calculado previamente en la Celda 4 de resultados
    r2_global = 0.0
    for res in resultados:
        if res['modelo'] == nombre_modelo:
            r2_global = res['R2']
            break

    # Calcular métricas de clasificación blanda por target discretizando en Bajo/Medio/Alto
    precisions, recalls, f1s = [], [], []
    for j, target in enumerate(TARGETS):
        valores_ref = df_combinado[target].dropna().values
        p33 = np.percentile(valores_ref, 33)
        p66 = np.percentile(valores_ref, 66)

        t_disc = np.zeros_like(y_true[:, j], dtype=int)
        t_disc[y_true[:, j] > p33] = 1
        t_disc[y_true[:, j] > p66] = 2

        p_disc = np.zeros_like(y_pred[:, j], dtype=int)
        p_disc[y_pred[:, j] > p33] = 1
        p_disc[y_pred[:, j] > p66] = 2

        precisions.append(precision_score(t_disc, p_disc, average='macro', zero_division=0))
        recalls.append(recall_score(t_disc, p_disc, average='macro', zero_division=0))
        f1s.append(f1_score(t_disc, p_disc, average='macro', zero_division=0))

    precision_m = np.mean(precisions)
    recall_m = np.mean(recalls)
    f1_m = np.mean(f1s)

    # Puntuación compuesta balanceada (R2 acotado a 0 si es muy bajo/negativo para visualización uniforme)
    r2_clipped = max(0, r2_global)
    score_compuesto = (r2_clipped + precision_m + recall_m + f1_m) / 4

    filas_radar.append({
        'Modelo': nombre_modelo,
        'R2': r2_global,
        'Precision': precision_m,
        'Recall': recall_m,
        'F1_Macro': f1_m,
        'Score_Compuesto': score_compuesto
    })

df_radar_orig = pd.DataFrame(filas_radar).sort_values(by='Score_Compuesto', ascending=False)

```

```
# Paso 4: Generación del Mosaico de Gráficos de Radar
categorias = ['R2', 'Precisión', 'Recall', 'F1-Macro']
N = len(categorias)
angulos = [n / float(N) * 2 * np.pi for n in range(N)]
angulos += angulos[:1]

fig, axes = plt.subplots(2, 4, figsize=(22, 11), subplot_kw=dict(projection='polar'))
axes = axes.flatten()

os.makedirs('output', exist_ok=True)

for idx, (index, fila) in enumerate(df_radar_orig.iterrows()):
    ax = axes[idx]

    # Valores mapeados (R2 acotado a 0 para mantener escala proporcional positiva de 0 a 1)
    valores = [max(0, fila['R2']), fila['Precisión'], fila['Recall'], fila['F1-Macro']]
    valores += valores[:1]

    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)

    plt.sca(ax)
    plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=10)
    ax.set_rlabel_position(0)
    plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=8)
    plt.ylim(0, 1)

    # Dibujar polígono en el mosaico
    ax.plot(angulos, valores, linewidth=2, linestyle='solid')
    ax.fill(angulos, valores, color=PALETTE['blue'], alpha=0.15)
    ax.set_title(f"{fila['Modelo']}\n(Score: {fila['Score-Compuesto']:.3f})", fontsize=11, fontweight='bold', pad=12)

    # Guardar gráfico radial individual
    fig_ind, ax_ind = plt.subplots(figsize=(6, 6), subplot_kw=dict(projection='polar'))
    ax_ind.set_theta_offset(np.pi / 2)
    ax_ind.set_theta_direction(-1)
    ax_ind.plot(angulos, valores, linewidth=2, linestyle='solid', color=PALETTE['green'])
    ax_ind.fill(angulos, valores, color=PALETTE['green'], alpha=0.15)
    ax_ind.set_title(f"Análisis Radial: {fila['Modelo']}\nScore Compuesto: {fila['Score-Compuesto']:.4f}", fontsize=12, fontweight='bold', pad=12)
    ax_ind.set_xticklabels(categorias)
    ax_ind.set_ylim(0, 1)

    nombre_archivo = f"output/regresion/graphs/radar-regresion_{fila['Modelo'].lower().replace(' ', '_')}.png"
    fig_ind.savefig(nombre_archivo, dpi=300, bbox_inches='tight')
    plt.close(fig_ind)

    # Formatear mosaico general y guardar
    fig.suptitle("Análisis Comparativo Radial Individualizado (Métricas Continuas R2 + Clasificación Blanda)", fontsize=16, fontweight='bold', pad=12)
    fig.tight_layout()
    fig.savefig('output/regresion/graphs/mosaico-radar-regresion.png', dpi=300, bbox_inches='tight')
    plt.show()

# Paso 5: Imprimir el Ranking Top 8 Final

print(" Top 8 modelos (Métricas de Regresión Original + Clasificación Blanda) ")

for i, (idx, fila) in enumerate(df_radar_orig.iterrows(), 1):
    print(f"Top {i}: {fila['Modelo']}<35 | Compuesto: {fila['Score-Compuesto']:.4f} | R2: {fila['R2']>7.4f} | P: {fila['Precisión']>0.8f}")
```

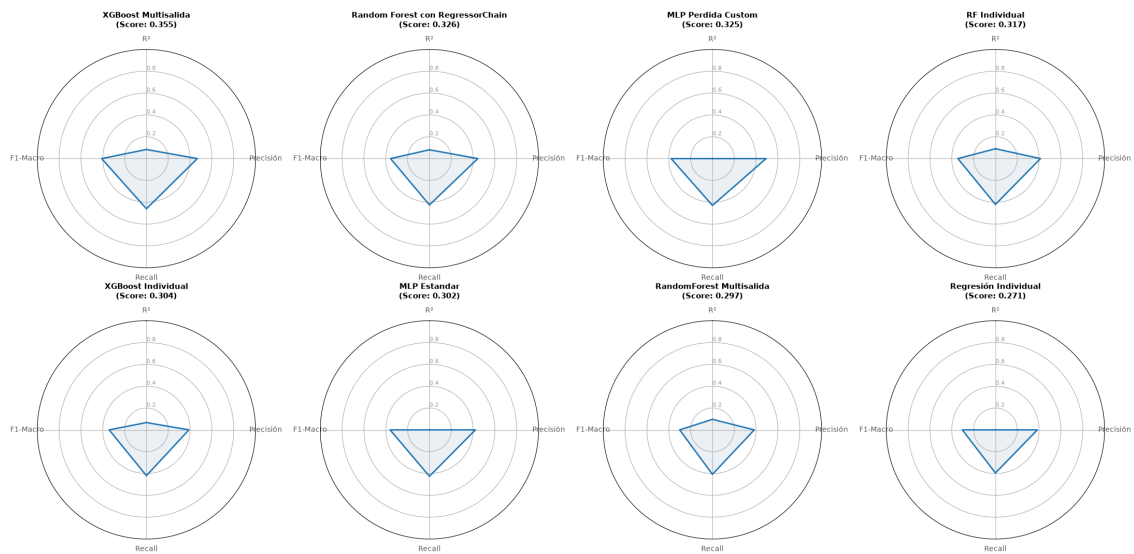
Iniciando Análisis Avanzado con Métricas de Clasificación Blanda...
[INFO] Cargado train.csv con éxito. Umbrales calculados sobre train + eval.

Generando predicciones para los 8 modelos...

```
[1/8] Recalculando RF Individual...
[2/8] Recalculando XGBoost Individual...
[3/8] Recalculando Regresión Individual (Ridge)...
[4/8] Recalculando RandomForest Multisalida...
[5/8] Recalculando XGBoost Multisalida...
[6/8] Recalculando Random Forest con RegressorChain...
[7/8] Recalculando MLP Estándar...
[8/8] Recalculando MLP Pérdida Custom...
```

¡Predicciones obtenidas con éxito!

Análisis Comparativo Radial Individualizado (Métricas Continuas R^2 + Clasificación Blanda)



Top 8 modelos (Métricas de Regresión Original + Clasificación Blanda)

Top 1: XGBoost Multisalida	Compuesto: 0.3549	R2: 0.0835	P: 0.4664	R: 0.4598	F1: 0.4101
Top 2: Random Forest con RegressorChain	Compuesto: 0.3263	R2: 0.0809	P: 0.4433	R: 0.4246	F1: 0.3564
Top 3: MLP Perdida Custom	Compuesto: 0.3248	R2: -0.1316	P: 0.4911	R: 0.4292	F1: 0.3789
Top 4: RF Individual	Compuesto: 0.3169	R2: 0.0904	P: 0.4125	R: 0.4203	F1: 0.3443
Top 5: XGBoost Individual	Compuesto: 0.3044	R2: 0.0670	P: 0.3894	R: 0.4198	F1: 0.3413
Top 6: MLP Estandar	Compuesto: 0.3017	R2: -0.0380	P: 0.4213	R: 0.4253	F1: 0.3603
Top 7: RandomForest Multisalida	Compuesto: 0.2968	R2: 0.0964	P: 0.3834	R: 0.4073	F1: 0.3002
Top 8: Regresión Individual	Compuesto: 0.2706	R2: -0.0164	P: 0.3839	R: 0.3952	F1: 0.3034

```
In [9]: # Paso 6: Metricas de clasificacion blanda (R2, Precision, Recall, F1) POR TARGET para cada modelo
print("Calculando metricas por target para el analisis por target...")
```

```
# Mapeo fijo modelo -> color, para que cada modelo tenga siempre el mismo color
# en todos los radares por target (facilita comparar visualmente entre targets)
modelos_lista = list(dicc_preds.keys())
color_por_modelo = {m: c for m, c in zip(modelos_lista, palette_cycle(len(modelos_lista)))}

filas_radar_target = []
for nombre_modelo, y_pred in dicc_preds.items():
    r2_por_target = r2_score(y_true, y_pred, multioutput='raw_values')

    for j, target in enumerate(TARGETS):
        valores_ref = df_combinado[target].dropna().values
        p33 = np.percentile(valores_ref, 33)
        p66 = np.percentile(valores_ref, 66)

        t_disc = np.zeros_like(y_true[:, j], dtype=int)
        t_disc[y_true[:, j] > p33] = 1
        t_disc[y_true[:, j] > p66] = 2

        p_disc = np.zeros_like(y_pred[:, j], dtype=int)
        p_disc[y_pred[:, j] > p33] = 1
        p_disc[y_pred[:, j] > p66] = 2

        precision_t = precision_score(t_disc, p_disc, average='macro', zero_division=0)
        recall_t = recall_score(t_disc, p_disc, average='macro', zero_division=0)
        f1_t = f1_score(t_disc, p_disc, average='macro', zero_division=0)
        r2_t = float(r2_por_target[j])
        score_t = (max(0, r2_t) + precision_t + recall_t + f1_t) / 4

        filas_radar_target.append({
            'Target': target,
            'Modelo': nombre_modelo,
            'R2': r2_t,
            'Precision': precision_t,
            'Recall': recall_t,
            'F1_Macro': f1_t,
            'Score_Compuesto': score_t
        })

df_radar_target = pd.DataFrame(filas_radar_target)
print(f"Metricas por target calculadas para {df_radar_target['Modelo'].nunique()} modelos "
      f"x {df_radar_target['Target'].nunique()} targets...")
```

Calculando metricas por target para el analisis por target...

Metricas por target calculadas para 8 modelos x 21 targets...

```
In [10]: # Paso 7: Radar por target - Top 8 modelos (ranking completo de los 8 modelos, por target)
os.makedirs('output/regresion/graphs/por_target', exist_ok=True)

n_targets = len(TARGETS)
n_cols_t = 3
n_rows_t = int(np.ceil(n_targets / n_cols_t))
```

```

fig_t, axes_t = plt.subplots(n_rows_t, n_cols_t, figsize=(6 * n_cols_t, 5.5 * n_rows_t),
                             subplot_kw=dict(projection='polar'))
axes_t = axes_t.flatten()

print(" Top 8 modelos por target (R2 + Clasificación) ")

for k, target in enumerate(TARGETS):
    ax = axes_t[k]
    df_t = (df_radar_target[df_radar_target['Target'] == target]
            .sort_values('Score_Compuesto', ascending=False)
            .reset_index(drop=True))

    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)
    plt.sca(ax)
    plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=8)
    ax.set_rlabel_position(0)
    plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=6)
    plt.ylim(0, 1)

    for i, fila in df_t.iterrows():
        valores_t = [max(0, fila['R2']), fila['Precision'], fila['Recall'], fila['F1_Macro']]
        valores_t += valores_t[1:]
        color_m = color_por_modelo[fila['Modelo']]
        ax.plot(angulos, valores_t, linewidth=1.4, linestyle='solid',
                color=color_m, label=fila['Modelo'] if k == 0 else None)
        ax.fill(angulos, valores_t, color=color_m, alpha=0.08)

    ax.set_title(f"{target}\n(Top 1: {df_t.iloc[0]['Modelo']})", fontsize=9, fontweight='bold', pad=10)

# Diagrama de araña individual por target
fig_ind_t, ax_ind_t = plt.subplots(figsize=(6, 6), subplot_kw=dict(projection='polar'))
ax_ind_t.set_theta_offset(np.pi / 2)
ax_ind_t.set_theta_direction(-1)
plt.sca(ax_ind_t)
plt.xticks(angulos[:-1], categorias, color=PALETTE['gray_dark'], size=10)
plt.yticks([0.2, 0.4, 0.6, 0.8], ["0.2", "0.4", "0.6", "0.8"], color=PALETTE['gray'], size=8)
plt.ylim(0, 1)
for i, fila in df_t.iterrows():
    color_m = color_por_modelo[fila['Modelo']]
    valores_t = [max(0, fila['R2']), fila['Precision'], fila['Recall'], fila['F1_Macro']]
    valores_t += valores_t[1:]
    ax_ind_t.plot(angulos, valores_t, linewidth=2, linestyle='solid', color=color_m, label=fila['Modelo'])
    ax_ind_t.fill(angulos, valores_t, color=color_m, alpha=0.08)
ax_ind_t.set_title(f"Análisis Radial por Target: {target}", fontsize=12, fontweight='bold', pad=15)
ax_ind_t.legend(loc='upper right', bbox_to_anchor=(1.4, 1.1), fontsize=7)
fig_ind_t.tight_layout()
fig_ind_t.savefig(f"output/regresion/graphs/por_target/radar_{target}.png", dpi=200, bbox_inches='tight')
plt.close(fig_ind_t)

# Ranking Top 8 impreso para este target
print(f"\n{target}:")
for i, fila in df_t.iterrows():
    print(f" Top {i+1}: {fila['Modelo'][:35]} | Compuesto: {fila['Score_Compuesto']:.4f} | "
          f"R2: {fila['R2']>7.4f} | P: {fila['Precision']:.4f} | R: {fila['Recall']:.4f} | "
          f"F1: {fila['F1_Macro']:.4f}")

for j in range(n_targets, len(axes_t)):
    axes_t[j].set_visible(False)

handles, labels = axes_t[0].get_legend_handles_labels()
fig_t.legend(handles, labels, loc='lower center', ncol=4, bbox_to_anchor=(0.5, -0.01), fontsize=10)
fig_t.suptitle("Análisis Radial por Target – Ranking de los 8 modelos (R² + Clasificación Blanda)",
               fontsize=16, fontweight='bold', y=1.01)
fig_t.tight_layout()
fig_t.savefig('output/regresion/graphs/mosaico_radar_por_target.png', dpi=250, bbox_inches='tight')
plt.show()

Top 8 modelos por target (R2 + Clasificación)
nematode_shannon_z:
Top 1: XGBoost Multisalida | Compuesto: 0.4013 | R2: 0.2228 | P: 0.5036 | R: 0.4738 | F1: 0.4049
Top 2: MLP Perdida Custom | Compuesto: 0.3130 | R2: 0.0903 | P: 0.4591 | R: 0.3883 | F1: 0.3143
Top 3: XGBoost Individual | Compuesto: 0.2938 | R2: 0.1637 | P: 0.2889 | R: 0.4065 | F1: 0.3160
Top 4: MLP Estandar | Compuesto: 0.2788 | R2: 0.1180 | P: 0.3053 | R: 0.3899 | F1: 0.3022
Top 5: RandomForest Multisalida | Compuesto: 0.2742 | R2: 0.1545 | P: 0.2791 | R: 0.3855 | F1: 0.2776
Top 6: Regresión Individual | Compuesto: 0.2714 | R2: 0.0112 | P: 0.4122 | R: 0.3615 | F1: 0.3008
Top 7: RF Individual | Compuesto: 0.2694 | R2: 0.1693 | P: 0.2564 | R: 0.3732 | F1: 0.2788
Top 8: Random Forest con RegressorChain | Compuesto: 0.2638 | R2: 0.1467 | P: 0.2564 | R: 0.3732 | F1: 0.2788
macro_shannon_z:
Top 1: XGBoost Multisalida | Compuesto: 0.5151 | R2: 0.3771 | P: 0.6217 | R: 0.5334 | F1: 0.5282
Top 2: RF Individual | Compuesto: 0.4743 | R2: 0.3199 | P: 0.6400 | R: 0.4832 | F1: 0.4541
Top 3: MLP Estandar | Compuesto: 0.4669 | R2: 0.2571 | P: 0.6148 | R: 0.5087 | F1: 0.4870
Top 4: Random Forest con RegressorChain | Compuesto: 0.4644 | R2: 0.2804 | P: 0.6679 | R: 0.4698 | F1: 0.4392
Top 5: MLP Perdida Custom | Compuesto: 0.4521 | R2: 0.2676 | P: 0.5754 | R: 0.4848 | F1: 0.4806
Top 6: XGBoost Individual | Compuesto: 0.3967 | R2: 0.2601 | P: 0.5141 | R: 0.4295 | F1: 0.3834
Top 7: RandomForest Multisalida | Compuesto: 0.3482 | R2: 0.2000 | P: 0.4633 | R: 0.4133 | F1: 0.3160
Top 8: Regresión Individual | Compuesto: 0.2953 | R2: 0.1257 | P: 0.3532 | R: 0.3988 | F1: 0.3033

```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

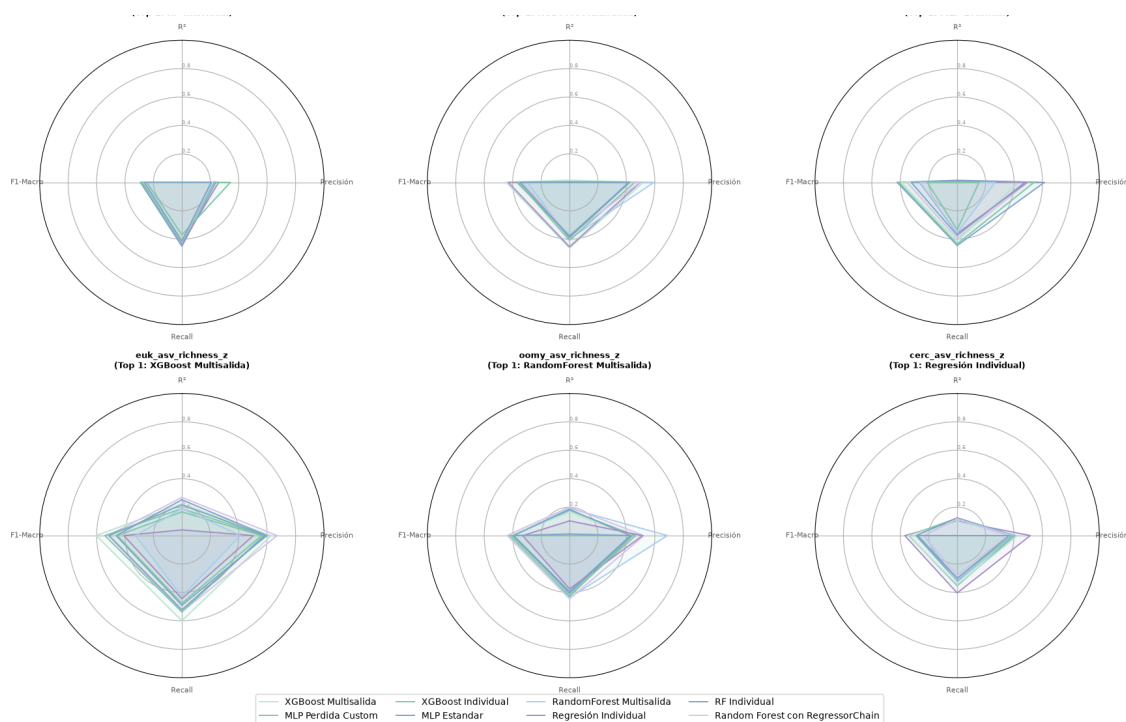
earthworm_shannon_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.7030	R2: 0.5375	P: 0.7703	R: 0.7578	F1: 0.7462	
Top 2: RF Individual	Compuesto: 0.6711	R2: 0.5041	P: 0.7425	R: 0.7275	F1: 0.7101	
Top 3: Random Forest con RegressorChain	Compuesto: 0.6678	R2: 0.4876	P: 0.7468	R: 0.7275	F1: 0.7093	
Top 4: XGBoost Individual	Compuesto: 0.6429	R2: 0.4946	P: 0.7051	R: 0.6925	F1: 0.6794	
Top 5: RandomForest Multisalida	Compuesto: 0.5822	R2: 0.4160	P: 0.6952	R: 0.6366	F1: 0.5808	
Top 6: MLP Estandar	Compuesto: 0.5802	R2: 0.4264	P: 0.6630	R: 0.6234	F1: 0.6080	
Top 7: MLP Perdida Custom	Compuesto: 0.5248	R2: 0.3750	P: 0.5887	R: 0.5768	F1: 0.5587	
Top 8: Regresión Individual	Compuesto: 0.3851	R2: 0.2395	P: 0.4279	R: 0.4720	F1: 0.4008	
orib_shannon_z:						
Top 1: MLP Perdida Custom	Compuesto: 0.4351	R2: 0.2216	P: 0.7284	R: 0.4717	F1: 0.3186	
Top 2: MLP Estandar	Compuesto: 0.3299	R2: 0.2356	P: 0.3918	R: 0.4359	F1: 0.2564	
Top 3: XGBoost Multisalida	Compuesto: 0.3094	R2: 0.2275	P: 0.3141	R: 0.4282	F1: 0.2677	
Top 4: RandomForest Multisalida	Compuesto: 0.2896	R2: 0.1693	P: 0.3889	R: 0.3974	F1: 0.2028	
Top 5: RF Individual	Compuesto: 0.2658	R2: 0.1794	P: 0.3286	R: 0.3641	F1: 0.1911	
Top 6: Random Forest con RegressorChain	Compuesto: 0.2413	R2: 0.2069	P: 0.2385	R: 0.3231	F1: 0.1969	
Top 7: XGBoost Individual	Compuesto: 0.2379	R2: 0.1147	P: 0.3046	R: 0.3718	F1: 0.1606	
Top 8: Regresión Individual	Compuesto: 0.2256	R2: 0.1137	P: 0.2992	R: 0.3385	F1: 0.1512	
meso_shannon_z:						
Top 1: MLP Perdida Custom	Compuesto: 0.1924	R2: -0.6189	P: 0.3597	R: 0.2296	F1: 0.1803	
Top 2: RandomForest Multisalida	Compuesto: 0.1247	R2: -0.1178	P: 0.0936	R: 0.2667	F1: 0.1385	
Top 3: XGBoost Individual	Compuesto: 0.1213	R2: -0.1868	P: 0.0962	R: 0.2500	F1: 0.1389	
Top 4: RF Individual	Compuesto: 0.1213	R2: -0.1731	P: 0.0962	R: 0.2500	F1: 0.1389	
Top 5: Regresión Individual	Compuesto: 0.1203	R2: -0.2499	P: 0.0943	R: 0.2500	F1: 0.1370	
Top 6: XGBoost Multisalida	Compuesto: 0.1202	R2: -0.2861	P: 0.1037	R: 0.2333	F1: 0.1436	
Top 7: Random Forest con RegressorChain	Compuesto: 0.1149	R2: -0.1640	P: 0.1143	R: 0.2000	F1: 0.1455	
Top 8: MLP Estandar	Compuesto: 0.1030	R2: -0.4898	P: 0.0889	R: 0.2000	F1: 0.1231	
coll_shannon_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.3556	R2: -0.3229	P: 0.6290	R: 0.5306	F1: 0.2628	
Top 2: MLP Perdida Custom	Compuesto: 0.3388	R2: -1.3439	P: 0.6250	R: 0.5102	F1: 0.2200	
Top 3: RF Individual	Compuesto: 0.2052	R2: -0.3489	P: 0.1231	R: 0.5000	F1: 0.1975	
Top 4: XGBoost Individual	Compuesto: 0.2052	R2: -0.3312	P: 0.1231	R: 0.5000	F1: 0.1975	
Top 5: RandomForest Multisalida	Compuesto: 0.2052	R2: -0.1953	P: 0.1231	R: 0.5000	F1: 0.1975	
Top 6: Regresión Individual	Compuesto: 0.2052	R2: -0.4998	P: 0.1231	R: 0.5000	F1: 0.1975	
Top 7: Random Forest con RegressorChain	Compuesto: 0.2052	R2: -0.6087	P: 0.1231	R: 0.5000	F1: 0.1975	
Top 8: MLP Estandar	Compuesto: 0.2052	R2: -0.8227	P: 0.1231	R: 0.5000	F1: 0.1975	
bac_shannon_z:						
Top 1: Random Forest con RegressorChain	Compuesto: 0.3583	R2: 0.0288	P: 0.5940	R: 0.4522	F1: 0.3582	
Top 2: XGBoost Individual	Compuesto: 0.3523	R2: 0.0092	P: 0.5667	R: 0.4450	F1: 0.3885	
Top 3: XGBoost Multisalida	Compuesto: 0.3378	R2: -0.0151	P: 0.4828	R: 0.4451	F1: 0.4232	
Top 4: Regresión Individual	Compuesto: 0.3155	R2: -0.0812	P: 0.4671	R: 0.4438	F1: 0.3511	
Top 5: MLP Estandar	Compuesto: 0.3089	R2: -0.0854	P: 0.4563	R: 0.4210	F1: 0.3584	
Top 6: MLP Perdida Custom	Compuesto: 0.3050	R2: -0.1004	P: 0.4511	R: 0.4126	F1: 0.3563	
Top 7: RandomForest Multisalida	Compuesto: 0.2682	R2: 0.0290	P: 0.2573	R: 0.4583	F1: 0.3284	
Top 8: RF Individual	Compuesto: 0.2426	R2: 0.0077	P: 0.2388	R: 0.4208	F1: 0.3030	
fun_shannon_z:						
Top 1: RF Individual	Compuesto: 0.2879	R2: -0.0455	P: 0.5322	R: 0.3881	F1: 0.2313	
Top 2: XGBoost Multisalida	Compuesto: 0.2779	R2: -0.0502	P: 0.4184	R: 0.3876	F1: 0.3056	
Top 3: Random Forest con RegressorChain	Compuesto: 0.2653	R2: -0.0219	P: 0.4819	R: 0.3615	F1: 0.2178	
Top 4: MLP Estandar	Compuesto: 0.2200	R2: -0.0757	P: 0.3145	R: 0.3373	F1: 0.2283	
Top 5: MLP Perdida Custom	Compuesto: 0.2190	R2: -0.0842	P: 0.3125	R: 0.3265	F1: 0.2371	
Top 6: RandomForest Multisalida	Compuesto: 0.1656	R2: -0.0127	P: 0.1864	R: 0.3270	F1: 0.1490	
Top 7: XGBoost Individual	Compuesto: 0.1345	R2: -0.0206	P: 0.0781	R: 0.3333	F1: 0.1266	
Top 8: Regresión Individual	Compuesto: 0.1338	R2: -0.0161	P: 0.0769	R: 0.3333	F1: 0.1250	
euk_shannon_z:						
Top 1: RandomForest Multisalida	Compuesto: 0.4391	R2: 0.1378	P: 0.7989	R: 0.4444	F1: 0.3755	
Top 2: RF Individual	Compuesto: 0.4391	R2: 0.1683	P: 0.6367	R: 0.4817	F1: 0.4695	
Top 3: Random Forest con RegressorChain	Compuesto: 0.4273	R2: 0.1624	P: 0.6293	R: 0.4658	F1: 0.4518	
Top 4: XGBoost Multisalida	Compuesto: 0.4042	R2: 0.0886	P: 0.5492	R: 0.4872	F1: 0.4917	
Top 5: MLP Estandar	Compuesto: 0.3940	R2: 0.1453	P: 0.5362	R: 0.4527	F1: 0.4419	
Top 6: XGBoost Individual	Compuesto: 0.3733	R2: 0.0912	P: 0.5303	R: 0.4382	F1: 0.4334	
Top 7: MLP Perdida Custom	Compuesto: 0.3713	R2: 0.0868	P: 0.4927	R: 0.4555	F1: 0.4501	
Top 8: Regresión Individual	Compuesto: 0.2908	R2: 0.0407	P: 0.4467	R: 0.3575	F1: 0.3182	
oomy_shannon_z:						
Top 1: MLP Estandar	Compuesto: 0.4294	R2: 0.1534	P: 0.6846	R: 0.4651	F1: 0.4147	
Top 2: XGBoost Multisalida	Compuesto: 0.4172	R2: 0.0620	P: 0.5660	R: 0.5250	F1: 0.5156	
Top 3: MLP Perdida Custom	Compuesto: 0.3970	R2: 0.0588	P: 0.5464	R: 0.4917	F1: 0.4910	
Top 4: XGBoost Individual	Compuesto: 0.3853	R2: 0.0965	P: 0.5317	R: 0.4655	F1: 0.4477	
Top 5: Regresión Individual	Compuesto: 0.3625	R2: 0.0857	P: 0.6149	R: 0.3948	F1: 0.3544	
Top 6: RandomForest Multisalida	Compuesto: 0.3615	R2: 0.1252	P: 0.5174	R: 0.4409	F1: 0.3626	
Top 7: Random Forest con RegressorChain	Compuesto: 0.3571	R2: 0.0913	P: 0.5182	R: 0.4218	F1: 0.3969	
Top 8: RF Individual	Compuesto: 0.3276	R2: 0.0832	P: 0.4772	R: 0.3893	F1: 0.3608	
cerc_shannon_z:						
Top 1: XGBoost Individual	Compuesto: 0.2852	R2: -0.0414	P: 0.4397	R: 0.3633	F1: 0.3378	
Top 2: Random Forest con RegressorChain	Compuesto: 0.2820	R2: -0.0006	P: 0.4247	R: 0.3698	F1: 0.3333	
Top 3: XGBoost Multisalida	Compuesto: 0.2723	R2: -0.0330	P: 0.3886	R: 0.3518	F1: 0.3489	
Top 4: MLP Estandar	Compuesto: 0.2666	R2: -0.1361	P: 0.3722	R: 0.3551	F1: 0.3389	
Top 5: RandomForest Multisalida	Compuesto: 0.2534	R2: -0.0041	P: 0.4107	R: 0.3330	F1: 0.2697	
Top 6: RF Individual	Compuesto: 0.2473	R2: -0.0318	P: 0.3718	R: 0.3296	F1: 0.2876	
Top 7: MLP Perdida Custom	Compuesto: 0.2395	R2: -0.1194	P: 0.3325	R: 0.3181	F1: 0.3075	
Top 8: Regresión Individual	Compuesto: 0.2282	R2: 0.0155	P: 0.3003	R: 0.3395	F1: 0.2576	
macro_order_richness_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.5626	R2: 0.4013	P: 0.6825	R: 0.5942	F1: 0.5722	
Top 2: Random Forest con RegressorChain	Compuesto: 0.5325	R2: 0.4069	P: 0.6775	R: 0.5372	F1: 0.5085	
Top 3: RF Individual	Compuesto: 0.5060	R2: 0.3417	P: 0.6667	R: 0.5271	F1: 0.4886	
Top 4: MLP Perdida Custom	Compuesto: 0.4938	R2: 0.3759	P: 0.6265	R: 0.4915	F1: 0.4813	
Top 5: MLP Estandar	Compuesto: 0.4667	R2: 0.3525	P: 0.5752	R: 0.4843	F1: 0.4548	
Top 6: XGBoost Individual	Compuesto: 0.4342	R2: 0.3082	P: 0.6272	R: 0.4304	F1: 0.3711	
Top 7: Regresión Individual	Compuesto: 0.3320	R2: 0.1521	P: 0.5701	R: 0.3484	F1: 0.2575	
Top 8: RandomForest Multisalida	Compuesto: 0.3203	R2: 0.2532	P: 0.3077	R: 0.4021	F1: 0.3182	

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

earthworm_richness_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.6163	R2: 0.5893	P: 0.6592	R: 0.6126	F1: 0.6043	
Top 2: XGBoost Individual	Compuesto: 0.5994	R2: 0.5171	P: 0.6515	R: 0.6168	F1: 0.6121	
Top 3: MLP Estandar	Compuesto: 0.5786	R2: 0.5176	P: 0.6124	R: 0.5967	F1: 0.5879	
Top 4: Random Forest con RegressorChain	Compuesto: 0.5753	R2: 0.5359	P: 0.6077	R: 0.5808	F1: 0.5767	
Top 5: MLP Perdida Custom	Compuesto: 0.5693	R2: 0.4671	P: 0.6301	R: 0.5942	F1: 0.5859	
Top 6: RF Individual	Compuesto: 0.5492	R2: 0.5722	P: 0.5611	R: 0.5441	F1: 0.5194	
Top 7: RandomForest Multisalida	Compuesto: 0.5150	R2: 0.4509	P: 0.5516	R: 0.5349	F1: 0.5226	
Top 8: Regresión Individual	Compuesto: 0.4861	R2: 0.2789	P: 0.5683	R: 0.5492	F1: 0.5479	
orib_species_richness_z:						
Top 1: MLP Perdida Custom	Compuesto: 0.4427	R2: 0.2542	P: 0.6449	R: 0.4451	F1: 0.4265	
Top 2: XGBoost Multisalida	Compuesto: 0.3275	R2: 0.2800	P: 0.3027	R: 0.3951	F1: 0.3323	
Top 3: Random Forest con RegressorChain	Compuesto: 0.3202	R2: 0.2026	P: 0.3154	R: 0.4136	F1: 0.3492	
Top 4: MLP Estandar	Compuesto: 0.3194	R2: 0.2596	P: 0.3039	R: 0.3889	F1: 0.3253	
Top 5: RF Individual	Compuesto: 0.3084	R2: 0.2034	P: 0.3027	R: 0.3951	F1: 0.3323	
Top 6: RandomForest Multisalida	Compuesto: 0.2954	R2: 0.2386	P: 0.2751	R: 0.3642	F1: 0.3039	
Top 7: Regresión Individual	Compuesto: 0.2875	R2: 0.0921	P: 0.3205	R: 0.4012	F1: 0.3362	
Top 8: XGBoost Individual	Compuesto: 0.2765	R2: 0.1259	P: 0.2889	R: 0.3765	F1: 0.3146	
meso_species_richness_z:						
Top 1: MLP Perdida Custom	Compuesto: 0.2193	R2: -0.3462	P: 0.2260	R: 0.3714	F1: 0.2799	
Top 2: Regresión Individual	Compuesto: 0.1995	R2: -0.0959	P: 0.2061	R: 0.3405	F1: 0.2514	
Top 3: MLP Estandar	Compuesto: 0.1778	R2: -0.2753	P: 0.1813	R: 0.3048	F1: 0.2252	
Top 4: XGBoost Individual	Compuesto: 0.1619	R2: -0.0592	P: 0.1603	R: 0.2833	F1: 0.2040	
Top 5: Random Forest con RegressorChain	Compuesto: 0.1552	R2: -0.0400	P: 0.1538	R: 0.2714	F1: 0.1957	
Top 6: RandomForest Multisalida	Compuesto: 0.1513	R2: -0.0255	P: 0.1356	R: 0.2857	F1: 0.1839	
Top 7: XGBoost Multisalida	Compuesto: 0.1486	R2: -0.1146	P: 0.1474	R: 0.2595	F1: 0.1873	
Top 8: RF Individual	Compuesto: 0.1300	R2: -0.0438	P: 0.1212	R: 0.2381	F1: 0.1606	
coll_species_richness_z:						
Top 1: RF Individual	Compuesto: 0.2451	R2: -0.5484	P: 0.2387	R: 0.4500	F1: 0.2917	
Top 2: Regresión Individual	Compuesto: 0.2430	R2: -0.7341	P: 0.2584	R: 0.4333	F1: 0.2804	
Top 3: MLP Perdida Custom	Compuesto: 0.2395	R2: -2.2411	P: 0.3419	R: 0.3698	F1: 0.2464	
Top 4: XGBoost Individual	Compuesto: 0.2343	R2: -0.5803	P: 0.2271	R: 0.4250	F1: 0.2850	
Top 5: Random Forest con RegressorChain	Compuesto: 0.2228	R2: -0.5799	P: 0.2178	R: 0.4167	F1: 0.2569	
Top 6: XGBoost Multisalida	Compuesto: 0.2226	R2: -0.6465	P: 0.2126	R: 0.4083	F1: 0.2694	
Top 7: RandomForest Multisalida	Compuesto: 0.2216	R2: -0.2920	P: 0.2130	R: 0.4167	F1: 0.2567	
Top 8: MLP Estandar	Compuesto: 0.2210	R2: -1.4628	P: 0.2060	R: 0.4125	F1: 0.2654	
bac_asv_richness_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.3611	R2: 0.0155	P: 0.5193	R: 0.4626	F1: 0.4469	
Top 2: Regresión Individual	Compuesto: 0.3457	R2: -0.0761	P: 0.4982	R: 0.4538	F1: 0.4310	
Top 3: RandomForest Multisalida	Compuesto: 0.3233	R2: 0.0044	P: 0.5943	R: 0.3982	F1: 0.2964	
Top 4: XGBoost Individual	Compuesto: 0.3073	R2: 0.0014	P: 0.4591	R: 0.4081	F1: 0.3605	
Top 5: Random Forest con RegressorChain	Compuesto: 0.3004	R2: -0.0052	P: 0.4958	R: 0.3762	F1: 0.3297	
Top 6: MLP Perdida Custom	Compuesto: 0.2927	R2: -0.1680	P: 0.4241	R: 0.3756	F1: 0.3713	
Top 7: RF Individual	Compuesto: 0.2907	R2: 0.0042	P: 0.4227	R: 0.3885	F1: 0.3474	
Top 8: MLP Estandar	Compuesto: 0.2863	R2: -0.1464	P: 0.4172	R: 0.3815	F1: 0.3464	
fun_asv_richness_z:						
Top 1: MLP Estandar	Compuesto: 0.3739	R2: 0.0145	P: 0.6141	R: 0.4444	F1: 0.4224	
Top 2: MLP Perdida Custom	Compuesto: 0.3468	R2: 0.0027	P: 0.5412	R: 0.4333	F1: 0.4099	
Top 3: Random Forest con RegressorChain	Compuesto: 0.3111	R2: -0.0186	P: 0.4969	R: 0.3956	F1: 0.3519	
Top 4: XGBoost Multisalida	Compuesto: 0.3095	R2: -0.0662	P: 0.4295	R: 0.4133	F1: 0.3953	
Top 5: RF Individual	Compuesto: 0.2934	R2: -0.0256	P: 0.4774	R: 0.3733	F1: 0.3227	
Top 6: Regresión Individual	Compuesto: 0.2813	R2: 0.0087	P: 0.4921	R: 0.3600	F1: 0.2644	
Top 7: RandomForest Multisalida	Compuesto: 0.2210	R2: -0.0068	P: 0.2670	R: 0.3556	F1: 0.2614	
Top 8: XGBoost Individual	Compuesto: 0.1744	R2: -0.0242	P: 0.1538	R: 0.3333	F1: 0.2105	
euk_asv_richness_z:						
Top 1: XGBoost Multisalida	Compuesto: 0.5132	R2: 0.2233	P: 0.6272	R: 0.5996	F1: 0.6028	
Top 2: Random Forest con RegressorChain	Compuesto: 0.4984	R2: 0.2690	P: 0.6709	R: 0.5354	F1: 0.5183	
Top 3: MLP Perdida Custom	Compuesto: 0.4613	R2: 0.1902	P: 0.5755	R: 0.5375	F1: 0.5420	
Top 4: MLP Estandar	Compuesto: 0.4594	R2: 0.2164	P: 0.5870	R: 0.5224	F1: 0.5118	
Top 5: RF Individual	Compuesto: 0.4509	R2: 0.2520	P: 0.5980	R: 0.4906	F1: 0.4628	
Top 6: XGBoost Individual	Compuesto: 0.4145	R2: 0.1671	P: 0.5605	R: 0.4762	F1: 0.4542	
Top 7: Regresión Individual	Compuesto: 0.3490	R2: 0.0400	P: 0.5033	R: 0.4444	F1: 0.4083	
Top 8: RandomForest Multisalida	Compuesto: 0.3393	R2: 0.1903	P: 0.4122	R: 0.4286	F1: 0.3262	
oomy_asv_richness_z:						
Top 1: RandomForest Multisalida	Compuesto: 0.4292	R2: 0.1917	P: 0.6885	R: 0.4368	F1: 0.3997	
Top 2: Random Forest con RegressorChain	Compuesto: 0.4021	R2: 0.2019	P: 0.5231	R: 0.4451	F1: 0.4383	
Top 3: XGBoost Individual	Compuesto: 0.3662	R2: 0.1776	P: 0.4650	R: 0.4166	F1: 0.4057	
Top 4: XGBoost Multisalida	Compuesto: 0.3587	R2: 0.1650	P: 0.4403	R: 0.4108	F1: 0.4188	
Top 5: RF Individual	Compuesto: 0.3498	R2: 0.1843	P: 0.4286	R: 0.3930	F1: 0.3935	
Top 6: Regresión Individual	Compuesto: 0.3308	R2: 0.1035	P: 0.5152	R: 0.3746	F1: 0.3299	
Top 7: MLP Perdida Custom	Compuesto: 0.3234	R2: -0.0546	P: 0.4438	R: 0.4281	F1: 0.4216	
Top 8: MLP Estandar	Compuesto: 0.3115	R2: 0.0113	P: 0.4339	R: 0.4062	F1: 0.3944	
cerc_asv_richness_z:						
Top 1: Regresión Individual	Compuesto: 0.3472	R2: 0.1019	P: 0.5148	R: 0.4037	F1: 0.3685	
Top 2: XGBoost Individual	Compuesto: 0.3053	R2: 0.1227	P: 0.4050	R: 0.3535	F1: 0.3400	
Top 3: XGBoost Multisalida	Compuesto: 0.3032	R2: 0.0983	P: 0.4254	R: 0.3450	F1: 0.3441	
Top 4: RF Individual	Compuesto: 0.2838	R2: 0.1250	P: 0.4021	R: 0.3194	F1: 0.2886	
Top 5: RandomForest Multisalida	Compuesto: 0.2684	R2: 0.1171	P: 0.3935	R: 0.3269	F1: 0.2362	
Top 6: Random Forest con RegressorChain	Compuesto: 0.2468	R2: 0.1171	P: 0.3560	R: 0.2795	F1: 0.2345	
Top 7: MLP Perdida Custom	Compuesto: 0.2415	R2: -0.0770	P: 0.3868	R: 0.3013	F1: 0.2777	
Top 8: MLP Estandar	Compuesto: 0.2359	R2: -0.0120	P: 0.3661	R: 0.3013	F1: 0.2763	

Análisis Radial por Target — Ranking de los 8 modelos (R^2 + Clasificación Blanda)





```
In [11]: # Paso 8: Resumen - Top 1 modelo por cada target
df_top1_target = (
    df_radar_target.sort_values('Score_Compuesto', ascending=False)
    .groupby('Target', sort=False).first()
    .reindex(TARGETS)
    .reset_index()[['Target', 'Modelo', 'Score_Compuesto', 'R2', 'Precision', 'Recall', 'F1_Macro']]
)

print(" Mejor modelo para cada target ")
print(df_top1_target.to_string(index=False))

conteo_top1 = df_top1_target['Modelo'].value_counts()
print("\nNumero de targets en los que cada modelo es el Top 1:")
for modelo, count in conteo_top1.items():
    print(f" {modelo}<35: {count} / {len(TARGETS)} targets")

# Grafico resumen: modelo Top 1 por target, coloreado por modelo (mismo color que en los radares)
fig, ax = plt.subplots(figsize=(10, 9))
colores_top1 = [color_por_modelo[m] for m in df_top1_target['Modelo']]
bars = ax.barh(df_top1_target['Target'], df_top1_target['Score_Compuesto'],
               color=colores_top1, edgecolor='white')
for bar, modelo in zip(bars, df_top1_target['Modelo']):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height() / 2, modelo, va='center', fontsize=8)
ax.set_xlabel('Score compuesto del modelo Top 1')
ax.set_title('Mejor modelo (Top 1) por target', fontsize=13, fontweight='bold')
ax.set_xlim(0, max(df_top1_target['Score_Compuesto']) * 1.3)
ax.invert_yaxis()

handles = [plt.Rectangle((0, 0), 1, 1, color=color_por_modelo[m]) for m in modelos_lista]
ax.legend(handles, modelos_lista, loc='lower right', fontsize=8, title='Modelo')

plt.tight_layout()
os.makedirs('output/regresion/graphs', exist_ok=True)
fig.savefig('output/regresion/graphs/resumen_top1_por_target.png', dpi=250, bbox_inches='tight')
plt.show()

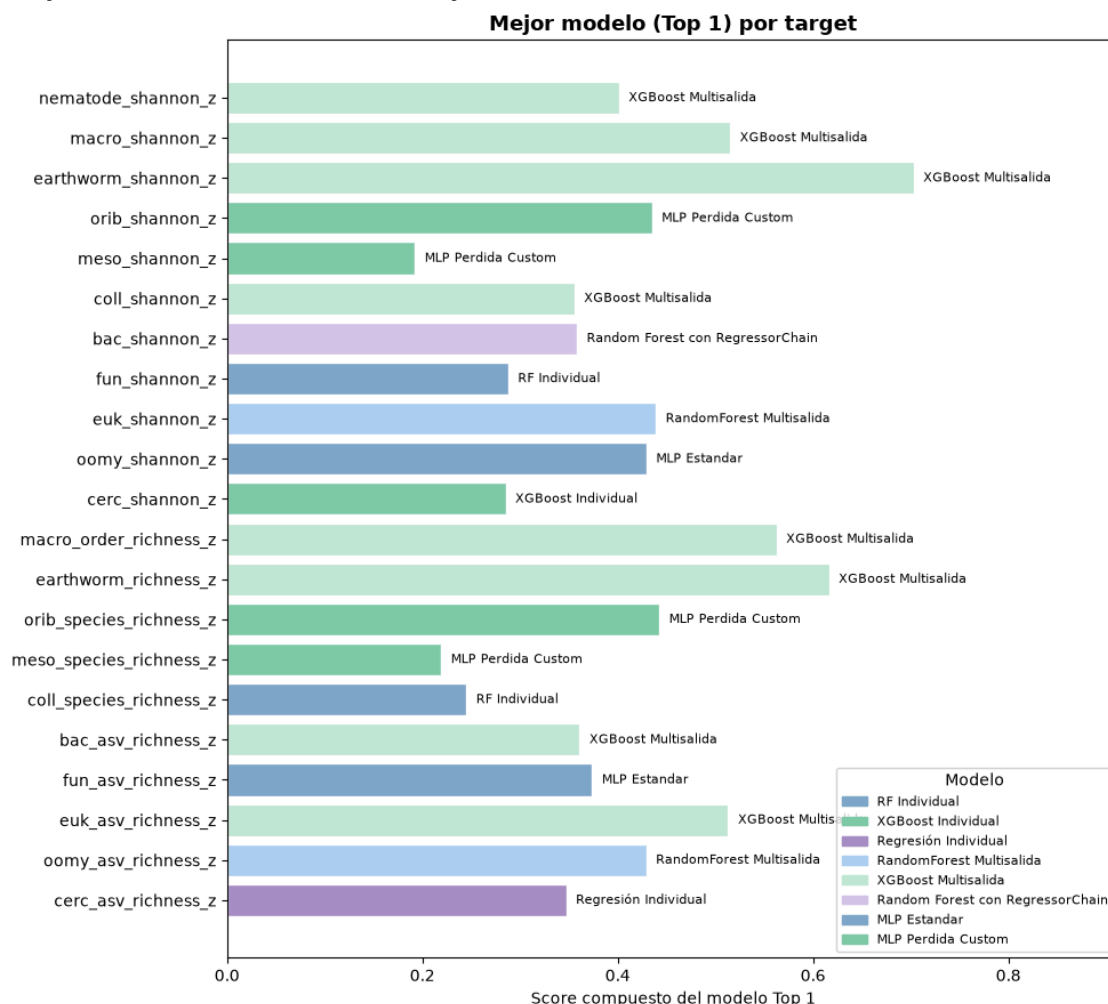
df_top1_target.to_csv('output/regresion/top1_modelo_por_target.csv', index=False)
print("\nResumen guardado en output/regresion/top1_modelo_por_target.csv")
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

Mejor modelo para cada target						
Target	Modelo	Score_Compuesto	R2	Precision	Recall	F1_Macro
nematode_shannon_z	XGBoost Multisalida	0.401273	0.222791	0.503589	0.473847	0.404867
macro_shannon_z	XGBoost Multisalida	0.515109	0.377138	0.621693	0.533402	0.528205
earthworm_shannon_z	XGBoost Multisalida	0.702971	0.537528	0.770330	0.757842	0.746183
orib_shannon_z	MLP Perdida Custom	0.435079	0.221609	0.728395	0.471706	0.318606
meso_shannon_z	MLP Perdida Custom	0.192409	-0.618857	0.359690	0.229630	0.180317
coll_shannon_z	XGBoost Multisalida	0.355616	-0.322850	0.629032	0.530612	0.262821
bac_shannon_z	Random Forest con RegressorChain	0.358293	0.028837	0.594017	0.452155	0.358162
fun_shannon_z	RF Individual	0.287878	-0.045471	0.532164	0.388068	0.231282
euk_shannon_z	RandomForest Multisalida	0.439141	0.137792	0.798851	0.444444	0.375477
oomy_shannon_z	MLP Estandar	0.429433	0.153375	0.684569	0.465079	0.414709
cerc_shannon_z	XGBoost Individual	0.285203	-0.041371	0.439683	0.363294	0.337835
macro_order_richness_z	XGBoost Multisalida	0.562561	0.401261	0.682540	0.594221	0.572222
earthworm_richness_z	XGBoost Multisalida	0.616329	0.589276	0.659201	0.612565	0.604277
orib_species_richness_z	MLP Perdida Custom	0.442673	0.254237	0.644928	0.445062	0.426468
meso_species_richness_z	MLP Perdida Custom	0.219318	-0.346185	0.225957	0.371429	0.279887
coll_species_richness_z	RF Individual	0.245113	-0.548401	0.238749	0.450000	0.291701
bac_asv_richness_z	XGBoost Multisalida	0.361079	0.015548	0.519312	0.462600	0.446856
fun_asv_richness_z	MLP Estandar	0.373855	0.014462	0.614141	0.444444	0.422373
euk_asv_richness_z	XGBoost Multisalida	0.513211	0.223251	0.627193	0.599567	0.602834
oomy_asv_richness_z	RandomForest Multisalida	0.429158	0.191662	0.688462	0.436768	0.399743
cerc_asv_richness_z	Regresión Individual	0.347224	0.101861	0.514815	0.403690	0.368529

Numero de targets en los que cada modelo es el Top 1:

- XGBoost Multisalida : 8 / 21 targets
- MLP Perdida Custom : 4 / 21 targets
- RF Individual : 2 / 21 targets
- RandomForest Multisalida : 2 / 21 targets
- MLP Estandar : 2 / 21 targets
- Random Forest con RegressorChain : 1 / 21 targets
- XGBoost Individual : 1 / 21 targets
- Regresión Individual : 1 / 21 targets



Resumen guardado en output/regresion/top1_modelo_por_target.csv

11.3.4.2. Comparador de clasificación

SOB4ES - Comparador de Modelos con Clasificación Ordinal

CRISP-ML(Q) - Fase 5: Evaluación y selección del modelo final

Este notebook extiende el comparador estándar (basado en R2, RMSE, MAE) añadiendo una **perspectiva de clasificación ordinal**: las predicciones y los valores reales se convierten en categorías discretas (Bajo / Medio / Alto) usando los percentiles 33 y 66 calculados sobre el conjunto combinado `train + eval`.

Esto permite:

- Generar **matrices de confusión** por target y por modelo, mostrando no solo si el modelo acierta sino qué tipo de error comete (confunde Bajo con Medio, o Bajo con Alto).
- Calcular métricas de clasificación adicionales que complementan el R2: **Accuracy**, **Cohen's Kappa** y **F1-macro**.

Datasets: `train.csv` (para calcular umbrales), `eval.csv` (para evaluar modelos).

Modelos comparados: los 8 enfoques implementados en el proyecto.

1.- Configuración

Carga del entorno, librerías y variables globales base.

```
In [1]: import os
import warnings
import joblib
import pandas as pd
import numpy as np
import itertools

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

import seaborn as sns

import torch
import torch.nn as nn

from sklearn.metrics import (
    r2_score, mean_squared_error, mean_absolute_error,
    confusion_matrix, accuracy_score, cohen_kappa_score,
    f1_score, classification_report
)

warnings.filterwarnings('ignore')

DATA_DIR = 'input/'
MODELS_BASE = 'output/models/'
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

CLASES = ['Bajo', 'Medio', 'Alto']

TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
```

```

'as_z',
'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

print('Configuracion cargada correctamente...')

# Paleta de colores unificada
PALETTE = {
    'blue': '#7FA6C9',
    'blue_light': '#AAD8F1',
    'green': '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple': '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray': '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Colormap secuencial (blanco → azul → morado) para heatmaps/matrices de confusion
PALETTE_CMAP = LinearSegmentedColormap.from_list(
    'sob4es_palette', ['#FFFFFF', PALETTE['purple_light'], PALETTE['purple']]
)

```

Configuracion cargada correctamente...

2.- Carga de datos

Se cargan `train.csv` y `eval.csv`. Los percentiles 33 y 66 para la clasificación ordinal se calculan sobre el **conjunto combinado train + eval**, garantizando que los umbrales reflejen la distribución global del dataset y no solo la de un subconjunto.

El modelo **nunca ha visto** `eval.csv` durante el entrenamiento ni el tuning, por lo que la evaluación sobre este conjunto es imparcial.

```

In [2]: df_train = pd.read_csv(os.path.join(DATA_DIR, 'train.csv'))
df_eval = pd.read_csv(os.path.join(DATA_DIR, 'eval.csv'))

X_eval = df_eval[FEATURES_AUTORIZADAS]
y_eval = df_eval[TARGETS]

# Conjunto combinado para calcular percentiles globales
df_combined = pd.concat([df_train[TARGETS], df_eval[TARGETS]], ignore_index=True)

print(f'train.csv : {len(df_train)} filas')
print(f'eval.csv : {len(df_eval)} filas')
print(f'Combinado : {len(df_combined)} filas (para calculo de percentiles)')

```

```

train.csv : 299 filas
eval.csv : 65 filas
Combinado : 364 filas (para calculo de percentiles)

```

3.- Cálculo de umbrales de clasificación

Cada target se divide en tres clases de igual tamaño aproximado usando los percentiles 33 y 66 calculados sobre `train + eval`:

Clase	Criterio
Bajo	Valor $\leq$ percentil 33
Medio	Percentil 33 < Valor $\leq$ percentil 66

Clase	Criterio
Alto	Valor > percentil 66

Este enfoque garantiza que cada clase represente aproximadamente un tercio del dataset en el conjunto combinado, produciendo clases equilibradas y evitando que una clase domine por tener un rango de valores mucho más amplio que las otras.

Los umbrales se calculan una sola vez aquí y se reutilizan para todos los modelos.

```
In [3]: # Calcular percentiles 33 y 66 por target sobre el conjunto combinado
umbrales = {}
for t in TARGETS:
    p33 = df_combined[t].quantile(0.33)
    p66 = df_combined[t].quantile(0.66)
    umbrales[t] = (p33, p66)

print('Umbrales de clasificación (p33, p66) por target:')
print(f' {"Target":30} {"p33":>8} {"p66":>8}')
print('-' * 50)
for t, (p33, p66) in umbrales.items():
    print(f' {t:30} {p33:8.4f} {p66:8.4f}')

def clasificar(valores, target):
    """
    Convierte un array de valores continuos en clases ordinales
    (0=Bajo, 1=Medio, 2=Alto) usando los umbrales globales del target.
    """
    p33, p66 = umbrales[target]
    return np.where(valores ≤ p33, 0, np.where(valores ≤ p66, 1, 2))
```

Umbrales de clasificación (p33, p66) por target:

Target	p33	p66
nematode_shannon_z	-0.1869	0.5169
macro_shannon_z	-0.3214	0.6288
earthworm_shannon_z	0.0299	0.4239
orib_shannon_z	-0.8646	0.5237
meso_shannon_z	-0.6420	-0.2622
coll_shannon_z	-0.6186	-0.6186
bac_shannon_z	-0.0287	0.3980
fun_shannon_z	-0.3097	0.4067
euk_shannon_z	-0.2973	0.3801
oomy_shannon_z	-0.2735	0.4768
cerc_shannon_z	-0.1777	0.3593
macro_order_richness_z	-0.6013	0.2751
earthworm_richness_z	-0.3821	-0.0079
orib_species_richness_z	-0.7099	0.2322
meso_species_richness_z	-0.7783	0.3988
coll_species_richness_z	-0.6982	-0.1721
bac_asv_richness_z	-0.2013	0.3079
fun_asv_richness_z	-0.3512	0.2940
euk_asv_richness_z	-0.4086	0.2268
oomy_asv_richness_z	-0.3641	0.4635
cerc_asv_richness_z	-0.3663	0.4012

4.- Métricas empleadas

Además del R2 estándar, se calculan las siguientes métricas sobre las clases ordinales.

Todas están justificadas por su complementariedad con el R2 en contextos ecológicos donde interesa más acertar la categoría de diversidad que el valor exacto:

- **R2 (coeficiente de determinación)** : Mide qué proporción de la varianza real explica el modelo sobre los valores continuos. Es la métrica principal de comparación entre modelos.
- **Accuracy (exactitud de clasificación)** : Proporción de muestras en las que el modelo asigna correctamente la clase Bajo/Medio/Alto. Una accuracy aleatoria sería 0.33 (1 de cada 3 clases). Se usa como métrica base de clasificación, aunque puede ser engañosa si las clases no están perfectamente equilibradas.
- **Cohen's Kappa** : Mide el acuerdo entre las clasificaciones del modelo y las reales, descontando el acuerdo que ocurriría por azar. Va de -1 a 1, donde 0 equivale a clasificación aleatoria y 1 a acuerdo perfecto. Se usa aquí en su variante **ponderada lineal** (`weights='linear'`), que penaliza más los errores grandes (confundir Bajo con Alto) que los pequeños (confundir Bajo con Medio). Esta ponderación es especialmente adecuada para variables ordinales como las clases de diversidad biológica.
- **F1-macro** : Media del F1 calculado por separado para cada clase, sin ponderar por el tamaño de cada una. Captura el rendimiento del modelo en cada nivel de diversidad por igual, lo que es útil cuando nos interesa que el modelo funcione bien tanto para suelos de baja diversidad como para los de alta diversidad, sin que una clase mayoritaria enmascare los errores en las minoritarias.
- **Matriz de confusión** : Tabla 3x3 que muestra cuántas muestras de cada clase real fueron predichas en cada clase. Permite identificar el patrón de errores del modelo: si tiende a confundir Bajo con Medio (error leve) o Bajo con Alto (error grave). Es la visualización más informativa para entender el comportamiento del clasificador.

5.- Funciones auxiliares

Funciones reutilizables para cargar modelos, generar predicciones y calcular todas las métricas sobre `eval.csv`.

```

In [4]: def calcular_metricas(y_true_cont, y_pred_cont, target):
    """
    Calcula metricas continuas (R2, RMSE, MAE) y de clasificacion ordinal
    (Accuracy, Kappa ponderado, F1-macro) para un target dado.
    """
    r2 = r2_score(y_true_cont, y_pred_cont)
    rmse = np.sqrt(mean_squared_error(y_true_cont, y_pred_cont))
    mae = mean_absolute_error(y_true_cont, y_pred_cont)

    y_true_cls = clasificar(y_true_cont, target)
    y_pred_cls = clasificar(y_pred_cont, target)

    acc = accuracy_score(y_true_cls, y_pred_cls)
    kappa = cohen_kappa_score(y_true_cls, y_pred_cls, weights='linear')
    f1 = f1_score(y_true_cls, y_pred_cls, average='macro', zero_division=0)
    cm = confusion_matrix(y_true_cls, y_pred_cls, labels=[0, 1, 2])

    return {
        'r2': round(r2, 4), 'rmse': round(rmse, 4), 'mae': round(mae, 4),
        'accuracy': round(acc, 4), 'kappa': round(kappa, 4), 'f1_macro': round(f1, 4),
        'confusion_matrix': cm,
        'y_true_cls': y_true_cls, 'y_pred_cls': y_pred_cls
    }

def metricas_globales(y_true_mat, y_pred_mat, nombre):
    """
    Calcula metricas globales (promedio sobre los 21 targets) para un modelo.
    Devuelve un dict con las medias y la lista de metricas por target.
    """
    resultados_por_target = {}
    for i, t in enumerate(TARGETS):
        resultados_por_target[t] = calcular_metricas(
            y_true_mat[:, i], y_pred_mat[:, i], t
        )

    r2_global = np.mean([v['r2'] for v in resultados_por_target.values()])
    acc_global = np.mean([v['accuracy'] for v in resultados_por_target.values()])
    kappa_global = np.mean([v['kappa'] for v in resultados_por_target.values()])
    f1_global = np.mean([v['f1_macro'] for v in resultados_por_target.values()])
    rmse_global = np.mean([v['rmse'] for v in resultados_por_target.values()])

    return {
        'modelo': nombre,
        'r2_global': round(r2_global, 4),
        'accuracy_global': round(acc_global, 4),
        'kappa_global': round(kappa_global, 4),
        'f1_global': round(f1_global, 4),
        'rmse_global': round(rmse_global, 4),
        'por_target': resultados_por_target
    }

def predecir_pkL_individual(carpeta, fn_nombre, n, X):
    """Carga n modelos individuales (uno por target) y devuelve matriz (n_eval x n_targets)."""
    y_pred = np.zeros((len(X), len(TARGETS)))
    for j, t in enumerate(TARGETS):
        preds = [joblib.load(os.path.join(MODELS_BASE, carpeta, fn_nombre(t, i))).predict(X)
                  for i in range(n)]
        y_pred[:, j] = np.mean(preds, axis=0)
    return y_pred

def predecir_pkL_multi(carpeta, prefijo, sufijo, n, X):
    """Carga n modelos multisalida y devuelve el promedio de predicciones."""
    preds = [joblib.load(os.path.join(MODELS_BASE, carpeta, f'{prefijo}{i}{sufijo}.pkl')).predict(X)
              for i in range(n)]
    return np.mean(preds, axis=0)

class MLPMultiSalida(nn.Module):
    def __init__(self, n_inputs, n_outputs, hidden_sizes, dropout_rate):
        super().__init__()
        layers, in_size = [], n_inputs
        for h in hidden_sizes:
            layers += [nn.Linear(in_size, h), nn.ReLU(), nn.Dropout(dropout_rate)]
            in_size = h
        layers.append(nn.Linear(in_size, n_outputs))
        self.red = nn.Sequential(*layers)
    def forward(self, x): return self.red(x)

def predecir_mlp(carpeta, prefijo, n, X, config_file):
    """Carga n redes PyTorch y devuelve el promedio de predicciones."""
    cfg = joblib.load(os.path.join(MODELS_BASE, carpeta, config_file))
    X_t = torch.tensor(X.values, dtype=torch.float32).to(DEVICE)
    preds = []
    for i in range(n):
        model = MLPMultiSalida(X.shape[1], len(TARGETS), cfg['hidden'], cfg['dropout']).to(DEVICE)
        model.load_state_dict(torch.load(
            os.path.join(MODELS_BASE, carpeta, f'{prefijo}{i}.pt'), map_location=DEVICE
        ))
        model.eval()

```

```

        with torch.no_grad():
            preds.append(model(X_t).cpu().numpy())
        return np.mean(preds, axis=0)

print('Funciones auxiliares definidas.')
```

Funciones auxiliares definidas.

6.- Predicciones de cada modelo

Se cargan todos los modelos desde disco y se generan predicciones sobre `eval.csv`.

Las predicciones son siempre valores continuos; la conversión a clases ordinales se realiza dentro de `calcular_metricas` usando los umbrales de la sección 3.

```

In [5]: resultados = {}
y_true_mat = y_eval.values

print('Cargando modelos y generando predicciones...\n')

# 1. Random Forest individual
print(' [1/8] Random Forest individual...')
y_pred = predecir_pkl_individual('rf', lambda t, i: f'rf_prod_{t}_m{i}.pkl', 5, X_eval)
resultados['RF Individual'] = metricas_globales(y_true_mat, y_pred, 'RF Individual')

# 2. XGBoost individual
print(' [2/8] XGBoost individual...')
y_pred = predecir_pkl_individual('xgb', lambda t, i: f'xgb_prod_{t}_exp{i}.pkl', 5, X_eval)
resultados['XGBoost Individual'] = metricas_globales(y_true_mat, y_pred, 'XGBoost Individual')

# 3. Ridge individual
print(' [3/8] Ridge Regression...')
y_pred_ridge = np.zeros((len(X_eval), len(TARGETS)))
for j, t in enumerate(TARGETS):
    m = joblib.load(os.path.join(MODELS_BASE, 'reg', f'ridge_prod_{t}.pkl'))
    y_pred_ridge[:, j] = m.predict(X_eval)
resultados['Ridge'] = metricas_globales(y_true_mat, y_pred_ridge, 'Ridge')

# 4. RF multisalida
print(' [4/8] Random Forest multisalida...')
y_pred = predecir_pkl_multi('rf_multi', 'rf_multi_m', '', 5, X_eval)
resultados['RF Multisalida'] = metricas_globales(y_true_mat, y_pred, 'RF Multisalida')

# 5. XGBoost multisalida
print(' [5/8] XGBoost multisalida...')
y_pred = predecir_pkl_multi('xgb_multi', 'xgb_multi_exp', '', 5, X_eval)
resultados['XGBoost Multisalida'] = metricas_globales(y_true_mat, y_pred, 'XGBoost Multisalida')

# 6. RegressorChain
print(' [6/8] Ensemble RegressorChain...')
N_CHAINS = 10
preds_chain = [joblib.load(os.path.join(MODELS_BASE, 'chain', f'chain_{i}.pkl')).predict(X_eval)
                for i in range(N_CHAINS)]
y_pred = np.mean(preds_chain, axis=0)
resultados['RegressorChain'] = metricas_globales(y_true_mat, y_pred, 'RegressorChain')

# 7. MLP estandar
print(' [7/8] MLP Multisalida estandar...')
y_pred = predecir_mlp('mlp', 'mlp_prod_', 5, X_eval, 'mlp_config.pkl')
resultados['MLP Estandar'] = metricas_globales(y_true_mat, y_pred, 'MLP Estandar')

# 8. MLP perdida personalizada
print(' [8/8] MLP perdida personalizada...')
y_pred = predecir_mlp('mlp_custom', 'mlp_custom_', 5, X_eval, 'mlp_custom_config.pkl')
resultados['MLP Custom'] = metricas_globales(y_true_mat, y_pred, 'MLP Custom')

print('\nPredicciones completadas.')
```

Cargando modelos y generando predicciones...

```

[1/8] Random Forest individual...
[2/8] XGBoost individual...
[3/8] Ridge Regression...
[4/8] Random Forest multisalida...
[5/8] XGBoost multisalida...
[6/8] Ensemble RegressorChain...
[7/8] MLP Multisalida estandar...
[8/8] MLP perdida personalizada...
```

Predicciones completadas.

7.- Tabla comparativa global

Se muestran todas las métricas globales (promedio sobre los 21 targets) para cada modelo, ordenadas por R2 descendente.

Guía de interpretación:

- **R2:** que proporción de la varianza real explica el modelo (cuanto mas cerca de 1, mejor).
- **Accuracy:** proporción de muestras clasificadas correctamente en Bajo/Medio/Alto. El azar da 0.33.

- **Kappa ponderado:** acuerdo con los valores reales descontando el azar, penalizando mas los errores graves (Bajo ↔ Alto).
- **F1-macro:** equilibrio entre precisión y recall promediado por igual entre las tres clases.
- **RMSE:** error cuadrático medio sobre los valores continuos.

```
In [6]: tabla = pd.DataFrame([{'Modelo': r['modelo'],
                              'R2': r['r2_global'],
                              'Accuracy': r['accuracy_global'],
                              'Kappa': r['kappa_global'],
                              'F1-macro': r['f1_global'],
                              'RMSE': r['rmse_global'],
                              } for r in resultados.values()]).sort_values('R2', ascending=False).reset_index(drop=True)

tabla.index += 1
print('Comparacion de modelos (ordenado por R2 descendente):')
print('=' * 75)
print(tabla.to_string())
print('=' * 75)
print(f'\nMejor modelo por R2      : {tabla.iloc[0]["Modelo"]} ({tabla.iloc[0]["R2"]})')
print(f'Mejor modelo por Kappa    : {tabla.sort_values("Kappa", ascending=False).iloc[0]["Modelo"]}')
print(f'Mejor modelo por F1-macro : {tabla.sort_values("F1-macro", ascending=False).iloc[0]["Modelo"]}')

Comparacion de modelos (ordenado por R2 descendente):
=====
      Modelo      R2  Accuracy  Kappa  F1-macro  RMSE
1  RF Multisalida  0.0964    0.3824  0.1331    0.3002  0.9078
2  RF Individual  0.0904    0.3971  0.1644    0.3443  0.9037
3  XGBoost Multisalida  0.0835    0.4403  0.2341    0.4101  0.9035
4  RegressorChain  0.0809    0.4022  0.1827    0.3564  0.9053
5  XGBoost Individual  0.0670    0.3978  0.1576    0.3413  0.9167
6      Ridge -0.0164    0.3744  0.1124    0.3034  0.9576
7  MLP Estandar -0.0380    0.4037  0.1726    0.3603  0.9473
8  MLP Custom -0.1316    0.4103  0.1864    0.3789  0.9743
=====

Mejor modelo por R2      : RF Multisalida (0.0964)
Mejor modelo por Kappa    : XGBoost Multisalida
Mejor modelo por F1-macro : XGBoost Multisalida
```

8.- Métricas por target

Tabla detallada con R2, Accuracy, Kappa y F1-macro para cada uno de los 21 targets y cada modelo.

Permite identificar en qué grupos biológicos falla cada enfoque y si los errores son de tipo leve (Bajo ↔ Medio) o grave (Bajo ↔ Alto).

```
In [7]: METRICA_KEYS = {
        'r2': 'r2_global',
        'accuracy': 'accuracy_global',
        'kappa': 'kappa_global',
        'f1_macro': 'f1_global',
        }

for metrica, key_global in METRICA_KEYS.items():
    filas = []
    for nombre, res in resultados.items():
        filas.append({'Target': 'GLOBAL', 'Modelo': nombre,
                     'Valor': res[key_global]})
    for t in TARGETS:
        filas.append({'Target': t, 'Modelo': nombre,
                     'Valor': res['por_target'][t][metrica]})

df_m = pd.DataFrame(filas).pivot(index='Target', columns='Modelo', values='Valor')
orden_cols = tabla['Modelo'].tolist()
df_m = df_m[orden_cols]
orden_rows = ['GLOBAL'] + TARGETS
df_m = df_m.loc[orden_rows]

print(f'\n{metrica.upper()} por target:')
print('=' * 120)
print(df_m.round(3).to_string())
print('=' * 120)
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

R2 por target:											
Modelo	RF	Multisalida	RF	Individual	XGBoost	Multisalida	RegressorChain	XGBoost	Individual	Ridge	MLP
Estandar	MLP	Custom									
Target											
GLOBAL			0.096	0.090	0.084		0.081		0.067	-0.016	
-0.038	-0.132										
nematode_shannon_z			0.154	0.169	0.223		0.147		0.164	0.011	
0.118	0.090										
macro_shannon_z			0.200	0.320	0.377		0.280		0.260	0.126	
0.257	0.268										
earthworm_shannon_z			0.416	0.504	0.538		0.488		0.495	0.240	
0.426	0.375										
orib_shannon_z			0.169	0.179	0.228		0.207		0.115	0.114	
0.236	0.222										
meso_shannon_z			-0.118	-0.173	-0.286		-0.164		-0.187	-0.250	
-0.490	-0.619										
coll_shannon_z			-0.195	-0.349	-0.323		-0.609		-0.331	-0.500	
-0.823	-1.344										
bac_shannon_z			0.029	0.008	-0.015		0.029		0.009	-0.081	
-0.085	-0.100										
fun_shannon_z			-0.013	-0.046	-0.050		-0.022		-0.021	-0.016	
-0.076	-0.084										
euk_shannon_z			0.138	0.168	0.089		0.162		0.091	0.041	
0.145	0.087										
oomy_shannon_z			0.125	0.083	0.062		0.091		0.096	0.086	
0.153	0.059										
cerc_shannon_z			-0.004	-0.032	-0.033		-0.001		-0.041	0.016	
-0.136	-0.119										
macro_order_richness_z			0.253	0.342	0.401		0.407		0.308	0.152	
0.352	0.376										
earthworm_richness_z			0.451	0.572	0.589		0.536		0.517	0.279	
0.518	0.467										
orib_species_richness_z			0.239	0.203	0.280		0.203		0.126	0.092	
0.260	0.254										
meso_species_richness_z			-0.026	-0.044	-0.115		-0.040		-0.059	-0.096	
-0.275	-0.346										
coll_species_richness_z			-0.292	-0.548	-0.646		-0.580		-0.580	-0.734	
-1.463	-2.241										
bac_asv_richness_z			0.004	0.004	0.016		-0.005		0.001	-0.076	
-0.146	-0.168										
fun_asv_richness_z			-0.007	-0.026	-0.066		-0.019		-0.024	0.009	
0.014	0.003										
euk_asv_richness_z			0.190	0.252	0.223		0.269		0.167	0.040	
0.216	0.190										
oomy_asv_richness_z			0.192	0.184	0.165		0.202		0.178	0.104	
0.011	-0.055										
cerc_asv_richness_z			0.117	0.125	0.098		0.117		0.123	0.102	
-0.012	-0.077										
=====											
ACCURACY por target:											
Modelo	RF	Multisalida	RF	Individual	XGBoost	Multisalida	RegressorChain	XGBoost	Individual	Ridge	MLP
Estandar	MLP	Custom									
Target											
GLOBAL			0.382	0.397	0.440		0.402		0.398	0.374	
0.404	0.410										
nematode_shannon_z			0.400	0.385	0.477		0.385		0.415	0.369	
0.400	0.400										
macro_shannon_z			0.446	0.508	0.554		0.492		0.462	0.431	
0.538	0.508										
earthworm_shannon_z			0.631	0.723	0.754		0.723		0.692	0.477	
0.631	0.585										
orib_shannon_z			0.231	0.215	0.292		0.215		0.200	0.185	
0.277	0.323										
meso_shannon_z			0.246	0.231	0.215		0.185		0.231	0.231	
0.185	0.246										
coll_shannon_z			0.246	0.246	0.292		0.246		0.246	0.246	
0.246	0.262										
bac_shannon_z			0.385	0.354	0.431		0.385		0.400	0.385	
0.369	0.369										
fun_shannon_z			0.231	0.292	0.323		0.277		0.231	0.231	
0.262	0.262										
euk_shannon_z			0.462	0.492	0.492		0.477		0.446	0.369	
0.462	0.462										
oomy_shannon_z			0.431	0.385	0.523		0.415		0.462	0.385	
0.462	0.492										
cerc_shannon_z			0.354	0.354	0.369		0.400		0.385	0.369	
0.369	0.338										
macro_order_richness_z			0.369	0.492	0.569		0.508		0.400	0.338	
0.462	0.477										
earthworm_richness_z			0.554	0.569	0.631		0.600		0.631	0.554	
0.615	0.615										
orib_species_richness_z			0.400	0.431	0.431		0.446		0.415	0.446	
0.431	0.477										
meso_species_richness_z			0.369	0.308	0.308		0.323		0.338	0.385	
0.338	0.369										
coll_species_richness_z			0.323	0.354	0.323		0.323		0.338	0.338	
0.323	0.292										
bac_asv_richness_z			0.338	0.354	0.446		0.338		0.369	0.431	
0.354	0.369										
fun_asv_richness_z			0.462	0.477	0.462		0.477		0.462	0.492	
0.538	0.523										

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

euk_asv_richness_z	0.431	0.492	0.600	0.538	0.477	0.446
0.523 0.538						
oomy_asv_richness_z	0.446	0.385	0.415	0.446	0.415	0.385
0.415 0.431						
cerc_asv_richness_z	0.277	0.292	0.338	0.246	0.338	0.369
0.277 0.277						

KAPPA por target:

Modelo	RF Multisalida	RF Individual	XGBoost Multisalida	RegressorChain	XGBoost Individual	Ridge	MLP
Estandar							
Target							
GLOBAL	0.133	0.164	0.234	0.183	0.158	0.112	
0.173 0.186							
nematode_shannon_z	0.054	0.043	0.219	0.043	0.100	0.057	
0.113 0.097							
macro_shannon_z	0.170	0.293	0.374	0.286	0.221	0.115	
0.331 0.327							
earthworm_shannon_z	0.483	0.546	0.632	0.592	0.561	0.216	
0.449 0.381							
orib_shannon_z	0.107	0.092	0.167	0.077	0.050	0.050	
0.170 0.228							
meso_shannon_z	-0.035	-0.024	-0.013	0.031	-0.024	-0.047	
-0.045 -0.017							
coll_shannon_z	0.000	0.000	0.031	0.000	0.000	0.000	
0.000 0.010							
bac_shannon_z	0.168	0.146	0.251	0.196	0.181	0.145	
0.134 0.156							
fun_shannon_z	0.001	0.045	0.077	0.040	-0.013	0.000	
-0.010 -0.018							
euk_shannon_z	0.213	0.313	0.321	0.287	0.252	0.091	
0.252 0.244							
oomy_shannon_z	0.186	0.160	0.335	0.201	0.258	0.171	
0.282 0.335							
cerc_shannon_z	0.065	0.070	0.119	0.144	0.138	0.052	
0.061 0.035							
macro_order_richness_z	0.177	0.388	0.456	0.399	0.224	0.078	
0.282 0.342							
earthworm_richness_z	0.425	0.435	0.492	0.454	0.487	0.388	
0.485 0.456							
orib_species_richness_z	0.114	0.158	0.158	0.188	0.127	0.155	
0.141 0.244							
meso_species_richness_z	-0.045	-0.070	-0.044	-0.044	-0.044	0.021	
0.007 0.079							
coll_species_richness_z	0.113	0.137	0.110	0.105	0.110	0.084	
0.141 0.105							
bac_asv_richness_z	0.113	0.112	0.264	0.138	0.140	0.207	
0.094 0.130							
fun_asv_richness_z	0.029	0.086	0.119	0.108	0.000	0.069	
0.208 0.177							
euk_asv_richness_z	0.164	0.273	0.454	0.352	0.254	0.178	
0.326 0.373							
oomy_asv_richness_z	0.228	0.194	0.245	0.250	0.186	0.125	
0.151 0.164							
cerc_asv_richness_z	0.065	0.055	0.148	-0.011	0.100	0.203	
0.053 0.068							

F1_MACRO por target:

Modelo	RF Multisalida	RF Individual	XGBoost Multisalida	RegressorChain	XGBoost Individual	Ridge	MLP
Estandar							
Target							
GLOBAL	0.300	0.344	0.410	0.356	0.341	0.303	
0.360 0.379							
nematode_shannon_z	0.278	0.279	0.405	0.279	0.316	0.301	
0.302 0.314							
macro_shannon_z	0.316	0.454	0.528	0.439	0.383	0.303	
0.487 0.481							
earthworm_shannon_z	0.581	0.710	0.746	0.709	0.679	0.401	
0.608 0.559							
orib_shannon_z	0.203	0.191	0.268	0.197	0.161	0.151	
0.256 0.319							
meso_shannon_z	0.138	0.139	0.144	0.146	0.139	0.137	
0.123 0.180							
coll_shannon_z	0.198	0.198	0.263	0.198	0.198	0.198	
0.198 0.220							
bac_shannon_z	0.328	0.303	0.423	0.358	0.388	0.351	
0.358 0.356							
fun_shannon_z	0.149	0.231	0.306	0.218	0.127	0.125	
0.228 0.237							
euk_shannon_z	0.376	0.470	0.492	0.452	0.433	0.318	
0.442 0.450							
oomy_shannon_z	0.363	0.361	0.516	0.397	0.448	0.354	
0.415 0.491							
cerc_shannon_z	0.270	0.288	0.349	0.333	0.338	0.258	
0.339 0.308							
macro_order_richness_z	0.318	0.489	0.572	0.508	0.371	0.258	
0.455 0.481							
earthworm_richness_z	0.523	0.519	0.604	0.577	0.612	0.548	
0.588 0.586							
orib_species_richness_z	0.304	0.332	0.332	0.349	0.315	0.336	
0.325 0.426							

meso_species_richness_z	0.184	0.161	0.187	0.196	0.204	0.251
0.225	0.280					
coll_species_richness_z	0.257	0.292	0.269	0.257	0.285	0.280
0.265	0.246					
bac_asv_richness_z	0.296	0.347	0.447	0.330	0.360	0.431
0.346	0.371					
fun_asv_richness_z	0.261	0.323	0.395	0.352	0.210	0.264
0.422	0.410					
euk_asv_richness_z	0.326	0.463	0.603	0.518	0.454	0.408
0.512	0.542					
oomy_asv_richness_z	0.400	0.394	0.419	0.438	0.406	0.330
0.394	0.422					
cerc_asv_richness_z	0.236	0.289	0.344	0.234	0.340	0.368
0.276	0.278					

9.- Matrices de confusión

Para cada modelo se muestra la matriz de confusión agregada: suma de las 21 matrices individuales (una por target), normalizada por filas para mostrar proporciones.

La diagonal principal representa los aciertos. Los elementos fuera de la diagonal muestran el tipo de error: los errores en la subdiagonal o superdiagonal inmediata (Bajo ↔ Medio o Medio ↔ Alto) son errores leves, mientras que los errores en las esquinas (Bajo ↔ Alto) son errores graves que el Kappa ponderado penaliza mas.

Se muestran adicionalmente las matrices de confusión individuales para los dos targets de referencia (earthworm_shannon_z y earthworm_richness_z).

```
In [8]: # Matrices de confusion agregadas (suma de los 21 targets, normalizada por filas)
n_modelos = len(resultados)
n_cols = 4
n_rows = (n_modelos + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, n_rows * 4))
axes = axes.flatten()

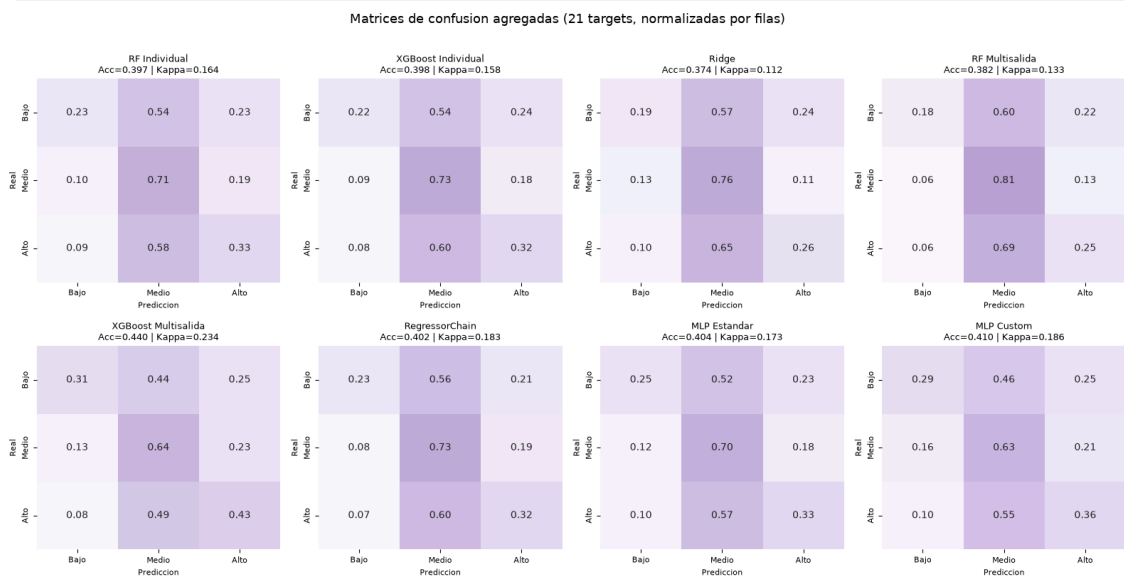
for idx, (nombre, res) in enumerate(resultados.items()):
    cm_agg = np.zeros((3, 3), dtype=int)
    for t in TARGETS:
        cm_agg += res['por_target'][t]['confusion_matrix']

    cm_norm = cm_agg.astype(float) / cm_agg.sum(axis=1, keepdims=True)

    sns.heatmap(cm_norm, ax=axes[idx], annot=True, fmt='.2f', cmap=PALETTE_CMAP,
                xticklabels=CLASES, yticklabels=CLASES,
                vmin=0, vmax=1, cbar=False)
    axes[idx].set_title(f'{nombre}\nAcc={res["accuracy_global"]:.3f} | Kappa={res["kappa_global"]:.3f}',
                        fontsize=9)
    axes[idx].set_xlabel('Prediccion', fontsize=8)
    axes[idx].set_ylabel('Real', fontsize=8)
    axes[idx].tick_params(labelsize=8)

for j in range(idx + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Matrices de confusion agregadas (21 targets, normalizadas por filas)', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()
```



```
In [9]: # Matrices de confusion individuales para los dos targets de referencia
for ref_target in ['earthworm_shannon_z', 'earthworm_richness_z']:
    n_modelos = len(resultados)
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    axes = axes.flatten()

    for idx, (nombre, res) in enumerate(resultados.items()):
        cm = res['por_target'][ref_target]['confusion_matrix']
        cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True).clip(min=1)

        sns.heatmap(cm_norm, ax=axes[idx], annot=True, fmt='.2f', cmap=PALETTE_CMAP,
                    xticklabels=CLASES, yticklabels=CLASES,
                    vmin=0, vmax=1, cbar=False)
        kappa_t = res['por_target'][ref_target]['kappa']
        acc_t = res['por_target'][ref_target]['accuracy']
        axes[idx].set_title(f'{nombre}\nAcc={acc_t:.3f} | Kappa={kappa_t:.3f}', fontsize=9)
        axes[idx].set_xlabel('Predicción', fontsize=8)
        axes[idx].set_ylabel('Real', fontsize=8)
        axes[idx].tick_params(labelsize=8)

    plt.suptitle(f'Matrices de confusion - {ref_target}', fontsize=13, y=1.01)
    plt.tight_layout()
    plt.show()
```



10.- Visualización comparativa de métricas

Gráficos de barras comparando todos los modelos en las cuatro métricas principales para facilitar la seleccion visual del mejor enfoque.

```
In [10]: metricas_plot = {
    'R2 global': [r['r2_global'] for r in resultados.values()],
    'Accuracy global': [r['accuracy_global'] for r in resultados.values()],
```

```

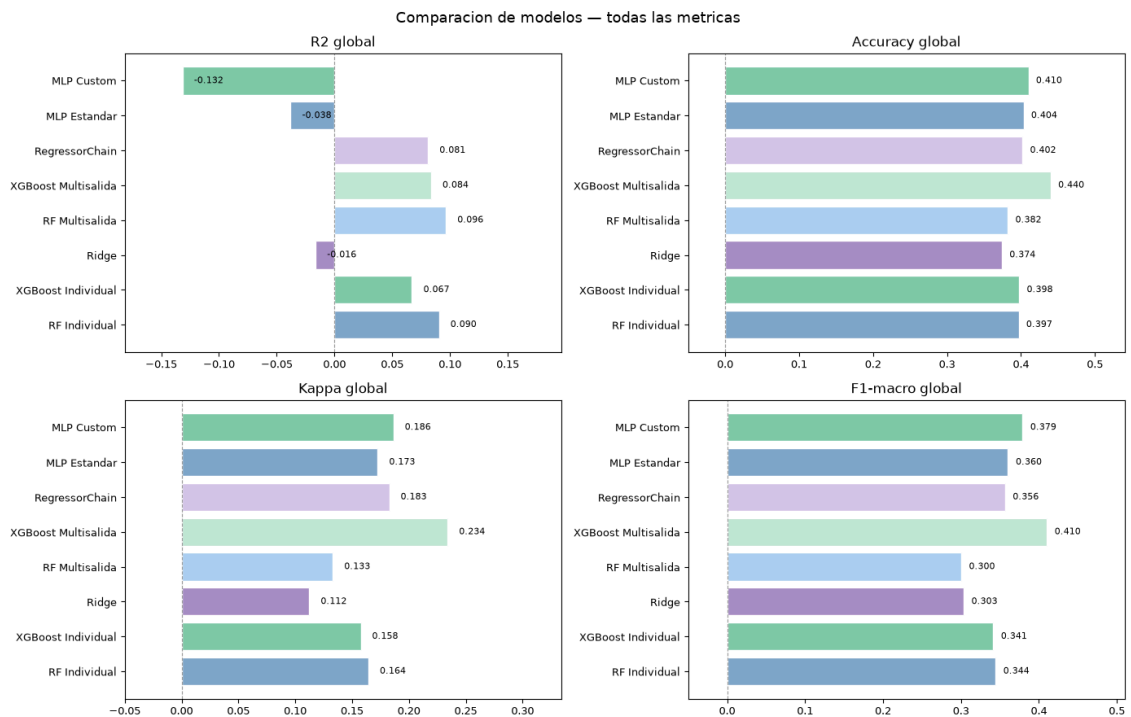
'Kappa global': [r['kappa_global'] for r in resultados.values()],
'F1-macro global': [r['f1_global'] for r in resultados.values()],
}
nombres = list(resultados.keys())
colores = palette_cycle(len(nombres))

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()

for ax, (titulo, valores) in zip(axes, metricas_plot.items()):
    bars = ax.barh(nombres, valores, color=colores, edgecolor='white')
    ax.set_title(titulo, fontsize=12)
    ax.set_xlim(min(0, min(valores)) - 0.05, max(valores) + 0.1)
    ax.axvline(0, color=PALETTE['gray'], linewidth=0.8, linestyle='--')
    for bar, val in zip(bars, valores):
        ax.text(val + 0.01, bar.get_y() + bar.get_height()/2,
                f'{val:.3f}', va='center', fontsize=8)
    ax.tick_params(labelsize=9)

plt.suptitle('Comparacion de modelos — todas las metricas', fontsize=13)
plt.tight_layout()
plt.show()

```



11.- Exportación de resultados

Se guardan las tablas comparativas en CSV para uso externo.

```

In [11]: os.makedirs('output/clasificacion', exist_ok=True)

tabla.to_csv('output/clasificacion/comparacion_global.csv', index=False)

# Tabla R2 por target
filas_r2 = []
for nombre, res in resultados.items():
    filas_r2.append({'Target': 'GLOBAL', 'Modelo': nombre, 'R2': res['r2_global']})
    for t in TARGETS:
        filas_r2.append({'Target': t, 'Modelo': nombre, 'R2': res['por_target'][t]['r2']})

pd.DataFrame(filas_r2).pivot(index='Target', columns='Modelo', values='R2') \
    .to_csv('output/clasificacion/r2_por_target.csv')

# Tabla Kappa por target
filas_k = []
for nombre, res in resultados.items():
    filas_k.append({'Target': 'GLOBAL', 'Modelo': nombre, 'Kappa': res['kappa_global']})
    for t in TARGETS:
        filas_k.append({'Target': t, 'Modelo': nombre, 'Kappa': res['por_target'][t]['kappa']})

pd.DataFrame(filas_k).pivot(index='Target', columns='Modelo', values='Kappa') \
    .to_csv('output/clasificacion/kappa_por_target.csv')

print('Resultados exportados:')
print(' output/clasificacion/comparacion_global.csv')

```

```
print(' output/clasificacion/r2_por_target.csv')
print(' output/clasificacion/kappa_por_target.csv')
```

Resultados exportados:
 output/clasificacion/comparacion_global.csv
 output/clasificacion/r2_por_target.csv
 output/clasificacion/kappa_por_target.csv

```
In [12]: from sklearn.metrics import precision_score, recall_score

print("Generando análisis radial individualizado por modelo...")

os.makedirs('output/clasificacion/graphs', exist_ok=True)

# 1. Extracción de métricas por modelo
filas_radar = []
for nombre, res in resultados.items():
    prec_targets, rec_targets = [], []
    for t in TARGETS:
        y_true_cls = res['por_target'][t]['y_true_cls']
        y_pred_cls = res['por_target'][t]['y_pred_cls']
        prec_targets.append(precision_score(y_true_cls, y_pred_cls, average='macro', zero_division=0))
        rec_targets.append(recall_score(y_true_cls, y_pred_cls, average='macro', zero_division=0))

    f1 = res['f1_global']
    precision = np.mean(prec_targets)
    recall = np.mean(rec_targets)

    # Puntuación compuesta para el orden del ranking (Media geométrica equilibrada)
    score_compuesto = (precision * recall * f1) ** (1/3)

    # Cambiamos 'F1-Score' por 'F1_Score' para evitar conflictos en itertuples()
    filas_radar.append({
        'Modelo': nombre,
        'Precision': precision,
        'Recall': recall,
        'F1_Score': f1,
        'Score_Compuesto': score_compuesto
    })

df_radar_individual = pd.DataFrame(filas_radar).sort_values(by='Score_Compuesto', ascending=False)

# 2. Configuración de la visualización radial conjunta (Matriz 2x4)
categorias = ['Precision', 'Recall', 'F1_Score']
num_vars = len(categorias)
angulos = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
angulos += angulos[:1]

fig, axes = plt.subplots(2, 4, figsize=(20, 10), subplot_kw=dict(polar=True))
axes = axes.flatten()
colores = palette_cycle(len(df_radar_individual))

for idx, fila in df_radar_individual.reset_index(drop=True).iterrows():
    ax = axes[idx]
    ax.set_theta_offset(np.pi / 2)
    ax.set_theta_direction(-1)

    valores = [fila['Precision'], fila['Recall'], fila['F1_Score']]
    valores += valores[:1]

    # Dibujar polígono en la cuadrícula general
    ax.plot(angulos, valores, color=colores[idx], linewidth=2, label=fila['Modelo'])
    ax.fill(angulos, valores, color=colores[idx], alpha=0.15)

    ax.set_xticks(angulos[:-1])
    ax.set_xticklabels(['Precisión', 'Recall', 'F1-Score'], fontsize=9, fontweight='bold')
    ax.set_ylim(0, 0.6) # Ajustar según tus límites de datos reales
    ax.set_title(f"{idx+1}. {fila['Modelo']}\n(Score: {fila['Score_Compuesto']:.3f})", fontsize=11, fontweight='bold', pad=5)

    # Guardar también como gráfico independiente para la memoria escrita
    fig_ind, ax_ind = plt.subplots(figsize=(5, 5), subplot_kw=dict(polar=True))
    ax_ind.set_theta_offset(np.pi / 2)
    ax_ind.set_theta_direction(-1)
    ax_ind.plot(angulos, valores, color=colores[idx], linewidth=2.5)
    ax_ind.fill(angulos, valores, color=colores[idx], alpha=0.2)
    ax_ind.set_xticks(angulos[:-1])
    ax_ind.set_xticklabels(['Precisión', 'Recall', 'F1-Score'], fontsize=10, fontweight='bold')
    ax_ind.set_ylim(0, 0.6)
    ax_ind.set_title(f"Perfil de Calidad: {fila['Modelo']}", fontsize=12, fontweight='bold')
    fig_ind.tight_layout()
    nombre_archivo = f"output/clasificacion/graphs/radar_{fila['Modelo'].lower().replace(' ', '_')}.png"
    fig_ind.savefig(nombre_archivo, dpi=300)
    plt.close(fig_ind)

# Ajustar y guardar el mosaico general
fig.suptitle("Análisis Comparativo Radial Individualizado por Arquitectura (Métricas Ordinales Macro)", fontsize=16, fontweight='bold')
fig.tight_layout()
fig.savefig('output/clasificacion/graphs/mosaico_radar_modelos.png', dpi=300, bbox_inches='tight')
plt.show()

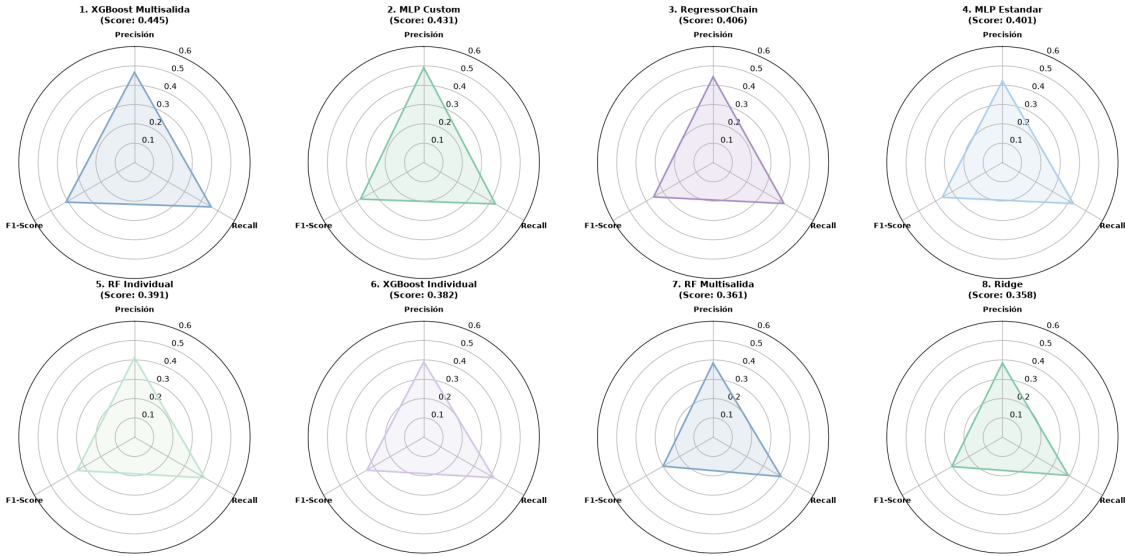
# 3. Imprimir el Top 8 oficial con identificadores válidos r.F1_Score (S mayúscula)

print("Top 8 modelos (Métricas de Clasificación Ordinal)\n")
```

```
for i, r in enumerate(df_radar_individual.itertuples(), 1):
    print(f"Top {i}: {r.Modelo:<35} | Compuesto: {r.Score_Compuesto:.4f} | F1: {r.F1_Score:.4f} | P: {r.Precision:.4f} | R:
```

Generando análisis radial individualizado por modelo...

Análisis Comparativo Radial Individualizado por Arquitectura (Métricas Ordinales Macro)



Top 8 modelos (Métricas de Clasificación Ordinal)

Top 1: XGBoost Multisalida	Compuesto: 0.4447	F1: 0.4101	P: 0.4664	R: 0.4598
Top 2: MLP Custom	Compuesto: 0.4306	F1: 0.3789	P: 0.4911	R: 0.4292
Top 3: RegressorChain	Compuesto: 0.4063	F1: 0.3564	P: 0.4433	R: 0.4246
Top 4: MLP Estandar	Compuesto: 0.4012	F1: 0.3603	P: 0.4213	R: 0.4253
Top 5: RF Individual	Compuesto: 0.3908	F1: 0.3443	P: 0.4125	R: 0.4203
Top 6: XGBoost Individual	Compuesto: 0.3821	F1: 0.3413	P: 0.3894	R: 0.4198
Top 7: RF Multisalida	Compuesto: 0.3606	F1: 0.3002	P: 0.3834	R: 0.4073
Top 8: Ridge	Compuesto: 0.3584	F1: 0.3034	P: 0.3839	R: 0.3952

11.3.4.3. Comparador entre ramas

SOB4ES - Extractor automatico de resultados entre ramas

Este notebook automatiza lo que se ha estado haciendo a mano hasta ahora: leer los notebooks de una rama de git (sin necesidad de hacer `checkout`, usando `git show` `rama:archivo`), extraer las métricas de evaluación, el número de variables activas, y detectar automáticamente dos problemas que ya han aparecido varias veces de forma manual:

1. **Variables que no coinciden** entre lo que dice `FEATURES_AUTORIZADAS` y lo que realmente se cargo en `X_train` (columna comentada pero no re-ejecutado).
2. **Celdas ejecutadas fuera de orden** (`execution_count` no creciente de arriba a abajo), que es la causa raíz de los resultados "pegados" de una ejecución anterior (el bug que se detectó en `regressorchain.ipynb` y `mlp_custom_loss.ipynb`).

Compara dos ramas cualesquiera (p. ej. `main` contra `prueba1-iteracion-2`) para los 8 notebooks de modelos, y genera la misma tabla comparativa que se ha ido pidiendo manualmente en el chat.

Requisito: tener el repositorio clonado localmente (funciona sobre tu copia local de git, sin necesidad de subir nada a ningún sitio). No hace falta hacer `git checkout` de las ramas - se leen los archivos directamente del historial de git.

0.- Configuración

```
In [1]: import os
import json
import re
import pandas as pd
from io import StringIO
from IPython.display import display
import subprocess

# --- EDITA ESTO ---
GIT_REPO_PATH = "." # Usar '.' si ejecutas el notebook dentro del repositorio

# Modifica para definir las ramas a comparar
BRANCHES = [
    "model-prep-var",
    # "model-prep-var-x-y", Añade más ramas según necesites
]

BRANCHES_BARRIDO = [
    "model-prep-rs-group", # AJUSTA: nombres reales de las 2 ramas
    "model-prep-rs-group-1",
]

# Notebooks a analizar
NOTEBOOKS = [
    "reg_model.ipynb",
    "rf_model.ipynb",
    "rf_multisalida.ipynb",
    "regressorchain.ipynb",
    "xgboost_model.ipynb",
    "xgb_multisalida.ipynb",
    "mlp_multisalida.ipynb",
    "mlp_custom_loss.ipynb",
]

NOTEBOOKS_SUBDIR = ""

# Lista completa de targets a extraer y comparar
TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z',
]

# Define los targets prioritarios que quieres destacar en la tabla comparativa
TARGETS_PRIORITARIOS = ["earthworm_shannon_z", "earthworm_richness_z"]
```

1.- Funciones de lectura desde git

`obtener_notebook_desde_git` usa `git show <rama><archivo>` para leer el contenido de un archivo tal y como está en una rama concreta, sin tocar el working directory ni hacer checkout. Si el repo es remoto y no lo tienes clonado, hay una alternativa comentada al final de la celda usando la API de GitHub (`raw.githubusercontent.com`).

```
In [2]: def obtener_notebook_desde_git(repo_path, branch, filename, subdir=""):
    """Lee un .ipynb tal y como está en una rama concreta, vía 'git show', sin checkout."""
    ruta_relativa = os.path.join(subdir, filename) if subdir else filename
    try:
        resultado = subprocess.run(
            ["git", "-C", repo_path, "show", f"{branch}:{ruta_relativa}"],
            capture_output=True,
            text=True,
            check=True,
        )
    except subprocess.CalledProcessError as e:
        print(
            f" [ERROR] No se pudo leer {filename} en la rama {branch}: {e.stderr.strip()}"
        )
        return None
    return json.loads(resultado.stdout)
```

```
def listar_ramas(repo_path):
    """Útil para comprobar el nombre exacto de las ramas disponibles (locales y remotas)."""
    resultado = subprocess.run(
        ["git", "-C", repo_path, "branch", "-a"],
        capture_output=True,
        text=True,
    )
    print(resultado.stdout)
```

2.- Funciones de extracción

- `extraer_features_activas`: parsea la lista `FEATURES_AUTORIZADAS` y separa las líneas comentadas (excluidas) de las activas.
- `extraer_shape_xtrain`: busca el print de `X_train`: (filas, columnas) para saber cuantas variables se usaron realmente en el entrenamiento.
- `detectar_celdas_desordenadas`: compara el `execution_count` de las celdas de código en el orden en que aparecen en el notebook; si no es creciente, señala un posible problema de ejecución (resultados de una corrida anterior, no de la actual).
- `extraer_metricas_targets`: busca en los outputs de texto, **a partir del marcador "Evaluacion final sobre eval.csv"**, las líneas con `R2/RMSE/MAE` para los targets pedidos. Restringir la búsqueda a ese bloque es importante: sin eso, el regex puede confundirse con el `R2` de entrenamiento/CV impreso en las celdas de tuning (mucho mas alto por sobreajuste) y dar una lectura falsa. Soporta dos formatos: modelos single-target (`R2+RMSE+MAE` por target) y modelos multisalida (solo `R2` por target, con `RMSE/MAE` unicamente a nivel global).

```
In [3]: def extraer_features_activas(nb_json):
    for c in nb_json["cells"]:
        src = "".join(c.get("source", []))
        if "FEATURES_AUTORIZADAS =" in src:
            m = re.search(r"FEATURES_AUTORIZADAS\s*=\s*[(.*)]", src, re.S)
            if not m:
                continue
            lineas = [l.strip() for l in m.group(1).split("\n") if l.strip()]
            activas, excluidas = [], []
            for linea in lineas:
                nombre_m = re.search(r"'([^\s]+)'", linea)
                if not nombre_m:
                    continue
                nombre = nombre_m.group(1)
                if linea.startswith("#"):
                    excluidas.append(nombre)
                else:
                    activas.append(nombre)
            return activas, excluidas
    return [], []

def extraer_shape_xtrain(nb_json):
    for c in nb_json["cells"]:
        for out in c.get("outputs", []):
            texto = "".join(out.get("text", []))
            m = re.search(r"X_train:\s*((\d+),\s*(\d+)\s*)", texto)
            if m:
                return int(m.group(1)), int(m.group(2))
    return None, None

def detectar_celdas_desordenadas(nb_json):
    """Devuelve una lista de avisos si el execution_count no es creciente."""
    avisos = []
    ultimo_exec = None
    for i, c in enumerate(nb_json["cells"]):
        if c.get("cell_type") != "code":
            continue
        exec_count = c.get("execution_count")
        if exec_count is None:
            continue
        if ultimo_exec is not None and exec_count < ultimo_exec:
            avisos.append(
                f"Celda #{i} tiene execution_count={exec_count}, "
                f"menor que una celda anterior ({ultimo_exec}) → posible resultado obsoleto"
            )
        ultimo_exec = exec_count
    return avisos

MARCADOR_EVAL = "Evaluacion final sobre eval.csv"

def extraer_metricas_targets(nb_json, targets):
    """Extrae R2/RMSE/MAE de la evaluación final para cada target."""
    resultados = {}
    patron_completo = {
        t: re.compile(
            rf"{re.escape(t)}\s+([\d-0-9.]+\s+([\d-0-9.]+\s+([\d-0-9.]+\s+))"
        )
        for t in targets
    }
    patron_solo_r2 = {
        t: re.compile(rf"{re.escape(t)}\s+([\d-0-9.]+\s+)\s*$", re.M)
        for t in targets
    }
    patron_global = re.compile(
        rf"R2\s+global:\s*([\d-0-9.]+\s+)?RMSE\s+global:\s*([\d-0-9.]+\s+)?MAE\s+global:\s*([\d-0-9.]+\s+)",
        re.S,
    )

    for c in nb_json["cells"]:
        # OJO: algunos notebooks (rf_multisalida, xgb_multisalida, regressorchain)
        # imprimen la cabecera "Evaluacion final..." y el bloque de métricas en
        # dos print() distintos, que Jupyter guarda como dos outputs SEPARADOS
        # dentro de la misma celda. Por eso hay que concatenar todos los outputs
        # de la celda ANTES de buscar el marcador, en vez de mirarlos uno a uno.
        texto_completo = "".join(
            "".join(out.get("text", []))
            for out in c.get("outputs", [])
            if out.get("output_type") == "stream"
        )
        if MARCADOR_EVAL not in texto_completo:
            continue
        texto = texto_completo[texto_completo.index(MARCADOR_EVAL):]

        g = patron_global.search(texto)
        global_metrics = (
```

```

        {
            "r2_global": float(g.group(1)),
            "rmse_global": float(g.group(2)),
            "mae_global": float(g.group(3)),
        }
        if g:
            else None
    )

    for t in targets:
        if t in resultados:
            continue
        m = patron_completo[t].search(texto)
        if m:
            resultados[t] = {
                "r2": float(m.group(1)),
                "rmse": float(m.group(2)),
                "mae": float(m.group(3)),
            }
            continue
        m2 = patron_solo_r2[t].search(texto)
        if m2:
            resultados[t] = {
                "r2": float(m2.group(1)),
                "rmse": None,
                "mae": None,
                "global": global_metrics,
            }
    }
    return resultados

```

```

def extraer_random_state(nb_json):
    """Busca el random_state usado para entrenar. Prioriza una constante de
    configuración tipo 'RANDOM_STATE = N'; si no existe, coge la primera
    aparición de 'random_state=N' que encuentre en el código."""
    codigo_completo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )
    m = re.search(r"RANDOM_STATE\s*=\s*(\d+)", codigo_completo)
    if m:
        return int(m.group(1))
    m2 = re.search(r"random_state\s*=\s*(\d+)", codigo_completo)
    if m2:
        return int(m2.group(1))
    return None

```

```

In [4]: def extraer_ruta_csv_barrido(nb_json):
    """Deduce la ruta del CSV de resultados del barrido de random_state
    leyendo el propio código del notebook (MODELS_DIR + dir_barrido +
    el nombre de archivo del .to_csv), en vez de asumir un nombre fijo tipo
    'ridge_barrido_rs.csv' -- así sirve igual para RF, XGBoost, etc."""
    codigo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )

    m_dir = re.search(r"MODELS_DIR\s*=\s*['\"]([^\"]+)['\"]", codigo)
    models_dir = m_dir.group(1) if m_dir else ""

    m_sub = re.search(
        r"dir_barrido\s*=\s*os.path.join\(\\s*MODELS_DIR\s*,\s*['\"]([^\"]+)['\"]\s*\)", codigo
    )
    if not m_sub:
        return None
    subdir = m_sub.group(1)

    m_file = re.search(
        r"\\.to_csv\(\\s*os.path.join\(\\s*dir_barrido\s*,\s*['\"]([^\"]+)['\"]\s*\)\s*\)", codigo
    )
    if not m_file:
        return None
    filename = m_file.group(1)

    return "/".join(p.strip("/") for p in [models_dir, subdir, filename] if p)

def obtener_csv_desde_git(repo_path, branch, ruta_csv):
    """Lee un CSV via 'git show', igual que obtener_notebook_desde_git pero
    parseando CSV en vez de JSON."""
    try:
        resultado = subprocess.run(
            ["git", "-C", repo_path, "show", f"{branch}:{ruta_csv}"],
            capture_output=True, text=True, check=True,
        )
    except subprocess.CalledProcessError as e:
        print(f" [ERROR] No se pudo leer {ruta_csv} en la rama {branch}: {e.stderr.strip()}")
        return None
    return pd.read_csv(StringIO(resultado.stdout))

def extraer_rango_esperado_rs(nb_json):
    """Lee 'RANDOM_STATES = range(a, b)' o 'range(a, b, paso)' del código y
    devuelve cuántas semillas se esperan, para poder comparar contra las
    filas que realmente tiene el CSV del barrido."""
    codigo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )
    m = re.search(
        r"RANDOM_STATES\s*=\s*range\(\\s*(\d+)\s*,\s*(\d+)\s*,\s*(?:,\s*(\d+)\s*)?\)", codigo
    )
    if not m:
        return None
    inicio, fin = int(m.group(1)), int(m.group(2))
    paso = int(m.group(3)) if m.group(3) else 1
    return len(range(inicio, fin, paso))

```

3.- Extracción de notebooks y por rama

```
In [5]: def analizar_notebook(repo_path, branch, filename, subdir, targets):
nb_json = obtener_notebook_desde_git(repo_path, branch, filename, subdir)
if nb_json is None:
    return None

activas, excluidas = extraer_features_activas(nb_json)
filas, columnas = extraer_shape_xtrain(nb_json)
metricas = extraer_metricas_targets(nb_json, targets)
avisos_orden = detectar_celdas_desordenadas(nb_json)
random_state = extraer_random_state(nb_json)

return {
    "n_features_lista": len(activas),
    "features_excluidas": excluidas,
    "x_train_shape": (filas, columnas),
    "coherente": (
        (columnas == len(activas)) if columnas is not None else None
    ),
    "metricas": metricas,
    "avisos_orden": avisos_orden,
    "random_state": random_state,
}

resultados_por_rama = {}

for branch in BRANCHES:
    print(f"Leyendo notebooks de la rama: {branch}...")
    resultados_por_rama[branch] = {}
    for nb_name in NOTEBOOKS:
        print(f"  {nb_name}")
        resultados_por_rama[branch][nb_name] = analizar_notebook(
            GIT_REPO_PATH,
            branch,
            nb_name,
            NOTEBOOKS_SUBDIR,
            TARGETS,
        )
    print()
```

Leyendo notebooks de la rama: model-prep-var...

reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb

4. Validaciones automaticas

Antes de comparar metricas, se comprueba automaticamente lo que hasta ahora se ha ido revisando a mano:

- ¿El numero de columnas de `X_train` coincide con el numero de variables activas en `FEATURES_AUTORIZADAS` ?
- ¿Hay celdas con `execution_count` fuera de orden (posible resultado obsoleto)?
- ¿Los valores de los targets prioritarios son sospechosamente identicos entre ambas ramas (posible notebook no re-ejecutado)?

```
In [6]: print("=" * 100)
print("VALIDACIONES AUTOMÁTICAS")
print("=" * 100)

base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    print(f"\n[{nb_name}]\n")

    rs_por_notebook = {}
    for branch in BRANCHES:
        r = resultados_por_rama.get(branch, {}).get(nb_name)
        if r is None:
            print(f" [ERROR - {branch}] No se pudo leer el notebook.")
            continue

        filas, columnas = r["x_train_shape"]
        rs_por_notebook[branch] = r.get("random_state")

        # con estas pruebas NO se eliminan variables: cualquier diferencia
        # en el número de columnas es sospechosa
        if r["coherente"] is False:
            print(
                f" [AVISO - {branch}] Se esperaban las mismas variables que en el resto "
                f"de ramas (esta prueba no elimina variables), pero X_train tiene {columnas} "
                f"columnas y FEATURES_AUTORIZADAS tiene {r['n_features_lista']} activas → revisar"
            )

        if r["avisos_orden"]:
            print(f" [AVISO - {branch}] Celdas ejecutadas fuera de orden:")
            for a in r["avisos_orden"]:
                print(f"   - {a}")

        m = re.search(r"(\d+)\s*$", branch)
        if m and r.get("random_state") is not None:
            rs_esperado = int(m.group(1))
            if r["random_state"] != rs_esperado:
                print(
                    f" [AVISO - {branch}] El nombre de la rama sugiere random_state={rs_esperado}, "
                    f"pero el notebook usa random_state={r['random_state']}"
                )

    valores_rs = set(v for v in rs_por_notebook.values() if v is not None)
    if len(valores_rs) > 1:
        print(f" [AVISO] Los notebooks de esta rama no usan todos el mismo random_state: {rs_por_notebook}")

    r_base = resultados_por_rama.get(base_branch, {}).get(nb_name)
    if r_base:
        for comp_branch in BRANCHES[1:]:
            r_comp = resultados_por_rama.get(comp_branch, {}).get(nb_name)
            if not r_comp:
                continue
            for t in TARGETS_PRIORITARIOS:
                m_base = r_base["metricas"].get(t)
                m_comp = r_comp["metricas"].get(t)
```

```

        if (
            m_base
            and m_comp
            and m_base.get("r2") == m_comp.get("r2")
        ):
            print(
                f" [AVISO] {t}: R2 IDENTICO en '{base_branch}' y '{comp_branch}' ({m_base['r2']}) "
                f"→ revisar si se re-ejecutó de verdad"
            )

=====
VALIDACIONES AUTOMÁTICAS
=====

[reg_model.ipynb]

[rf_model.ipynb]

[rf_multisalida.ipynb]

[regressorchain.ipynb]

[xgboost_model.ipynb]

[xgb_multisalida.ipynb]

[mlp_multisalida.ipynb]

[mlp_custom_loss.ipynb]

```

5- Tablas comparativas

En esta sección se generarán múltiples tablas para facilitar la comparación de resultados.

```

In [7]:
filas_tabla = []
base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    fila = {"Notebook": nb_name}

    for t in TARGETS_PRIORITARIOS:
        # Obtener R2 de la rama base
        r_base = resultados_por_rama.get(base_branch, {}).get(nb_name)
        m_base = r_base["metricas"].get(t) if r_base else None
        r2_base = m_base["r2"] if m_base else None

        fila[f"{t}_R2 ({base_branch})"] = r2_base

        # R2 y Delta para las ramas a comparar
        for branch in BRANCHES[1:]:
            r_branch = resultados_por_rama.get(branch, {}).get(nb_name)
            m_branch = r_branch["metricas"].get(t) if r_branch else None
            r2_val = m_branch["r2"] if m_branch else None

            fila[f"{t}_R2 ({branch})"] = r2_val

            delta = (
                round(r2_val - r2_base, 4)
                if (r2_val is not None and r2_base is not None)
                else None
            )
            fila[f"{t}_delta ({branch})"] = delta

        filas_tabla.append(fila)

df_comparativa = pd.DataFrame(filas_tabla)

print(f"Comparación entre {len(BRANCHES)} ramas (Base: {base_branch}): \n")
print(df_comparativa.to_string(index=False))

# Guardar resultado en CSV
os.makedirs("output/comparativas", exist_ok=True)
nombre_salida = f"comparativa_{'_vs_'}.join(BRANCHES)}.csv".replace("/", "-")
ruta_csv = f"output/comparativas/{nombre_salida}"
df_comparativa.to_csv(ruta_csv, index=False)
print(f"\nGuardado en: {ruta_csv}")

Comparación entre 1 ramas (Base: model-prep-var):

    Notebook    earthworm_shannon_z_R2 (model-prep-var)    earthworm_richness_z_R2 (model-prep-var)
reg_model.ipynb                0.2395                0.2789
rf_model.ipynb                 0.5041                0.5722
rf_multisalida.ipynb           0.4160                0.4509
regressorchain.ipynb           0.4876                0.5359
xgboost_model.ipynb            0.4946                0.5171
xgb_multisalida.ipynb          0.5375                0.5893
mlp_multisalida.ipynb          0.3583                0.4539
mlp_custom_loss.ipynb          0.3438                0.4543

Guardado en: output/comparativas/comparativa_model-prep-var.csv

```

```

In [8]:
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

def construir_df_largo_barrido():
    filas = []
    for branch in BRANCHES:
        for nb_name in NOTEBOOKS:
            res = resultados_por_rama.get(branch, {}).get(nb_name)
            if not res:
                continue
            rs = res.get("random_state")
            m_dict = res.get("metricas", {})

            # R2 global (mismo criterio que en la tabla general de la sección 5.1)
            r2_glob = None
            for t_info in m_dict.values():
                if t_info and t_info.get("global"):
                    r2_glob = t_info["global"].get("r2_global")
                    break
            if r2_glob is None and m_dict:
                r2_vals = [v["r2"] for v in m_dict.values() if v and v.get("r2") is not None]
                if r2_vals:
                    r2_glob = sum(r2_vals) / len(r2_vals)

```

```

        if r2_glob is not None:
            filas.append({"Notebook": nb_name, "branch": branch, "random_state": rs, "target": "GLOBAL", "r2": r2_glob})

        for t in TARGETS_PRIORITARIOS:
            info = m_dict.get(t)
            if info and info.get("r2") is not None:
                filas.append({"Notebook": nb_name, "branch": branch, "random_state": rs, "target": t, "r2": info["r2"]})

    return pd.DataFrame(filas)

df_barrido = construir_df_largo_barrido()
print(f"Filas recogidas: {len(df_barrido)} (esperado: hasta {len(BRANCHES)} semillas x {len(NOTEBOOKS)} notebooks x 3 targets)")
df_barrido.head()

```

Filas recogidas: 24 (esperado: hasta 1 semillas x 8 notebooks x 3 targets)

```
Out[8]:
```

	Notebook	branch	random_state	target	r2
0	reg_model.ipynb	model-prep-var	42	GLOBAL	-0.016376
1	reg_model.ipynb	model-prep-var	42	earthworm_shannon_z	0.239500
2	reg_model.ipynb	model-prep-var	42	earthworm_richness_z	0.278900
3	rf_model.ipynb	model-prep-var	42	GLOBAL	0.090362
4	rf_model.ipynb	model-prep-var	42	earthworm_shannon_z	0.504100

5.1.- Tabla de comparación general

```

In [9]:
filas_general = []
base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    fila = {"Notebook": nb_name}

    for branch in BRANCHES:
        res = resultados_por_rama.get(branch, {}).get(nb_name)

        if res:
            # Dimensiones de X_train
            shape_str = (
                f"{res['x_train_shape']}[0]}x{res['x_train_shape']}[1]}"
                if res["x_train_shape"][0]
                else "N/A"
            )
            fila[f"N_Vars ({branch})"] = res["n_features_lista"]
            fila[f"Shape ({branch})"] = shape_str

            # Extracción de métricas globales
            m_dict = res.get("metricas", {})
            r2_glob, rmse_glob, mae_glob = None, None, None

            # 1. Intentar obtener 'global' si el notebook es multitarget
            for t_info in m_dict.values():
                if t_info and t_info.get("global"):
                    r2_glob = t_info["global"].get("r2_global")
                    rmse_glob = t_info["global"].get("rmse_global")
                    mae_glob = t_info["global"].get("mae_global")
                    break

            # 2. Si es single-target, promediar el R2 de los targets evaluados
            if r2_glob is None and m_dict:
                r2_vals = [
                    v["r2"]
                    for v in m_dict.values()
                    if v and v.get("r2") is not None
                ]
                if r2_vals:
                    r2_glob = round(sum(r2_vals) / len(r2_vals), 4)

            fila[f"R2_Global ({branch})"] = r2_glob
            if rmse_glob is not None:
                fila[f"RMSE_Global ({branch})"] = rmse_glob
            if mae_glob is not None:
                fila[f"MAE_Global ({branch})"] = mae_glob

            # Calcular Delta R2 Global respecto al baseline
            if branch != base_branch:
                r2_base = fila.get(f"R2_Global ({base_branch})")
                fila[f"Delta_R2_Global ({branch})"] = (
                    round(r2_glob - r2_base, 4)
                    if (r2_glob is not None and r2_base is not None)
                    else None
                )
            else:
                fila[f"N_Vars ({branch})"] = "Error"
                fila[f"R2_Global ({branch})"] = None

        filas_general.append(fila)

df_general = pd.DataFrame(filas_general)

print("=" * 100)
print(f"1. TABLA COMPARATIVA GENERAL (Base: {base_branch})")
print("=" * 100)
print(df_general.to_markdown(index=False))

# Guardar CSV
os.makedirs("output/comparativas", exist_ok=True)
ruta_csv_gen = f"output/comparativas/general_{'_vs_'}.join(BRANCHES)}.csv"
df_general.to_csv(ruta_csv_gen, index=False)
print(f"\nGuardado en: {ruta_csv_gen}")

```

1. TABLA COMPARATIVA GENERAL (Base: model-prep-var)

Notebook	N_Vars (model-prep-var)	Shape (model-prep-var)	R2_Global (model-prep-var)	RMSE_Global (model-prep-var)	MAE_Global (model-prep-var)
reg_model.ipynb	34	299x34	-0.0164	nan	
rf_model.ipynb	34	299x34	0.0904	nan	
rf_multisalida.ipynb	34	299x34	0.0964	0.9198	
regressorchain.ipynb	34	299x34	0.0809	0.9184	
xgboost_model.ipynb	34	299x34	0.067	nan	
xgb_multisalida.ipynb	34	299x34	0.0835	nan	
mlp_multisalida.ipynb	34	299x34	0.0017	0.942	
mlp_custom_loss.ipynb	34	299x34	-0.0978	0.9714	

Guardado en: output/comparativas/general_model-prep-var.csv

5.2.- Comparación por targets

```
In [10]: from collections import Counter
from IPython.display import display, HTML

base_branch = BRANCHES[0]
tablas_por_target = {}
html_partes = []

def obtener_label_rama(branch):
    """Etiqueta la rama con el random_state usado, tomando el valor más
    frecuente entre los 8 notebooks (por si alguno estuviera desincronizado)."""
    conteos = Counter()
    for nb_name in NOTEBOOKS:
        res = resultados_por_rama.get(branch, {}).get(nb_name)
        if res and res.get("random_state") is not None:
            conteos[res["random_state"]] += 1
    if not conteos:
        return branch
    rs_comun = conteos.most_common(1)[0][0]
    return f"rs={rs_comun}"

LABELS_RAMAS = {branch: obtener_label_rama(branch) for branch in BRANCHES}

os.makedirs("output/comparativas", exist_ok=True)

for target in TARGETS:
    filas_target = []

    for nb_name in NOTEBOOKS:
        fila = {"Notebook": nb_name}

        for branch in BRANCHES:
            res = resultados_por_rama.get(branch, {}).get(nb_name)
            m = (
                res["metricas"].get(target)
                if (res and res.get("metricas"))
                else None
            )
            fila[f"R2 ({LABELS_RAMAS[branch]})"] = m["r2"] if m else None

        filas_target.append(fila)

    df_target = pd.DataFrame(filas_target)

    cols_r2 = [c for c in df_target.columns if c != "Notebook"]
    fila_media = {"Notebook": "MEDIA GLOBAL"}
    for col in cols_r2:
        fila_media[col] = df_target[col].mean(skipna=True)
    df_target = pd.concat([df_target, pd.DataFrame([fila_media])], ignore_index=True)

    tablas_por_target[target] = df_target

    html_target = df_target.to_html(
        index=False, na_rep="--", float_format=lambda x: f"{x:.4f}"
    )

    print("=" * 100)
    print(f"#2. TABLA COMPARATIVA POR TARGET: {target} (Base: {LABELS_RAMAS[base_branch]})")
    print("=" * 100)
    display(HTML(html_target))

    ruta_html_target = f"output/comparativas/por_target_{target}_{'_'}.join(BRANCHES)}.html"
    with open(ruta_html_target, "w", encoding="utf-8") as f:
        f.write(html_target)
    print(f"Guardado en: {ruta_html_target}\n")

    html_partes.append(f"<h2>{target}</h2>\n{html_target}")

html_final = (
    "<html><head><meta charset='utf-8'>"
    "<style>table{border-collapse:collapse;margin-bottom:30px;}"
    "<th,td{border:1px solid #ccc;padding:4px 8px;text-align:right;}"
    "<th{background:#f0f0f0;}"
    "<td{border:1px solid #ccc;padding:4px 8px;text-align:right;}"
    "</th><td>{text-align:left;}"
    "</td></tr></thead><tbody>"
    f"<tr><td>Comparativa por targets (Base: {LABELS_RAMAS[base_branch]})</td></tr>"
    + "\n".join(html_partes)
    + "</tbody></html>"
)

ruta_html_final = f"output/comparativas/todas_las_tablas_{'_'}.join(BRANCHES)}.html"
with open(ruta_html_final, "w", encoding="utf-8") as f:
    f.write(html_final)

print("=" * 100)
print(f"#Export final con todas las tablas guardado en: {ruta_html_final}")
print("=" * 100)
```

=====

2. TABLA COMPARATIVA POR TARGET: nematode_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0112
rf_model.ipynb	0.1693
rf_multisalida.ipynb	0.1545
regressorchain.ipynb	0.1467
xgboost_model.ipynb	0.1637
xgb_multisalida.ipynb	0.2228
mlp_multisalida.ipynb	0.0656
mlp_custom_loss.ipynb	0.0520
MEDIA GLOBAL	0.1232

Guardado en: output/comparativas/por_target_nematode_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: macro_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.1257
rf_model.ipynb	0.3199
rf_multisalida.ipynb	0.2000
regressorchain.ipynb	0.2804
xgboost_model.ipynb	0.2601
xgb_multisalida.ipynb	0.3771
mlp_multisalida.ipynb	0.2101
mlp_custom_loss.ipynb	0.1817
MEDIA GLOBAL	0.2444

Guardado en: output/comparativas/por_target_macro_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: earthworm_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.2395
rf_model.ipynb	0.5041
rf_multisalida.ipynb	0.4160
regressorchain.ipynb	0.4876
xgboost_model.ipynb	0.4946
xgb_multisalida.ipynb	0.5375
mlp_multisalida.ipynb	0.3583
mlp_custom_loss.ipynb	0.3438
MEDIA GLOBAL	0.4227

Guardado en: output/comparativas/por_target_earthworm_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: orib_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.1137
rf_model.ipynb	0.1794
rf_multisalida.ipynb	0.1693
regressorchain.ipynb	0.2069
xgboost_model.ipynb	0.1147
xgb_multisalida.ipynb	0.2275
mlp_multisalida.ipynb	0.2277
mlp_custom_loss.ipynb	0.1990
MEDIA GLOBAL	0.1798

Guardado en: output/comparativas/por_target_orib_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: meso_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.2499
rf_model.ipynb	-0.1731
rf_multisalida.ipynb	-0.1178
regressorchain.ipynb	-0.1640
xgboost_model.ipynb	-0.1868
xgb_multisalida.ipynb	-0.2861
mlp_multisalida.ipynb	-0.2084
mlp_custom_loss.ipynb	-0.4116
MEDIA GLOBAL	-0.2247

Guardado en: output/comparativas/por_target_meso_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: coll_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.4998
rf_model.ipynb	-0.3489
rf_multisalida.ipynb	-0.1953
regressorchain.ipynb	-0.6087
xgboost_model.ipynb	-0.3312
xgb_multisalida.ipynb	-0.3229
mlp_multisalida.ipynb	-0.7628
mlp_custom_loss.ipynb	-1.2843
MEDIA GLOBAL	-0.5442

Guardado en: output/comparativas/por_target_coll_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: bac_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.0812
rf_model.ipynb	0.0077
rf_multisalida.ipynb	0.0290
regressorchain.ipynb	0.0288
xgboost_model.ipynb	0.0092
xgb_multisalida.ipynb	-0.0151
mlp_multisalida.ipynb	0.0032
mlp_custom_loss.ipynb	-0.0494
MEDIA GLOBAL	-0.0085

Guardado en: output/comparativas/por_target_bac_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: fun_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.0161
rf_model.ipynb	-0.0455
rf_multisalida.ipynb	-0.0127
regressorchain.ipynb	-0.0219
xgboost_model.ipynb	-0.0206
xgb_multisalida.ipynb	-0.0502
mlp_multisalida.ipynb	-0.0615
mlp_custom_loss.ipynb	-0.0859
MEDIA GLOBAL	-0.0393

Guardado en: output/comparativas/por_target_fun_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: euk_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0407
rf_model.ipynb	0.1683
rf_multisalida.ipynb	0.1378
regressorchain.ipynb	0.1624
xgboost_model.ipynb	0.0912
xgb_multisalida.ipynb	0.0886
mlp_multisalida.ipynb	0.1323
mlp_custom_loss.ipynb	0.0952
MEDIA GLOBAL	0.1146

Guardado en: output/comparativas/por_target_euk_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: oomy_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0857
rf_model.ipynb	0.0832
rf_multisalida.ipynb	0.1252
regressorchain.ipynb	0.0913
xgboost_model.ipynb	0.0965
xgb_multisalida.ipynb	0.0620
mlp_multisalida.ipynb	0.2119
mlp_custom_loss.ipynb	0.2024
MEDIA GLOBAL	0.1198

Guardado en: output/comparativas/por_target_oomy_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: cerc_shannon_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0155
rf_model.ipynb	-0.0318
rf_multisalida.ipynb	-0.0041
regressorchain.ipynb	-0.0006
xgboost_model.ipynb	-0.0414
xgb_multisalida.ipynb	-0.0330
mlp_multisalida.ipynb	-0.0668
mlp_custom_loss.ipynb	-0.1223
MEDIA GLOBAL	-0.0356

Guardado en: output/comparativas/por_target_cerc_shannon_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: macro_order_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.1521
rf_model.ipynb	0.3417
rf_multisalida.ipynb	0.2532
regressorchain.ipynb	0.4069
xgboost_model.ipynb	0.3082
xgb_multisalida.ipynb	0.4013
mlp_multisalida.ipynb	0.2683
mlp_custom_loss.ipynb	0.2420
MEDIA GLOBAL	0.2967

Guardado en: output/comparativas/por_target_macro_order_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: earthworm_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.2789
rf_model.ipynb	0.5722
rf_multisalida.ipynb	0.4509
regressorchain.ipynb	0.5359
xgboost_model.ipynb	0.5171
xgb_multisalida.ipynb	0.5893
mlp_multisalida.ipynb	0.4539
mlp_custom_loss.ipynb	0.4543
MEDIA GLOBAL	0.4816

Guardado en: output/comparativas/por_target_earthworm_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: orib_species_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0921
rf_model.ipynb	0.2034
rf_multisalida.ipynb	0.2386
regressorchain.ipynb	0.2026
xgboost_model.ipynb	0.1259
xgb_multisalida.ipynb	0.2800
mlp_multisalida.ipynb	0.2700
mlp_custom_loss.ipynb	0.2477
MEDIA GLOBAL	0.2075

Guardado en: output/comparativas/por_target_orib_species_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: meso_species_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.0959
rf_model.ipynb	-0.0438
rf_multisalida.ipynb	-0.0255
regressorchain.ipynb	-0.0400
xgboost_model.ipynb	-0.0592
xgb_multisalida.ipynb	-0.1146
mlp_multisalida.ipynb	-0.0625
mlp_custom_loss.ipynb	-0.2028
MEDIA GLOBAL	-0.0805

Guardado en: output/comparativas/por_target_meso_species_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: coll_species_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.7341
rf_model.ipynb	-0.5484
rf_multisalida.ipynb	-0.2920
regressorchain.ipynb	-0.5799
xgboost_model.ipynb	-0.5803
xgb_multisalida.ipynb	-0.6465
mlp_multisalida.ipynb	-1.3183
mlp_custom_loss.ipynb	-2.1265
MEDIA GLOBAL	-0.8532

Guardado en: output/comparativas/por_target_coll_species_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: bac_asv_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	-0.0761
rf_model.ipynb	0.0042
rf_multisalida.ipynb	0.0044
regressorchain.ipynb	-0.0052
xgboost_model.ipynb	0.0014
xgb_multisalida.ipynb	0.0155
mlp_multisalida.ipynb	-0.0443
mlp_custom_loss.ipynb	-0.0900
MEDIA GLOBAL	-0.0238

Guardado en: output/comparativas/por_target_bac_asv_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: fun_asv_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0087
rf_model.ipynb	-0.0256
rf_multisalida.ipynb	-0.0068
regressorchain.ipynb	-0.0186
xgboost_model.ipynb	-0.0242
xgb_multisalida.ipynb	-0.0662
mlp_multisalida.ipynb	0.0101
mlp_custom_loss.ipynb	0.0043
MEDIA GLOBAL	-0.0148

Guardado en: output/comparativas/por_target_fun_asv_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: euk_asv_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.0400
rf_model.ipynb	0.2520
rf_multisalida.ipynb	0.1903
regressorchain.ipynb	0.2690
xgboost_model.ipynb	0.1671
xgb_multisalida.ipynb	0.2233
mlp_multisalida.ipynb	0.2201
mlp_custom_loss.ipynb	0.2118
MEDIA GLOBAL	0.1967

Guardado en: output/comparativas/por_target_euk_asv_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: oomy_asv_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.1035
rf_model.ipynb	0.1843
rf_multisalida.ipynb	0.1917
regressorchain.ipynb	0.2019
xgboost_model.ipynb	0.1776
xgb_multisalida.ipynb	0.1650
mlp_multisalida.ipynb	0.0911
mlp_custom_loss.ipynb	0.0849
MEDIA GLOBAL	0.1500

Guardado en: output/comparativas/por_target_oomy_asv_richness_z_model-prep-var.html

=====

2. TABLA COMPARATIVA POR TARGET: cerc_asv_richness_z (Base: rs=42)

=====

Notebook	R2 (rs=42)
reg_model.ipynb	0.1019
rf_model.ipynb	0.1250
rf_multisalida.ipynb	0.1171
regressorchain.ipynb	0.1171
xgboost_model.ipynb	0.1227
xgb_multisalida.ipynb	0.0983
mlp_multisalida.ipynb	0.0380
mlp_custom_loss.ipynb	-0.0008
MEDIA GLOBAL	0.0899

Guardado en: output/comparativas/por_target_cerc_asv_richness_z_model-prep-var.html

Export final con todas las tablas guardado en: output/comparativas/todas_las_tablas_model-prep-var.html

7. Ranking de modelos

Se calcula directamente a partir de las métricas ya extraídas. Rank 1 = mejor modelo (mayor R²) en esa rama/target.

```
In [11]: print("=" * 100)
print("7.1.- RANKING POR RAMA (R2 global, 1 = mejor modelo)")
print("=" * 100)

r2_global_cols = [c for c in df_general.columns if c.startswith("R2_Global (")]
ranking_global = df_general.set_index("Notebook")[r2_global_cols].rank(ascending=False, axis=0)
ranking_global.columns = [c.replace("R2_Global (", "").rstrip(")") for c in ranking_global.columns]

print(ranking_global.to_string())

resumen_ranking = pd.DataFrame({
    "Ranking medio": ranking_global.mean(axis=1).round(2),
    "Desviacion (estabilidad)": ranking_global.std(axis=1).round(2),
}).sort_values("Ranking medio")

print("\n" + "=" * 100)
print("7.2.- RANKING MEDIO Y ESTABILIDAD ENTRE RAMAS")
print("=" * 100)
print(resumen_ranking.to_string())

os.makedirs("output/comparativas", exist_ok=True)
ruta_ranking = f"output/comparativas/ranking_global_{'_vs_'}.join(BRANCHES)}.csv"
ranking_global.to_csv(ruta_ranking)
print(f"\nGuardado en: {ruta_ranking}")

=====
7.1.- RANKING POR RAMA (R2 global, 1 = mejor modelo)
=====
model-prep-var

Notebook
reg_model.ipynb      7.0
rf_model.ipynb       2.0
rf_multisalida.ipynb 1.0
regressorchain.ipynb 4.0
xgboost_model.ipynb  5.0
xgb_multisalida.ipynb 3.0
mlp_multisalida.ipynb 6.0
mlp_custom_loss.ipynb 8.0

=====
7.2.- RANKING MEDIO Y ESTABILIDAD ENTRE RAMAS
=====
Ranking medio  Desviacion (estabilidad)

Notebook
rf_multisalida.ipynb      1.0      NaN
rf_model.ipynb            2.0      NaN
xgb_multisalida.ipynb     3.0      NaN
regressorchain.ipynb     4.0      NaN
xgboost_model.ipynb      5.0      NaN
mlp_multisalida.ipynb     6.0      NaN
reg_model.ipynb           7.0      NaN
mlp_custom_loss.ipynb    8.0      NaN

Guardado en: output/comparativas/ranking_global_model-prep-var.csv
```

8. Barrido de random_state

Dos ramas, cada una entrena un modelo por semilla dentro de un rango y guarda el resultado en un CSV propio (MODELS_DIR/barrido_rs_<modelo>/<...>.csv). Aquí se lee ese CSV directamente via `git show`, no se parsea el notebook.

```
In [12]: barrido_por_rama = {}

for branch in BRANCHES_BARRIDO:
    print(f"Leyendo barridos de la rama: {branch}...")
    barrido_por_rama[branch] = {}
    conteos_rama = {}

    for nb_name in NOTEBOOKS:
        nb_json = obtener_notebook_desde_git(GIT_REPO_PATH, branch, nb_name, NOTEBOOKS_SUBDIR)
        if nb_json is None:
            continue
        ruta_csv = extraer_ruta_csv_barrido(nb_json)
        if ruta_csv is None:
            print(f" [AVISO] {nb_name}: no se encontró la celda de barrido (¿no tiene Prueba 3 todavía?)")
            continue

        df_csv = obtener_csv_desde_git(GIT_REPO_PATH, branch, ruta_csv)
        if df_csv is None:
            barrido_por_rama[branch][nb_name] = None
            continue

        n_real = len(df_csv)
        n_esperado = extraer_rango_esperado_rs(nb_json)
        conteos_rama[nb_name] = n_real
```

```

etiqueta_check = ""
if n_esperado is not None and n_real != n_esperado:
    etiqueta_check = f" [AVISO] se esperaban {n_esperado} semillas (RANDOM_STATES en el código), CSV tiene {n_real}"

print(f" {nb_name}: {ruta_csv} → {n_real} semillas{etiqueta_check}")
barrido_por_rama[branch][nb_name] = df_csv

# dentro de la misma rama, todos los notebooks deberían barrer el mismo
# rango de semillas → si uno se queda corto respecto al resto, avisar
if len(set(conteos_rama.values())) > 1:
    print(f" [AVISO] No todos los notebooks de '{branch}' tienen el mismo número de semillas: {conteos_rama}")
print()

```

Leyendo barridos de la rama: model-prep-rs-group...

```

reg_model.ipynb: output/models/reg/barrido_rs_ridge/ridge_barrido_rs.csv → 101 semillas
rf_model.ipynb: output/models/rf/barrido_rs_rf/rf_barrido_rs.csv → 101 semillas
rf_multisalida.ipynb: output/models/rf_multi/barrido_rs_rf_multi/rf_multi_barrido_rs.csv → 101 semillas
regressorchain.ipynb: output/models/chain/barrido_rs_chain/chain_barrido_rs.csv → 101 semillas
xgboost_model.ipynb: output/models/xgb/barrido_rs_xgb/xgb_barrido_rs.csv → 101 semillas
xgb_multisalida.ipynb: output/models/xgb_multi/barrido_rs_xgb_multi/xgb_multi_barrido_rs.csv → 101 semillas
mlp_multisalida.ipynb: output/models/mlp/barrido_rs_mlp/mlp_barrido_rs.csv → 101 semillas
mlp_custom_loss.ipynb: output/models/mlp_custom/barrido_rs_mlp_custom/mlp_custom_barrido_rs.csv → 101 semillas

```

Leyendo barridos de la rama: model-prep-rs-group-1...

```

reg_model.ipynb: output/models/reg/barrido_rs_ridge/ridge_barrido_rs.csv → 491 semillas
rf_model.ipynb: output/models/rf/barrido_rs_rf/rf_barrido_rs.csv → 491 semillas
rf_multisalida.ipynb: output/models/rf_multi/barrido_rs_rf_multi/rf_multi_barrido_rs.csv → 491 semillas
regressorchain.ipynb: output/models/chain/barrido_rs_chain/chain_barrido_rs.csv → 491 semillas
xgboost_model.ipynb: output/models/xgb/barrido_rs_xgb/xgb_barrido_rs.csv → 491 semillas
xgb_multisalida.ipynb: output/models/xgb_multi/barrido_rs_xgb_multi/xgb_multi_barrido_rs.csv → 491 semillas
mlp_multisalida.ipynb: output/models/mlp/barrido_rs_mlp/mlp_barrido_rs.csv → 491 semillas
mlp_custom_loss.ipynb: output/models/mlp_custom/barrido_rs_mlp_custom/mlp_custom_barrido_rs.csv → 491 semillas

```

```

In [13]: def construir_df_largo_desde_csv():
    filas = []
    for branch, notebooks_dict in barrido_por_rama.items():
        for nb_name, df_csv in notebooks_dict.items():
            if df_csv is None or "random_state" not in df_csv.columns:
                continue
            r2_cols = [c for c in df_csv.columns if c.endswith("_r2")]
            for _, row in df_csv.iterrows():
                rs = row["random_state"]
                filas.append({
                    "Notebook": nb_name, "branch": branch, "random_state": rs,
                    "target": "GLOBAL", "r2": row[r2_cols].mean(),
                })
            for t in TARGETS_PRIORITARIOS:
                col = f"{t}_r2"
                if col in df_csv.columns:
                    filas.append({
                        "Notebook": nb_name, "branch": branch, "random_state": rs,
                        "target": t, "r2": row[col],
                    })
    return pd.DataFrame(filas)

df_barrido = construir_df_largo_desde_csv()
print(f"Filas recogidas: {len(df_barrido)}")
df_barrido.head()

```

Filas recogidas: 14208

```

Out[13]:

```

	Notebook	branch	random_state	target	r2
0	reg_model.ipynb	model-prep-rs-group	0.0	GLOBAL	-0.016377
1	reg_model.ipynb	model-prep-rs-group	0.0	earthworm_shannon_z	0.239544
2	reg_model.ipynb	model-prep-rs-group	0.0	earthworm_richness_z	0.278931
3	reg_model.ipynb	model-prep-rs-group	1.0	GLOBAL	-0.016377
4	reg_model.ipynb	model-prep-rs-group	1.0	earthworm_shannon_z	0.239544

```

In [14]: print("=" * 100)
print("8.1.- ESTADÍSTICA DEL BARRIDO (media ± desviación típica sobre las semillas, por rama)")
print("=" * 100)

os.makedirs("output/comparativas", exist_ok=True)

for target in ["GLOBAL"] + TARGETS_PRIORITARIOS:
    sub = df_barrido[df_barrido["target"] == target]
    resumen = sub.groupby(["Notebook", "branch"])["r2"].agg(
        media="mean", desviacion="std", minimo="min", maximo="max", n="count"
    ).round(4).sort_values("media", ascending=False)
    print(f"\n--- {target} ---")
    print(resumen.to_string())
    resumen.to_csv(f"output/comparativas/barrido_resumen_{target}.csv")

```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

=====

8.1.- ESTADÍSTICA DEL BARRIDO (media ± desviación típica sobre las semillas, por rama)

=====

```

--- GLOBAL ---

```

		media	desviacion	minimo	maximo	n
Notebook	branch					
rf_multisalida.ipynb	model-prep-rs-group	0.0953	0.0030	0.0869	0.1032	101
	model-prep-rs-group-1	0.0953	0.0033	0.0839	0.1053	491
rf_model.ipynb	model-prep-rs-group	0.0892	0.0042	0.0768	0.1029	491
	model-prep-rs-group	0.0890	0.0040	0.0796	0.0996	101
xgb_multisalida.ipynb	model-prep-rs-group	0.0786	0.0049	0.0639	0.0949	491
	model-prep-rs-group	0.0780	0.0055	0.0635	0.0880	101
regressorchain.ipynb	model-prep-rs-group	0.0643	0.0116	0.0341	0.0988	491
	model-prep-rs-group	0.0635	0.0104	0.0402	0.0976	101
reg_model.ipynb	model-prep-rs-group	-0.0164	0.0000	-0.0164	-0.0164	491
	model-prep-rs-group	-0.0164	0.0000	-0.0164	-0.0164	101
mlp_multisalida.ipynb	model-prep-rs-group	-0.0271	0.0342	-0.1196	0.0606	101
mlp_custom_loss.ipynb	model-prep-rs-group	-0.0495	0.0312	-0.1351	0.0185	101
	model-prep-rs-group	-0.0588	0.0348	-0.1737	0.0461	491
xgboost_model.ipynb	model-prep-rs-group	-0.0636	0.0000	-0.0636	-0.0636	101
	model-prep-rs-group	-0.0636	0.0000	-0.0636	-0.0636	491
mlp_multisalida.ipynb	model-prep-rs-group	-0.0678	0.0516	-0.2609	0.0568	491

```

--- earthworm_shannon_z ---

```

		media	desviacion	minimo	maximo	n
Notebook	branch					
xgb_multisalida.ipynb	model-prep-rs-group	0.5343	0.0108	0.5032	0.5612	101
	model-prep-rs-group	0.5330	0.0111	0.4981	0.5688	491
rf_model.ipynb	model-prep-rs-group	0.5018	0.0104	0.4718	0.5302	491
	model-prep-rs-group	0.4990	0.0091	0.4720	0.5195	101
regressorchain.ipynb	model-prep-rs-group	0.4792	0.0157	0.4307	0.5209	491
	model-prep-rs-group	0.4789	0.0144	0.4445	0.5134	101
rf_multisalida.ipynb	model-prep-rs-group	0.4142	0.0092	0.3876	0.4433	491
	model-prep-rs-group	0.4140	0.0091	0.3802	0.4350	101
xgboost_model.ipynb	model-prep-rs-group	0.3898	0.0000	0.3898	0.3898	101
	model-prep-rs-group	0.3898	0.0000	0.3898	0.3898	491
mlp_multisalida.ipynb	model-prep-rs-group	0.3653	0.0404	0.1936	0.4770	491
mlp_custom_loss.ipynb	model-prep-rs-group	0.3464	0.0446	0.1304	0.4571	491
mlp_multisalida.ipynb	model-prep-rs-group	0.3452	0.0361	0.2589	0.4198	101
mlp_custom_loss.ipynb	model-prep-rs-group	0.3169	0.0466	0.1959	0.4450	101
reg_model.ipynb	model-prep-rs-group	0.2395	0.0000	0.2395	0.2395	101
	model-prep-rs-group	0.2395	0.0000	0.2395	0.2395	491

```

--- earthworm_richness_z ---

```

		media	desviacion	minimo	maximo	n
Notebook	branch					
xgb_multisalida.ipynb	model-prep-rs-group	0.5867	0.0127	0.5505	0.6251	491
	model-prep-rs-group	0.5866	0.0112	0.5597	0.6106	101
rf_model.ipynb	model-prep-rs-group	0.5643	0.0103	0.5426	0.5873	101
	model-prep-rs-group	0.5635	0.0106	0.5353	0.5900	491
regressorchain.ipynb	model-prep-rs-group	0.5279	0.0121	0.4924	0.5664	491
	model-prep-rs-group	0.5270	0.0118	0.5046	0.5590	101
xgboost_model.ipynb	model-prep-rs-group	0.4895	0.0000	0.4895	0.4895	491
	model-prep-rs-group	0.4895	0.0000	0.4895	0.4895	101
mlp_multisalida.ipynb	model-prep-rs-group	0.4592	0.0455	0.2597	0.5618	491
rf_multisalida.ipynb	model-prep-rs-group	0.4494	0.0086	0.4200	0.4808	491
	model-prep-rs-group	0.4493	0.0087	0.4178	0.4733	101
mlp_custom_loss.ipynb	model-prep-rs-group	0.4491	0.0402	0.2841	0.5469	491
mlp_multisalida.ipynb	model-prep-rs-group	0.4354	0.0325	0.3511	0.5132	101
mlp_custom_loss.ipynb	model-prep-rs-group	0.4185	0.0410	0.2991	0.5315	101
reg_model.ipynb	model-prep-rs-group	0.2789	0.0000	0.2789	0.2789	101
	model-prep-rs-group	0.2789	0.0000	0.2789	0.2789	491

```

In [15]: import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

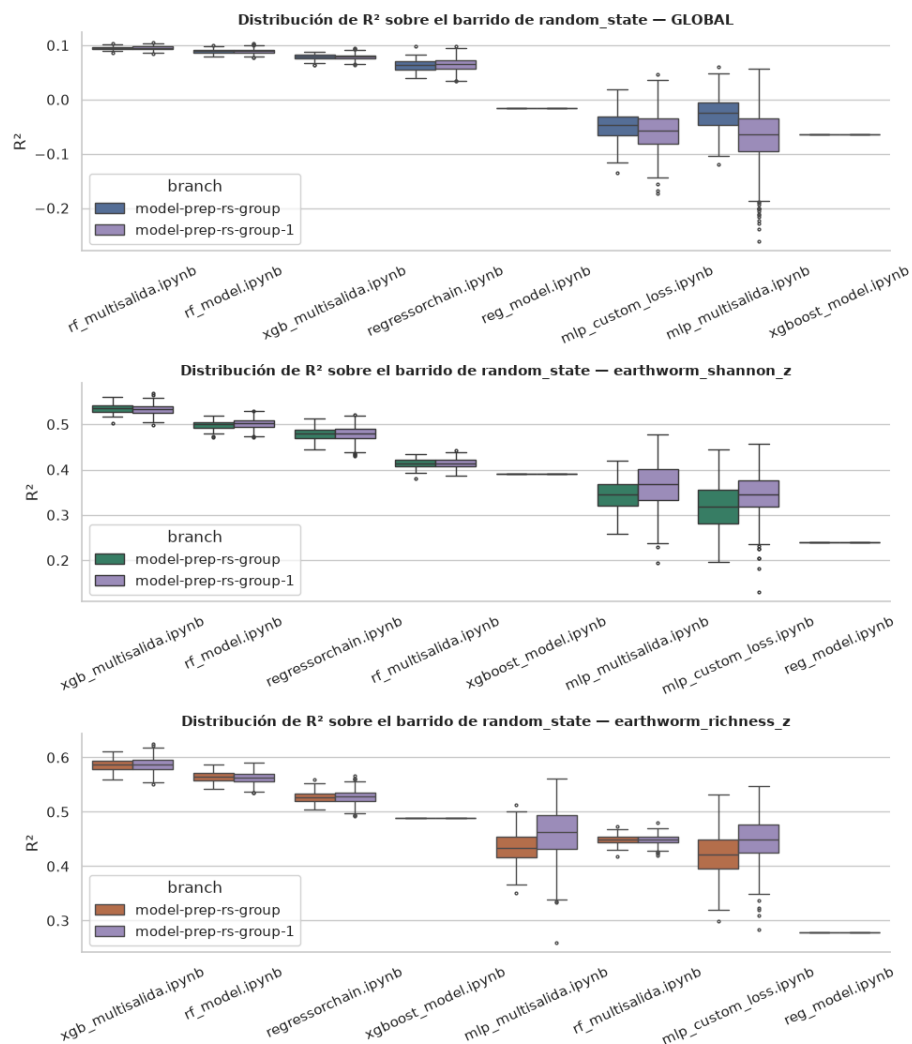
TARGET_COLORS = {"GLOBAL": "#4A6FAS", "earthworm_shannon_z": "#2E8B6B", "earthworm_richness_z": "#C9683A"}

targets_a_graficar = ["GLOBAL"] + TARGETS_PRIORITARIOS
fig, axes = plt.subplots(len(targets_a_graficar), 1, figsize=(10, 3.8 * len(targets_a_graficar)), squeeze=False)

for ax, target in zip(axes[:, 0], targets_a_graficar):
    sub = df_barrido[df_barrido["target"] == target]
    orden = sub.groupby("Notebook")["r2"].mean().sort_values(ascending=False).index
    sns.boxplot(data=sub, x="Notebook", y="r2", hue="branch", order=orden, ax=ax, fliersize=2,
                palette=TARGET_COLORS.get(target, "#4A6FAS"), "#9B85C4"])
    ax.set_title(f"Distribución de R² sobre el barrido de random_state - {target}", fontsize=11, fontweight="bold")
    ax.set_xlabel("")
    ax.set_ylabel("R²")
    ax.tick_params(axis="x", rotation=25)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.tight_layout()
fig.savefig("output/comparativas/barrido_boxplot.png", dpi=160, bbox_inches="tight")
plt.show()

```



```
In [16]: print("=" * 100)
print("8.3.- RANKING MEDIO DE MODELOS A LO LARGO DE LAS SEMILLAS, POR RAMA")
print("=" * 100)

for branch in BRANCHES_BARRIDO:
    sub = df_barrido[(df_barrido["target"] == "GLOBAL") & (df_barrido["branch"] == branch)]
    pivot = sub.pivot(index="random_state", columns="Notebook", values="r2")
    ranking = pivot.rank(ascending=False, axis=1)
    resumen_rank = pd.DataFrame({
        "Ranking medio": ranking.mean().round(2),
        "Desviacion": ranking.std().round(2),
    }).sort_values("Ranking medio")
    print(f"\n--- {branch} ({len(pivot)} semillas) ---")
    print(resumen_rank.to_string())
```

```
=====
8.3.- RANKING MEDIO DE MODELOS A LO LARGO DE LAS SEMILLAS, POR RAMA
=====

--- model-prep-rs-group (101 semillas) ---
Ranking medio Desviacion
Notebook
rf_multisalida.ipynb      1.11      0.34
rf_model.ipynb           1.96      0.37
xgb_multisalida.ipynb     3.04      0.37
regressorchain.ipynb     3.90      0.41
reg_model.ipynb          5.48      0.63
mlp_multisalida.ipynb     6.02      0.99
mlp_custom_loss.ipynb     6.87      0.87
xgboost_model.ipynb      7.62      0.51

--- model-prep-rs-group-1 (491 semillas) ---
Ranking medio Desviacion
Notebook
rf_multisalida.ipynb      1.12      0.34
rf_model.ipynb           1.95      0.42
xgb_multisalida.ipynb     3.08      0.41
regressorchain.ipynb     3.85      0.42
reg_model.ipynb          5.25      0.51
mlp_custom_loss.ipynb     6.77      0.88
mlp_multisalida.ipynb     6.92      1.09
xgboost_model.ipynb      7.06      0.82
```

11.3.5. Notebook para la comparación de variables (Prueba I)

SOB4ES - Comparador de variables

CRISP-ML(Q) - Fase 4: ranking de variables candidatas a eliminar

Este notebook lee **todos** los archivos `variables_menos_relevantes_*.csv` que cada notebook de modelado exporta (uno por modelo, con sus 10 variables menos relevantes), los cruza, y genera un **ranking general de candidatas a eliminar** del conjunto de features: las variables que se repiten como "poco relevantes" en mas modelos, con metodos de importancia distintos (SHAP, permutacion, coeficientes, gain), son la señal mas solida.

Formatos de archivo que reconoce

Cada notebook de modelado exporta su propio `variables_menos_relevantes_{modelo}.csv` con alguno de estos formatos, y el notebook detecta automaticamente cual es:

1. **Formato estandar** — columna `variable` + una o mas columnas numericas de importancia, ya ordenado de menor a mayor importancia (la primera fila es la variable menos relevante). Es el que usan RF, XGBoost, RF/XGB Multisalida, RegressorChain y los MLP.
2. **Formato Ridge** — columna `GLOBAL_MENOS_RELEVANTES` con el bottom-10 global ya calculado (mas columnas por target, que aqui no se usan).
3. **Formato roto** — solo una columna de valores numericos, sin nombres de variable (p. ej. si se exporto con `to_csv(index=False)` sobre una Series sin resetear el indice). Estos archivos **no se pueden usar** — se detectan y se listan aparte, sin bloquear el resto del analisis.

Como identifica el modelo cada archivo

El nombre de cada modelo se deriva del propio nombre de archivo (todo lo que va despues de `variables_menos_relevantes_` o `variables_menos_revelantes_`, por si hay alguna errata). Si aparecen nombres de archivo ambiguos o no reconocidos, se muestran tal cual aparecen para que se puedan revisar a mano.

```
In [1]: import os
import glob
import re
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

# Directorio donde estan los variables_menos_relevantes_*.csv de cada notebook.
# Por defecto, el mismo directorio que este notebook.
DATA_DIR = './output/var/'

PALETTE = {
    'blue': '#7FA6C9', 'blue_light': '#AAD0F1', 'green': '#7FC8A9', 'green_light': '#BEE6D3',
    'purple': '#A98FC7', 'purple_light': '#D6C7EA', 'gray': '#9B9B9B', 'gray_dark': '#5C5C5C',
    'red': '#D98866',
}

PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

# Nombres "bonitos" para los sufijos de archivo mas habituales (ajustar si cambian
# las convenciones de nombres en el proyecto). Si un archivo no aparece aqui, se usa
# su nombre de archivo tal cual, para que sea facil detectarlo y revisarlo a mano.
NOMBRES_MODELO = {
    'rf': 'Random Forest',
    'rf_multisalida': 'RF Multisalida',
    'xgboost': 'XGBoost',
    'xgb': 'XGB Multisalida',
    'ridge': 'Ridge',
    # 'cadena_shap': 'RegressorChain',
    'cadena': 'RegressorChain',
    'mlp_multisalida': 'MLP Multisalida',
    'mlp_custom': 'MLP Perdida Personalizada',
    # 'mlp_permutation': 'MLP (variante sin confirmar - revisar origen)',
}

print('Librerias y configuracion cargadas correctamente...')
```

Librerias y configuracion cargadas correctamente...

1.- Descubrimiento y carga de archivos

Se buscan todos los `variables_menos_relevantes_*.csv` (y la variante con la errata `variables_menos_revelantes_*.csv`) en `DATA_DIR`, se detecta el formato de cada uno, y se extrae su bottom-10 (lista de variables menos relevantes, ordenada de la mas a la menos "poco relevante" dentro de ese top-10).

```
In [2]: def nombre_modelo_desde_archivo(path):
    base = os.path.basename(path)
    base = re.sub(r'\.csv$', '', base, flags=re.IGNORECASE)
    base = re.sub(r'^variables_menos_(relevantes|revelantes)_?', '', base, flags=re.IGNORECASE)
    base = re.sub(r'^(s*)(d+)$', '', base) # quita sufijos tipo '(1)' de descargas duplicadas
    clave = base.strip().lower()

    # 1. Coincidencia exacta con el diccionario de nombres conocidos
    if clave in NOMBRES_MODELO:
        return NOMBRES_MODELO[clave]

    # 2. Heuristica por palabras clave, para no depender de que el nombre de
    # archivo sea exactamente igual al esperado (mayusculas, guiones, etc.)
    contiene = lambda *palabras: all(p in clave for p in palabras)
    if contiene('mlp') and contiene('custom'):
        return 'MLP Perdida Personalizada'
    # if contiene('mlp') and contiene('permutation'):
    # return 'MLP (variante sin confirmar - revisar origen)'
    if contiene('mlp') and ('multi' in clave or 'salida' in clave):
        return 'MLP Multisalida'
    if contiene('rf') and ('multi' in clave or 'salida' in clave):
        return 'RF Multisalida'
    if clave in ('rf', 'random_forest', 'randomforest'):
        return 'Random Forest'
    if contiene('xgb') and ('multi' in clave or 'salida' in clave):
        return 'XGB Multisalida'
    if clave in ('xgboost', 'xgb'):
        return 'XGBoost' if clave == 'xgboost' else 'XGB Multisalida'
    if 'ridge' in clave:
        return 'Ridge'
    if 'cadena' in clave or 'chain' in clave:
        return 'RegressorChain'
```

```

else:
    pass
# 3. Si nada coincide, se devuelve el nombre limpio tal cual (para poder
# identificarlo a simple vista y, si hace falta, anadirlo al diccionario)
return base if base else 'desconocido'

def cargar_bottom10(path):
    """Devuelve (lista_bottom10, motivo_error). Si motivo_error no es None,
    el archivo no se pudo usar y lista_bottom10 sera None."""
    try:
        df = pd.read_csv(path)
    except Exception as e:
        return None, f'no se pudo leer el archivo ({e})'

    columnas = list(df.columns)

    # Formato Ridge: columna GLOBAL_MENOS_RELEVANTES ya calculada
    if 'GLOBAL_MENOS_RELEVANTES' in columnas:
        return df['GLOBAL_MENOS_RELEVANTES'].tolist(), None

    # Formato estandar: columna 'variable' (+ una o mas columnas numericas)
    if 'variable' in columnas:
        return df['variable'].tolist(), None

    # Formato roto: ninguna columna de texto reconocible → no se puede usar
    return None, ('no tiene columna de nombres de variable reconocible '
        f'(columnas encontradas: {columnas}) - hace falta re-exportar '
        'conservando el nombre de la variable')

patrones = sorted(
    glob.glob(os.path.join(DATA_DIR, 'variables_menos_relevantes_*.csv')) +
    glob.glob(os.path.join(DATA_DIR, 'variables_menos_relevantes_*.csv'))
)

bottom10_por_modelo = {}
archivos_no_usables = {}

for path in patrones:
    modelo = nombre_modelo_desde_archivo(path)
    bottom10, error = cargar_bottom10(path)
    if error is None:
        bottom10_por_modelo[modelo] = bottom10
    else:
        archivos_no_usables[os.path.basename(path)] = error

print(f'Archivos encontrados: {len(patrones)}')
print(f'Modelos utilizables: {len(bottom10_por_modelo)}')
for modelo, lst in bottom10_por_modelo.items():
    print(f' OK   {modelo:45} ({len(lst)} variables)')

if archivos_no_usables:
    print(f'\nArchivos NO utilizables: {len(archivos_no_usables)}')
    for nombre, motivo in archivos_no_usables.items():
        print(f' --   {nombre}: {motivo}')

if not bottom10_por_modelo:
    print('\nNo se pudo cargar ningun archivo utilizable. Revisa DATA_DIR y los nombres de archivo.')

MODELOS_DISPONIBLES = list(bottom10_por_modelo.keys())

Archivos encontrados: 8
Modelos utilizables: 8
OK RegressorChain (10 variables)
OK MLP Multisalida (10 variables)
OK Random Forest (10 variables)
OK RF Multisalida (10 variables)
OK Ridge (10 variables)
OK XGB Multisalida (10 variables)
OK XGBoost (10 variables)
OK MLP Perdida Personalizada (10 variables)

```

2.- Bottom-10 por modelo (tal cual se cargo de cada archivo)

```

In [3]: for modelo, lst in bottom10_por_modelo.items():
        print(f'{modelo}')
        print('-' * 60)
        for i, var in enumerate(lst, 1):
            print(f' {i:2d}. {var}')
        print()

```

RegressorChain	1. mo_z 2. clay_content_z 3. soil_moisture_z 4. dem_pendiente_deg_z 5. dem_orientacion_deg_z 6. zn_z 7. plot_total_n_z 8. cu_z 9. total_plant_cover_z 10. eu_clay_content_z
MLP Multisalida	1. cu_z 2. as_z 3. plot_total_n_z 4. ni_z 5. pb_z 6. plot_total_organic_c_z 7. soil_moisture_z 8. eu_sand_content_z 9. mo_z 10. k_z
Random Forest	1. total_plant_cover_z 2. mo_z 3. aggregate_stability_z 4. ni_z 5. dem_pendiente_deg_z 6. bulk_density_z 7. cu_z 8. dem_orientacion_deg_z 9. as_z 10. gee_ndvi_verano_z
RF Multisalida	1. total_plant_cover_z 2. dem_orientacion_deg_z 3. aggregate_stability_z 4. mo_z 5. bulk_density_z 6. dem_pendiente_deg_z 7. cu_z 8. plot_total_organic_c_z 9. ni_z 10. k_z
Ridge	1. cu_z 2. plot_total_organic_c_z 3. ni_z 4. as_z 5. eu_sand_content_z 6. eu_water_holding_capacity_z 7. sand_content_z 8. eu_zn_z 9. mo_z 10. dem_orientacion_deg_z
XGB Multisalida	1. total_plant_cover_z 2. aggregate_stability_z 3. eu_zn_z 4. ni_z 5. cu_z 6. eu_organic_carbon_octop_z 7. eu_sand_content_z 8. clay_content_z 9. eu_water_holding_capacity_z 10. eu_silt_content_z
XGBoost	1. eu_silt_content_z 2. eu_sand_content_z 3. ni_z 4. plot_total_organic_c_z 5. eu_zn_z 6. dem_orientacion_deg_z 7. clay_content_z 8. cu_z 9. dem_pendiente_deg_z 10. gee_ndvi_verano_z
MLP Perdida Personalizada	1. soil_moisture_z 2. cu_z 3. sand_content_z 4. ni_z 5. as_z 6. eu_sand_content_z 7. eu_water_holding_capacity_z 8. plot_total_organic_c_z 9. eu_organic_carbon_octop_z 10. dem_orientacion_deg_z

3.- Ranking general de variables a eliminar

Para cada variable se calcula:

- **frecuencia:** en cuantos modelos aparece dentro de su bottom-10.
- **posicion media:** la posicion media (1 = la variable menos relevante de ese modelo, 10 = la decima menos relevante) en los modelos donde aparece. A igualdad de frecuencia, una posicion media mas baja indica que, cuando aparece, suele ser de las peores de todas (mas prioritaria para eliminar).

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

El ranking final ordena primero por frecuencia (descendente) y despues por posicion media (ascendente).

```
In [4]: if MODELOS_DISPONIBLES:
    registros = []
    for modelo, lst in bottom10_por_modelo.items():
        for posicion, var in enumerate(lst, 1):
            registros.append({'variable': var, 'modelo': modelo, 'posicion': posicion})
    df_registros = pd.DataFrame(registros)

    resumen = df_registros.groupby('variable').agg(
        frecuencia=('modelo', 'nunique'),
        posicion_media=('posicion', 'mean'),
    ).sort_values(['frecuencia', 'posicion_media'], ascending=[False, True])

    resumen['modelos'] = resumen.index.map(
        lambda v: sorted(df_registros.loc[df_registros['variable'] == v, 'modelo'].unique())
    )

    n_modelos = len(MODELOS_DISPONIBLES)
    print('=' * 90)
    print(f'RANKING GENERAL DE VARIABLES A ELIMINAR (sobre {n_modelos} modelos utilizables)')
    print('=' * 90)
    for i, (var, fila) in enumerate(resumen.iterrows(), 1):
        print(f'{i:2d}. {var:32} en {fila.frecuencia}/{n_modelos} modelos '
              f'f{posicion_media: {fila.posicion_media:.1f}/10}')
        print(f'      → {fila.modelos}')

    umbral = max(2, (n_modelos + 1) // 2)
    candidatas_fuertes = resumen[resumen['frecuencia'] ≥ umbral]
    print(f'\nCandidatas fuertes a eliminar (aparecen en {umbral} o mas de {n_modelos} modelos):')
    for var in candidatas_fuertes.index:
        print(f'  - {var}')
else:
    resumen = pd.DataFrame()
    print('Sin modelos disponibles, no se puede calcular el ranking.')
```

```
=====
RANKING GENERAL DE VARIABLES A ELIMINAR (sobre 8 modelos utilizables)
=====
1. cu_z                en 8/8 modelos (posicion media: 4.9/10)
   → ['MLP Multisalida', 'MLP Perdida Personalizada', 'RF Multisalida', 'Random Forest', 'RegressorChain', 'Ridge', 'XGB Multisalida', 'XGBoost']
2. ni_z                en 7/8 modelos (posicion media: 4.4/10)
   → ['MLP Multisalida', 'MLP Perdida Personalizada', 'RF Multisalida', 'Random Forest', 'Ridge', 'XGB Multisalida', 'XGBoost']
3. dem_orientacion_deg_z en 6/8 modelos (posicion media: 6.8/10)
   → ['MLP Perdida Personalizada', 'RF Multisalida', 'Random Forest', 'RegressorChain', 'Ridge', 'XGBBoost']
4. mo_z                en 5/8 modelos (posicion media: 5.0/10)
   → ['MLP Multisalida', 'RF Multisalida', 'Random Forest', 'RegressorChain', 'Ridge']
5. eu_sand_content_z    en 5/8 modelos (posicion media: 5.6/10)
   → ['MLP Multisalida', 'MLP Perdida Personalizada', 'Ridge', 'XGB Multisalida', 'XGBoost']
6. plot_total_organic_c_z en 5/8 modelos (posicion media: 5.6/10)
   → ['MLP Multisalida', 'MLP Perdida Personalizada', 'RF Multisalida', 'Ridge', 'XGBoost']
7. total_plant_cover_z  en 4/8 modelos (posicion media: 3.0/10)
   → ['RF Multisalida', 'Random Forest', 'RegressorChain', 'XGB Multisalida']
8. as_z                en 4/8 modelos (posicion media: 5.0/10)
   → ['MLP Multisalida', 'MLP Perdida Personalizada', 'Random Forest', 'Ridge']
9. dem_pendiente_deg_z  en 4/8 modelos (posicion media: 6.0/10)
   → ['RF Multisalida', 'Random Forest', 'RegressorChain', 'XGBoost']
10. aggregate_stability_z en 3/8 modelos (posicion media: 2.7/10)
    → ['RF Multisalida', 'Random Forest', 'XGB Multisalida']
11. soil_moisture_z      en 3/8 modelos (posicion media: 3.7/10)
    → ['MLP Multisalida', 'MLP Perdida Personalizada', 'RegressorChain']
12. eu_zn_z              en 3/8 modelos (posicion media: 5.3/10)
    → ['Ridge', 'XGB Multisalida', 'XGBoost']
13. clay_content_z       en 3/8 modelos (posicion media: 5.7/10)
    → ['RegressorChain', 'XGB Multisalida', 'XGBoost']
14. eu_water_holding_capacity_z en 3/8 modelos (posicion media: 7.3/10)
    → ['MLP Perdida Personalizada', 'Ridge', 'XGB Multisalida']
15. plot_total_n_z       en 2/8 modelos (posicion media: 5.0/10)
    → ['MLP Multisalida', 'RegressorChain']
16. sand_content_z       en 2/8 modelos (posicion media: 5.0/10)
    → ['MLP Perdida Personalizada', 'Ridge']
17. bulk_density_z       en 2/8 modelos (posicion media: 5.5/10)
    → ['RF Multisalida', 'Random Forest']
18. eu_silt_content_z     en 2/8 modelos (posicion media: 5.5/10)
    → ['XGB Multisalida', 'XGBoost']
19. eu_organic_carbon_octop_z en 2/8 modelos (posicion media: 7.5/10)
    → ['MLP Perdida Personalizada', 'XGB Multisalida']
20. gee_ndvi_verano_z     en 2/8 modelos (posicion media: 10.0/10)
    → ['Random Forest', 'XGBoost']
21. k_z                 en 2/8 modelos (posicion media: 10.0/10)
    → ['MLP Multisalida', 'RF Multisalida']
22. pb_z                 en 1/8 modelos (posicion media: 5.0/10)
    → ['MLP Multisalida']
23. zn_z                 en 1/8 modelos (posicion media: 6.0/10)
    → ['RegressorChain']
24. eu_clay_content_z     en 1/8 modelos (posicion media: 10.0/10)
    → ['RegressorChain']

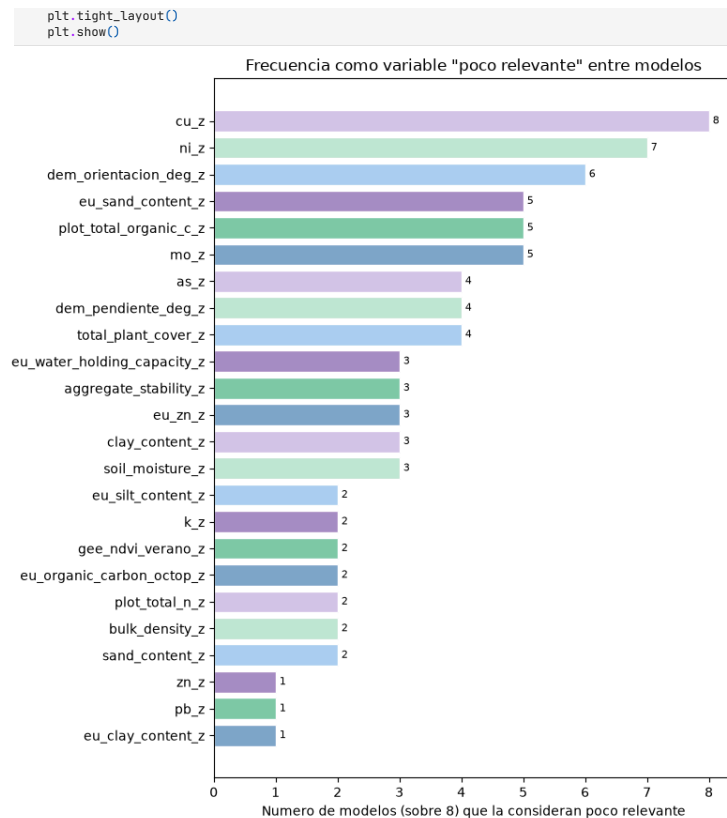
Candidatas fuertes a eliminar (aparecen en 4 o mas de 8 modelos):
- cu_z
- ni_z
- dem_orientacion_deg_z
- mo_z
- eu_sand_content_z
- plot_total_organic_c_z
- total_plant_cover_z
- as_z
- dem_pendiente_deg_z
```

4.- Visualizacion

4.1 Frecuencia: en cuantos modelos aparece cada variable como "poco relevante"

```
In [5]: if not resumen.empty:
    frecuencia_ordenada = resumen['frecuencia'].sort_values()

    fig, ax = plt.subplots(figsize=(8, max(4, 0.35 * len(frecuencia_ordenada))))
    bars = ax.barh(frecuencia_ordenada.index, frecuencia_ordenada.values,
                   color=palette_cycle(len(frecuencia_ordenada)), edgecolor='white')
    ax.bar_label(bars, fmt='%d', padding=3, fontsize=8)
    ax.set_xlabel(f'Numero de modelos (sobre {len(MODELOS_DISPONIBLES)}) que la consideran poco relevante')
    ax.set_title('Frecuencia como variable "poco relevante" entre modelos', fontsize=12)
```



4.2 Mapa de calor: posicion dentro del bottom-10 (variable x modelo)

Cada celda es la posición de esa variable dentro del bottom-10 de ese modelo (1 = la más "poco relevante" de todas, 10 = la decima). Las celdas vacías significan que esa variable no aparece en el bottom-10 de ese modelo (no necesariamente que sea muy importante ahí, solo que no está entre las 10 peores). Ordenado por frecuencia y luego por posición media.

```
In [6]: if not resumen.empty:
orden_filas = resumen.index
tabla_posiciones = pd.DataFrame(index=orden_filas, columns=MODELOS_DISPONIBLES, dtype=float)
for modelo, lst in bottom10_por_modelo.items():
    for posicion, var in enumerate(lst, 1):
        if var in tabla_posiciones.index:
            tabla_posiciones.loc[var, modelo] = posicion

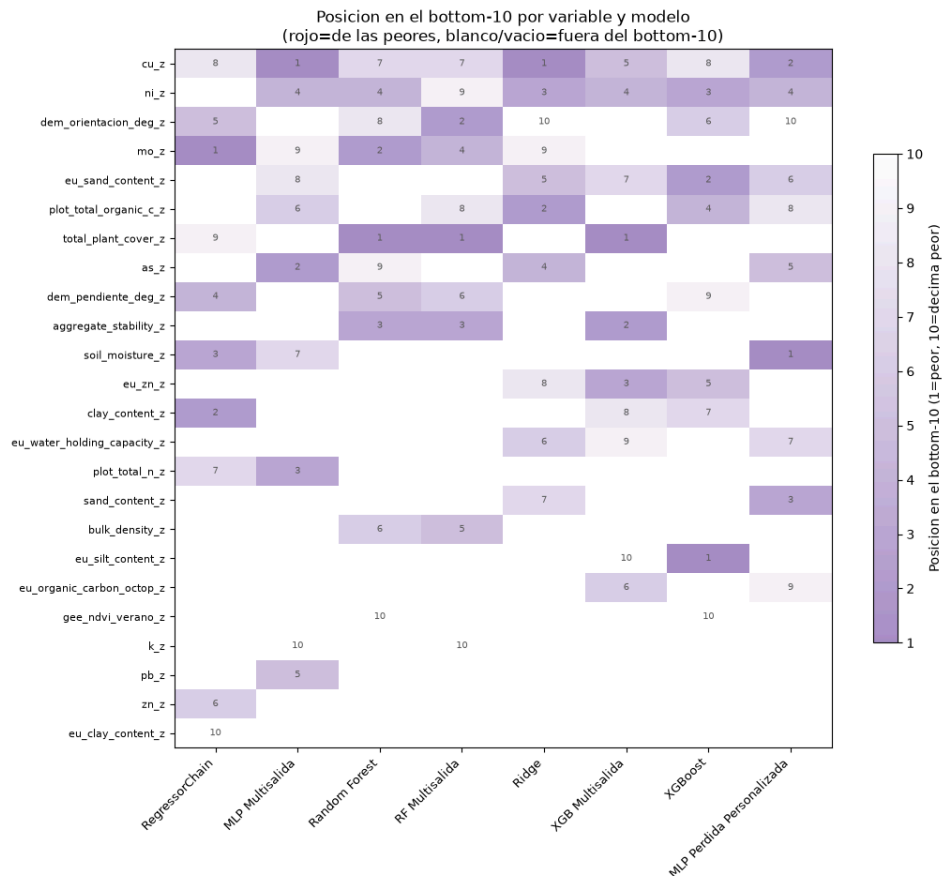
cmap_pos = LinearSegmentedColormap.from_list('sob4es_pos', [PALETTE['purple'], '#FFFFFF'])

fig, ax = plt.subplots(figsize=(1.1 * len(MODELOS_DISPONIBLES) + 2, max(4, 0.4 * len(orden_filas))))
data = tabla_posiciones.values.astype(float)
im = ax.imshow(np.ma.masked_invalid(data), cmap=cmap_pos, vmin=1, vmax=10, aspect='auto')

ax.set_xticks(range(len(MODELOS_DISPONIBLES)))
ax.set_xticklabels(MODELOS_DISPONIBLES, rotation=45, ha='right', fontsize=9)
ax.set_yticks(range(len(orden_filas)))
ax.set_yticklabels(orden_filas, fontsize=8)

for i in range(len(orden_filas)):
    for j in range(len(MODELOS_DISPONIBLES)):
        val = data[i, j]
        if not np.isnan(val):
            ax.text(j, i, f'{val:.0f}', ha='center', va='center', fontsize=7, color=PALETTE['gray_dark'])

fig.colorbar(im, ax=ax, label='Posicion en el bottom-10 (1=peor, 10=decima peor)', shrink=0.7)
ax.set_title('Posicion en el bottom-10 por variable y modelo\n(rojo=de las peores, blanco/vacio=fuera del bottom-10)', fontsize=12)
plt.tight_layout()
plt.show()
```



5.- Archivos que no se pudieron usar

Si alguno de los `variables_menos_relevantes_*.csv` no aparece en las secciones anteriores, es porque no se pudo interpretar correctamente. Aquí se explica el motivo de cada uno para poder corregirlo (normalmente hace falta re-exportar el CSV conservando el nombre de la variable como columna o como índice, en vez de solo los valores numericos).

```
In [7]: if archivos_no_usables:
        for nombre, motivo in archivos_no_usables.items():
            print(f'{nombre}:')
            print(f' {motivo}')
            print()
        else:
            print('Todos los archivos encontrados se pudieron usar.')
```

Todos los archivos encontrados se pudieron usar.

6.- Conclusiones y como usar el ranking

- Las variables en "candidatas fuertes a eliminar" (Seccion 3) son las que mas modelos, con metodos de importancia distintos entre si (SHAP, permutacion, coeficientes, gain), coinciden en señalar como poco relevantes. Es la señal mas robusta de todo el analisis.
- Como probarlo:** quita esas variables de `FEATURES_AUTORIZADAS`, reentrena (tuning + produccion) los modelos afectados, y compara el R2 sobre `eval.csv` con `comparativa_modelos.ipynb` antes y despues. Si el R2 se mantiene o mejora con menos variables, confirma que esas variables solo aportaban ruido/sobreajuste.
- Revisa la Seccion 5 antes de dar el ranking por definitivo: cualquier modelo listado ahí no esta participando en el conteo, y su ausencia puede sesgar el resultado (sobre todo si son modelos con metodos de importancia distintos a los que sí se cargaron).
- Ante cualquier variable que aparezca en pocos modelos pero con posicion medio muy baja (p. ej. 1-2), vale la pena revisarla igualmente: puede que solo la calculen 1-2 modelos con ese metodo en concreto, pero que dentro de esos sea consistentemente la peor de todas.
- Como siempre, la relevancia estadística es un criterio mas, no el unico: si una variable tiene sentido de dominio (ecologico/edafologico) fuerte, vale la pena mantenerla en el modelo aunque su importancia estadística sea baja, sobre todo si el coste de mantenerla es bajo.

11.3.6. Notebook para la comparación de barrido de rs (Prueba III)

SOB4ES - Comparador del barrido de random_state (Prueba 3)

Este notebook consolida los resultados generados por el barrido de `random_state` (0-100) en los 8 notebooks de modelos de SOB4ES (`xgboost-model`, `xgb_multisalida`, `rf-model`, `rf_multisalida`, `regressorchain`, `reg-model`, `mlp_multisalida`, `mlp_custom_loss`).

Objetivo: determinar qué modelo o modelos son consistentemente los más prominentes en el ranking a lo largo de 101 semillas distintas, en lugar de fiarse de una única ejecución con `random_state=42` (ver Prueba 2, que ya demostró que el ranking con una sola semilla no es estable).

Requisito previo: haber ejecutado la celda de barrido en cada uno de los 8 notebooks de modelos, lo que genera un `*_barrido_rs.csv` por modelo bajo `output/models/barrido_rs_<modelo>/`.

Este notebook tiene dos bloques independientes:

- **Bloque A — Ranking global:** compara los modelos usando la media de R^2 de los 2 targets prioritarios (`earthworm_shannon_z`, `earthworm_richness_z`). Da una conclusión rápida de "qué modelo es el más prominente en general".
- **Bloque B — Ranking por target:** compara los modelos target a target sobre los 21 targets del proyecto, para detectar si un mismo modelo domina en todos o si conviene uno distinto según el target.

0. Configuración

```
In [2]: import os
import itertools
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Paleta de colores unificada (identica a la usada en comparador_modelos.ipynb)
PALETTE = {
    'blue':      '#7FA6C9',
    'blue_light': '#AADO0F1',
    'green':     '#7FC8A9',
    'green_light': '#BEE6D3',
    'purple':    '#A98FC7',
    'purple_light': '#D6C7EA',
    'gray':      '#9B9B9B',
    'gray_dark': '#5C5C5C',
}

# Secuencia ciclica para graficos con N series/categorias (N puede superar 6)
PALETTE_CYCLE = [PALETTE['blue'], PALETTE['green'], PALETTE['purple'],
                  PALETTE['blue_light'], PALETTE['green_light'], PALETTE['purple_light']]

def palette_cycle(n):
    """Devuelve una lista de n colores repitiendo PALETTE_CYCLE si hace falta."""
    return list(itertools.islice(itertools.cycle(PALETTE_CYCLE), n))

os.makedirs('output/regresion', exist_ok=True)
os.makedirs('output/regresion/graphs', exist_ok=True)
```

Bloque A — Ranking global (targets prioritarios)

Usa la media de R^2 de `earthworm_shannon_z` y `earthworm_richness_z` como criterio único de ranking por semilla.

A.1 Carga y unificación de los CSVs del barrido

Se cargan los 8 `*_barrido_rs.csv`. Si alguno no existe todavía (falta ejecutar ese barrido en su notebook), se avisa y se continúa con los que sí están disponibles.

```
In [2]: BARRIDO_PATHS = {
    'XGBoost Individual':      'output/models/xgb/barrido_rs_xgb/xgb_barrido_rs.csv',
    'XGBoost Multisalida':    'output/models/xgb_multi/barrido_rs_xgb_multi/xgb_multi_barrido_rs.csv',
    'RF Individual':          'output/models/rf/barrido_rs_rf/rf_barrido_rs.csv',
    'RandomForest Multisalida': 'output/models/rf_multi/barrido_rs_rf_multi/rf_multi_barrido_rs.csv',
    'Random Forest con RegressorChain': 'output/models/chain/barrido_rs_chain/chain_barrido_rs.csv',
    'Regresión Individual':    'output/models/reg/barrido_rs_ridge/ridge_barrido_rs.csv',
    'MLP Estandar':           'output/models/mlp/barrido_rs_mlp/mlp_barrido_rs.csv',
    'MLP Perdida Custom':     'output/models/mlp_custom/barrido_rs_mlp_custom/mlp_custom_barrido_rs.csv',
}

COLS_COMUNES = ['random_state', 'earthworm_shannon_z_r2', 'earthworm_richness_z_r2']

dfs_barrido = []
print(f'\nTotal de filas combinadas: {len(df_barrido_todos)}')
for nombre_modelo, ruta in BARRIDO_PATHS.items():
    if not os.path.exists(ruta):
        print(f' [AVISO] No encontrado: {ruta} (¿falta ejecutar ese barrido?)')
        continue
    df_m = pd.read_csv(ruta)[COLS_COMUNES].copy()
    df_m['Modelo'] = nombre_modelo
    df_m['r2_medio_prioritarios'] = df_m[['earthworm_shannon_z_r2', 'earthworm_richness_z_r2']].mean(axis=1)
    dfs_barrido.append(df_m)
    print(f' {nombre_modelo}<35} → {len(df_m)} semillas cargadas')

df_barrido_todos = pd.concat(dfs_barrido, ignore_index=True)
print(f'\nTotal de filas combinadas: {len(df_barrido_todos)}')
f'({df_barrido_todos["Modelo"].nunique()} modelos x {df_barrido_todos["random_state"].nunique()} semillas)'
```

Cargando resultados del barrido de random_state por modelo...

```
XGBoost Individual      → 101 semillas cargadas
XGBoost Multisalida    → 101 semillas cargadas
RF Individual           → 101 semillas cargadas
RandomForest Multisalida → 101 semillas cargadas
Random Forest con RegressorChain → 101 semillas cargadas
Regresión Individual    → 101 semillas cargadas
MLP Estandar            → 101 semillas cargadas
MLP Perdida Custom     → 101 semillas cargadas
```

Total de filas combinadas: 808 (8 modelos x 101 semillas)

A.2 Ranking por semilla

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

Para cada `random_state`, se ordenan los 8 modelos de mayor a menor R^2 medio y se guarda la posición de cada uno (1º a 8º).

Se usa `sort_values` + `cumcount` en vez de `groupby().apply()` para evitar que versiones recientes de pandas (2.2+) excluyan la columna de agrupación del resultado.

```
In [3]: df_rankings = (
    df_barrido_todos
    .sort_values(['random_state', 'r2_medio_prioritarios'], ascending=[True, False])
    .reset_index(drop=True)
)
df_rankings['posicion'] = df_rankings.groupby('random_state').cumcount() + 1

print('Ejemplo de ranking para random_state=42:')
print(df_rankings[df_rankings['random_state'] == 42]
      [['posicion', 'Modelo', 'earthworm_shannon_z_r2', 'earthworm_richness_z_r2']]
      .to_string(index=False))
```

Ejemplo de ranking para random_state=42:

posicion	Modelo	earthworm_shannon_z_r2	earthworm_richness_z_r2
1	XGBoost Multisalida	0.542808	0.579899
2	RF Individual	0.495710	0.579899
3	Random Forest con RegressorChain	0.458899	0.533491
4	MLP Perdida Custom	0.444963	0.531502
5	XGBoost Individual	0.389882	0.489547
6	RandomForest Multisalida	0.488728	0.442448
7	MLP Estandar	0.331738	0.435985
8	Regresión Individual	0.239544	0.278931

A.3 Agregación final: estabilidad y frecuencia en el ranking

Esta es la tabla clave de la Prueba 3: para cada modelo, calcula el R^2 medio/std a lo largo de las 101 semillas y cuántas veces queda en el top-1 / top-3 del ranking.

```
In [4]: resumen_barrido = df_barrido_todos.groupby('Modelo').agg(
    r2_medio = ('r2_medio_prioritarios', 'mean'),
    r2_std = ('r2_medio_prioritarios', 'std'),
    r2_min = ('r2_medio_prioritarios', 'min'),
    r2_max = ('r2_medio_prioritarios', 'max'),
).reset_index()

n_semillas = df_barrido_todos['random_state'].nunique()

for k in [1, 2, 3, 5]:
    frecuencia_k = (df_rankings[df_rankings['posicion'] <= k]['Modelo']
                    .value_counts().reindex(resumen_barrido['Modelo']).fillna(0).astype(int))
    resumen_barrido[f'veces_top{k}'] = resumen_barrido['Modelo'].map(frecuencia_k)
    resumen_barrido[f'pct_top{k}'] = (resumen_barrido[f'veces_top{k}'] / n_semillas * 100).round(1)

resumen_barrido = resumen_barrido.sort_values('veces_top1', ascending=False).reset_index(drop=True)
resumen_barrido.index += 1

print('Resumen del barrido de random_state (0-100) por modelo:')
print('=' * 120)
print(resumen_barrido.round(4).to_string())
print('=' * 120)
print(f'\nModelo mas prominente (mas veces en el top-1): {resumen_barrido.iloc[0]['Modelo']} "
      f"({resumen_barrido.iloc[0]['veces_top1']}/{n_semillas} semillas)')
```

Resumen del barrido de random_state (0-100) por modelo:

	Modelo	r2_medio	r2_std	r2_min	r2_max	veces_top1	pct_top1	veces_top2	pct_top2	veces_top3	pct_top3	veces_top5	pct_top
5													
1	XGBoost Multisalida	0.5685	0.0876	0.5415	0.5826	101	100.0	101	100.0	101	100.0	101	100.
0													
2	MLP Estandar	0.3983	0.0330	0.3874	0.4681	0	0.0	0	0.0	0	0.0	12	11.
9													
3	RF Individual	0.5316	0.0874	0.5132	0.5492	0	0.0	101	100.0	101	100.0	101	100.
0													
4	MLP Perdida Custom	0.3677	0.0422	0.2581	0.4882	0	0.0	0	0.0	0	0.0	3	3.
0													
5	Random Forest con RegressorChain	0.5030	0.0111	0.4825	0.5284	0	0.0	0	0.0	101	100.0	101	100.
0													
6	RandomForest Multisalida	0.4317	0.0086	0.3990	0.4519	0	0.0	0	0.0	0	0.0	86	85.
1													
7	Regresión Individual	0.2592	0.0000	0.2592	0.2592	0	0.0	0	0.0	0	0.0	0	0.
0													
8	XGBoost Individual	0.4397	0.0000	0.4397	0.4397	0	0.0	0	0.0	0	0.0	101	100.
0													

Modelo mas prominente (mas veces en el top-1): XGBoost Multisalida (101/101 semillas)

A.4 Gráficos: dispersión de R^2 y frecuencia en el top-1

```
In [5]: orden_modelos_barrido = resumen_barrido.sort_values('r2_medio')['Modelo']

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Boxplot de R2 medio (shannon+richness) por modelo a lo largo de las 101 semillas
data_box = [df_barrido_todos[df_barrido_todos['Modelo'] == m]['r2_medio_prioritarios'].values
             for m in orden_modelos_barrido]
bp = axes[0].boxplot(data_box, orientation='horizontal', patch_artist=True, tick_labels=orden_modelos_barrido)
for patch, color in zip(bp['boxes'], palette_cycle(len(orden_modelos_barrido))):
    patch.set_facecolor(color)
axes[0].set_xlabel('R2 medio (shannon + richness) por semilla')
axes[0].set_title('Dispersión de R2 por modelo\n(101 random_states, 0-100)', fontsize=12)

# Barras de frecuencia en el top-1
_orden_top1 = resumen_barrido.sort_values('veces_top1')
axes[1].barh(_orden_top1['Modelo'], _orden_top1['pct_top1'], color=PALETTE['green'], edgecolor='white')
for i, v in enumerate(_orden_top1['pct_top1']):
    axes[1].text(v + 1, i, f'{v:.0f}%', va='center', fontsize=9)
axes[1].set_xlabel('% de semillas en las que el modelo queda 1o')
axes[1].set_title('Frecuencia en el top-1 del ranking', fontsize=12)
axes[1].set_xlim(0, 100)

plt.tight_layout()
fig.savefig('output/regresion/graphs/barrido_rs_resumen.png', dpi=250, bbox_inches='tight')
plt.show()

# -----
# Grafico: frecuencia en el top-2
# -----
_orden_top2 = resumen_barrido.sort_values('veces_top2')
```

```
fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(_orden_top2['Modelo'], _orden_top2['pct_top2'], color=PALETTE['blue'], edgecolor='white')
for i, v in enumerate(_orden_top2['pct_top2']):
    ax.text(v + 1, i, f'{v:.0f}%', va='center', fontsize=9)
ax.set_xlabel('% de semillas en las que el modelo queda en el top-2')
ax.set_title('Frecuencia en el top-2 del ranking\n(101 random_states, 0-100)', fontsize=12)
ax.set_xlim(0, 100)

plt.tight_layout()
fig.savefig('output/regresion/graphs/barrido_rs_top2.png', dpi=250, bbox_inches='tight')
plt.show()

# -----
# Grafico: frecuencia en el top-3
# -----
_orden_top3 = resumen_barrido.sort_values('veces_top3')

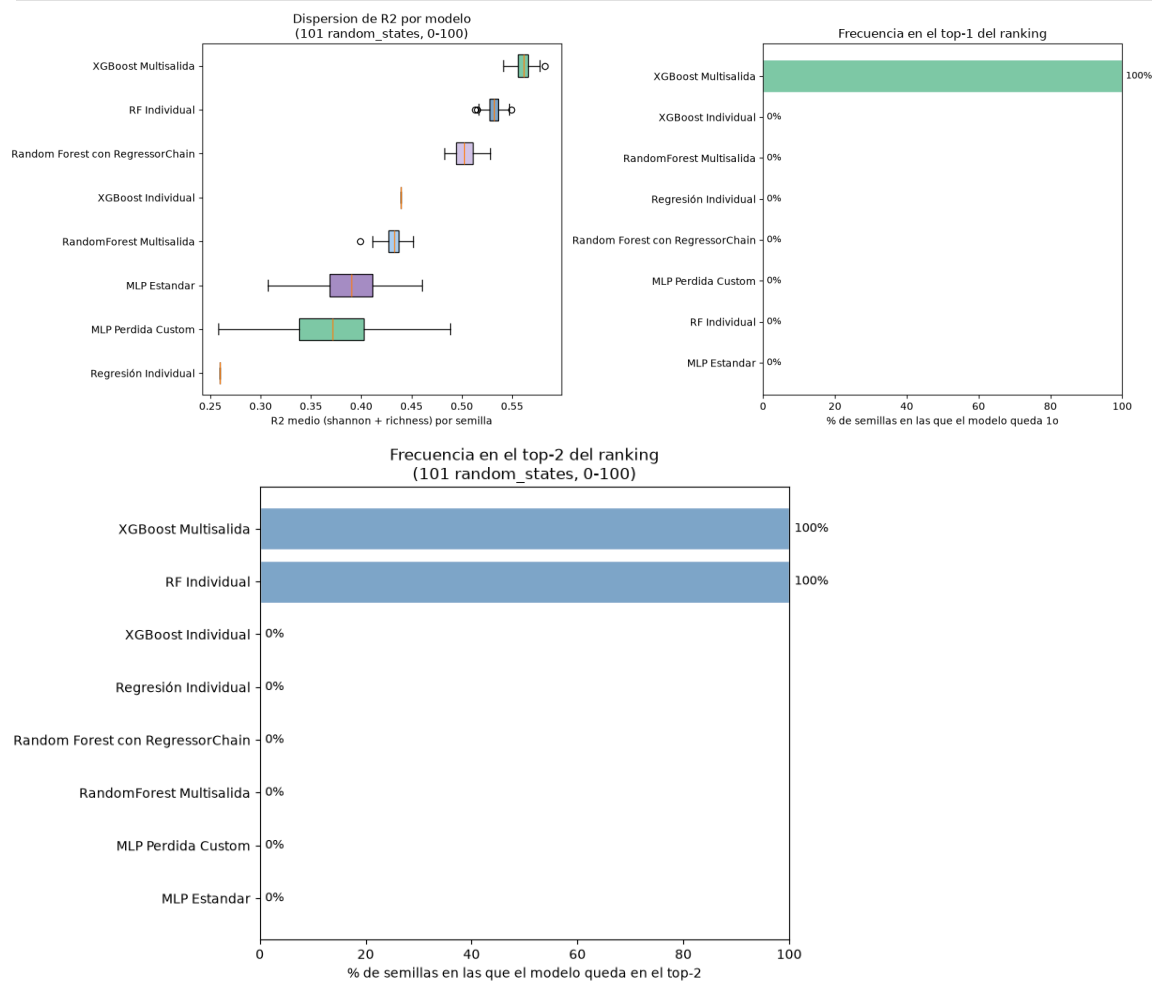
fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(_orden_top3['Modelo'], _orden_top3['pct_top3'], color=PALETTE['purple'], edgecolor='white')
for i, v in enumerate(_orden_top3['pct_top3']):
    ax.text(v + 1, i, f'{v:.0f}%', va='center', fontsize=9)
ax.set_xlabel('% de semillas en las que el modelo queda en el top-3')
ax.set_title('Frecuencia en el top-3 del ranking\n(101 random_states, 0-100)', fontsize=12)
ax.set_xlim(0, 100)

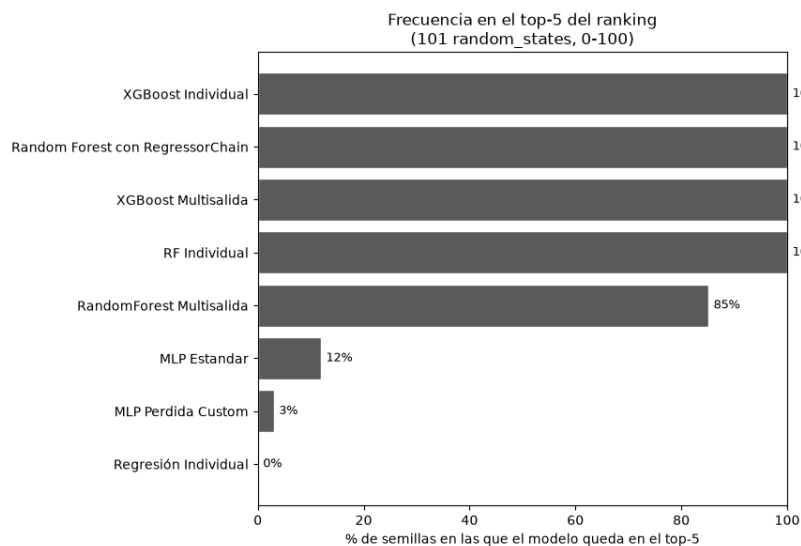
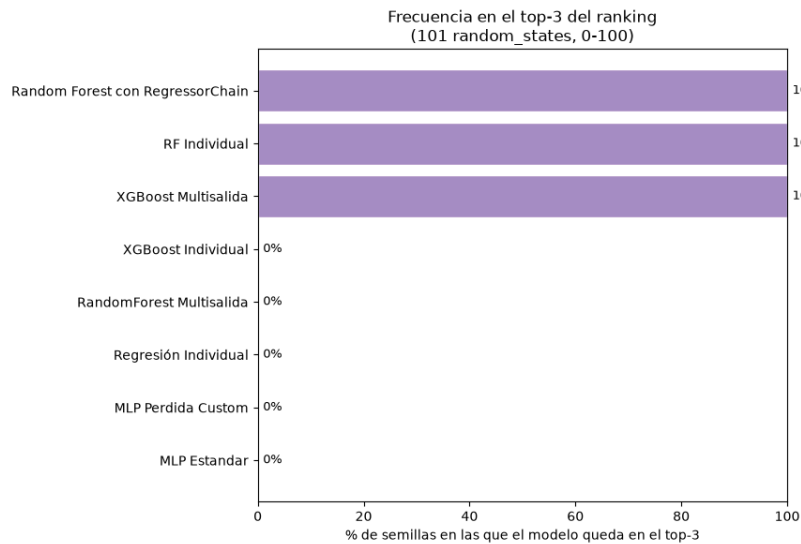
plt.tight_layout()
fig.savefig('output/regresion/graphs/barrido_rs_top3.png', dpi=250, bbox_inches='tight')
plt.show()

# -----
# Grafico: frecuencia en el top-5
# -----
_orden_top5 = resumen_barrido.sort_values('veces_top5')

fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(_orden_top5['Modelo'], _orden_top5['pct_top5'], color=PALETTE['gray_dark'], edgecolor='white')
for i, v in enumerate(_orden_top5['pct_top5']):
    ax.text(v + 1, i, f'{v:.0f}%', va='center', fontsize=9)
ax.set_xlabel('% de semillas en las que el modelo queda en el top-5')
ax.set_title('Frecuencia en el top-5 del ranking\n(101 random_states, 0-100)', fontsize=12)
ax.set_xlim(0, 100)

plt.tight_layout()
fig.savefig('output/regresion/graphs/barrido_rs_top5.png', dpi=250, bbox_inches='tight')
plt.show()
```





A.5 Exportación de resultados (ranking global)

```
In [6]: df_barrido_todos.to_csv('output/regresion/barrido_rs_todos_los_modelos.csv', index=False)
df_rankings.to_csv('output/regresion/barrido_rs_rankings_por_semilla.csv', index=False)
resumen_barrido.to_csv('output/regresion/barrido_rs_resumen.csv', index=False)

print('Guardado:')
print(' output/regresion/barrido_rs_todos_los_modelos.csv (una fila por modelo x semilla)')
print(' output/regresion/barrido_rs_rankings_por_semilla.csv (posicion de cada modelo en cada semilla)')
print(' output/regresion/barrido_rs_resumen.csv (tabla resumen para el informe de la Prueba 3)')

Guardado:
output/regresion/barrido_rs_todos_los_modelos.csv (una fila por modelo x semilla)
output/regresion/barrido_rs_rankings_por_semilla.csv (posicion de cada modelo en cada semilla)
output/regresion/barrido_rs_resumen.csv (tabla resumen para el informe de la Prueba 3)
```

Bloque B — Ranking por target (los 21 targets)

El bloque A resume el rendimiento en 2 targets prioritarios. Este bloque desglosa el mismo análisis en los 21 targets del proyecto, para ver si el modelo más prominente cambia según el target.

B.1 Carga en formato largo (modelo x semilla x target)

Reutiliza `BARRIDO_PATHS` del bloque A. Convierte cada CSV a formato largo (`melt`) para tener una fila por combinación de modelo, semilla y target.

```
In [7]: filas_largas = []
print('Cargando resultados del barrido de random_state por modelo (todos los targets)...\n')
for nombre_modelo, ruta in BARRIDO_PATHS.items():
    if not os.path.exists(ruta):
        print(f' [AVISO] No encontrado: {ruta} (¿falta ejecutar ese barrido?)')
        continue
    df_m = pd.read_csv(ruta)

    # columnas de R2 por target: '<target>_r2'
    cols_r2 = [c for c in df_m.columns if c.endswith('_r2') and c not in ('r2_global',)]

    df_largo = df_m[['random_state'] + cols_r2].melt(
```

```

        id_vars='random_state', var_name='target', value_name='r2'
    )
    df_largo['target'] = df_largo['target'].str.removesuffix('_r2')
    df_largo['Modelo'] = nombre_modelo
    filas_largas.append(df_largo)
    print(f'    {nombre_modelo}<35} → {df_m["random_state"].nunique()} semillas x {len(cols_r2)} targets')

df_barrido_por_target = pd.concat(filas_largas, ignore_index=True)
print(f'\nTotal de filas combinadas: {len(df_barrido_por_target)} '
      f'({df_barrido_por_target["Modelo"].nunique()} modelos x '
      f'{df_barrido_por_target["random_state"].nunique()} semillas x '
      f'{df_barrido_por_target["target"].nunique()} targets')

```

Cargando resultados del barrido de random_state por modelo (todos los targets)...

```

XGBoost Individual          → 101 semillas x 21 targets
XGBoost Multisalida         → 101 semillas x 21 targets
RF Individual               → 101 semillas x 21 targets
RandomForest Multisalida    → 101 semillas x 21 targets
Random Forest con RegressorChain → 101 semillas x 21 targets
Regresión Individual        → 101 semillas x 21 targets
MLP Estandar               → 101 semillas x 21 targets
MLP Perdida Custom          → 101 semillas x 21 targets

```

Total de filas combinadas: 16968 (8 modelos x 101 semillas x 21 targets)

B.2 Ranking por semilla y por target

Igual que en el bloque A, pero ahora la agrupación es `(random_state, target)`: cada target tiene su propio ranking de 8 modelos por semilla.

```

In [8]: df_rankings_target = (
    df_barrido_por_target
    .sort_values(['random_state', 'target', 'r2'], ascending=[True, True, False])
    .reset_index(drop=True)
)
df_rankings_target['posicion'] = df_rankings_target.groupby(['random_state', 'target']).cumcount() + 1

print('Ejemplo de ranking para random_state=42, target earthworm_shannon_z:')
print(df_rankings_target[(df_rankings_target['random_state'] == 42) & (df_rankings_target['target'] == 'earthworm_shannon_z')
      [['posicion', 'Modelo', 'r2']]
      .to_string(index=False))

```

Ejemplo de ranking para random_state=42, target earthworm_shannon_z:

posicion	Modelo	r2
1	XGBoost Multisalida	0.542088
2	RF Individual	0.495710
3	Random Forest con RegressorChain	0.450899
4	MLP Perdida Custom	0.444963
5	RandomForest Multisalida	0.408728
6	XGBoost Individual	0.389802
7	MLP Estandar	0.331738
8	Regresión Individual	0.239544

B.3 Agregación final por target

Para cada combinación `(target, modelo)`: R^2 medio/std y frecuencia en el top-1/top-3 a lo largo de las 101 semillas. `ganador_por_target` resume, para cada uno de los 21 targets, cuál es el modelo más prominente.

```

In [9]: resumen_por_target = df_barrido_por_target.groupby(['target', 'Modelo']).agg(
    r2_medio = ('r2', 'mean'),
    r2_std   = ('r2', 'std'),
).reset_index()

n_semillas = df_barrido_por_target['random_state'].nunique()

frecuencia_top1 = df_rankings_target[df_rankings_target['posicion'] == 1].groupby(['target', 'Modelo']).size()
frecuencia_top3 = df_rankings_target[df_rankings_target['posicion'] <= 3].groupby(['target', 'Modelo']).size()

resumen_por_target = resumen_por_target.set_index(['target', 'Modelo'])
resumen_por_target['veces_top1'] = frecuencia_top1
resumen_por_target['veces_top3'] = frecuencia_top3
resumen_por_target[['veces_top1', 'veces_top3']] = resumen_por_target[['veces_top1', 'veces_top3']].fillna(0).astype(int)
resumen_por_target['pct_top1'] = (resumen_por_target['veces_top1'] / n_semillas * 100).round(1)
resumen_por_target['pct_top3'] = (resumen_por_target['veces_top3'] / n_semillas * 100).round(1)
resumen_por_target = resumen_por_target.reset_index()

# Tabla resumen: para cada target, quien gana mas veces el top-1
ganador_por_target = (
    resumen_por_target
    .sort_values(['target', 'veces_top1'], ascending=[True, False])
    .groupby('target')
    .first()
    [['Modelo', 'veces_top1', 'pct_top1', 'r2_medio']]
    .rename(columns={'Modelo': 'modelo_ganador'})
)

print('Modelo mas prominente (top-1 mas frecuente) por target:')
print('=' * 90)
print(ganador_por_target.round(4).to_string())

```

Modelo mas prominente (top-1 mas frecuente) por target:

target	modelo_ganador	veces_top1	pct_top1	r2_medio
bac_asv_richness_z	XGBoost Multisalida	42	41.6	0.0106
bac_shannon_z	RandomForest Multisalida	52	51.5	0.0282
cerc_asv_richness_z	RF Individual	50	49.5	0.1227
cerc_shannon_z	Regresión Individual	87	86.1	0.0155
coll_shannon_z	RandomForest Multisalida	100	99.0	-0.1947
coll_species_richness_z	RandomForest Multisalida	101	100.0	-0.2973
earthworm_richness_z	XGBoost Multisalida	93	92.1	0.5866
earthworm_shannon_z	XGBoost Multisalida	101	100.0	0.5343
euk_asv_richness_z	Random Forest con RegressorChain	48	47.5	0.2547
euk_shannon_z	RF Individual	75	74.3	0.1684
fun_asv_richness_z	MLP Estandar	50	49.5	0.0101
fun_shannon_z	Regresión Individual	40	39.6	-0.0161
macro_order_richness_z	XGBoost Multisalida	80	79.2	0.4084
macro_shannon_z	XGBoost Multisalida	101	100.0	0.3732
meso_shannon_z	RandomForest Multisalida	84	83.2	-0.1218
meso_species_richness_z	RandomForest Multisalida	72	71.3	-0.0256
nematode_shannon_z	XGBoost Multisalida	100	99.0	0.2109
oomy_asv_richness_z	Random Forest con RegressorChain	61	60.4	0.1995
oomy_shannon_z	MLP Estandar	101	100.0	0.1938
orib_shannon_z	XGBoost Multisalida	69	68.3	0.2283
orib_species_richness_z	XGBoost Multisalida	80	79.2	0.2787

B.4 Gráfico: heatmap de dominancia por target

Un vistazo rápido a si hay un modelo que domine en todos los targets o si conviene un modelo distinto según el target.

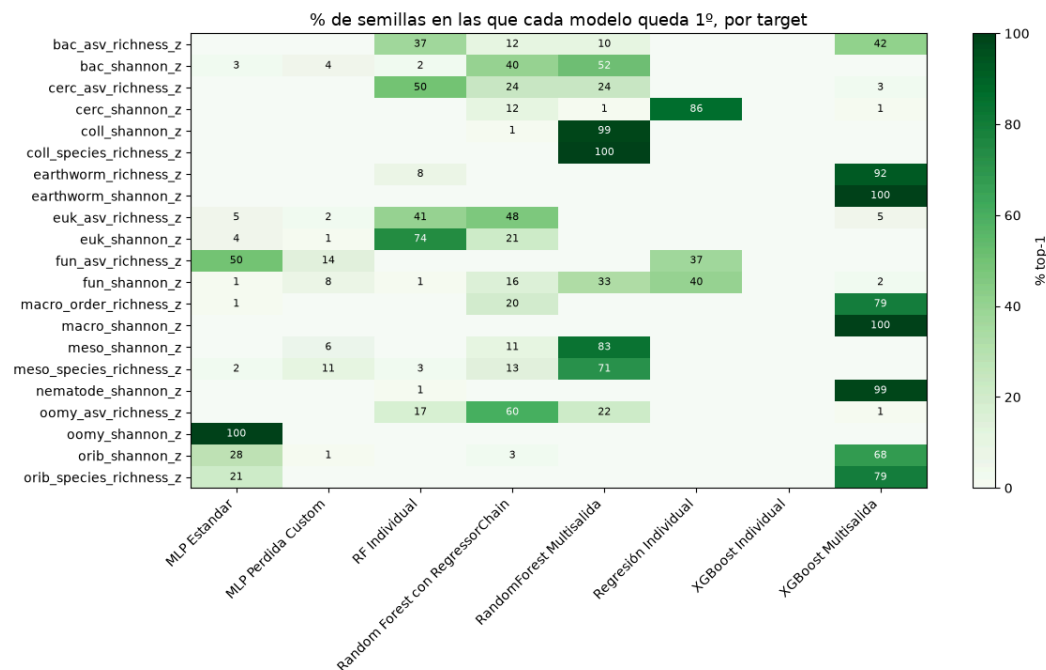
```
In [10]: pivot_top1 = resumen_por_target.pivot(index='target', columns='Modelo', values='pct_top1').fillna(0)

fig, ax = plt.subplots(figsize=(12, max(6, 0.35 * len(pivot_top1))))
im = ax.imshow(pivot_top1.values, cmap='Greens', aspect='auto', vmin=0, vmax=100)

ax.set_xticks(range(len(pivot_top1.columns)))
ax.set_xticklabels(pivot_top1.columns, rotation=45, ha='right')
ax.set_yticks(range(len(pivot_top1.index)))
ax.set_yticklabels(pivot_top1.index)

for i in range(pivot_top1.shape[0]):
    for j in range(pivot_top1.shape[1]):
        val = pivot_top1.values[i, j]
        if val > 0:
            ax.text(j, i, f'{val:.0f}', ha='center', va='center',
                    color='white' if val > 50 else 'black', fontsize=8)

ax.set_title('% de semillas en las que cada modelo queda 1º, por target', fontsize=13)
plt.colorbar(im, ax=ax, label='% top-1')
plt.tight_layout()
fig.savefig('output/regresion/graphs/barrido_rs_top1_por_target.png', dpi=250, bbox_inches='tight')
plt.show()
```



B.5 Exportación de resultados por target

```
In [11]: df_barrido_por_target.to_csv('output/regresion/barrido_rs_todos_los_modelos_por_target.csv', index=False)
df_rankings_target.to_csv('output/regresion/barrido_rs_rankings_por_semilla_target.csv', index=False)
resumen_por_target.to_csv('output/regresion/barrido_rs_resumen_por_target.csv', index=False)
ganador_por_target.to_csv('output/regresion/barrido_rs_ganador_por_target.csv')

print('Guardado:')
print(' barrido_rs_todos_los_modelos_por_target.csv (modelo x semilla x target)')
print(' barrido_rs_rankings_por_semilla_target.csv (posicion de cada modelo, por semilla y target)')
print(' barrido_rs_resumen_por_target.csv (media/std/top1/top3 por modelo y target)')
print(' barrido_rs_ganador_por_target.csv (modelo mas prominente por target)')
```

Guardado:
barrido_rs_todos_los_modelos_por_target.csv (modelo x semilla x target)
barrido_rs_rankings_por_semilla_target.csv (posicion de cada modelo, por semilla y target)
barrido_rs_resumen_por_target.csv (media/std/top1/top3 por modelo y target)
barrido_rs_ganador_por_target.csv (modelo mas prominente por target)

Conclusiones (Prueba 3)

El barrido de 101 semillas descarta la idea de un único modelo "ganador universal": aunque XGBoost Multisalida es el más prominente en el ranking global (99% de las semillas en el top-1, R^2 medio 0.5605) y en los dos targets prioritarios del proyecto, el desglose target a target muestra que **el modelo más robusto depende del target**. La recomendación es adoptar una estrategia de modelo específico por grupo de targets, no un modelo único para los 21.

Modelo recomendado por target (según frecuencia de top-1 a lo largo de las 101 semillas):

Modelo	Targets donde es el más prominente	Nº targets
XGBoost Multisalida	earthworm_shannon_z (92%), earthworm_richness_z (94%), macro_shannon_z (100%), nematode_shannon_z (98%), macro_order_richness_z (58%), bac_asv_richness_z (55%)	6
Random Forest Multisalida	bac_shannon_z (59%), meso_shannon_z (96%), meso_species_richness_z (86%), oomy_shannon_z (72%), coll_shannon_z (99%), coll_species_richness_z (100%)	6
RF con RegressorChain	euk_asv_richness_z (67%), euk_shannon_z (38%), oomy_asv_richness_z (75%)	3
MLP Estándar	fun_asv_richness_z (55%), orib_shannon_z (71%), orib_species_richness_z (65%)	3
Ridge (Regresión Individual)	cerc_shannon_z (58%), fun_shannon_z (47%)	2
RF Individual	cerc_asv_richness_z (44%)	1

*Los targets marcados con asterisco (coll_shannon_z , coll_species_richness_z) tienen R^2 medio negativo (-0.19 y -0.30) incluso con el modelo ganador — ningún modelo alcanza un ajuste aceptable ahí. "Ganar" en estos casos significa ser el menos malo, no un modelo fiable; se recomienda no usar sus predicciones para conclusiones ecológicas sin más validación, o tratarlos como targets no modelizables con los datos actuales.

Para los dos targets prioritarios del TFG (earthworm_shannon_z , earthworm_richness_z), la elección de XGBoost Multisalida queda respaldada de forma robusta por las 101 semillas (92-94% de dominancia), consistente con el resultado global.

Siguiente paso: antes de fijar esta tabla como definitiva, confirmar que XGB_OPTIMAL_PARAMS no tiene subsample / colsample_* en 1.0 de forma que anule por completo la influencia de random_state en XGBoost Individual (ver aviso técnico en el desarrollo de esta prueba) — de ser así, no afecta a esta conclusión (que se basa en XGBoost Multisalida, no en el individual), pero conviene dejarlo anotado como limitación conocida del experimento.

11.3.7. Notebooks para la prueba de ensamblado (Prueba IV)

11.3.7.1. Notebook de ensamblado de predicciones sin meta-modelos (Prueba IV.I)

Prueba 4 - Variantes de ensamblado de modelos

Este notebook hace lo siguiente:

1. Carga los modelos entrenados del pipeline SOB4ES: 3 familias single-target (Ridge, RF, XGBoost) y 5 familias multisalida (RF, XGBoost, RegressorChain, MLP, MLP con loss personalizada).
2. Ejecuta el smoke test habitual sobre **todo eval.csv** (no una muestra).
3. Separa eval.csv en meta-train/holdout, y calcula sobre el meta-train los pesos, ranking y coeficientes necesarios para las variantes que se calibran con datos.
4. Compara, todas sobre el mismo holdout, **5 formas de combinar los modelos base**:
 - **Media simple**: peso igual para todos.
 - **Media ponderada (R2)**: pesos proporcionales al R2 de cada modelo.
 - **Top-k**: media simple, pero solo entre los K_TOP mejores modelos.
 - **Mediana**: mas robusta a un modelo atípico (como Ridge) que la media.
 - **Stacking convexo**: pesos $w \geq 0$, $\sum(w) = 1$, optimizados en meta-train.

Por que hay un holdout (y no se evalua todo sobre eval.csv completo)

La media simple y la mediana no calibran nada con datos, así que podrían evaluarse sobre las 65 filas de eval.csv sin problema. Pero la media ponderada, el top-k y el stacking convexo **si calibran algo con datos** (pesos, ranking, coeficientes): si se calibraran y evaluaran sobre las mismas filas, el resultado quedaría inflado (el metodo "ha visto" la respuesta correcta de antemano). Para que las 5 variantes sean comparables entre si en igualdad de condiciones, todas se evalúan sobre el mismo holdout que no se ha usado para calibrar nada.

1. Configuración y carga de librerías

- **MODELS_DIR**: carpeta donde estan guardados los modelos entrenados.
- **EVAL_PATH**: ruta a eval.csv.
- **FEATURES_AUTORIZADAS**: las 34 variables de entrada (mismo orden que en el resto del pipeline).
- **TARGETS_ORDER**: los 21 targets **en el mismo orden** en que los modelos multisalida fueron entrenados (necesario para poder indexar su salida). Si no lo recuerdas de memoria, miralo en el notebook de entrenamiento multisalida (columna y usada en .fit).
- **TARGETS_TO_TEST**: el subconjunto de targets con los que se prueba este ensamblado (por defecto los dos prioritarios + uno mas a elegir).
- **SHOW_ENSEMBLE_MEMBERS**: si es **True**, la seccion 3b imprime el R2 de cada miembro individual de cada familia multisalida antes de promediar (util para depurar). No afecta a los resultados finales, es solo informativo.

```
In [1]: import json
import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.optimize import minimize
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Librerías cargadas correctamente...")

MODELS_DIR = Path("output/models")
EVAL_PATH = Path("input/eval.csv")

TARGETS_ORDER = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
    'zn_z',
    'soil_ph_z',
    'plot_total_organic_c_z',
    'plot_total_n_z',
    'gee_temp_media_C_z',
    'gee_humedad_rel_pct_z',
    'gee_ndvi_verano_z',
]
```

```
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

# Targets con los que se prueba el ensamblado por media + otros métodos.
TARGETS_TO_TEST = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

SHOW_ENSEMBLE_MEMBERS = False # True para depurar el detalle de cada miembro de cada familia

META_TEST_SIZE = 0.3 # proporción de eval.csv reservada como holdout para evaluar las variantes
K_TOP = 4 # número de modelos que se quedan en la variante top-k

assert all(t in TARGETS_ORDER for t in TARGETS_TO_TEST), (
    "Algun target de TARGETS_TO_TEST no está en TARGETS_ORDER - revisa la configuración."
)
```

Librerías cargadas correctamente...

2.- Carga de eval.csv

```
In [2]: eval_df = pd.read_csv(EVAL_PATH)

missing_features = [f for f in FEATURES_AUTORIZADAS if f not in eval_df.columns]
missing_targets = [t for t in TARGETS_ORDER if t not in eval_df.columns]
if missing_features:
    print(f"[AVISO] Faltan {len(missing_features)} features en eval.csv: {missing_features[:5]}...")
if missing_targets:
    print(f"[AVISO] Faltan {len(missing_targets)} targets en eval.csv: {missing_targets[:5]}...")

X_eval = eval_df[FEATURES_AUTORIZADAS]
y_eval = eval_df[TARGETS_ORDER]

print(f"eval.csv: {eval_df.shape[0]} filas, {eval_df.shape[1]} columnas")
print(f"X_eval: {X_eval.shape} | y_eval: {y_eval.shape}")

eval.csv: 65 filas, 71 columnas
X_eval: (65, 34) | y_eval: (65, 21)
```

3.- Definición y carga de los modelos base

Todos los modelos son ensamblados internos de varios miembros entrenados por separado, **salvo Ridge, que es un único modelo por target (sin ensamblado)**. El resto - single-target (RF single-target, XGBoost single-target, un ensamblado por target) y multisalida (RF, XGBoost, RegressorChain, MLP, MLP custom loss, un ensamblado por familia) - si son ensamblados: 5 miembros en todos salvo RegressorChain, que tiene 10.

`EnsembleWrapper` carga todos los miembros disponibles de una familia (y, si es single-target, de un target concreto) y promedia su predicción, de forma que el resto del notebook trata cada familia como un único modelo ya consolidado:

- Si es **single-target**, cada miembro ya predice directamente el target → se promedian las predicciones tal cual.
- Si es **multisalida**, cada miembro predice los 21 targets a la vez → primero se selecciona la columna del target pedido en cada miembro y después se promedia entre miembros.

Notas sobre el formato de cada tipo de modelo:

- **Ridge, RF y XGBoost** (single y multi) se guardan como `.pkl` y se cargan igual, con `joblib.load`.
- **MLP y MLP custom loss**: cada miembro tiene sus propios pesos (`.pt`, un `state_dict`), pero el config (`.pkl`) con la arquitectura usada al entrenar es **único por familia** (no uno por miembro): todos los miembros de "MLP multisalida" comparten el mismo config, y lo mismo para "MLP custom loss".

Si un fichero (o un miembro del ensamblado) no existe o falla la carga, se **omite con un aviso** en vez de interrumpir el notebook, así puedes ejecutar esto aunque todavía no tengas todos los modelos entrenados.

```
In [3]: def load_sklearn(path):
        return joblib.load(path)

def load_mlp_member(weights_path, config_path):
    import torch
    import torch.nn as nn

    config = joblib.load(config_path)

    def _cfg_get(key, default):
        # El config puede venir como dict o como un objeto (p.ej. dataclass/Namespace).
        if isinstance(config, dict):
            return config.get(key, default)
        return getattr(config, key, default)

    class GenericMLP(nn.Module):
        def __init__(self, input_dim, hidden_dims, output_dim, dropout=0.0):
```

```

        super().__init__()
        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers.append(nn.Linear(prev_dim, h))
            layers.append(nn.ReLU())
            if dropout > 0:
                layers.append(nn.Dropout(dropout))
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))
        self.red = nn.Sequential(*layers)

    def forward(self, x):
        return self.red(x)

    def predict(self, X):
        self.eval()
        with torch.no_grad():
            X_t = torch.as_tensor(np.asarray(X), dtype=torch.float32)
            return self.red(X_t).numpy()

model = GenericMLP(
    input_dim=cfg_get("input_dim", len(FEATURES_AUTORIZADAS)),
    hidden_dims=cfg_get("hidden", [64, 32]),
    output_dim=cfg_get("output_dim", len(TARGETS_ORDER)),
    dropout=cfg_get("dropout", 0.0),
)
state_dict = torch.load(Path(weights_path), map_location="cpu")
model.load_state_dict(state_dict)
model.eval()
return model

class EnsembleWrapper:
    '''Modelo (single-target o multialida) guardado como ensamblado de varios miembros
    entrenados por separado. predict() promedia la predicción de todos los miembros
    disponibles. Si falta algún miembro (archivo no encontrado, error de carga),
    simplemente se promedia con los que si se cargaron.

    kind="single": cada miembro predice directamente el target → se promedian tal cual.
    kind="multi": cada miembro predice todos los targets a la vez → primero se
    selecciona la columna del target pedido en cada miembro y despues se promedia.
    '''

    def __init__(self, key, label, members, kind, targets_order=None):
        self.key = key
        self.label = label
        self.members = members
        self.kind = kind
        self.targets_order = targets_order

    def predict(self, X, target=None):
        member_preds = []
        for model in self.members:
            preds = np.asarray(model.predict(X))
            if self.kind == "multi":
                if preds.ndim == 1:
                    preds = preds.reshape(-1, 1)
                    idx = self.targets_order.index(target)
                    preds = preds[:, idx]
                else:
                    preds = preds.reshape(-1)
            member_preds.append(preds)
        return np.mean(np.column_stack(member_preds), axis=1)

def load_ensemble_members(loader, path_template, n_members, loader_kwargs=None, **template_kwargs):
    '''Carga los n_members ficheros de un ensamblado, tolerando fallos parciales.
    loader_kwargs son argumentos fijos que se pasan a loader() en cada miembro (p.ej.
    un config compartido por toda la familia, en vez de uno por miembro).
    Devuelve (members, errors), donde errors es una lista de (índice, tipo_error, mensaje).
    '''
    loader_kwargs = loader_kwargs or {}
    members = []
    errors = []
    for i in range(n_members):
        path = Path(path_template.format(i=i, **template_kwargs))
        try:
            members.append(loader(path, **loader_kwargs))
        except Exception as e:
            errors.append((i, type(e).__name__, str(e)))
    return members, errors

SINGLE_MODEL_CONFIGS = [
    {"key": "ridge", "label": "Ridge", "loader": load_skllearn, "path_template": str(MODELS_DIR / "reg/ridge_prod_{target}.pkl"), "n_members": 1},
    {"key": "rf_single", "label": "RF single-target", "loader": load_skllearn, "path_template": str(MODELS_DIR / "rf/rf_prod_{target}_m{i}.pkl"), "n_members": 1},
    {"key": "xgb_single", "label": "XGBoost single-target", "loader": load_skllearn, "path_template": str(MODELS_DIR / "xgb/xgb_prod_{target}_exp{i}.pkl"), "n_members": 1}
]

MULTI_MODEL_CONFIGS = [
    {"key": "rf_multi", "label": "RF multialida", "loader": load_skllearn, "path_template": str(MODELS_DIR / "rf_multi/rf_multi_m{i}.pkl"), "n_members": 1},
    {"key": "xgb_multi", "label": "XGBoost multialida", "loader": load_skllearn, "path_template": str(MODELS_DIR / "xgb_multi/xgb_multi_exp{i}.pkl"), "n_members": 1},
    {"key": "regchain", "label": "RegressorChain", "loader": load_skllearn, "path_template": str(MODELS_DIR / "chain/chain_{i}.pkl"), "n_members": 1},
    {"key": "mlp_multi", "label": "MLP multialida", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp/mlp_prod_{i}.pt"), "n_members": 1},
    {"loader_kwargs": {"config_path": str(MODELS_DIR / "mlp/mlp_config.pkl")}},
    {"key": "mlp_custom", "label": "MLP custom loss", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp_custom/mlp_custom_{i}.pt"), "n_members": 1},
    {"loader_kwargs": {"config_path": str(MODELS_DIR / "mlp_custom/mlp_custom_config.pkl")}},
]

loaded_single = {} # (key, target) → EnsembleWrapper
loaded_multi = {} # key → EnsembleWrapper

for cfg in SINGLE_MODEL_CONFIGS:
    target in TARGETS_TO_TEST:
        members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
                                                loader_kwargs=cfg.get("loader_kwargs"), target=target)
        if members:
            loaded_single[cfg["key"], target] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="single")
            print(f"OK {cfg['label']:22s} [{target}] cargado ({len(members)}/{cfg['n_members']} miembros)")
            if errors:
                print(f" [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][1]}: {errors[0][2]}")
        else:
            print(f"FALLO {cfg['label']:22s} [{target}] omitido → no se cargo ningun miembro 0/{cfg['n_members']}")
            if errors:
                print(f" → primer error: {errors[0][1]}: {errors[0][2]}")

for cfg in MULTI_MODEL_CONFIGS:
```

```

members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
                                       loader_kwargs=cfg.get("loader_kwargs"))

if members:
    loaded_multi[cfg["key"]] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="multi", targets_order=TARGETS_ORDER)
    print(f"OK      {cfg['label']:22s} cargado ({len(members)}/{cfg['n_members']} miembros)")
    if errors:
        print(f"      [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][1]}: {errors[0][2]}")
    else:
        print(f"FALLO {cfg['label']:22s} omitido → no se cargo ningun miembro (0/{cfg['n_members']})")
    if errors:
        print(f"      → primer error: {errors[0][1]}: {errors[0][2]}")

print(f"\nModelos disponibles: {len(loaded_single)} single-target (por target) + {len(loaded_multi)} familias multisalida")

OK Ridge      [nematode_shannon_z] cargado (1/1 miembros)
OK Ridge      [macro_shannon_z] cargado (1/1 miembros)
OK Ridge      [earthworm_shannon_z] cargado (1/1 miembros)
OK Ridge      [orib_shannon_z] cargado (1/1 miembros)
OK Ridge      [meso_shannon_z] cargado (1/1 miembros)
OK Ridge      [coll_shannon_z] cargado (1/1 miembros)
OK Ridge      [bac_shannon_z] cargado (1/1 miembros)
OK Ridge      [fun_shannon_z] cargado (1/1 miembros)
OK Ridge      [euk_shannon_z] cargado (1/1 miembros)
OK Ridge      [oomy_shannon_z] cargado (1/1 miembros)
OK Ridge      [cerc_shannon_z] cargado (1/1 miembros)
OK Ridge      [macro_order_richness_z] cargado (1/1 miembros)
OK Ridge      [earthworm_richness_z] cargado (1/1 miembros)
OK Ridge      [orib_species_richness_z] cargado (1/1 miembros)
OK Ridge      [meso_species_richness_z] cargado (1/1 miembros)
OK Ridge      [coll_species_richness_z] cargado (1/1 miembros)
OK Ridge      [bac_asv_richness_z] cargado (1/1 miembros)
OK Ridge      [fun_asv_richness_z] cargado (1/1 miembros)
OK Ridge      [euk_asv_richness_z] cargado (1/1 miembros)
OK Ridge      [oomy_asv_richness_z] cargado (1/1 miembros)
OK Ridge      [cerc_asv_richness_z] cargado (1/1 miembros)
OK RF single-target [nematode_shannon_z] cargado (5/5 miembros)
OK RF single-target [macro_shannon_z] cargado (5/5 miembros)
OK RF single-target [earthworm_shannon_z] cargado (5/5 miembros)
OK RF single-target [orib_shannon_z] cargado (5/5 miembros)
OK RF single-target [meso_shannon_z] cargado (5/5 miembros)
OK RF single-target [coll_shannon_z] cargado (5/5 miembros)
OK RF single-target [bac_shannon_z] cargado (5/5 miembros)
OK RF single-target [fun_shannon_z] cargado (5/5 miembros)
OK RF single-target [euk_shannon_z] cargado (5/5 miembros)
OK RF single-target [oomy_shannon_z] cargado (5/5 miembros)
OK RF single-target [cerc_shannon_z] cargado (5/5 miembros)
OK RF single-target [macro_order_richness_z] cargado (5/5 miembros)
OK RF single-target [earthworm_richness_z] cargado (5/5 miembros)
OK RF single-target [orib_species_richness_z] cargado (5/5 miembros)
OK RF single-target [meso_species_richness_z] cargado (5/5 miembros)
OK RF single-target [coll_species_richness_z] cargado (5/5 miembros)
OK RF single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [nematode_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_order_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK RF multisalida cargado (5/5 miembros)
OK XGBoost multisalida cargado (5/5 miembros)
OK RegressorChain cargado (10/10 miembros)
OK MLP multisalida cargado (5/5 miembros)
OK MLP custom loss cargado (5/5 miembros)

```

Modelos disponibles: 63 single-target (por target) + 5 familias multisalida

3b.- (Opcional) Inspeccion de miembros individuales de cada ensamblado

Solo se ejecuta si `SHOW_ENSEMBLE_MEMBERS = True` en la seccion 1. Util para depurar si un miembro concreto (de un modelo single-target o de una familia multisalida) esta dando resultados muy distintos al resto antes de que se pierda esa informacion al promediar. No afecta a los resultados de las secciones siguientes.

```

In [4]: if SHOW_ENSEMBLE_MEMBERS:
        print("--- Modelos single-target ---")
        for (key, target), wrapper in loaded_single.items():
            y_true_t = y_eval[target].values
            print(f"\n{wrapper.Label} [{target}] ({len(wrapper.members)} miembros cargados)")
            for i, model in enumerate(wrapper.members):
                preds = np.asarray(model.predict(X_eval)).reshape(-1)
                r2 = r2_score(y_true_t, preds)
                print(f"  miembro {i} | R2={r2:.4f}")

        print("\n--- Familias multisalida ---")
        for key, wrapper in loaded_multi.items():
            print(f"\n{wrapper.Label} ({len(wrapper.members)} miembros cargados)")
            for target in TARGETS_TO_TEST:
                idx = TARGETS_ORDER.index(target)
                y_true_t = y_eval[target].values
                for i, model in enumerate(wrapper.members):
                    preds = np.asarray(model.predict(X_eval))
                    if preds.ndim == 1:
                        preds = preds.reshape(-1, 1)
                    preds_t = preds[:, idx]
                    r2 = r2_score(y_true_t, preds_t)
                    print(f"  miembro {i} | {target:22s} | R2={r2:.4f}")
            else:
                print("SHOW_ENSEMBLE_MEMBERS = False → seccion omitida (cambia el flag en la seccion 1 para activarla).")

```

SHOW_ENSEMBLE_MEMBERS = False → seccion omitida (cambia el flag en la seccion 1 para activarla).

4.- Smoke test sobre eval.csv completo

Mismas comprobaciones que el smoke test habitual de los notebooks de modelo (predice sin error, sin NaN/Inf, tamaño de salida correcto), pero aquí sobre **todas** las filas de eval.csv en vez de una muestra pequeña. Se reutilizan las predicciones para las secciones siguientes, así no se predice dos veces. Cada familia multisalida ya llega aquí como una única predicción promediada entre sus miembros.

```
In [5]: def smoke_test(wrapper, X, y_true, target):
    try:
        preds = wrapper.predict(X, target=target)
    except Exception as e:
        print(f"[SMOKE TEST] {wrapper.Label:22s} | {target:22s} | ERROR al predecir -> {type(e).__name__}: {e}")
        return None

    preds = np.asarray(preds, dtype=float).reshape(-1)
    shape_ok = preds.shape[0] == len(X)
    n_nan = int(np.isnan(preds).sum())
    n_inf = int(np.isinf(preds).sum())
    ok = shape_ok and n_nan == 0 and n_inf == 0
    r2 = r2_score(y_true, preds) if ok else float("nan")
    status = "OK" if ok else "REVISAR"
    print(f"[SMOKE TEST] {wrapper.Label:22s} | {target:22s} | filas={len(preds):5d} | "
          f"NaN={n_nan} | Inf={n_inf} | R2={r2: .4f} | {status}")
    return preds if ok else None

# predictions[target][label] = np.array de predicciones (solo si paso el smoke test)
predictions = {} for t in TARGETS_TO_TEST:

for target in TARGETS_TO_TEST:
    y_true_t = y_eval[target].values
    print(f"\n--- {target} ---")

    for (key, t), wrapper in loaded_single.items():
        if t != target:
            continue
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
        if preds is not None:
            predictions[target][wrapper.Label] = preds

    for key, wrapper in loaded_multi.items():
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
        if preds is not None:
            predictions[target][wrapper.Label] = preds

--- nematode_shannon_z ---
[SMOKE TEST] Ridge | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0112 | OK
[SMOKE TEST] RF single-target | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1366 | OK
[SMOKE TEST] XGBoost single-target | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1637 | OK
[SMOKE TEST] RF multisalida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1545 | OK
[SMOKE TEST] XGBoost multisalida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2228 | OK
[SMOKE TEST] RegressorChain | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1467 | OK
[SMOKE TEST] MLP multisalida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0829 | OK
[SMOKE TEST] MLP custom loss | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0668 | OK

--- macro_shannon_z ---
[SMOKE TEST] Ridge | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1257 | OK
[SMOKE TEST] RF single-target | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2797 | OK
[SMOKE TEST] XGBoost single-target | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2601 | OK
[SMOKE TEST] RF multisalida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2080 | OK
[SMOKE TEST] XGBoost multisalida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3771 | OK
[SMOKE TEST] RegressorChain | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2804 | OK
[SMOKE TEST] MLP multisalida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2396 | OK
[SMOKE TEST] MLP custom loss | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2704 | OK

--- earthworm_shannon_z ---
[SMOKE TEST] Ridge | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2395 | OK
[SMOKE TEST] RF single-target | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5059 | OK
[SMOKE TEST] XGBoost single-target | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4946 | OK
[SMOKE TEST] RF multisalida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4160 | OK
[SMOKE TEST] XGBoost multisalida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5375 | OK
[SMOKE TEST] RegressorChain | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4876 | OK
[SMOKE TEST] MLP multisalida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3538 | OK
[SMOKE TEST] MLP custom loss | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3770 | OK

--- orib_shannon_z ---
[SMOKE TEST] Ridge | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1137 | OK
[SMOKE TEST] RF single-target | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1644 | OK
[SMOKE TEST] XGBoost single-target | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1147 | OK
[SMOKE TEST] RF multisalida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1693 | OK
[SMOKE TEST] XGBoost multisalida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2275 | OK
[SMOKE TEST] RegressorChain | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2069 | OK
[SMOKE TEST] MLP multisalida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2250 | OK
[SMOKE TEST] MLP custom loss | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2552 | OK

--- meso_shannon_z ---
[SMOKE TEST] Ridge | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2499 | OK
[SMOKE TEST] RF single-target | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1895 | OK
[SMOKE TEST] XGBoost single-target | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1868 | OK
[SMOKE TEST] RF multisalida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1178 | OK
[SMOKE TEST] XGBoost multisalida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2861 | OK
[SMOKE TEST] RegressorChain | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1640 | OK
[SMOKE TEST] MLP multisalida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.4925 | OK
[SMOKE TEST] MLP custom loss | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.4289 | OK

--- coll_shannon_z ---
[SMOKE TEST] Ridge | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.4998 | OK
[SMOKE TEST] RF single-target | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.4168 | OK
[SMOKE TEST] XGBoost single-target | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.3312 | OK
[SMOKE TEST] RF multisalida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1953 | OK
[SMOKE TEST] XGBoost multisalida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.3229 | OK
[SMOKE TEST] RegressorChain | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.6087 | OK
[SMOKE TEST] MLP multisalida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-1.2378 | OK
[SMOKE TEST] MLP custom loss | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-1.2015 | OK

--- bac_shannon_z ---
[SMOKE TEST] Ridge | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0812 | OK
[SMOKE TEST] RF single-target | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0037 | OK
[SMOKE TEST] XGBoost single-target | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0092 | OK
[SMOKE TEST] RF multisalida | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0290 | OK
[SMOKE TEST] XGBoost multisalida | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0151 | OK
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
[SMOKE TEST] RegressorChain          | bac_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.0288 | OK
[SMOKE TEST] MLP multisalida         | bac_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0334 | OK
[SMOKE TEST] MLP custom loss         | bac_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.1218 | OK

--- fun_shannon_z ---
[SMOKE TEST] Ridge                   | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0161 | OK
[SMOKE TEST] RF single-target        | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0337 | OK
[SMOKE TEST] XGBoost single-target   | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0206 | OK
[SMOKE TEST] RF multisalida          | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0127 | OK
[SMOKE TEST] XGBoost multisalida     | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0502 | OK
[SMOKE TEST] RegressorChain          | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0219 | OK
[SMOKE TEST] MLP multisalida         | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.1246 | OK
[SMOKE TEST] MLP custom loss         | fun_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2=-0.0672 | OK

--- euk_shannon_z ---
[SMOKE TEST] Ridge                   | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.0407 | OK
[SMOKE TEST] RF single-target        | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.1575 | OK
[SMOKE TEST] XGBoost single-target   | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.0912 | OK
[SMOKE TEST] RF multisalida          | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.1378 | OK
[SMOKE TEST] XGBoost multisalida     | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.0886 | OK
[SMOKE TEST] RegressorChain          | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.1624 | OK
[SMOKE TEST] MLP multisalida         | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.1451 | OK
[SMOKE TEST] MLP custom loss         | euk_shannon_z          | filas= 65 | NaN=0 | Inf=0 | R2= 0.1025 | OK

--- oomy_shannon_z ---
[SMOKE TEST] Ridge                   | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0857 | OK
[SMOKE TEST] RF single-target        | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0998 | OK
[SMOKE TEST] XGBoost single-target   | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0965 | OK
[SMOKE TEST] RF multisalida          | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.1252 | OK
[SMOKE TEST] XGBoost multisalida     | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0620 | OK
[SMOKE TEST] RegressorChain          | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0913 | OK
[SMOKE TEST] MLP multisalida         | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.1362 | OK
[SMOKE TEST] MLP custom loss         | oomy_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0429 | OK

--- cerc_shannon_z ---
[SMOKE TEST] Ridge                   | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2= 0.0155 | OK
[SMOKE TEST] RF single-target        | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0404 | OK
[SMOKE TEST] XGBoost single-target   | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0414 | OK
[SMOKE TEST] RF multisalida          | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0041 | OK
[SMOKE TEST] XGBoost multisalida     | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0330 | OK
[SMOKE TEST] RegressorChain          | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0006 | OK
[SMOKE TEST] MLP multisalida         | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0855 | OK
[SMOKE TEST] MLP custom loss         | cerc_shannon_z         | filas= 65 | NaN=0 | Inf=0 | R2=-0.0613 | OK

--- macro_order_richness_z ---
[SMOKE TEST] Ridge                   | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1521 | OK
[SMOKE TEST] RF single-target        | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3058 | OK
[SMOKE TEST] XGBoost single-target   | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3082 | OK
[SMOKE TEST] RF multisalida          | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2532 | OK
[SMOKE TEST] XGBoost multisalida     | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4013 | OK
[SMOKE TEST] RegressorChain          | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4069 | OK
[SMOKE TEST] MLP multisalida         | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3447 | OK
[SMOKE TEST] MLP custom loss         | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3581 | OK

--- earthworm_richness_z ---
[SMOKE TEST] Ridge                   | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.2789 | OK
[SMOKE TEST] RF single-target        | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.5492 | OK
[SMOKE TEST] XGBoost single-target   | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.5171 | OK
[SMOKE TEST] RF multisalida          | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.4509 | OK
[SMOKE TEST] XGBoost multisalida     | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.5893 | OK
[SMOKE TEST] RegressorChain          | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.5359 | OK
[SMOKE TEST] MLP multisalida         | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.4628 | OK
[SMOKE TEST] MLP custom loss         | earthworm_richness_z   | filas= 65 | NaN=0 | Inf=0 | R2= 0.4854 | OK

--- orib_species_richness_z ---
[SMOKE TEST] Ridge                   | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0921 | OK
[SMOKE TEST] RF single-target        | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2407 | OK
[SMOKE TEST] XGBoost single-target   | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1259 | OK
[SMOKE TEST] RF multisalida          | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2386 | OK
[SMOKE TEST] XGBoost multisalida     | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2800 | OK
[SMOKE TEST] RegressorChain          | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2026 | OK
[SMOKE TEST] MLP multisalida         | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2783 | OK
[SMOKE TEST] MLP custom loss         | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2981 | OK

--- meso_species_richness_z ---
[SMOKE TEST] Ridge                   | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0959 | OK
[SMOKE TEST] RF single-target        | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0453 | OK
[SMOKE TEST] XGBoost single-target   | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0592 | OK
[SMOKE TEST] RF multisalida          | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0255 | OK
[SMOKE TEST] XGBoost multisalida     | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1146 | OK
[SMOKE TEST] RegressorChain          | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0400 | OK
[SMOKE TEST] MLP multisalida         | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2681 | OK
[SMOKE TEST] MLP custom loss         | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2198 | OK

--- coll_species_richness_z ---
[SMOKE TEST] Ridge                   | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.7341 | OK
[SMOKE TEST] RF single-target        | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5461 | OK
[SMOKE TEST] XGBoost single-target   | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5803 | OK
[SMOKE TEST] RF multisalida          | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2920 | OK
[SMOKE TEST] XGBoost multisalida     | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.6465 | OK
[SMOKE TEST] RegressorChain          | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5799 | OK
[SMOKE TEST] MLP multisalida         | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-2.0859 | OK
[SMOKE TEST] MLP custom loss         | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-1.9873 | OK

--- bac_asv_richness_z ---
[SMOKE TEST] Ridge                   | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0761 | OK
[SMOKE TEST] RF single-target        | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0004 | OK
[SMOKE TEST] XGBoost single-target   | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0014 | OK
[SMOKE TEST] RF multisalida          | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0044 | OK
[SMOKE TEST] XGBoost multisalida     | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0155 | OK
[SMOKE TEST] RegressorChain          | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0052 | OK
[SMOKE TEST] MLP multisalida         | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0799 | OK
[SMOKE TEST] MLP custom loss         | bac_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.1811 | OK

--- fun_asv_richness_z ---
[SMOKE TEST] Ridge                   | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0087 | OK
[SMOKE TEST] RF single-target        | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0144 | OK
[SMOKE TEST] XGBoost single-target   | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0242 | OK
[SMOKE TEST] RF multisalida          | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0060 | OK
[SMOKE TEST] XGBoost multisalida     | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0662 | OK
[SMOKE TEST] RegressorChain          | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2=-0.0186 | OK
[SMOKE TEST] MLP multisalida         | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0105 | OK
[SMOKE TEST] MLP custom loss         | fun_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0069 | OK

--- euk_asv_richness_z ---
[SMOKE TEST] Ridge                   | euk_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.0400 | OK
[SMOKE TEST] RF single-target        | euk_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.2360 | OK
[SMOKE TEST] XGBoost single-target   | euk_asv_richness_z     | filas= 65 | NaN=0 | Inf=0 | R2= 0.1671 | OK
```

```
[SMOKE TEST] RF multialida          | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1903 | OK
[SMOKE TEST] XGBoost multialida     | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2233 | OK
[SMOKE TEST] RegressorChain          | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2690 | OK
[SMOKE TEST] MLP multialida          | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2470 | OK
[SMOKE TEST] MLP custom loss         | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1970 | OK

--- oomy_asv_richness_z ---
[SMOKE TEST] Ridge                   | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1035 | OK
[SMOKE TEST] RF single-target        | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1895 | OK
[SMOKE TEST] XGBoost single-target   | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1776 | OK
[SMOKE TEST] RF multialida           | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1917 | OK
[SMOKE TEST] XGBoost multialida      | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1650 | OK
[SMOKE TEST] RegressorChain          | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2019 | OK
[SMOKE TEST] MLP multialida          | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0640 | OK
[SMOKE TEST] MLP custom loss         | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0188 | OK

--- cerc_asv_richness_z ---
[SMOKE TEST] Ridge                   | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1019 | OK
[SMOKE TEST] RF single-target        | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1207 | OK
[SMOKE TEST] XGBoost single-target   | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1227 | OK
[SMOKE TEST] RF multialida           | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1171 | OK
[SMOKE TEST] XGBoost multialida      | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0983 | OK
[SMOKE TEST] RegressorChain          | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1171 | OK
[SMOKE TEST] MLP multialida          | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0285 | OK
[SMOKE TEST] MLP custom loss         | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0845 | OK
```

5.- Particion meta-train / holdout

Las variantes de la seccion 7 (media ponderada, top-k, stacking convexo) **calibran algo con datos** (pesos, ranking, coeficientes), así que evaluarlas sobre las mismas filas que se usan para calibrarlas inflaría el resultado (el metodo "ha visto" la respuesta correcta de antemano). Por eso se separan las filas de `eval.csv` en dos bloques: uno para calibrar (meta-train) y otro para evaluar todo -- modelos individuales y las 5 variantes de combinacion por igual -- (holdout). La mediana y la media simple no calibran nada, pero se evaluan tambien sobre el holdout para que la comparacion final sea homogenea.

```
In [6]: idx_all = np.arange(len(eval_df))
        idx_train, idx_holdout = train_test_split(idx_all, test_size=META_TEST_SIZE, random_state=RANDOM_STATE)
```

```
print(f"Filas totales en eval.csv: {len(idx_all)}")
print(f"Filas para calibrar pesos/seleccion (meta-train): {len(idx_train)}")
print(f"Filas para evaluar todo (holdout): {len(idx_holdout)}")
```

```
Filas totales en eval.csv: 65
Filas para calibrar pesos/seleccion (meta-train): 45
Filas para evaluar todo (holdout): 20
```

6.- Calculo de pesos, ranking y coeficientes de stacking (sobre meta-train)

Para cada target:

- **R2 por modelo** en meta-train (base para las variantes ponderada y top-k).
- **Pesos de la media ponderada:** $\max(R2, 0)$ normalizado para sumar 1 (un modelo con R2 negativo en meta-train no deberia restar, así que se recorta a 0 antes de normalizar).
- **Ranking para top-k:** los K_TOP modelos con mejor R2 en meta-train.
- **Coeficientes del stacking con pesos no negativos que suman 1:** se resuelve un problema de optimizacion (SLSQP) que minimiza el error cuadratico en meta-train sujeto a $w \geq 0$ y $\sum(w) = 1$.

```
In [7]: def fit_convex_weights(X_meta_train, y_meta_train):
        '''Encuentra pesos w ≥ 0, sum(w) = 1, que minimizan el MSE en meta-train.'''
        n_models = X_meta_train.shape[1]
        w0 = np.full(n_models, 1.0 / n_models)

        def objective(w):
            preds = X_meta_train @ w
            return np.mean((preds - y_meta_train) ** 2)

        constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1.0}]
        bounds = [(0.0, 1.0)] * n_models

        result = minimize(objective, w0, method="SLSQP", bounds=bounds, constraints=constraints)
        if not result.success:
            print(f" [AVISO] Optimizacion de pesos no convergio ({result.message}); se usan pesos uniformes.")
            return w0
        return result.x

variant_info = {} # target → dict con model_labels, weights, top_k_labels, convex_weights

for target in TARGETS_TO_TEST:
    model_labels = list(predictions[target].keys())
    if not model_labels:
        print(f" [AVISO] {target}: no hay modelos disponibles, no se pueden calcular pesos.")
        continue

    X_full = np.column_stack([predictions[target][lbl] for lbl in model_labels])
    y_target = y_eval[target].values
    X_train, y_train = X_full[idx_train], y_target[idx_train]

    r2_train = {lbl: r2_score(y_train, X_train[:, i]) for i, lbl in enumerate(model_labels)}

    raw_weights = np.array([max(r2_train[lbl], 0.0) for lbl in model_labels])
    if raw_weights.sum() == 0:
        print(f" [AVISO] {target}: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.")
        weighted_weights = np.full(len(model_labels), 1.0 / len(model_labels))
    else:
        weighted_weights = raw_weights / raw_weights.sum()

    k = min(K_TOP, len(model_labels))
    top_k_labels = sorted(model_labels, key=lambda lbl: r2_train[lbl], reverse=True)[:k]

    convex_weights = fit_convex_weights(X_train, y_train)

    variant_info[target] = {
        "model_labels": model_labels,
        "r2_train": r2_train,
        "weighted_weights": weighted_weights,
        "top_k_labels": top_k_labels,
        "convex_weights": convex_weights,
    }

    print(f"\n--- {target} ---")
    print(f"R2 en meta-train por modelo:", {k: round(v, 3) for k, v in r2_train.items()})
    print(f"Pesos media ponderada: ", {lbl: round(w, 3) for lbl, w in zip(model_labels, weighted_weights)})
    print(f"Top-{k}: {top_k_labels}")
    print(f"Pesos stacking convexo: ", {lbl: round(w, 3) for lbl, w in zip(model_labels, convex_weights)})
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
--- nematode_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.038, 'RF single-target': 0.146, 'XGBoost single-target': 0.173, 'RF multisalida': 0.178, 'XGBoost multisalida': 0.201,
'RegressorChain': 0.144, 'MLP multisalida': 0.108, 'MLP custom loss': 0.067}
Pesos media ponderada: {'Ridge': np.float64(0.036), 'RF single-target': np.float64(0.138), 'XGBoost single-target': np.float64(0.164), 'RF multisalida': n
p.float64(0.169), 'XGBoost multisalida': np.float64(0.19), 'RegressorChain': np.float64(0.137), 'MLP multisalida': np.float64(0.103), 'MLP custom loss': np.floa
t64(0.063)}
Top-4: ['XGBoost multisalida', 'RF multisalida', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.319), 'XGBoost multisalida': np.float64(0.681), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- macro_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.084, 'RF single-target': 0.181, 'XGBoost single-target': 0.146, 'RF multisalida': 0.107, 'XGBoost multisalida': 0.305,
'RegressorChain': 0.151, 'MLP multisalida': 0.152, 'MLP custom loss': 0.158}
Pesos media ponderada: {'Ridge': np.float64(0.065), 'RF single-target': np.float64(0.141), 'XGBoost single-target': np.float64(0.114), 'RF multisalida': n
p.float64(0.083), 'XGBoost multisalida': np.float64(0.237), 'RegressorChain': np.float64(0.118), 'MLP multisalida': np.float64(0.118), 'MLP custom loss': np.flo
at64(0.123)}
Top-4: ['XGBoost multisalida', 'RF single-target', 'MLP custom loss', 'MLP multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.0), 'XGBoost multisalida': np.float64(1.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- earthworm_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.306, 'RF single-target': 0.529, 'XGBoost single-target': 0.526, 'RF multisalida': 0.419, 'XGBoost multisalida': 0.578,
'RegressorChain': 0.506, 'MLP multisalida': 0.427, 'MLP custom loss': 0.435}
Pesos media ponderada: {'Ridge': np.float64(0.082), 'RF single-target': np.float64(0.142), 'XGBoost single-target': np.float64(0.141), 'RF multisalida': n
p.float64(0.112), 'XGBoost multisalida': np.float64(0.155), 'RegressorChain': np.float64(0.136), 'MLP multisalida': np.float64(0.115), 'MLP custom loss': np.flo
at64(0.117)}
Top-4: ['XGBoost multisalida', 'RF single-target', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.0), 'XGBoost multisalida': np.float64(1.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- orib_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.163, 'RF single-target': 0.23, 'XGBoost single-target': 0.171, 'RF multisalida': 0.234, 'XGBoost multisalida': 0.331,
'RegressorChain': 0.293, 'MLP multisalida': 0.328, 'MLP custom loss': 0.35}
Pesos media ponderada: {'Ridge': np.float64(0.078), 'RF single-target': np.float64(0.11), 'XGBoost single-target': np.float64(0.081), 'RF multisalida': n
p.float64(0.111), 'XGBoost multisalida': np.float64(0.158), 'RegressorChain': np.float64(0.14), 'MLP multisalida': np.float64(0.156), 'MLP custom loss': np.floa
t64(0.166)}
Top-4: ['MLP custom loss', 'XGBoost multisalida', 'MLP multisalida', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.0), 'XGBoost multisalida': np.float64(0.413), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.587)}
[AVISO] meso_shannon_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- meso_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.419, 'RF single-target': -0.449, 'XGBoost single-target': -0.44, 'RF multisalida': -0.295, 'XGBoost multisalida': -0.5
38, 'RegressorChain': -0.455, 'MLP multisalida': -0.449, 'MLP custom loss': -0.399}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64(0.125), 'RF multisalida': n
p.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP multisalida': np.float64(0.125), 'MLP custom loss': np.flo
at64(0.125)}
Top-4: ['RF multisalida', 'MLP custom loss', 'Ridge', 'XGBoost single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.76), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.24)}
[AVISO] coll_shannon_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- coll_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.519, 'RF single-target': -0.515, 'XGBoost single-target': -0.408, 'RF multisalida': -0.253, 'XGBoost multisalida': -0.
372, 'RegressorChain': -0.76, 'MLP multisalida': -0.725, 'MLP custom loss': -0.57}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64(0.125), 'RF multisalida': n
p.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP multisalida': np.float64(0.125), 'MLP custom loss': np.flo
at64(0.125)}
Top-4: ['RF multisalida', 'XGBoost multisalida', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.871), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.129)}

--- bac_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.053, 'RF single-target': 0.03, 'XGBoost single-target': 0.037, 'RF multisalida': 0.063, 'XGBoost multisalida': 0.005,
'RegressorChain': 0.039, 'MLP multisalida': 0.016, 'MLP custom loss': -0.052}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.158), 'XGBoost single-target': np.float64(0.196), 'RF multisalida': np.
float64(0.352), 'XGBoost multisalida': np.float64(0.026), 'RegressorChain': np.float64(0.206), 'MLP multisalida': np.float64(0.082), 'MLP custom loss': np.floa
t64(0.0)}
Top-4: ['RF multisalida', 'RegressorChain', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.971), 'XGBoost multisalida': np.float64(0.029), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- fun_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.0, 'RF single-target': 0.011, 'XGBoost single-target': 0.024, 'RF multisalida': 0.012, 'XGBoost multisalida': 0.038,
'RegressorChain': 0.021, 'MLP multisalida': -0.105, 'MLP custom loss': -0.032}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.101), 'XGBoost single-target': np.float64(0.227), 'RF multisalida': np.
float64(0.115), 'XGBoost multisalida': np.float64(0.36), 'RegressorChain': np.float64(0.197), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64
(0.0)}
Top-4: ['XGBoost multisalida', 'XGBoost single-target', 'RegressorChain', 'RF multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.323), 'RF multisalida': np.fl
oat64(0.0), 'XGBoost multisalida': np.float64(0.677), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- euk_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.015, 'RF single-target': 0.241, 'XGBoost single-target': 0.184, 'RF multisalida': 0.164, 'XGBoost multisalida': 0.209,
'RegressorChain': 0.257, 'MLP multisalida': 0.239, 'MLP custom loss': 0.142}
Pesos media ponderada: {'Ridge': np.float64(0.01), 'RF single-target': np.float64(0.166), 'XGBoost single-target': np.float64(0.127), 'RF multisalida': n
p.float64(0.113), 'XGBoost multisalida': np.float64(0.144), 'RegressorChain': np.float64(0.177), 'MLP multisalida': np.float64(0.165), 'MLP custom loss': np.flo
at64(0.098)}
Top-4: ['RegressorChain', 'RF single-target', 'MLP multisalida', 'XGBoost multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.floa
t64(0.0), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.591), 'MLP multisalida': np.float64(0.409), 'MLP custom loss': np.float64(0.0)}

--- oomy_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.058, 'RF single-target': 0.064, 'XGBoost single-target': 0.087, 'RF multisalida': 0.096, 'XGBoost multisalida': 0.046,
'RegressorChain': 0.067, 'MLP multisalida': 0.09, 'MLP custom loss': -0.02}
Pesos media ponderada: {'Ridge': np.float64(0.115), 'RF single-target': np.float64(0.127), 'XGBoost single-target': np.float64(0.171), 'RF multisalida': n
p.float64(0.19), 'XGBoost multisalida': np.float64(0.09), 'RegressorChain': np.float64(0.131), 'MLP multisalida': np.float64(0.177), 'MLP custom loss': np.float
64(0.0)}
Top-4: ['RF multisalida', 'MLP multisalida', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.068), 'RF multisalida': np.fl
oat64(0.493), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.438), 'MLP custom loss': np.float64(0.
0)}
[AVISO] cerc_shannon_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- cerc_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.018, 'RF single-target': -0.092, 'XGBoost single-target': -0.112, 'RF multisalida': -0.072, 'XGBoost multisalida': -0.
038, 'RegressorChain': -0.023, 'MLP multisalida': -0.17, 'MLP custom loss': -0.116}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64(0.125), 'RF multisalida': n
p.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP multisalida': np.float64(0.125), 'MLP custom loss': np.flo
at64(0.125)}
Top-4: ['Ridge', 'RegressorChain', 'XGBoost multisalida', 'RF multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.59), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.flo
at64(0.0), 'XGBoost multisalida': np.float64(0.41), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- macro_order_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.072, 'RF single-target': 0.17, 'XGBoost single-target': 0.157, 'RF multisalida': 0.148, 'XGBoost multisalida': 0.27, 'R
egressorChain': 0.328, 'MLP multisalida': 0.247, 'MLP custom loss': 0.236}
Pesos media ponderada: {'Ridge': np.float64(0.044), 'RF single-target': np.float64(0.104), 'XGBoost single-target': np.float64(0.096), 'RF multisalida': n
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
p.float64(0.091), 'XGBoost multisalida': np.float64(0.166), 'RegressorChain': np.float64(0.202), 'MLP multisalida': np.float64(0.152), 'MLP custom loss': np.float64(0.145)}
Top-4:
['RegressorChain', 'XGBoost multisalida', 'MLP multisalida', 'MLP custom loss']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.878), 'MLP multisalida': np.float64(0.122), 'MLP custom loss': np.float64(0.0)}

--- earthworm_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.366, 'RF single-target': 0.558, 'XGBoost single-target': 0.52, 'RF multisalida': 0.453, 'XGBoost multisalida': 0.605, 'RegressorChain': 0.55, 'MLP multisalida': 0.522, 'MLP custom loss': 0.536}
Pesos media ponderada:
{'Ridge': np.float64(0.089), 'RF single-target': np.float64(0.136), 'XGBoost single-target': np.float64(0.126), 'RF multisalida': np.float64(0.11), 'XGBoost multisalida': np.float64(0.147), 'RegressorChain': np.float64(0.134), 'MLP multisalida': np.float64(0.127), 'MLP custom loss': np.float64(0.13)}
Top-4:
['XGBoost multisalida', 'RF single-target', 'RegressorChain', 'MLP custom loss']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.91), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.09)}

--- orib_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.128, 'RF single-target': 0.304, 'XGBoost single-target': 0.161, 'RF multisalida': 0.294, 'XGBoost multisalida': 0.372, 'RegressorChain': 0.257, 'MLP multisalida': 0.383, 'MLP custom loss': 0.412}
Pesos media ponderada:
{'Ridge': np.float64(0.056), 'RF single-target': np.float64(0.132), 'XGBoost single-target': np.float64(0.07), 'RF multisalida': np.float64(0.127), 'XGBoost multisalida': np.float64(0.161), 'RegressorChain': np.float64(0.111), 'MLP multisalida': np.float64(0.166), 'MLP custom loss': np.float64(0.178)}
Top-4:
['MLP custom loss', 'MLP multisalida', 'XGBoost multisalida', 'RF single-target']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.183), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.817)}
[AVISO] meso_species_richness_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- meso_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.217, 'RF single-target': -0.208, 'XGBoost single-target': -0.236, 'RF multisalida': -0.121, 'XGBoost multisalida': -0.27, 'RegressorChain': -0.193, 'MLP multisalida': -0.179, 'MLP custom loss': -0.135}
Pesos media ponderada:
{'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP multisalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4:
['RF multisalida', 'MLP custom loss', 'MLP multisalida', 'RegressorChain']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.537), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.463)}
[AVISO] coll_species_richness_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- coll_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.606, 'RF single-target': -0.629, 'XGBoost single-target': -0.639, 'RF multisalida': -0.368, 'XGBoost multisalida': -0.724, 'RegressorChain': -0.699, 'MLP multisalida': -1.088, 'MLP custom loss': -0.824}
Pesos media ponderada:
{'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP multisalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4:
['RF multisalida', 'Ridge', 'RF single-target', 'XGBoost single-target']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.837), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.163)}

--- bac_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.072, 'RF single-target': 0.032, 'XGBoost single-target': 0.02, 'RF multisalida': 0.038, 'XGBoost multisalida': 0.037, 'RegressorChain': 0.021, 'MLP multisalida': -0.021, 'MLP custom loss': -0.107}
Pesos media ponderada:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.217), 'XGBoost single-target': np.float64(0.137), 'RF multisalida': np.float64(0.255), 'XGBoost multisalida': np.float64(0.251), 'RegressorChain': np.float64(0.14), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4:
['RF multisalida', 'XGBoost multisalida', 'RF single-target', 'RegressorChain']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.506), 'XGBoost multisalida': np.float64(0.494), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- fun_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.068, 'RF single-target': 0.047, 'XGBoost single-target': 0.021, 'RF multisalida': 0.05, 'XGBoost multisalida': 0.007, 'RegressorChain': 0.053, 'MLP multisalida': 0.073, 'MLP custom loss': 0.068}
Pesos media ponderada:
{'Ridge': np.float64(0.176), 'RF single-target': np.float64(0.122), 'XGBoost single-target': np.float64(0.055), 'RF multisalida': np.float64(0.129), 'XGBoost multisalida': np.float64(0.017), 'RegressorChain': np.float64(0.136), 'MLP multisalida': np.float64(0.189), 'MLP custom loss': np.float64(0.175)}
Top-4:
['MLP multisalida', 'Ridge', 'MLP custom loss', 'RegressorChain']
Pesos stacking convexo:
{'Ridge': np.float64(0.224), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.534), 'MLP custom loss': np.float64(0.242)}

--- euk_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.087, 'RF single-target': 0.228, 'XGBoost single-target': 0.185, 'RF multisalida': 0.174, 'XGBoost multisalida': 0.299, 'RegressorChain': 0.296, 'MLP multisalida': 0.264, 'MLP custom loss': 0.2}
Pesos media ponderada:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.139), 'XGBoost single-target': np.float64(0.112), 'RF multisalida': np.float64(0.106), 'XGBoost multisalida': np.float64(0.182), 'RegressorChain': np.float64(0.18), 'MLP multisalida': np.float64(0.16), 'MLP custom loss': np.float64(0.122)}
Top-4:
['XGBoost multisalida', 'RegressorChain', 'MLP multisalida', 'RF single-target']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.399), 'RegressorChain': np.float64(0.423), 'MLP multisalida': np.float64(0.178), 'MLP custom loss': np.float64(0.0)}

--- oomv_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.056, 'RF single-target': 0.112, 'XGBoost single-target': 0.113, 'RF multisalida': 0.098, 'XGBoost multisalida': 0.087, 'RegressorChain': 0.129, 'MLP multisalida': -0.067, 'MLP custom loss': -0.087}
Pesos media ponderada:
{'Ridge': np.float64(0.094), 'RF single-target': np.float64(0.189), 'XGBoost single-target': np.float64(0.19), 'RF multisalida': np.float64(0.164), 'XGBoost multisalida': np.float64(0.146), 'RegressorChain': np.float64(0.218), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4:
['RegressorChain', 'XGBoost single-target', 'RF single-target', 'RF multisalida']
Pesos stacking convexo:
{'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.137), 'RegressorChain': np.float64(0.863), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- cerc_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.056, 'RF single-target': 0.021, 'XGBoost single-target': 0.044, 'RF multisalida': 0.053, 'XGBoost multisalida': -0.029, 'RegressorChain': 0.021, 'MLP multisalida': -0.019, 'MLP custom loss': -0.073}
Pesos media ponderada:
{'Ridge': np.float64(0.288), 'RF single-target': np.float64(0.106), 'XGBoost single-target': np.float64(0.226), 'RF multisalida': np.float64(0.272), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.109), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4:
['Ridge', 'RF multisalida', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo:
{'Ridge': np.float64(0.488), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.32), 'RF multisalida': np.float64(0.192), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
```

7.- Evaluación de todas las variantes sobre el holdout

Se calculan 5 formas de combinar los modelos disponibles, todas sobre las mismas filas de holdout:

1. **Media simple:** la de siempre, peso igual para todos.
2. **Media ponderada (R2):** pesos proporcionales al R2 de cada modelo en meta-train.
3. **Top-k:** media simple, pero solo entre los K_{TOP} mejores modelos.
4. **Mediana:** mas robusta a un modelo atípico (como Ridge) que la media.
5. **Stacking convexo:** combinacion lineal con pesos $w \geq 0$, $\sum(w) = 1$, optimizados en meta-train.

```
In [8]: holdout_predictions = {t: {} for t in TARGETS_TO_TEST}

for target in TARGETS_TO_TEST:
    if target not in variant_info:
```

```

        continue
    info = variant_info[target]
    model_labels = info["model_labels"]

    X_holdout = np.column_stack([predictions[target][lbl] for lbl in model_labels][idx_holdout])

    # Modelos individuales, recortados al holdout
    for i, lbl in enumerate(model_labels):
        holdout_predictions[target][lbl] = X_holdout[:, i]

    # 1. Media simple
    holdout_predictions[target]["Media simple"] = X_holdout.mean(axis=1)

    # 2. Media ponderada por R2
    holdout_predictions[target]["Media ponderada (R2)"] = X_holdout @ info["weighted_weights"]

    # 3. Top-k
    idx_top_k = [model_labels.index(lbl) for lbl in info["top_k_labels"]]
    holdout_predictions[target][f"Top-{len(idx_top_k)}"] = X_holdout[:, idx_top_k].mean(axis=1)

    # 4. Mediana
    holdout_predictions[target]["Mediana"] = np.median(X_holdout, axis=1)

    # 5. Stacking convexo (pesos ≥ 0, suman 1)
    holdout_predictions[target]["Stacking convexo"] = X_holdout @ info["convex_weights"]

```

8.- Comparativa: modelos individuales vs. las 5 variantes de combinacion

```

In [9]: COMBINATION_LABELS = {"Media simple", "Media ponderada (R2)", "Mediana", "Stacking convexo"}
        COMBINATION_LABELS |= {lbl for lbl in holdout_predictions.get(TARGETS_TO_TEST[0], {}) if lbl.startswith("Top-")}

        rows = []
        for target in TARGETS_TO_TEST:
            if target not in variant_info:
                continue
            y_true_holdout = y_eval[target].values[idx_holdout]
            for label, preds in holdout_predictions[target].items():
                rmse = mean_squared_error(y_true_holdout, preds) ** 0.5
                rows.append({
                    "target": target,
                    "modelo": label,
                    "R2": r2_score(y_true_holdout, preds),
                    "RMSE": rmse,
                    "MAE": mean_absolute_error(y_true_holdout, preds),
                    "tipo": "combinacion" if label in COMBINATION_LABELS else "individual",
                })

        if not rows:
            print("[AVISO] No hay predicciones de holdout disponibles todavía. "
                  "Revisa que los modelos se hayan cargado (sección 3), que el smoke test haya "
                  "pasado (sección 4) y que los pesos se hayan calculado (sección 6).")
            results_df = pd.DataFrame(columns=["target", "modelo", "R2", "RMSE", "MAE", "tipo"])
        else:
            results_df = pd.DataFrame(rows).sort_values(["target", "R2"], ascending=[True, False]).reset_index(drop=True)

        results_df

```

```

Out[9]:

```

	target	modelo	R2	RMSE	MAE	tipo
0	bac_asv_richness_z	XGBoost multisalida	-0.089010	0.855012	0.627395	individual
1	bac_asv_richness_z	XGBoost single-target	-0.090581	0.855628	0.626837	individual
2	bac_asv_richness_z	Ridge	-0.097045	0.858160	0.620105	individual
3	bac_asv_richness_z	Stacking convexo	-0.108851	0.862766	0.633342	combinacion
4	bac_asv_richness_z	Media ponderada (R2)	-0.116122	0.865590	0.634717	combinacion
...	...	...	...	...	...	...
268	orib_species_richness_z	Top-4	-0.138941	0.956782	0.712581	combinacion
269	orib_species_richness_z	XGBoost multisalida	-0.181075	0.974319	0.686219	individual
270	orib_species_richness_z	Stacking convexo	-0.218877	0.989788	0.722996	combinacion
271	orib_species_richness_z	MLP multisalida	-0.244074	0.999967	0.752825	individual
272	orib_species_richness_z	MLP custom loss	-0.271366	1.010875	0.731857	individual

273 rows x 6 columns

8.1.- Resumen agregado por target

Con muchos targets a la vez, `results_df` tiene demasiadas filas para leerla a ojo. Esta tabla se queda con una fila por target: el mejor modelo individual, la mejor variante de combinacion, cual de los dos gana, y si el mejor R2 de ese target es positivo (es decir, si hay alguna señal predictiva real) o no.

R2 <= 0 significa que ni el mejor modelo hace mejor que predecir siempre la media -- en esos targets el problema no es que metodo de ensamblado usar, sino que los modelos base no tienen señal útil que combinar.

```

In [10]: if results_df.empty:
        print("[AVISO] results_df esta vacio. Ejecuta primero las secciones 3-8.")
        summary_df = pd.DataFrame(columns=[
            "target", "mejor_individual", "R2_mejor_individual",
            "mejor_combinacion", "R2_mejor_combinacion", "gana_combinacion", "mejor_R2_global",
        ])
    else:
        summary_rows = []
        for target in results_df["target"].unique():
            sub = results_df[results_df["target"] == target]

            individuales = sub[sub["tipo"] == "individual"].sort_values("R2", ascending=False)
            combinaciones = sub[sub["tipo"] != "individual"].sort_values("R2", ascending=False)

            mejor_individual = individuales.iloc[0] if not individuales.empty else None
            mejor_combinacion = combinaciones.iloc[0] if not combinaciones.empty else None

            r2_individual = mejor_individual["R2"] if mejor_individual is not None else float("-inf")
            r2_combinacion = mejor_combinacion["R2"] if mejor_combinacion is not None else float("-inf")

            summary_rows.append({
                "target": target,
                "mejor_individual": mejor_individual["modelo"] if mejor_individual is not None else None,
                "R2_mejor_individual": r2_individual,
                "mejor_combinacion": mejor_combinacion["modelo"] if mejor_combinacion is not None else None,
            })

```

```

        "R2_mejor_combinacion": r2_combinacion,
        "gana_combinacion": bool(r2_combinacion > r2_individual),
        "mejor_R2_global": max(r2_individual, r2_combinacion),
    })

summary_df = pd.DataFrame(summary_rows).sort_values("mejor_R2_global", ascending=False).reset_index(drop=True)

n_targets = len(summary_df)
n_gana_combo = int(summary_df["gana_combinacion"].sum())
n_r2_positivo = int((summary_df["mejor_R2_global"] > 0).sum())

print(f"Targets evaluados: {n_targets}")
print(f"Targets donde alguna combinacion supera al mejor individual: "
      f"{n_gana_combo}/{n_targets} ({n_gana_combo / n_targets:.0%})")
print(f"Targets con señal predictiva real (mejor R2 > 0): "
      f"{n_r2_positivo}/{n_targets} ({n_r2_positivo / n_targets:.0%})")
print(f"Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): "
      f"{n_targets - n_r2_positivo}")

summary_df

Targets evaluados: 21
Targets donde alguna combinacion supera al mejor individual: 4/21 (19%)
Targets con señal predictiva real (mejor R2 > 0): 15/21 (71%)
Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): 6
Out[10]:

```

	target	mejor_individual	R2_mejor_individual	mejor_combinacion	R2_mejor_combinacion	gana_combinacion	mejor_R2_global
0	macro_order_richness_z	XGBoost multisalida	0.509020	Top-4	0.450304	False	0.509020
1	earthworm_richness_z	XGBoost multisalida	0.508708	Stacking convexo	0.496985	False	0.508708
2	cerc_asv_richness_z	XGBoost multisalida	0.404936	Mediana	0.324205	False	0.404936
3	macro_shannon_z	RegressorChain	0.372242	Stacking convexo	0.370340	False	0.372242
4	earthworm_shannon_z	RegressorChain	0.366510	Top-4	0.351522	False	0.366510
5	oomy_asv_richness_z	RF multisalida	0.302448	Stacking convexo	0.279431	False	0.302448
6	nematode_shannon_z	XGBoost multisalida	0.280802	Stacking convexo	0.238977	False	0.280802
7	euk_asv_richness_z	RF single-target	0.219539	Mediana	0.217647	False	0.219539
8	cerc_shannon_z	RF multisalida	0.121626	Mediana	0.069647	False	0.121626
9	oomy_shannon_z	MLP multisalida	0.109946	Stacking convexo	0.084285	False	0.109946
10	euk_shannon_z	RF multisalida	0.103819	Mediana	0.057647	False	0.103819
11	meso_species_richness_z	RF single-target	0.053125	Mediana	0.043725	False	0.053125
12	coll_shannon_z	RF multisalida	-0.058985	Mediana	0.032299	True	0.032299
13	meso_shannon_z	RegressorChain	0.023491	Mediana	-0.058045	False	0.023491
14	orib_shannon_z	MLP custom loss	-0.001389	Media ponderada (R2)	0.011586	True	0.011586
15	coll_species_richness_z	RF multisalida	-0.069098	Stacking convexo	-0.015200	True	-0.015200
16	fun_shannon_z	Ridge	-0.045406	Mediana	-0.095235	False	-0.045406
17	orib_species_richness_z	RF multisalida	-0.054500	Mediana	-0.070825	False	-0.054500
18	fun_asv_richness_z	Ridge	-0.087334	Stacking convexo	-0.086309	True	-0.086309
19	bac_asv_richness_z	XGBoost multisalida	-0.089010	Stacking convexo	-0.108851	False	-0.089010
20	bac_shannon_z	RegressorChain	-0.129290	Top-4	-0.244984	False	-0.129290

9.- Visualización

Barras de R2 por modelo y target sobre el holdout. Las variantes de combinacion se resaltan con borde negro para distinguirlas de los modelos individuales.

```

In [11]: if results_df.empty:
print("[AVISO] results_df esta vacio, no hay nada que graficar. Ejecuta primero las secciones 3-8.")
else:
    targets_present = [t for t in TARGETS_TO_TEST if t in results_df["target"].unique()]
    palette = sns.color_palette("Set2", n_colors=max(len(targets_present), 3))
    TARGET_COLORS = dict(zip(targets_present, palette))

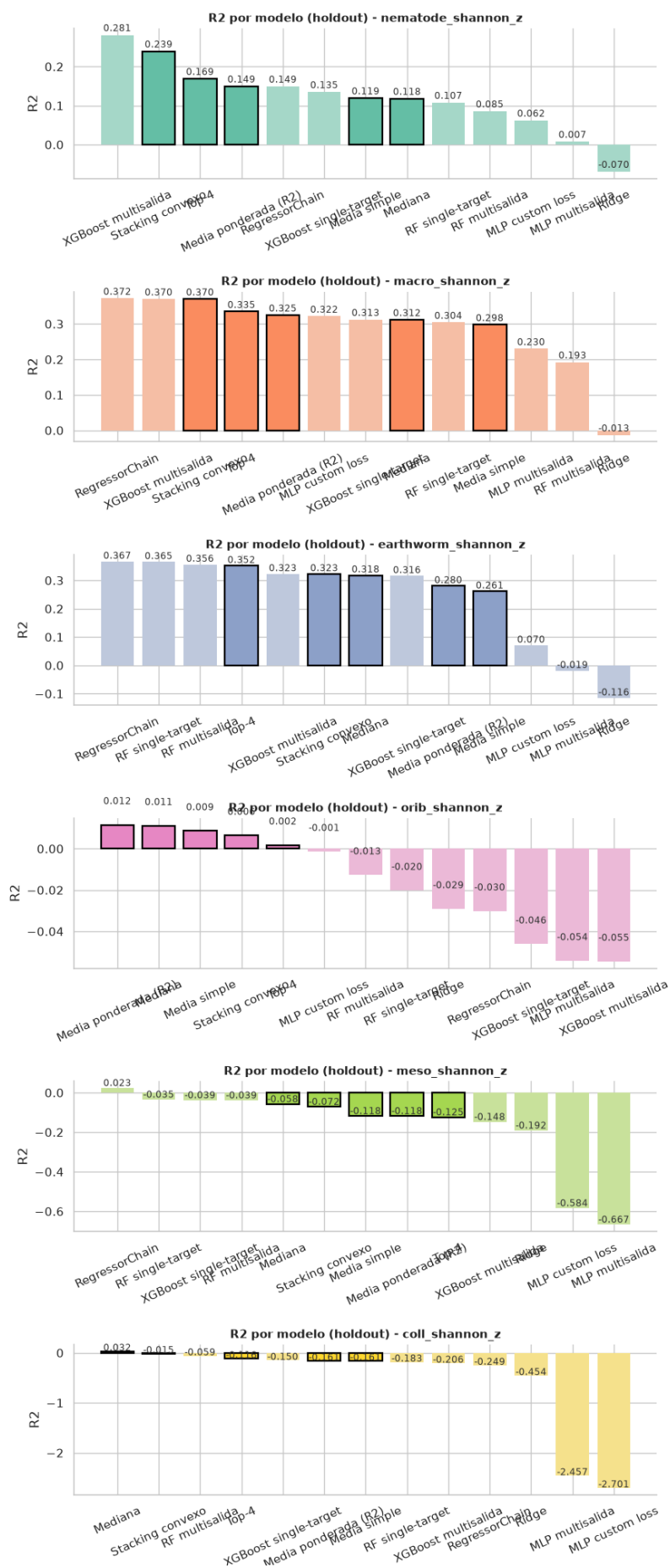
    fig, axes = plt.subplots(len(targets_present), 1, figsize=(9, 3.4 * len(targets_present)), squeeze=False)

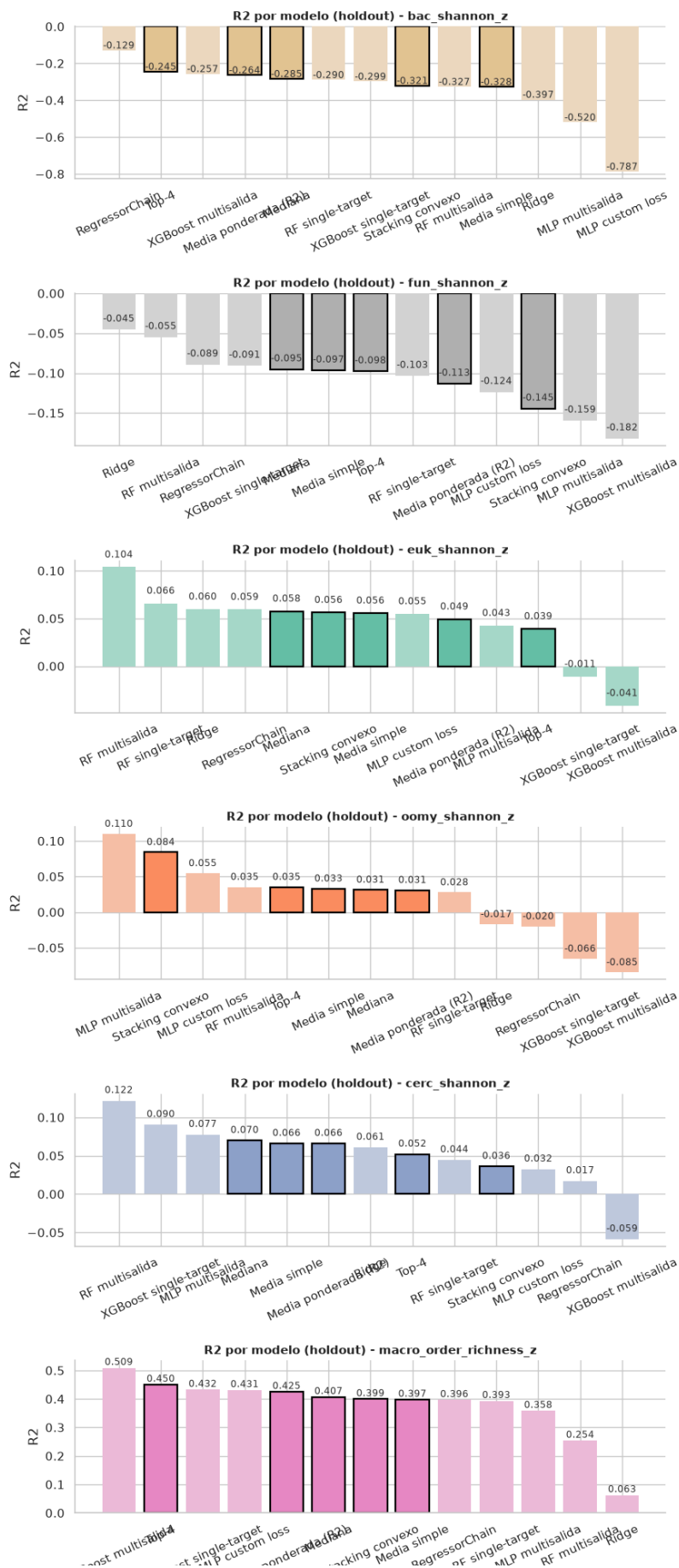
    for ax, target in zip(axes[:, 0], targets_present):
        sub = results_df[results_df["target"] == target].sort_values("R2", ascending=False)
        base_color = TARGET_COLORS[target]
        is_highlighted = sub["tipo"] != "individual"
        colors = [base_color if h else sns.light_palette(base_color, n_colors=3)[1] for h in is_highlighted]
        edgecolors = ["black" if h else "none" for h in is_highlighted]
        linewidths = [1.6 if h else 0 for h in is_highlighted]

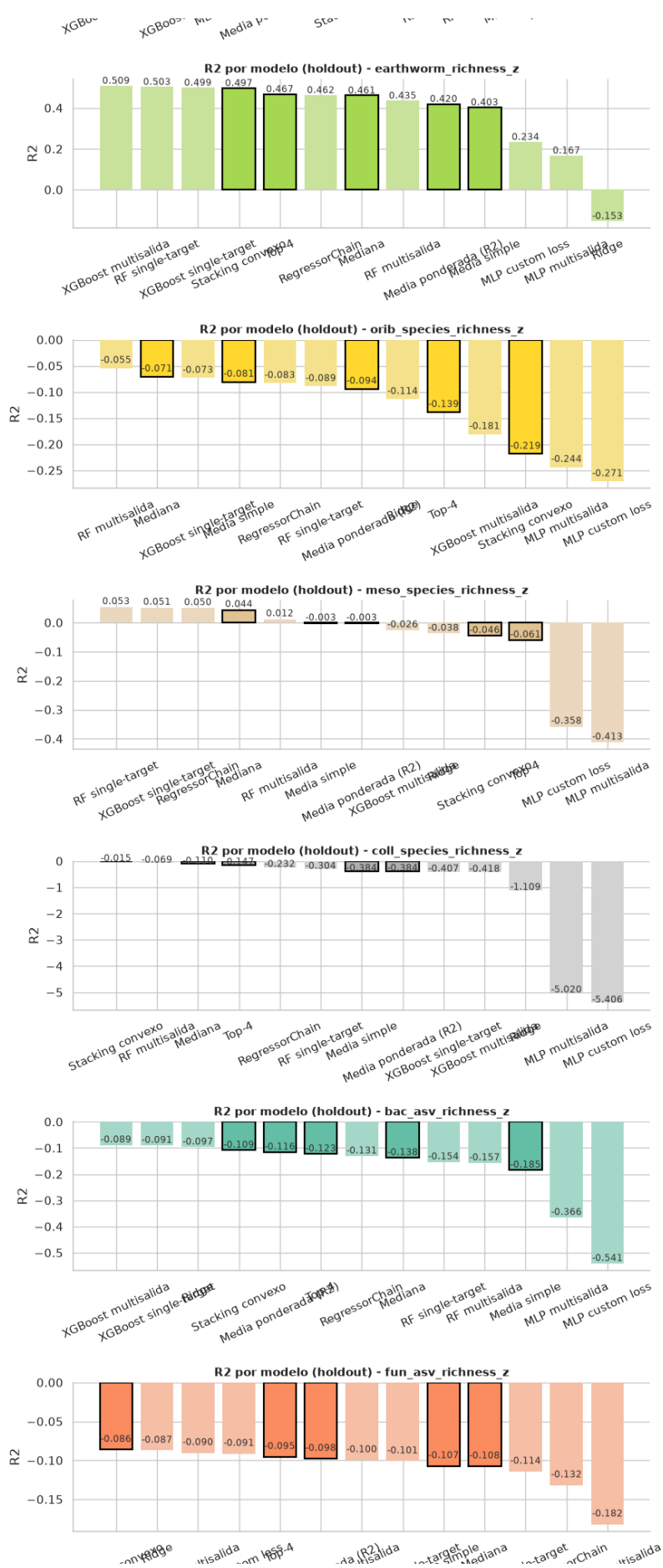
        bars = ax.bar(sub["modelo"], sub["R2"], color=colors, edgecolor=edgecolors, linewidth=linewidths)
        for bar, r2 in zip(bars, sub["R2"]):
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f"{r2:.3f}",
                    ha="center", fontsize=8.5, color="#333333")
        ax.set_title(f"R2 por modelo (holdout) - {target}", fontsize=11, fontweight="bold")
        ax.set_ylabel("R2")
        ax.tick_params(axis="x", rotation=25)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)

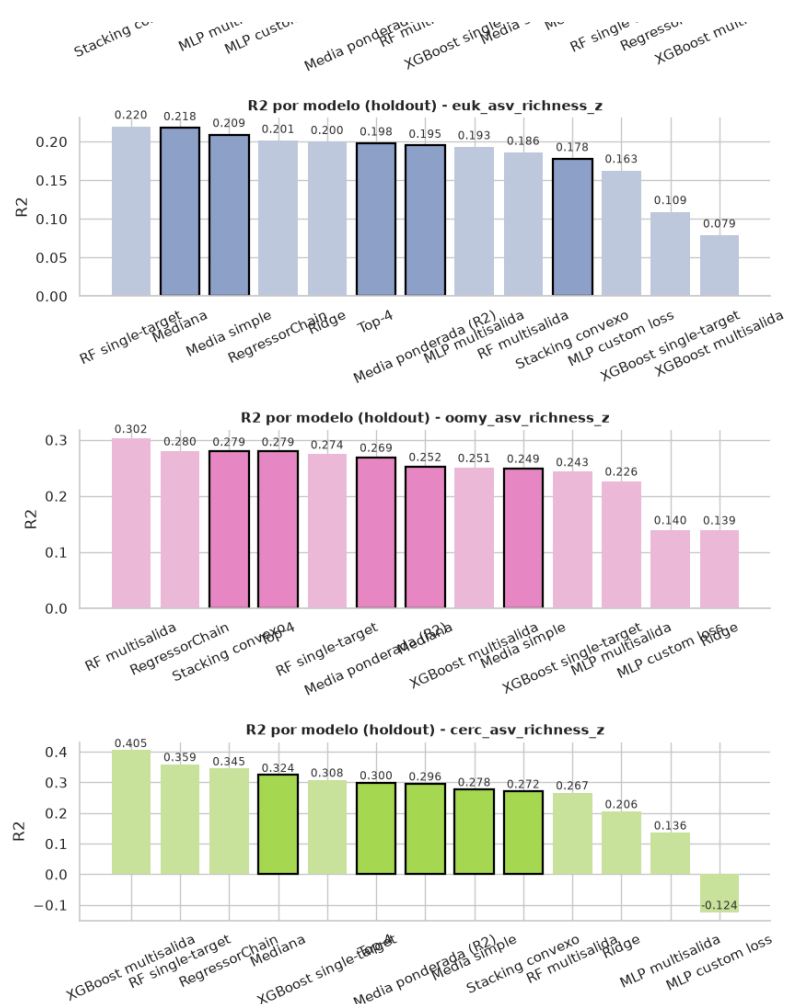
    fig.tight_layout()
    fig.savefig("output/r2_comparativa_sin_modelos.png", dpi=300, bbox_inches="tight")
    plt.show()

```









10. Conclusiones

[COMPLETAR tras ejecutar con los modelos y `eval.csv` reales]

- ¿Alguna de las variantes (ponderada, top-k, mediana, stacking convexo) mejora la media simple en el holdout? ¿Cual gana en cada target?
- ¿Alguna variante supera al mejor modelo individual (no solo a la media simple)? Si ninguna lo hace, puede que para estos targets compense mas quedarse con el mejor modelo individual que con cualquier combinacion.
- ¿La variante ganadora es consistente entre targets, o cambia segun cual se pruebe?
- ¿Los pesos de la media ponderada y del stacking convexo (seccion 6) coinciden en darle poco peso a los modelos mas debiles (p.ej. Ridge, MLP)?
- Dado que el holdout es pequeño, ¿los resultados son estables si cambias `RANDOM_STATE` en el `train_test_split`? Merece la pena repetir con varias semillas antes de decidir cual variante llevar a produccion.
- Dentro de cada ensamblado interno (tanto en los modelos single-target como en las familias multisalida), ¿los miembros son consistentes entre si (ver seccion 3b) o hay alguno que desentona y conviene revisar?

11.3.7.2. Notebook de ensamblado de predicciones con meta-modelos (Prueba IV.II)

Prueba 5 - Ensamblado por stacking (meta-modelo)

Este notebook parte de la Prueba 4 (ensamblado por media) y prueba una alternativa: en vez de promediar a mano las predicciones de los modelos base, se entrena un **meta-modelo** que aprende, a partir de los valores reales, como combinar mejor las salidas de los modelos base.

Pasos:

1. Carga los mismos modelos base que la Prueba 4 (single-target + familias multisalida, cada una como ensamblado interno de varios miembros).
2. Ejecuta el smoke test habitual sobre todo `eval.csv`.
3. Separa `eval.csv` en dos bloques: uno para **entrenar el meta-modelo** (meta-train) y otro para **evaluarlo** (meta-test / holdout), para no medir el meta-modelo con los mismos datos con los que se ha entrenado.
4. Para cada target, entrena un meta-modelo (Ridge) que recibe como entrada las predicciones de los modelos base y como salida el valor real.
5. Repite el smoke test sobre el meta-modelo, esta vez sobre el holdout.
6. Compara el error del meta-modelo frente a cada modelo base y frente al ensamblado simple por media (Prueba 4), todo medido sobre el mismo holdout.

1.- Configuración y carga de librerías

Misma configuración que la Prueba 4, pero en este caso se añaden las siguientes variables los meta-modelos:

- `META_TEST_SIZE`: proporción de filas de `eval.csv` que se reservan como holdout para evaluar los meta-modelos (el resto se usa para entrenarlos).
- `META_MODEL_FACTORIES`: diccionario `{nombre: funcion_que_crea_el_modelo}`, con varios meta-modelos candidatos a probar en paralelo (Ridge, Lasso, ElasticNet, regresión lineal sin regularizar, y un Random Forest como candidato no lineal). Todos se entrenan y evalúan igual, y se comparan al final en la misma tabla. Se puede añadir o quitar candidatos libremente.

Nota: Si el tamaño general del dataset es bajo, hacer uso solamente de los datos de eval.csv. Los resultados de esta prueba de deben de tomar un uno o varios granos de sal, para tener datos realmente verídicos se recomendaría realizar una sub-prueba aún más grande. Si se obtienen más datos lo recomendable sería recurrir a CV (Validación Cruzada) o a hacer uso de un valor para el hold-out aún más grande.

```
In [1]: import json
import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import ElasticNet, Lasso, LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Librerías cargadas correctamente...")

MODELS_DIR = Path("output/models")
EVAL_PATH = Path("input/eval.csv")

TARGETS_ORDER = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
    'silt_content_z',
    'sand_content_z',
    'aggregate_stability_z',
    'bulk_density_z',
    'soil_moisture_z',
    'as_z',
    'cu_z',
    'k_z',
    'mo_z',
    'ni_z',
    'p_z',
    'pb_z',
    'zn_z',
    'soil_ph_z',
    'plot_total_organic_c_z',
    'plot_total_n_z',
    'gee_temp_media_C_z',
    'gee_humedad_rel_pct_z',
    'gee_ndvi_verano_z',
    'dem_elevacion_m_z',
    'dem_pendiente_deg_z',
    'dem_orientacion_deg_z',
    'eu_clay_content_z',
    'eu_sand_content_z',
    'eu_silt_content_z',
    'eu_water_holding_capacity_z',

```

```

'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

TARGETS_TO_TEST = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

META_TEST_SIZE = 0.3 # proporcion de eval.csv reservada como holdout para evaluar los meta-modelos

# Varios meta-modelos candidatos, todos entrenados y evaluados igual. Anade/quita libremente.
META_MODEL_FACTORIES = {
    "Ridge": lambda: Ridge(random_state=RANDOM_STATE),
    "Lasso": lambda: Lasso(alpha=0.1, random_state=RANDOM_STATE),
    "ElasticNet": lambda: ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=RANDOM_STATE),
    "Linear": lambda: LinearRegression(),
    "RandomForest": lambda: RandomForestRegressor(n_estimators=200, max_depth=3, random_state=RANDOM_STATE),
}

assert all(t in TARGETS_ORDER for t in TARGETS_TO_TEST), (
    "Algun target de TARGETS_TO_TEST no esta en TARGETS_ORDER, revisa la configuracion..."
)

```

Librerías cargadas correctamente...

2.- Carga de eval.csv

```

In [2]: eval_df = pd.read_csv(EVAL_PATH)

missing_features = [f for f in FEATURES_AUTORIZADAS if f not in eval_df.columns]
missing_targets = [t for t in TARGETS_ORDER if t not in eval_df.columns]
if missing_features:
    print(f"[AVISO] Faltan {len(missing_features)} features en eval.csv: {missing_features[:5]}...")
if missing_targets:
    print(f"[AVISO] Faltan {len(missing_targets)} targets en eval.csv: {missing_targets[:5]}...")

X_eval = eval_df[FEATURES_AUTORIZADAS]
y_eval = eval_df[TARGETS_ORDER]

print(f"eval.csv: {eval_df.shape[0]} filas, {eval_df.shape[1]} columnas")
print(f"X_eval: {X_eval.shape} | y_eval: {y_eval.shape}")

eval.csv: 65 filas, 71 columnas
X_eval: (65, 34) | y_eval: (65, 21)

```

3.- Definición y carga de los modelos base

Igual que en la Prueba 4: todos los modelos son ensamblados internos de varios miembros entrenados por separado, **salvo Ridge, que es un unico modelo por target (sin ensamblado)**.

El resto de modelos, single-target (RF single-target, XGBoost single-target, un ensamblado por target) y multisalida (RF, XGBoost, RegressorChain, MLP, MLP custom loss, un ensamblado por familia), si son ensamblados: 5 miembros en todos salvo RegressorChain, que tiene 10.

`EnsembleWrapper`: promedia internamente los miembros de cada ensamblado (para Ridge, con un unico miembro, no cambia el resultado), así que a partir de aquí cada modelo base (single o multi) se trata como un unico modelo consolidado.

Ridge, RF y XGBoost se guardan como `.pkl` (misma carga con `joblib.load`). El MLP y el MLP con loss personalizada tienen sus propios pesos por miembro (`.pt`), pero el config (`.pkl`) con la arquitectura usada al entrenar es **unico por familia** (no uno por miembro).

```

In [3]: def load_sklearn(path):
    return joblib.load(path)

def load_mlp_member(weights_path, config_path):
    import torch
    import torch.nn as nn

    config = joblib.load(config_path)

    def _cfg_get(key, default):
        # El config puede venir como dict o como un objeto (p.ej. dataclass/Namespace).
        if isinstance(config, dict):
            return config.get(key, default)
        return getattr(config, key, default)

    class GenericMLP(nn.Module):
        def __init__(self, input_dim, hidden_dims, output_dim, dropout=0.0):
            super().__init__()
            layers = []
            prev_dim = input_dim
            for h in hidden_dims:
                layers.append(nn.Linear(prev_dim, h))
                layers.append(nn.ReLU())
                if dropout > 0:
                    layers.append(nn.Dropout(dropout))
                prev_dim = h
            layers.append(nn.Linear(prev_dim, output_dim))

```

```

self.red = nn.Sequential(*layers)

def forward(self, x):
    return self.red(x)

def predict(self, X):
    self.eval()
    with torch.no_grad():
        X_t = torch.as_tensor(np.asarray(X), dtype=torch.float32)
        return self.red(X_t).numpy()

model = GenericMLP(
    input_dim=_cfg.get("input_dim", len(FEATURES_AUTORIZADAS)),
    hidden_dims=_cfg.get("hidden", [64, 32]),
    output_dim=_cfg.get("output_dim", len(TARGETS_ORDER)),
    dropout=_cfg.get("dropout", 0.8),
)

state_dict = torch.load(Path(weights_path), map_location="cpu")
model.load_state_dict(state_dict)
model.eval()
return model

class EnsembleWrapper:
    '''Modelo (single-target o multisalida) guardado como ensamblado de varios miembros
    entrenados por separado. predict() promedia la prediccion de todos los miembros
    disponibles. Si falta algun miembro (fichero no encontrado, error de carga),
    simplemente se promedia con los que si se cargaron.

    kind="single": cada miembro predice directamente el target → se promedian tal cual.
    kind="multi": cada miembro predice todos los targets a la vez → primero se
    selecciona la columna del target pedido en cada miembro y despues se promedia.
    '''

    def __init__(self, key, label, members, kind, targets_order=None):
        self.key = key
        self.label = label
        self.members = members
        self.kind = kind
        self.targets_order = targets_order

    def predict(self, X, target=None):
        member_preds = []
        for model in self.members:
            preds = np.asarray(model.predict(X))
            if self.kind == "multi":
                if preds.ndim == 1:
                    preds = preds.reshape(-1, 1)
                    idx = self.targets_order.index(target)
                    preds = preds[:, idx]
                else:
                    preds = preds.reshape(-1)
            member_preds.append(preds)
        return np.mean(np.column_stack(member_preds), axis=1)

def load_ensemble_members(loader, path_template, n_members, loader_kwargs=None, **template_kwargs):
    '''Carga los n_members ficheros de un ensamblado, tolerando fallos parciales.
    loader_kwargs son argumentos fijos que se pasan a loader() en cada miembro (p.ej.
    un config compartido por toda la familia, en vez de uno por miembro).
    Devuelve (members, errors), donde errors es una lista de (indice, tipo_error, mensaje).
    '''
    loader_kwargs = loader_kwargs or {}
    members = []
    errors = []
    for i in range(n_members):
        path = Path(path_template.format(i=i, **template_kwargs))
        try:
            members.append(loader(path, **loader_kwargs))
        except Exception as e:
            errors.append((i, type(e).__name__, str(e)))
    return members, errors

SINGLE_MODEL_CONFIGS = [
    {"key": "ridge", "label": "Ridge", "loader": load_sklearn, "path_template": str(MODELS_DIR / "reg/ridge_prod_{target}.pkl"), "n_members": 1},
    {"key": "rf_single", "label": "RF single-target", "loader": load_sklearn, "path_template": str(MODELS_DIR / "rf/rf_prod_{target}_m{i}.pkl"), "n_members": 1},
    {"key": "xgb_single", "label": "XGBoost single-target", "loader": load_sklearn, "path_template": str(MODELS_DIR / "xgb/xgb_prod_{target}_exp{i}.pkl"), "n_members": 1}
]

MULTI_MODEL_CONFIGS = [
    {"key": "rf_multi", "label": "RF multisalida", "loader": load_sklearn, "path_template": str(MODELS_DIR / "rf_multi/rf_multi_m{i}.pkl"), "n_members": 1},
    {"key": "xgb_multi", "label": "XGBoost multisalida", "loader": load_sklearn, "path_template": str(MODELS_DIR / "xgb_multi/xgb_multi_exp{i}.pkl"), "n_members": 1},
    {"key": "regchain", "label": "RegressorChain", "loader": load_sklearn, "path_template": str(MODELS_DIR / "chain/chain_{i}.pkl"), "n_members": 1},
    {"key": "mlp_multi", "label": "MLP multisalida", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp/mlp_prod_{i}.pt"), "n_members": 1},
    {"key": "mlp_custom", "label": "MLP custom loss", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp_custom/mlp_custom_{i}.pt"), "n_members": 1},
    {"key": "mlp_custom_loss", "label": "MLP custom loss", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp_custom/mlp_custom_{i}.pt"), "n_members": 1}
]

loaded_single = {} # (key, target) → EnsembleWrapper
loaded_multi = {} # key → EnsembleWrapper

for cfg in SINGLE_MODEL_CONFIGS:
    for target in TARGETS_TO_TEST:
        members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
        loader_kwargs=cfg.get("loader_kwargs"), target=target)
        if members:
            loaded_single[cfg["key"], target] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="single")
            print(f"OK {cfg['label']:22s} [{target}] cargado ({len(members)}/{cfg['n_members']} miembros)")
            if errors:
                print(f" [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][1]}: {errors[0][2]}")
        else:
            print(f"FALLO {cfg['label']:22s} [{target}] omitido → no se cargo ningun miembro (0/{cfg['n_members']})")
            if errors:
                print(f" → primer error: {errors[0][1]}: {errors[0][2]}")

for cfg in MULTI_MODEL_CONFIGS:
    members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
    loader_kwargs=cfg.get("loader_kwargs"))
    if members:
        loaded_multi[cfg["key"]] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="multi", targets_order=TARGETS_ORDER)
        print(f"OK {cfg['label']:22s} cargado ({len(members)}/{cfg['n_members']} miembros)")
        if errors:
            print(f" [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][1]}: {errors[0][2]}")
    else:
        print(f"FALLO {cfg['label']:22s} omitido → no se cargo ningun miembro (0/{cfg['n_members']})")
        if errors:
            print(f" → primer error: {errors[0][1]}: {errors[0][2]}")

```

```
print(f"      → primer error: {errors[0][1]}: {errors[0][2]}")

print(f"\nModelos base disponibles: {len(loaded_single)} single-target (por target) + {len(loaded_multi)} familias multisalida")

OK Ridge [nematode_shannon_z] cargado (1/1 miembros)
OK Ridge [macro_shannon_z] cargado (1/1 miembros)
OK Ridge [earthworm_shannon_z] cargado (1/1 miembros)
OK Ridge [orib_shannon_z] cargado (1/1 miembros)
OK Ridge [meso_shannon_z] cargado (1/1 miembros)
OK Ridge [coll_shannon_z] cargado (1/1 miembros)
OK Ridge [bac_shannon_z] cargado (1/1 miembros)
OK Ridge [fun_shannon_z] cargado (1/1 miembros)
OK Ridge [euk_shannon_z] cargado (1/1 miembros)
OK Ridge [oomy_shannon_z] cargado (1/1 miembros)
OK Ridge [cerc_shannon_z] cargado (1/1 miembros)
OK Ridge [macro_order_richness_z] cargado (1/1 miembros)
OK Ridge [earthworm_richness_z] cargado (1/1 miembros)
OK Ridge [orib_species_richness_z] cargado (1/1 miembros)
OK Ridge [meso_species_richness_z] cargado (1/1 miembros)
OK Ridge [coll_species_richness_z] cargado (1/1 miembros)
OK Ridge [bac_asv_richness_z] cargado (1/1 miembros)
OK Ridge [fun_asv_richness_z] cargado (1/1 miembros)
OK Ridge [euk_asv_richness_z] cargado (1/1 miembros)
OK Ridge [oomy_asv_richness_z] cargado (1/1 miembros)
OK Ridge [cerc_asv_richness_z] cargado (1/1 miembros)
OK RF single-target [nematode_shannon_z] cargado (5/5 miembros)
OK RF single-target [macro_shannon_z] cargado (5/5 miembros)
OK RF single-target [earthworm_shannon_z] cargado (5/5 miembros)
OK RF single-target [orib_shannon_z] cargado (5/5 miembros)
OK RF single-target [meso_shannon_z] cargado (5/5 miembros)
OK RF single-target [coll_shannon_z] cargado (5/5 miembros)
OK RF single-target [bac_shannon_z] cargado (5/5 miembros)
OK RF single-target [fun_shannon_z] cargado (5/5 miembros)
OK RF single-target [euk_shannon_z] cargado (5/5 miembros)
OK RF single-target [oomy_shannon_z] cargado (5/5 miembros)
OK RF single-target [cerc_shannon_z] cargado (5/5 miembros)
OK RF single-target [macro_order_richness_z] cargado (5/5 miembros)
OK RF single-target [earthworm_richness_z] cargado (5/5 miembros)
OK RF single-target [orib_species_richness_z] cargado (5/5 miembros)
OK RF single-target [meso_species_richness_z] cargado (5/5 miembros)
OK RF single-target [coll_species_richness_z] cargado (5/5 miembros)
OK RF single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [nematode_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_order_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK RF multisalida cargado (5/5 miembros)
OK XGBoost multisalida cargado (5/5 miembros)
OK RegressorChain cargado (10/10 miembros)
OK MLP multisalida cargado (5/5 miembros)
OK MLP custom loss cargado (5/5 miembros)
```

Modelos base disponibles: 63 single-target (por target) + 5 familias multisalida

4.- Smoke test sobre eval.csv completo

Igual que en la Prueba 4: se predice una unica vez sobre todo `eval.csv` y se reutilizan las predicciones despues, tanto para entrenar como para evaluar el meta-modelo (respetando la particion train/holdout de la seccion 5).

```
In [4]: def smoke_test(wrapper, X, y_true, target):
    try:
        preds = wrapper.predict(X, target=target)
    except Exception as e:
        print(f"[SMOKE TEST] {wrapper.Label:22s} | {target:22s} | ERROR al predecir → {type(e).__name__}: {e}")
        return None

    preds = np.asarray(preds, dtype=float).reshape(-1)
    shape_ok = preds.shape[0] == len(X)
    n_nan = int(np.isnan(preds).sum())
    n_inf = int(np.isinf(preds).sum())
    ok = shape_ok and n_nan == 0 and n_inf == 0
    r2 = r2_score(y_true, preds) if ok else float("nan")
    status = "OK" if ok else "REVISAR"
    print(f"[SMOKE TEST] {wrapper.Label:22s} | {target:22s} | filas={len(preds):5d} | "
          f"NaN={n_nan} | Inf={n_inf} | R2={r2: .4f} | {status}")
    return preds if ok else None

# predictions[target][label] = np.array de predicciones sobre TOD0 eval.csv (solo si paso el smoke test)
predictions = {} for t in TARGETS_TO_TEST:

for target in TARGETS_TO_TEST:
    y_true_t = y_eval[target].values
    print(f"\n--- {target} ---")

    for (key, t), wrapper in loaded_single.items():
        if t != target:
            continue
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
        if preds is not None:
            predictions[target][wrapper.Label] = preds

    for key, wrapper in loaded_multi.items():
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
if preds is not None:
    predictions[target][wrapper.Label] = preds

--- nematode_shannon_z ---
[SMOKE TEST] Ridge | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0112 | OK
[SMOKE TEST] RF single-target | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1366 | OK
[SMOKE TEST] XGBoost single-target | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1637 | OK
[SMOKE TEST] RF multialida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1545 | OK
[SMOKE TEST] XGBoost multialida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2228 | OK
[SMOKE TEST] RegressorChain | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1467 | OK
[SMOKE TEST] MLP multialida | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0829 | OK
[SMOKE TEST] MLP custom loss | nematode_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0668 | OK

--- macro_shannon_z ---
[SMOKE TEST] Ridge | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1257 | OK
[SMOKE TEST] RF single-target | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2797 | OK
[SMOKE TEST] XGBoost single-target | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2681 | OK
[SMOKE TEST] RF multialida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2080 | OK
[SMOKE TEST] XGBoost multialida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3771 | OK
[SMOKE TEST] RegressorChain | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2804 | OK
[SMOKE TEST] MLP multialida | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2396 | OK
[SMOKE TEST] MLP custom loss | macro_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2704 | OK

--- earthworm_shannon_z ---
[SMOKE TEST] Ridge | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2395 | OK
[SMOKE TEST] RF single-target | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5059 | OK
[SMOKE TEST] XGBoost single-target | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4946 | OK
[SMOKE TEST] RF multialida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4160 | OK
[SMOKE TEST] XGBoost multialida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5375 | OK
[SMOKE TEST] RegressorChain | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4876 | OK
[SMOKE TEST] MLP multialida | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3538 | OK
[SMOKE TEST] MLP custom loss | earthworm_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3770 | OK

--- orib_shannon_z ---
[SMOKE TEST] Ridge | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1137 | OK
[SMOKE TEST] RF single-target | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1644 | OK
[SMOKE TEST] XGBoost single-target | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1147 | OK
[SMOKE TEST] RF multialida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1693 | OK
[SMOKE TEST] XGBoost multialida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2275 | OK
[SMOKE TEST] RegressorChain | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2069 | OK
[SMOKE TEST] MLP multialida | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2250 | OK
[SMOKE TEST] MLP custom loss | orib_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2552 | OK

--- meso_shannon_z ---
[SMOKE TEST] Ridge | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.2499 | OK
[SMOKE TEST] RF single-target | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1895 | OK
[SMOKE TEST] XGBoost single-target | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1868 | OK
[SMOKE TEST] RF multialida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1178 | OK
[SMOKE TEST] XGBoost multialida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.2861 | OK
[SMOKE TEST] RegressorChain | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1640 | OK
[SMOKE TEST] MLP multialida | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.4925 | OK
[SMOKE TEST] MLP custom loss | meso_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.4289 | OK

--- coll_shannon_z ---
[SMOKE TEST] Ridge | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.4998 | OK
[SMOKE TEST] RF single-target | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.4168 | OK
[SMOKE TEST] XGBoost single-target | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.3312 | OK
[SMOKE TEST] RF multialida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1953 | OK
[SMOKE TEST] XGBoost multialida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.3229 | OK
[SMOKE TEST] RegressorChain | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.6087 | OK
[SMOKE TEST] MLP multialida | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -1.2378 | OK
[SMOKE TEST] MLP custom loss | coll_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -1.2015 | OK

--- bac_shannon_z ---
[SMOKE TEST] Ridge | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0812 | OK
[SMOKE TEST] RF single-target | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0037 | OK
[SMOKE TEST] XGBoost single-target | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0092 | OK
[SMOKE TEST] RF multialida | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0290 | OK
[SMOKE TEST] XGBoost multialida | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0151 | OK
[SMOKE TEST] RegressorChain | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0288 | OK
[SMOKE TEST] MLP multialida | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0334 | OK
[SMOKE TEST] MLP custom loss | bac_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1218 | OK

--- fun_shannon_z ---
[SMOKE TEST] Ridge | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0161 | OK
[SMOKE TEST] RF single-target | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0337 | OK
[SMOKE TEST] XGBoost single-target | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0206 | OK
[SMOKE TEST] RF multialida | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0127 | OK
[SMOKE TEST] XGBoost multialida | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0502 | OK
[SMOKE TEST] RegressorChain | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0219 | OK
[SMOKE TEST] MLP multialida | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.1246 | OK
[SMOKE TEST] MLP custom loss | fun_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0672 | OK

--- euk_shannon_z ---
[SMOKE TEST] Ridge | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0407 | OK
[SMOKE TEST] RF single-target | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1575 | OK
[SMOKE TEST] XGBoost single-target | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0912 | OK
[SMOKE TEST] RF multialida | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1378 | OK
[SMOKE TEST] XGBoost multialida | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0886 | OK
[SMOKE TEST] RegressorChain | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1624 | OK
[SMOKE TEST] MLP multialida | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1451 | OK
[SMOKE TEST] MLP custom loss | euk_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1025 | OK

--- oomy_shannon_z ---
[SMOKE TEST] Ridge | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0857 | OK
[SMOKE TEST] RF single-target | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0998 | OK
[SMOKE TEST] XGBoost single-target | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0965 | OK
[SMOKE TEST] RF multialida | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1252 | OK
[SMOKE TEST] XGBoost multialida | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0620 | OK
[SMOKE TEST] RegressorChain | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0913 | OK
[SMOKE TEST] MLP multialida | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1362 | OK
[SMOKE TEST] MLP custom loss | oomy_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0429 | OK

--- cerc_shannon_z ---
[SMOKE TEST] Ridge | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0155 | OK
[SMOKE TEST] RF single-target | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0404 | OK
[SMOKE TEST] XGBoost single-target | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0414 | OK
[SMOKE TEST] RF multialida | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0041 | OK
[SMOKE TEST] XGBoost multialida | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0330 | OK
[SMOKE TEST] RegressorChain | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0006 | OK
[SMOKE TEST] MLP multialida | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0855 | OK
[SMOKE TEST] MLP custom loss | cerc_shannon_z | filas= 65 | NaN=0 | Inf=0 | R2= -0.0613 | OK

--- macro_order_richness_z ---
[SMOKE TEST] Ridge | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1521 | OK
[SMOKE TEST] RF single-target | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3058 | OK
[SMOKE TEST] XGBoost single-target | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3082 | OK
```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
[SMOKE TEST] RF multisalida | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2532 | OK
[SMOKE TEST] XGBoost multisalida | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4013 | OK
[SMOKE TEST] RegressorChain | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4069 | OK
[SMOKE TEST] MLP multisalida | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3447 | OK
[SMOKE TEST] MLP custom loss | macro_order_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.3581 | OK

--- earthworm_richness_z ---
[SMOKE TEST] Ridge | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2789 | OK
[SMOKE TEST] RF single-target | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5492 | OK
[SMOKE TEST] XGBoost single-target | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5171 | OK
[SMOKE TEST] RF multisalida | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4509 | OK
[SMOKE TEST] XGBoost multisalida | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5893 | OK
[SMOKE TEST] RegressorChain | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.5359 | OK
[SMOKE TEST] MLP multisalida | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4628 | OK
[SMOKE TEST] MLP custom loss | earthworm_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.4854 | OK

--- orib_species_richness_z ---
[SMOKE TEST] Ridge | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0921 | OK
[SMOKE TEST] RF single-target | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2407 | OK
[SMOKE TEST] XGBoost single-target | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1259 | OK
[SMOKE TEST] RF multisalida | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2386 | OK
[SMOKE TEST] XGBoost multisalida | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2800 | OK
[SMOKE TEST] RegressorChain | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2026 | OK
[SMOKE TEST] MLP multisalida | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2783 | OK
[SMOKE TEST] MLP custom loss | orib_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2981 | OK

--- meso_species_richness_z ---
[SMOKE TEST] Ridge | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0959 | OK
[SMOKE TEST] RF single-target | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0453 | OK
[SMOKE TEST] XGBoost single-target | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0592 | OK
[SMOKE TEST] RF multisalida | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0255 | OK
[SMOKE TEST] XGBoost multisalida | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1146 | OK
[SMOKE TEST] RegressorChain | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0400 | OK
[SMOKE TEST] MLP multisalida | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2681 | OK
[SMOKE TEST] MLP custom loss | meso_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2198 | OK

--- coll_species_richness_z ---
[SMOKE TEST] Ridge | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.7341 | OK
[SMOKE TEST] RF single-target | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5461 | OK
[SMOKE TEST] XGBoost single-target | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5803 | OK
[SMOKE TEST] RF multisalida | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.2920 | OK
[SMOKE TEST] XGBoost multisalida | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.6465 | OK
[SMOKE TEST] RegressorChain | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.5799 | OK
[SMOKE TEST] MLP multisalida | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-2.0859 | OK
[SMOKE TEST] MLP custom loss | coll_species_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-1.9873 | OK

--- bac_asv_richness_z ---
[SMOKE TEST] Ridge | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0761 | OK
[SMOKE TEST] RF single-target | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0004 | OK
[SMOKE TEST] XGBoost single-target | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0014 | OK
[SMOKE TEST] RF multisalida | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0044 | OK
[SMOKE TEST] XGBoost multisalida | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0155 | OK
[SMOKE TEST] RegressorChain | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0052 | OK
[SMOKE TEST] MLP multisalida | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0799 | OK
[SMOKE TEST] MLP custom loss | bac_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.1811 | OK

--- fun_asv_richness_z ---
[SMOKE TEST] Ridge | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0087 | OK
[SMOKE TEST] RF single-target | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0144 | OK
[SMOKE TEST] XGBoost single-target | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0242 | OK
[SMOKE TEST] RF multisalida | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0068 | OK
[SMOKE TEST] XGBoost multisalida | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0662 | OK
[SMOKE TEST] RegressorChain | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0186 | OK
[SMOKE TEST] MLP multisalida | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0105 | OK
[SMOKE TEST] MLP custom loss | fun_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0069 | OK

--- euk_asv_richness_z ---
[SMOKE TEST] Ridge | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0400 | OK
[SMOKE TEST] RF single-target | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2360 | OK
[SMOKE TEST] XGBoost single-target | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1671 | OK
[SMOKE TEST] RF multisalida | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1903 | OK
[SMOKE TEST] XGBoost multisalida | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2233 | OK
[SMOKE TEST] RegressorChain | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2690 | OK
[SMOKE TEST] MLP multisalida | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2470 | OK
[SMOKE TEST] MLP custom loss | euk_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1970 | OK

--- oomy_asv_richness_z ---
[SMOKE TEST] Ridge | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1035 | OK
[SMOKE TEST] RF single-target | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1895 | OK
[SMOKE TEST] XGBoost single-target | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1776 | OK
[SMOKE TEST] RF multisalida | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1917 | OK
[SMOKE TEST] XGBoost multisalida | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1650 | OK
[SMOKE TEST] RegressorChain | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.2019 | OK
[SMOKE TEST] MLP multisalida | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0640 | OK
[SMOKE TEST] MLP custom loss | oomy_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0188 | OK

--- cerc_asv_richness_z ---
[SMOKE TEST] Ridge | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1019 | OK
[SMOKE TEST] RF single-target | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1207 | OK
[SMOKE TEST] XGBoost single-target | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1227 | OK
[SMOKE TEST] RF multisalida | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1171 | OK
[SMOKE TEST] XGBoost multisalida | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0983 | OK
[SMOKE TEST] RegressorChain | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.1171 | OK
[SMOKE TEST] MLP multisalida | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2= 0.0285 | OK
[SMOKE TEST] MLP custom loss | cerc_asv_richness_z | filas= 65 | NaN=0 | Inf=0 | R2=-0.0845 | OK
```

5.- Particion meta-train / meta-holdout

Se separan las filas de `eval.csv` en dos bloques usando los mismos índices para todos los targets: uno para entrenar el meta-modelo y otro (holdout) para evaluarlo después, sin que haya visto esas filas durante el entrenamiento.

```
In [5]: idx_all = np.arange(len(eval_df))
        idx_train, idx_holdout = train_test_split(idx_all, test_size=META_TEST_SIZE, random_state=RANDOM_STATE)

        print(f"Filas totales en eval.csv: {len(idx_all)}")
        print(f"Filas para entrenar el meta-modelo (meta-train): {len(idx_train)}")
        print(f"Filas para evaluar el meta-modelo (holdout): {len(idx_holdout)}")
```

```
Filas totales en eval.csv: 65
Filas para entrenar el meta-modelo (meta-train): 45
Filas para evaluar el meta-modelo (holdout): 20
```

6.- Entrenamiento de los meta-modelos por target

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

Para cada target y cada candidato de `META_MODEL_FACTORIES`, la entrada es la matriz de predicciones de todos los modelos base disponibles (una columna por modelo) sobre las filas de meta-train, y la salida es el valor real de ese target en esas mismas filas.

```
In [6]: meta_models = {} # target → {"model_labels": [...], "models": {nombre_meta: modelo_entrenado}}
stacked_features = {} # target → matriz (n_filas_eval x n_modelos_base), mismo orden que eval_df

for target in TARGETS_TO_TEST:
    model_labels = list(predictions[target].keys())
    if not model_labels:
        print(f"[AVISO] {target}: no hay modelos base disponibles, no se pueden entrenar los meta-modelos.")
        continue

    X_meta = np.column_stack([predictions[target][lbl] for lbl in model_labels])
    stacked_features[target] = X_meta

    y_target = y_eval[target].values
    X_meta_train = X_meta[idx_train]
    y_meta_train = y_target[idx_train]

    trained = {}
    for meta_name, factory in META_MODEL_FACTORIES.items():
        meta_model = factory()
        meta_model.fit(X_meta_train, y_meta_train)
        trained[meta_name] = meta_model

    meta_models[target] = {"model_labels": model_labels, "models": trained}
    print(f"{target}: {len(trained)} meta-modelos entrenados con {len(model_labels)} modelos base → {list(trained)}")
```

```
nematode_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
macro_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
earthworm_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
orib_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
meso_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
coll_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
bac_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
fun_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
euk_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
oomy_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
cerc_shannon_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
macro_order_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
earthworm_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
orib_species_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
meso_species_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
coll_species_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
bac_asv_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
fun_asv_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
euk_asv_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
oomy_asv_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
cerc_asv_richness_z: 5 meta-modelos entrenados con 8 modelos base → ['Ridge', 'Lasso', 'ElasticNet', 'Linear', 'RandomForest']
```

7.- Smoke test de los meta-modelos sobre el holdout

Mismas comprobaciones que antes (sin error, sin NaN/Inf, tamaño correcto), aplicadas a cada meta-modelo sobre las filas de holdout que no ha visto en el entrenamiento. También se guardan aquí las predicciones de cada modelo base y del ensamblado simple por media, recortadas al mismo holdout, para poder comparar todo en igualdad de condiciones en la sección 8.

```
In [7]: holdout_predictions = {} for t in TARGETS_TO_TEST:

for target in TARGETS_TO_TEST:
    if target not in meta_models:
        continue

    y_true_holdout = y_eval[target].values[idx_holdout]
    print(f"\n--- {target} (holdout, {len(idx_holdout)} filas) ---")

    # Modelos base, recortados al holdout
    for label in meta_models[target]["model_labels"]:
        preds_full = predictions[target][label]
        preds_holdout = np.asarray(preds_full, dtype=float)[idx_holdout]
        n_nan = int(np.isnan(preds_holdout).sum())
        n_inf = int(np.isinf(preds_holdout).sum())
        ok = n_nan == 0 and n_inf == 0
        r2 = r2_score(y_true_holdout, preds_holdout) if ok else float("nan")
        status = "OK" if ok else "REVISAR"
        print(f"[SMOKE TEST] {label:22s} | filas={len(preds_holdout):5d} | NaN={n_nan} | Inf={n_inf} | R2={r2: .4f} | {status}")
    if ok:
        holdout_predictions[target][label] = preds_holdout

    # Ensamblado simple por media, calculado solo con las filas del holdout
    model_labels = meta_models[target]["model_labels"]
    stacked_holdout = np.column_stack([predictions[target][lbl] for lbl in model_labels])[idx_holdout]
    mean_ensemble_holdout = stacked_holdout.mean(axis=1)
    holdout_predictions[target]["Ensamblado (media)"] = mean_ensemble_holdout

    # Cada meta-modelo candidato
    X_meta_holdout = stacked_features[target][idx_holdout]
    for meta_name, meta_model in meta_models[target]["models"].items():
        label = f"Meta-modelo ({meta_name})"
        try:
            meta_preds = np.asarray(meta_model.predict(X_meta_holdout), dtype=float).reshape(-1)
            n_nan = int(np.isnan(meta_preds).sum())
            n_inf = int(np.isinf(meta_preds).sum())
            ok = meta_preds.shape[0] == len(idx_holdout) and n_nan == 0 and n_inf == 0
            r2 = r2_score(y_true_holdout, meta_preds) if ok else float("nan")
            status = "OK" if ok else "REVISAR"
            print(f"[SMOKE TEST] {label:22s} | filas={len(meta_preds):5d} | NaN={n_nan} | Inf={n_inf} | R2={r2: .4f} | {status}")
            if ok:
                holdout_predictions[target][label] = meta_preds
        except Exception as e:
            print(f"[SMOKE TEST] {label:22s} | ERROR al predecir → {type(e).__name__}: {e}")
```

```

--- nematode_shannon_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0695 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.1074 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.1346 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.0853 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.2808 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.1490 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.0073 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.0617 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2599 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2238 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2206 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1860 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1941 | OK

--- macro_shannon_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0129 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3044 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3127 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.1926 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.3703 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.3722 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.2301 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.3217 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1676 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1009 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1222 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0225 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1182 | OK

--- earthworm_shannon_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.1160 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3653 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3157 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.3560 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.3227 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.3665 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.0194 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.0703 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3012 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3263 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3273 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2114 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3237 | OK

--- orib_shannon_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0291 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.0202 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.0462 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0127 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0546 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2=-0.0302 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0543 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.0014 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0782 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0146 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0269 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.1755 | OK

```

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

[SMOKE TEST] Meta-modelo (RandomForest)		filas=	20		NaN=0		Inf=0		R2=-0.2093		OK
--- meso_shannon_z (holdout, 20 filas) ---											
[SMOKE TEST] Ridge		filas=	20		NaN=0		Inf=0		R2=0.1916		OK
[SMOKE TEST] RF single-target		filas=	20		NaN=0		Inf=0		R2=0.0353		OK
[SMOKE TEST] XGBoost single-target		filas=	20		NaN=0		Inf=0		R2=0.0386		OK
[SMOKE TEST] RF multitalisida		filas=	20		NaN=0		Inf=0		R2=0.0392		OK
[SMOKE TEST] XGBoost multitalisida		filas=	20		NaN=0		Inf=0		R2=0.1482		OK
[SMOKE TEST] RegressorChain		filas=	20		NaN=0		Inf=0		R2=0.0235		OK
[SMOKE TEST] MLP multitalisida		filas=	20		NaN=0		Inf=0		R2=0.6674		OK
[SMOKE TEST] MLP custom loss		filas=	20		NaN=0		Inf=0		R2=0.5844		OK
[SMOKE TEST] Meta-modelo (Ridge)		filas=	20		NaN=0		Inf=0		R2=0.2047		OK
[SMOKE TEST] Meta-modelo (Lasso)		filas=	20		NaN=0		Inf=0		R2=0.1273		OK
[SMOKE TEST] Meta-modelo (ElasticNet)		filas=	20		NaN=0		Inf=0		R2=0.1295		OK
[SMOKE TEST] Meta-modelo (Linear)		filas=	20		NaN=0		Inf=0		R2=0.1146		OK
[SMOKE TEST] Meta-modelo (RandomForest)		filas=	20		NaN=0		Inf=0		R2=0.3326		OK
--- coll_shannon_z (holdout, 20 filas) ---											
[SMOKE TEST] Ridge		filas=	20		NaN=0		Inf=0		R2=0.4537		OK
[SMOKE TEST] RF single-target		filas=	20		NaN=0		Inf=0		R2=0.1833		OK
[SMOKE TEST] XGBoost single-target		filas=	20		NaN=0		Inf=0		R2=0.1497		OK
[SMOKE TEST] RF multitalisida		filas=	20		NaN=0		Inf=0		R2=0.0590		OK
[SMOKE TEST] XGBoost multitalisida		filas=	20		NaN=0		Inf=0		R2=0.2065		OK
[SMOKE TEST] RegressorChain		filas=	20		NaN=0		Inf=0		R2=0.2493		OK
[SMOKE TEST] MLP multitalisida		filas=	20		NaN=0		Inf=0		R2=2.4566		OK
[SMOKE TEST] MLP custom loss		filas=	20		NaN=0		Inf=0		R2=2.7013		OK
[SMOKE TEST] Meta-modelo (Ridge)		filas=	20		NaN=0		Inf=0		R2=0.2805		OK
[SMOKE TEST] Meta-modelo (Lasso)		filas=	20		NaN=0		Inf=0		R2=0.0581		OK
[SMOKE TEST] Meta-modelo (ElasticNet)		filas=	20		NaN=0		Inf=0		R2=0.1611		OK
[SMOKE TEST] Meta-modelo (Linear)		filas=	20		NaN=0		Inf=0		R2=0.4077		OK
[SMOKE TEST] Meta-modelo (RandomForest)		filas=	20		NaN=0		Inf=0		R2=0.3693		OK
--- bac_shannon_z (holdout, 20 filas) ---											
[SMOKE TEST] Ridge		filas=	20		NaN=0		Inf=0		R2=0.3969		OK
[SMOKE TEST] RF single-target		filas=	20		NaN=0		Inf=0		R2=0.2900		OK
[SMOKE TEST] XGBoost single-target		filas=	20		NaN=0		Inf=0		R2=0.2986		OK
[SMOKE TEST] RF multitalisida		filas=	20		NaN=0		Inf=0		R2=0.3272		OK
[SMOKE TEST] XGBoost multitalisida		filas=	20		NaN=0		Inf=0		R2=0.2574		OK
[SMOKE TEST] RegressorChain		filas=	20		NaN=0		Inf=0		R2=0.1293		OK
[SMOKE TEST] MLP multitalisida		filas=	20		NaN=0		Inf=0		R2=0.5203		OK
[SMOKE TEST] MLP custom loss		filas=	20		NaN=0		Inf=0		R2=0.7874		OK
[SMOKE TEST] Meta-modelo (Ridge)		filas=	20		NaN=0		Inf=0		R2=0.1651		OK
[SMOKE TEST] Meta-modelo (Lasso)		filas=	20		NaN=0		Inf=0		R2=0.1001		OK
[SMOKE TEST] Meta-modelo (ElasticNet)		filas=	20		NaN=0		Inf=0		R2=0.0500		OK
[SMOKE TEST] Meta-modelo (Linear)		filas=	20		NaN=0		Inf=0		R2=0.6071		OK
[SMOKE TEST] Meta-modelo (RandomForest)		filas=	20		NaN=0		Inf=0		R2=0.9267		OK
--- fun_shannon_z (holdout, 20 filas) ---											
[SMOKE TEST] Ridge		filas=	20		NaN=0		Inf=0		R2=0.0454		OK
[SMOKE TEST] RF single-target		filas=	20		NaN=0		Inf=0		R2=0.1035		OK
[SMOKE TEST] XGBoost single-target		filas=	20								

Desarrollo y evaluación de modelos de aprendizaje automático para la predicción de la biodiversidad del suelo

```
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2270 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3036 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3142 | OK

--- earthworm_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.1531 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.5028 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.4989 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.4353 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.5087 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.4625 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.1671 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.2336 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.5225 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.5175 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.4737 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.4934 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1883 | OK

--- orib_species_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.1141 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.0893 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.0734 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0545 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.1811 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2=-0.0834 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.2441 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.2714 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2=-0.4362 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2=-0.3359 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2=-0.3580 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.2950 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2=-0.5907 | OK

--- meso_species_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0381 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.0531 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.0512 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.0116 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0264 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.0499 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.4126 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.3581 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2=-0.1662 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0630 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0636 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.1760 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2=-0.2422 | OK

--- coll_species_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-1.1093 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.3038 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.4074 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0691 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.4185 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2=-0.2315 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-5.0199 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-5.4060 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1627 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0570 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2366 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3629 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.5233 | OK

--- bac_asv_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0970 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.1536 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.0906 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.1571 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0890 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2=-0.1310 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.3656 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.5413 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0164 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0183 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0269 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.3580 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2=-0.2139 | OK

--- fun_asv_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2=-0.0873 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.1143 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2=-0.1011 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0995 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.1820 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2=-0.1319 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2=-0.0904 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.0914 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0995 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0189 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0286 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.0661 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2=-0.2787 | OK

--- euk_asv_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2= 0.1997 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.2195 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.1091 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.1859 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.0789 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.2012 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.1930 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.1625 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0312 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1352 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1997 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2=-0.5930 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0659 | OK

--- oomy_asv_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2= 0.1394 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.2743 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.2432 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.3024 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.2506 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.2802 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.2264 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2= 0.1398 | OK
```

```
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2587 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1086 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1647 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2334 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1872 | OK

--- cerc_asv_richness_z (holdout, 20 filas) ---
[SMOKE TEST] Ridge | filas= 20 | NaN=0 | Inf=0 | R2= 0.2057 | OK
[SMOKE TEST] RF single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3594 | OK
[SMOKE TEST] XGBoost single-target | filas= 20 | NaN=0 | Inf=0 | R2= 0.3082 | OK
[SMOKE TEST] RF multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.2667 | OK
[SMOKE TEST] XGBoost multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.4049 | OK
[SMOKE TEST] RegressorChain | filas= 20 | NaN=0 | Inf=0 | R2= 0.3455 | OK
[SMOKE TEST] MLP multisalida | filas= 20 | NaN=0 | Inf=0 | R2= 0.1363 | OK
[SMOKE TEST] MLP custom loss | filas= 20 | NaN=0 | Inf=0 | R2=-0.1239 | OK
[SMOKE TEST] Meta-modelo (Ridge) | filas= 20 | NaN=0 | Inf=0 | R2= 0.3590 | OK
[SMOKE TEST] Meta-modelo (Lasso) | filas= 20 | NaN=0 | Inf=0 | R2= 0.1213 | OK
[SMOKE TEST] Meta-modelo (ElasticNet) | filas= 20 | NaN=0 | Inf=0 | R2= 0.2181 | OK
[SMOKE TEST] Meta-modelo (Linear) | filas= 20 | NaN=0 | Inf=0 | R2= 0.0939 | OK
[SMOKE TEST] Meta-modelo (RandomForest) | filas= 20 | NaN=0 | Inf=0 | R2= 0.4917 | OK
```

8.- Comparativa: modelos individuales vs. ensamblado por media vs. meta-modelo

Todas las metricas de esta tabla se calculan sobre el mismo holdout, para que la comparacion entre modelos base, ensamblado simple y meta-modelo sea justa.

```
In [8]: rows = []
for target in TARGETS_TO_TEST:
    if target not in meta_models:
        continue
    y_true_holdout = y_eval[target].values[idx_holdout]
    for label, preds in holdout_predictions[target].items():
        rmse = mean_squared_error(y_true_holdout, preds) ** 0.5
        rows.append({
            "target": target,
            "modelo": label,
            "R2": r2_score(y_true_holdout, preds),
            "RMSE": rmse,
            "MAE": mean_absolute_error(y_true_holdout, preds),
            "tipo": "meta-modelo" if label.startswith("Meta-modelo (") else ("ensamblado media" if label == "Ensamblado (media)" else "individual"),
        })

if not rows:
    print("[AVISO] No hay predicciones de holdout disponibles todavia. "
          "Revisa que los modelos base se hayan cargado (seccion 3), que el smoke test "
          "haya pasado (seccion 4) y que los meta-modelos se hayan entrenado (seccion 6).")
    results_df = pd.DataFrame(columns=["target", "modelo", "R2", "RMSE", "MAE", "tipo"])
else:
    results_df = pd.DataFrame(rows).sort_values(["target", "R2"], ascending=[True, False]).reset_index(drop=True)

results_df
```

```
Out[8]:
```

	target	modelo	R2	RMSE	MAE	tipo
0	bac_asv_richness_z	Meta-modelo (ElasticNet)	0.026856	0.808248	0.594321	meta-modelo
1	bac_asv_richness_z	Meta-modelo (Lasso)	0.018270	0.811806	0.617058	meta-modelo
2	bac_asv_richness_z	Meta-modelo (Ridge)	-0.016433	0.826030	0.592349	meta-modelo
3	bac_asv_richness_z	XGBoost multisalida	-0.089010	0.855012	0.627395	individual
4	bac_asv_richness_z	XGBoost single-target	-0.090581	0.855628	0.626837	individual
...	...	...	...	...	...	...
289	orib_species_richness_z	Meta-modelo (Linear)	-0.295042	1.020244	0.726350	meta-modelo
290	orib_species_richness_z	Meta-modelo (Lasso)	-0.315931	1.028440	0.734192	meta-modelo
291	orib_species_richness_z	Meta-modelo (ElasticNet)	-0.358041	1.044765	0.748035	meta-modelo
292	orib_species_richness_z	Meta-modelo (Ridge)	-0.436228	1.074420	0.769499	meta-modelo
293	orib_species_richness_z	Meta-modelo (RandomForest)	-0.590671	1.130714	0.867258	meta-modelo

294 rows x 6 columns

8.1.- Resumen agregado por target

Con muchos targets a la vez, `results_df` tiene demasiadas filas para leerla a ojo. Esta tabla se queda con una fila por target: el mejor modelo individual, el mejor meta-modelo (o ensamblado por media), cual de los dos gana, y si el mejor R2 de ese target es positivo (es decir, si hay alguna señal predictiva real) o no.

R2 <= 0 significa que ni el mejor modelo hace mejor que predecir siempre la media -- en esos targets el problema no es que meta-modelo usar, sino que los modelos base no tienen señal útil que aprender a combinar. En targets sin señal, además, conviene fijarse en si los meta-modelos quedan claramente peor que el mejor individual (mas sobreajuste al meta-train cuanto menos señal real hay para aprender).

```
In [9]: if results_df.empty:
    print("[AVISO] results_df esta vacio. Ejecuta primero las secciones 3-8.")
    summary_df = pd.DataFrame(columns=[
        "target", "mejor_individual", "R2_mejor_individual",
        "mejor_no_individual", "R2_mejor_no_individual", "gana_no_individual", "mejor_R2_global",
    ])
else:
    summary_rows = []
    for target in results_df["target"].unique():
        sub = results_df[results_df["target"] == target]

        individuales = sub[sub["tipo"] == "individual"].sort_values("R2", ascending=False)
        no_individuales = sub[sub["tipo"] != "individual"].sort_values("R2", ascending=False)

        mejor_individual = individuales.iloc[0] if not individuales.empty else None
        mejor_no_individual = no_individuales.iloc[0] if not no_individuales.empty else None

        r2_individual = mejor_individual["R2"] if mejor_individual is not None else float("-inf")
        r2_no_individual = mejor_no_individual["R2"] if mejor_no_individual is not None else float("-inf")

        summary_rows.append({
            "target": target,
            "mejor_individual": mejor_individual["modelo"] if mejor_individual is not None else None,
            "R2_mejor_individual": r2_individual,
            "mejor_no_individual": mejor_no_individual["modelo"] if mejor_no_individual is not None else None,
            "R2_mejor_no_individual": r2_no_individual,
            "gana_no_individual": bool(r2_no_individual > r2_individual),
            "mejor_R2_global": max(r2_individual, r2_no_individual),
        })
```

```
summary_df = pd.DataFrame(summary_rows).sort_values("mejor_R2_global", ascending=False).reset_index(drop=True)

n_targets = len(summary_df)
n_gana_no_individual = int(summary_df["gana_no_individual"].sum())
n_r2_positivo = int((summary_df["mejor_R2_global"] > 0).sum())

print(f"Targets evaluados: {n_targets}")
print(f"Targets donde algun meta-modelo/ensamblado supera al mejor individual: "
      f"{(n_gana_no_individual)/n_targets} ({n_gana_no_individual / n_targets:.0%})")
print(f"Targets con señal predictiva real (mejor R2 > 0): "
      f"{(n_r2_positivo)/n_targets} ({n_r2_positivo / n_targets:.0%})")
print(f"Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): "
      f"{(n_targets - n_r2_positivo)}")
```

summary_df

Targets evaluados: 21

Targets donde algun meta-modelo/ensamblado supera al mejor individual: 9/21 (43%)

Targets con señal predictiva real (mejor R2 > 0): 18/21 (86%)

Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): 3

Out[9]:	target	mejor_individual	R2_mejor_individual	mejor_no_individual	R2_mejor_no_individual	gana_no_individual	mejor_R2_global
0	coll_species_richness_z	RF multisalida	-0.069098	Meta-modelo (RandomForest)	0.523332	True	0.523332
1	earthworm_richness_z	XGBoost multisalida	0.508708	Meta-modelo (Ridge)	0.522522	True	0.522522
2	macro_order_richness_z	XGBoost multisalida	0.509020	Ensamblado (media)	0.397477	False	0.509020
3	cerc_asv_richness_z	XGBoost multisalida	0.404936	Meta-modelo (RandomForest)	0.491682	True	0.491682
4	coll_shannon_z	RF multisalida	-0.058985	Meta-modelo (Linear)	0.407715	True	0.407715
5	macro_shannon_z	RegressorChain	0.372242	Ensamblado (media)	0.297913	False	0.372242
6	earthworm_shannon_z	RegressorChain	0.366510	Meta-modelo (ElasticNet)	0.327297	False	0.366510
7	oomy_asv_richness_z	RF multisalida	0.302448	Meta-modelo (Ridge)	0.258726	False	0.302448
8	nematode_shannon_z	XGBoost multisalida	0.280802	Meta-modelo (Ridge)	0.259927	False	0.280802
9	euk_asv_richness_z	RF single-target	0.219539	Ensamblado (media)	0.208970	False	0.219539
10	cerc_shannon_z	RF multisalida	0.121626	Ensamblado (media)	0.065739	False	0.121626
11	oomy_shannon_z	MLP multisalida	0.109946	Ensamblado (media)	0.033054	False	0.109946
12	euk_shannon_z	RF multisalida	0.103819	Meta-modelo (ElasticNet)	0.072138	False	0.103819
13	meso_species_richness_z	RF single-target	0.053125	Ensamblado (media)	-0.002852	False	0.053125
14	bac_asv_richness_z	XGBoost multisalida	-0.089010	Meta-modelo (ElasticNet)	0.026856	True	0.026856
15	meso_shannon_z	RegressorChain	0.023491	Meta-modelo (Linear)	-0.114632	False	0.023491
16	orib_shannon_z	MLP custom loss	-0.001389	Meta-modelo (Lasso)	0.014629	True	0.014629
17	fun_shannon_z	Ridge	-0.045406	Meta-modelo (Linear)	0.006084	True	0.006084
18	fun_asv_richness_z	Ridge	-0.087334	Meta-modelo (Lasso)	-0.018913	True	-0.018913
19	bac_shannon_z	RegressorChain	-0.129290	Meta-modelo (ElasticNet)	-0.050019	True	-0.050019
20	orib_species_richness_z	RF multisalida	-0.054500	Ensamblado (media)	-0.080917	False	-0.054500

9.- Visualización

Barra de R2 por modelo y target sobre el holdout. Los meta-modelos y el ensamblado por media se resaltan con borde negro para distinguirlos de los modelos base.

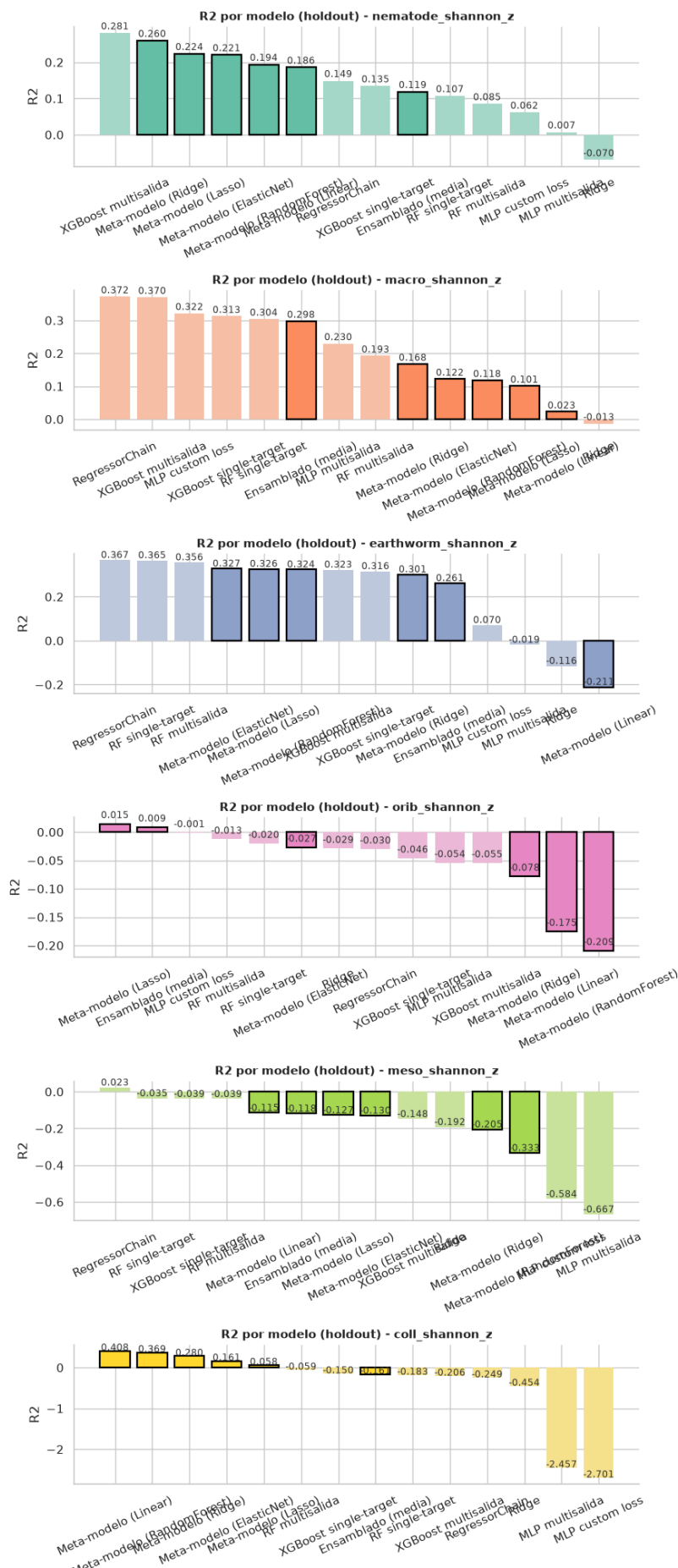
```
In [10]: if results_df.empty:
print("[AVISO] results_df esta vacio, no hay nada que graficar. Ejecuta primero las secciones 3-8.")
else:
    targets_present = [t for t in TARGETS_TO_TEST if t in results_df["target"].unique()]
    palette = sns.color_palette("Set2", n_colors=max(len(targets_present), 3))
    TARGET_COLORS = dict(zip(targets_present, palette))

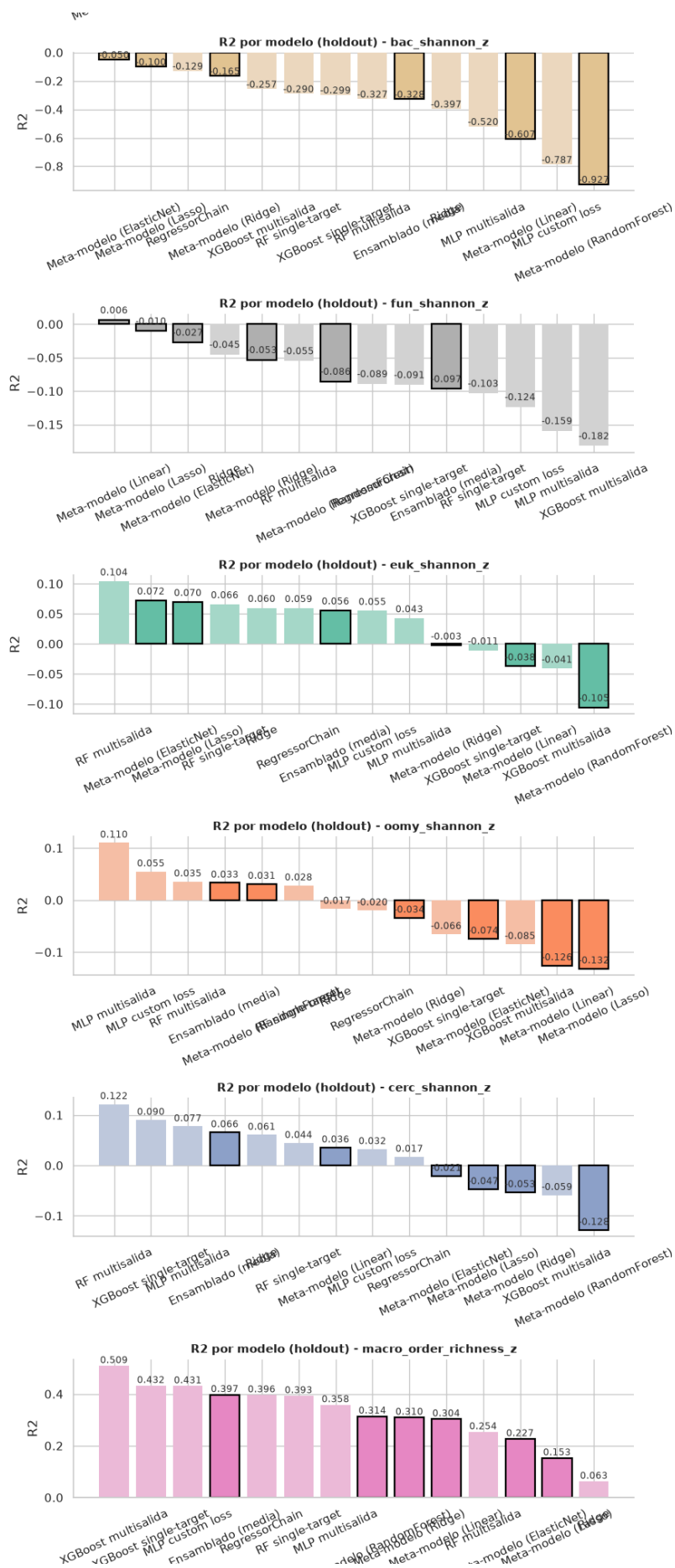
    fig, axes = plt.subplots(len(targets_present), 1, figsize=(9, 3.4 * len(targets_present)), squeeze=False)

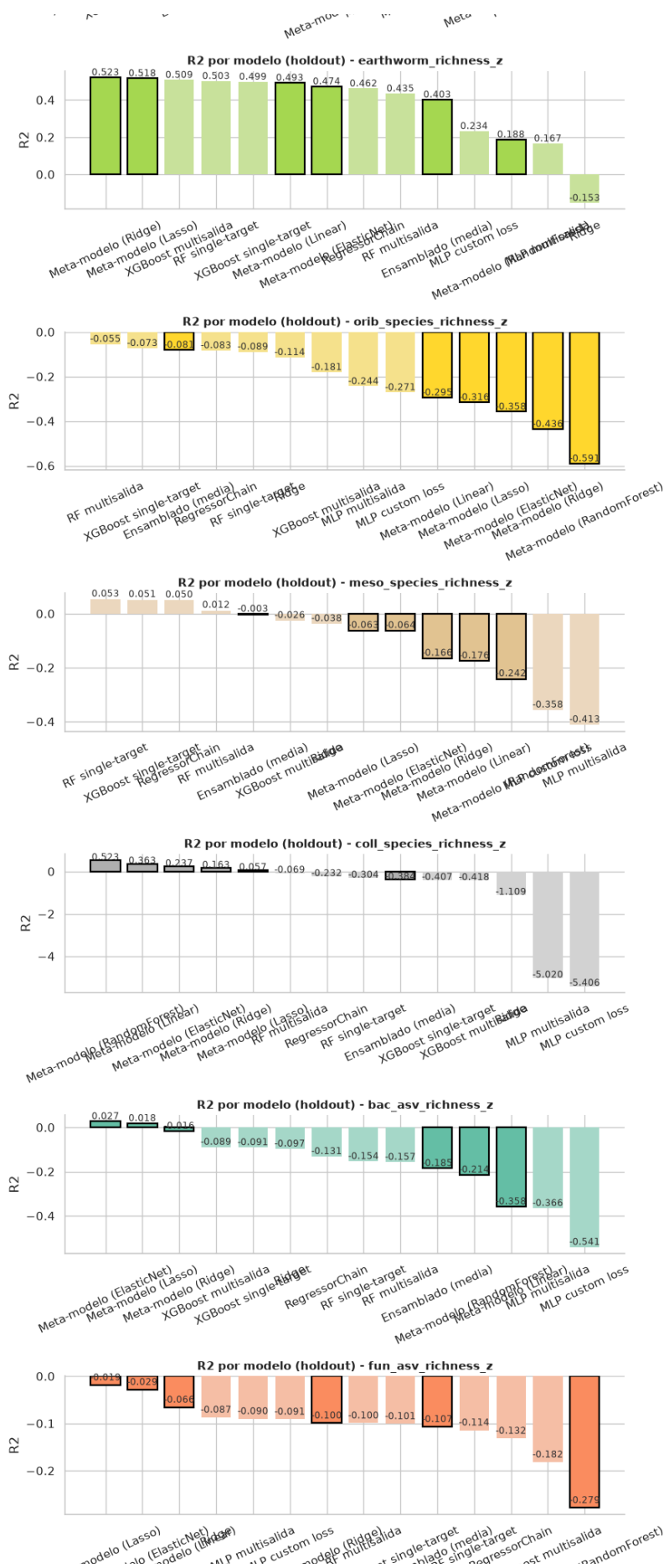
    for ax, target in zip(axes[:, 0], targets_present):
        sub = results_df[results_df["target"] == target].sort_values("R2", ascending=False)
        base_color = TARGET_COLORS[target]
        is_highlighted = sub["tipo"] != "individual"
        colors = [base_color if h else sns.light_palette(base_color, n_colors=3)[1] for h in is_highlighted]
        edgecolors = ["black" if h else "none" for h in is_highlighted]
        linewidths = [1.6 if h else 0 for h in is_highlighted]

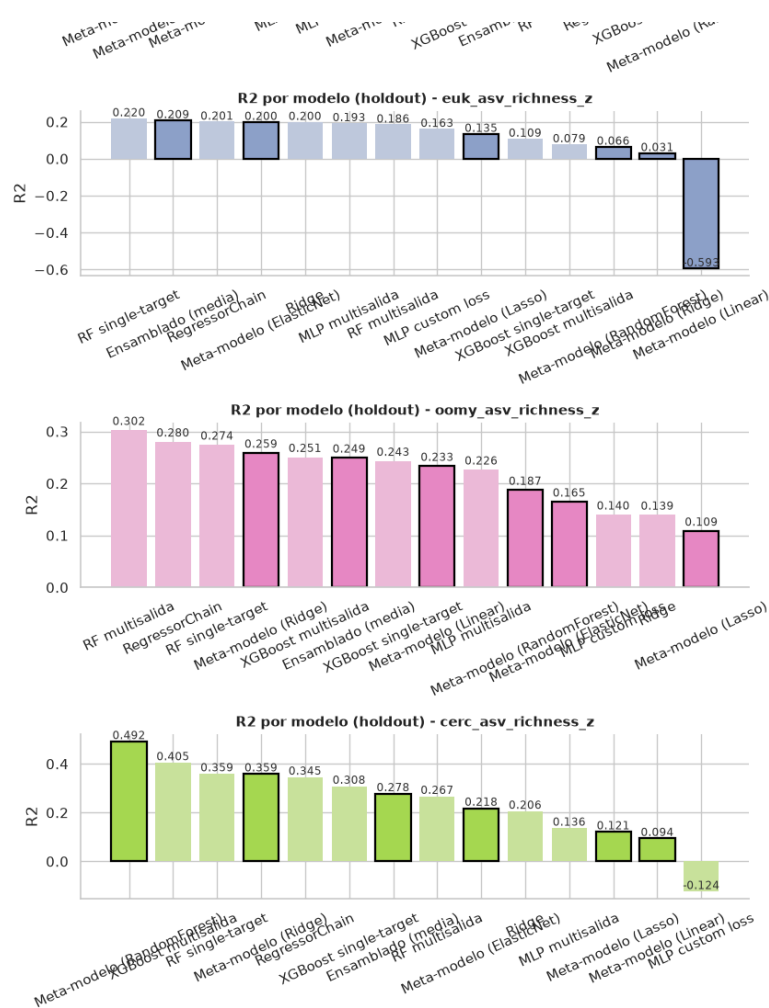
        bars = ax.bar(sub["modelo"], sub["R2"], color=colors, edgecolor=edgecolors, linewidth=linewidths)
        for bar, r2 in zip(bars, sub["R2"]):
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f"{r2:.3f}",
                    ha="center", fontsize=8.5, color="#333333")
        ax.set_title(f"R2 por modelo (holdout) - {target}", fontsize=11, fontweight="bold")
        ax.set_ylabel("R2")
        ax.tick_params(axis="x", rotation=25)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)

    fig.tight_layout()
    fig.savefig("output/r2_comparativa_con_modelos.png", dpi=300, bbox_inches="tight")
    plt.show()
```









10. Conclusiones

[COMPLETAR tras ejecutar con los modelos y `eval.csv` reales]

- ¿Algun meta-modelo mejora el R2 en el holdout respecto al ensamblado simple por media y respecto al mejor modelo individual, en alguno de los targets probados?
- ¿Es siempre el mismo meta-modelo el que gana en los distintos targets, o cambia segun el target? Si Ridge/Lasso/ElasticNet (regularizados) generalizan mejor que Linear/RandomForest (mas parametros libres) en el target con menos datos o mas ruido, es una señal de sobreajuste en los candidatos menos regularizados.
- ¿La mejora (si la hay) es consistente entre targets o depende mucho de cual se pruebe?
- Dado que el holdout es pequeño, ¿los resultados parecen estables o muy sensibles al `RANDOM_STATE` usado en el `train_test_split`? Se puede repetir el notebook con otras semillas para comprobarlo, target a target y meta-modelo a meta-modelo.
- ¿Compensa la complejidad extra de mantener y versionar un meta-modelo (ademas de los modelos base) frente a quedarse con el ensamblado simple por media o con el mejor modelo individual por target?
- Si se dispone de mas datos en el futuro, ¿merece la pena repetir este notebook con validacion cruzada (predicciones out-of-fold) en vez de un unico split train/holdout, para reducir el riesgo de sobreajustar los meta-modelos a una particion concreta?

11.4. Pruebas llevadas a cabo

En este anexo se indican las pruebas adicionales realizadas y los resultados que estas mismas han mostrado. Todas estas pruebas, junto con sus iteraciones o fases se han realizado en ramas independientes, por lo tanto los resultados se podrán comparar de forma independiente dentro del repositorio de GitHub. La distribución de las ramas se puede ver en el **Anexo IV**.

11.4.1. Prueba de eliminación de variables

11.4.2. Prueba de sensibilidad al random state

11.4.3. Prueba de barrido de random_state

11.4.4. Prueba de ensamblado de predicciones

11.5. Distribución de ramas y notebooks

Como se indica en **Lenguajes de programación, control de versiones y librerías**, este TFG ha empleado git como sistema de control de versiones, alojado en un repositorio de GitHub^[28].

A diferencia de un flujo de trabajo con una única rama principal, en el que cada modelo y cada prueba adicional se acumularían de forma secuencial sobre el mismo historial, se optó por un esquema de ramas independientes por prueba. Esto responde al carácter exploratorio de este TFG.

A lo largo del trabajo se han entrenado múltiples variantes de un mismo modelo, en específico las variantes multisalida (véase Sección 11.2.6, Sección 5.3.3.1 para la variante de XGBoost y Sección 11.2.3, Sección 5.3.2.1 para la variante de Random Forest) y las variantes elaboradas para las pruebas de **Eliminación de variables**, **Sensibilidad de random_state** y **Barrido de random_states**. Esto ha sido necesario para poder comparar los resultados de las diferentes variables entre sí, independientemente de que se haya hecho de forma manual o automatizada, sin que los cambios realizados sobrescribiesen los resultados de las demás.

Para proporcionar una mayor visibilidad e interpretabilidad al trabajo realizado, se ha decidido aplicar las siguientes medidas:

1. Alojar cada prueba/modificación realizada dentro de ramas independientes para tener una mejor forma de comparar los resultados entre varias ramas.
2. Establecer una nomenclatura común para las ramas que formen parte de una misma prueba, de forma que el propio nombre de la rama sea autoexplicativo sobre los orígenes de su prueba o objetivo.
3. Todas las ramas relacionadas con los modelos irán precedidas por el prefijo `model-prep`.
4. Ramas auxiliares o con funciones específicas deberán seguir una nomenclatura similar, describiendo brevemente su propósito (por ejemplo, `documentation` y `data-prep`).

Esta nomenclatura jerárquica permite, además, identificar de un vistazo qué ramas están relacionadas entre sí, es decir todas las variantes o iteraciones de una prueba comparten el mismo prefijo (por ejemplo, `model-prep-var`) y se diferencian haciendo uso de un sufijo numérico que indica la iteración o número de variante de cada prueba. A partir de estas medidas y una vez realizadas todas las pruebas, las cuales se pueden ver dentro del **Anexo V**, el repositorio en su estado final presenta la siguiente distribución de ramas en base al objetivo o funcionalidad de las mismas, la cual se puede ver en la siguiente tabla:

Objetivo/Prueba	Número de ramas	Nombre de las ramas
Documentación	1	documentation
Preparación de datos	1	data-prep
Eliminación de variables	8	model-prep-var
		model-prep-var-1
		model-prep-var-2
		model-prep-var-3
		model-prep-var-1-2
		model-prep-var-1-3
		model-prep-var-2-3
		model-prep-var-1-2-3
Sensibilidad de random_state	2	model-prep-rs
		model-prep-rs-1
Barrido de random_state	2	model-prep-rs-group
		model-prep-rs-group-1
Ensamblado de predicciones	2	model-prep-mixin
		model-prep-mixin-1

Tabla 24: Distribución de ramas por objetivo/prueba

Objetivo/Prueba	Número de ramas
documentation	Rama en la que se encuentra toda la documentación realizada para este TFG, en formato Typst.
data-prep	Rama en la que se encuentra todo lo relacionado a las primeras dos capas de la arquitectura (ingesta y procesamiento de los datos).
model-prep	Rama en la que se encuentran las primeras versiones de los modelos desarrollados, a lo largo de las iteraciones se han ido realizando cambios, algunos de dichos cambios se han pasado a los modelos de esta rama.
model-prep-var	Rama en la que se hacen las pruebas iniciales para la prueba de eliminación de variables.
model-prep-var-1	Rama en la que se elimina la variable <code>cu_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-2	Rama en la que se elimina la variable <code>ni_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-3	Rama en la que se elimina la variable <code>mo_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-1-2	Rama en la que se eliminan las variables <code>cu_z</code> y <code>ni_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-1-3	Rama en la que se eliminan las variables <code>cu_z</code> y <code>mo_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-2-3	Rama en la que se eliminan las variables <code>ni_z</code> y <code>mo_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-var-1-2-3	Rama en la que se eliminan las variables <code>cu_z</code> , <code>ni_z</code> y <code>mo_z</code> para comprobar cómo se comportan los modelos entrenados.
model-prep-rs	Rama en la que se hace la primera iteración de la prueba de sensibilidad de <code>random_state</code> . El valor de <code>random_state</code> de esta iteración es de 27.
model-prep-rs-1	Rama en la que se hace la segunda iteración de la prueba de sensibilidad de <code>random_state</code> . El valor de <code>random_state</code> de esta iteración es de 128.
model-prep-rs-group	Rama en la que se hace la primera iteración de la prueba de barrido de <code>random_state</code> . El rango de valores de esta iteración es de 0 a 100 (inclusive).
model-prep-rs-group-1	Rama en la que se hace la segunda iteración de la prueba de barrido de <code>random_state</code> . El rango de valores de esta iteración es de 0 a 100 (inclusive).
model-prep-mixin	Rama en la que se hace la primera iteración de la prueba de ensamblado de predicciones, en esta se prueba con 8 de los 21 targets.
model-prep-mixin-1	Rama en la que se hace la primera iteración de la prueba de ensamblado de predicciones, en esta se prueba con todos los targets.

Tabla 25: Resumen de las ramas creadas.

11.5.1. Notebooks empleados

Además de la distribución en ramas, cada modelo desarrollado, como también cada prueba adicional realizada, se corresponde con uno o varios notebooks de Jupyter independientes. Esta separación permite ejecutar, depurar y reentrenar cada modelo o prueba de forma aislada sin afectar al resto y facilita de forma aislada sin afecta al resto, y facilita además la ejecución automatizada mediante los scripts auxiliares descritos en **Anexo IV**.

Todos los notebooks de modelado comparten una misma estructura interna, dividida en las siguientes celdas o bloques, con el objetivo de mantener una experiencia de desarrollo homogénea entre modelos y facilitar así tanto la comparación entre ellos como la incorporación de nuevos modelos en un futuro:

- **Carga de datos y configuración:** lectura de `train.csv`, `test.csv` y `eval.csv`, junto con la carga del `scaler.pkl` correspondiente y la definición de las semillas (`random_state`) y demás parámetros de configuración del notebook.
- **Búsqueda de hiperparámetros:** ejecución de `RandomizedSearchCV` sobre el conjunto de entrenamiento y validación cruzada, con el grid de parámetros específico del modelo.
- **Validación cruzada del modelo final:** ejecución de `cross_validate` con `RepeatedKfold` sobre el mejor conjunto de hiperparámetros encontrado, para comprobar la estabilidad del modelo antes de entrenarlo de forma definitiva.
- **Entrenamiento final:** ajuste del modelo sobre la totalidad del conjunto de entrenamiento con los hiperparámetros seleccionados.
- **Evaluación preliminar:** cálculo de las métricas de regresión (R^2 , RMSE, MAE) sobre el conjunto de evaluación, junto con una primera aproximación a la importancia de variables.
- **Persistencia:** guardado del modelo entrenado en un archivo `.pkl` mediante `joblib`, para su uso posterior en la Capa de evaluación de modelos.

Sobre esta estructura común, cada notebook incorpora las particularidades propias de su modelo (por ejemplo, las celdas de detección de GPU en los notebooks de XGBoost, o la definición de la arquitectura de capas en los notebooks de MLP).

La siguiente tabla resume los notebooks principales empleados a lo largo de este TFG:

Notebook	Responsabilidad
reg_model	Entrenamiento del modelo de Regresión Ridge. Al tratarse del modelo base con menor coste computacional del proyecto, se emplea además como referencia mínima de rendimiento (baseline) frente a la que se comparan el resto de modelos.
rf_model	Entrenamiento del modelo Random Forest base (salida simple): un modelo independiente por target. Incluye el cálculo de la importancia de variables por impureza media (MDI) y por permutación, ambas empleadas posteriormente como complemento a SHAP.
rf_multisalida	Entrenamiento del modelo Random Forest con salida múltiple: un mismo conjunto de árboles predice simultáneamente los 21 targets. Reutiliza la mayor parte de la estructura de <code>rf_model</code> , adaptando la preparación de los datos para que y sea una matriz en vez de un vector.
regressorchain	Entrenamiento del modelo Random Forest pero envuelto en Regressor-Chain para que devuelva cadenas de predicciones. Implementa la estrategia de Ensembles of Chains (ECC), entrenando varias cadenas con orden aleatorio de targets y semilla distinta por iteración, y promediando sus predicciones finales.
xgboost_model	Entrenamiento del modelo XGBoost base (salida simple). Incorpora una celda de detección de GPU/CUDA para configurar automáticamente los parámetros de dispositivo según el equipo en el que se ejecute el notebook.

Notebook	Responsabilidad
xgb_multisalida	Entrenamiento del modelo XGBoost con múltiples salidas y con soporte de predicción multi-target, empleando el parámetro <code>multi_strategy="multi_output_tree"</code> para que las divisiones del árbol capturen relaciones compartidas entre targets. Comparte la misma lógica de detección de GPU que <code>xgboost_model</code> .
mlp_multisalida	Entrenamiento de una red neuronal con salida múltiple, con la función de pérdida MSE estándar calculada conjuntamente sobre los 21 targets. Implementado sobre PyTorch, incluye la definición de la arquitectura de capas, el bucle de entrenamiento con el optimizador Adam y las celdas de seguimiento de la curva de pérdida por época.
mlp_custom_loss	Entrenamiento de una red neuronal con pérdida modificable, que combina el MSE estándar con un término configurable que penaliza la divergencia entre la matriz de correlación de las predicciones y la matriz de correlación real de los targets del lote. Reutiliza la arquitectura definida en <code>mlp_multisalida</code> , sustituyendo únicamente la función de pérdida empleada durante el entrenamiento.

Tabla 26: Notebooks de entrenamiento de modelos.

Todos estos notebooks comparten una misma estructura interna (carga de datos y configuración, búsqueda de hiperparámetros, validación cruzada del modelo final, entrenamiento final, evaluación preliminar y persistencia del modelo en .pkl mediante joblib), con el objetivo de mantener una experiencia de desarrollo homogénea entre modelos y facilitar así tanto la comparación entre ellos como la incorporación de nuevos modelos en un futuro. Sobre esta estructura común, cada notebook incorpora las particularidades propias de su modelo, como las ya mencionadas en la tabla anterior.

11.5.1.1. Comparación de resultados

Una vez entrenados todos los modelos anteriores, los siguientes notebooks se encargan de cargarlos (o los datos exportados por los mismos) y compararlos entre sí, cada uno desde uno de los dos puntos de vista descritos en la **Capa de evaluación de modelos**:

Notebook	Responsabilidad
comparador_modelos	Carga de todos los modelos entrenados (.pkl) y generación de métricas comparativas desde el punto de vista numérico, las cuales se han descrito ya en la Capa de evaluación de modelos .
comparador_clasificación	Carga de todos los modelos entrenados (.pkl) y generación de métricas comparativas desde el punto de vista ordinal (clasificación), las cuales se han descrito ya en la Capa de evaluación de modelos .
comparador_variables	Carga todos los archivos de variables menos relevantes (.csv) y genera a partir de los datos el ranking de variables a eliminar para la Prueba de eliminación de variables
comparador_barrido_rs	Carga de todos los modelos entrenados (.pkl) y generación de métricas comparativas desde el punto de vista ordinal (clasificación) y numérico, las cuales se han descrito ya en la Capa de evaluación de modelos . Forma parte de la Prueba de barrido de random_state

Tabla 27: Notebooks de comparación de resultados.

11.5.1.2. Pruebas de ensamblado

Vinculados a las ramas **model-prep-mixin** y **model-prep-mixin-1** (véase la tabla de distribución de ramas anterior), estos dos notebooks exploran si combinar las predicciones de varios modelos base mejora el resultado obtenido por cualquiera de ellos por separado:

Notebook	Responsabilidad
prueba_ensamblado_con_modelos	Carga de todos los modelos entrenados previamente y entrena un conjunto de meta-modelos adicionales con las siguientes características: las entradas del meta-modelo son las salidas de los modelos base; el meta-modelo se entrena con una parte de los valores de eval.csv; y se comprueba si la diferencia entre las predicciones de los meta-modelos y el valor real es menor que la diferencia obtenida con los modelos independientes.
prueba_ensamblado	Carga de todos los modelos entrenados y realiza comparaciones a partir de las siguientes condiciones: comprueba si la diferencia de la predicción combinada con el valor real es menor que la diferencia de las predicciones individuales con el valor real. Dichas comprobaciones se hacen de forma independiente con cada uno de los métodos de combinación empleados.

Tabla 28: Notebooks de pruebas de ensamblado.

La diferencia entre ambos enfoques es la siguiente:

- **prueba_ensamblado_con_modelos** aprende una combinación mediante un meta-modelo entrenado sobre parte del conjunto de evaluación (stacking),
- **prueba_ensamblado** prueba distintos métodos de combinación fijos y predefinidos (por ejemplo, un promedio simple o ponderado de las predicciones), sin entrenar ningún modelo adicional sobre ellas.

Los detalles sobre esta prueba y sus resultados se pueden ver en el **Anexo IV**.

Los notebooks correspondientes al resto de las pruebas adicionales (eliminación de variables, sensibilidad y barrido de random_state) reutilizan la misma estructura y el mismo código base que los notebooks de entrenamiento de modelos descritos anteriormente, variando únicamente los parámetros o las variables de entrada propias de cada prueba (por ejemplo, eliminando cu_z, ni_z y/o mo_z del conjunto de variables predictoras, o cambiando el valor de random_state). Esta reutilización deliberada del mismo código, en vez de crear notebooks nuevos y no relacionados por cada prueba, es la que permite que las diferencias observadas entre ramas se deban únicamente al cambio introducido en cada prueba, y no a diferencias accidentales en la implementación.

Cada notebook, una vez ejecutado, deja constancia de sus resultados de dos formas complementarias: por un lado, las propias celdas de salida del notebook quedan guardadas junto con el código tras la ejecución (véase el script de automatización del **Anexo V**, que realiza esta ejecución modificando los propios notebooks).

Por otro, los archivos .pkl de los modelos entrenados y los archivos de resultados exportados por los notebooks de comparación quedan disponibles para su análisis posterior sin necesidad de reejecutar ningún notebook.